"十二五"普通高等教育本科国家级规划教材

面向21世纪课程教材
Textbook Series for 21st Century

电子技术基础

数字部分（第7版）

华中科技大学电子技术课程组　编
主　编　康华光　张　林
副主编　秦　臻
参　编　罗　杰

高等教育出版社

"十二五"普通高等教育本科国家级规划教材

面向21世纪课程教材
Textbook Series for 21st Century

电子技术基础

数字部分（第7版）

华中科技大学电子技术课程组　编
主　编　康华光　张　林
副主编　秦　臻
参　编　罗　杰

高等教育出版社·北京

内容简介

本书为“十二五”普通高等教育本科国家级规划教材，曾荣获全国高等学校优秀教材奖、优秀教材全国特等奖、科技进步二等奖和优秀教材一等奖。

本次修订进一步弱化了中规模器件属性，突出了组合与时序基本单元模块的原理，加强了用 Verilog HDL 设计实现逻辑电路的相关内容，数模与模数转换器的内容更加贴近实际应用。

全书共 11 章，分别是：数字逻辑概论，逻辑代数与硬件描述语言基础，逻辑门电路，组合逻辑电路，锁存器和触发器，时序逻辑电路，半导体存储器，FPGA 和 CPLD，脉冲波形的变换与产生，数模与模数转换器，数字系统设计基础。附录中列出电气简图用图形符号——二进制逻辑单元（GB/T 4728.12—2008）简介，常用基本逻辑符号对照表，编写 Verilog HDL 测试模块，ModelSim 仿真软件的使用，Quartus Prime 软件的使用。

本书可作为高等学校电气类、电子信息类、自动化类等专业“数字电子技术基础”课程的教材，也可供相关工程技术人员参考。

图书在版编目（CIP）数据

电子技术基础. 数字部分 / 华中科技大学电子技术课程组编；康华光，张林主编. -- 7 版. -- 北京：高等教育出版社，2021.8

ISBN 978-7-04-055664-3

Ⅰ. ①电… Ⅱ. ①华… ②康… ③张… Ⅲ. ①电子技术-高等学校-教材 Ⅳ. ①TN

中国版本图书馆 CIP 数据核字（2021）第 026987 号

Dianzi Jishu Jichu Shuzi Bufen

策划编辑 欧阳舟	责任编辑 王耀锋	封面设计 张 楠	版式设计 张 杰
插图绘制 于 博	责任校对 张 薇	责任印制 刁 毅	

出版发行	高等教育出版社	网 址	http://www.hep.edu.cn
社 址	北京市西城区德外大街 4 号		http://www.hep.com.cn
邮政编码	100120	网上订购	http://www.hepmall.com.cn
印 刷	山东临沂新华印刷物流集团有限责任公司		http://www.hepmall.com
开 本	787 mm×1092 mm 1/16		http://www.hepmall.cn
印 张	33.5	版 次	1979 年 3 月第 1 版
字 数	780 千字		2021 年 8 月第 7 版
购书热线	010-58581118	印 次	2021 年 8 月第 1 次印刷
咨询电话	400-810-0598	定 价	68.00 元

本书如有缺页、倒页、脱页等质量问题，请到所购图书销售部门联系调换

版权所有 侵权必究

物 料 号 55664-00

电子技术基础
数字部分
（第7版）

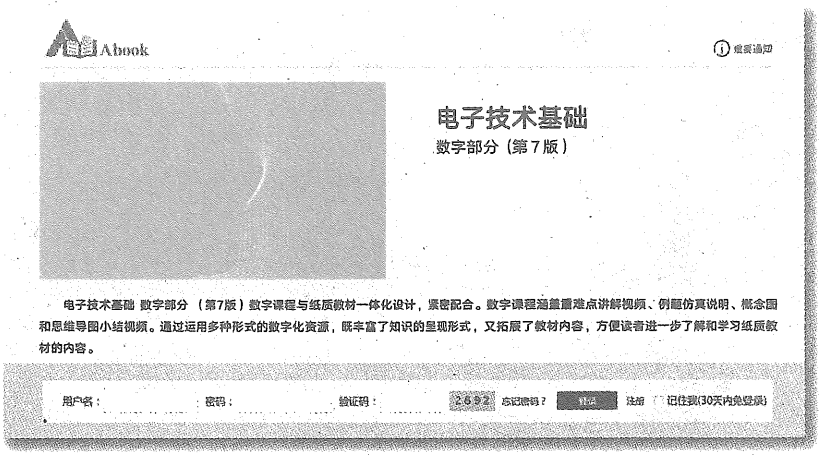
康华光　张　林

1　计算机访问http://abook.hep.com.cn/1257822，或手机扫描二维码、下载并安装Abook应用。

2　注册并登录，进入“我的课程”。

3　输入封底数字课程账号（20位密码，刮开涂层可见），或通过Abook应用扫描封底数字课程账号二维码，完成课程绑定。

4　单击“进入课程”按钮，开始本数字课程的学习。



课程绑定后一年为数字课程使用有效期。受硬件限制，部分内容无法在手机端显示，请按提示通过计算机访问学习。

如有使用问题，请发邮件至abook@hep.com.cn。

电子技术基础
数字部分
（第7版）

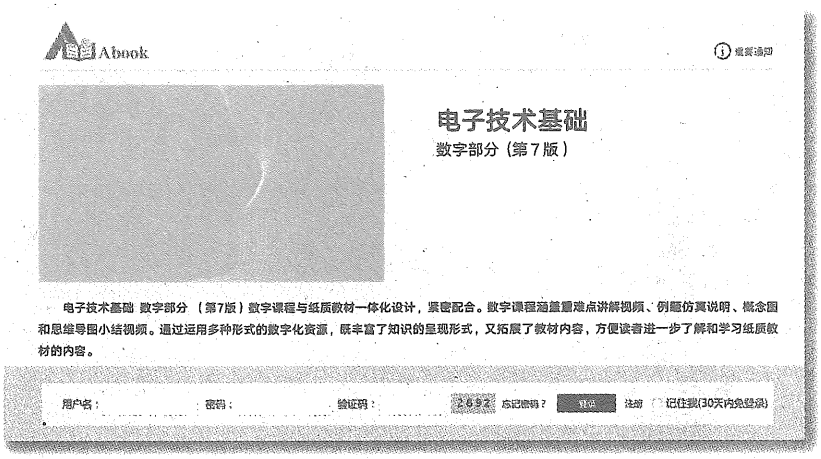
康华光　张　林

1　计算机访问http://abook.hep.com.cn/1257822，或手机扫描二维码、下载并安装Abook应用。

2　注册并登录，进入“我的课程”。

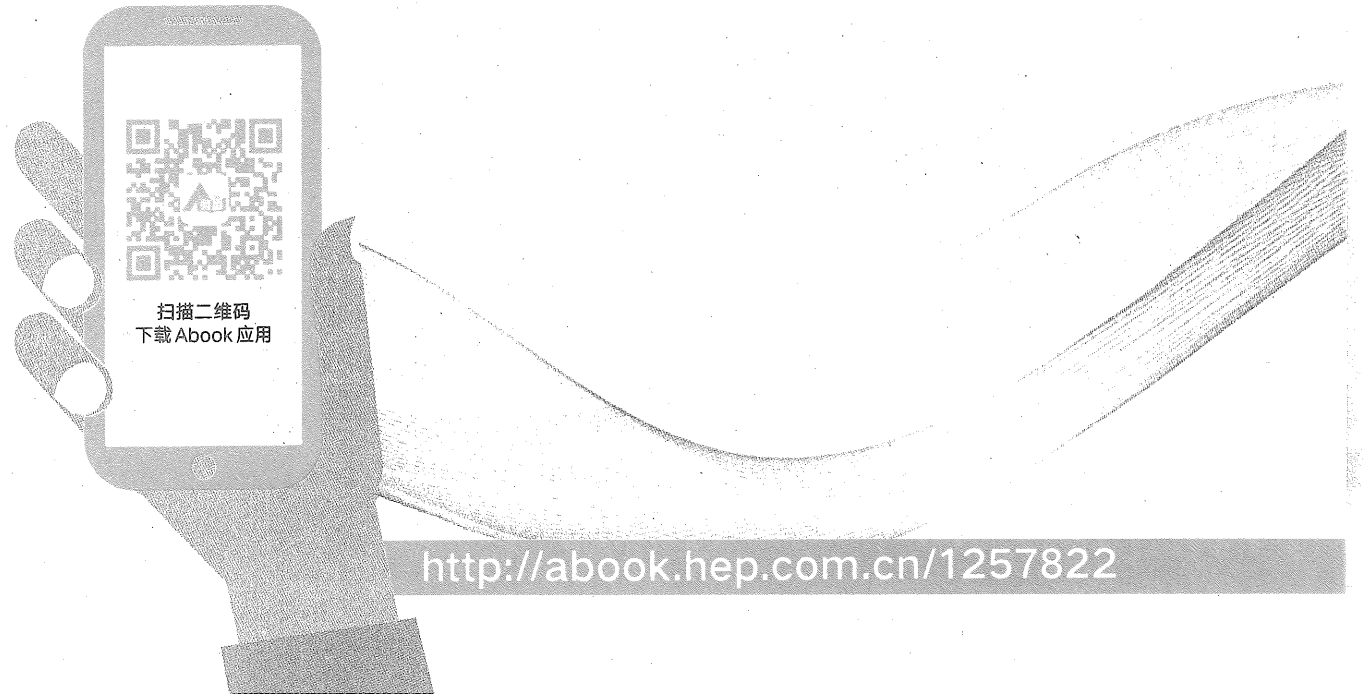
3　输入封底数字课程账号（20位密码，刮开涂层可见），或通过Abook应用扫描封底数字课程账号二维码，完成课程绑定。

4　单击“进入课程”按钮，开始本数字课程的学习。



课程绑定后一年为数字课程使用有效期。受硬件限制，部分内容无法在手机端显示，请按提示通过计算机访问学习。

如有使用问题，请发邮件至abook@hep.com.cn。



与本教材配套的实验教材是罗杰、陈大钦主编的《电子技术基础实验》(高等教育出版社出版,2017 年)。

本书模拟部分分别由四川大学周群教授、大连理工大学林秋华教授主审,参加审阅的还有四川大学的刘婕副教授和刘雪山副教授。数字部分由浙江大学张德华教授主审。他们认真审阅了本书,提出了不少中肯的修改意见,在此表示衷心的感谢。原编者王岩教授也仔细审阅了模拟部分全稿,并提出了宝贵意见,也在此表示衷心感谢。

另外,原编者康华光、陈大钦、王岩、彭容修、瞿安连、杨华等前辈为本套教材的建设和发展做出了巨大贡献;也对本次修订工作给予了大力支持和帮助。特在此向他们致以崇高的敬意!

第 6 版发行期间,承全国各兄弟院校师生给我们以鼓励,提出了不少宝贵意见和建议,编者在此一并致以谢忱。编者邮箱:campzh@ hust.edu.cn。

编者

2020 年 7 月于武汉华中科技大学

作者声明

未经本书作者和高等教育出版社书面许可，任何单位和个人均不得以任何形式将《电子技术基础　数字部分》（第7版）中的习题解答后出版，不得翻印或在出版物中选编、摘录本书的内容；否则，将依照《中华人民共和国著作权法》追究法律责任。

第 7 版序

《电子技术基础(模拟、数字)》是高等学校电子信息类、电气类、自动化类等专业的重要基础课程教材。根据当今电子技术发展及教材使用实际情况,我们对前一版教材进行了本次修订。

一、遵循原则

(1) 保持内容的科学性、先进性,以适应电子技术的发展。

(2) 坚守基础性特征,精选内容,科学合理地控制内容的深度和广度,以适应我国高校的教学要求,力争好教好学。

(3) 加强与工程实际应用的联系。

(4) 适当引入新形态教材形式,以便更好地辅助读者学习。

二、主要变动

1. 模拟部分

(1) 将差模增益、共模增益和共模抑制比等概念提前到第 2 章运算放大器介绍,更加贴近运放实际应用。补充了应用实例——心电信号放大电路。

(2) 不再采用 MOS 管和 BJT 并重的方式组织第 4、5 章内容,改为以 MOSFET 为主介绍三极管放大电路,而简要介绍 BJT 放大电路。并将 4.6 节更新为 MOSFET 大信号工作及开关应用。

(3) 重新组织编排了第 6 章并缩减了内容,补充了短路时间常数法和开路时间常数法。

(4) 合并了 7.2 节和 7.3 节并重新梳理了内容。集成运算放大器补充了全差分运算放大器和电流反馈运算放大器,将 LF356 移至习题。加强了运放主要参数对实际应用影响的相关内容,补充了运放在单电源下工作的内容。

(5) 第 8 章用 MOS 管放大电路更换了部分 BJT 放大电路。

(6) 第 9 章新增了丁类(D 类)功率放大电路原理简介,更新了集成功率放大器。

(7) 第 10 章重写了有源滤波电路,使之更加符合当前实际设计应用情况。

(8) 重写了 11.2 线性稳压电路和 11.3 开关式稳压电路,补充了低压差稳压电路。

(9) 删除每章的"SPICE 仿真例题"一节,将部分内容分散到了正文适当的位置和习题中。

删除了第12章电子电路的计算机辅助分析与设计的内容。

(10) 在新媒体方面,有若干重难点讲解视频、例题仿真说明、各章的概念图和思维导图小结视频,附录A PSpice/SPICE软件简介,以及软件使用操作演示视频等。读者都可扫码浏览观看。

2. 数字部分

(1) 注重组合和时序单元电路的模块化。压缩和删减了较少使用或已经淘汰的集成芯片内部结构的详细介绍。

(2) 根据Verilog HDL最新版本更新了语法规则,并调整了描述顺序(先举例,再讲语法)。增加了测试激励模块的简介。仿真软件改用ModelSim并在附录中介绍测试模块的编写方法及软件的使用说明。

(3) 第3章缩减了TTL逻辑门电路的内容,删除了ECL逻辑门电路,增加了差分信号传输内容的介绍。

(4) 第4章注重单元电路的工作原理,弱化产品芯片型号及电路内部结构的介绍,增加了移位器等内容。在4.6节增加了实例引用带参数模块的例题。扩充了Verilog习题。

(5) 第5章注重触发器的原理功能介绍,缩减了实际芯片内部电路的相关内容。删除了传输延迟触发器。

(6) 第6章删除了已淘汰产品GAL的内容。

(7) 第7章补充了Flash存储器的相关内容。

(8) 第8章调整了CPLD和FPGA的顺序,补充了CPLD中逻辑宏单元的内容。

(9) 第9章精简了单稳态和施密特内部电路的介绍,修改了多谐振荡电路的描述。补充了相关电路芯片信息。新增了并联谐振型晶体振荡电路。

(10) 第10章增加了电阻串联分压式D/A转换器,修改了双极性输出的描述,以及误差的相关描述。补充了半闪速型、流水线型A/D转换器。

(11) 第11章精简了ASM图介绍,重新组织了交通信号控制系统及数字密码锁的设计内容,便于用CAD方法设计。

(12) 在新媒体方面,有若干难点视频讲解,Verilog例题仿真说明文档,每章的思维导图,附录D ModelSim仿真软件简介,以及软件使用操作演示视频等。读者都可扫码浏览观看。

参加模拟部分修订工作的有张林(第1~7章、附录及17个新媒体文档、15个视频)、秦臻(第8章及3个新媒体文档)、邓天平(第9、11章及1个新媒体文档、9个视频)、罗杰(第10章)等。参加数字部分修订工作的有秦臻(第1、3、4、5、11章、附录A、B及6个新媒体文档、2个视频)、罗杰(第2、8、9、10章、各章Verilog语言部分、附录C、D、E及12个新媒体文档、14个视频)、张林(第6、7章及2个新媒体文档、1个视频)。张林为主编,负责全书的策划、组织和定稿。

第 6 版序

《电子技术基础(模拟、数字)》,是电子电气类专业的技术基础课程教材,该套教材自 1979 年春由高等教育出版社出版发行以来,深受广大读者的欢迎。根据当前电子技术发展的新形势,在第 5 版的基础上,推陈出新。如今电子技术发展的现实是,MOSFETs 器件在电子产品中已占统治地位。为了适应这一发展形势,新版教材大力加强了 MOSFET 的相关内容。现就模拟和数字两部分提出如下的修订思路。

一、模拟部分

1. 运算放大器是模拟部分的核心内容。在第 2 章中,首先把它理想化,称为理想运算放大器,实际的运放将在第 7 章(模拟集成电路)中讲述。这样的安排是为了让学生易于入门,分散难点,也为了让教师根据各专业的要求作相应的选择。讲完第 2 章后,即可在教师的指导下进行运放的基本实验,可使学生对该课程产生兴趣,并有初步的成就感。

2. 由于半导体材料和器件制造工艺的进步,场效应管(MOSFETs)与双极结型三极管(BJTs)相比显示出新的优越性而获得较广泛的应用。考虑到历史和现实将第 4、5 两章写成相互独立的内容,教师可以自由选择其中任一章先讲,后讲的章节可以加快进度。

3. 频率响应一章,除一般知识外,可就 MOSFETs 和 BJTs 的相关电路有选择性地讲述。例如重点介绍 MOS 管及共源放大电路的高低频响应,最后介绍扩展频带的方法。

4. 模拟集成电路一章的内容丰富,可有选择性地讲述 MOSFETs 和 BJTs 的相关电路。至于运算放大器,也可按同样方法处理。

5. 反馈电子电路是电子电路的重要内容,通过大量的例题和习题以阐明负反馈的基本概念与分析方法,对反馈电路的稳定问题也作了简明分析。

二、数字部分

1. 现代数字电路和系统基本上不再使用中规模集成芯片搭建,而是采用 CPLD 或 FPGA 实现,甚至将系统集成在单一芯片上。其设计过程是将组合与时序单元电路作为基本模块由高层调用。因此,教材力求在弱化中规模集成芯片应用的同时,将组合与时序单元电路作为宏模型介绍。

2. 便携设备的发展要求CMOS集成电路的电源电压越来越低，导致低电压、超低电压器件的广泛使用。教材加强了低电源电压器件及其接口内容介绍，同时削减了TTL系列的内容。

3. 增加了CMOS通用电路中小尺寸逻辑与宽总线内容的介绍。小尺寸逻辑芯片是用来修改完善大规模集成芯片之间连线或外围电路的。与中规模器件相比，体积更小，速度更快。宽总线芯片是为满足计算机总线驱动而产生的。

4. 为了便于学生掌握Verilog描述单元电路的方法，加强了Verilog描述组合及时序单元电路的例题。

5. 当用指定器件实现电路设计时，力求成本低、速度快。介绍了EDA工具实现优化设计时，需要用到多乘积项的共用、提取公因子、函数分解等方法。

6. 增加了时钟同步状态机的同步问题。当数字系统的结构复杂、工作速度快时，时钟同步问题也越来越突出。由时钟偏移等问题引起触发器误翻转会造成系统的误动作。因此要在设计上避免这类问题的出现。

在本版修订工作中，重新改编了例题、复习思考题和习题，以利读者深入理解教材内容。SPICE部分和Verilog语言部分的内容，供各校师生灵活选用。

参加本版模拟部分修订工作的有张林(第1、3、12章及附录A、B、C)、王岩(第2、7、11章)、陈大钦(第4、9、10章)、杨华(第5、6、8章)等。参加数字部分修订工作的有秦臻(第1、3、4、11章及附录B、C)、罗杰(第2章及附录A)、瞿安连(第5、6章)、张林(第7、8章)、彭容修(第9、10章)。康华光为主编，负责全书的策划、组织和定稿。陈大钦、张林为模拟部分的副主编；秦臻、张林为数字部分的副主编，协助主编工作。此外，张林还完成了模拟电路的SPICE分析；罗杰还完成了数字电路的Verilog语言描述。

电子技术基础是一门实践性很强的课程，与本教材配套的实验教材是由高等教育出版社出版的，陈大钦、罗杰主编的《电子技术基础实验》。

本书由哈尔滨工业大学蔡惟铮教授主审，参加审阅的还有王淑娟教授、杨春玲教授和王立欣教授。他们认真审阅了本书，提出了不少中肯的修改意见，在此表示衷心的感谢。第5版发行期间，承全国各兄弟院校师生给我们以鼓励，寄来了不少宝贵意见和建议，编者在此一并致以谢忱。

康华光

2013年5月于武汉华中科技大学

第5版序

当代电子技术的迅速发展，为人们的文化、物质生活提供了优越的条件，数码摄像机、家庭影院、空调、电子计算机等，都是典型的电子技术应用实例，可谓琳琅满目、异彩纷呈。至于电子技术在科技领域的应用，更是起着龙头作用，例如通信工程、测控技术、空间科学等比比皆是。而计算机的普及，也为大学生们提供了良好的学习平台。

本版是在前版的基础上修订而成，在修订过程中，参考了教育部组织编写的《电子技术基础(A)课程基本要求》，提出了如下的思路：精选内容，推陈出新；讲清基本概念、基本电路的工作原理和基本分析方法。对于较简单的电路，可用手工的方法进行近似计算；对于较复杂的电路，则可利用计算机及相应的软件进行仿真分析和设计。具体考虑有如下几点：

1. 简述信号与电子系统的概念，为学习模拟电路和数字电路提供引导性的背景知识。

2. 由于微电子学与制造工艺的进步，特别是在数字电路中，与双极型器件的性能相比，MOS器件具有明显的优势。

3. 在模拟电路中增加了器件建模的内容，并利用SPICE软件对电路作具体的仿真分析与设计。在数字电路中增加了用Verilog语言建模的内容，借助Quartus Ⅱ集成开发软件对电路进行仿真分析与设计。

现在，硬件与软件之间的界限越来越模糊，模拟电路或数字电路均属硬件，在利用软件对电路进行辅助设计时，不能轻视硬件，应引导学生全面发展。

4. 重新改编了例题、复习思考题和习题，以便读者深入理解教材内容。SPICE部分和Verilog语言部分的内容，供各院校师生灵活选用。

参加本版模拟部分修订工作的有张林(第1、3、11章及附录A、B、C)、王岩(第2、6、10章)、杨华(第4、7章)、陈大钦(第5、8、9章)等。参加数字部分修订工作的有秦臻(第1、3、4、10章及附录A、C、D)、罗杰(第2章及附录B)、瞿安连(第5、6章)、张林(第7章)、彭容修(第8、9章)等。康华光为主编，负责全书的策划、组织和定稿。陈大钦、张林为模拟部分的副主编，邹寿彬、秦臻为数字部分的副主编，协助主编工作。此外，张林还完成了模拟电路的SPICE分析；罗杰还完成了数字电路的Verilog语言描述。

本书由哈尔滨工业大学蔡惟铮教授主审，参加模拟部分审阅的为王淑娟教授，数字部分审阅的为杨春玲教授，在此表示衷心的感谢。第4版发行期间，承全国各兄弟院校师生给我们以鼓励，寄来了不少宝贵意见和建议，编者在此一并致以谢忱。

康华光

2005年7月于武汉华中科技大学

第 4 版序

在电子技术日新月异的形势下，为了培养面向 21 世纪的电子技术人才，本书在第 3 版的基础上，经过教学改革与实践，对其内容作了较大的修改和更新，使之更符合电子信息时代的要求。在修订过程中，依照 1995 年教育部（原国家教委）颁发的《高等工业学校电子技术基础课程教学基本要求》，提出了如下的总思路：精选内容，推陈出新；讲清基本概念、基本电路的工作原理和基本分析方法。对主要的技术指标，采用工程近似方法进行计算，至于更全面的分析与设计则可借助 CAD 技术来实现。这将有利于读者开拓思路。具体考虑有如下几点：

1. 加强电子系统与信号的概念，为学习模拟电路和数字电路提供了引导性的背景知识。

2. 增加了部分新器件的内容，如砷化镓场效应管（MSFET）、VMOS 功率器件、BiCMOS 门电路、现场可编程逻辑器件（如 CPLD、FPGA）等，以适应新技术发展的需要。

3. 将三端有源器件（BJT、FET）的 6 种电路组态（共射、共集、共基和共源、共漏、共栅）归结为 3 种通用的电路组态，即反相电压放大器、电压跟随器和电流跟随器，这就有利于电子电路的分析与综合，也为学习和使用 BiFET 和 BiCMOS 等一类新型集成电路器件奠定了基础。

4. 根据当前教学上的需要与设备条件的可能性，模拟部分增设了“电子电路的计算机辅助分析与设计”一章；为电子电路仿真与设计作了引导性的介绍，并利用 PSPICE 软件解题；数字部分增设了“数字系统设计基础”一章，将硬件描述语言 ABEL 和 ISP Synario 软件作为数字系统设计的入门性工具，并附有设计实例。

5. 为便于读者深入理解教材内容，加强了例题，其中部分电路具有实用性。同时也重编了具有启发意义的复习思考题和习题，并附有少量的 CAD 例题和习题供各校师生灵活选用。

参加本版模拟部分修订工作的有瞿安连（第 1 章）、康华光（第 2、3、7 章）、陈大钦（第 4、5、8、9 章）、王岩（第 6、8、10 章）、张林（第 11 章及附录）等同志。参加数字部分修订工作的有康华光（第 1、2 章及附录 A、B、C、E、F、G）、邹寿彬（第 3、4、5 章）、杨华（第 6、7 章）、张林、李玲（第 8 章）、彭容修（第 9、10 章）、秦臻（第 11 章）、罗杰（附录 D 和第 8 章的部分内容）。康华光同志为主编，负责全书的策划、组织和定稿。陈大钦和邹寿彬同志分别为模拟部分和数字部分的副主编，协助主编工作。此外，杨华同志负责重编了模拟部分第 2、3、7 章的习题和第 1 章的校订工

作;张林和罗杰二位同志协助有关各章的编者,完成了全书的CAD例题和习题的解答工作。

本书由东南大学衣承斌教授主审,参加审阅的,模拟部分为刘京南教授、李桂安副教授;数字部分为皇甫正贤教授、戴义宝副教授。第3版发行期间,承全国各兄弟院校师生给我们以鼓励,寄来了不少宝贵意见和建议,编者在此一并致以衷心的谢意。

编者

1998年7月于武汉华中理工大学

第 3 版序

自本书第 1 版问世以来，已经历了近十年。在这期间，电子技术领域发生了迅猛而巨大的变化。新技术革命和教学改革的不断深入，促使本教材不断改进完善，第 3 版现在与读者见面了。

新版是在第 2 版的基础上，经过改革试验、总结提高、修改增删而成的。在修订工作中，依照 1987 年经国家教委批准的《高等工业学校电子技术课程教学基本要求》，在保证基本教学内容的前提下，为适应电子技术不断发展的新形势和教学上的灵活性以及因材施教的需要，本版适当增加了部分加宽加深的选讲内容，具体考虑如下：

1. 新版在体系上做了较大的调整。在模拟部分中，将“模拟集成电路”一章的位置提前，以致有可能在“反馈放大器”以及后续各章中，均以模拟集成电路为对象进行讨论，这就形成了以模拟集成电路为主干的体系。数字部分则直接以小规模数字集成电路引路，逐步向中大规模集成电路深入，几乎大部分内容都纳入“组合逻辑”和“时序逻辑”两大类电路之中。

2. 在保证基本理论完整性的原则下，删去或精简了一些分立元件电路内容，增强了集成电路的应用，并引入模拟乘法器、开关电容滤波器、压控振荡器、锁相环、直流变换器、门阵列、算术逻辑单元、动态存储器、集成 A/D 与 D/A 转换器等新技术内容。

3. 为了开拓学生的知识广度，新增了“调制与解调”一章。

4. 本书数字部分的内容安排与讲述方法，注意到了与“微处理器基础”的密切联系，以利于压缩学时，提高教学效果。

5. 为了贯彻理论联系实际的原则，书中以不同的方式，安排了一定数量的电路实例，并注意阅读电子电路图和查阅电子器件手册的训练。

6. 教材正文与例题、习题紧密配合。例题是正文的补充。某些内容则有意地让读者通过习题来掌握，以调节教学节律，利于理解深化。

7. 在编排上，对于加宽加深的内容，均注有 * 号，以便于教师选讲和读者自学参考。

本版仍沿用从模拟到数字的体系，若有需要，亦可按从数字到模拟的体系讲授，只需将模拟部分的“半导体二极管和三极管”一章移到数字部分之前讲授即可。

参加新版模拟部分修订工作的有汤之璋(第1章)、康华光(第1、2、6、7章)、王岩(第5、8、11章)和陈大钦(第3、4、8、9、10章及附录A)等同志。参加数字部分修订工作的有康华光(第1、2章)、邹寿彬(第3、4、7章)和赵德宝(第5、6章及附录A)等同志。康华光同志为主编,负责全书的组织和定稿。陈大钦和邹寿彬同志分别为模拟和数字部分的副主编,协助主编工作。在修订过程中,得到了汤之璋教授的支持与帮助。赵德宝、瞿安连、肖锡湘同志协助校订了模拟部分的原稿。陈大钦、瞿安连同志协助校订了数字部分的原稿。丁素芳、罗杰、杨晓安和汪菊华等同志绘制了全书的插图。教研室的其他同志也参加了部分工作。

本书由南京工学院李士雄教授主审,负责组织审稿工作的为衣承斌副教授,参加审阅的,模拟部分为衣承斌、陈黎明、陈天授副教授,李桂安讲师;数字部分为丁康源副教授,郑虎申、严振祥、皇甫正贤讲师。在第2版发行期间,承全国许多师生给我们以鼓励,寄来了不少宝贵意见和建议,编者谨此一并致以谢忱。

本版虽有所改进提高,但离教学改革的要求尚远。敬希读者予以批评指正。

编者

1987年8月于武昌华工园

第 2 版序

本书是在第 1 版的试用基础上，并按照高等工业学校《电子技术基础教学大纲（草案）》（四年制自动化类和电力类专业试用），总结提高、修改增删而成的。主要做了下列几方面的工作：（1）从本课程的目的和任务出发，在保证打好基础的前提下，精选了内容，例如删去了“电子电路的计算机辅助分析”一章，适当精简了器件内部的物理过程、放大器的频率特性分析、分立元件电路以及设计方面的内容等，在篇幅上有较大的缩减；（2）删繁就简改写了第二、四、六章的大部分内容。同时，将第一版的第九、十章各分为两章，以利于教学；（3）增加了部分新内容，如集成运算放大器的应用电路，中规模数字集成电路等；（4）加强了电路分析方法，如用“虚短”的概念分析集成运算放大器的线性应用电路；在数字电路中，突出了组合逻辑与时序逻辑电路的分析方法；（5）近几年来，由于大规模集成电路的飞速发展，出现了微处理机对各个科学技术领域的渗透，为此，我们充实了“MOS 数字集成电路”一章的内容；（6）重新整理并增删了各章所附的思考题和习题。此外，在编排上，把基本内容排大字，选讲内容排小字，自学参考内容既排小字，又带＊号。

本版各章基本上由原编者修订，参加的人员有汤之璋、康华光、陈婉儿、王岩、陈大钦、邹寿彬、朱立群等同志，全书由康华光同志定稿。在修订过程中，得到了汤之璋教授的帮助与指导，陈婉儿同志协助校阅了第一至第六章的书稿，肖锡湘、陈晓天、丘小云、石友惠、罗玉兰以及其他同志参加了许多工作。

本书由南京工学院李士雄教授主审，参加审阅工作的还有陈天授、陈黎明、皇甫正贤、郑虎申等同志；在本书第 1 版的试用期间，承全国有关兄弟院校的师生寄来不少宝贵意见和建议，编者在此深表谢忱。

本版内容虽有所改进，但离教学要求尚有差距，恳请使用本教材的师生和其他读者予以批评指正，以便不断提高。

编者

1982 年 10 月于武汉

第1版序

本书是根据高等学校工科基础课电工、无线电类教材编写会议(1977年11月合肥会议)所制订的"电子技术基础"(电力类)教材编写大纲编写的。在编写过程中,我们力图以马列主义、毛泽东思想为指导,运用辩证唯物主义观点和方法来阐明本学科的规律。

"电子技术基础"是电力工程类各专业的一门技术基础课,它是研究各种半导体器件的性能、电路及其应用的学科。从本学科内容大的方面来划分,本书上、中两册属模拟电子技术,下册属数字电子技术;前者主要是讨论线性电路,后者则着重讨论脉冲数字电路。

教材中注意总结我们近年来的教学实践经验,加强了基础理论,如加强了半导体的物理基础和电路的基本分析方法;同时也注意吸取国内外的先进技术,如加强了线性集成电路和数字集成电路(包括中、大规模集成电路)的原理和应用,新增了电子电路的计算机辅助分析等内容。

在内容的安排上,注意贯彻从实际出发,由浅入深、由特殊到一般、从感性上升到理性等原则。通过各种半导体器件及其电路来阐明电子技术中的基本概念、基本原理和基本分析方法。对于基本的和常用的半导体电路(包括脉冲数字电路),除了作定性的分析外,还介绍了工程计算或设计方法。为了加深对课堂知识的理解,列举了若干电路实例,并配有一定数量的例题、思考题和习题。

在使用本教材时,请注意下列几点:

(1) 本课程是在学完普通物理学和电工原理的大部分内容之后开设的,课程之间的相互配合和衔接非常重要。例如,在第一章用能带理论来解释半导体内两种载流子——电子和空穴的导电规律时,应以普通物理学中讲的固体能带理论为基础;又如在分析放大器时,既讨论了稳态分析(频域),也介绍了瞬态分析(时域),在"运算放大器"一章中,又有积分、微分电路以及其他应用,这些内容应以电工原理中的无源线性电路的瞬态分析为基础,只有配合得好,才能取得满意的效果。

(2) 本教材是按课程总学时数约200(包括实验课等环节)而编写的,除了基本内容之外,还编入了部分较深入的内容,这些内容均在标题前注有星号(*)或用小字排印,自成体系。不同专业可按学时多少,由教师灵活选择,也可供读者自学参考。

(3) 课程中各个教学环节的配合十分重要,除了课堂讲授外,还必须通过习题课和实验课等环节加以补充,有些内容可以把这几个环节有机地结合起来。对于实验课,必须予以高度重视,通过实验课,不仅可以验证理论,加深对理论知识的理解,更重要的是,可以学会电子测试技术,使理论紧密结合实践。

参加本书编写工作的有汤之璋(第一章)、陈婉儿(第一、二、九章)、陈大钦(第三、五、十章)、康华光(第四、十一章)、王岩(第六、七、十三章)、林家瑞(第六章)、邹寿彬(第八、十二章)、周劲青(第十一章)和江庚和(第十三章)等同志,最后由康华光同志定稿。在编写过程中,张瑾、朱立群、赵月怀、肖锡湘、杨华、石友惠、汪菊华、罗玉兰以及其他同志参加了许多工作,给予很大支持。

本书由南京工学院李士雄副教授主审,参加主审工作的还有江正战、张志明、衣承斌、陈黎明和丁康源等同志。

在武汉和南京举行的审稿会上,承西安交通大学沈尚贤教授、清华大学童诗白教授、浙江大学邓汉馨副教授、上海交通大学徐俊荣副教授以及重庆大学、山东工学院、沈阳机电学院、合肥工业大学、大连工学院、湖南大学、华南工学院、同济大学、哈尔滨工业大学、天津大学、太原工学院和昆明工学院等兄弟院校的教师代表对初稿进行了认真的审阅,并提出了许多宝贵的意见。

在编写本书第八章(电子电路的计算机辅助分析)的过程中,承中国科学院湖北岩体土力学研究所计算机室协助解题。

对所有为本教材进行审阅并提出宝贵意见以及在编写出版过程中给予热情帮助和支持的同志们,我们在此一并表示衷心的感谢。

由于我们的水平有限,加之时间比较仓促,书中错误和不妥之处,在所难免,殷切希望使用本教材的师生及其他读者,给予批评指正。

编者

1979年3月

本书常用符号表

A_0、A_1、A_2、…	第 0、1、2、…位译码器地址输入
$A>B$、$A=B$、$A<B$	数字比较器大于、等于、小于
BCD	二-十进制码
C	传输门高电平控制信号
CAS	动态存储器列地址选通信号
CE	存储器片选输入端
CEP	计数器并行进位允许输入端
CET	计数器进位允许输入端
CP	时钟脉冲
CP_n	时序电路中第 n 个触发器的时钟信号
cp_n	异步时序电路第 n 个触发器的时钟有效标志
CR	清零输入端
CS	片选信号输入
C_{ext}	外接电容端
C_{PD}	CMOS 电路的功耗电容
C_L	负载电容
$\overline{C}$	传输门低电平控制信号
D	数据
D	数据输入
D	D 锁存器或触发器的 D 输入端
DP	延时-功耗积
D_{PO}	移位寄存器并行输出
D_{SI}	移位寄存器串行输入端
D_{SL}	移位寄存器左移串行输入端
D_{SO}	移位寄存器串行输出端
D_{SR}	移位寄存器右移串行输入端
DNL	级差非线性误差
E	使能控制端
E	锁存器使能输入端
E	时序电路激励信号向量
EC	计数允许输入端
$EI;EO$	使能输入;使能输出
FF_n	时序电路中第 n 个触发器
f_{CP}	时钟频率
f_{cmax}	最高时钟频率
f_s	采样频率
G	逻辑门
G	进位产生变量
I	时序电路的输入信号向量
INL	整体非线性误差
I_{BS}	临界饱和基极电流
i_B	基极电流瞬时值(交、直流混合量)
I_{CS}	集电极饱和电流
i_C	集电极电流瞬时值
i_D	漏极电流瞬时值
I_{IH}	高电平输入电流
I_{IL}	低电平输入电流
I_{OH}	高电平输出电流
I_{OL}	低电平输出电流
I_{OZ}	流过截止 MOS 管的漏电流
J	JK 触发器的 J 输入端
K	JK 触发器的 K 输入端
K_n	MOS 管电导常数
k'_n	MOS 管制造工艺常数
LE	锁存允许输入端
N_I	扇入数
N_O	扇出数
O	时序电路的输出信号向量
OE	储存器输出使能输入端

PE	并行置数允许输入端	t_{Gpd}	门延迟时间
P_D	功耗	t_f	下降时间
P_T	CMOS 管导通功耗	t_{pHL}	高电平到低电平的传输延迟时间
P_L	CMOS 管带容性负载的动态功耗	t_{pLH}	低电平到高电平的传输延迟时间
Q	锁存器或触发器的输出端		
Q^n	触发器的现态	t_{pd}	平均传输延迟时间
Q^{n+1}	触发器的次态	t_{SU}	建立时间
q	占空比	t_{set}	D/A 转换器的稳定时间
R	*SR* 锁存器或 *SR* 触发器的复位(置 0)输入端	t_W	脉冲宽度
		t_r	上升时间
RAS	动态储存器行地址选通信号	v_{BE}	三极管发射结电压
$RESET$	时序电路的复位信号	V_{CC}	TTL 电路电源电压
R_{ext}	外接电阻端	V_{CES}	BJT 的饱和压降
R_D	锁存器或触发器的直接复位(置 0)输入端	v_{CE}	三极管集电极-发射极电压
		V_{DD}	CMOS 电路电源电压
r_{ds}	MOS 管漏源之间的交流等效电阻	v_{DS}	MOS 管漏源之间电压
		v_{GS}	MOS 管栅源之间电压
R_L	负载电阻	V_I	输入电压
R_{on}	MOS 管导通时的电阻	v_I	输入电压瞬时值
R_P	OD 门外接的上拉电阻	V_{LSB}	D/A 转换器输出电压的步长
S	*SR* 锁存器或 *SR* 触发器的置位(置 1)输入端	V_{NH}	高电平噪声容限电压
		V_{NL}	低电平噪声容限电压
S	时序电路的状态向量	V_O	输出电压
S_D	锁存器或触发器的直接置位(置 1)输入端	v_O	输出电压瞬时值
		V_{OH}	输出高电平时的电压
S^n	时序电路的现态	V_{OL}	输出低电平时的电压
S^{n+1}	时序电路的次态	V_{REF}	参考电压
T	周期	V_T	MOS 管阈值电压
T	*T* 触发器的 *T* 输入端	V_{TH}	阈值电压
TC	计数器进位输出	V_{T+}	施密特触发特性的正向阈值电压
T_{CP}	时钟周期		
T_N	N 沟道 MOSFET	V_{T-}	施密特触发特性的负向阈值电压
T_P	P 沟道 MOSFET		
t	时间	WE	储存器写使能输入端
t_{COMpd}	同步时序电路中组合电路的延迟时间	W/L	MOS 管沟道的宽长比
		×	任意态,无关项
t_H	保持时间	τ	时间常数

目　录

1　数字逻辑概论 …… 1
1.1　数字信号与数字电路 …… 1
1.1.1　数字技术的发展及其应用 …… 1
1.1.2　数字集成电路的分类及特点 …… 3
1.1.3　模拟信号和数字信号 …… 6
1.1.4　数字信号的描述方法 …… 7
1.2　数制 …… 11
1.2.1　十进制 …… 11
1.2.2　二进制 …… 11
1.2.3　十-二进制之间的转换 …… 14
1.2.4　十六进制和八进制 …… 16
1.3　二进制数的算术运算 …… 18
1.3.1　无符号二进制数的算术运算 …… 18
1.3.2　带符号二进制数的减法运算 …… 19
1.4　二进制代码 …… 23
1.4.1　二-十进制码 …… 23
1.4.2　格雷码 …… 24
1.4.3　ASCII 码 …… 27
1.5　二值逻辑变量与基本逻辑运算 …… 28
1.6　逻辑函数及其表示方法 …… 33
1.6.1　逻辑函数的几种表示方法 …… 33
1.6.2　逻辑函数表示方法之间的转换 …… 35
小结 …… 37
习题 …… 37

2　逻辑代数与硬件描述语言基础 …… 41
2.1　逻辑代数的基本定律和规则 …… 41
2.1.1　逻辑代数的基本定律和恒等式 …… 41
2.1.2　逻辑代数的基本规则或定理 …… 43
2.2　逻辑函数表达式的形式 …… 45
2.2.1　逻辑函数表达式的基本形式 …… 45
2.2.2　最小项与最小项表达式 …… 45
2.2.3　最大项与最大项表达式 …… 47
2.3　逻辑函数的代数化简法 …… 49
2.4　逻辑函数的卡诺图化简法 …… 52
2.4.1　用卡诺图表示逻辑函数 …… 52
2.4.2　用卡诺图化简逻辑函数 …… 55
2.5　硬件描述语言 Verilog HDL 基础 …… 58
2.5.1　Verilog HDL 模块的基本结构 …… 59
2.5.2　逻辑功能的仿真与测试 …… 61
2.5.3　Verilog HDL 的基本语法规则 …… 64

2.5.4 数据类型 …… 66
2.5.5 运算符及其优先级 …… 68
2.5.6 Verilog HDL 内部的基本门级元件 …… 70
小结 …… 73
习题 …… 74

3 逻辑门电路 …… 78
3.1 逻辑门电路简介 …… 78
3.1.1 各种逻辑门电路系列简介 …… 78
3.1.2 开关电路 …… 80
3.2 基本 CMOS 逻辑门电路 …… 81
3.2.1 MOS 管及其开关特性 …… 81
3.2.2 CMOS 反相器 …… 85
3.2.3 CMOS 与非门和或非门 …… 89
3.2.4 CMOS 传输门 …… 91
3.3 CMOS 逻辑门的其他输出结构及参数 …… 93
3.3.1 CMOS 逻辑门的保护和缓冲电路 …… 93
3.3.2 CMOS 漏极开路门和三态输出门电路 …… 94
3.3.3 CMOS 逻辑门电路的主要技术参数 …… 100
3.4 类 NMOS 和 BiCMOS 逻辑门电路 …… 106
3.4.1 类 NMOS 门电路 …… 106
3.4.2 BiCMOS 门电路 …… 108
3.5 TTL 逻辑门电路 …… 109
3.5.1 BJT 的开关特性 …… 109
3.5.2 TTL 反相器的基本电路 …… 110
3.5.3 抗饱和 TTL 门电路 …… 111
3.6 逻辑描述中的几个问题 …… 113
3.6.1 正负逻辑问题 …… 113
3.6.2 基本逻辑门的等效符号及其应用 …… 114
3.7 逻辑门电路使用中的几个实际问题 …… 116
3.7.1 各系列逻辑门电路之间的接口问题 …… 116
3.7.2 逻辑门电路驱动其他负载时的接口 …… 121
3.7.3 抗干扰措施 …… 122
3.7.4 差分信号传输 …… 123
3.7.5 CMOS 电路中的小尺寸逻辑和宽总线系列 …… 124
*3.8 用 Verilog HDL 描述 CMOS 门电路 …… 127
3.8.1 CMOS 门电路的 Verilog 建模 …… 127
3.8.2 CMOS 传输门电路的 Verilog 建模 …… 128
小结 …… 129
习题 …… 130

4 组合逻辑电路 …… 140
4.1 组合逻辑电路的分析 …… 140
4.1.1 组合逻辑电路的定义 …… 140
4.1.2 组合逻辑电路的分析方法 …… 140
4.2 组合逻辑电路的设计 …… 143
4.2.1 组合逻辑电路的设计过程 …… 143
4.2.2 组合逻辑电路的优化实现 …… 146
4.3 组合逻辑电路中的竞争-冒险 …… 149
4.3.1 产生竞争-冒险的原因 …… 149
4.3.2 消去竞争-冒险的方法 …… 150
4.4 常用组合逻辑电路模块 …… 152
4.4.1 编码器 …… 152
4.4.2 译码器/数据分配器 …… 157
4.4.3 数据选择器 …… 169

4.4.4 数值比较器 …… 178
4.4.5 算术运算电路 …… 181
4.5 组合逻辑的可编程电路实现 …… 189
4.5.1 PLD 的电路表示、编程技术及分类 …… 189
4.5.2 组合逻辑电路的 PLD 实现 …… 193
4.6 用 Verilog HDL 描述组合逻辑电路 …… 195
4.6.1 组合逻辑电路的行为级建模 …… 196
4.6.2 分模块、分层次的电路设计 …… 202
小结 …… 206
习题 …… 206

5 锁存器和触发器 …… 220
5.1 基本双稳态电路 …… 220
5.2 *SR* 锁存器 …… 221
5.2.1 基本 *SR* 锁存器 …… 221
5.2.2 门控 *SR* 锁存器 …… 225
5.3 *D* 锁存器 …… 227
5.3.1 *D* 锁存器的电路结构 …… 227
5.3.2 *D* 锁存器集成电路 …… 229
5.3.3 *D* 锁存器的动态特性 …… 230
5.4 *D* 触发器 …… 232
5.4.1 主从 *D* 触发器 …… 232
5.4.2 具有清零和置数输入的主从 *D* 触发器 …… 233
5.4.3 具有使能控制的主从 *D* 触发器 …… 236
5.4.4 其他结构的 *D* 触发器 …… 236
5.4.5 *D* 触发器的动态特性 …… 237
5.5 触发器的逻辑功能 …… 238
5.5.1 *D* 触发器 …… 239
5.5.2 *JK* 触发器 …… 240
5.5.3 *T* 触发器 …… 241
5.5.4 *SR* 触发器 …… 242
5.5.5 *D* 触发器逻辑功能的转换 …… 243
5.6 用 Verilog HDL 描述锁存器和触发器 …… 244
5.6.1 时序逻辑电路建模基础 …… 244
5.6.2 锁存器和触发器的 Verilog 建模实例 …… 246
小结 …… 249
习题 …… 249

6 时序逻辑电路 …… 259
6.1 时序逻辑电路的基本概念 …… 259
6.1.1 时序逻辑电路的基本结构与分类 …… 259
6.1.2 时序逻辑电路功能的表达 …… 262
6.2 同步时序逻辑电路的分析 …… 266
6.2.1 分析同步时序逻辑电路的一般步骤 …… 266
6.2.2 同步时序逻辑电路分析举例 …… 266
6.3 同步时序逻辑电路的设计 …… 272
6.3.1 设计同步时序逻辑电路的一般步骤 …… 273
6.3.2 同步时序逻辑电路设计举例 …… 274
6.3.3 同步时序逻辑电路中的时钟偏移 …… 285
6.4 异步时序逻辑电路的分析 …… 287
6.5 常用时序逻辑电路模块 …… 291
6.5.1 寄存器和移位寄存器 …… 291
6.5.2 计数器 …… 295
6.6 时序逻辑的可编程电路实现 …… 308
6.7 用 Verilog HDL 描述时序逻辑电路 …… 310

6.7.1 移位寄存器的 Verilog 建模 …… 310
6.7.2 计数器的 Verilog 建模 …… 311
6.7.3 状态转换图的 Verilog 建模 …… 313
6.7.4 数字钟的 Verilog 建模 …… 315
小结 …… 318
习题 …… 319

7 半导体存储器 …… 333
7.1 只读存储器 …… 333
7.1.1 ROM 的基本结构 …… 333
7.1.2 二维译码与存储阵列 …… 335
7.1.3 可编程 ROM …… 336
7.1.4 ROM 实例 …… 338
7.1.5 ROM 应用举例 …… 340
7.2 随机存取存储器 …… 341
7.2.1 SRAM …… 341
7.2.2 同步 SRAM …… 344
7.2.3 DRAM …… 345
7.2.4 存储容量的扩展 …… 347
7.2.5 RAM 应用举例 …… 349
小结 …… 351
习题 …… 351

8 FPGA 和 CPLD …… 354
8.1 现场可编程门阵列(FPGA) …… 354
8.1.1 FPGA 中编程实现逻辑功能的基本原理 …… 354
8.1.2 FPGA 的结构简介 …… 356
8.2 复杂可编程逻辑器件(CPLD)简介 …… 361
8.2.1 传统的 CPLD 结构 …… 361
8.2.2 基于查找表的 CPLD 结构 …… 364
8.3 可编程逻辑器件开发过程简介 …… 365
8.4 用 EDA 技术和可编程器件设计举例 …… 368
小结 …… 370
习题 …… 371

9 脉冲波形的变换与产生 …… 373
9.1 单稳态电路 …… 373
9.1.1 逻辑门组成的单稳态电路 …… 373
9.1.2 集成单稳态电路 …… 375
9.1.3 单稳态电路的应用 …… 378
9.2 施密特触发电路 …… 380
9.2.1 逻辑门组成的施密特触发电路 …… 380
9.2.2 集成施密特触发电路 …… 382
9.2.3 施密特触发电路的应用 …… 382
9.3 多谐振荡电路 …… 384
9.3.1 逻辑门组成的多谐振荡电路 …… 384
9.3.2 施密特触发电路组成的多谐振荡电路 …… 387
9.3.3 石英晶体多谐振荡电路 …… 387
9.4 555 定时器及其应用 …… 390
9.4.1 555 定时器 …… 390
9.4.2 用 555 组成的施密特触发电路 …… 391
9.4.3 用 555 组成的单稳态电路 …… 393
9.4.4 用 555 组成的多谐振荡电路 …… 395
小结 …… 396
习题 …… 396

10 数模与模数转换器 …… 402
10.1 D/A 转换器 …… 402

10.1.1 权电阻网络 D/A 转换器 …… 402
10.1.2 倒 T 形电阻网络 D/A 转换器 …… 404
10.1.3 权电流型 D/A 转换器 …… 408
10.1.4 电阻串联分压式 D/A 转换器 …… 410
10.1.5 D/A 转换器的双极性输出方式 …… 412
10.1.6 D/A 转换器的主要技术指标 …… 414
10.1.7 D/A 转换器的应用 …… 418
10.2 A/D 转换器 …… 420
10.2.1 A/D 转换的一般工作过程 …… 420
10.2.2 并行比较型 A/D 转换器 …… 423
10.2.3 逐次比较型 A/D 转换器 …… 427
10.2.4 双积分型 A/D 转换器 …… 430
10.2.5 A/D 转换器的主要技术指标 …… 433
10.2.6 集成 A/D 转换器及其应用 …… 434
小结 …… 437
习题 …… 437

11 数字系统设计基础 …… 441
11.1 数字系统概述 …… 441
11.1.1 数字系统的组成 …… 441
11.1.2 数字系统的设计方法 …… 442
11.1.3 数字系统的实现 …… 444
11.2 算法状态机 …… 444
11.2.1 ASM 图形符号 …… 444
11.2.2 ASM 图举例 …… 446
11.3 交通信号灯控制系统 …… 448
11.3.1 交通信号灯控制系统 ASM 图 …… 448
11.3.2 用典型电路基本模块设计交通灯控制系统 …… 451
11.3.3 交通灯控制系统 Verilog 建模 …… 453
11.4 数字密码锁 …… 460
11.4.1 数字密码锁的 ASM 图 …… 460
11.4.2 用典型电路基本模块设计数字密码锁 …… 464
11.4.3 数字密码锁 Verilog 建模 …… 466
小结 …… 471
习题 …… 472

附录 A 电气简图用图形符号——二进制逻辑单元(GB/T 4728.12—2008)简介 …… 475
A.1 二进制逻辑单元图形符号的组成 …… 475
A.2 限定性符号 …… 476
A.3 关联标注法 …… 479

附录 B 常用基本逻辑符号对照表 …… 481

附录 C 编写 Verilog HDL 测试模块 …… 483
C.1 测试模块的基本结构 …… 483
C.2 组合逻辑电路的测试 …… 483
C.3 时序逻辑电路的测试 …… 487
C.4 编译指令、系统任务和系统函数 …… 490
C.4.1 Verilog HDL 编译器指令 …… 490
C.4.2 Verilog HDL 系统任务 …… 491

C.4.3 Verilog HDL 系统函数 …… 494

附录 D ModelSim 仿真软件的使用 …… 496

附录 E Quartus Prime 软件的使用 …… 497

参考文献 …… 498

索引(汉英对照) …… 499

1 数字逻辑概论

引言

随着现代电子技术的发展，人们正处于一个信息时代。每天都要通过电视、广播、通信、互联网等多种媒体获取大量的信息。而现代信息的存储、处理和传输几乎都采用数字技术。在人们的日常生活中，常用的计算机、电视机、音响系统、视频记录设备、远距离通信等电子设备或电子系统中，无一不采用数字电路或数字系统。因此，数字电子技术的应用越来越广泛。

本章首先介绍数字技术的发展及应用、数字集成电路的分类及特点、模拟信号和数字信号以及数字信号的描述方法。然后讨论数制、二进制的算术运算、二进制代码和数字逻辑的基本运算。

文档 1.1
第 1 章思维导图

1.1 数字信号与数字电路

1.1.1 数字技术的发展及其应用

电子技术是 20 世纪以来发展最迅速、应用最广泛的技术，已经渗透到人类生活的各个方面。特别是数字电子技术，取得了令人瞩目的进步。

电子技术的发展是以电子器件的发展为基础的。20 世纪初直至中叶，主要使用真空管，也称电子管。随着固体微电子学的进步，第一只晶体管于 1947 年问世，开创了电子技术的新领域。随后，在 20 世纪 60 年代初，模拟和数字集成电路相继上市。20 世纪 80 年代末，微处理器每个芯片的晶体管数目突破百万大关。到 20 世纪 90 年代末，可以制造包含千万个晶体管的芯片。当前的制造技术可以在芯片上集成几十亿个晶体管。

数字技术应用的典型代表是电子计算机，它是伴随着电子技术的发展而发展的，经历了电子管、晶体管、集成电路以及微处理器四个发展阶段。计算机技术的影响已遍及人类经济生活的各个领域，使人类进入了一个数字化时代。现在数字技术已经在工业控制、医疗器械、交通工具、家用电器、手机等各种设备或产品中嵌入微处理器或微计算机，使这些产品向智能化方向发展。在过去的几十年间，集成电路的集成度和性能以惊人的速度发展着，印证了摩尔定律，即单个芯片上集成的晶体管数量每 18 个月就翻一番。由于高集成度产生的热量、半导体材料物理极限及加工工艺等问题的限制，摩尔定律终将失效，行业不再单纯追求

高集成度，而是开始寻求新型材料。

由于数字信息具有便于存储、处理和传输的特点，使得许多传统使用模拟技术的领域越来越多地运用数字技术。

音频信息存储 20世纪90年代以前，声音的存储主要以模拟信号的方式记录在唱片或磁带上，而两者的携带和保存都不方便。现在是将声音转换成数字信号存储在数字记录设备，如CD(compact disc)上。普通CD通常使用的取样频率为44.1 kHz，量化位数为16位，同时有两股声音被录制(分别流向立体声系统的两个扬声器)，而且可以存储长达74 min的音乐，这样一张CD盘上储存的数字信息总量超过700 MB。清晰度更高的CD，其音频采样率可达192 kHz，量化位数为24位。

视频信息存储 早期的视频信息存储主要以记录模拟信号的录像带为主。现在采用数字技术存储处理传输视频信息。如现在的蓝光光盘BD(blu-ray disc)，采用MPEG4[①]的高压缩比数字格式，双层容量为40 GB，可以录制2小时的高清视频。

照相机 传统的模拟相机是用卤化银感光胶片记录影像，胶片成像过程需要严格的加工工艺和技术，而且胶片不便于保存和传输。数码相机是将影像的光信号转换为数字信号，以像素阵列的形式进行存储。例如1920×1080的像素阵列中，每个像素的红、绿、蓝三原色均是8位，则其数据量近于50 MB。如果用JPEG[②]图形格式进行压缩处理，压缩比率采用20∶1，处理后的数据量只为原来的5%，便于进行网络的远距离传输。如今，数码相机不仅完全取代模拟胶片相机，而且集成在手机、无人机等智能产品中。

交通灯控制系统 交通灯于1920年问世。早期的红黄绿灯是用机电定时器控制的。后来使用继电器构成的控制器，根据道路上传感器检测的信号进行控制。现在采用计算机控制，可以将监测系统检测到的车辆流量信息送到系统计算机，经计算后进行合理的时间分配。如某路口东西方向堵塞，则将该路口东西方向的绿灯自动延时，并将附近区域东西方向的红灯也自动地延时，堵塞解除后，信号灯恢复正常状态。

电子技术的发展及应用正在改变着人类的生产方式、生活方式及思维方式。人类的学习、生活、工作所使用的物品都会是数字化的产品。在任何时间、任何地点，用任何设备所获得的信息多以数字的形式存在。但是，无论数字技术如何发展，也不能代替模拟技术。自然界中绝大多数物理量都是模拟量，数字技术不能直接接受模拟信号进行处理，也无法将处理后的数字信号直接送到外部物理世界。因此，模拟技术在电子系统中是不可缺少的。由于模拟技术难度远高于数字技术，其发展自然较慢。实际电子系统一般是模拟电路和数字电路的结合，在发展数字技术的同时，也应重视模拟技术的发展。

---

① MPEG(Moving Picture Experts Group)是世界数字视频和音频压缩比的标准化组织制定的，用于多媒体运动图像和伴音的数据压缩编码的国际标准。MPEG标准主要包括MPEG1、MPEG2、MPEG4、MPEG7和MPEG21等，以适用于不同带宽和数字影像质量的要求。最高压缩比可达200∶1。

② JPEG(Joint Photographic Experts Group)是国际标准化组织(International Standard Organization，ISO)和国际电报电话咨询委员会(Consultative Committee for International Telegraph and Telephone，CCITT)联合制定的静止图像压缩编码标准。JPEG有多种压缩级别，压缩比率通常在10∶1到40∶1之间。

1.1.2 数字集成电路的分类及特点

电子电路可以分为模拟电路和数字电路。根据电路的结构特点及其对输入信号的响应规则的不同,数字电路可分为组合逻辑电路和时序逻辑电路。数字电路中的电子器件,如二极管、三极管(FET、BJT)处于开关状态,时而导通,时而截止,构成电子开关。这些电子开关是组成逻辑门电路的基本器件。逻辑门电路又是数字电路的基本单元。如果将这些门电路集成在一片半导体芯片上就构成数字集成电路。

1. 数字集成电路的分类

数字电路的发展历史与模拟电路一样,经历了由电子管、晶体管等半导体分立器件到集成电路的发展过程,但比模拟电路发展得更快。由于制造工艺的不断改进,产生出种类繁多的标准化、系列化的集成电路产品。

衡量集成电路设计和制造技术水平的两个主要参数是集成度与特征尺寸。集成度是指每一芯片所包含的晶体管的个数。特征尺寸是指集成电路中半导体器件加工的最小线条宽度。集成度与特征尺寸是相关的。当集成电路芯片的面积一定时,集成度越高,特征尺寸就越小。所以,特征尺寸也成为一个重要指标。

在过去的几十年中,以硅为主要加工材料的微电子制造工艺从开始的微米级加工水平,经历亚微米级、深亚微米级,到现在的纳米级技术,集成电路芯片的集成度越来越高,成本越来越低。

从集成度来说,数字集成电路可分为小规模集成电路(small scale integration,SSI)、中规模集成电路(medium scale integration,MSI)、大规模集成电路(large scale integration,LSI)、超大规模集成电路(very large scale integration,VLSI)和甚大规模集成电路(ultra large scale integration,ULSI)五类。表 1.1.1 列出了数字集成电路的集成度分类。

表 1.1.1 数字集成电路的集成度分类

分类	晶体管的个数	典型集成电路
小规模	小于 100 个	逻辑门、触发器
中规模	$10^2 \sim 10^3$	计数器、加法器
大规模	$10^3 \sim 10^5$	小型存储器、门阵列
超大规模	$10^5 \sim 10^6$	大型存储器、微处理器
甚大规模	10^6以上	可编程逻辑器件、多功能专用集成电路

按照结构和工艺分为双极型、MOS 型和双极-MOS 型。三极管-三极管(transistor-transistor logic,TTL)逻辑门电路问世较早,其工艺经过不断改进,曾广泛应用于中小规模集成电路设计中。随着金属-氧化物-半导体(MOS)工艺特别是互补 MOS(complementary metal-oxide-semiconductor,CMOS)工艺的发展,大大提高了电路的集成度和工作速度,且明显降低了功耗,因此 TTL 的主导地位已被 CMOS 器件所取代。

根据集成芯片逻辑功能的特点,可分为通用型和专用型。小规模、中规模数字集成电路

的逻辑功能是固定的，构成大型逻辑电路时所用芯片种类和数量多、体积大、可靠性差。后来发展了专用集成电路(application specific integrated circuit，ASIC)和可编程逻辑器件(programmable logic device，PLD)。

ASIC 是根据用户特定要求和电子系统的特定需要而设计制造的专用集成电路。ASIC 芯片的制作可以采用全定制或半定制的方法。全定制适用于生产批量的成熟产品，由半导体生产厂家制造。对于生产批量小或研究试制阶段的产品，可以采用半定制方法。可编程逻辑器件 PLD 属于可编程 ASIC，是一种由用户定义和设置逻辑功能的集成电路。PLD 种类较多，目前广泛应用的主要有复杂可编程逻辑器件(complex programmable logic device，CPLD)和现场可编程门阵列(field programmable gate array，FPGA)。用户通过软件编程，将自己设计的数字系统制作在厂家生产的 CPLD 或 FPGA 芯片上，便得到所需的系统级芯片。与 ASIC 相比，PLD 不需要介入芯片的布局布线和工艺问题，而且可以随时改变其逻辑功能，使用灵活。

2. 数字集成电路的特点

与模拟电路相比，数字电路主要有下列优点。

(1) 稳定性高，抗干扰能力强

一般而言，对于一个给定的输入信号，如果噪声信号不使其超过阈值，数字电路的输出状态不变。因此数字电路的工作可靠，稳定性好，抗干扰能力强。而模拟电路的输出则随着外界温度和电源电压的变化，以及器件的老化等因素而发生变化。

(2) 易于设计

数字电路是对用 0 和 1 表示的数字信号进行逻辑运算和处理，使用的数学工具是逻辑代数。数字电路只要能够可靠地区分 0 和 1 两种状态就可以正常工作。因此，数字电路的分析与设计相对较容易。

(3) 便于集成，成本低廉

将广泛使用的电路大批量生产为集成芯片，不仅体积小，通用性强，还可以降低制造成本。小规模的电路或系统的设计及组成可以采用通用芯片实现。大规模的系统可以采用可编程器件实现。

(4) 可编程性及保密性

现代数字系统的设计，大多采用可编程逻辑器件。用户根据需要用硬件描述语言(hardware description language，HDL)在计算机上完成电路设计和仿真，并写入芯片，这为用户研制开发产品带来了极大的方便和灵活性。为了保证芯片内部信息的安全性，可以对芯片进行加密保护。

(5) 高速度，低功耗

随着集成电路工艺的发展，数字器件的工作速度越来越高，而功耗越来越低。集成电路中单管的开关速度可以做到小于 10 ps。整体器件中，信号从输入到输出的传输时间低于 1 ns。百万门以上超大规模集成芯片的功耗，可以低于毫瓦级。

(6) 可扩展性

数字技术发展迅速，对电路和系统进行设计时，都增加了扩展端口，以便将来能够对电路或系统进行扩展或升级。

由于具有这些优点，可以肯定，数字电路在众多领域取代模拟电路的趋势将会继续发展

下去。

3. 数字电路的分析、设计与测试

(1) 数字电路的分析方法

数字电路处理的是数字信号,电路中的半导体器件工作在开关状态,如 MOS 管工作在可变电阻区或截止区,所以不能采用模拟电路使用的小信号模型等方法进行分析。数字电路又称为逻辑电路,主要研究对象是电路的输出与输入之间的逻辑关系,所采用的分析工具是逻辑代数,表达电路输出与输入的关系主要用真值表、功能表、逻辑表达式或波形图。

随着计算机技术的发展,借助电子设计自动化(electronic design automation,EDA)技术,可以更直观、更全面地分析电路的行为,以便检查是否存在逻辑错误或时序错误。EDA 技术包括计算机在过程规划、设计、工程及制造等不同领域的应用。其中之一是用于集成电路设计仿真到元器件布局布线的各种计算机辅助设计(computer aided design,CAD)工具,如 Xilinx 公司的 Vivado、Intel(前身为 Altera)公司的 Quartus 等。

(2) 数字电路的设计方法

数字电路的设计是根据给定的逻辑功能要求,设计出符合要求的逻辑电路。设计过程一般有方案的提出、验证和修改三个阶段。设计方法分为传统设计方法和现代设计方法。传统设计方法是自下而上的设计方法,硬件电路设计全过程都是由人工完成,验证和调试是在电路构成后进行的,电路存在问题只能在验证后发现。如果存在的问题较大,有可能重新设计电路,因而设计周期长,资源浪费大。现代设计方法是自上而下的设计方法。设计者提出方案后,借助于计算机及 CAD 工具完成电路的详细设计过程,如果发现错误,可以重复设计过程直至得到满意的电路,然后用可编程逻辑器件 PLD 进行硬件电路的实现。这种方法缩短了设计周期,节省了设计费用,提高了设计质量。因此基于 CAD 的设计方法已成为设计数字电路的基本方法。

CAD 软件的种类较多,大多数软件包含以下主要工具。

原理图输入　设计者可以如同在纸上画电路一样,将逻辑电路图输入到计算机。

HDL 文本输入　硬件描述语言是用文本的形式描述硬件电路。它虽然没有图形输入那么直观,但更便于进行大规模数字系统的设计。因此,成为主要的输入方法。常用的 HDL 语言有 VHDL(very high speed integrated circuit)和 Verilog HDL 等。

测试平台与仿真　当逻辑电路的设计输入到计算机后,需要在测试平台上编写或绘制激励信号,对电路进行功能仿真和时序仿真,以便验证电路是否满足设计要求。

综合工具　将 HDL 描述的电路的逻辑关系,转换为门和触发器等元件及其相互连接的电路形式。

(3) 数字电路的测试技术

数字电路在正确设计和安装后,必须经过严格的测试方可使用。测试中小规模数字电路时,常使用数字电压表、数字示波器或逻辑分析仪等设备,以便验证测试结果是否与理论分析一致。

集成电路的复杂程度越高,测试越困难。大规模集成电路的测试系统由计算机和测试仪等硬件,以及操作系统和测试需要的各种软件组成。对于系统芯片的测试成本几乎占芯片成本的一半。集成电路测试面临的最大挑战是如何降低测试成本。

1.1.3 模拟信号和数字信号

1. 模拟信号

人们在自然界感知的许多物理量中，有一些物理量如速度、压力、温度、声音、重量以及位置等具有一个共同的特点，即它们在时间上是连续变化的，幅值上也是连续取值的。这种连续变化的物理量称为模拟量，表示模拟量的信号称为模拟信号，处理模拟信号的电子电路称为模拟电路。在工程技术上，为了便于处理和分析，通常用传感器将模拟量转换为与之成比例的电压或电流信号，然后再送到电子系统中进一步处理。图 1.1.1(a)中给出了由热电偶得到的一个模拟电压信号波形。

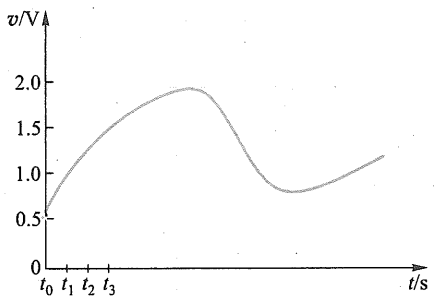
2. 数字信号

与模拟量相对应的另一类物理量称为数字量。它们是在一系列离散的时刻取值，数值的大小和每次的增减都是量化单位的整数倍，即它们是一系列时间离散、数值也离散的信号。表示数字量的信号称为数字信号。将处理数字信号的电子电路称为数字电路。例如用温度计测量某一天内的温度变化，仅在整点时刻读取数据，并且对数据进行量化，若某次温度计的读数为 30.35…℃，取 1 ℃作为量化单位，则温度记录值为 30 ℃。这样，一天内的温度记录在时间上和数值上都是离散的，温度是以 1 ℃为单位增加或减少。显然，用数字信号也可以表示其他各种物理量的大小，只是存在着一定的误差，误差取决于量化单位大小的选择。

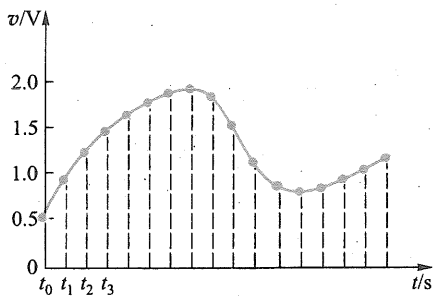
随着计算机的广泛应用，绝大多数电子系统都采用计算机对信号进行处理。由于计算机无法直接处理模拟信号，所以需要将模拟信号转换为数字信号。

3. 模拟量的数字表示

图 1.1.1 所示为转换过程中的各种波形图，图 1.1.1(a)所示是模拟电压信号。首先对模拟信号取样。图 1.1.1(b)所示为模拟信号通过取样电路后，变成时间离散、幅值连续的取样信号，t_0、t_1、t_2、…为取样时间点。这里，幅值连续是指各取样点的幅值没有量化，仍然与对应的模拟信号的幅值相同，如图 1.1.1(a)和图 1.1.1(b)中 t_1 处的幅值均为模拟量 0.915…V。然后对取样信号进行量化，即数字化。选取一个量化单位，将取样信号除以量化单位并取整数结果，得到时间离散、数值也离散的数字量。最后对得到的数字量进行编码，生成用 **0** 和 **1** 表示的数字信号，如图 1.1.1(c)所示。图中以 0.1 V 作为量化单位，对 t_1 处的幅值 0.915…V 进行量化，量化后数值为 9。该值用 8 位二进制数表示为 **00001001**。如果取样点足够多，量



(a)



(b)

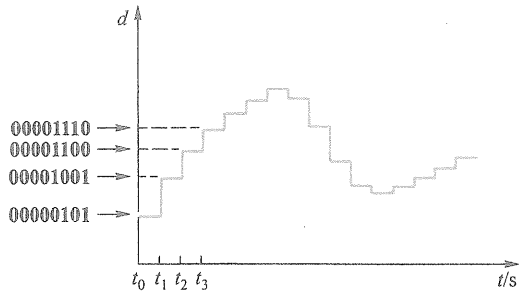


图 1.1.1 模拟量的数字表示

(a) 模拟电压信号 (b) 取样信号 (c) 数字信号

化单位足够小,数字信号可以较真实地反映模拟信号。关于模数和数模转换的详细讨论见本书第 10 章的内容。

1.1.4 数字信号的描述方法

模拟信号可以用数学表达式或波形图等表示。数字信号则用 0、1 两种值表示,即二值数字逻辑(binary digital logic);或高、低电平组成的数字波形,即逻辑电平(logic level)表示。

1. 二值数字逻辑和逻辑电平

在数字电路中,可以用 0 和 1 组成的二进制数表示数量的大小,也可以用 0 和 1 表示两种不同的逻辑状态。当表示数量时,两个二进制数可以进行数值运算,常称为算术运算,将在 1.3 节介绍。当用 0 和 1 描述客观世界存在的彼此相互关联又相互对立的事物状态时,例如,是与非,真与假,开与关,低与高,通与断等,这里的 0 和 1 不是数值,而是逻辑 0 和逻辑 1。这种只有两种对立逻辑状态的逻辑关系称为二值数字逻辑或简称数字逻辑。

在电路中,可以很方便地用电子器件的开与关来实现二值数字逻辑。也就是以高、低电平分别表示逻辑 1 和 0 两种状态。在分析实际数字电路时,考虑的是信号之间的逻辑关系,只要能区别出表示逻辑状态的高、低电平,就可以忽略高、低电平的具体数值。表 1.1.2 所示为某类 CMOS 器件的电压范围与逻辑电平之间的关系。当信号电压在 3.5~5 V 范围内,都表示高电平;在 0~1.5 V 范围内,都表示低电平。这些表示数字信号电压的高、低电平通常称为逻辑电平。应当注意,逻辑电平不是物理量,而是物理量的相对表示。

表 1.1.2 电压范围与逻辑电平的关系

电压	二值逻辑	电平
3.5~5 V	1	H(高电平)
0~1.5 V	0	L(低电平)

图 1.1.2 所示为用逻辑电平描述的数字波形,用逻辑 0 表示低电平,逻辑 1 表示高电平。其中图 1.1.2(a)所示波形标出时间及幅值。通常在分析一个数字系统时,由于电路采用相同的逻辑电平标准,一般可以不标出高、低电平的电压值。时间轴也可以不标,如图 1.1.2(b)所示。

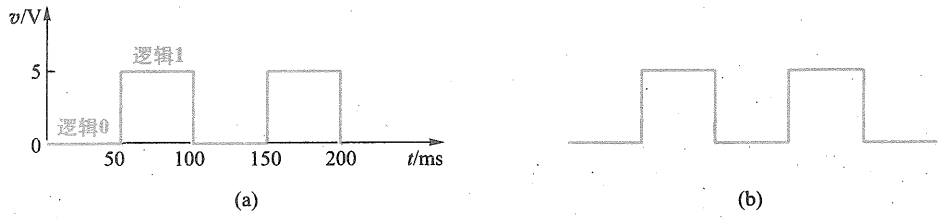


图 1.1.2 数字波形

(a) 标明时间及幅值的数字波形 (b) 数字波形的常规表示

2. 数字波形

(1) 数字波形的两种类型

数字波形是逻辑电平对时间的图形表示。数字信号有两种传输波形,一种是非归零型,另一种是归零型。在图 1.1.3 中,一定的时间间隔 T,称为 1 位(1bit),或者一拍。如果在一个时间拍内用高电平代表 1,低电平代表 0,称为非归零型,如图 1.1.3(a)所示。如果在一个时间拍内有脉冲代表 1,无脉冲代表 0,称为归零型,如图 1.1.3(b)所示。两者的区别在于高电平的表示方法不同,在一个时间拍内,非归零型信号持续为高电平,而归零型信号的高电平持续一段时间后会归零。只有作为时序控制信号使用的时钟脉冲是归零型,除此之外的大多数数字信号基本都是非归零型,非归零型信号使用较为广泛。

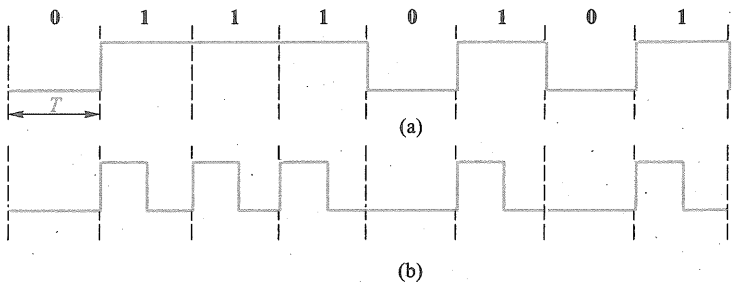


图 1.1.3 数字信号的传输波形

(a) 非归零型信号 (b) 归零型信号

数字信号只有两种取值,故称为二值信号,数字波形又称为二值位形图。非归零型信号的每位数据占用一拍时间。每秒钟所传输数据的位数称为数据率或比特率(Bit Rate)。

例 1.1.1 某通信系统每秒钟传输 1544000 位(1.544 兆位)数据,求每位数据的时间。

解:按题意,每位数据的时间为

$$\left(\frac{1.544\times10^{6}}{1\ \text{s}}\right)^{-1}=647.67\times10^{-9}\ \text{s}\approx648\ \text{ns}$$

(2) 周期性和非周期性

与模拟波形相同,数字波形亦有周期性与非周期性之分。图 1.1.4(a)、(b)所示分别为周期性与非周期性数字波形举例。

周期性数字波形常用周期 T 和频率 f 来描述。脉冲波形的脉冲宽度用 t_w 表示,它表示脉冲的作用时间。另一个重要参数是占空比 q,它表示脉冲宽度 t_w 占整个周期 T 的百分数,

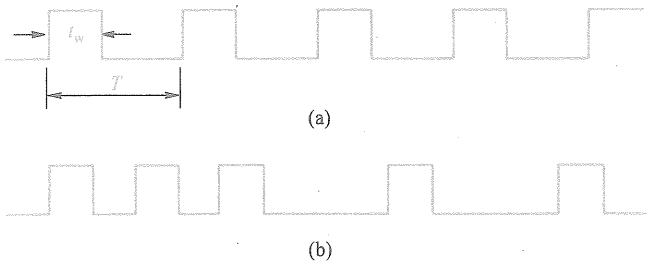


图 1.1.4 数字波形

(a) 周期性数字波形 (b) 非周期性数字波形

常用下式来表示

$$q(\%)=\frac{t_w}{T}\times 100\% \tag{1.1.1}$$

当占空比为 50%时,称此时的矩形脉冲为方波。

例 1.1.2 设周期性数字波形的高电平持续 6 ms,低电平持续 10 ms,求占空比 q。

解:因数字波形的脉冲宽度 $t_w=6$ ms,周期 $T=6$ ms+10 ms=16 ms。

$$q=\frac{6\text{ ms}}{16\text{ ms}}\times 100\%=37.5\%$$

(3) 实际数字信号波形

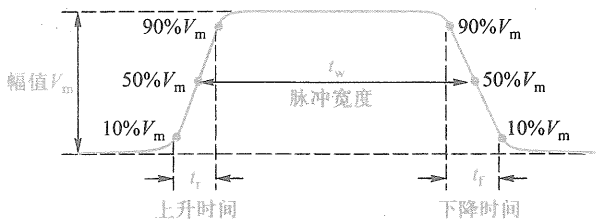


图 1.1.5 非理想脉冲波形

在实际的数字系统中,数字信号并没有那么理想。当矩形脉冲从低电平跳变到高电平,或从高电平跳到低电平时,边沿没有那么陡峭,而要经历一个过渡过程,分别用上升时间 t_r 和下降时间 t_f 描述,如图 1.1.5 所示。若脉冲幅值为 V_m,将矩形脉冲上升沿从 10%V_m 到 90%V_m 时所经历的时间称为上升时间 t_r。下降时间则相反,将矩形脉冲下降沿从 90%V_m 下降到 10%V_m 时所经历的时间称为下降时间 t_f。将脉冲上升沿的 50%V_m 到下降沿的 50%V_m 两个时间点所跨越的时间称为脉冲宽度 t_w,对于不同类型的器件和电路,其上升和下降时间各不相同。一般数字信号上升和下降时间的典型值约为几纳秒(ns)。

例 1.1.3 试绘出一幅值为 5 V 的脉冲波形,设它的占空比为 50%,脉冲宽度为 100 ns,上升时间为 10 ns,而下降时间为 20 ns。

解:根据题意,所绘出的脉冲波形如图 1.1.6 所示。

本书所用的数字波形大多数将画成理想波形。实际上,每一波形均有上升时间 t_r 和下降时间 t_f。当我们关注各信号之间的逻辑关系时,不必在每一波形上表示出上升和下降时

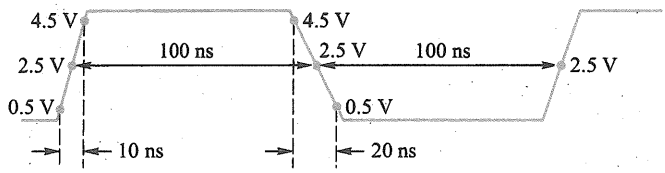


图 1.1.6 例 1.1.3 的波形图

间。只有当研究数字电路的翻转特性时,才绘出各信号的上升时间和下降时间。

(4) 波形图和时序图

数字波形是表达数字电路动态特性的有效工具之一。将数字电路输入变量的每一种取值与相应的输出值按照时间顺序依次排列得到的图形,称为**波形图**(waveform)。

在时序电路中,电路的状态和输出对时钟脉冲序列及输入信号响应的波形图又称为**时序图**(timing diagram)。当用示波器、逻辑分析仪测试电路,或用 CAD 工具仿真时,时序图可以用来描述逻辑电路的行为。对于实际构成电子电路的逻辑门,其输出值的变化相对输入的变化存在一定的延迟,我们采用反映延迟时间的时序图表示。为便于比较,图 1.1.7 所示为 D 触发器的时序图。图 1.1.7(a)中,CP 为时钟控制输入,D 为数据输入,Q 为输出。在图 1.1.7(b)中为保证电路稳定工作,D 输入端的信号比 CP 上升沿提前至少 t_{SU}(setup time,建立时间),并且要求其在 CP 上升沿来之后保持至少 t_H(hold time,保持时间),其他时间参数在后续章节介绍。通常数字集成电路,例如存储器和时序逻辑器件的数据手册里均有时序图,以便于进行数字系统的分析、设计和应用。

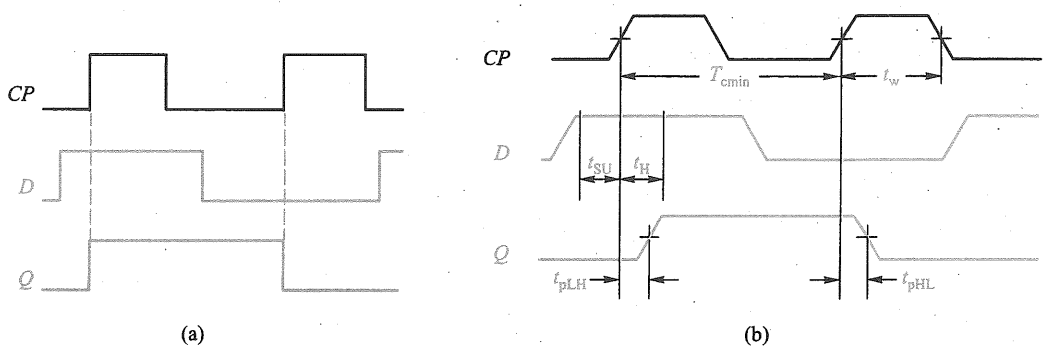


图 1.1.7 D 触发器的波形图

(a) 时序图 (b) 反映延迟时间的时序图

复习思考题

1.1.1 衡量集成电路有哪两个主要的参数?

1.1.2 数字电路从集成度来看可分为几大类?

1.1.3 数字电路有哪些优点?

1.1.4 为什么数字逻辑称为二值数字逻辑?

1.1.5 实际数字波形和理想数字波形有什么不同?

1.2 数制

人们在日常生活中经常遇到计数问题，并且习惯于用十进制数。而在数字系统，例如计算机中，通常采用二进制数，有时也采用十六进制数或八进制数。这种多位数码的构成方式以及从低位到高位的进位规则称为数制。

1.2.1 十进制

众所周知，中国的算盘是一个十进制计数器。任何一个数都可以用 0，1，2，3，4，5，6，7，8，9 这 10 个数码中的一个或几个，按一定的规律排列起来表示，其计数规律是“逢十进一”即 9+1=10，其中左边的“1”为十位数，右边的“0”为个位数，也就是 $10=1\times10^1+0\times10^0$。所谓十进制就是以 10 为基数的计数体制。

这样，每一数码处于不同的位置时，它所代表的数值是不同的。例如，十进制数 4 587.29 可以表示为

$$4\ 587.29=4\times10^3+5\times10^2+8\times10^1+7\times10^0+2\times10^{-1}+9\times10^{-2}$$

式中 10^3、10^2、10^1 和 10^0 分别为千位、百位、十位和个位数码的权，而小数点以右数码的权值是 10 的负幂。这与算盘横梁上所标示的个、十、百、千的位权是相同的。

一般地说，任意十进制数可表示为

$$(N)_D^{①}=\sum_{i=-\infty}^{\infty}K_i\times10^i \tag{1.2.1}$$

式中 K_i 为基数“10”的第 i 次幂的系数，它可以是 0~9 中任何一个数字。

如果将式(1.2.1)中的 10 用字母 R 来代替，就可以得到任意进制数的表达式

$$(N)_R=\sum_{i=-\infty}^{\infty}K_i\times R^i \tag{1.2.2}$$

式中 K_i 是第 i 次幂的系数，根据基数 R 的不同，它的取值为 0 到 $R-1$ 个不同的数码。例如对于十进制数，R 为 10，所以 K_i 的取值为 0~9 共 10 个数码。

数字电路不便于存储或处理十进制数。因为构成数字电路的基本思路是把电路的状态与数码对应起来。二进制只有 0 和 1 两个数码，分别用数字电路的两种状态来表示。而十进制的 10 个数码要求电路有 10 个完全不同的状态，这样使得电路很复杂，因此在数字电路中不直接处理十进制数。

1.2.2 二进制

1. 二进制的表示方法

二进制数中，只有 0 和 1 两个数码，并且计数规律是“逢二进一”，即 1+1=10(读为“壹零”)。必须注意，这里的“10”与十进制数的“10”是完全不同的，它并不代表数“拾”。左边的“1”表示 2^1 的系数，右边的“0”表示 2^0 的系数，也就是 $10=1\times2^1+0\times2^0$。因此，所谓二进制

① 下标 D(decimal)表示十进制。

就是以 2 为基数的计数体制。在计算机中，二进制数中的 1 位称为 1bit（binary digital 的缩写），8bit 组成一个字节，称为 byte。

根据式（1.2.2），任意二进制数可表示为

$$(N)_B^{①} = \sum_{i=-\infty}^{\infty} K_i \times 2^i \tag{1.2.3}$$

式中 K_i 为基数“2”的第 i 次幂的系数，它可以是 0 或者 1。这样式（1.2.3）也可以作为二进制数转换为十进制数的转换公式。

例 1.2.1　试将二进制数 $(\mathbf{1010110})_B$ 转换为十进制数。

解：将每一位二进制数与其位权相乘，然后相加便得相应的十进制数。

$$(\mathbf{1010110})_B = 1\times2^6+0\times2^5+1\times2^4+0\times2^3+1\times2^2+1\times2^1+0\times2^0 = (86)_D$$

2. 二进制的优点

与十进制相比较，二进制具有一定的优点，因此它在计算机技术中被广泛采用。

（1）二进制的数字电路简单可靠，所用元件少

二进制数的每一位只有 **0** 和 **1** 两个状态，可用任何具有两个不同稳定状态的元件来表示，如 MOS 管的导通与截止，继电器触点的闭合与断开，灯的亮和不亮等。只要规定其中一种状态表示 **1**，另一种状态表示 **0**，就可以表示二进制数。这样，数码的存储、分析和传输，就可以用简单而可靠的方式进行。

（2）二进制的基本运算规则简单，运算操作方便

二进制只有两个数码 **0** 和 **1**，基本运算规则简单。

但是，采用二进制也有缺点。用二进制表示一个数时，位数太多，例如，十进制数 49 表示为二进制时，即为 **110001**，不便于书写和阅读，使用不方便。因此，通常在阅读和书写时采用十进制数。在送入计算机时，计算机先将十进制数转换成数字系统能接受的二进制数；而在运算结束后，再将二进制数转换为十进制数，输出十进制结果。

3. 二进制数的波形表示

在数字电路中，二值数据常用数字波形来表示。图 1.2.1 所示为 16 位数据 **0010111100111010** 的波形表示。数据的波形表示比较直观，也便于用示波器进行监视。图 1.2.2 所示为一计数器的波形，图中最左列标出了二进制数的位权（2^0、2^1、2^2、2^3）以及最低有效位（least significant bit，LSB）和最高有效位（most significant bit，MSB），最后一行标出了从 0 到 15 的等效十进制数。

从图 1.2.2 还可看出，每一位的波形均为方波，其占空比均为 50%，但从低位到高位，每一波形的频率逐位减半。

图 1.2.1　16 位数据的波形表示

① 下标 B（binary）表示二进制。

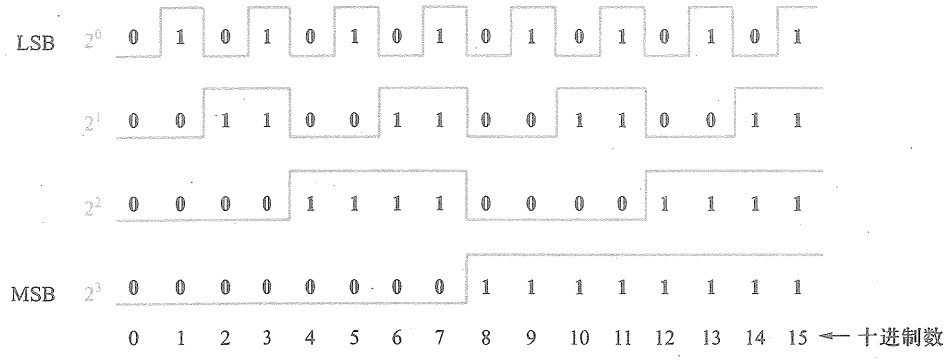


图 1.2.2 用二进制数表示 0~15 波形图

4. 二进制数据的传输

二进制数据从一处传输到另一处，可以采用串行的方式或并行的方式。对于串行传输方式，一组数据在时钟脉冲的控制下逐位传送。串行方式所需的设备简单，只需一根数据传输线和一个共同接地端等。计算机与打印机之间通过通用串行总线 USB(universal serial bus)连接，数据传输采用串行方式。USB3.0 的数据传输速度可达 5Gbps(bits per second)(实际的 USB 是采用差分传输，见 3.7.4 节)。

以 8 位二进制数为例，串行传输的示意图如图 1.2.3 所示，图中只画出数据传输线。图 1.2.3(a)表示二进制数 **00110110** 从计算机串行传送到打印机。图 1.2.3(b)是在时钟脉冲 *CP* 的控制下，数据由最高位 MSB 到最低位 LSB 依次传输的波形图。每一个时钟周期传送一位数，并且在时钟脉冲的下降沿完成。

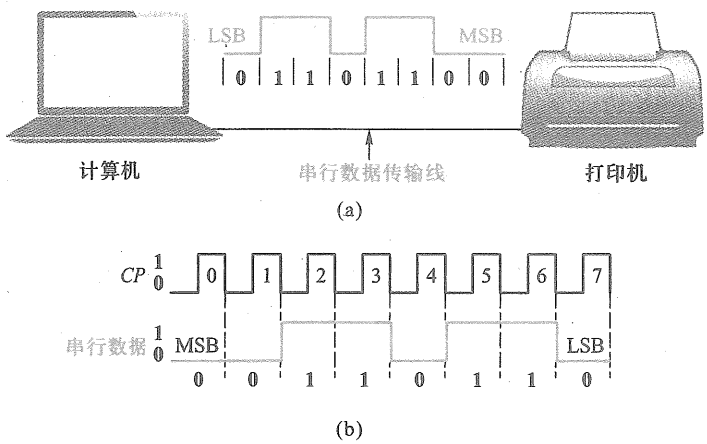


图 1.2.3 二进制数据的串行传输

(a) 计算机与打印机之间的串行通信 (b) 二进制数据的串行表示

若要求传输速度快，则可采用并行传输方式，即将一组二进制数据同时传送。大多数计算机和一些数字系统内部都是以并行方式传输数据。我们仍然以 8 位数为例，并行传输的示意图如图 1.2.4 所示，图中只画出数据传输线。图(b)中最左列标出了二进制数的位权(2^0、2^1、…、2^7)以及最低位 LSB 和最高位 MSB。传输 8 位数据所需的时间为一个时钟脉冲

周期，只有串行传输时间的 1/8。但所需设备复杂，需用 8 条数据传输线和其他部件。

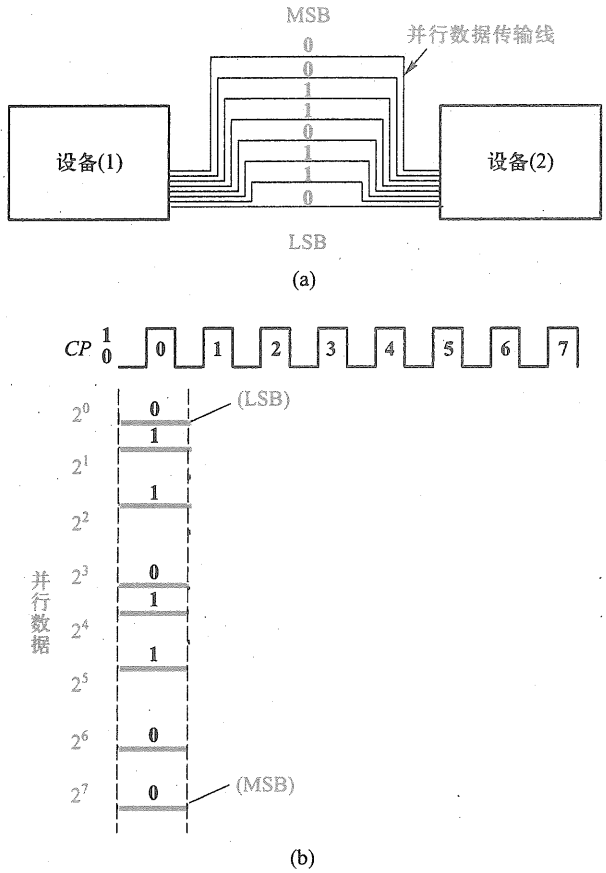


图 1.2.4 并行传输数据的示意图

(a) 设备之间的并行通信 (b) 二进制数据的并行表示

1.2.3 十-二进制之间的转换

既然同一个数可以用二进制和十进制两种不同形式来表示，那么两者之间必然有一定的转换关系。前面已经介绍了二进制数转换为十进制数的方法，是将每位二进制数与其权相乘，然后相加便得到相应的十进制数。十进制数转换为二进制数时，整数部分和小数部分的方法不同，下面分别介绍。

对于整数部分可写成

$$(N)_D = b_n \times 2^n + b_{n-1} \times 2^{n-1} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 \tag{1.2.4}$$

式中 b_n、b_{n-1}、…、b_1、b_0是二进制数各位数字。将等式两边分别除以 2，得

$$\frac{1}{2}(N)_D = b_n \times 2^{n-1} + b_{n-1} \times 2^{n-2} + \cdots + b_1 \times 2^0 + \frac{b_0}{2} \tag{1.2.5}$$

由此可知，将十进制数除以 2，其余数为 b_0。得到的商为

$$b_n \times 2^{n-1} + b_{n-1} \times 2^{n-2} + \cdots + b_1$$

可以写成

$$b_n\times 2^{n-1}+b_{n-1}\times 2^{n-2}+\cdots+b_1=2\left(b_n\times 2^{n-2}+b_{n-1}\times 2^{n-3}+\cdots+b_2+\frac{b_1}{2}\right) \tag{1.2.6}$$

将式(1.2.6)再除以2,其余数为 b_1。不难推知,将十进制整数每除以一次2,就可根据余数得到二进制数的1位数字。因此,只要连续除以2直到商为0,就可由所有的余数求出二进制数。

例 1.2.2 将十进制数 $(37)_D$ 转换为二进制数。

解:根据上述原理,可将 $(37)_D$ 按如下的步骤转换为二进制数:

$$\begin{array}{r|l} 2 & 37 \cdots\cdots \text{余 } 1 \cdots\cdots b_0 \\ \hline 2 & 18 \cdots\cdots \text{余 } 0 \cdots\cdots b_1 \\ \hline 2 & 9 \cdots\cdots \text{余 } 1 \cdots\cdots b_2 \\ \hline 2 & 4 \cdots\cdots \text{余 } 0 \cdots\cdots b_3 \\ \hline 2 & 2 \cdots\cdots \text{余 } 0 \cdots\cdots b_4 \\ \hline 2 & 1 \cdots\cdots \text{余 } 1 \cdots\cdots b_5 \\ \hline & 0 \end{array}$$

由上得

$$(37)_D=(\mathbf{100101})_B$$

当十进制数较大时,不必逐次除2,而是将十进制数和与其相当的2的幂项对比,使转换过程得到简化。

例 1.2.3 将 $(133)_D$ 转换为二进制数。

解:由于 2^7 为128,而 $133-128=5=2^2+2^0$,所以对应二进制数 $b_7=1, b_2=1, b_0=1$,其余各系数均为0,所以得

$$(133)_D=(\mathbf{10000101})_B$$

值得指出,数字电路或数字系统处理的二进制数多数是4、8、16、32或64位等。因此,数据的位数需配成规格化的位数,如例1.2.2中转换结果为**100101**,如将它配成8位,则在该二进制数的最左边补充两个**0**,其值不变,即

$$\mathbf{100101=00100101}$$

对于二进制的小数部分可写成

$$(N)_D=b_{-1}\times 2^{-1}+b_{-2}\times 2^{-2}+\cdots+b_{-(n-1)}\times 2^{-(n-1)}+b_{-n}\times 2^{-n} \tag{1.2.7}$$

将上式两边分别乘以2,得

$$2\times(N)_D=b_{-1}\times 2^{0}+b_{-2}\times 2^{-1}+\cdots+b_{-(n-1)}\times 2^{-(n-2)}+b_{-n}\times 2^{-(n-1)} \tag{1.2.8}$$

由此可见,将十进制小数乘以2,所得乘积的整数即为 b_{-1}。不难推知,每次乘以2得到的积中的整数部分就是二进制小数相应位权的系数,将积中的小数部分再乘以2,直到小数部分为0,或小数部分不为0,但满足误差要求进行"四舍五入"为止,就可完成将十进制小数转换成二进制小数。

例 1.2.4 将$(0.706)_D$转换为二进制数，要求其误差不大于2^{-10}。

解：按上面介绍的方法计算，可得b_{-1}、b_{-2}、…、b_{-9}如下：

$$0.706\times2=1.412\cdots\cdots\mathbf{1}\cdots\cdots b_{-1}$$
$$0.412\times2=0.824\cdots\cdots\mathbf{0}\cdots\cdots b_{-2}$$
$$0.824\times2=1.648\cdots\cdots\mathbf{1}\cdots\cdots b_{-3}$$
$$0.648\times2=1.296\cdots\cdots\mathbf{1}\cdots\cdots b_{-4}$$
$$0.296\times2=0.592\cdots\cdots\mathbf{0}\cdots\cdots b_{-5}$$
$$0.592\times2=1.184\cdots\cdots\mathbf{1}\cdots\cdots b_{-6}$$
$$0.184\times2=0.368\cdots\cdots\mathbf{0}\cdots\cdots b_{-7}$$
$$0.368\times2=0.736\cdots\cdots\mathbf{0}\cdots\cdots b_{-8}$$
$$0.736\times2=1.472\cdots\cdots\mathbf{1}\cdots\cdots b_{-9}$$
$$0.472\times2=0.944\cdots\cdots\mathbf{0}\cdots\cdots b_{-10}$$

由于最后的小数大于 0.5，根据“四舍五入”的原则，b_{-10}应为 1。所以，$(0.706)_D=(\mathbf{0.1011010011})_B$，其误差$\varepsilon<2^{-10}$。

1.2.4 十六进制和八进制

由于二进制数位数多，不便书写和记忆，因此在计算机资料中也常采用十六进制数或八进制数来表示。

1. 十六进制

十六进制数采用 16 个数码，分别为 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F。其中 A、B、C、D、E、F 依次相当于十进制数中的 10、11、12、13、14、15。十六进制数是“逢十六进一”，见表 1.2.1，它是以 16 为基数的计数体制。

根据式(1.2.2)，任意十六进制数可表达如下：

$$(N)_H{}^{①}=\sum_{i=-\infty}^{\infty}K_i\times16^i \tag{1.2.9}$$

式中K_i为基数 16 的第 i 次幂的系数。

例 1.2.5 将十六进制数$(4E6)_H$转换为十进制数。

解：$(4E6)_H=4\times16^2+14\times16^1+6\times16^0=1\ 254$

2. 十六-二进制之间的转换

4 位二进制数有 16 个状态，而 1 位十六进制数有 16 个不同的数码，因此二进制转换成为十六进制非常简单，以小数点为基准，整数部分从右到左每 4 位一组，不足 4 位的在高位补 0；小数部分从左到右每 4 位一组，不足 4 位的在低位补 0。将每组 4 位二进制数用 1 位十六进制数表示即可。

例 1.2.6 将二进制数$(\mathbf{10111001011.01001})_B$转换为十六进制数。

解：将 4 位二进制数分为一组，用相应的十六进制数代替即得

$$(\mathbf{0101\quad 1100\quad 1011.\quad 0100\quad 1000})_B=(5CB.48)_H$$

---

① 下标 H(hexadecimal)表示十六进制。$(59)_H$也可写成 59H。

例 1.2.7 将十六进制数$(F15.6)_H$转换为二进制数。

解：将每位十六进制数用 4 位二进制数代替即得相应的二进制数

$$(F15.6)_H=(1111\quad 0001\quad 0101.\quad 011)_B$$

3. 八进制

同样，八进制数由 0~7 共 8 个数码表示，且“逢八进一”。任意八进制数可表示为

$$(N)_O{}^{①}=\sum_{i=-\infty}^{\infty}K_i\times 8^i \tag{1.2.10}$$

式中K_i为基数 8 的第 i 次幂的系数。

4. 八-二进制之间的转换

同理，对于八进制数，可将 3 位二进制数分为一组，然后每组用 1 位八进制数表示。如将例 1.2.6 中的二进制数变换为八进制数，则可写成

$$(010\quad 111\quad 001\quad 011.\quad 010\quad 010)_B=(2\ 713.22)_O$$

至于十进制数变换为十六进制数，可先将十进制数变换为二进制数，再由二进制数转换为十六进制数。

为便于对照，将十进制数 0~20 与二进制、八进制及十六进制数之间的关系列于表 1.2.1 中。

表 1.2.1 几种数制之间的关系对照表

十进制数	二进制数	八进制数	十六进制数
0	0 0 0 0 0	0	0
1	0 0 0 0 1	1	1
2	0 0 0 1 0	2	2
3	0 0 0 1 1	3	3
4	0 0 1 0 0	4	4
5	0 0 1 0 1	5	5
6	0 0 1 1 0	6	6
7	0 0 1 1 1	7	7
8	0 1 0 0 0	10	8
9	0 1 0 0 1	11	9
10	0 1 0 1 0	12	A
11	0 1 0 1 1	13	B
12	0 1 1 0 0	14	C
13	0 1 1 0 1	15	D

① 下标 O(octal)表示八进制。某些文献中也用下标 2、8、10 及 16 分别表示二进制、八进制、十进制及十六进制。

续表

十进制数	二进制数	八进制数	十六进制数
14	01110	16	E
15	01111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
19	10011	23	13
20	10100	24	14

复习思考题

1.2.1 为什么在计算机或数字系统中通常采用二进制数?

1.2.2 在二进制数中,其位权的规律是什么?

1.2.3 十六进制数主要用于何种场合?

1.2.4 二进制与十六进制之间如何进行转换?

1.3 二进制数的算术运算

在数字电路中,0 和 1 既可以表示逻辑状态,又可以表示数量大小。当表示数量时,两个二进制数可以进行算术运算。下面介绍无符号二进制数和有符号二进制数的算术运算。

1.3.1 无符号二进制数的算术运算

无符号二进制数是指所有位都用于表示数的大小。无符号二进制数的加、减、乘、除四种运算的运算规则与十进制数类似,两者唯一的区别在于进位或借位规则不同。无符号二进制数的运算规则如下。

加法规则:

$$0+0=0,0+1=1,1+1=\boxed{1}0$$

方框中的 1 是进位位,表示两个 1 相加“逢二进一”。

减法规则:

$$0-0=0,1-1=0,1-0=1,0-1=\boxed{1}1$$

方框中的 1 是借位位,表示 0 减 1 时不够减,向高位借 1。

乘法规则:

$$0\times0=0,0\times1=0,1\times0=0,1\times1=1$$

除法规则:

$$0\div1=0,1\div1=1$$

注意,除数不能为 0,否则无意义

例 1.3.1 计算两个二进制数 1010 和 0011 的和、差、积和商。

解：

$$\begin{array}{r} 1\ 0\ 1\ 0 \\ +\ 0\ 0\ 1\ 1 \\ \hline 1\ 1\ 0\ 1 \end{array} \qquad \begin{array}{r} 1\ 0\ 1\ 0 \\ -\ 0\ 0\ 1\ 1 \\ \hline 0\ 1\ 1\ 1 \end{array}$$

$$\begin{array}{r} 1\ 0\ 1\ 0 \\ \times\ 0\ 0\ 1\ 1 \\ \hline 1\ 0\ 1\ 0 \\ 1\ 0\ 1\ 0 \\ 0\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0 \\ \hline 0\ 0\ 1\ 1\ 1\ 1\ 0 \end{array} \qquad \begin{array}{r} \quad\ \ 1\ 1 \\ 0\ 0\ 1\ 1 \overline{)\ 1\ 0\ 1\ 0} \\ \quad\ \ 0\ 1\ 1 \\ \hline \quad\ \ 1\ 0\ 0 \\ \quad\ \ 0\ 1\ 1 \\ \hline \quad\ \ 1 \end{array}$$

……余数

所以 1010+0011=1101，1010−0011=0111，1010×0011=11110，1010÷0011=11…余 1。

二进制数的加法运算是基础，数字系统中的各种算术运算最终都将通过它来实现。

由于无符号二进制数中无法表示负数，则要求被减数一定大于减数。

由上述运算过程可见，乘法运算是由左移被乘数与加法运算组成的。除法运算是由右移除数与减法运算组成的。

1.3.2 带符号二进制数的减法运算

前面只考虑了无符号二进制数，当涉及正、负数时需要用有符号的二进制数表示。在日常生活中，通常在一个数的前面用“+”表示正数，用“−”表示负数。但数字系统只能识别 0 和 1，因此，在二进制数前面增加一个符号位。带符号的二进制数通常有 3 种表示方法：原码、反码和补码。在这 3 种方法中规定，用 0 表示“+”，用 1 表示“−”。

1. 二进制数的原码、反码和补码

（1）原码

原码的表示方法：正数的符号位为 0，负数的符号位为 1，数值用其绝对值的二进制数表示。例如

$$+5=\boxed{0}101$$

$$-5=\boxed{1}101$$

原码简单直观，但做减法时需要判断两个数绝对值的大小，用绝对值大的数减去小的数，并且差值的符号由绝对值大的数的符号确定，这一过程较复杂。在数字系统中，是用更简单的补码加法运算实现减法运算的。为了便于得到补码，引入反码的概念。

（2）反码

反码又称为“1 的补码”（one's complement），正数的反码与原码相同。一个数值位为 n 位的二进制负数的原码、反码分别为 N 和 $(N)_{反}$（均不包括符号位），则

$$(N)_{反}=(2^n-1)-N \tag{1.3.1}$$

以-5为例，如果 $n=3$，$N=101$，则 $(N)_{反}=(2^3-1)-101=111-101=010$，即 $\boxed{1}101$ 的反码为 $\boxed{1}010$。

求二进制负数的反码的简单方法是，符号位不变，将原码的数值位逐位求反得到。

(3) 补码

教学视频 1.1：二进制数的补码

补码又称为“2的补码(two's complement)”，正数的补码与原码相同。一个数值位为 n 位的二进制负数的原码、补码分别为 N 和 $(N)_{补}$(均不包括符号位)，则

$$(N)_{补}=2^n-N \tag{1.3.2}$$

以-5为例，如果 $n=3$，$N=101$，则 $(N)_{补}=2^3-101=1000-101=011$。即 $\boxed{1}101$ 的补码为 $\boxed{1}011$。

根据式(1.3.1)和式(1.3.2)可以得出

$$(N)_{补}=(N)_{反}+1 \tag{1.3.3}$$

求二进制负数的补码的简单方法是，在其反码的最低位加1得到。

综上所述，当二进制数为正数时，其补码、反码与原码相同。

当二进制数为负数时，将原码的数值位逐位求反(即得到反码)，然后在最低位加1得到补码。

例 1.3.2 分别计算出 $A=+6$ 和 $B=-6$ 的4位二进制的原码、反码和补码。

解：A 和 B 的绝对值均为6。除最高位为符号位外，还有3位为数值位。其原码、反码和补码分别为

$$A_{原}=\boxed{0}110 \qquad B_{原}=\boxed{1}110$$

$$A_{反}=\boxed{0}110 \qquad B_{反}=\boxed{1}001$$

$$A_{补}=\boxed{0}110 \qquad B_{补}=\boxed{1}010$$

4位带符号的二进制数的原码、反码和补码对照表如表1.3.1所示。对于-0和+0，原码和反码用两组不同的4位二进制数表示，原码分别用1000和0000表示，反码用1111和0000表示。而补码只用一组二进制数0000表示-0和+0，可以用多出一组二进制数表示-8。因此，它们表示的数值范围分别为，原码是-7~+7，反码也是-7~+7，补码是-8~+7。由此可以推知，对于 n 位带符号的二进制数的原码、反码和补码的数值范围分别为：

原码 $-(2^{n-1}-1)\sim+(2^{n-1}-1)$

反码 $-(2^{n-1}-1)\sim+(2^{n-1}-1)$

补码 $-2^{n-1}\sim+(2^{n-1}-1)$

表 1.3.1 4位二进制数原码、反码、补码对照表

十进制数	二进制数		
	原码	反码	补码
-8	—	—	1000
-7	1111	1000	1001

续表

十进制数	二进制数		
	原码	反码	补码
-6	1 1 1 0	1 0 0 1	1 0 1 0
-5	1 1 0 1	1 0 1 0	1 0 1 1
-4	1 1 0 0	1 0 1 1	1 1 0 0
-3	1 0 1 1	1 1 0 0	1 1 0 1
-2	1 0 1 0	1 1 0 1	1 1 1 0
-1	1 0 0 1	1 1 1 0	1 1 1 1
-0	1 0 0 0	1 1 1 1	0 0 0 0
+0	0 0 0 0	0 0 0 0	0 0 0 0
+1	0 0 0 1	0 0 0 1	0 0 0 1
+2	0 0 1 0	0 0 1 0	0 0 1 0
+3	0 0 1 1	0 0 1 1	0 0 1 1
+4	0 1 0 0	0 1 0 0	0 1 0 0
+5	0 1 0 1	0 1 0 1	0 1 0 1
+6	0 1 1 0	0 1 1 0	0 1 1 0
+7	0 1 1 1	0 1 1 1	0 1 1 1

2. 二进制数减法的补码运算

我们先从生活中常见的指针式钟表解释补码运算的原理，如图 1.3.1 所示。如果在 3 点钟时发现时针停在 11 点，需要将其调整到 3 点。有两种调整方法：一是将时针按照逆时针方向减少 8 格，调整到 3 点，即 11-8=3。二是将时针按照顺时针方向增加 4 格，调整到 3 点，即 11+4=15。由于钟表盘的最大数为 12，将超过 12 的“进位”丢掉后，剩下的余数即为需要的结果，即 15-12=3。

知识拓展文档 1.1：补数与补码

由此说明 11-8 可以用 11+4 完成（舍去进位）。由于表盘共有 12 个数，称 12 为进位的模，而 4 为-8 对模 12 的补码，同样 8 为-4 对模 12 的补码。同理，5 为-7 对模 12 的补码，7 为-5 对模 12 的补码……

这里引入了“模”的概念，用 M 表示（也写作 mod M）。所谓“模”是指一个计数系统的进位基数。例如，时钟从 1 到 12 有 12 个数，它的模是 12。4 位二进制数计数范围为 0000~1111 共 16 个数，所以模为 16。

从上述例中可以得出一个结论，减去某个数可以用加上它的负数的补码来完成，并舍去进位。这个结论同样适用于二进制数的运算。

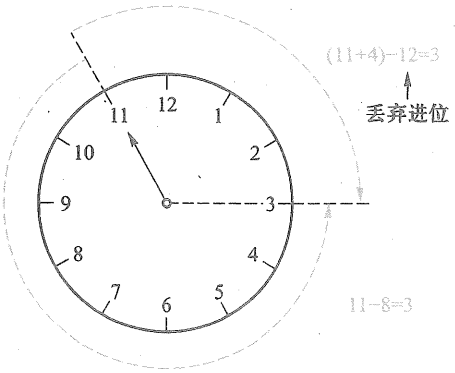


图 1.3.1 补码运算的图解

在计算机等数字系统中，均采用补码进行带符号二进制数的减法运算。

设 A 和 B 均为 n 位正数，则补码的运算规则如下（mod 2^n）：

$$(A+B)_{补}=(A)_{补}+(B)_{补}$$
$$(A-B)_{补}=(A)_{补}+(-B)_{补}$$
$$(-A+B)_{补}=(-A)_{补}+(B)_{补}$$
$$(-A-B)_{补}=(-A)_{补}+(-B)_{补}$$

进行二进制补码运算时，必须注意：

（1）参与运算的是补码，运算结果仍是补码。

（2）被加数与加数的补码均采用相同的位数，如果运算结果超过了模，将产生进位，则应丢弃进位才能得到正确结果。

（3）符号位与数值位一起参与运算，结果的符号位由运算结果确定。

教学视频 1.2：补码的加减运算

例 1.3.3 试用 4 位二进制补码计算 5-2。

解： 因为 $(5-2)_{补}=(5)_{补}+(-2)_{补}$

$=0101+1110$

$=0011$

```
  0 1 0 1
+ 1 1 1 0
[1] 0 0 1 1
```

自动丢弃

两个二进制补码相加时，方框中的 1 是进位位，在计算中会自动丢失，因为运算是以 4 位二进制补码表示的，计算结果仍然保留 4 位数。0011 是正数的补码，就是其原码，所以 5-2=3。

3. 溢出

例 1.3.4 试用 4 位二进制补码计算 5+7。

解： 因为 $(5+7)_{补}=(5)_{补}+(7)_{补}$

$=0101+0111$

$=1100$

```
  0 1 0 1
+ 0 1 1 1
  1 1 0 0
```

计算结果 1100 表示-4，而实际正确的结果应该为 12。错误产生的原因在于 4 位二进制补码中，有 3 位是数值位，它所表示的范围为-8~+7，而本题的结果需要 4 位数值位表示，因而产生溢出。解决溢出的办法是进行位扩展，即用 5 位以上的二进制补码表示，就不会产生溢出了。

4. 溢出的判别

两个符号相反的数相加不会产生溢出，但两个符号相同的数相加有可能产生溢出。如何判断是否产生溢出？根据下列同符号 4 位二进制补码的计算结果可以推知。

```
 +4      0 1 0 0        -5      1 0 1 1
+)+3   + 0 0 1 1       +)-3   + 1 1 0 1
 +7    [0] 0 1 1 1      -8    [1] 1 0 0 0
```

(a)　　(b)

```
 +2      0 0 1 0        -3      1 1 0 1
+)+6   + 0 1 1 0       +)-6   + 1 0 1 0
 +8    [0] 1 0 0 0      -9    [1] 0 1 1 1
```

(c)　　(d)

4 位二进制补码表示的数值范围为-8~+7，所以(a)和(b)没有产生溢出，结果是正确的。(c)和(d)的运算结果应分别是+8 和-9，均超过了允许的范围，产生溢出。比较 4 种情况可以得出溢出的判别方法，如果两个加数的符号相同，而和的符号与它们的符号不同，则运算结果是错误的，产生溢出。

复习思考题

1.3.1 为什么说二进制数的加法运算是算术运算的基础？

1.3.2 二进制数乘法、除法运算过程各有什么规律？

1.3.3 说明反码与补码之间的关系。

1.3.4 简要说明由加补码完成减法运算的原理。

1.3.5 说明溢出产生的原因及解决的办法。

1.4 二进制代码

在数字系统中，数码不仅表示数的大小，还可以表示文字符号(包括控制符)。表示文字符号信息的二进制数码称为代码。以一定的规则编制代码，用以表示十进制数值、字母、符号等的过程称为编码。将代码还原成所表示的十进制数、字母、符号等的过程称为解码或译码。

若所需编码的信息有 N 项，则需要的二进制数码的位数 n 应满足如下关系

$$2^n \geqslant N$$

1.4.1 二-十进制码

二-十进制码就是用 4 位二进制数来表示 1 位十进制数中的 0~9 十个数码，即二进制编码的十进制码(binary-coded-decimal，BCD 码)。4 位二进制数有 16 种不同的组合方式，即 16 种代码，根据不同的规则从中选择 10 种来表示十进制的 10 个数码。其方案有很多种。表 1.4.1 列出了几种常用的 BCD 码。

表 1.4.1 几种常见的 BCD 码

十进制数	有权码			无权码	
	8421 码	2421 码	5421 码	余 3 码	余 3 循环码
0	0000	0000	0000	0011	0010
1	0001	0001	0001	0100	0110
2	0010	0010	0010	0101	0111
3	0011	0011	0011	0110	0101
4	0100	0100	0100	0111	0100
5	0101	1011	1000	1000	1100
6	0110	1100	1001	1001	1101
7	0111	1101	1010	1010	1111
8	1000	1110	1011	1011	1110
9	1001	1111	1100	1100	1010

8421BCD 码是最常用的一种。它由 4 位自然二进制数 **0000** 到 **1111** 的 16 种组合中前 10 种构成，其余 6 种组合是无效的。其编码中每位的值都是固定数，称为位权，从左到右的位权分别为 8、4、2、1，因此称为 8421BCD 码。它属于有权码。

2421 码也是有权码。从左到右的位权分别为 2、4、2 和 1。它的特点是，将任意一个十进制数 N 的代码各位取反，所得代码正好表示 N 对 9 的补码。例如 2 的代码 **0010** 各位取反所得代码 **1101** 正好是 9-2=7 的代码。这种特性称为自补性。具有自补特性的代码称为自补码。

5421 码也是有权码，它各位的权依次为 5、4、2、1。在一般情况下，有权码的十进制数与二进制数之间可用下式来表示

$$(N)_D = W_3 b_3 + W_2 b_2 + W_1 b_1 + W_0 b_0 \tag{1.4.1}$$

式中 $W_3 \sim W_0$ 为二进制码中各位的权。

余 3 码是自补码，与 2421 码有类似的自补性。当两个十进制数之和是 10 时，相应的二进制数之和是 16。例如 1 和 9、2 和 8、…、6 和 4。该特点便于求 10 的补码。余 3 码是无权码，它的每一位没有固定的权值，不能用式(1.4.1)来表示其编码关系，但其编码可以由 8421 码加 3(**0011**)得出。

余 3 循环码也是一种无权码，它的特点是具有相邻性，任意两个相邻代码之间仅有 1 位取值不同，例如 4 和 5 两个代码 **0100** 和 **1100** 仅 b_3 不同。余 3 循环码可以看成是将格雷码首尾各 3 种状态去掉后得到的。下面介绍格雷码。

1.4.2 格雷码

格雷码(Gray code)也是一种常见的无权码，与二进制码一样，可以是任意多位。表 1.4.2 给出了 4 位格雷码的编码顺序。它也具有相邻性，即两个相邻代码之间仅有 1 位取值不同，并且 0 和最大数(2^n-1)之间也只有 1 位不同，因此它是一种循环码。格雷码的相邻性特点使它在传输时引起的误差较小，属于可靠性编码。

格雷码的缺点是不能直接进行算术运算。这是因为格雷码是无权码，其每一位的权值不是固定的。

1. 格雷码的构成方法

(1) 1 位格雷码有 2 个代码，**0** 和 **1**。

(2) (n+1)位格雷码有 2^{n+1} 组代码，其中前 2^n 组代码是将 n 位格雷码按顺序排列，最高位补 0。

(3) (n+1)位格雷码的后 2^n 组代码是将 n 位格雷码按倒序排列，最高位补 **1**。

例如，2 位格雷码 G_1G_0，前 2 组代码的 G_0 从小到大依次为 **0**、**1**，高位 G_1 补 **0**。后 2 组代码的 G_0 与前 2 组顺序相反，从小到大依次为 **1**、**0**，G_1 补 **1**，如图 1.4.1(a)所示。箭头表示 G_0 前、后 2 组码的顺序相反。

表 1.4.2 格 雷 码

十进制数	二进制码				格雷码			
	b_3	b_2	b_1	b_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

3 位格雷码 $G_2G_1G_0$，前 4 组代码的 G_1G_0从小到大依次为 00、01、11、10，最高位 G_2补 0。后 4 组的 G_1G_0顺序与前 4 组相反，依次为 10、11、01、00，G_2补 1，如图 1.4.1(b)所示。箭头表示 G_1G_0前、后 2 组码的顺序相反。以此类推，更多位格雷码以反射轴划分，最高位是相反的，其余位则是对称的。

教学视频 1.3：格雷码的构成

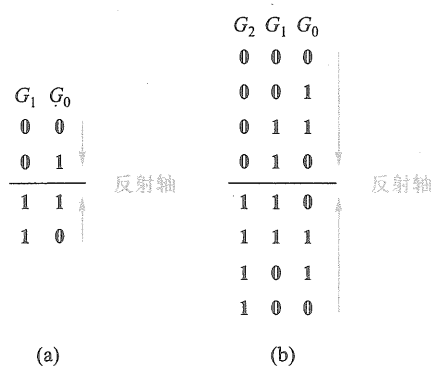


图 1.4.1 格雷码的构成方法

(a) 2 位格雷码 (b) 3 位格雷码

2. 格雷码与二进制码的转换

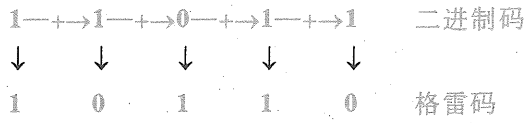
格雷码和二进制码之间经常需要转换，二进制码到格雷码的转换方法如下：

(1) 格雷码的最高位(最左边)与二进制码的最高位相同。

(2) 从左到右，逐一将二进制码相邻的 2 位相加(舍去进位)，作为格雷码的下一位。

例 1.4.1 将二进制码 **11011** 转换成格雷码。

解：转换过程如下：



所以，格雷码为 **10110**。

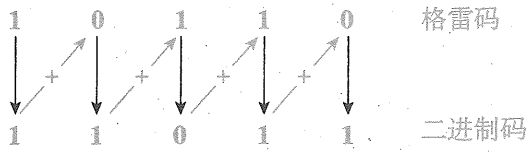
格雷码到二进制码的转换方法如下：

(1) 二进制码的最高位(最左边)与格雷码的最高位相同。

(2) 将产生的每一位二进制码，与下一位(右侧)相邻的格雷码相加(舍去进位)，作为二进制码的下一位。

例 1.4.2 将格雷码 **10110** 转换成二进制码。

解：转换过程如下：



所以，二进制码为 **11011**。

格雷码和二进制码相互转换的另一种方法是用异或表达式(参见 4.2.1 节)。

3. 格雷码的应用

格雷码是一种错误最小化的编码，在模数转换、汽车制动、数控机床、伺服电机、机器人等领域有广泛的应用。

在需要将模拟量转换成用相邻二进制数序列表示数字量的系统中。当模拟量发生微小变化而引起数字量的变化时，例如在传输的递增计数值从 7 变到 8 时，若采用 8421BCD 码则数码从 **0111** 变到 **1000**，如果每一位数码变化不能保持同步，则可能出现 **1111**、**1110**、**1100**……瞬间错误数码。若采用格雷码，其变化是从 **0100** 到 **1100**，只有 b_3 位从 **0** 变成 **1**，其余 3 位保持不变。这样可以避免瞬间错误数码的出现。

光电码盘用于需要指示角度位置的系统。由一个随物体旋转的码盘，和一组与码道对应的光电发射器和接收器组成，当码盘旋转到不同位置时，接收器输出一组表示码盘角位移的数字量。为简单起见，假设码盘上有 8 个扇区，对应 8 组编码，每组 3 位数码，亮的区域有光反射为 **0**，暗的区域没有光反射为 **1**。

如果使用二进制编码，如图 1.4.2(a)所示，当码盘转到过渡边界时，例如编码 **001** 到 **010**，将有 2 位数发生变化。由于机械装置的偏差，使得最外圈的接收器提前输出 **0**，结果出现 **000** 错误编码。最坏情况是 3 位数都发生变化，如 **111** 到 **000**，及 **011** 到 **100** 的边界。

如果采用格雷码，则可以避免上述问题出现，如图 1.4.2(b)所示。每个区域边界只有 1 位变化，接收器输出的编码只有两种可能，要么表示边界这一边 001，要么表示另一边 011。因此，不会出现错误编码。

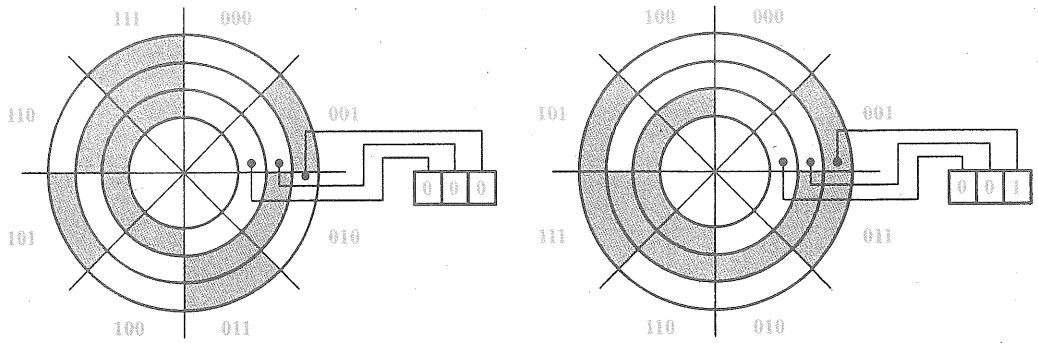


图 1.4.2 光电编码器

(a) 二进制编码码盘 (b) 格雷码编码码盘

1.4.3 ASCII 码

计算机不仅用于处理数字，而且用于处理字母、符号等文字信息。人们通过键盘上的字母、符号和数值向计算机发送数据和指令，每一个键符可用一个二进制码来表示，美国标准信息交换码(American Standard Code for Information Interchange，ASCII)是目前国际上最通用的一种字符码。它是用 7 位二进制码来表示 128 个十进制数、英文大小写字母、控制符、运算符以及特殊符号，如表 1.4.3A 所示，其中一些字符的含义如表 1.4.3B 所示。

表 1.4.3A ASCII 码

$b_3b_2b_1b_0$	$b_6b_5b_4$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{

续表

$b_3b_2b_1b_0$	$b_6b_5b_4$							
	000	001	010	011	100	101	110	111
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

表 1.4.3B ASCII 码中各字符的含义

字符	含义	字符	含义
NUL	Null 空白	DC1	Device control 1 设备控制 1
SOH	Start of heading 标题开始	DC2	Device control 2 设备控制 2
STX	Start of text 文本开始	DC3	Device control 3 设备控制 3
ETX	End of text 文本结束	DC4	Device control 4 设备控制 4
EOT	End of transmission 传输结束	NAK	Negative acknowledge 否认
ENQ	Enquiry 询问	SYN	Synchronous idle 同步空转
ACK	Acknowledge 确认	ETB	End of transmission block 块传输结束
BEL	Bell 报警	CAN	Cancel 取消
BS	Backspace 退一格	EM	End of medium 纸尽
HT	Horizontal tab 水平列表	SUB	Substitute 替换
LF	Line feed 换行	ESC	Escape 脱离
VT	Vertical tab 垂直列表	FS	File separator 文件分隔符
FF	Form feed 走纸	GS	Group separator 组分隔符
CR	Carriage return 回车	RS	Record separator 记录分隔符
SO	Shift out 移出	US	Unit separator 单元分隔符
SI	Shift in 移入	SP	Space 空格
DLE	Data link escape 数据链路换码	DEL	Delete 删除

复习思考题

1.4.1 什么叫 BCD 码？试列举几种常用的 BCD 码。

1.4.2 为什么 8421BCD 码用的较普遍？

1.4.3 格雷码有什么特点，用于什么场合？

1.4.4 格雷码与二进制码之间如何进行转换？

1.4.5 什么是 ASCII 码？试说明它的用途。

1.5 二值逻辑变量与基本逻辑运算

当 0 和 1 表示逻辑状态时，两个二进制数码按照某种指定的因果关系进行的运算称为逻辑运算。逻辑运算与算术运算完全不同，它所使用的数学工具是逻辑代数（又称为布尔

代数)。与普通代数一样,它由逻辑变量和逻辑运算组成。变量可以用 A、B、C、x、y、z 等字母组成。所不同的是,在普通代数中,变量的取值可以是任意的,而在逻辑代数中,逻辑变量只有两个可取的值,即 0 和 1,因而称为二值逻辑变量。这里,0 和 1 并不表示数量的大小,而是用来表示完全对立的逻辑状态。

在逻辑代数中,有与、或、非三种基本的逻辑运算。众所周知,运算是一种函数关系,它可以用语言描述,亦可用逻辑代数表达式描述,还可用表格或图形来描述。输入逻辑变量所有取值的组合与其对应的输出逻辑函数值构成的表格,称为真值表。用规定的逻辑符号表示的图形称为逻辑图。下面分别讨论三种基本的逻辑运算。

1. 与运算

图 1.5.1(a)所示为一个简单的与逻辑电路,电源 E 通过开关 A 和 B 向灯供电,只有 A 与 B 同时闭合时,灯才亮。A 和 B 中只要有一个断开或者两个均断开时,则灯不亮。在这个电路中,开关 A、B 与灯 L 的逻辑关系是:"只有当一件事的几个条件全部具备之后,这件事才发生"。这种关系称为与逻辑。如果用二值逻辑 0 和 1 来表示开关和灯的状态,设开关断开和灯不亮均用 0 表示,而开关闭合和灯亮均用 1 表示,则可得出其真值表,如表 1.5.1 所示。其中 L 表示灯的状态。则与运算逻辑表达式为

$$L=A\cdot B \tag{1.5.1}$$

式中小圆点"·"表示 A、B 的与运算,也称为逻辑乘。在不致引起混淆的前提下,乘号"·"被省略。在某些文献中,也采用符号"∧"、"∩"表示与运算。实现与运算的逻辑电路称为与门,其逻辑符号[①]如图 1.5.1(b)和图 1.5.1(c)所示。其中图(b)为特异形符号,图 1.5.1(c)为矩形符号。

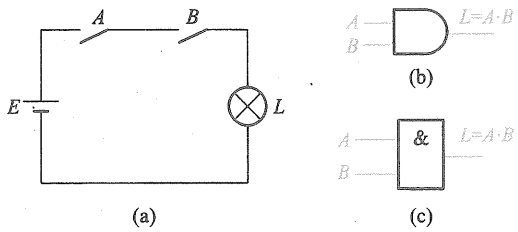


图 1.5.1 与逻辑运算

(a) 电路图 (b) 特异形符号 (c) 矩形符号

表 1.5.1 与逻辑真值表

A	B	$L=A\cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

① 电气和电子工程师协会(institute of electrical and electronics engineers,简称 IEEE)标准中,门电路的图形符号有两种,矩形符号和特异形符号(见附录 B)。中国国家标准(GB/T 4728.12—2008)采用矩形符号。本书所用的逻辑图形符号为特异形符号。

2. 或运算

图 1.5.2(a)所示为一个简单的或逻辑电路,电源 E 通过开关 A 或 B 向灯供电。只要开关 A 或 B 闭合或两者均闭合,则灯亮。而当 A 和 B 均断开时,则灯不亮。此电路中,开关 A、B 与灯 L 的逻辑关系是:"当一件事情的几个条件中只要有一个条件得到满足,这件事就会发生"。这种关系称为或逻辑。或是指 A 闭合或 B 闭合,即任一个条件具备的意思。仿照前述,可以得出用 0、1 表示的或逻辑真值表如表 1.5.2 所示。或运算逻辑表达式为

$$L=A+B \tag{1.5.2}$$

式中符号"+"表示 A、B 的或运算。在某些文献中,也采用符号 V、U 表示。实现或运算的逻辑电路称为或门,其逻辑符号如图 1.5.2(b)和图 1.5.2(c)所示。

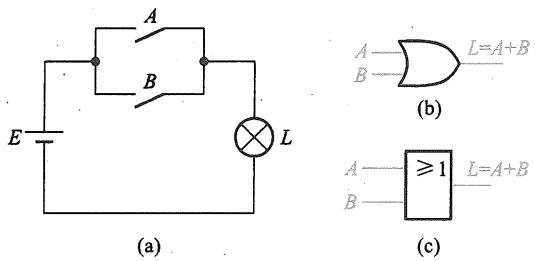


图 1.5.2 或逻辑运算

(a) 电路图 (b) 特异形符号 (c) 矩形符号

表 1.5.2 或逻辑真值表

A	B	$L=A+B$
0	0	0
0	1	1
1	0	1
1	1	1

3. 非运算

如图 1.5.3(a)所示,电压源 E 通过一继电器触点向灯供电,NC 为继电器 A 的动断触点,即通常 A 不通电,动断触点闭合,灯亮;而当 A 通电时,动断触点断开,灯不亮。由此可得出其逻辑关系为:"一件事情的发生是以其相反的条件为依据"。这种逻辑关系称为非逻辑。若用 0 和 1 来表示继电器和灯的状态,A 不通电和灯不亮用 0 表示,而 A 通电和灯亮用 1 表示,则得出非逻辑的真值表如表 1.5.3 所示。可以看出,L 与 A 总是处于相反的逻辑状态。非运算逻辑表达式为

$$L=\overline{A} \tag{1.5.3}$$

式中字母 A 上方的短线"-"表示非运算。在某些文献中,也采用"~""¬"或"′"表示。在逻辑运算中,通常将 A 称为原变量,而将 $\overline{A}$ 称为反变量或非变量。

实现非运算的电路称为非门,或称为反相器,其逻辑符号如图 1.5.3(b)、(c)所示,小圆圈表示非运算。

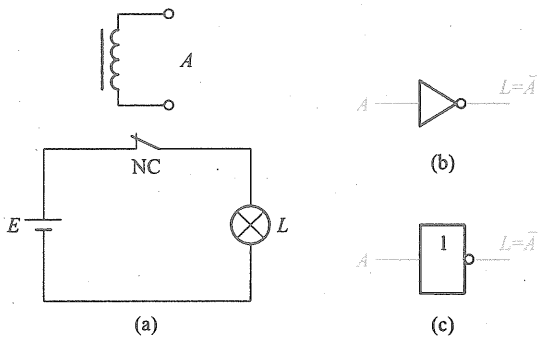


图 1.5.3 非逻辑运算

(a) 电路图 (b) 特异形符号 (c) 矩形符号

表 1.5.3 非逻辑真值表

A	$L=\overline{A}$
0	1
1	0

上述逻辑运算可以推广到多变量的情况。

4. 常用复合逻辑运算

在实际逻辑运算中，除了与、或、非三种基本运算外，还经常使用它们的组合逻辑运算，例如与非、或非、异或和同或。

与非运算是与运算和非运算的组合。逻辑符号和真值表分别如图 1.5.4 和表 1.5.4 所示。逻辑表达式为

$$L=\overline{A\cdot B} \tag{1.5.4}$$

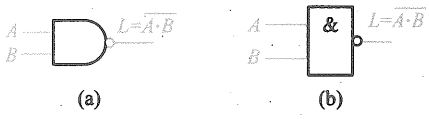


图 1.5.4 与非逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.4 与非逻辑真值表

A	B	L
0	0	1
0	1	1
1	0	1
1	1	0

或非运算是或运算和非运算的组合。逻辑符号和真值表分别如图 1.5.5 和表 1.5.5 所示。逻辑表达式为

$$L=\overline{A+B} \tag{1.5.5}$$

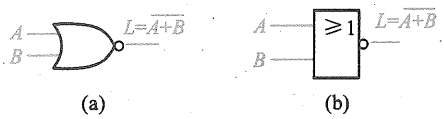


图 1.5.5 或非逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.5 或非逻辑真值表

A	B	L
0	0	1
0	1	0
1	0	0
1	1	0

异或的逻辑关系是:当两个输入状态相同时,输出为 0;当两个输入状态不同时,输出为 1。逻辑符号和真值表分别如图 1.5.6 和表 1.5.6 所示。逻辑表达式为

$$L=\overline{A}B+A\overline{B}=A\oplus B \tag{1.5.6}$$

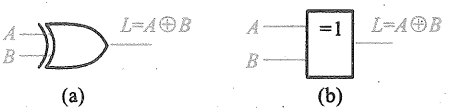


图 1.5.6 异或逻辑符号

(a) 特异形符号 (b) 矩形符号

表 1.5.6 异或逻辑真值表

A	B	L
0	0	0
0	1	1
1	0	1
1	1	0

同或和异或的逻辑关系刚好相反:当两个输入状态相同时,输出为 1;当两个输入状态不同时,输出为 0。逻辑符号和真值表分别如图 1.5.7 和表 1.5.7 所示。逻辑表达式为

$$L=AB+\overline{A}\,\overline{B}=A\odot B \tag{1.5.7}$$

图 1.5.7 同或逻辑符号
(a) 特异形符号 (b) 矩形符号

表 1.5.7 同或逻辑真值表

A	B	L
0	0	1
0	1	0
1	0	0
1	1	1

复习思考题

1.5.1 试列举几种相关的实例说明与、或、非逻辑关系，并列写出三种逻辑运算的表达式。

1.5.2 根据真值表判断异或和同或具有怎样的逻辑关系。

1.6 逻辑函数及其表示方法

从上一节介绍的逻辑运算中可以知道，逻辑变量分为两种：输入逻辑变量和输出逻辑变量。描述输入逻辑变量和输出逻辑变量之间的因果关系称为逻辑函数。由于逻辑变量是只取 0 或 1 的二值逻辑变量，因此逻辑函数也是二值逻辑函数。

1.6.1 逻辑函数的几种表示方法

通常一个比较复杂的逻辑电路，往往受多个输入逻辑变量控制。描述输出变量与输入变量之间逻辑函数的方法有真值表、逻辑函数表达式、逻辑图、波形图和卡诺图等。下面举一个简单实例介绍前四种逻辑函数的表示。

图 1.6.1 是一个控制楼梯照明灯的电路，单刀双掷开关 A 装在楼下，B 装在楼上，这样在楼下开灯后，可在楼上关灯；同样，也可在楼上开灯，而在楼下关灯。因为只有当两个开关都向上扳或向下扳时，灯才亮；而一个向上扳、另一个向下扳时，灯就不亮。

1. 真值表

将输入变量所有可能的取值与相应的函数值列成表格，就得到真值表。

图 1.6.1 电路的逻辑关系可用真值表来描述。设 L 表示灯的状态，即 $L=1$ 表示灯亮，$L=0$ 表示灯灭。A、B 为 1 表示开关向上扳，为 0 表示开关向下扳。A、B 的取值有 4 种组合，00、01、10、11，对每一种组合确定输出 L 的值，则 L 与 A、B 逻辑关系的真值表如表 1.6.1 所示。

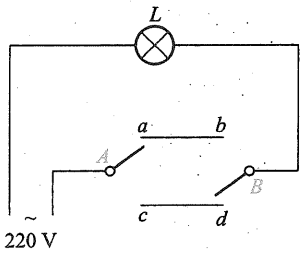


图 1.6.1 逻辑电路举例

表 1.6.1 图 1.6.1 的真值表

A	B	L
0	0	1
0	1	0
1	0	0
1	1	1

2. 逻辑函数表达式

逻辑函数表达式是用与、或、非等运算组合起来，表示逻辑函数与逻辑变量之间关系的逻辑代数式（简称逻辑表达式）。

由表 1.6.1 所示真值表可知，在 A、B 状态的四种组合中，只有第一（$A=B=0$）和第四（$A=B=1$）两种组合使灯亮（$L=1$），即乘积项$\overline{A}\,\overline{B}$和 AB 为 1 时，使 $L=1$。输出 L 为两个乘积项之和，即

$$L=\overline{A}\,\overline{B}+AB \tag{1.6.1}$$

根据真值表写逻辑表达式的方法：输入变量之间是与的关系，而输出状态之间的组合则是或的关系。对于变量 A、B 或输出 L，凡取 1 值的用原变量表示，取 0 值用反变量表示。

3. 逻辑图

用与、或、非等逻辑符号表示逻辑函数中各变量之间的逻辑关系所得到的图形称为逻辑图。

将式（1.6.1）中所有的与、或、非运算符号用相应的逻辑符号代替，并按照逻辑运算的先后次序将这些逻辑符号连接起来，就得到图 1.6.1 所示电路对应的逻辑图，如图 1.6.2（a）所示。式（1.6.1）表示的是同或逻辑关系，为简便起见，也可以用同或逻辑符号表示，得到图 1.6.2（b）所示逻辑图。

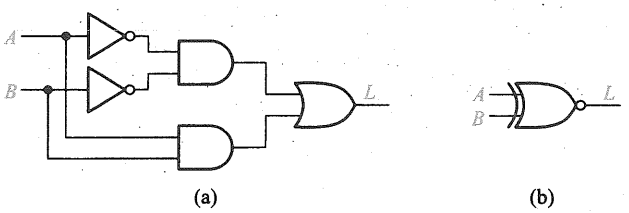


图 1.6.2 图 1.6.1 电路的逻辑图

（a）由与、或、非逻辑符号构成的逻辑图 （b）由同或逻辑符号构成的逻辑图

4. 波形图

对输入变量随时间变化的每一种取值，求出相应的输出值。并将输入和输出关系按时间顺序依次排列得到的图形，称为波形图。

图 1.6.3 所示的波形图中，在 t_1 时间段内，A、B 输入端均为低电平 0，根据式(1.6.1)或表 1.6.1 可知，此时输出 L 为高电平 1。依照此方法，可得出 t_2、t_3和 t_4时间段内输出 L 的波形图。从图中可以直观地看出，对于同或逻辑关系，只要输入 A 和 B 相同，输出为 1；A 和 B 不相同时，输出为 0。

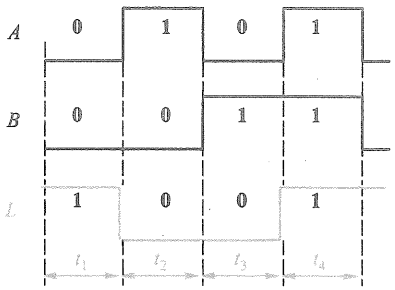


图 1.6.3 图 1.6.1 所示电路的波形图

上述四种不同的表示方法所描述的是同一逻辑关系，因此它们之间有着必然的联系，可以从一种表示方法，得到其他表示方法。

1.6.2 逻辑函数表示方法之间的转换

逻辑函数的表示方法归纳起来有六种：真值表、逻辑函数表达式、逻辑图、波形图、卡诺图和 HDL。卡诺图及 HDL 表示方法将在第 2 章介绍。

同一逻辑问题可以用六种不同方法描述。因此，这些方法本质是相通的，可以相互转换。这里只介绍真值表与逻辑表达式及逻辑图之间的转换。

1. 真值表到逻辑图的转换

通常从给定的真值表不能直接得到逻辑图。首先根据真值表写出逻辑表达式，依照逻辑表达式画出逻辑图。

转换步骤：

(1) 根据真值表写出逻辑表达式。

(2) 用公式法或卡诺图法化简得到简化的逻辑表达式。

(3) 根据逻辑表达式画出逻辑图。

例 1.6.1 已知真值表如表 1.6.2 所示，试画出用与、或、非门实现的逻辑图。

解：(1) 写出真值表中使输出为 1 的所有输入变量乘积项，凡取 1 值的用原变量表示，取 0 值的用反变量表示。然后将这些乘积项相加，即得到输出逻辑函数表达式。

当输入为 $A=0$、$B=1$、$C=1$ 时，使乘积项 $\overline{A}BC=1$，此时 $L=1$。同理乘积项 $AB\overline{C}$ 使 $L=1$。输出逻辑函数表达式就是这两个乘积项之和，即

$$L=\overline{A}BC+AB\overline{C} \tag{1.6.2}$$

(2) 公式法或卡诺图法化简将在第 2 章介绍，式(1.6.2)不需要化简。

(3) 将式(1.6.2)中所有的与、或、非运算符号用相应的逻辑符号代替，并按照逻辑运算的先后次序将它们连接起来，得到如图 1.6.4 所示的逻辑图。

表 1.6.2 例 1.6.1 的真值表

A	B	C	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

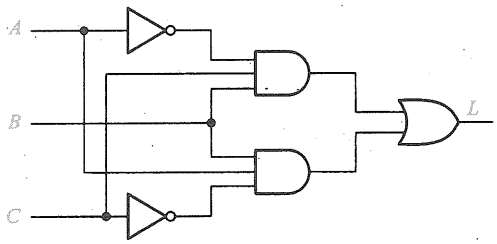


图 1.6.4 例 1.6.1 的逻辑图

2. 逻辑图到真值表的转换

从逻辑图不能直接得到真值表，转换步骤如下：

(1) 从逻辑图的输入端到输出端，逐级写出每个逻辑符号输出端的表达式，直到写出最后的输出变量的逻辑表达式。

(2) 化简变换，求简化的逻辑表达式。

(3) 将输入变量可能的取值逐个代入表达式进行计算，并将结果列表，即得真值表。

例 1.6.2 已知逻辑图如图 1.6.5 所示，列出它的真值表。

解：从输入端开始写出每个逻辑符号输出端的表达式，得到最后的表达式为

$$L=\overline{A}B+A\,\overline{B}$$

该表达式不需要化简变换。

输入变量 A、B 取值有四种组合，其中 $\overline{A}B$ 和 $A\,\overline{B}$ 使 L 为 1，得到真值表如表 1.6.3 所示。

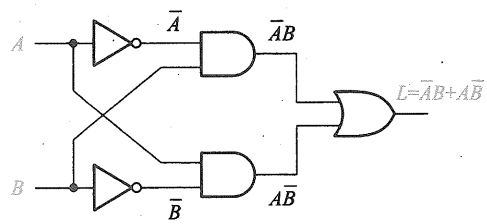


图 1.6.5 例 1.6.2 的逻辑图

表 1.6.3 例 1.6.2 的真值表

A	B	L
0	0	0
0	1	1
1	0	1
1	1	0

复习思考题

1.6.1 逻辑函数都有哪些表示方法?

1.6.2 你能根据给出的一种逻辑函数表示方法转换为其他的描述方式吗?

小 结

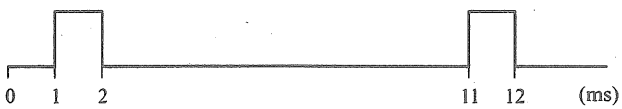
- 由于模拟信号具有连续性,实用上难于存储、分析和传输;数字电路或数字系统传输和处理 0 和 1 表示的信息,因此,便于克服这些困难。
- 用 0 和 1 可以组成二进制数表示数量的大小,也可以表示对立的两种逻辑状态。
- 数字系统中常用二进制数来表示数值。此外常用的还有八进制、十进制和十六进制。各种进制表示的数值之间可以相互转换。
- 与十进制数类似,二进制数也有加、减、乘、除四种运算,加法是各种运算的基础。
- 二进制数可以用原码、反码或补码表示。在数字系统或计算机中常采用二进制补码表示有符号的数,并进行有关运算。
- 数字系统中用来表示十进制数的代码有 8421 码、2421 码、5421 码、余 3 码、余 3 循环码、格雷码等。也有用 7 位二进制数来表示符号-数字混合码,如 ASCII 码。
- 与、或、非是逻辑运算中的三种基本运算,其他的逻辑运算可以由这三种基本运算构成。数字逻辑是计算机的基础。逻辑函数的描述方法有真值表、逻辑函数表达式、逻辑图、波形图和卡诺图等。

习 题

1.1 数字信号与数字电路

1.1.1 试按表 1.1.1 所列的数字集成电路的分类为依据,指出下列 IC 器件属于何种集成度器件:(1) 微处理器;(2) 计数器;(3) 加法器;(4) 逻辑门;(5) 4 兆位存储器。

1.1.2 一周期性数字波形如图题 1.1.2 所示,试计算:



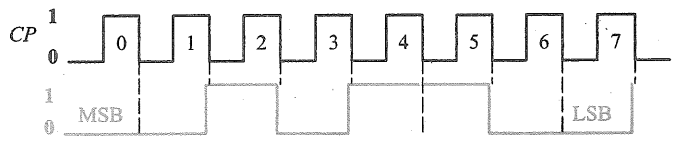
图题 1.1.2

(1) 周期;(2) 频率;(3) 占空比。

1.2 数制

1.2.1 一数字波形如图题 1.2.1 所示,时钟频率为 4 kHz,试确定:

(1) 它所表示的二进制数;(2) 串行方式传送 8 位数据所需要的时间;(3) 以 8 位并行方式传送数据时需要的时间。



图题 1.2.1

1.2.2 将下列二进制数转换为十进制数:

(1) $(1011)_B$ (2) $(0.101)_B$ (3) $(10.01)_B$ (4) $(1110.1001)_B$

1.2.3 将下列二进制数转换为八进制数和十六进制数:

(1) $(101001)_B$ (2) $(1011.0101)_B$ (3) $(11010.11001)_B$ (4) $(110111.011101)_B$

1.2.4 将下列十进制数转换为二进制数、八进制数和十六进制数:

(1) $(43)_D$ (2) $(59)_D$ (3) $(127)_D$ (4) $(234)_D$

1.2.5 将下列十进制数转换为二进制数(要求保留小数点后 8 位):

(1) $(0.34)_D$ (2) $(7.2)_D$ (3) $(45.62)_D$ (4) $(1002.456)_D$

1.2.6 将下列十进制数转换为二进制数和十六进制数(要求转换误差不大于 2^{-4}):

(1) $(0.9)_D$ (2) $(4.8)_D$ (3) $(15.27)_D$ (4) $(254.35)_D$

1.2.7 将下列十六制数转换为二进制数:

(1) $(7)_H$ (2) $(D4)_H$ (3) $(23F.45)_H$ (4) $(A040.51)_H$

1.2.8 将下列十六进制数转换为十进制数:

(1) $(13)_H$ (2) $(103.2)_H$ (3) $(A3.0C)_H$ (4) $(A45D.0BC)_H$

1.2.9 试比较下列各数,并指出从大到小的排列顺序:

(1) $(74)_D$ (2) $(1101101)_B$ (3) $(165)_O$ (4) $(10E)_H$

1.3 二进制数的算术运算

1.3.1 写出下列二进制数的原码、反码和补码:

(1) $(+1110)_B$ (2) $(+10110)_B$ (3) $(-1110)_B$ (4) $(-10110)_B$

1.3.2 写出下列有符号二进制补码所表示的十进制数:

(1) 01011 (2) 0010111 (3) 11101000 (4) 11011001

1.3.3 试用 8 位二进制补码表示下列十进制数:

(1) +11 (2) +68 (3) −24 (4) −90

1.3.4 试用 8 位二进制补码计算下列各式,并用十进制数表示结果:

(1) 12+9 (2) 11−3 (3) −29−25 (4) −120+30

1.4 二进制代码

1.4.1 将下列十进制数转换为 8421BCD 码:

(1) 43 (2) 127 (3) 254.25 (4) 2.718

1.4.2 将下列数码分别作为自然二进制数和 8421BCD 码时，求出相应的十进制数。

(1) 10010111 (2) 100010010011 (3) 000101001001 (4) 10000100.10010001

1.4.3 将下列二进制数转换为格雷码：

(1) 101 (2) 1011 (3) 11010 (4) 101101

1.4.4 将下列格雷码转换为二进制数：

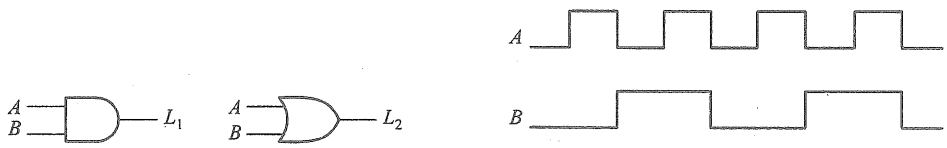
(1) 110 (2) 1111 (3) 10110 (4) 110111

1.4.5 试用十六进制数写出下列字符的 ASCII 码的表示：

(1) + (2) @ (3) you (4) 43

1.5 二值逻辑变量与基本逻辑运算

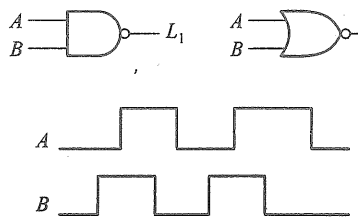
1.5.1 在图题 1.5.1 中，已知输入信号 A、B 的波形，画出各逻辑门输出的波形。



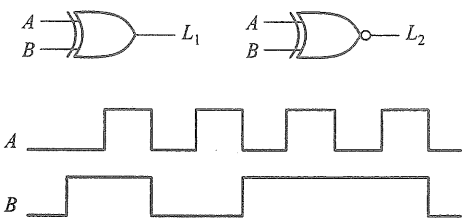
图题 1.5.1

1.5.2 在图题 1.5.2 中，已知输入信号 A、B 的波形，画出各逻辑门输出的波形。

1.5.3 在图题 1.5.3 中，已知输入信号 A、B 的波形，画出各逻辑门输出的波形。



图题 1.5.2



图题 1.5.3

1.6 逻辑函数及其表示方法

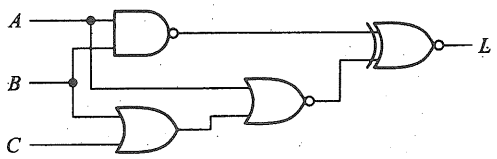
1.6.1 已知逻辑函数的真值表如表题 1.6.1 所示，试写出其逻辑函数表达式。

表题 1.6.1 题 1.6.1 的真值表

A	B	C	L
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

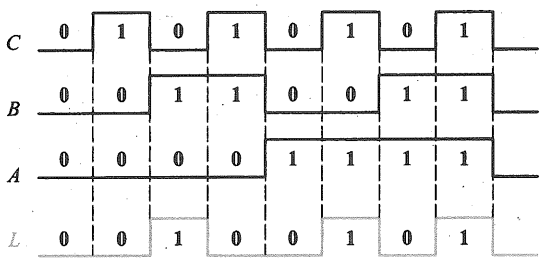
1.6.2 已知逻辑函数 $L=A+\overline{B}C+\overline{A}B\,\overline{C}$，试求其对应的真值表。

1.6.3 已知逻辑图如图题 1.6.3 所示，试求 L 的逻辑函数表达式。



图题 1.6.3

1.6.4 已知逻辑函数 L 的波形图如图题 1.6.4 所示，试求其真值表和逻辑函数表达式。



图题 1.6.4

文档 1.2：
第 1 章部分
习题答案

2 逻辑代数与硬件描述语言基础

引言

本章将首先介绍分析和设计数字电路的数学工具——逻辑代数[①]，从逻辑变量、基本定律和定理、逻辑函数及其化简方法逐步加以讨论。然后介绍在数字电路仿真和设计中使用的一种硬件描述语言——Verilog HDL 的基础知识。

文档 2.1：第 2 章思维导图

2.1 逻辑代数的基本定律和规则

逻辑代数是 1854 年问世的，最早用于开关和继电器网络的分析及化简，随着半导体器件制造工艺的发展，各种性能良好的微电子开关器件不断涌现，逻辑代数成为分析和设计逻辑电路不可缺少的数学工具。利用这种数学工具，可以把逻辑电路输入和输出之间的关系用代数方程表示出来。

逻辑代数有一系列的定律、定理和规则，用它们对数学表达式进行处理，可以完成对逻辑电路的化简、变换、分析和设计。

2.1.1 逻辑代数的基本定律和恒等式

表 2.1.1 列出了常用的逻辑代数基本定律和恒等式。等式中的字母（例如 A、B、C）为逻辑变量，其值可以取 0 或 1，代表逻辑信号的两种可能状态之一。

表 2.1.1 逻辑代数定律、定理和恒等式

名称	基本定律和常用恒等式	
自等律	$A+0=A$	$A\cdot 1=A$
0-1 律	$A+1=1$	$A\cdot 0=0$
重叠律	$A+A=A$	$A\cdot A=A$
互补律	$A+\overline{A}=1$	$A\cdot\overline{A}=0$
还原律	$\overline{\overline{A}}=A$	

① 又称布尔代数，是英国数学家 G.Boole（1815—1864）提出的。

续表

名称	基本定律和常用恒等式	
交换律	$A+B=B+A$	$A\cdot B=B\cdot A$
结合律	$(A+B)+C=A+(B+C)$	$(A\cdot B)\cdot C=A\cdot(B\cdot C)$
分配律	$A\cdot(B+C)=A\cdot B+A\cdot C$	$A+B\cdot C=(A+B)\cdot(A+C)$
吸收律	$A+A\cdot B=A$	$A\cdot(A+B)=A$
反演律(摩根定理①)	$\overline{A\cdot B\cdot C\cdots}=\overline{A}+\overline{B}+\overline{C}+\cdots$	$\overline{A+B+C\cdots}=\overline{A}\cdot\overline{B}\cdot\overline{C}\cdots$
其他常用恒等式	$A+\overline{A}\cdot B=A+B$ $A\cdot B+\overline{A}\cdot C+B\cdot C=A\cdot B+\overline{A}\cdot C$	$A\cdot(\overline{A}+B)=A\cdot B$ $A\cdot B+\overline{A}\cdot C+B\cdot C\cdot D=A\cdot B+\overline{A}\cdot C$

表2.1.1 中除了还原律和最后一组恒等式外,其他基本定律或恒等式是成对出现的,具有对偶性。为了进一步简化表达式的形式,在不产生混淆的前提下,可以省略与运算符“·”。

在以上所有定律中,数学家摩根提出的反演律具有特殊重要的意义。反演律又称为摩根定理,它经常用于求一个原函数的非函数或者对逻辑函数进行变换。

用完全归纳法可以证明表 2.1.1 所列等式的正确性,方法是:分别列出等式左边表达式与右边表达式的真值表,如果等式两边的真值表相同,说明等式成立。

例 2.1.1 用完全归纳法证明两变量反演律(摩根定理)的正确性。

$$\overline{A\cdot B}=\overline{A}+\overline{B},\quad \overline{A+B}=\overline{A}\cdot\overline{B}$$

证:列出 A、B 所有可能的取值情况,并求出相应表达式的结果,得到表 2.1.2 所示的真值表。

表 2.1.2 摩根定理的证明

A B	$\overline{A}$ $\overline{B}$	$\overline{A\cdot B}$	$\overline{A}+\overline{B}$	$\overline{A+B}$	$\overline{A}\cdot\overline{B}$
0 0	1 1	$\overline{0\cdot 0}=1$	1	$\overline{0+0}=1$	1
0 1	1 0	$\overline{0\cdot 1}=1$	1	$\overline{0+1}=0$	0
1 0	0 1	$\overline{1\cdot 0}=1$	1	$\overline{1+0}=0$	0
1 1	0 0	$\overline{1\cdot 1}=0$	0	$\overline{1+1}=0$	0

分别将表中第 3 列和第 4 列进行比较、第 5 列和第 6 列进行比较,可见上面每个等式两边的真值表相同,故等式成立。

表 2.1.1 中的常用恒等式可以用其他基本定律加以证明,下面举例进行说明。

例 2.1.2 试证明 $A+\overline{A}\cdot B=A+B$ 成立。

证:$A+\overline{A}B=(A+\overline{A})\cdot(A+B)$ (运用定律 $A+B\cdot C=(A+B)\cdot(A+C)$)

$=1\cdot(A+B)=A+B$ (运用定律 $A+\overline{A}=1$)

例 2.1.3 试证明 $A\cdot B+\overline{A}\cdot C+B\cdot C=A\cdot B+\overline{A}\cdot C$ 成立。

① “摩根”是 De Mogen 的译称。

证：$A\cdot B+\overline{A}\cdot C+B\cdot C=A\cdot B+\overline{A}\cdot C+(A+\overline{A})\cdot B\cdot C$ （运用定律 $A+\overline{A}=1$）

$=A\cdot B+\overline{A}\cdot C+A\cdot B\cdot C+\overline{A}\cdot B\cdot C$ （运用定律 $A\cdot(B+C)=A\cdot B+A\cdot C$）

$=A\cdot B\cdot(1+C)+\overline{A}\cdot C\cdot(1+B)$ （运用定律 $A\cdot(B+C)=A\cdot B+A\cdot C$）

$=A\cdot B+\overline{A}\cdot C$ （运用定律 $A+1=1$ 和 $A\cdot 1=A$）

这个恒等式说明，若两个乘积项中分别包含因子 A 和 $\overline{A}$，而这两个乘积项的其余因子组成第三个乘积项时，则第三个乘积项是多余的，可以消去。

上面这些基本定理对化简逻辑函数表达式十分有用，也就是说，可以用来减少表达式中乘积项的数量。

例 2.1.4 试化简下列逻辑函数表达式：

$$L=(A+B)\cdot(\overline{A}+B)$$

解：运用分配律将表达式展开，得到

$L=A\cdot\overline{A}+A\cdot B+B\cdot\overline{A}+B\cdot B$

$=0+A\cdot B+B\cdot\overline{A}+B$ （运用定理 $A\cdot\overline{A}=0$ 和 $A\cdot A=A$）

$=A\cdot B+B\cdot\overline{A}+B$ （运用定理 $A+0=A$）

$=B\cdot(A+\overline{A}+1)$ （运用定理 $A\cdot(B+C)=A\cdot B+A\cdot C$）

$=B\cdot 1=B$ （运用定理 $A+1=1$ 和 $A\cdot 1=A$）

注意，本节列出的基本定律和恒等式反映了逻辑关系，而不是数量之间的关系，在运算中不能简单套用初等代数的运算规则。如初等代数中的移项规则就不能用，这是因为逻辑代数中没有减法的缘故。

2.1.2 逻辑代数的基本规则或定理

1. 代入规则

在任何一个逻辑等式中，如果用一个函数代替等式两边出现的某变量 A，则等式依然成立，这个规则称为代入规则。

例如，在 $B\cdot(A+C)=B\cdot A+B\cdot C$ 中，将所有出现 A 的地方都用函数 $E+F$ 代替，则等式仍成立，即得

$$B\cdot[(E+F)+C]=B\cdot(E+F)+B\cdot C=B\cdot E+B\cdot F+B\cdot C$$

对所有基本定律或定理都可以使用代入规则。例如对于二变量的摩根定理 $\overline{A\cdot B}=\overline{A}+\overline{B}$，若用 $C\cdot D$ 代替等式中的 A，则 $\overline{(C\cdot D)\cdot B}=\overline{C\cdot D}+\overline{B}=\overline{C}+\overline{D}+\overline{B}$，以此类推，任意多个变量的摩根定理都成立。

2. 反演规则

根据摩根定理，由原函数 L 的表达式，求它的非函数 $\overline{L}$ 时，可以将 L 中的与（·）换成或（+），或（+）换成与（·）；再将原变量换为非变量（如 A 换成 $\overline{A}$），非变量换为原变量；并将 1 换成 0，0 换成 1；那么所得的逻辑函数式就是 $\overline{L}$。这个规则称为反演规则。

利用反演规则，可以比较容易求出一个原函数的非函数。运用反演规则时必须注意以下两个原则：

（1）保持原来的运算优先级，即先进行与运算，后进行或运算。并注意优先考虑括号内的运算。

(2) 对于非变量以外的非号应保留不变。

例 2.1.5 试求 $L=\overline{A}\cdot\overline{B}+C\cdot D+0$ 的非函数 $\overline{L}$。

解：按照反演规则，得

$$\overline{L}=(A+B)\cdot(\overline{C}+\overline{D})\cdot 1=(A+B)\cdot(\overline{C}+\overline{D})$$

例 2.1.6 试求 $L=A+B\cdot\overline{C}+\overline{D+E}$ 的非函数 $\overline{L}$。

解：按照反演规则，并保留非变量以外的非号不变，得

$$\overline{L}=\overline{A}\cdot\overline{(\overline{B}+C)\cdot\overline{\overline{D}\cdot E}}$$

3. 对偶规则

设 L 是一个逻辑表达式，若把 L 中的“与、或互换，0、1 互换”，那么就得到一个新的逻辑函数式，这就是 L 的对偶式，记作 L'。**与、或互换就是把与(·)换成或(+)，或(+)换成与(·)；0、1 互换就是把 1 换成 0，0 换成 1。**变换时需注意保持原式中“先括号、然后与、最后或”的运算顺序。

例如，$L=(A+\overline{B})\cdot(A+C)$，则 $L'=A\cdot\overline{B}+A\cdot C$。

对偶规则为：当某个逻辑表达式相等，则它们的对偶式也相等。

对偶性意味着逻辑代数中每个逻辑恒等式可以用两种不同的表达式进行表示。即任何一个定理的对偶表达式也是正确的。例如，表 2.1.1 中吸收律 $A+\overline{A}\cdot B=A+B$ 成立，利用对偶规则，它的对偶式 $A\cdot(\overline{A}+B)=A\cdot B$ 也是成立的。

注意，对偶变换时，逻辑变量本身及其位置均保持不变。

4. 香农展开定理

香农展开定理(Shannon's expansion theorem)是指任何一个逻辑函数 $f(x_1,x_2,\cdots,x_{i-1},x_i,x_{i+1},\cdots,x_n)$ 都可以重新表示为

$$f(x_1,x_2,\cdots,x_{i-1},x_i,x_{i+1},\cdots,x_n)=\overline{x_i}\cdot f(x_1,x_2,\cdots,x_{i-1},0,x_{i+1},\cdots,x_n)+x_i\cdot f(x_1,x_2,\cdots,x_{i-1},1,x_{i+1},\cdots,x_n)$$

例 2.1.7 利用香农展开定理将变量 A 从下列 3 变量函数 L 中分解出来。

$$L(A,B,C)=AB+BC+AC$$

解：按照香农展开定理，得到

$$\begin{aligned}L(A,B,C)&=\overline{A}\cdot L(0,B,C)+A\cdot L(1,B,C)\\&=\overline{A}\cdot(0\cdot B+BC+0\cdot B)+A\cdot(1\cdot B+BC+1\cdot C)\\&=\overline{A}\cdot(BC)+A\cdot(B+C)\\&=\overline{A}\cdot L_0+A\cdot L_1\end{aligned}$$

式中，$L_0=BC$，$L_1=B+C$。可见，经过展开，3 变量函数 L 转换为两个 2 变量函数 L_0 和 L_1。这说明使用香农展开定理，可以减少函数自变量的数目，降低函数的复杂度。

顺便指出，可以对一个函数进行多次分解，每展开 1 次，新形成的子函数式中的变量个数比原来函数中变量个数将减少 1 个，此特性在 EDA 综合工具的算法中得到重要应用。

复习思考题

2.1.1 (1) $A+1=?$ (2) $A\cdot 0=?$ (3) $A\cdot A=?$ (4) $A+A=?$

2.1.2 试用完全归纳法证明 $A+B\cdot C=(A+B)\cdot(A+C)$。

2.1.3 写出三变量摩根定理的表达式，并用完全归纳法进行证明。

2.2 逻辑函数表达式的形式

任何一个逻辑函数，其表达式的形式都不是唯一的。下面先介绍逻辑函数的“与-或”“或-与”表达式，接着介绍最小项的概念及逻辑函数的“最小项之和”表达式，然后再介绍最大项的概念及逻辑函数的“最大项之积”表达式。

2.2.1 逻辑函数表达式的基本形式

1. 与-或表达式

与-或表达式是指由若干与项进行或逻辑运算构成的表达式。例如有一个逻辑函数式为

$$L=A\cdot C+\overline{C}\cdot D$$

式中，$A\cdot C$ 和 $\overline{C}\cdot D$ 两项都是由与（逻辑乘）运算把变量连接起来的，故称为与项（或乘积项），然后将这两个与项用或运算符连接起来，称这种类型的表达式为与-或式，或称之为“积之和（sum of products，SOP）”表达式。

2. 或-与表达式

或-与表达式是指由若干或项进行与逻辑运算构成的表达式。例如有一个逻辑函数式为

$$L=(A+C)\cdot(B+\overline{C})\cdot D$$

式中，$(A+C)$ 和 $(B+\overline{C})$ 两项是由或（逻辑加）运算符把变量连接起来的，故称为或项，D 是单个变量，可以认为是 $(D+0)$，然后将这三个或项用与运算符连接起来，称这种类型的表达式为或-与式，或称之为“和之积（products of sum，POS）”表达式。

通常可以将逻辑函数式表示成混合形式。例如，函数 $L=A\cdot(B\cdot C+\overline{B}\cdot\overline{C})+A\cdot(B\cdot\overline{C}+\overline{B}\cdot C)$，既不是与-或式，也不是或-与式，但经过变换可以转化成上述两种基本形式。

2.2.2 最小项与最小项表达式

逻辑函数的最小项表达式是建立在最小项基础之上的，下面先介绍最小项的定义和性质。

1. 最小项的定义和性质

对于有 n 个变量的逻辑函数，若有一个乘积项包含了全部的 n 个变量，每个变量都以它的原变量或非变量的形式在乘积项中出现，且仅出现一次，则称该乘积项为最小项。例如，三个变量 A、B、C 的最小项有 $\overline{A}\,\overline{B}\,\overline{C}$、$AB\overline{C}$、$ABC$ 等，而 $\overline{A}B$、$A\overline{B}C\overline{A}$、$A(B+C)$ 等则不是最小项。

一般 n 个变量的最小项应有 2^n 个，最小项通常用 m_i 表示，下标 i 即最小项编号，用十进制数表示。将最小项中的原变量用 1 表示，非变量用 0 表示，可得到最小项的编号。例如，以三个变量的乘积项 $\overline{A}BC$ 为例，它的二进制取值为 011，可以用十进制数 3 表示，所以把最小项 $\overline{A}BC$ 记作 m_3。三个变量 A、B、C 的全部 8 个最小项及其最小项的代表符号如表 2.2.1 所示。

表 2.2.1 三变量最小项、最大项的代表符号

行号	变量取值			最小项	最大项
	A	B	C		
0	0	0	0	$m_0=\overline{A}\,\overline{B}\,\overline{C}$	$M_0=A+B+C$
1	0	0	1	$m_1=\overline{A}\,\overline{B}C$	$M_1=A+B+\overline{C}$
2	0	1	0	$m_2=\overline{A}B\overline{C}$	$M_2=A+\overline{B}+C$
3	0	1	1	$m_3=\overline{A}BC$	$M_3=A+\overline{B}+\overline{C}$
4	1	0	0	$m_4=A\overline{B}\,\overline{C}$	$M_4=\overline{A}+B+C$
5	1	0	1	$m_5=A\overline{B}C$	$M_5=\overline{A}+B+\overline{C}$
6	1	1	0	$m_6=AB\overline{C}$	$M_6=\overline{A}+\overline{B}+C$
7	1	1	1	$m_7=ABC$	$M_7=\overline{A}+\overline{B}+\overline{C}$

为了分析最小项的性质，我们列出三变量 A、B、C 所有最小项的真值表，如表 2.2.2 所示。

表 2.2.2 三变量最小项真值表

A	B	C	$\overline{A}\,\overline{B}\,\overline{C}$	$\overline{A}\,\overline{B}C$	$\overline{A}B\overline{C}$	$\overline{A}BC$	$A\overline{B}\,\overline{C}$	$A\overline{B}C$	$AB\overline{C}$	ABC
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

观察表 2.2.2 可以看出，最小项具有下列性质。

（1）任意一个最小项，输入变量只有一组取值使其值为 1，而其他各组取值均使其为 0。并且，最小项不同，使其值为 1 的输入变量取值也不同。以 $AB\overline{C}$ 为例，只有当 ABC 取 110 时，最小项 $m_6=AB\overline{C}=1$，取其他值时，m_6 均为 0。

（2）任意两个不同的最小项之积为 0。

（3）所有最小项之和为 1。

2. 最小项表达式

由若干最小项相或构成的逻辑表达式称为最小项表达式，也称为标准与-或表达式。

例 2.2.1 将逻辑函数 $L(A,B,C)=AB+\overline{A}C$ 变换成最小项之和表达式。

解：利用公式 $A+\overline{A}=1$，将逻辑函数中的每一个乘积项都化成包含所有变量 A、B、C 的项，即

$$\begin{aligned}L(A,B,C)&=AB+\overline{A}C\\&=AB(C+\overline{C})+\overline{A}(B+\overline{B})C\\&=ABC+AB\overline{C}+\overline{A}BC+\overline{A}\,\overline{B}C\end{aligned}$$

此式为四个最小项之和，是一个标准“与-或”表达式。为了简便，在表达式中常用最小项的编号表示，上式又可写为

$$L(A,B,C)=m_7+m_6+m_3+m_1=\sum m(1,3,6,7)$$

由此可见，任一个逻辑函数都能变换成唯一的最小项表达式。

例 2.2.2　将逻辑函数 $L(A,B,C)=\overline{(AB+\overline{A}\,\overline{B}+\overline{C})\overline{AB}}$ 变换成最小项表达式。

解：（1）多次利用摩根定理去掉非号，直至最后得到一个只在单个变量上有非号的表达式，即

$$\begin{aligned}L(A,B,C)&=\overline{(AB+\overline{A}\,\overline{B}+\overline{C})\overline{AB}}\\&=\overline{(AB+\overline{A}\,\overline{B}+\overline{C})}+AB\\&=(\overline{AB}\cdot\overline{\overline{A}\,\overline{B}}\cdot C)+AB\\&=(\overline{A}+\overline{B})(A+B)C+AB\end{aligned}$$

（2）利用分配律去掉括号，直至得到一个与-或表达式

$$\begin{aligned}L(A,B,C)&=(\overline{A}+\overline{B})(A+B)C+AB\\&=\overline{A}BC+A\overline{B}C+AB\end{aligned}$$

（3）式中 AB 不是最小项，用 $(C+\overline{C})$ 进行配项，可得

$$\begin{aligned}L(A,B,C)&=\overline{A}BC+A\overline{B}C+AB(C+\overline{C})\\&=\overline{A}BC+A\overline{B}C+ABC+AB\overline{C}\\&=m_3+m_5+m_6+m_7\\&=\sum m(3,5,6,7)\end{aligned}$$

2.2.3　最大项与最大项表达式

逻辑函数的最大项表达式是建立在最大项基础之上的，下面先介绍最大项的定义和性质。

1. 最大项的定义和性质

对于有 n 个变量的函数来说，若有一个或项包含了全部的 n 个变量，每个变量都以它的原变量或者非变量的形式在或项中出现，且仅出现一次，则称该或项为最大项。

一般 n 个变量的最大项应有 2^n 个。最大项通常用 M_i 表示，下标编号 i 用于区别不同的最大项。对于一个最大项，输入变量只有一组二进制数使其取值为 0，与该二进制数对应的十进制数就是该最大项的下标编号。

三个变量 A、B、C 的全部 8 个最大项及其最大项的代表符号如表 2.2.1 所示。表中每一行列出的二进制数都使与其对应的最大项的值为 0。例如，A、B、C 的取值为 010，其对应十进制数为 2，它使最大项 $(A+\overline{B}+C)=0$，所以把 $(A+\overline{B}+C)$ 记作 M_2。

根据最大项的定义，得到最大项具有下列性质：

（1）任意一个最大项，输入变量只有一组取值使得它的值为0，而在变量取其他各组值时，这个最大项的值都是1。并且，最大项不同，使其值为0的输入变量取值也不同。

（2）任意两个不同的最大项之和为1。

（3）所有最大项之积为0。

2. 最小项和最大项的关系

根据最小项和最大项的性质可知，相同变量构成的最小项与最大项之间存在互补关系，即

$$m_i=\overline{M_i} \qquad \text{或者} \qquad M_i=\overline{m_i}$$

例如，$m_2=\overline{A}B\overline{C}$，则$\overline{m}_2=\overline{\overline{A}B\overline{C}}=A+\overline{B}+C=M_2$。

例 2.2.3　将逻辑函数 $L(A,B,C)=A\cdot B+\overline{A}\cdot C$ 变换成最大项之积表达式。

解：（1）多次利用摩根定律，将函数表达式变换成或-与表达式，即

$$\begin{aligned}L(A,B,C)&=\overline{\overline{A\cdot B+\overline{A}\cdot C}}=\overline{(\overline{A}+\overline{B})(A+\overline{C})}\\&=\overline{\overline{A}\cdot A+A\cdot\overline{B}+\overline{A}\cdot\overline{C}+\overline{B}\cdot\overline{C}}\\&=(\overline{A}+B)\cdot(A+C)\cdot(B+C)\end{aligned}$$

（2）利用公式$\overline{A}\cdot A=0$和 $A+B\cdot C=(A+B)\cdot(A+C)$，将或-与式中非最大项扩展成最大项，即

$$\begin{aligned}L(A,B,C)&=(\overline{A}+B+0)\cdot(A+C+0)\cdot(B+C+0)\\&=(\overline{A}+B+C\cdot\overline{C})\cdot(A+C+B\cdot\overline{B})\cdot(B+C+A\cdot\overline{A})\\&=(\overline{A}+B+C)\cdot(\overline{A}+B+\overline{C})\cdot(A+C+B)\cdot(A+C+\overline{B})\cdot(B+C+A)\cdot(B+C+\overline{A})\\&=(\overline{A}+B+C)\cdot(\overline{A}+B+\overline{C})\cdot(A+B+C)\cdot(A+\overline{B}+C)\end{aligned}$$

此式为四个最大项之积，是一个标准“或-与”表达式。

为了简便，在表达式中常用最大项（或最大项的下标编号）表示，上式又可写为

$$L(A,B,C)=M_4\cdot M_5\cdot M_0\cdot M_2=\prod M(0,2,4,5)$$

由此可见，任一个逻辑函数经过变换，都能表示成唯一的最大项表达式。

例 2.2.4　一个逻辑电路有三个输入逻辑变量 A、B、C，它的真值表如表 2.2.3 所示，试写出该逻辑函数的最小项表达式和最大项表达式。

表 2.2.3　例 2.2.4 的真值表

行号	输入变量			输出
	A	B	C	L
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	$1\rightarrow m_3$
4	1	0	0	0

续表

行号	输入变量			输出
	A	B	C	L
5	1	0	1	$1\rightarrow m_5$
6	1	1	0	$1\rightarrow m_6$
7	1	1	1	0

解：（1）根据真值表求最小项表达式的一般方法是：

① 写出使函数 $L=1$ 的各行所对应的最小项；

② 将这些最小项相加，即得到最小项表达式。

$$\begin{aligned} L(A,B,C) &= m_3+m_5+m_6 \\ &= \sum m(3,5,6) \\ &= \overline{A}\cdot B\cdot C+A\cdot\overline{B}\cdot C+A\cdot B\cdot\overline{C} \end{aligned}$$

（2）根据真值表求最大项表达式的一般方法是：

① 写出使函数 $L=0$ 的各行所对应的最大项；

② 将这些最大项相乘，即得到最大项表达式。

$$\begin{aligned} L(A,B,C) &= M_0\cdot M_1\cdot M_2\cdot M_4\cdot M_7 \\ &= \prod M(0,1,2,4,7) \\ &= (A+B+C)\cdot(A+B+\overline{C})\cdot(A+\overline{B}+C)\cdot(\overline{A}+B+C)\cdot(\overline{A}+\overline{B}+\overline{C}) \end{aligned}$$

由上面的推导可知，由未出现在最小项表达式中的各标号组成的最大项之积就构成了最大项表达式，反之亦然。

复习思考题

2.2.1 对于有 n 个变量的函数来说，它的最小项有多少个？每个最小项中有多少个变量？

2.2.2 对于 n 个变量的函数，任意两个最小项的乘积是多少？全体最小项之和是多少？

2.2.3 相同变量构成的最小项与最大项之间存在什么关系？试以两个变量为例进行说明。

2.2.4 已知逻辑函数的最小项表达式 $L(A,B,C,D)=\sum m(1,2,3,7,8,9,10,11,12,14)$，试写出其最大项表达式。

2.3 逻辑函数的代数化简法

在设计逻辑电路时，根据实际要求直接归纳出来的逻辑函数式往往不是最简的形式，这就需要对逻辑函数式进行化简，其目的是为了降低电路实现的成本，以较少的门实现电路。

例如，图 2.3.1(a)所示电路可以简化为图 2.3.1(b)所示的电路，两个电路具有完全相同的逻辑功能。显然，简化电路使用较少的门，比原来电路的体积小且成本低。简化电路的连

线较少，减少了电路可能潜在的故障，可靠性得到进一步的改善。

简化逻辑函数的方法很多，本节将介绍代数化简法，下一节介绍卡诺图法。

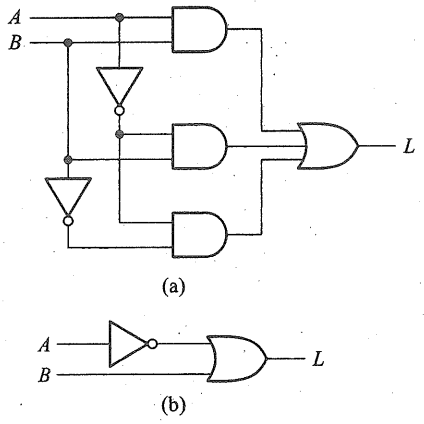


图 2.3.1 具有相同逻辑功能的两种电路实现

（a）标准积之和的电路 （b）成本最低的电路

一个逻辑函数可以有多种不同的逻辑表达式，如**与-或**表达式、**与非-与非**表达式、**或-与**表达式、**或非-或非**表达式以及**与-或-非**表达式等。通常**与-或**表达式易于转换为其他类型的函数式，所以下面着重讨论**与-或**表达式的化简。

在若干个具有相同逻辑关系的**与-或**表达式中，将其中包含的乘积项个数最少，且每个乘积项中变量数最少的表达式称为最简**与-或**表达式。

逻辑函数化简就是要消去**与-或**表达式中多余的乘积项和每个乘积项中多余的变量，以得到逻辑函数的最简**与-或**表达式。有了最简**与-或**表达式以后，再用公式变换就可以得到其他类型的函数式。

1. 逻辑函数的化简

代数法是运用逻辑代数中的定理、恒等式或规则对逻辑函数进行化简，这种方法需要一些技巧，没有固定的步骤。下面是经常使用的方法。

（1）并项法

利用公式 $A+\overline{A}=1$，将两项合并成一项，并消去一个变量。

例 2.3.1 试用并项法化简逻辑函数表达式 $L=A(BC+\overline{B}\,\overline{C})+A(B\overline{C}+\overline{B}C)$。

解：$L=ABC+A\,\overline{B}\,\overline{C}+AB\overline{C}+A\,\overline{B}C$

$=AB(C+\overline{C})+A\overline{B}(C+\overline{C})$

$=A(B+\overline{B})=A$

（2）吸收法

利用公式 $A+AB=A$，消去多余的项 AB。根据代入规则，A、B 可以是任何一个复杂的逻辑式。

例 2.3.2 试用吸收法化简逻辑函数表达式 $L=\overline{A}B+\overline{A}BCDE+\overline{A}BCDF$。

解：$L=\overline{A}B+\overline{A}BCD(E+F)=\overline{A}B$

（3）消去法

利用 $A+\overline{A}B=A+B$，消去多余的因子。

例 2.3.3 试用消去法化简逻辑函数表达式 $L=AB+\overline{A}C+\overline{B}C$。

解：$L=AB+(\overline{A}+\overline{B})C$

$=AB+\overline{AB}C=AB+C$

(4) 配项法

先利用 $A=A(B+\overline{B})$，增加必要的乘积项，再用并项或吸收的办法使项数减少。

例 2.3.4 试用配项法化简逻辑函数表达式 $L=AB+\overline{A}\,\overline{C}+B\overline{C}$。

解：$L=AB+\overline{A}\,\overline{C}+(A+\overline{A})B\overline{C}$

$=AB+\overline{A}\,\overline{C}+AB\,\overline{C}+\overline{A}B\overline{C}$

$=(AB+AB\overline{C})+(\overline{A}\,\overline{C}+\overline{A}\,\overline{C}B)$

$=AB+\overline{A}\,\overline{C}$

使用配项的方法要有一定的经验，否则越配越繁。通常对逻辑表达式进行化简，要综合使用上述技巧。以下再举一例。

例 2.3.5 化简 $L=AD+A\,\overline{D}+AB+\overline{A}C+BD+A\,\overline{B}EF+\overline{B}EF$

解：$L=A(D+\overline{D})+AB+\overline{A}C+BD+A\,\overline{B}EF+\overline{B}EF$ （利用 $A+\overline{A}=1$）

$=A+\overline{A}C+BD+\overline{B}EF$ （利用 $A+AB=A$）

$=A+C+BD+\overline{B}EF$ （利用 $A+\overline{A}B=A+B$）

2. 逻辑函数形式的变换

如果用门电路实现前面介绍的逻辑函数，需要用到非门、与门和或门。通常在一片集成电路芯片中只有一种门电路，为了减少门电路的种类，需要对逻辑函数表达式进行变换。

例 2.3.6 已知逻辑函数表达式为

$$L=\overline{A}B\overline{D}+A\,\overline{B}\,\overline{D}+\overline{A}BD+A\,\overline{B}\,\overline{C}D+A\overline{B}CD$$

要求：(1) 试求最简的与-或逻辑函数表达式，并画出相应的逻辑图；

(2) 仅用与非门画出表达式的逻辑图。

解：$L=\overline{A}B(\overline{D}+D)+A\,\overline{B}\,\overline{D}+A\,\overline{B}(\overline{C}+C)D$ （分配律）

$=\overline{A}B+A\,\overline{B}\,\overline{D}+A\,\overline{B}D$ （利用 $A+\overline{A}=1$）

$=\overline{A}B+A\overline{B}(D+\overline{D})$ （利用 $A+\overline{A}=1$）

$=\overline{A}B+A\overline{B}$ （与-或表达式）

$=\overline{\overline{A}B+A\overline{B}}$ （先利用 $\overline{\overline{A}}=A$，再用摩根定理）

$=\overline{\overline{A}B}\cdot\overline{A\overline{B}}$ （与非-与非表达式）

图 2.3.2(a) 是根据最简与-或表达式画出的逻辑图，它用到与门、或门和非门三种类型的门；而图 2.3.2(b) 是根据与非-与非表达式画出的逻辑图，它只用到两输入端与非门一种类型。注意，图 2.3.2(b) 中用两输入与非门来实现非门（即将两个输入端连接在一起）。

将与-或表达式变换成与非-与非表达式时，首先对与-或表达式取两次非，然后按照摩根定理分开下面的非号。将与-或表达式变换成或非-或非表达式时，首先对与-或表达式中的每个乘积项单独取两次非，然后按照摩根定理分开下面的非号。下面再举一例说明逻辑函数的变换。

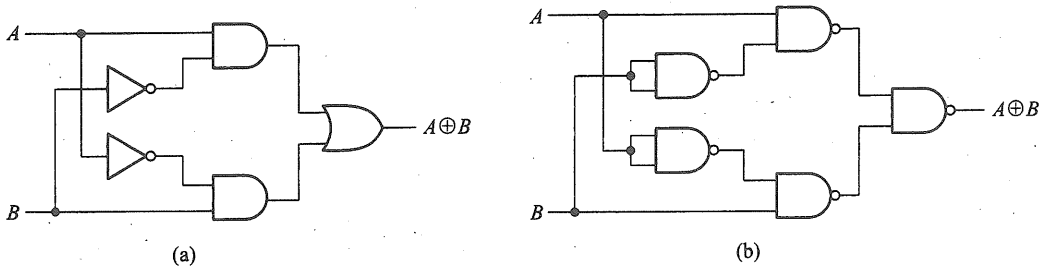


图 2.3.2 例 2.3.6 的逻辑图

(a) 与-或表达式的实现 (b) 与非门的实现

例 2.3.7 试对逻辑表达式 $L=\overline{A}\,\overline{B}C+A\overline{B}\,\overline{C}$ 进行变换，仅用或非门画出该表达式的逻辑图。

解：$L=\overline{\overline{\overline{A}\,\overline{B}C+A\overline{B}\,\overline{C}}}$

$=\overline{\overline{A+B+\overline{C}}+\overline{\overline{A}+B+C}}$ （摩根定理）

$=\overline{\overline{A+B+\overline{C}}+\overline{\overline{A}+B+C}}$ （或非-或非表达式）

逻辑图如图 2.3.3 所示。

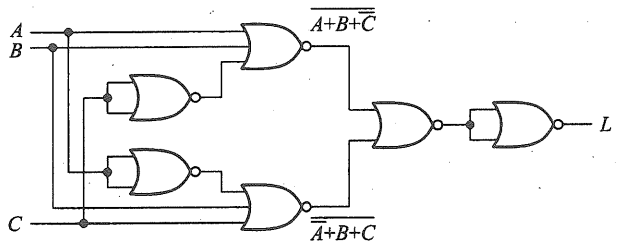


图 2.3.3 例 2.3.7 的逻辑图

复习思考题

2.3.1 对逻辑函数进行化简的目的是什么？最简与-或表达式的标准是什么？

2.3.2 对逻辑函数进行变换的目的是什么？

2.4 逻辑函数的卡诺图化简法

利用代数法可使逻辑函数变成较简单的形式，但经代数法化简得到的逻辑表达式是否为最简式较难判断。本节介绍的卡诺图[①]法可以比较简便地得到最简的逻辑表达式。

2.4.1 用卡诺图表示逻辑函数

1. 卡诺图的引出

一个逻辑函数的卡诺图就是将此函数的最小项表达式中的各最小项相应地填入一个特

① 美国工程师 Karnaugh 在 20 世纪 50 年代提出。

定的方格图内，此方格图称为卡诺图。因此，卡诺图是逻辑函数的一种图形表示。

下面从一变量卡诺图开始讨论，逐步过渡到多变量卡诺图。

对于 n 个变量的逻辑函数有 2^n 个最小项，因此一变量的逻辑函数有 2 个最小项。设变量为 D，则其两个最小项是 $\overline{D}$ 和 D，分别记为 $m_0=\overline{D}$，$m_1=D$。用两个相邻的方格表示这两个最小项，如图 2.4.1(a)所示。方格上的 $\overline{D}$ 和 D 分别表示非变量和原变量。为简明起见，非变量 $\overline{D}$ 可以不标出，只标出原变量 D，得到图 2.4.1(b)。图 2.4.1(c)是另一种更简单的画法，图中 m_0、m_1 只用其下标编号来表示。

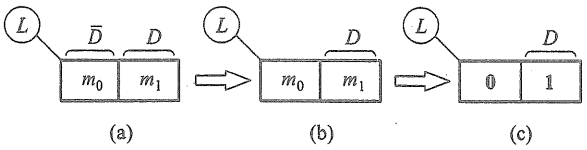


图 2.4.1 一变量卡诺图

二变量逻辑函数的最小项有 $2^2=4$ 项，如果逻辑函数的变量用 C、D 表示，则 $m_0=\overline{C}\cdot\overline{D}$，$m_1=\overline{C}\cdot D$，$m_2=C\cdot\overline{D}$，$m_3=C\cdot D$。用四个相邻的方格来表示这四个最小项。

将一变量卡诺图按照图 2.4.2(a)所示箭头方向展开，得到二变量卡诺图如图 2.4.2(b)所示。新增加的两个方格标以变量 C，新方格最小项的编号相对原方格编号增加了 $2^{n-1}=2^{2-1}=2$，即方格 0 后面为方格 2，方格 1 后面为方格 3。按照展开规律，中间两格 m_1 和 m_3 包含原变量 D，右边两方格 m_3 和 m_2 包含原变量 C，图中清楚地展示了原变量所包含的方格。

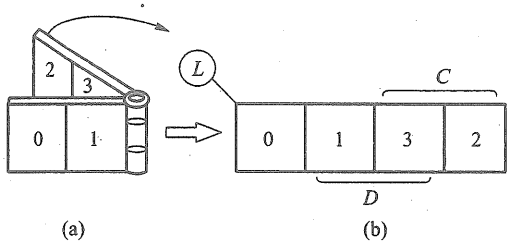


图 2.4.2 二变量卡诺图

综上所述，可归纳“折叠展开”的法则如下：

① 新增加的方格按展开方向应标以新变量；

② 新方格内最小项编号应为展开前对应方格编号加上 2^{n-1}。

按照同样的方法，从二变量卡诺图展开获得三变量卡诺图，如图 2.4.3 所示。三变量逻辑函数 $L(B,C,D)$ 有 8 个最小项，可用 8 个相邻的方格来表示。新增加的四个方格标以变量 B，并且其最小项编号相对原方格的编号加上 $2^{n-1}=2^{3-1}=4$。方格所处的位置，与相应的最小项对应，例如，2 号方格处于变量为 $\overline{B}$、C、$\overline{D}$ 的区域，则 $m_2=\overline{B}C\overline{D}$，余类推。

同理，可得四变量卡诺图，如图 2.4.4 所示。在使用时，根据方格所在区域对应的变量 A、B、C、D 的取值，可直接填入相应的最小项。

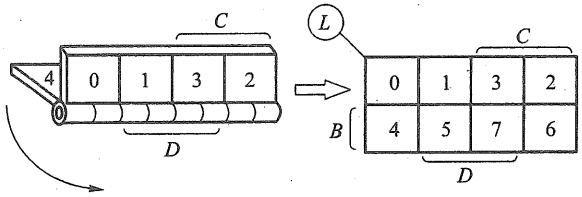


图 2.4.3 三变量卡诺图

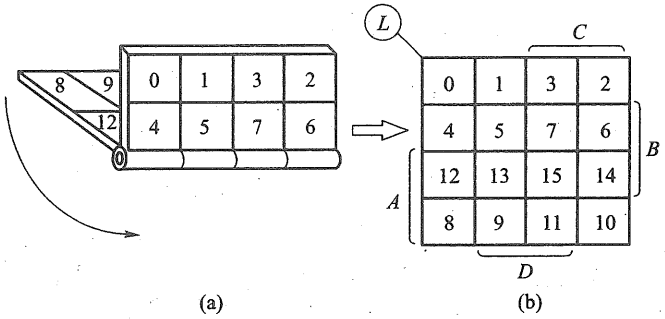


图 2.4.4 四变量卡诺图

2. 卡诺图的特点

卡诺图的特点是：几何位置相邻的最小项在逻辑上也是相邻的。即相邻的两个最小项只有一个变量不同，这是用卡诺图化简逻辑函数的主要依据。

现以四变量卡诺图为例，并将最小项用变量表示出来，如图 2.4.5 所示。图中相邻的方格内只有一个变量不同，例如，$m_4=\overline{A}B\overline{C}\,\overline{D}$，$m_5=\overline{A}B\overline{C}D$，这两个相邻最小项之间的差别仅在 D 和 $\overline{D}$，m_5 和 m_{13} 之间的差别在于 A 和 $\overline{A}$。要特别指出的是，卡诺图同一行最左端和最右端的方格是相邻的，同一列最上端和最下端两个方格也是相邻的。这个特点说明在几何位置上卡诺图具有上下、左右封闭的特性。

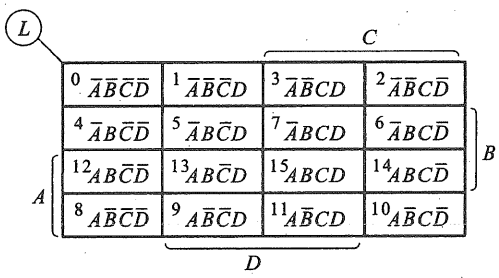


图 2.4.5 填入最小项的卡诺图

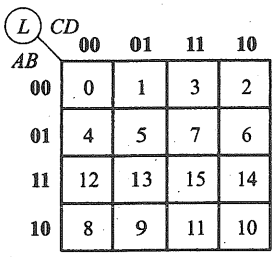


图 2.4.6 图 2.4.5 的简化表示法

3. 卡诺图的简化表示法

图 2.4.6 所示为图 2.4.5 所示的卡诺图简化形式。变量 A、B、C、D 的每组取值，与对应方格内的最小项编号一一对应。例如，$AB=11$，$CD=01$，即 1101 对应最小项 m_{13}。

4. 已知逻辑函数,画出卡诺图

当逻辑函数为最小项表达式时,在卡诺图对应最小项的方格填上 1,其余的方格填上 0(也可用空格表示),就可以得到相应的卡诺图。也就是说,任何逻辑函数都等于其卡诺图中为 1 的方格所对应的最小项之和。

例如,要画出逻辑函数 $L(A,B,C,D)=\sum m(0,1,2,3,4,8,10,11,14,15)$ 的卡诺图时,首先画出四变量卡诺图,并将逻辑函数表达式中最小项对应的方格填入 1,其余填入 0,即可得图 2.4.7 所示的 $L(A,B,C,D)$ 的卡诺图。

当逻辑函数的表达式为其他形式时,可将其变换为最小项表达式后,再作出卡诺图。

例 2.4.1 画出 $L(A,B,C,D)=(\overline{A}+\overline{B}+\overline{C}+\overline{D})(\overline{A}+\overline{B}+C+\overline{D})(\overline{A}+B+\overline{C}+D)(A+\overline{B}+\overline{C}+D)(A+B+C+D)$ 的卡诺图。

解:(1) 由摩根定理,将上式变换成

$$\begin{aligned}\overline{L}&=ABCD+AB\overline{C}D+A\overline{B}C\overline{D}+\overline{A}BC\overline{D}+\overline{A}\,\overline{B}\,\overline{C}\,\overline{D}\\&=\sum m(15,13,10,6,0)\end{aligned}$$

(2) 因上式为 $\overline{L}$,故最小项对应的方格内填入 0,其余填入 1,得到图 2.4.8 所示的卡诺图。

L / AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	0	0
11	0	0	1	1
10	1	0	1	1

图 2.4.7 $L(A,B,C,D)$ 的卡诺图

L / AB \ CD	00	01	11	10
00	0	1	1	1
01	1	1	1	0
11	1	0	0	1
10	1	1	1	0

图 2.4.8 例 2.4.1 的卡诺图

2.4.2 用卡诺图化简逻辑函数

1. 化简的依据

卡诺图具有相邻性,若两个相邻的方格均为 1,则这两个最小项之和有一个变量可以被消去,例如,图 2.4.5 所示卡诺图中的方格 5 和方格 7,其最小项之和为 $\overline{A}B\overline{C}D+\overline{A}BCD=\overline{A}BD(\overline{C}+C)=\overline{A}BD$,将两个方格中的不相同的因子 C 和 $\overline{C}$ 消去,保留不变的因子。

若四个相邻的方格为 1,则这四个最小项之和有两个变量可以被消去,如图 2.4.5 所示四变量卡诺图中的方格 2、3、7、6,它们的最小项之和

$$\overline{A}\,\overline{B}C\overline{D}+\overline{A}BCD+\overline{A}BC\overline{D}+\overline{A}BC\overline{D}=\overline{A}BC(D+\overline{D})+\overline{A}BC(D+\overline{D})=\overline{A}\,\overline{B}C+\overline{A}BC=\overline{A}C$$

消去了四个方格中不相同的两个因子,使逻辑表达式得到简化。这就是卡诺图法化简逻辑函数的基本原理。

2. 化简的步骤

用卡诺图化简逻辑函数的步骤如下:

(1) 将逻辑函数写成最小项表达式。

(2) 按最小项表达式填卡诺图,凡式中包含了的最小项,其对应方格填 **1**,其余方格填 **0**。

(3) 找出为 **1** 的相邻最小项,用虚线(或者细实线)画一个包围圈,每个包围圈含 2^n 个方格,写出每个包围圈的乘积项。

(4) 将所有包围圈对应的乘积项相加。

有时也可以由真值表直接填卡诺图,以上的(1)(2)两步就合为一步。

画包围圈的原则:

(1) 包围圈内的方格数必定是 2^n 个,n 等于 0、1、2、3、…。

(2) 相邻方格包括上下底相邻,左右边相邻和四个角两两相邻。

(3) 同一方格可以被不同的包围圈重复包围,但新增包围圈中一定要有新的方格,否则该包围圈为多余。

(4) 包围圈内的方格数要尽可能多,包围圈的数目要尽可能少。

化简逻辑函数后,一个包围圈对应一个乘积项,包围圈越大,所得乘积项中的变量越少。包围圈个数越少,乘积项个数也越少,得到的**与-或**表达式也最简。

例 2.4.2 用卡诺图法化简下列逻辑函数,求 L 的最简**与-或**表达式。

$$L(A,B,C,D)=\sum m(0,2,5,7,8,10,13,15)$$

解:(1) 由 L 画出卡诺图,如图 2.4.9 所示。

(2) 找出 **1** 的相邻最小项,用虚线画包围圈,合并最小项,得最简**与-或**表达式为

$$L=BD+\overline{B}\,\overline{D}$$

例 2.4.3 用卡诺图法化简下列逻辑函数

$$L(A,B,C,D)=(\overline{A}\,\overline{B}+B\,\overline{D})\,\overline{C}+BD(\overline{\overline{A}\,\overline{C}})+\overline{D}\,\overline{(\overline{A}+\overline{B})}$$

解:(1) 将逻辑函数化简,得到**与-或**表达式。

$$\begin{aligned}L(A,B,C,D)&=\overline{A}\,\overline{B}\,\overline{C}+B\,\overline{C}\,\overline{D}+BD(A+C)+\overline{D}AB\\&=\overline{A}\,\overline{B}\,\overline{C}+B\,\overline{C}\,\overline{D}+ABD+BCD+AB\,\overline{D}\end{aligned}$$

(2) 由逻辑表达式直接作卡诺图,如图 2.4.10 所示。

(3) 画包围圈合并最小项,得简化的**与-或**表达式为

$$L=AB+BCD+\overline{A}\,\overline{B}\,\overline{C}+\overline{A}\,\overline{C}\,\overline{D}$$

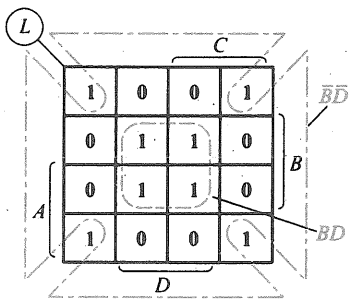


图 2.4.9 例 2.4.2 的卡诺图

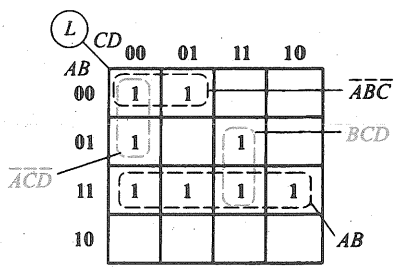


图 2.4.10 例 2.4.3 的卡诺图

注意,此例中包含最小项 m_4 的包围圈还有一种画法,即将 m_4 和 m_{12} 画成一个包围圈,得

到下面另一种简化的与-或表达式。可见，包围圈画的不同，得到的表达式也不同。

$$L=AB+BCD+\overline{A}\,\overline{B}\,\overline{C}+B\,\overline{C}\,\overline{D}$$

利用卡诺图化简时，如果卡诺图中大多数方格为 1，用包围 1 的方法需要画很多包围圈。这时采用包围 0 的方法化简更为简单。即求出非函数$\overline{L}$，再对$\overline{L}$求非，下面举例说明。

例 2.4.4 化简下列逻辑函数

$$L(A,B,C,D)=\sum m(0\sim3,5\sim11,13\sim15)$$

解：由 L 画出卡诺图，如图 2.4.11(a)所示。

方法一：用包围 1 的方法化简，如图 2.4.11(b)所示，得

$$L=\overline{B}+C+D$$

方法二：用包围 0 的方法化简，如图 2.4.11(c)所示，得

$$\overline{L}=B\,\overline{C}\,\overline{D}$$

对$\overline{L}$求非

$$L=\overline{B}+C+D$$

可见，两种方法结果相同。

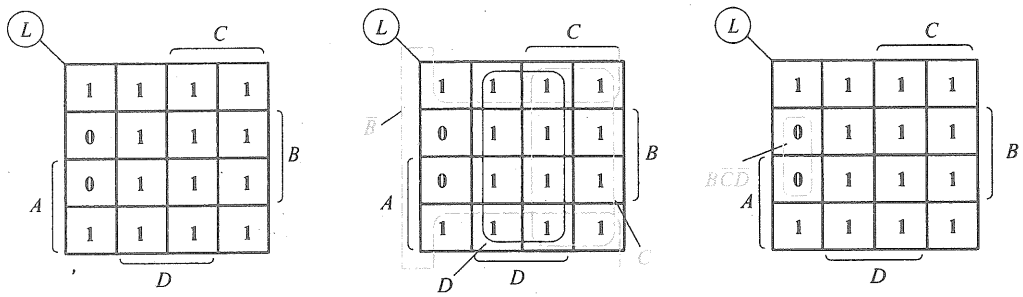


图 2.4.11 例 2.4.4 的卡诺图

3. 具有无关项的化简

在实际工作中，当逻辑变量被赋予特定含义时，有一些变量的取值组合是不应该出现的，这些变量取值组合对应的最小项称为约束项。例如，在 8421BCD 码中，1010～1111 这六种组合是不使用的代码(应该被限制)，在输入端不应该出现，因此，这六种组合对应的最小项为约束项。而在有的情况下，受客观条件所限，有些最小项根本不会出现。例如，落地电风扇上有 4 个互锁的琴键开关(用 A_0～A_3表示)。按下 A_0时，电扇停转，A_1、A_2和 A_3用来控制电扇的风速(按下 A_3时，风速最大)，每次只有一个开关能够被按下，即对应于变量 $A_0A_1A_2A_3$的组合只会出现 1000、0100、0010 和 0001，其他取值组合客观上是不会出现的，于是其函数值可以是任意的(它的值可以取 0 或取 1)，将变量取这些值所对应的最小项称为任意项。

教学视频 2.1：具有无关项的逻辑函数化简

约束项和任意项统称为无关项。合理利用无关项，可以使逻辑函数得到进一步简化。

无关项的值可以取 0 或取 1，具体取什么值，应该以得到的逻辑函数式最简为准。下面举一个利用无关项化简的例子。

例 2.4.5 试写出判断 1 位十进制数(以 8421BCD 码表示)奇偶性电路的输出逻辑函数的最小项表达式。当十进制数为奇数时,电路输出为 **1**;否则输出为 **0**。并求简化的逻辑函数表达式。

解:用 8421BCD 码表示十进制数,输入变量用 A、B、C、D 表示,输出用 L 表示。当十进制数为 1、3、5、7、9 时,L 为 **1**,当十进制数为 0、2、4、6、8 时,L 为 **0**。注意,8421BCD 码只有十个数,其余的 10、11、12、13、14、15 六种输入不应该出现,它们所对应的最小项就是无关项。与用 m 表示最小项类似,用 d 表示无关项,其对应的函数值通常以×(或 ϕ)表示。则

$$L=\sum m(1,3,5,7,9)+\sum d(10,11,12,13,14,15)$$

根据 L 的表达式填写四变量卡诺图,如图 2.4.12 所示。画包围圈,利用无关项,将最小项 m_{13}、m_{15}、m_{11} 对应的方格视为 1,可以得到最大的包围圈,由此得到逻辑表达式

$$L=D$$

若不利用无关项,$L=\overline{A}D+B\,\overline{C}D$,结果复杂得多。

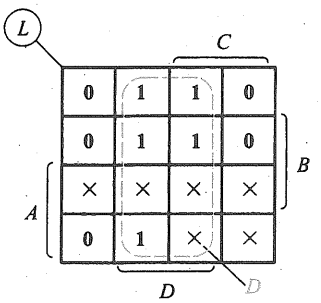


图 2.4.12 例 2.4.5 的卡诺图

以上讨论的是从一变量到四变量的卡诺图,并以四变量卡诺图为例,分析了逻辑函数的化简方法。至于更多变量,如五变量或六变量卡诺图的应用,可参阅相关文献。

复习思考题

2.4.1 什么是无关项?

2.4.2 使用卡诺图化简逻辑函数的依据是什么?

2.5 硬件描述语言 Verilog HDL 基础

文档 2.2:硬件描述语言(HDL)发展简介

硬件描述语言(HDL)类似于计算机高级程序设计语言(如 C 语言等),它是一种以文本形式来描述数字系统硬件结构和功能的语言,用它可以表示逻辑电路图、逻辑表达式,还可以表示更复杂的数字逻辑系统所完成的逻辑功能(即行为)。人们还可以用 HDL 编写设计说明文档,这种文档易于存储和修改,适用于不同的设计人员之间进行技术交流,还能被计算机识别和处理,计算机对 HDL 的处理包括两个方面:逻辑仿真和逻辑综合。

逻辑仿真是指用计算机仿真软件对数字逻辑电路的结构和行为进行预测,仿真器对 HDL 描述进行解释,以文本形式或时序波形图形式给出电路的输出。在电路被实现之前,设计人员根据仿真结果可以初步判断电路的逻辑功能是否正确。在仿真期间,如果发现设计中存在的错误,可以对 HDL 描述进行修改,直至满足设计要求为止。

逻辑综合是指将 HDL 描述的电路的逻辑关系,转换为门和触发器等元件及其相互之间的连接关系表(常称为**门级网表**)的过程,即将 HDL 代码转换成真实的硬件电路。它类似于高级程序设计语言中对一个程序进行编译,得到目标代码的过程。所不同的是,逻辑综合不会产生目标代码,而是产生门级元件及其连接关系的数据库,根据这个数据库可以制作出集成电路或印制电路板(printed circuit board,PCB)。

目前,Verilog HDL(简称 Verilog)和 VHDL 在数字设计领域的使用非常广泛,它们都已经被 IEEE 认可为工业标准。尽管这两种语言在很多方面都有所不同,但在学习逻辑电路时,设计者使用任何一种语言都可以完成自己的任务,并且 Verilog 的句法根源出自通用的 C 语言,较 VHDL 易学易用。作为入门,本书以 Verilog 为主讲解硬件描述语言的基本知识。

2.5.1 Verilog HDL 模块的基本结构

模块(**module**)是 Verilog 描述电路的基本单元,它可以表示一个简单的门电路,也可以表示功能复杂的数字电路。通常逻辑电路的功能可以使用 Verilog 的一个或多个模块进行描述(也称为建模),不同的模块之间通过端口进行连接。

定义模块时总是以关键词[①] **module** 开始,并以关键词 **endmodule** 结束。模块的组成如图 2.5.1 所示。[②]

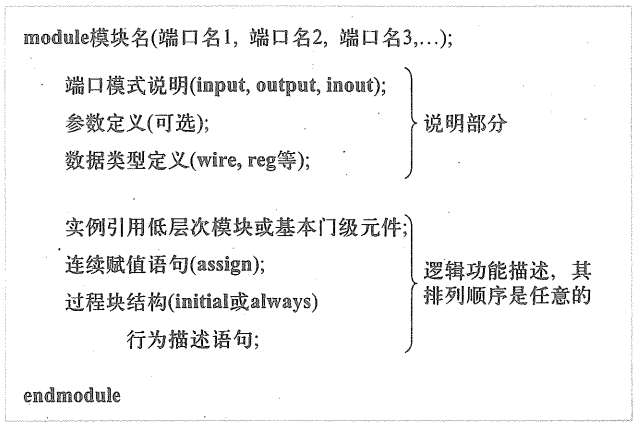


图 2.5.1 Verilog HDL 模块组成

module 后面紧跟着“模块名”,模块名是模块唯一的标识符;在模块名后面的圆括号中列出该模块的输入、输出端口名称,各个端口名称之间以逗号分隔。在 Verilog 中,通常以 **input**(输入)、**output**(输出)、**inout**(双向端口)来说明信号流经端口的方向;“参数定义”是将数值常量用符号常量代替,以增加程序的可读性和可修改性,它是一个可选择的语句。“数据类型定义”部分用来指定模块内所用的数据对象为连线(**wire**)类型还是寄存器(**reg**)类型。

接着,描述模块将要完成的逻辑功能。通常有三种不同的描述风格:一是实例引用低层次模块的方法,即调用其他已定义过的层次较低的子模块对整个电路的结构进行描述,或者直接调用 Verilog 内部预先定义的基本门级元件描述电路的结构,通常将这种方法称为**结构描述方式**(对仅使用基本门级元件描述电路功能的方式,也称为**门级描述方式**);二是使用连续赋值语句(**assign**)对电路的逻辑功能进行描述,通常称之为**数据流描述方式**,该方式特别便于对组合逻辑电路建模;三是使用过程块语句结构(包括 **initial** 结构和 **always** 结构两

① 关键词是 Verilog 本身规定的特殊字符串,用于表示特定的含义。本书为清晰起见,将关键词以粗体字印刷。

② 为了与程序保持一致,本书 Verilog HDL 部分的变量均与程序一致,采用正体。

种)和比较抽象的高级程序语句对电路的逻辑功能进行描述,通常称之为行为描述方式。行为描述侧重于模块的逻辑功能,不涉及实现该模块的详细硬件电路结构。

设计人员可以选用这三种方式中的任意一种或混合使用几种方式对电路的功能进行描述,并且这几种描述方式在程序中排列的先后顺序是任意的。这些描述方式将在 4.6 节、5.6节和 6.7 节做进一步介绍。除此之外,在 3.8 节中还介绍了开关级描述方式。

例 2.5.1　用 Verilog 对图 2.5.2 所示简单逻辑电路进行建模。

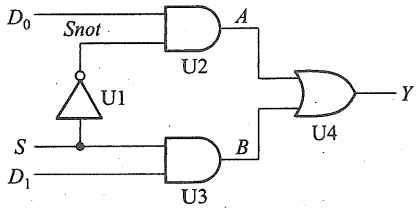


图 2.5.2 · 简单的门电路

解:该电路的逻辑表达式为

$$Y=D_0\cdot\overline{S}+D_1\cdot S$$

当 $S=0$时,输出 $Y=D_0$,当 $S=1$ 时,$Y=D_1$。可见,该电路根据输入控制信号 S 的值,来选择两路输入信号(D_0、D_1)中的一路作为输出。因此,该电路被称为 2 选 1 数据选择器。

下面使用两种不同的描述方式对该电路建模。图 2.5.3 是 Verilog 的门级描述方式,而图 2.5.4 则是数据流的描述方式。

在 Verilog 设计中,使用文件的方式来保存模块的描述,文件的扩展名为 *.v,图 2.5.3 和图 2.5.4 中代码的第 1 行均给出了文件名。双斜线"//"后面是注释,到本行结尾处,注释结束。

```
// 文件名: mux2to1.v
module mux2to1(D0,D1,S,Y);
  input D0,D1,S;   //输入端口
  output Y;        //输出端口
  wire Snot,A,B ;  //内部节点

//电路逻辑功能描述
  not U1(Snot,S); //实例名 U1 可省略
  and U2(A,D0,Snot);
  and U3(B,D1,S);
  or U4(Y,A,B);
endmodule
```

图 2.5.3　Verilog HDL 门级描述

```
//文件名: mux2to1_df.v
module mux2to1_df (
  input D0,D1,S,      //输入端口
  output wire Y       //输出端口及数据类型
);

  //电路功能描述
  assign Y=(~S & D0) |(S & D1);

endmodule
```

图 2.5.4　Verilog HDL 数据流描述

图 2.5.3 是根据图 2.5.2 所示的电路图进行描述的。首先描述了电路的输入、输出端口,以及电路内部的节点信号,接着描述了电路的结构(即逻辑功能),这里直接调用 Verilog 语言内部预先定义的基本门级元件(**not**、**and**、**or**)来描述逻辑图中的元件以及元件之间的连线关系。在每个逻辑门元件后面跟着一个实例名(U1、U2 等)和由圆括号括起来、以逗号分隔的电路端口;Verilog 规定:输出端口总是位于圆括号内左边第 1 个位置,输入端口跟在后面。例如,对于实例名为 U4 的或门,其输出端口是 Y、输入端口是 A 和 B,实例名可以直接使用,不需要预先定义。实际上,调用基本门级元件时,实例名可以省略不写。

每一条语句以分号结尾,但模块最后的 **endmodule** 后面没有分号。由于这个模块是用来描述电路逻辑功能的,故将该模块称为设计块。

图 2.5.4 的代码采用了数据流描述方式。这里,模块的声明如下:

```
module mux2to1_df(input D0,D1,S,output wire Y);
```

这是修订版本 Verilog-2001/2005 标准的写法。在模块的端口列表中,直接声明端口方向,不再需要独立的 **input** 和 **output** 语句,这使代码变得更加紧凑。本书后面的代码将采用这种方式。

电路功能是根据前面推导出来的逻辑表达式描述的。由关键词 **assign** 开始,后面跟着由操作数和运算符(类似于 C 语言中的运算符)组成的逻辑表达式。使用逻辑表达式编写 Verilog 代码,具有简明、易懂的优点。

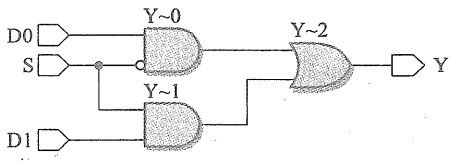


图 2.5.5 图 2.5.4 代码逻辑综合的结果

将图 2.5.4 的代码送到逻辑综合软件[①]进行综合,得到如图 2.5.5 所示的逻辑图,它与图 2.5.2 所示的门级电路完成的功能相同。

2.5.2 逻辑功能的仿真与测试

一旦完成了设计块后,接下来就要测试这个设计块描述的逻辑功能是否正确。为此必须在输入端口加入测试信号,以便检测输出端口的结果是否正确,这一过程常称为搭建测试平台(test bench)。

根据仿真软件的不同,搭建测试平台的方法也不同,下面以 ModelSim 软件[②]为例进行说明。

在对一个设计块进行仿真时,需要准备一个测试模块。该模块大致由三部分组成:第一部分实例引用被测试的模块(即设计块)。第二部分是给输入信号赋各种不同的组合值,即激励信号。第三部分指定测试结果的显示格式,并指定输出文件名。由于测试模块的主要任务是给设计块提供激励信号,所以也称为激励块[③]。

激励块通常是顶层模块,同样用 Verilog HDL 来描述。它是以 **module** 开始并以 **endmodule** 结尾的,内部包括模块名、数据类型的声明、低层次模块的实例引用和行为语句块(**initial** 或者 **always**),但是不需要声明端口。

在激励块中,通常会使用编译指令`**timescale** 将时间单位与实际时间进行关联。编译指令(compiler directives)为仿真工具提供有关如何解释 Verilog 模型的附加信息。编译指令放在模块定义之前,并以反引号(即`)开头。`**timescale** 的格式为:

```
`timescale time_unit/time_precision
```

其中,time_unit 为仿真时间单位,time_precision 为仿真时间的精度(即最小分辨度)。注意,时间单位通常大于或等于时间精度。

---

① 这是 IntelFPGA 开发软件 Quartus Prime 18.1 进行逻辑综合的结果。

② 这里使用与 Quartus Prime 18.1 软件配套的 ModelSim-Intel FPGA Starter Edition 10.5b。

③ 有关激励块的相关知识见附录 C。

图 2.5.6 是 2 选 1 数据选择器激励块代码，文件名为 test_mux2to1_df.v。第 1 条语句用 `timescale 将激励块中所有延迟时间的单位设置成 1 ns，模块后面的#5 代表延迟 5 ns。

```
//文件名 : test_mux2to1_df.v
`timescale 1ns/1ns                    //时间单位为 1 ns, 精确度为 1ns
module test_mux2to1_df;               //激励块（顶层模块）没有端口列表
  reg PD0, PD1, PS;                   //声明输入信号
  wire PY;                            //声明输出信号

  //实例引用设计块
  mux2to1_df  t_mux (PD0, PD1, PS, PY);    //按照端口位置进行连接

initial begin                         //激励信号
      PS = 0; PD1 = 0; PD0 = 0;       //语句1
  #5  PS = 0; PD1 = 0; PD0 = 1;       //语句2
  #5  PS = 0; PD1 = 1; PD0 = 0;       //语句3
  #5  PS = 0; PD1 = 1; PD0 = 1;       //语句4
  #5  PS = 1; PD1 = 0; PD0 = 0;       //语句5
  #5  PS = 1; PD1 = 0; PD0 = 1;       //语句6
  #5  PS = 1; PD1 = 1; PD0 = 0;       //语句7
  #5  PS = 1; PD1 = 1; PD0 = 1;       //语句8
  #5  PS = 0; PD1 = 0; PD0 = 0;       //语句9
  #5  $stop;                          //语句10
      end
  initial begin   //输出部分
      $monitor($time, ":\tS = %b\tD1 = %b\tD0 = %b\tY = %b",PS,PD1,PD0,PY);
    end
endmodule
```

图 2.5.6 激励块代码

文档 2.3：例 2.5.1 中 2 选 1 数据选择器的仿真步骤

接着，声明激励块的名称为 test_mux2to1_df。在激励块中，可以重新定义一套信号端口名称，也可以与设计块中的端口名相同，但输入信号的数据类型要求为 **reg**，以便保持激励值不变，直至执行到下一条激励语句为止。输出信号的数据类型要求为 **wire**，以便能随时跟踪激励信号的变化。

接下来，实例引用（调用）设计块，按照端口排列顺序一一对应地将激励块中的信号端口与设计块中的端口相连接，在本例中，将 PD0、PD1、PS、PY 分别对应地连接到设计块 mux2to1_df 中的端口 D0、D1、S、Y。

initial 语句下面的过程赋值语句给出了激励信号的输入值。仿真时，刚进入 **initial** 的时刻为 0，此时执行语句 1，将 PS、PD1 和 PD0 的值初始化

为 0,隔 5 ns 后,执行语句 2,将 PD0 的值设置为 1,PS、PD1 的值仍保持 0 不变;再隔 5 ns,执行语句 3,将 PD0 的值设置为 0,将 PD1 的值设置为 1,PS 的值仍保持 0 不变……语句 10 执行后,**initial** 语句永远被挂起。

最后面的 **initial** 语句描述了要监视的输出信号,它和前面的 **initial** 语句是同时并行执行的。代码中的 **$monitor**、**$time** 和 **$stop** 为 Verilog HDL 的系统任务,**$monitor** 将信息以指定的格式输出到屏幕上,双引号括起来的是要显示的内容,%b 代表它后面的信号用二进制格式显示,\t 代表水平制表符,**$time** 将返回当前的仿真时间,**$stop** 为停止仿真,但不退出仿真环境。常用的系统任务和编译指令将在附录 C 中介绍。

对例 2.5.1 进行仿真[①]时,打开任意一个文本编辑器(或者使用 ModelSim 软件自带的编辑器),输入设计块和激励块源文件,并存放在一个新建的子目录中。然后,在 ModelSim 软件创建一个新的工程设计项目,添加已经存在的源文件(mux2to1_df.v 和 test_mux2to1_df.v),并编译,最后进行逻辑功能仿真,得到图 2.5.7 所示的波形。同时,在 ModelSim 的 Transcript 窗口以文本方式显示的结果如图 2.5.8 所示。注意,如果激励模块中没有 **$monitor** 语句,仿真结果中就只有波形图,而不会出现图 2.5.8 所示的结果。

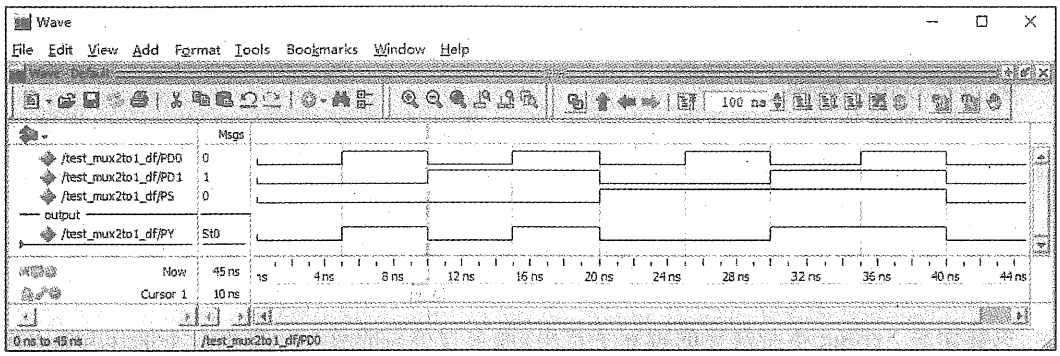


图 2.5.7 例 2.5.1 的仿真输出波形

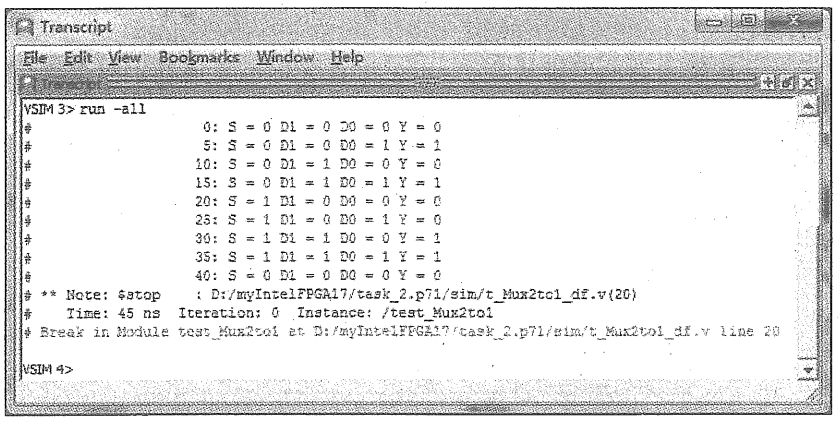


图 2.5.8 以文本方式显示的仿真结果

① 仿真的详细步骤见附录 D。

由图 2.5.7 可知,在 0~20 ns 期间,由于 PS=0,所以输出 PY 与输入 PD0 相同;在 20~40 ns 期间,PS=1,故输出 PY 与输入 PD1 相同。表明该设计块描述的逻辑功能是正确的。

对一个实际的门电路来说,信号从输入端传到输出端存在着延时,在使用 Verilog 建模时,有时需要说明门电路的延时,有关内容可以阅读专门书籍。

2.5.3 Verilog HDL 的基本语法规则

要用 Verilog 对逻辑电路建模,必须了解 Verilog 语言的基本语法规则。下面进行简单介绍。

1. 间隔符

Verilog 的间隔符包括空格符(\b)、TAB 键(\t)、换行符(\n)及换页符。如果间隔符并非出现在字符串中,则该间隔符被忽略。所以编写程序时,可以跨越多行书写,也可以在一行内书写。

2. 注释符

Verilog 支持两种形式的注释符:/*---*/和//。其中,/*---*/用于书写多行注释;//为单行注释符,以双斜线//开始到行尾结束。注释只是为了改善程序的可读性,在编译时不起作用。

3. 标识符和关键词

给对象(如模块名、电路的输入与输出端口、变量等)取名所用的字符串称为标识符。标识符(Identifier)通常是由英文字母、数字、$符或者下画线组成的字符序列,但标识符的第一个字符必须是英文字母或下画线。此外,标识符是区分大、小写英文字母的。例如,A 和 a 是两个不同的标识符;clk、counter8、_net、bus_A 等都是合法的标识符,而 2cp、$latch、a*b 则是非法的标识符。

关键词是 Verilog 本身规定的特殊字符串,通常为小写的英文字符串。不能作为标识符使用,在 Verilog 中有 100 个左右预定义的关键词。例如,**module**、**endmodule**、**input**、**output**、**wire**、**reg**、**and** 等都是关键词。

4. 逻辑值集合

为了表示数字逻辑电路的逻辑状态,Verilog 语言规定了 4 种基本的逻辑值(也被称为四值逻辑系统),如表 2.5.1 所示。

表 2.5.1 4 种基本的逻辑值

0	逻辑 0、逻辑假
1	逻辑 1、逻辑真
x 或 X	不确定的值(未知状态)
z 或 Z	高阻态

5. 常量及其表示

Verilog 中有两种类型的常量:整数型常量和实数型常量。

整数型常量有两种不同的表示方法:

(1) 使用简单的十进制数形式表示整数,例如:30、-2 都是十进制数表示的整数。用这种方法表示的整数被认为是有符号的常量,负数可以使用补码形式表示。

(2) 使用带基数的形式表示整数,由 4 个部分组成,其格式为①:

<+/-><size>'<signed>radix integer_number

其中<+/->表示常量是正整数还是负整数,当常量为正整数时,前面的正号可以省略;<size>表示常量对应二进制数的位数(位宽);<signed>用小写 s 或大写 S 表示有符号的数;radix 为 b 或 B(表示二进制数)、d 或 D(表示十进制数)、h 或 H(表示十六进制数)、o 或 O(表示八进制数)之一,integer_number 是基于基数(radix)格式的数值。数值中的 x、z 以及十六进制数的 a~f 是不区分大小写的。下面是一些常数的示例:

4'b1x0x　　4 位的二进制数

8'he3　　8 位十六进制数

8'b1001_0011　　在数字之间可以增加下画线,以改善可读性

基数格式的数中如果没有 s,则通常为无符号数。另外,表示整数位宽的常数是可选的,如果没有定义位宽,则该数的位宽至少为 32 位。例如,

'hAF　　32 位十六进制数,无符号

'sb1011　　32 位二进制数,有符号的-5

若定义的位数较少,则数值最左边的多余位将被截断。若定义的位数多于数值所需要的位数时,对无符号数则在数值的左边填 0 补齐,而对有符号数则在左边填符号位补齐。但是如果数值的最左边一位为 x 或 z,则相应地用 x 或 z 在左边补齐。例如,

3'sb10100　　等同于 3'sb100

10'bx0x1　　左边补 x,结果为 xx_xxxx_x0x1

8'sb101101　　左边补符号位(1),结果为 1110_1101

实数型常量也有两种不同的表示方法:

(1) 使用十进制数表示法。例如,0.1、2.0、5.67 等都表示实数型常量。

(2) 使用科学记数法。例如,23_5.1e2、3.6E2、5E-4 等,它们以十进制数表示分别为 23 510.0、360.0 和 0.0005。

6. 参数

参数是由一个标识符和一个常量组成的,参数经常用于指定延迟、变量的位宽等。参数声明语句的格式为:

parameter<signed><[msb:lsb]>参数名 1=常量表达式 1,参数名 2=常量表达式 2,…;

其中,signed 表示参数为有符号的数;[msb:lsb]用于指定参数值的范围。下面是一些参数声明的示例:

```
parameter N=4;
parameter S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;//声明 4 个无符号数值
parameter signed [3:0]MEM_DR=-5,CPU_SPI=6;//声明 2 个有符号的 4 位数值
```

标识符 N 在代码中可以用在表示数字 4 的地方,而 S0 可以用于替换数值 2'b00,等等。

① 格式中的尖括号<>中的内容是可选的,下文沿用这一规定。

注意:使用参数声明语句只能对参数赋一次值。参数是局部的,仅在声明该参数的模块内部起作用。参数值可以在编译时被改变。可以使用 **defparam**(即重新定义参数)语句来改变参数值,也可以通过在实例引用模块的语句中指定新的参数值而加以修改。

7. 字符串

字符串是双引号内的字符序列,不允许分成多行书写。在表达式和赋值语句中,字符串要转换成无符号整数,用一串 8 位 ASCII 值表示,每一个 8 位 ASCII 值代表一个字符。例如:字符串"ab"等价于 16'h6162。存储字符串"INTERNAL ERROR",需要定义 8×14 位的变量。字符串中的特殊字符必须用转义字符"\"来说明。

2.5.4 数据类型

Verilog 有两大类数据类型:一类是线网类型(net type),另一类是变量[①]类型(variable type)。

1. 线网类型

线网类型是硬件电路中元件之间实际连线的抽象。线网的值由驱动元件的值决定。如果没有驱动元件连接到线网,则线网的默认值为高阻态 z(线网 **trireg** 除外,它的默认值为 x)。例如,图 2.5.9 所示线网 L 跟与门 G1 的输出相连,线网 L 的值由与门的驱动信号 a 和 b 所决定,即 L = a&b。a、b 的值发生变化,线网 L 的值会立即跟着变化。

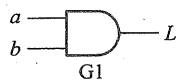


图 2.5.9 线网示意图

线网类型有 **wire**、**wand**、**wor**、**tri**、**triand**、**trior**、**trireg** 等多种。常用的线网类型由关键词 **wire** 进行声明,其格式如下:

wire<**signed**><[msb:lsb]>net_name1,net_name2,…,net_nameN;

关键词 **signed** 是可选的,用 **signed** 声明的线网是有符号数(以 2 的补码形式保存)的线网;没有 **signed** 时表示无符号数的线网。位宽[msb:lsb]也是可选的,不指定位宽时,默认为 1 位宽的线网。

在 Verilog 模块中,如果没有明确定义输入、输出端口的数据类型,则默认为是 1 位宽的 **wire** 型。下面是一些例子:

```
wire a,b,L;                    //声明 3 个 1 位的线网(连线)
wire signed [7:0]usb_data;     //声明一个 8 位宽的线网,其数值为 2 的补码形式
wire [32:1]busA,busB,busC;     //声明 3 个位宽为 32 的线网
```

2. 变量类型

线网(**wire**)提供了一种逻辑元件之间相互连接的方式,但在行为描述方式中,却不能给线网赋值。为了实现这个目的,Verilog 提供了变量。变量表示一个抽象的数据存储单元,它只能在 **initial** 或 **always** 内部被赋值,在一条行为描述语句中给变量赋值后,这个值会保持不变,直到被再次赋值为止。变量在没有被赋值前,它的默认值是 x。

Verilog 有 5 种不同的变量类型,如表 2.5.2 所示。

---

① 在 IEEE 1364-2001 标准公布之前,该类型被称作"寄存器类型(register type)"。

表 2.5.2 变量类型及其说明

寄存器类型	功能说明
reg	用于行为描述,表示存储逻辑值的变量。其值可以取 0、1、x 和 z
integer	32 位带符号的整数型变量
real,realtime	64 位带符号的实数型变量,默认值为 0
time	64 位无符号的时间型变量

最常用的变量类型是用关键词 reg 声明的寄存器类型,其格式如下:

reg<signed><[msb:lsb]>reg_name1,reg_name2,…,reg_nameN;

reg 型变量的值通常被解释为无符号数,但可以用 signed 声明它是有符号变量,此时 reg 保存的就是有符号数(以 2 的补码形式保存)。位宽[msb:lsb]也是可选的,如果没有指定位宽,则默认为是 1 位宽的 reg 型变量。下面是一些例子:

```
reg clock;                    //声明一个无符号寄存器型变量
reg [3:0]counter;             //声明一个无符号的 4 位寄存器变量
reg signed [16:1]sim_counter; //sim_counter 以 2 的补码形式保存有符号数
```

在 Verilog 中,常常将位宽为 1 的线网(例如 wire)或变量(例如 reg)称为标量,而将位宽大于 1 的线网或变量称为向量(vector)。

integer、real、realtime 和 time 等变量都是纯数学的抽象描述,不对应任何具体的硬件电路。integer 型变量通常用于对整数型常量进行存储和运算,在算术运算中,integer 型数据被视为有符号的数,用二进制补码的形式存储。而 reg 型数据通常被当作无符号数来处理。注意 integer 型变量不能使用位向量,例如 integer[3:0]num;的定义是错误的。

real 和 realtime 型变量完全相同,通常用它们对实数型常量进行存储和运算,实数不能定义位宽,其默认值为 0。当实数值被赋给一个 integer 型变量时,只保留整数部分的值,小数点后面的值被截掉。real 型变量的应用举例如下:

```
real delta;          //声明一个实数型变量
initial
 begin
   delta=4e10;  //给 delta 赋值
   delta=2.13;
 end
integer i;           //声明一个整型变量
initial
   i=delta;          //i 得到的值是 2(只将实数 2.13 的整数部分赋给 i)
```

time 型变量主要用于存储仿真的时间,它只存储无符号数。每个 time 型变量存储一个至少 64 位的时间值。为了得到当前的仿真时间,常调用系统函数 $time。time 型变量的应用举例如下:

```
time current_time;           //声明一个时间类型的变量
```

```
initial
    current_time = $time;  //保存当前的仿真时间到变量 current_time 中
```

2.5.5 运算符及其优先级

1. 运算符

数据流建模时,会用到运算符来描述逻辑函数表达式。表 2.5.3 列出了 Verilog 语言提供的运算符及其符号。按照功能可以大致分为算术运算符、逻辑运算符、关系运算符、移位运算符等几大类。

表 2.5.3 Verilog HDL 的运算符

类型	符号	功能说明	类型	符号	功能说明
算术运算符	+	二进制加	关系运算符	>	大于
	-	二进制减		<	小于
	-	2 的补码		>=	大于或等于
	*	二进制乘		<=	小于或等于
	/	二进制除		==	相等
	%	取余(求模)		!=	不相等
	**	指数幂		===	case 中使用的等于
				!==	case 中使用的不等于
按位逻辑运算符	~	非(取反)	缩位运算符(一元运算符)	&	缩位与
	&	按位与		~&	缩位与非
	|	按位或		|	缩位或
	^	按位异或		~|	缩位或非
	^~或~^	按位同或		^	缩位异或
				^~或~^	缩位同或
逻辑运算符	!	逻辑非	移位运算符	>>	逻辑右移
	&&	逻辑与		<<	逻辑左移
	||	逻辑或		>>>	算术右移
				<<<	算术左移
位拼接运算符	{,}	将多个操作数拼接成一个操作数	条件运算符	?:	根据条件表达式是否成立,选择表达式
	{{}}				

算术运算符的操作数是数值。运算符+、-、*、/用来对一对操作数求和、差、积、商。取余运算符(%)求两个数相除的余数。指数幂运算符(**)是 Verilog-2001 中新增加的,底数和指数可以是实数型、整数型或带符号数,结果为双精度浮点数值。

按位(bitwise)运算是将两个操作数的对应位进行相应的逻辑运算,操作数是几位数,则运算结果也是几位数。取反运算符(~)是一元运算符,它对一个向量操作数进行取反,结果还是向量。

缩位(reduction)运算符是一元运算符,它和按位运算符是不同的。缩位运算是对单个操作数的各位进行相应运算(从右到左一位一位地进行运算),产生的结果是 1 位二进制数。注意,负数不能使用缩位运算符。

逻辑运算的结果为 1 位,它用来判断条件的真假,如果条件为真,结果为 1;如果条件是假,结果为 0;如果条件是不确定的,结果将是 x。其中,逻辑运算符! 为一元运算符,而 && 和 || 为二元运算符。如果操作数只有 1 位,那么 1 代表逻辑 1,0 代表逻辑 0;如果操作数由多位组成,若操作数的每一位都是 0,则认为该操作数具有逻辑 0 值;反之,若操作数中某一位为 1,则认为该操作数具有逻辑 1 值;如果任一个操作数为 x 或 z,则逻辑运算的结果为不定态 x。

假设 A、B 的值分别为 4'b1010 和 4'b1111,表 2.5.4 为上述三种不同运算的比较。

表 2.5.4 按位运算、缩位运算和逻辑运算的比较

按位运算	~A=0101 ~B=0000	A&B=1010	A|B=1111	A^B=0101	A~^B=1010
缩位运算	&A=1&0&1&0 =0	~&A=1 &B=1	|A=1 ~|B=0	^A=0 ^B=0	~^A=1 ~^B=1
逻辑运算	!A=!(1010) =!(1)=0	A && B =1 && 1=1	A || B=1 || 1=1		

位拼接运算符的作用是将两个或多个信号的某些位拼接起来成为一个新的操作数,进行运算操作。例如,设 A=1'b1,B=2'b10,C=2'b00,若将操作数 B、C 拼接起来,则得到{B,C}=4'b1000;若将操作数 A、操作数 B 的第 1 位和 C 的第 0 位拼接起来,则得到一个 3 位向量的新操作数,即{A,B[1],C[0]}=3'b110。若将操作数 A、B、C 和 3'b101 拼接起来,则得到一个 8 位向量的新操作数,即{A,B,C,3'b101}=8'b11000101。

对同一个操作数重复拼接还可以使用双重大括号构成的运算符{{}},例如{4{A}}=4'b1111,{2{A},2{B},C}=8'b11101000。注意,参与拼接的操作数必须标明位宽。

关系运算的结果为 1 位,可用来判断条件的真假,如果条件为真,结果为 1;如果条件是假,结果为 0;如果条件是不确定的,结果将是 x。在 Verilog 四值逻辑(0、1、x、z)系统中,关系运算符===和!==按位测试两个数值相同还是不同。例如,如果 A=4'b1xx0、B=4'b1xx0,那么 A===B 的结果为真,但是 A==B 的结果为 x。

Verilog-2001 有逻辑移位运算符和算术移位运算符。逻辑移位时,空出的位用 0 填充,例如,逻辑右移时,左边空出的位用 0 填充。算术右移时,空出的位要用左边最高位(MSB)的数值来填充。例如,A=11010,则执行 A>>>1 后,结果为 A=11101。逻辑左移与算术左移没有区别。

2. 运算符的优先级

Verilog 主要运算符的优先级如表 2.5.5 所示。优先级的顺序从下向上依次增加。列在同一行的运算符优先级别相同。所有运算符(条件运算符?:除外)在表达式中都是从左向右结合的,使用圆括号()可以改变运算的先后顺序。为了避免发生误解,推荐使用圆括号

去隔开不同的表达式,以使代码的含义更明确易懂。

表 2.5.5 Verilog HDL 运算符的优先级

类型	符号	优先级别
取反	! ~ -(求 2 的补码)	最高优先级
算术	* / + -	
移位	>> << >>> <<<	
关系	< <= > >=	
等于	== ! = === ! ==	
缩位	& ~& ^ ^~ | ~|	
逻辑	&& ||	
条件	? :	最低优先级

2.5.6 Verilog HDL 内部的基本门级元件

Verilog 语言包含 12 个预先定义的基本门级元件[①](gate level primitives)模型,如表 2.5.6 所示。在门级建模时,可以直接调用这些基本门级元件将逻辑图转换成 Verilog 代码。

注意,门级元件均用小写的关键词来表示,当使用这些元件进行逻辑仿真时,仿真软件会根据程序的描述给每个元件中的信号分配逻辑 0、逻辑 1、不确定态 x 和高阻态 z 这 4 个值之一。下面介绍这些元件的用法。

表 2.5.6 Verilog 语言内置的 12 个基本门级元件

元件符号	功能说明	元件符号	功能说明
and	多输入端的与门	**nand**	多输入端的与非门
or	多输入端的或门	**nor**	多输入端的或非门
xor	多输入端的异或门	**xnor**	多输入端的异或非门
buf	多输出端的缓冲器	**not**	多输出端的反相器
bufif1	控制信号高电平有效的三态缓冲器	**notif1**	控制信号高电平有效的三态反相器
bufif0	控制信号低电平有效的三态缓冲器	**notif0**	控制信号低电平有效的三态反相器

① 有的文献翻译成门级原语。

1. 多输入门

and、**nand**、**or**、**nor**、**xor** 和 **xnor** 是具有多个输入的逻辑门，它们的共同特点是：只允许有一个输出，但可以有多个输入。

与门元件(**and**)模型的示意图如图 2.5.10 所示，逻辑真值表如表 2.5.7 所示，两个输入分别列于表中行的顶部和列的左侧，行、列输入的运算结果列在表的中间部位。注意，只有当行、列输入都是 1 时，行列交叉处的输出才为 1；而任意一个输入为 0，输出就为 0；另外，如果有一个输入为 x 或者 z，则输出为 x。多输入门的输出不可能为高阻态 z。

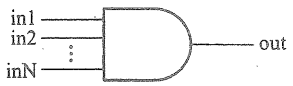


图 2.5.10 多输入与门元件模型

表 2.5.7 两输入 and 真值表

and		输入 1			
		0	1	x	z
输入 2	0	0	0	0	0
	1	0	1	x	x
	x	0	x	x	x
	z	0	x	x	x

与门元件的一般调用形式为：

```
and  A1(out,in1,in2,…,inN);
```

其中，实例名[①] A1 可以省略。在后面的圆括号中列出该门的端口，输出列在前，输入列在后，并用逗号分开。

nand、**or**、**nor**、**xor**、**xnor** 的调用形式与之类似，不再赘述。表 2.5.8 和表 2.5.9 是两输入端 **or** 和 **xor** 元件的逻辑真值表。

表 2.5.8 两输入 or 真值表

or		输入 1			
		0	1	x	z
输入 2	0	0	1	x	x
	1	1	1	1	1
	x	x	1	x	x
	z	x	1	x	x

① 是英文 Instance name 的译称。

表 2.5.9 两输入 xor 真值表

xor		输入 1			
		0	1	x	z
输入 2	0	0	1	x	x
	1	1	0	x	x
	x	x	x	x	x
	z	x	x	x	x

2. 多输出门

buf、**not** 是具有多个输出端的逻辑门，它们的共同特点是：允许有多个输出，但只有一个输入。一般的调用形式为：

```
buf  B1(out1,out2,…,outN,in);
not  N1(out1,out2,…,outN,in);
```

其中，实例名 B1、N1 可以省略。图 2.5.11 所示为多个输出端反相器的示意图，其逻辑真值表如表 2.5.10 所示。**buf** 逻辑真值表与 **not** 的类似，但其输出的 **0**、**1** 值与 **not** 的相反。

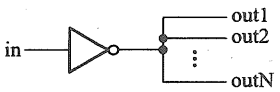


图 2.5.11 门级元件 not 的模型

表 2.5.10 not 真值表

not	输入			
	0	1	x	z
输出	1	0	x	x

3. 三态门①

bufif1②、**bufif0**、**notif1** 和 **notif0** 是三态门元件模型，这些门有一个输出、一个数据输入和一个控制输入。如果输入控制信号无效，则三态门的输出为高阻态 z。一般的调用形式为：

```
bufif1  B1(out,in,ctrl); bufif0  B0(out,in,ctrl);
notif1  N1(out,in,ctrl); notif0  N0(out,in,ctrl);
```

其中，实例名 B1、B0、N1、N0 可以省略。

图 2.5.12 所示是 **bufif1** 和 **notif1** 的示意图。

图 2.5.12 三态门元件模型

(a) bufif1 (b) notif1

① 本书第 3 章将介绍三态门的有关内容。

② 该关键词可分两部分理解，buf 是 buffer 的缩写，表示该元件完成缓冲器的功能；后面的 if1 是如果为 1，表示完成该功能所需的条件。其他 3 个关键词的理解与此类似。

表 2.5.11 和表 2.5.12 给出了 **bufif1** 和 **notif1** 的逻辑真值表，表中，0/z 表明三态门的输出可能是 0，也可能是高阻态 z，主要由输入数据信号和控制信号的强度决定。有关信号强度的讨论超出了本书的范畴。

表 2.5.11 bufif1 真值表

bufif1		控制输入			
		0	1	x	z
数据输入	0	z	0	0/z	0/z
	1	z	1	1/z	1/z
	x	z	x	x	x
	z	z	x	x	x

表 2.5.12 notif1 真值表

notif1		控制输入			
		0	1	x	z
数据输入	0	z	1	1/z	1/z
	1	z	0	0/z	0/z
	x	z	x	x	x
	z	z	x	x	x

小 结

- 逻辑代数是分析和设计逻辑电路的数学工具。一个逻辑问题可用逻辑函数来描述，逻辑函数可用真值表、逻辑表达式、卡诺图和逻辑图表达，这四种表示方法各具特点，可根据需要选用。
- 逻辑函数的**与-或**表达式和**或-与**表达式是两种常见的形式，但其表达式的形式不是唯一的。任一个逻辑函数经过变换，都能得到唯一的最小项表达式或者是最大项表达式。
- 硬件描述语言(HDL)是一种以文本形式来描述数字系统硬件的结构和行为的语言，可以用来描述逻辑电路图、逻辑表达式，还可以描述更复杂的数字逻辑系统所完成的逻辑功能(即行为)。计算机对 HDL 的处理包括两个方面：逻辑仿真和逻辑综合。
- 利用先进的 EDA 软件工具和 HDL 设计数字系统已经成为当今主流设计方法。使用 HDL 描述的逻辑电路可以直接被编程到可编程逻辑器件中。
- 模块(module)是 Verilog 描述电路的基本单元，通常可以使用一个或多个模块来描述逻辑电路的功能，用来描述电路逻辑功能的模块称为**设计块**。
- 激励块是用来给设计块提供激励信号的 Verilog 模块，目的是测试设计块所描述的逻辑

功能是否正确。

习 题

2.1 逻辑代数的基本定律和规则

2.1.1 用真值表证明下列恒等式：

(1) $(A\oplus B)C=A\oplus(B\oplus C)$　　(2) $\overline{A\oplus B}=\overline{A}\,\overline{B}+AB$

2.1.2 用逻辑代数定律证明下列等式：

(1) $A+\overline{A}B=A+B$　　(2) $(A+B)(A+\overline{B})=A$

(3) $AB+\overline{A}C+BCD=AB+\overline{A}C$　　(4) $(A+B)(B+C)(\overline{A}+C)=(A+B)(\overline{A}+C)$

2.1.3 应用反演规则和对偶规则，求下列函数的非函数和对偶函数(用与-或形式表示)。

(1) $L=A\cdot B+\overline{A}\cdot\overline{B}$　　(2) $L=\overline{A}\cdot\overline{B}+\overline{\overline{A}\cdot B\cdot\overline{C}}\cdot D$

2.1.4 某逻辑函数的对偶式 $L'=\overline{\overline{A}B}\cdot(\overline{B\overline{C}}+\overline{D})$，则该逻辑函数 $L=$ _____，其反演式 $\overline{L}=$ _____。

2.2 逻辑函数表达式的形式

2.2.1 将下列各式转换成与-或形式：

(1) $\overline{A\oplus B}\oplus\overline{C\oplus D}$　　(2) $\overline{\overline{A+B}+\overline{C+D}}+\overline{\overline{C+D}+\overline{A+D}}$

(3) $\overline{\overline{AC}\cdot\overline{BD}\,\overline{BC}\cdot\overline{AB}}$

2.2.2 列出逻辑函数 $L(A,B,C)=A\overline{B}+B\overline{C}$ 的真值表，并写出该函数的最小项表达式。

2.2.3 试写出下列各个函数的最小项表达式。

(1) $L=A\overline{C}D+\overline{B}C\,\overline{D}+ABCD$　　(2) $L=\overline{\overline{A}(B+\overline{C})}$

(3) $L=\overline{\overline{AB}+ABD}(B+\overline{C}D)$　　(4) $L=\overline{(A\overline{B}+B\overline{C})\overline{AB}}$

2.2.4 判断下列等式是否成立？

(1) $\overline{A}C+AB\overline{C}+\overline{A}B+A\overline{B}=\overline{B}C+A\overline{C}+B\overline{C}+\overline{A}BC$

(2) $(A+C)(\overline{A}+\overline{B}+\overline{C})(\overline{A}+B)=(A+B)(B+C)(\overline{A}+\overline{C})$

2.2.5 列出逻辑函数 $L(A,B,C)=\overline{A}\cdot C+A\cdot\overline{B}\cdot\overline{C}$ 的真值表，并写出该函数的最大项表达式。

2.2.6 已知下列函数的最大项表达式，试写出其最小项表达式。

(1) $L(A,B)=\prod M(0,1,2)$　　(2) $L(A,B,C)=\prod M(0,1,3,6,7)$

2.2.7 已知下列函数的最小项表达式，试写出其最大项表达式。

(1) $L(A,B)=\sum m(1,2)$　　(2) $L(A,B,C,D)=\sum m(1,2,5,6)$

2.3 逻辑函数的代数化简法

2.3.1 用代数法将下列各式化简成最简的与-或表达式：

(1) $\overline{\overline{(\overline{A}+B)}+\overline{(A+B)}+\overline{(\overline{AB})}\,\overline{(A\overline{B})}}$　　(2) $\overline{B}+ABC+\overline{AC}+\overline{AB}$

(3) $ABC\overline{D}+ABD+BC\overline{D}+ABCD+B\overline{C}$　　(4) $\overline{\overline{AC+\overline{A}BC}+\overline{B}C+AB\overline{C}}$

2.3.2 已知逻辑函数表达式为 $L=\overline{A}BC\overline{D}$，画出实现该式的逻辑电路图，限使用非门和 2 输

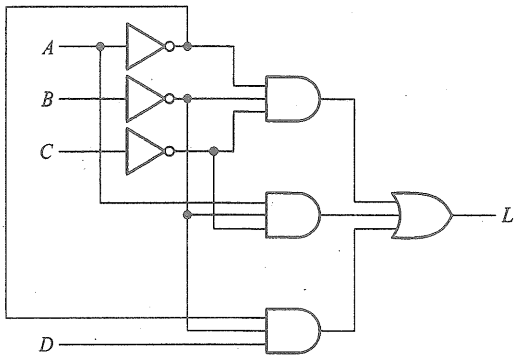
入与非门。

2.3.3 画出实现下列逻辑表达式的逻辑电路图，限使用非门和2输入与非门。

(1) $L=AB+AC$ (2) $L=\overline{D(A+C)}$ (3) $L=\overline{(A+B)(C+D)}$

2.3.4 已知逻辑函数表达式为 $L=A\overline{B}+\overline{A}C$，画出实现该式的逻辑电路图，限使用非门和2输入或非门。

2.3.5 写出图题2.3.5所示电路的逻辑表达式，将其转换成与非-与非表达式，然后画出仅用非门和与非门实现的逻辑图。



图题2.3.5

2.4 逻辑函数的卡诺图化简法

2.4.1 已知逻辑函数 $L(A,B,C)=A\overline{B}+B\overline{C}+C\overline{A}$，试用真值表、卡诺图和逻辑图(限用非门和与非门)表示。

2.4.2 已知函数 $L(A,B,C,D)$ 的卡诺图如图题2.4.2所示，试写出函数 L 的最简与-或表达式。

L / AB \ CD	00	01	11	10
00		1		1
01	1			
11	1			
10		1		1

图题2.4.2

2.4.3 用卡诺图法化简下列各式。

(1) $A\overline{B}CD+AB\overline{C}D+A\overline{B}+A\overline{D}+A\overline{B}C$

(2) $A\overline{B}CD+D(\overline{B}\,\overline{C}D)+(A+C)B\overline{D}+\overline{A}\overline{(\overline{B}+C)}$

(3) $L(A,B,C,D)=\sum m(0,2,4,8,10,12)$

(4) $L(A,B,C,D)=\sum m(0,4,6,13,14,15)+\sum d(1,2,3,5,7,9,10,11)$

2.4.4 用卡诺图化简法，求下列函数的最简或-与表达式。

(1) $L(A,B,C,D)=A\overline{C}+AD+\overline{B}\,\overline{C}+\overline{B}D$

(2) $L(A,B,C,D)=\sum m(3,4,5,7,13,14,15)$

(3) $L(A,B,C,D)=(\overline{A}+D)(B+\overline{D})(A+B)$

(4) $L(A,B,C,D)=\prod M(3,4,6,7,11,12,13,14,15)$

2.5 硬件描述语言 Verilog HDL 基础

2.5.1 在 Verilog 语言中，下列标识符是否正确？

(1) system1 (2) 2reg (3) FourBit_Adder (4) exec$ (5) _2to1mux

2.5.2 Verilog 语言规定的4种基本逻辑值是什么？

2.5.3 在 Verilog 程序中，如果没有说明输入、输出信号的数据类型，试问它们的数据类型是什么？

2.5.4 填空题：

(1) 问下列运算的二进制值是多少？

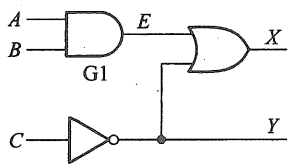
```
reg[3:0] m;
m=4'b1010;  //{2{m}}的二进制值是_____;
```

(2) 假设 m=4'b0101，按要求填写下列运算的结果。

&m=_____，　　|m=_____，

^m=_____，　　~^m=_____。

2.5.5 下列 Verilog 模块描述了图题 2.5.5 所示的电路，但程序中每一行有一个语法错误，请改正。(注意：基本门级元件的实例名可以省略。)



图题 2.5.5

```
module Ex1(A,B,C,X,Y)
  input A,B,C
  output X,Y
  reg E;
  and G1(A,B,E);
  NOT(Y,C);
  OR(X,E,Y);
endmodule;
```

2.5.6 根据下面的 Verilog HDL 描述，画出该模块的逻辑电路图。

```
module circuit(A,B,L);
  input A,B;
  output L;
  wire a1,a2,Anot,Bnot;
  and G1(a1,A,B);
  and G2(a2,Anot,Bnot);
  not(Anot,A);
  not(Bnot,B);
  or(L,a1,a2);
endmodule
```

2.5.7 使用 Verilog 连续赋值语句描述下列逻辑函数表示的电路。

(1) $Sum=\overline{A}B+A\overline{B}$，$C_0=AB$

(2) $Y_0=\overline{S}_0D_0+S_0D_1$，$Y_1=\overline{S}_0D_2+S_0D_3$

2.5.8 写出图题 2.3.5 所示电路的 Verilog 门级描述代码。

2.5.9 假设 A=4'b1010，B=4'b1101，执行下列运算后，Ynot、Yand、Yor、Yxor 和 Yxnor 的结果是多少？

```
module Bitwise(
        input[3:0] A,B,
```

```
        output [3:0] Ynot, Yand, Yor, Yxor, Yxnor
    );
        assign   Ynot = ~A;
        assign   Yand = A & B;
        assign   Yor = A | B;
        assign  Yxor = A ^ B;
        assign  Yxnor = A ~^ B;
    endmodule
```

2.5.10 假设 A = 4'b1010,执行下列运算后,red_and、red_nand、red_or、red_nor、red_xor 和 red_xnor 的结果是多少?

```
    module Reduction(
        input [3:0] A,
        output red_and, red_nand, red_or, red_nor, red_xor, red_xnor
        );
        assign   red_and = &A,
                 red_nand = ~&A,
                 red_or = |A,
                 red_nor = ~|A,
                 red_xor = ^A,
                 red_xnor = ^~A;
    endmodule
```

文档 2.4:
第 2 章部分
习题答案

3 逻辑门电路

引言

文档 3.1：第 3 章思维导图

在第 1 章我们介绍了与、或、非三种基本逻辑运算，并介绍了逻辑变量与逻辑函数的关系。对于逻辑门只给出了逻辑符号，没有涉及内部的结构和原理。

逻辑门电路是数字电路的基本逻辑单元。本章重点讨论 CMOS 门电路，简单介绍 NMOS 门电路、BiCMOS 门电路及双极结型三极管（biploar junction transistor，BJT）构成的 TTL 门电路。门电路中的 MOS 管或 BJT 管工作在开关状态。对于 CMOS 门电路，首先介绍 MOS 管的开关特性，然后介绍由它们构成的基本逻辑门的电路结构和工作原理，着重阐述其逻辑功能和外部输入特性、输出特性，以及其他电气特性，以便正确使用这些门电路。

3.1 逻辑门电路简介

3.1.1 各种逻辑门电路系列简介

实现基本逻辑运算和常用逻辑运算的单元电路称为逻辑门电路，简称门电路。逻辑门电路是组成各种数字电路的基本单元电路。将构成门电路的元器件制作在一块半导体芯片上，再封装起来，便构成了集成门电路。

按照制造门电路三极管的不同，分为 MOS 型、双极型和混合型。MOS 型集成逻辑门有 CMOS、NMOS、PMOS，双极型集成逻辑门主要有 TTL，混合型集成逻辑门有 BiCMOS。

CMOS 逻辑门电路是目前使用最广泛、占主导地位的集成电路。与 TTL 逻辑门相比，早期的 CMOS 电路速度慢、功耗低。后来随着制造工艺的不断改进，CMOS 电路的集成度、工作速度、功耗和抗干扰能力远优于 TTL。因此，几乎所有的 CPU、存储器、PLD 器件和专用集成电路（ASIC）都采用 CMOS 工艺制造，且费用较低。因此，出现种类繁多的 CMOS 逻辑系列。

早期生产的 CMOS 门电路为 4000 系列，其工作速度较慢，与 TTL 不兼容，但功耗低、工作电压范围宽、抗干扰能力强。随后出现了高速 CMOS 器件 HC/HCT（high-speed CMOS/high-speed CMOS，TTL compatible）系列。与 4000 系列相比，其工作速度快、带负载能力强。

HCT 系列与 TTL 兼容，可与 TTL 器件互换使用。另一种 CMOS 系列是 AHC/AHCT（advanced high-speed CMOS/ advanced high-speed CMOS，TTL compatible）系列，其工作速度达到 HC/HCT 系列两倍之多。

后来，随着便携式设备（如笔记本电脑、数码相机、手机等）的发展，要求使用体积小、重量轻、功耗低的半导体器件，因此先后推出了低电压 CMOS 器件 LVC（low-voltage CMOS）系列，速度和性能比 LVC 更好的 ALVC（advanced low-voltage CMOS）系列，超低电压 AUC（advanced ultra-low-voltage CMOS）系列，以及低功耗 AUP（advanced ultra-low-power CMOS）系列，并且半导体制造工艺的进步使它们的成本更低、速度更快。不同的 CMOS 系列器件对电源电压要求不一样，表 3.1.1 列出几种 CMOS 集成电路的电源电压范围、典型值和速度特点。

中小规模集成电路芯片的名称以 54 或 74 开始，后加不同系列缩写字母及数字表示，如 54/74HC00。中间字母表示不同系列，如 HC 系列。最后的数字表示不同逻辑功能芯片的编号，如 00 表示 4 个 2 输入与非门，即一个芯片中封装了 4 个与非门，如图 3.1.1 所示。图 3.1.1（a）所示为双列直插（dual in-line package，DIP）封装的芯片，图 3.1.1（b）所示为 74HC00 引脚排列图。54 和 74 系列的区别是 54 系列适用的温度范围更宽，测试和筛选标准更严格。

表 3.1.1 几种 CMOS 电路的电源电压值及速度特点

参数、特点	类型					
	4000	74HC/ HCT	74AHC/AHCT	74LVC	74ALVC	74AUC
电源电压范围/V	3~18	2~6/4.5~5.5	2~5.5/4.5~5.5	1.65~5.5	1.65~3.6	0.8~2.7
典型值/V	5	5	5	3.3	2.5	1.8
速度特点	速度慢与 TTL 不兼容	速度快与 TTL 兼容	速度是 HC 的两倍与 TTL 兼容	速度比 AHC 快	速度是 LVC 的两倍	速度更快

CMOS 是数字逻辑电路的主流工艺技术，但 CMOS 技术却不适合用在射频电路中。因此 BiMOS 成为射频系统中用得最多的工艺技术。BiCMOS 集成电路是将 BJT 的高速性和高驱动能力，以及 CMOS 的高密度、低功耗和低成本等优点结合起来，在高速、大电流驱动场合，例如大规模集成电路的接口，可以采用 BiCMOS 缓冲器、驱动器、锁存器或触发器等驱动。性能优于硅的新型材料 SiGe 大多数都是 BiMOS 工艺，主要用于通信领域射频前端，如手机、蓝牙技术、高速光通信等设备，而且 BiMOS 技术既可用于数字集成电路，也可用于模拟集成电路。

TTL 是应用最早，技术比较成熟的集成电路，曾被广泛使用。大规模集成电路的发展，要求每个逻辑单元电路的结构简单，并且功耗低。TTL 电路不满足这个条件，因此逐渐被 CMOS 电路所取代，退出其主导地位。

对于 TTL 门电路，最早是 74 系列。后来随着集成电路的发展，工作速度和功耗逐步改善，先后推出 74S、74LS、74AS 和 74ALS 等系列。由于 TTL 技术在整个数字集成电路设计领

域中的历史地位和影响，目前主要应用于教育或是简单的中小规模数字电路。

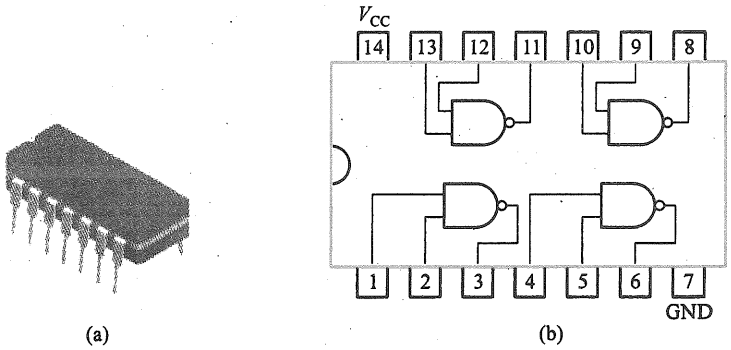


图 3.1.1 2 输入与非门 74HC00

（a）双列直插封装 （b）引脚排列图

3.1.2 开关电路

在二值数字逻辑中，逻辑变量的取值不是 0 就是 1。在数字电路中，与其对应的是电子器件的“闭合”和“断开”两种状态。图 3.1.2 所示为开关电路示意图。当开关 S 断开时，输出电压 $v_O = V_{CC}$，输出逻辑 1，如图 3.1.2(a)所示。反之，当开关 S 接通时，输出电压 $v_O = 0$，输出逻辑 0，如图 3.1.2(b)所示。

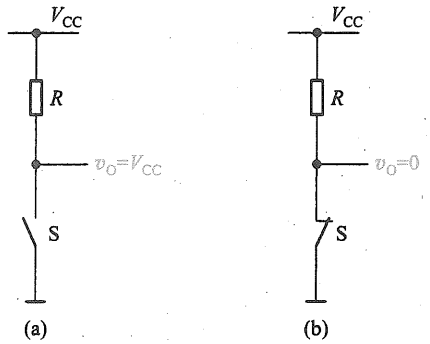


图 3.1.2 开关电路示意图

（a）输出逻辑 1 （b）输出逻辑 0

早期的开关由继电器构成，后来使用 BJT 或 MOS 管作为开关。BJT 或 MOS 管相当于一个受控开关，当其工作在截止状态时，相当于开关断开，输出高电平；当其工作在饱和导通状态时，相当于开关闭合，输出低电平。

复习思考题

3.1.1 CMOS 逻辑门电路有哪些优点？

3.1.2 74HC 与 74HCT 系列的区别是什么？

3.1.3 现在的 CPU、PLD 和 ASIC 芯片主要采用什么工艺制造？

线基本重合，可以用一条曲线来表示。这条曲线与横坐标的交点即为阈值电压 V_T。

3. 其他类型的 MOS 管

（1）P 沟道增强型 MOS 管

与 N 沟道 MOS 管相反，P 沟道 MOS 管是在 N 型衬底上制作两个高浓度的 P 区，导电沟道为 P 型，载流子为空穴。其符号如图 3.2.5 所示。通常将衬底与源极相连，或接电源。为吸引空穴形成导电沟道，栅极接电源负极，与衬底相连的源极接电源的正极，即 v_{GS} 为负值，因此阈值电压 V_T 也为负值。而 i_D 的实际方向为流出漏极，与通常的假定方向正好相反。图 3.2.6 所示为 P 沟道增强型 MOS 管的输出特性和转移特性曲线。

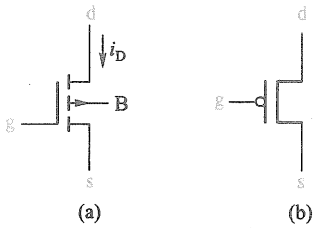


图 3.2.5 P 沟道增强型 MOS 管符号

（a）标准符号 （b）简化符号

对于增强型 PMOS 管，当 $v_{GS}>V_T$ 时，PMOS 管处于截止区；当 $v_{GS}\leqslant V_T$、$v_{DS}>v_{GS}-V_T$ 时，PMOS 管工作在可变电阻区；当 $v_{GS}\leqslant V_T$、$v_{DS}\leqslant v_{GS}-V_T$ 时，PMOS 管工作在恒流区。

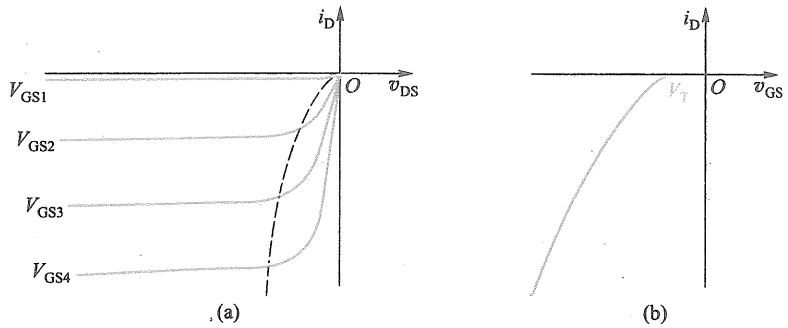


图 3.2.6 P 沟道增强型 MOS 管的特性曲线

（a）输出特性曲线 （b）转移特性曲线

（2）N 沟道耗尽型 MOS 管

N 沟道耗尽型 MOS 管的结构与增强型基本相同，最大的区别是在生产过程中，在 SiO_2 绝缘层中掺入大量正离子，其结构示意图与符号如图 3.2.7 所示。$v_{GS}=0$ 时，由于正离子的作用，将电子吸引到栅极下面的衬底表面形成 N 型沟道。当 $v_{GS}>0$ 时，沟道变宽。在 v_{DS} 作用下，i_D 的数值更大。v_{GS} 为负，沟道变窄，i_D 减小，当 v_{GS} 达到某一负的阈值电压 V_T 时，沟道完全被夹断，即使加 v_{DS}，也不会有漏极电流 i_D，MOS 管截止。

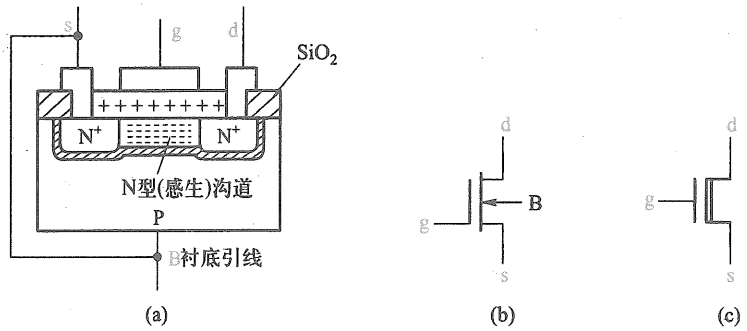


图 3.2.7 N 沟道耗尽型 MOS 管

（a）结构示意图 （b）标准符号 （c）简化符号

(3) P 沟道耗尽型 MOS 管

P 沟道耗尽型 MOS 管的结构与增强型基本相同,但在 SiO_2 绝缘层中掺入大量负离子,形成 P 型导电沟道,其阈值电压 V_T 为正值。v_{GS} 可以是负值、零或正值。v_{GS} 为负值时沟道变宽,i_D 数值变大;v_{GS} 为正值时沟道变窄,i_D 数值变小。当 v_{GS} 达到 V_T 时,沟道完全被夹断,MOS 管截止。P 沟道耗尽型 MOS 管的符号如图 3.2.8 所示。

4. MOS 管开关电路

用增强型 NMOS 管替代图 3.1.2 所示的开关 S 构成的电路如图 3.2.9 所示。MOS 管的作用对应于有触点开关 S 的“断开”和“闭合”,但在速度和可靠性方面比机械开关优越得多。

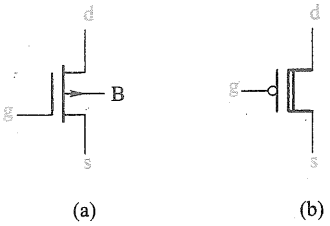


图 3.2.8 P 沟道耗尽型 MOS 管符号

(a) 标准符号 (b) 简化符号

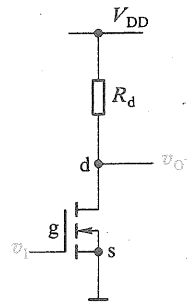


图 3.2.9 MOS 管开关电路

当 $v_I<V_T$ 时,MOS 管处于截止状态,输出电压 $v_O=V_{DD}$。此时器件不消耗功率。

当 $v_I>V_T$,并且足够大时,使得 MOS 管工作在可变电阻区。d、s 之间的导通电阻 R_{on} 可由式(3.2.1)确定。由于 v_{GS} 足够大,所以 R_{on} 很小,使得 R_d 远远大于 R_{on},电路输出为低电平。

由此可见,MOS 管相当于一个由 v_{GS} 控制的无触点开关。当输入为低电平时,MOS 管截止,其等效电阻约为 1 MΩ 以上,相当于开关“断开”,输出为高电平,其等效电路如图 3.2.10(a)所示;当输入为高电平时,MOS 管工作在可变电阻区,MOS 管导通时等效电阻 R_{on} 约在 1 kΩ 以内。不同 CMOS 集成系列的 MOS 管导通电阻不同,有的可以达到 10 Ω,甚至更低。

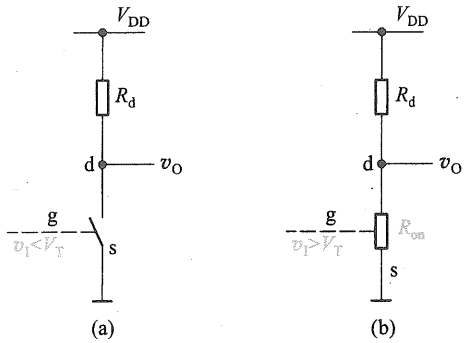


图 3.2.10 MOS 管的开关等效电路

(a) 截止时的等效电路 (b) 导通时的等效电路

5. MOS 管开关电路的动态特性

在图 3.2.9 所示 MOS 管开关电路的输入端，加一个理想的脉冲波形，如图 3.2.11(a)所示。由于 MOS 管中栅极与衬底间电容 C_{gb}(即数据手册中的输入电容 C_i)、漏极与衬底间电容 C_{db}、栅极与漏极电容 C_{gd}以及导通电阻等的存在，使其在导通和闭合两种状态之间转换时，不可避免地受到电容充、放电过程的影响。输出电压 v_O的波形已不是与输入一样的理想脉冲，如图 3.2.11(b)所示。上升沿和下降沿的变化都变得缓慢了，而且输出 v_O的变化滞后于输入 v_I的变化，t_{pHL}为输出 v_O由高电平跳变为低电平的传输延迟时间，t_{pLH}为输出 v_O由低电平跳变为高电平的传输延迟时间。

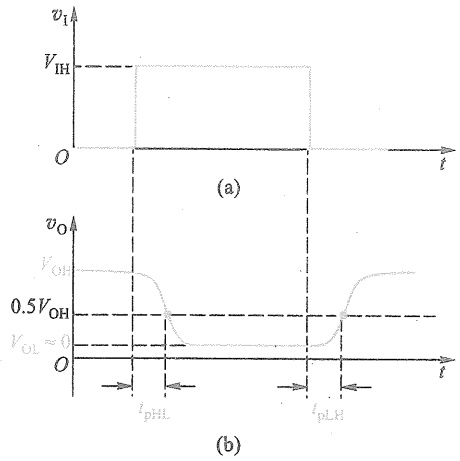


图 3.2.11 MOS 管开关电路电压波形
(a) 输入电压波形 (b) 输出电压波形

图 3.2.9 所示电路中 R_d的作用是：当输入为高电平时，流过导通 NMOS 管的电流很大，R_d起限流作用(通常为千欧级)，但此时其上消耗的功率也很大。为了克服这个缺点，用另一个 PMOS 管替代电阻 R_d，就构成了 CMOS 反相器。

3.2.2 CMOS 反相器

由 N 沟道和 P 沟道增强型 MOS 管组成的电路称为互补 MOS 或 CMOS 电路。CMOS 反相器是构成 CMOS 逻辑电路的基本单元电路之一，另一个基本单元电路——传输门将在3.2.4节介绍。下面讨论 CMOS 反相器的工作原理。

CMOS 反相器电路如图 3.2.12 所示，由两只增强型 MOS 管组成，其中 T_N为 NMOS 管，T_P为 PMOS 管。两只 MOS 管的栅极连在一起作为输入端；它们的漏极连在一起作为输出端。按照图中标明的电压与电流方向，$v_I = v_{GSN}$，$v_O = v_{DSN}$，并设 $i_{DN} = i_{DP} = i_D$。NMOS 管和 PMOS 管的阈值电压分别用 V_{TN}和 V_{TP}表示。为了让电路正常工作，要求电源电压 V_{DD}大于两 MOS 管阈值电压的绝对值之和，即 $V_{DD} > (|V_{TN}| + |V_{TP}|)$。

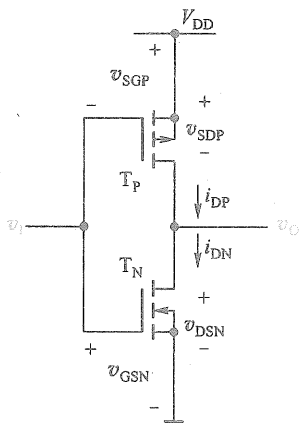


图 3.2.12 CMOS 反相器

1. 工作原理

首先，我们考虑两种极限情况：当 v_I 处于逻辑 0 时，相应的电压近似为 0 V；而当 v_I 处于逻辑 1 时，相应的电压近似为 V_{DD}。

当输入为高电平 $v_I = V_{DD}$ 时，根据图 3.2.12 所示电路图可知，$v_{GSN} = V_{DD} > V_{TN}$，$T_N$ 管导通，并且导通电阻很低，通常为十欧以下；而 $v_{SGP} = 0 < |V_{TP}|$，T_P 管截止，等效电阻很高，可达兆欧级以上。因此，反相器输出为低电平，输出电压 $v_O = V_{OL} \approx 0$，而通过两管的电流接近于零。电路的功耗极低。

当输入为低电平 $v_I = 0$，即 $v_{GSN} = 0 < V_{TN}$，T_N 管截止；而 $v_{SGP} = V_{DD} > |V_{TP}|$，$T_P$ 管导通。反相器输出为高电平，输出电压 $v_O = V_{OH} \approx V_{DD}$。

由此可知，当输入 $v_I = V_{DD}$ 时，输出 $v_O \approx 0$；而输入 $v_I = 0$，输出 $v_O \approx V_{DD}$。输出与输入之间为逻辑**非**的关系，也称**非**门为反相器(inverter)。CMOS 反相器近似于一个理想的逻辑单元，其输出电压接近于零或 V_{DD}。

通过上述分析可以看出 CMOS 反相器具有以下几个重要特点。

第一，当反相器处于稳态时，无论输入 v_I 是高电平还是低电平，T_N 和 T_P 中总是一个处于导通而另一个处于截止的状态。截止管的等效电阻很大，流过 T_N 和 T_P 的静态电流非常小。因此 CMOS 反相器的静态功耗非常低，几乎为零。这是 CMOS 电路最显著的优点。

第二，MOS 管导通电阻很低，而截止电阻很高。当反相器输出端接电容性负载，其输出端由低电平变为高电平时，T_P 导通，电源通过其导通电阻给负载电容提供快速充电回路。反相器输出电平由高变低时，T_N 导通，负载电容通过其较小的导通电阻快速放电。与图 3.2.9 所示的电路相比较，CMOS 反相器的开关速度更快，具有更强的带负载能力。

第三，MOS 管栅极是绝缘的，$I_G \approx 0$，反相器的输入电阻非常高。因此，理论上，反相器可以驱动任意数目的同类门而不会对输出电平造成影响。但是，负载门输入端具有栅极电容和杂散电容，负载门数目的增加将使反相器的负载电容增加，从而影响反相器的开关速度。

2. 电压传输特性和电流传输特性

CMOS 反相器的电压传输特性是指其输出电压 v_O 随输入电压 v_I 变化的关系曲线，如图 3.2.13(a)所示。电流传输特性是指漏极电流 i_D 随输入电压 v_I 变化的曲线，如图 3.2.13(b)所示。根据 T_N 和 T_P 两管工作情况的不同，可将传输特性曲线分为五段。

在传输特性曲线的 *AB* 或 *EF* 段，根据前述 CMOS 反相器工作原理的两种极限情况分析可知，不论输出为高电平或低电平，总有一只 MOS 管工作在截止区，因此，流过两管的电流接近零值。

在 *BC* 或 *DE* 段，T_N 和 T_P 两管中，总有一个工作在恒流区，另一个工作在可变电阻区，此时两管中的电流比较大，传输特性变化比较快，两管在 $v_I = V_{DD}/2$ 处转换状态。因此，将状态转换处的电压定义为 CMOS 反相器的阈值电压 V_{TH}①，$V_{TH} = V_{DD}/2$。

在 *CD* 段，由于 T_N 和 T_P 两管均工作在恒流区，此时 $v_I = V_{DD}/2$，电流 i_D 达到最大值。

从 *B* 到 *E* 之间，即 $V_{TN} < v_I < V_{DD} - |V_{TP}|$ 时，T_N 和 T_P 处于同时导通的过渡区域，传输特性变化急剧，产生一个较大的电流尖峰。因而导致有较大的功耗。使用时应避免使两管长时间

① V_{TH} 系 threshold voltage 之意。

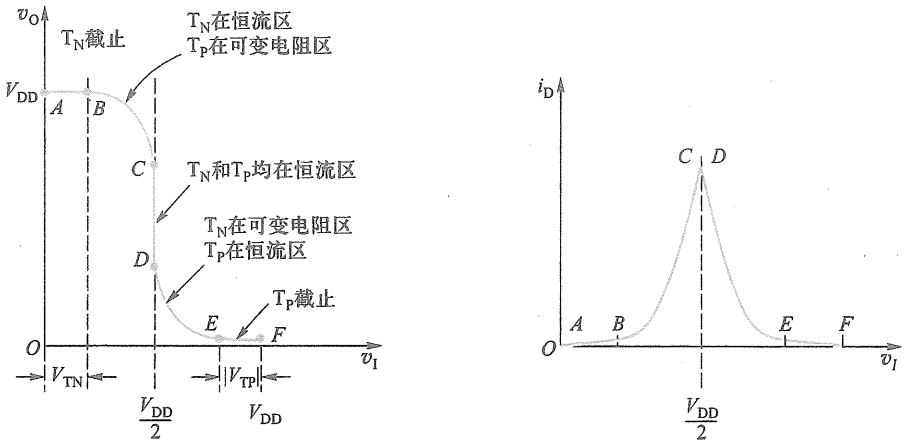


图 3.2.13 CMOS 反相器的传输特性

(a) 电压传输特性 (b) 电流传输特性

工作在此区域,以防止功耗过大而损坏。

3. 输入逻辑电平

由图 3.2.13(a)可见,当输入电压从 0 V 开始逐渐增加时,输出高电平维持一段时间没有改变。同样,当输入电压由 V_{DD} 开始降低时,输出低电平也维持一段时间没有改变。因此,在反相器的输出逻辑状态没有发生明显改变时,输入高、低电平值允许有一个波动范围,如图 3.2.14(a)所示。对于不同系列的集成电路,输入高、低电平所对应的电压范围也不同。因此,各种集成门电路都规定了输入低电平的上限值 $V_{IL(max)}$ 和输入高电平的下限值 $V_{IH(min)}$。

以 74LVC 系列 CMOS 逻辑电路为例,在 3.3 V 工作电源条件下,$V_{IL(max)}$ = 0.8 V,$V_{IH(min)}$ = 2.0 V,即 v_I 在 0~0.8 V 范围内,v_O 为高电平;v_I 在 2.0~3.3 V 范围内,v_O 为低电平。

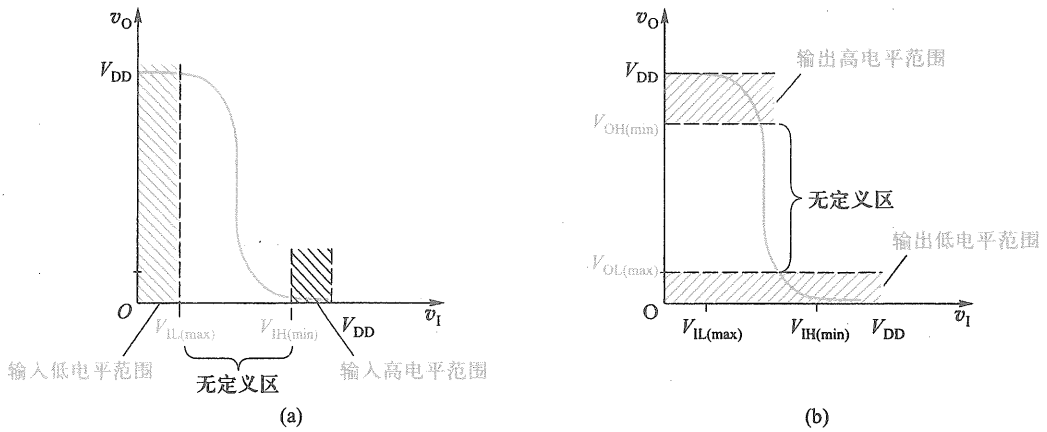


图 3.2.14 门电路输入和输出逻辑电平范围

(a) 输入高、低电平 (b) 输出高、低电平

4. 输出逻辑电平

对于图 3.2.12 所示的 CMOS 反相器，输出高、低电平值也允许有一个波动范围，如图 3.2.14(b)所示。当输入为高电平时，则 $v_{GSN}=v_I=V_{IH}$，T_N 管导通，T_P 管截止，输出为低电平，即 $v_O=V_{OL}$，等效电路如图 3.2.15(a)所示。图中 R_L 为 CMOS 反相器所带负载的等效电阻。此时负载电流 I_{OL} 从负载流向 T_N 管，称为灌电流。

当 v_I 足够大时，T_N 管工作在可变电阻区($v_{DS}<v_{GS}-V_T$)，参见图 3.2.4(a)。此时 T_N 管的导通电阻 $R_{on(N)}$ 与 v_{GSN} 的关系可由式(3.2.1)确定。

需要特别注意，由于 $V_{OL}=R_{on(N)}I_{OL}$，所以 V_{OL} 的值与 T_N 管导通电阻 $R_{on(N)}$ 和负载电流 I_{OL} 两个因素有关。若 $R_{on(N)}$ 不变，V_{OL} 随着 I_{OL} 增加(由 R_L 引起)而升高。若 I_{OL} 不变，由式(3.2.1)可知，当 v_I 的高电平值越低，v_{GS} 越小，则 $R_{on(N)}$ 越大，导致 V_{OL} 也越高。因此，集成电路数据手册给出了在不同 I_{OL} 条件下，输出低电平的值 V_{OL}。由于器件参数的分散性，即使在相同条件下，多个同型号器件的输出低电平的值也略有区别，但它们的 V_{OL} 值不超过 $V_{OL(max)}$。例如 CMOS 系列与非门 74LVC00 在 3.3 V 工作电源条件下，当 $I_{OL}=100\ \mu A$ 时，$V_{OL(max)}=0.1$ V。

同理，当输入为低电平时，T_N 管截止，T_P 管导通，输出为高电平，即 $v_O=V_{OH}$，等效电路如图 3.2.15(b)所示。R_L 为 CMOS 反相器所带负载的等效电阻，$R_{on(P)}$ 为 T_P 管的导通电阻。此时负载电流 I_{OH} 从 T_P 管流向负载，称为拉电流。

V_{OH} 的值与 T_P 管导通电阻 $R_{on(P)}$ 和负载电流 I_{OH} 两个因素有关。由于 $V_{OH}=V_{DD}-R_{on(P)}I_{OH}$，随着 I_{OH} 的增加(由 R_L 引起)，V_{OH} 将降低。集成电路数据手册给出了不同 I_{OH} 条件下，输出高电平的值 V_{OH}。同样，在相同条件下，多个同型号器件的输出高电平的值也略有区别，但它们的 V_{OH} 值不低于 $V_{OH(min)}$。例如 74LVC00 在 3.3 V 工作电源条件下，当 $I_{OH}=-100\ \mu A$(规定电流从器件流出为负)，$V_{OH(min)}=3.1$ V；当 $I_{OH}=-12$ mA 时，$V_{OH(min)}=2.7$ V。

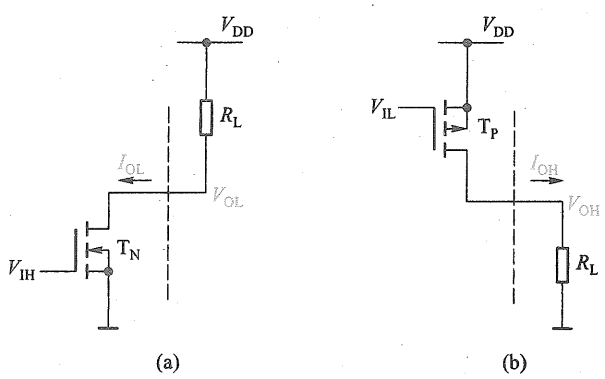


图 3.2.15 CMOS 反相器的输出特性

(a)输出低电平等效电路 (b) 输出高电平等效电路

5. 工作速度

CMOS 电路用于驱动其他 MOS 器件时，其负载的阻抗是电容性的，如图 3.2.16(a)所示。当 $v_I=0$ 时，T_N 截止，T_P 导通，由 V_{DD} 通过 T_P 向负载电容 C_L 充电，如图 3.2.16(b)所示。此时 $|V_{GSP}|=V_{DD}$，根据式(3.2.1)可知，$|V_{GSP}|$ 达到最大值使 T_P 的导通电阻变小，致使充电回路的时间常数较小。类似地，当 $v_I=V_{DD}$ 时，T_N 导通，T_P 截止，电容 C_L 的放电回路如图 3.2.16

(c)所示。由于电路具有互补对称的性质,T_N与T_P的导通电阻相等,因此,传输延迟时间t_{pLH}和t_{pHL}是基本相等的。

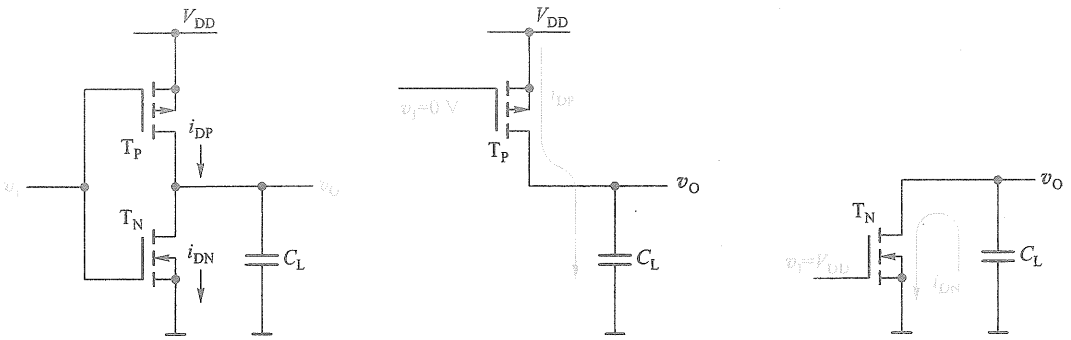


图 3.2.16 CMOS 反相器在电容负载下的工作情况

(a) 电路图 (b) 负载电容充电 (c) 负载电容放电

3.2.3 CMOS 与非门和或非门

在 CMOS 系列基本逻辑门电路中,除上述介绍的非门(反相器)外,还有与门、或门、与非门、或非门、异或门等电路。下面重点介绍与非门和或非门。

1. 与非门电路

图 3.2.17 是 2 输入端 CMOS 与非门电路,其中包括两个串联的增强型 NMOS 管和两个并联的增强型 PMOS 管。每个输入端连到一个 N 沟道和一个 P 沟道 MOS 管的栅极。电路输出与输入信号逻辑关系及各个 MOS 管的工作状态如表 3.2.1 所示。当输入端 A、B 有一个为低电平时,就会使与它相连的 NMOS 管截止,PMOS 管导通,输出为高电平;仅当 A、B 全为高电平时,才会使两个串联的 NMOS 管都导通,使两个并联的 PMOS 管都截止,输出为低电平。

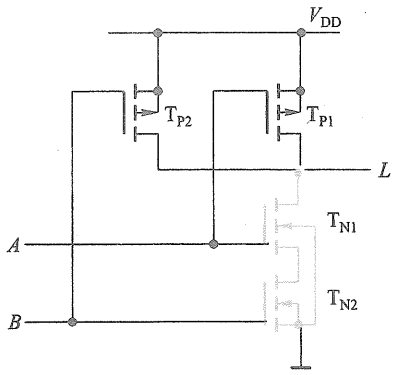


图 3.2.17 CMOS 与非门

因此,这种电路具有与非的逻辑功能,即

$$L=\overline{A\cdot B}$$

显然,n 个输入端的与非门必须有 n 个 NMOS 管串联和 n 个 PMOS 管并联。

表 3.2.1 与非门输入输出关系及各 MOS 管工作状态

A	B	T_{N1}	T_{P1}	T_{N2}	T_{P2}	L
0	0	截止	导通	截止	导通	1
0	1	截止	导通	导通	截止	1
1	0	导通	截止	截止	导通	1
1	1	导通	截止	导通	截止	0

2. 或非门电路

图 3.2.18 是 2 输入端 CMOS 或非门电路，其中包括两个并联的增强型 NMOS 管和两个串联的增强型 PMOS 管。

电路输出与输入信号逻辑关系及各个 MOS 管的工作状态如表 3.2.2 所示。当输入端 A、B 只要有一个为高电平时，就会使与它相连的 NMOS 管导通，而 PMOS 管截止，输出为低电平；仅当 A、B 全为低电平时，两个并联 NMOS 管都截止，两个串联的 PMOS 管都导通，输出为高电平。

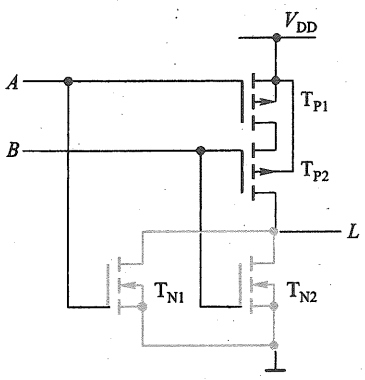


图 3.2.18 CMOS 或非门

因此，这种电路具有或非的逻辑功能，其逻辑表达式为

$$L=\overline{A+B}$$

n 个输入端的或非门必须有 n 个 NMOS 管并联和 n 个 PMOS 管串联。

从以上 CMOS 与非门和或非门电路可知，输入端的数目越多，则串联的 MOS 管也越多。若串联的 MOS 管全部导通时，其总的导通电阻会增加，以致影响输出电平，使与非门的低电平升高，使或非门的高电平降低。因此 CMOS 逻辑门电路的输入端不宜过多。

表 3.2.2 或非门输入输出关系及各 MOS 管工作状态

A	B	T_{N1}	T_{P1}	T_{N2}	T_{P2}	L
0	0	截止	导通	截止	导通	1
0	1	截止	导通	导通	截止	0
1	0	导通	截止	截止	导通	0
1	1	导通	截止	导通	截止	0

教学视频 3.1：CMOS 异或门的分析

例 3.2.1 试分析图 3.2.19 所示的 CMOS 逻辑门电路，写出其逻辑表达式，说明其逻辑功能。

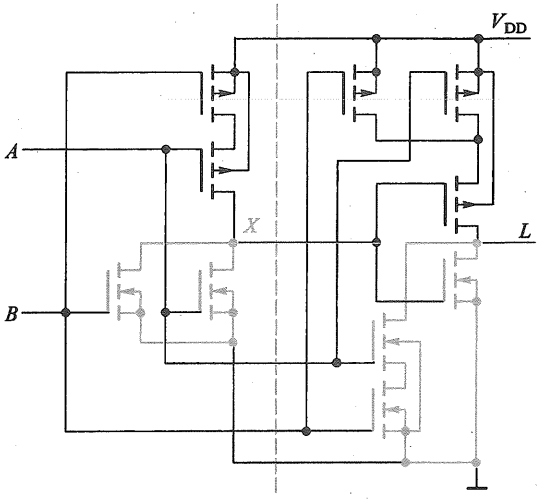


图 3.2.19 例 3.2.1 的 CMOS 逻辑门电路

解：从电路结构可以看出，虚线左边两个 NMOS 管并联，分别与它们输入端相连的两个 PMOS 管串联，是或非门结构，其输出 $X=\overline{A+B}$。虚线右边的两个 NMOS 管串联相与，即 $A\cdot B$，然后与另一个 NMOS 管并联相或即 $A\cdot B+X$，同时三个 PMOS 管也有相应的连接，所以是与或非门结构。与或非门的输出 $L=\overline{A\cdot B+X}$，将 X 的表达式代入得

$$L=\overline{A\cdot B+X}=\overline{A\cdot B+\overline{A+B}}=\overline{A\cdot B+\overline{A}\cdot\overline{B}}=\overline{A\odot B}=A\oplus B$$

电路实现异或逻辑功能。如果在异或门的后面增加一级反相器就构成异或非门，此时其表达式为 $\overline{L}=A\cdot B+\overline{A}\cdot\overline{B}$，可以实现同或逻辑功能。

3.2.4 CMOS 传输门

传输门（transmission gate，TG）的应用比较广泛，不仅可以作为基本单元电路构成各种逻辑电路，用于数字信号的传输，而且可以在取样-保持电路、斩波电路、模数和数模转换等电路中传输模拟信号，因而又称为模拟开关。

1. 传输门的结构及工作原理

CMOS 传输门由一个 P 沟道和一个 N 沟道增强型 MOS 管并联而成，如图 3.2.20（a）所示。图 3.2.20（b）是它的逻辑符号。T_N 和 T_P 是结构完全对称的，所以栅极的引出端画在符号横线的中间。它们的漏极和源极可以互换，因而传输门的输入和输出端可以互换使用，即为双向器件。设它们的阈值电压 $V_{TN}=|V_{TP}|=V_T$，C 和 $\overline{C}$ 是一对互补的控制信号。衬底电极与普通 MOS 管不同，其连接的原则是：NMOS 管的衬底连接到电路中最低电位点，而 PMOS 管的衬底连接到电路中最高电位点，这是为了使衬底与漏源极之间形成的 PN 结反向偏置，防止电流从漏极直接流入衬底。因此，将 NMOS 管的衬底连接到地电位，PMOS 管的衬底接 V_{DD} 电压。

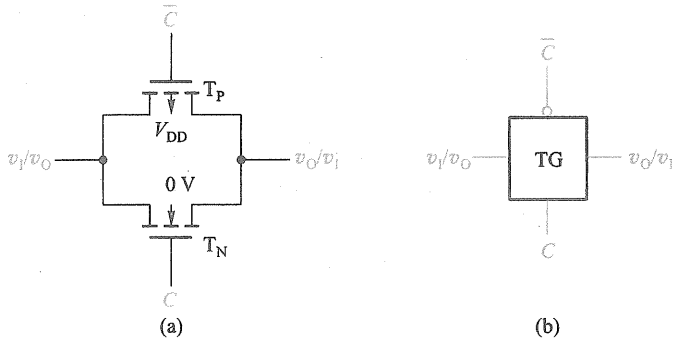


图 3.2.20 CMOS 传输门

（a）电路结构 （b）逻辑符号

传输门的工作情况如下：当 C 端接 0 V，$\overline{C}$ 端接 V_{DD} 时，输入信号 v_I 的取值在 0～V_{DD} 范围内，T_N 和 T_P 同时截止，输入和输出之间呈高阻态，传输门是断开的。

当 C 端接 V_{DD}，$\overline{C}$ 端接 0 V 时，v_I 在 0 V～($V_{DD}-V_T$) 的范围内，T_N 导通。v_I 在 V_T～V_{DD} 的范围内，T_P 将导通。由此可知，当 v_I 在 0 V～V_{DD} 之间变化时，T_N 和 T_P 至少有一个导通，使 v_I 与 v_O 之间的导通电阻很小，传输门导通。

进一步分析还可看到，当输入电压变化时，会使两管的栅源电压 v_{GS} 均发生变化。而

MOS 管漏源间的等效电阻是 v_{GS}的函数，如式(3.2.1)所示。因此，两管漏源间的等效电阻随输入电压的变化而变化。一管导通的程度越深，另一管的导通程度则相应地减小。也就是当一管的等效电阻减小，则另一管的等效电阻就增加。由于具有互补作用的两管并联在一起，使传输门导通电阻的变化相对各单管等效电阻的变化小得多，这是 CMOS 传输门的优点。

2. 模拟开关

当 CMOS 传输门用作模拟开关时，若输入信号的变化范围为$-V_{SS}$到$+V_{DD}$，则 T_N和 T_P的衬底分别接$-V_{SS}$和$+V_{DD}$。互补控制端 C 和$\overline{C}$的控制电压分别为$+V_{DD}$和$-V_{SS}$，传输门导通。C和$\overline{C}$的控制电压分别为$-V_{SS}$和$+V_{DD}$，传输门断开。

模拟开关的导通电阻与输出端的负载构成分压器，输出电压是两者对输入电压分压产生的。因此，导通电阻的稳定可以使输出电压随输入电压的变化呈线性关系。但模拟开关的导通电阻不是恒定的，因此，推出了多种改进的电路，其目的是为了使导通电阻尽可能小，并且在输入信号的变化范围内使导通电阻尽可能保持不变。

一般的模拟开关导通电阻值在几十欧以下，有些精密 CMOS 模拟开关的导通电阻达到 1 Ω 以下。当它与输入阻抗为兆欧级的运放或输入电阻达 10^{10} Ω 以上的 MOS 电路串接时，可以忽略不计。

3. 传输门在数字电路中的应用

传输门的传输延迟时间短、结构简单，可用于构成各种逻辑电路，如数据选择/分配器、触发器等。

由 CMOS 传输门构成的**异或**门电路如图 3.2.21 所示，输入信号 B 作为传输门的控制信号。当 $B=0$ 时，TG_1截止，TG_2导通，$L=A$；当 $B=1$ 时，TG_1导通，TG_2截止，$L=\overline{A}$。L 与 A、B 是**异或**逻辑关系，即 $L=A\overline{B}+\overline{A}B=A\oplus B$。

由 CMOS 传输门构成的 2 选 1 数据选择器如图 3.2.22 所示。A、B 为数据输入信号，C 为选择控制信号。

当 $C=0$ 时，TG_1导通，TG_2截止，输入信号 A 被传到输出端，$L=A$。而当 $C=1$ 时，TG_1截止，TG_2导通，输入信号 B 被传到输出端，$L=B$。

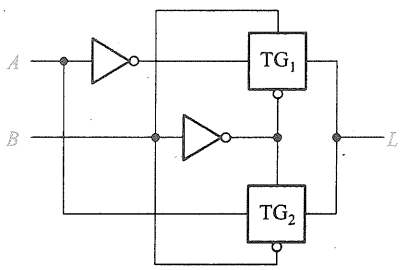


图 3.2.21 传输门构成的异或门

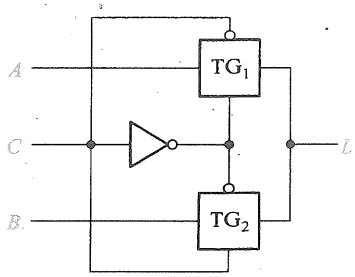


图 3.2.22 传输门构成的数据选择器

复习思考题

3.2.1 在数字电路中 MOS 管工作在输出特性的什么区域，如何保证其工作在这些区域？

3.2.2 阈值电压 V_T的含义是什么？

3.2.3 N 沟道增强型 MOS 管的导通电阻 R_{on}与 v_{GS}有怎样的关系？

3.2.4 CMOS 反相器的静态功耗是指什么？为什么其静态功耗几乎等于零？

3.2.5 CMOS 反相器在什么时候电流 i_D 达到最大值？

3.2.6 CMOS 电路的输出低电平 V_{OL} 与负载电流 I_{OL} 有怎样的关系？输出高电平 V_{OH} 与负载电流 I_{OH} 又有怎样的关系？

3.2.7 增加 CMOS 与非门输入端会增加串联的 NMOS 管数目，对电路输出电平有什么影响？增加或非门的输入端呢？

3.2.8 为什么 CMOS 传输门又称为模拟开关？传输门在数字电路和模拟电路中的连接方式有什么不同？

3.3 CMOS 逻辑门的其他输出结构及参数

实际 CMOS 逻辑门电路的输入和输出端都有保护电路和缓冲电路。另外，使用时如果需要将两个 CMOS 逻辑门的输出端连在一起，则需要选择漏极开路的逻辑门。如果希望对 CMOS 逻辑门电路的输出加以控制，则选择三态输出逻辑门电路。

3.3.1 CMOS 逻辑门的保护和缓冲电路

实际的 CMOS 逻辑门通常有输入、输出保护电路和缓冲电路。主要原因是 CMOS 门电路的输入端是 MOS 管的栅极，在栅极与沟道之间是很薄的 SiO_2 层，极易被击穿。而输入电阻高达 $10^{12}\ \Omega$ 以上，输入电容为几皮法。电路在使用前输入端是悬空的，只要外界有很小的静电源，都会在输入端积累电荷而将栅极击穿。其次，由 3.2.3 节可知，在 CMOS 与非门和或非门电路中，输入端的数目不同，则串联的 MOS 管数目也不同，若串联的 MOS 管全部导通，其总的导通电阻也不同，则输出电平也不同。另外，门电路所带负载门数目不同时，其输出特性也不相同。这些输入、输出特性的不同，会给电路设计工作带来麻烦。因此，CMOS 门电路的每个输入和输出端都接一个反相器作为缓冲电路，使得逻辑门电路的输入或输出特性具有统一的参数，同时也为了提高输出端的驱动能力。

以 74HC/HCT 系列为例，其电路结构如图 3.3.1 所示。图中的基本逻辑功能电路可以是前面介绍的反相器、与非门、或非门或者它们的组合电路，这里只画了一个输入端，若是多输入端的门，每个输入端都有输入保护电路。

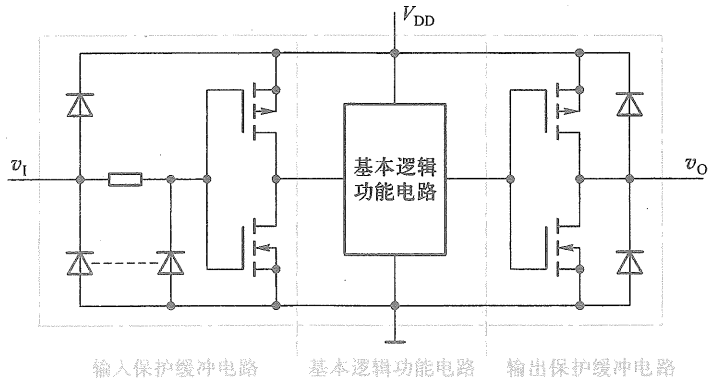


图 3.3.1 实际集成 CMOS 门电路结构图

1. 输入保护电路

图 3.3.2 所示为输入保护电路及缓冲电路。图中 C_N 和 C_P 分别表示 T_N 和 T_P 的栅极等效电容，二极管 D_1 和 D_2 的正向导通压降为 $V_{DF}=0.5\sim0.7$ V。D_2 是分布式二极管结构，用虚线和两个二极管表示。这种分布式二极管结构可以通过较大的电流，使得输入引脚上的静电荷得以释放，从而保护了 MOS 管的栅极绝缘层。二极管的反向击穿电压约为 30 V，小于栅极 SiO_2 层的击穿电压。

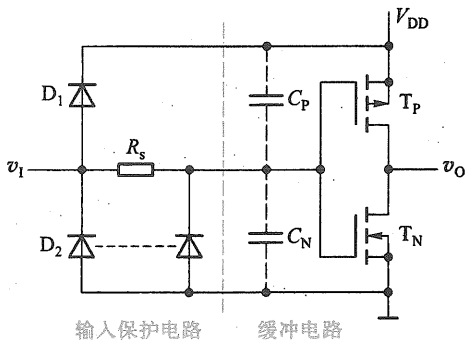


图 3.3.2 输入保护电路及缓冲电路

输入电压在正常范围内（$0\leqslant v_I\leqslant V_{DD}$），保护电路不起作用。当 $v_I>(V_{DD}+V_{DF})$ 或 $v_I<-V_{DF}$ 时，二极管 D_1 或 D_2 首先导通，MOS 管的栅极电位被限制在 $-V_{DF}$ 到 $(V_{DD}+V_{DF})$ 之间，使栅极的 SiO_2 层不会被击穿。如果输入电平发生突变时的过冲电压很大或持续时间较长，可能使流过 D_1 或 D_2 的电流过大而损坏二极管，进而使 MOS 管栅极被击穿。

另外，电阻 R_s 和 MOS 管的栅极电容组成积分网络，使输入信号的过冲电压延迟一段时间才作用到栅极上，而且幅度有所衰减。为减小这种延迟对电路动态性能的影响，R_s 值不宜过大，通常 R_s 为 2 kΩ 左右。

逻辑门电路输出端也接入静电保护二极管，确保输出不超出正常的工作范围。

2. 反相缓冲电路

图 3.3.3 所示为带缓冲级的 CMOS 与非门电路的逻辑符号。由于输入、输出端加了反相器作为缓冲电路，所以电路输出与输入的逻辑关系也发生了变化。图中的基本逻辑电路是或非门，增加了缓冲器后的逻辑关系为与非功能，即

$$L=\overline{\overline{A}+\overline{B}}=\overline{A\cdot B}$$

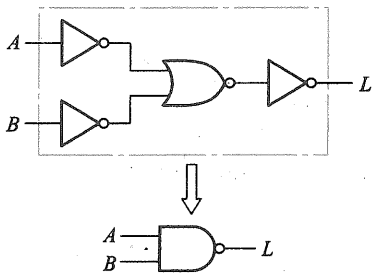


图 3.3.3 带缓冲级的 CMOS 与非门电路的逻辑符号

3.3.2 CMOS 漏极开路门和三态输出门电路

前面讨论了具有输入、输出缓冲电路的 CMOS 电路。如果从输出端看，还有另外两种输出结构的 CMOS 门电路，漏极开路门（open drain，OD）和三态输出门（tristate logic，TSL）。下面分别加以讨论。

1. CMOS 漏极开路门

（1）漏极开路门的结构及工作原理

通常 CMOS 门电路都有反相器作输出缓冲电路。而在工程实践中，有时为了方便，将两个门的输出端并联以实现与逻辑功能（称为线与）。现在来考察这种情况，如果将两个 CMOS 与非门 G_1 和 G_2 的输出端连接在一起，如图 3.3.4 所示，并设 G_1 的输出处于高电平，T_{N1} 截止，T_{P1} 导通；而 G_2 的输出为低电平，T_{N2} 导通，T_{P2} 截止。这样，从 G_1 的 T_{P1} 到 G_2 的 T_{N2} 将形

成一低阻通路，从而产生很大的电流，有可能导致器件的损毁，并且无法确定输出是高电平还是低电平。因此，普通门电路的输出端是禁止线与的。这一问题可以采用 OD 门来解决。另外，OD 门也可以驱动大电流负载，或者实现逻辑电平变换。

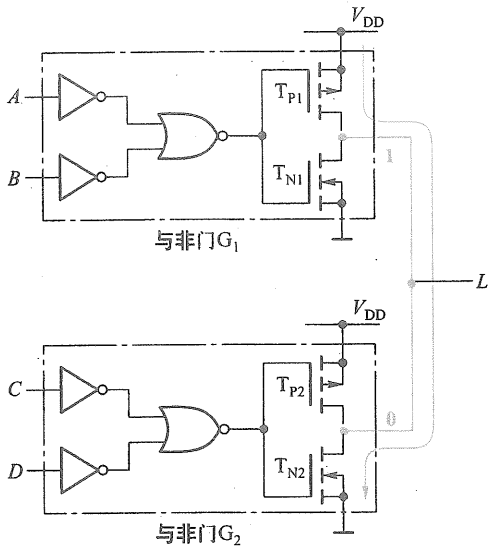


图 3.3.4 普通 CMOS 与非门输出端相连

漏极开路是指 CMOS 门电路的输出电路只有 NMOS 管，并且它的漏极是开路的。漏极开路的与非门电路及逻辑符号如图 3.3.5(a)和 3.3.5(b)所示，其中图标“◇”表示漏极开路之意。

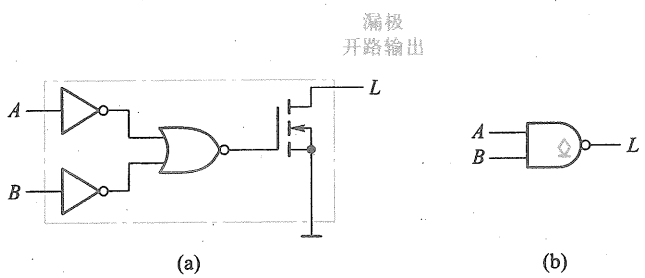


图 3.3.5 漏极开路(OD)与非门
(a) 电路结构 (b) 逻辑符号

使用 OD 门时必须在漏极和电源 V_{DD} 之间，外接一个上拉电阻 R_p[①]。图 3.3.6 所示为两个 OD 与非门实现线与。将两个门电路输出端接在一起，通过上拉电阻接电源。由图 3.3.6(a)可见，当两个与非门的输出全为 1 时，输出为 1；只要其中一个为 0 时，输出为 0。所以该电路的输出符合与逻辑功能，即 $L=\overline{AB}\cdot\overline{CD}$。图 3.3.6(b)所示为两个 OD 与非门线与的逻辑图。

① 上拉电阻系 pull-up resistor 的译称。

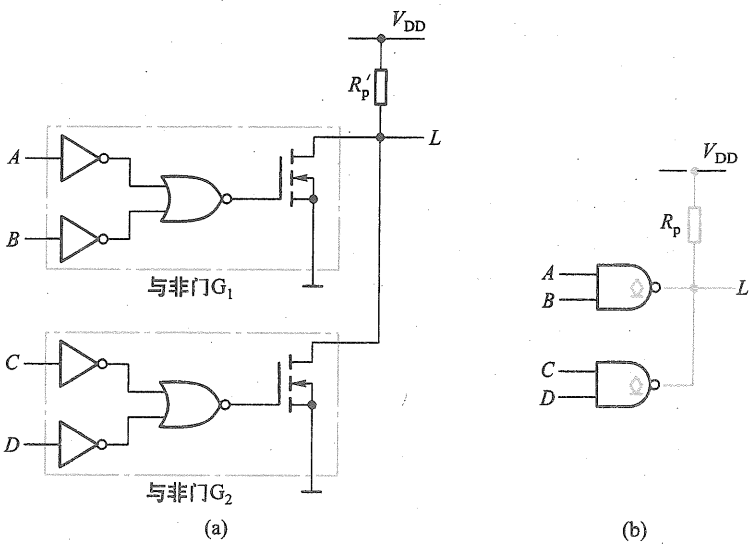


图 3.3.6 漏极开路(OD)与非门的线与

(a) 线与连接图 (b) 逻辑图

(2) 上拉电阻对 OD 门动态性能的影响

当其他门电路作为 OD 门的负载时,OD 门称为驱动门,其后所接的门电路称为负载门。由于驱动门的输出电容、负载门的输入电容以及接线电容的存在,上拉电阻 R_p 的大小必将影响 OD 门的开关速度,R_p 的值越小,负载电容的充电时间常数也越小,因而开关速度越快。但上拉电阻不能任意地减小,它必须保证 OD 门输出端的电流不能超过允许的最大值 $I_{OL(max)}$。

对于 74HC/HCT 系列 CMOS 门电路,$I_{OL(max)}$ = 4 mA,因此 R_p 必须大于 $V_{DD}/I_{OL(max)}$ = 5 V/4 mA = 1.25 kΩ。与普通 CMOS 电路相比,R_p 的值比 PMOS 管导通电阻大,因而,OD 门从低电平到高电平的转换速度比普通 CMOS 门慢。

图 3.3.7 所示为 OD 门驱动其他门电路,由于负载门的输入电容以及接线电容的存在,用等效电容 C_L 做负载分析电路的工作情况,取上拉电阻 R_p 为 1.5 kΩ。由于输出状态发生变化时,电容的充、放电作用会对输出波形产生影响,所以主要考虑负载电容 C_L,并假设 C_L 为 100 pF,下面分两种情况进行讨论。

当 OD 门输出由高电平变为低电平时,其等效电路如图 3.3.7(b)所示,如果 NMOS 管导通时的等效电阻 $R_{on(N)}$ 为 100 Ω,因此充了电的电容主要通过 NMOS 管放电,放电时间常数 τ_{HL} = 100 Ω×100 pF = 10 ns。

当输出由低电平跳变为高电平时,其等效电路如图 3.3.7(c)所示,此时电源通过 R_p 向 C_L 充电,充电时间常数 τ_{LH} = 1.5 kΩ×100 pF = 150 ns,导致输出波形的上升沿时间很长。因此,当工作速度较快时,应尽量避免用 OD 门驱动大的电容性负载。

(3) 上拉电阻的计算

选择上拉电阻 R_p 的值时要考虑多种因素。一方面,从上面分析可知,如果负载具有电容性,R_p 的值越小,电容的充电时间常数也越小,因而开关速度越快,但功耗也越大。另一

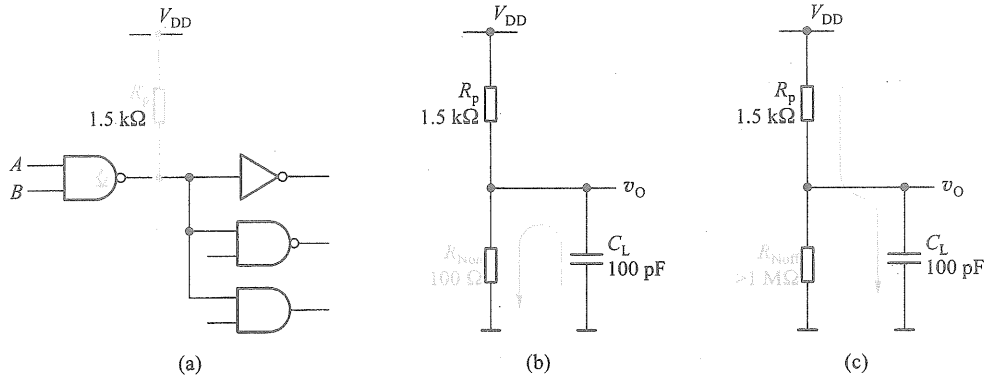


图 3.3.7 OD 门驱动电容性负载

(a) 逻辑图 (b) 输出为低电平时的等效电路 (c) 输出为高电平时的等效电路

方面,多个 OD 门的输出端线与在一起,当只有一个门导通,输出为低电平,其他门均截止时,负载电流将全部流向导通的 OD 门,这是一种最不利的情况,此时上拉电阻 R_p 具有限制电流的作用,其取值不能太小。应保证 I_{OL} 不超过额定值 $I_{OL(max)}$。

下面分两种情况计算 R_p 的最小值 $R_{p(min)}$ 及最大值 $R_{p(max)}$。

① 当输出为低电平,并联的 OD 门中只有一个导通。

由图 3.3.8(a)可见,为保证导通 OD 门的输出电流 $I_{OL} \leqslant I_{OL(max)}$,对于所有截止的 OD 门,忽略流过截止 NMOS 管的漏电流 I_{OZ}。于是,$I_{OL}=I_{Rp}+I_{IL(total)} \leqslant I_{OL(max)}$,并且 $V_{OL}=V_{OL(max)}$。求得 R_p 上的压降为 $V_{DD}-V_{OL(max)}$,则流过 R_p 的电流为 $(V_{DD}-V_{OL(max)})/R_p=I_{OL(max)}-I_{IL(total)}$。此时 R_p 应满足方程

$$R_p \geqslant \frac{V_{DD}-V_{OL(max)}}{I_{OL(max)}-I_{IL(total)}}$$

因此 R_p 的最小值 $R_{p(min)}$ 可按下式来确定:

$$R_{p(min)}=\frac{V_{DD}-V_{OL(max)}}{I_{OL(max)}-I_{IL(total)}} \tag{3.3.1}$$

式中,V_{DD} 为直流电源电压;$V_{OL(max)}$ 为驱动门 V_{OL} 最大值;$I_{OL(max)}$ 为驱动门 I_{OL} 最大值;$I_{IL(total)}$ 为 CMOS 负载门低电平输入电流 I_{IL} 总和,$I_{IL(total)}=nI_{IL}$,n 为并联的输入端数目。

② 当所有 OD 门输出均为高电平。

由图 3.3.8(b)可见,为使得输出高电平不低于规定的 V_{OH} 的最小值,即 $V_{OH} \geqslant V_{OH(min)}$,则 R_p 的选择不能过大。流过 R_p 的电流 $I_{Rp}=(V_{DD}-V_{OH})/R_p$,应满足 $I_{Rp}=I_{OZ(total)}+I_{IH(total)}$。则 $V_{OH}=V_{DD}-R_p(I_{OZ(total)}+I_{IH(total)}) \geqslant V_{OH(min)}$。

因此,R_p 的最大值 $R_{p(max)}$ 可按下式来确定:

$$R_{p(max)}=\frac{V_{DD}-V_{OH(min)}}{I_{OZ(total)}+I_{IH(total)}} \tag{3.3.2}$$

式中,$V_{OH(min)}$ 为驱动门 V_{OH} 最小值;$I_{OZ(total)}$ 为全部驱动门输出高电平时的漏电流总和;$I_{IH(total)}$ 为 CMOS 负载门高电平输入电流 I_{IH} 总和。$I_{IH(total)}=nI_{IH}$,n 为负载门并联的输入端数目。

实际上,R_p 的值选在 $R_{p(min)}$ 和 $R_{p(max)}$ 之间,若要求电路速度快,选用 R_p 的值接近 $R_{p(min)}$

的标准值。若要求电路功耗小，选用 R_p 的值接近 $R_{p(max)}$ 的标准值。

式(3.3.1)和式(3.3.2)中已考虑电流的方向，因此，所有电流参数均取正值。

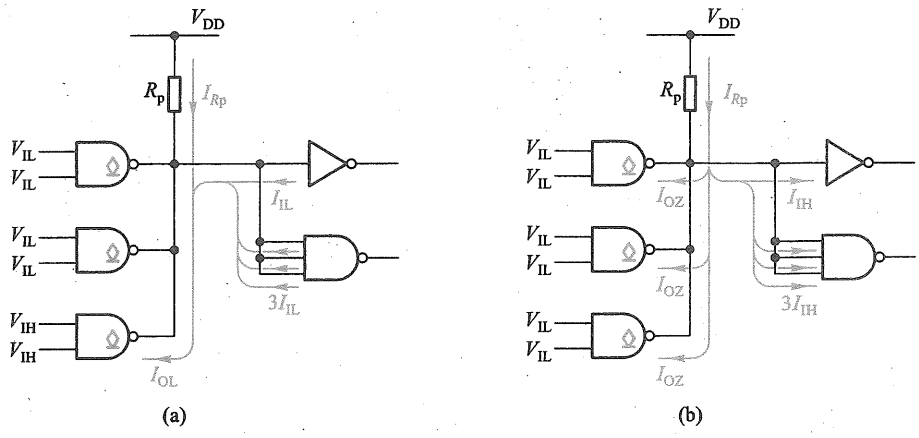


图 3.3.8 计算 OD 门上拉电阻 R_p 的工作情况

(a) $R_{p(min)}$ 的工作情况 (b) $R_{p(max)}$ 的工作情况

例 3.3.1 设 74HC03 中的 3 个漏极开路**与非门作线与**连接，驱动 74HC04 中的一个反相器和 74HC10 中的一个 3 输入**与非门**，电路如图 3.3.8 所示，试确定一合适大小的上拉电阻 R_p。已知 $V_{DD}=5$ V，OD 门输出低电平 $V_{OL(max)}=0.33$ V 时的输出电流 $I_{OL(max)}=4$ mA，输出高电平 $V_{OH(min)}=4.4$ V 时的漏电流 $I_{OZ}=5$ μA。负载门高电平和低电平输入电流最大值 $I_{IH(max)}=I_{IL(max)}=1$ μA。

解：(1) 当**线与**的 OD 门输出为低电平时，式(3.3.1)中

$$I_{IL(total)}=nI_{IL(max)}=4\times1\ \mu A=0.004\ mA$$

得

$$R_{p(min)}=\frac{V_{DD}-V_{OL(max)}}{I_{OL(max)}-I_{IL(total)}}=\frac{5\ V-0.33\ V}{4\ mA-0.004\ mA}\approx1.17\ k\Omega$$

(2) 当**线与**的 OD 门输出为高电平时，式(3.3.2)中

$$I_{OZ(total)}=3\times5\ \mu A=0.015\ mA$$

$$I_{IH(total)}=4\times1\ \mu A=0.004\ mA$$

得

$$R_{p(max)}=\frac{V_{DD}-V_{OH(min)}}{I_{OZ(total)}+I_{IH(total)}}=\frac{5\ V-4.4\ V}{0.015\ mA+0.004\ mA}\approx31.58\ k\Omega$$

根据上述计算，R_p 的值可在 1.17 kΩ 至 31.58 kΩ 之间选取。为使电路有较快的开关速度，可选用一标准值为 2 kΩ 的电阻。

除了可以实现**线与**的逻辑功能外，OD 门也用来驱动发光二极管。发光二极管发光时需要的电流较大。图 3.3.9(a)所示为用 74HC05 中的 1 个漏极开路反相器驱动发光二极管。发光二极管发光时要求有几毫安的电流通过，74HC/HCT 系列 CMOS 门电路的最大灌电流或拉电流为 4 mA。当输入为高电平时，输出为低电平，此时发光二极管发光，否则输出为高电平时二极管熄灭。若驱动指示灯(12 V，20 mA)，74HC/HCT 系列门电路不能满足要求，可以选用 74AC05 或 74ACT05，其灌电流为 24 mA。

OD 门电路的另一个功能是实现逻辑电平变换，例如，可将 3.3 V 高电平转换为 5 V 高电平，如图 3.3.9(b)所示。

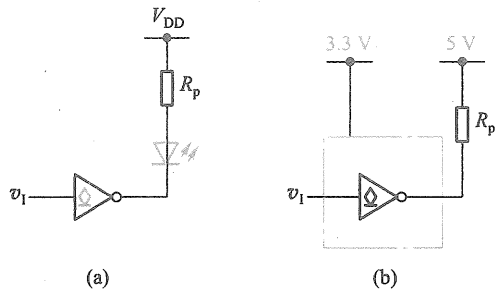


图 3.3.9 OD 门电路的应用

(a) 驱动发光二极管 (b) 逻辑电平变换

2. 三态输出门电路

利用 OD 门虽然可以实现线与的功能，但外接电阻 R_p 的选择要受到一定的限制，因此影响了工作速度。同时它省去了 PMOS 有源负载，使得带负载能力下降。为保持互补输出级的优点，又可以与总线连接，开发了一种三态输出门(简称三态门)电路，它的输出除了具有一般门电路的两种状态，即输出高、低电平外，还具有高输出阻抗的第三状态，称为高阻态，又称为禁止态。

图 3.3.10(a)所示为高电平使能的三态输出缓冲器，其中 A 是输入端，L 为输出端，EN (enable)是控制信号输入端，也称为使能端，图 3.3.10(b)是它的逻辑符号。

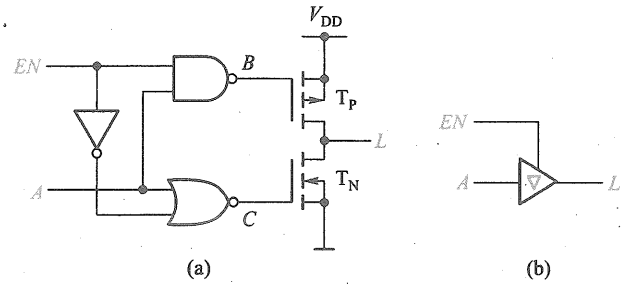


图 3.3.10 高电平使能三态输出缓冲器

(a) 电路结构 (b) 逻辑符号

当使能端 $EN=1$ 时，如果 $A=0$，则 $B=1$，$C=1$，使得 T_N 导通，同时 T_P 截止，输出端 $L=0$；如果 $A=1$，则 $B=0$，$C=0$，使得 T_N 截止，T_P 导通，输出端 $L=1$。

当使能端 $EN=0$ 时，不论 A 的取值为何，都使得 $B=1$，$C=0$，则 T_N 和 T_P 均截止，电路的输出端既不是低电平，又不是高电平，而是开路，这就是第三种高阻工作状态。

由以上分析可知，当 EN 为有效的高电平时，电路处于正常逻辑工作状态，$L=A$。而当 EN 为低电平时，电路处于高阻状态。高电平使能(简称高使能)的三态输出缓冲器的真值表如表 3.3.1 所示，其中×表示 A 可以是 0 或 1。

在实际应用中，除上述介绍的高使能三态输出缓冲器外，还有其他不同形式的电路结构

(见习题 3.3.7 所示)。使能端可以是高电平或低电平有效,输出与输入可以是同相或反相,其目的都是为用户提供一个合适的动态特性。其他三态缓冲器或反相器的符号如图 3.3.11 所示,对于**与非**、**或非**等逻辑门也有三态输出结构。

表 3.3.1 高使能三态输出缓冲器的真值表

使能 EN	输入 A	输出 L
1	0	0
1	1	1
0	×	高阻

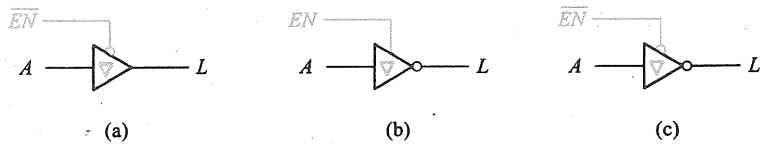


图 3.3.11 其他三态输出电路逻辑符号

(a) 低使能三态缓冲器 (b) 高使能三态反相器 (c) 低使能三态反相器

三态输出门电路主要用于总线传输,如计算机或微处理器系统,其连接形式如图 3.3.12 所示。为避免两个不同的信号在总线上引起冲突,任何时刻只有一个门电路的使能端 EN 为 1,该门电路的信号被传到总线上,而其他三态输出电路处于高阻状态。这样就可以按一定顺序将各个门电路的输出信号分时送到总线上。

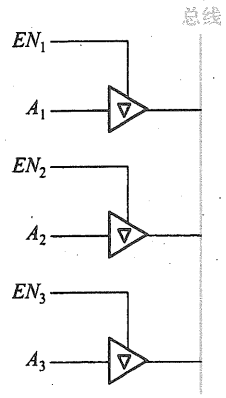


图 3.3.12 三态输出电路构成总线传输结构

接到总线上的三态输出电路,在任何时刻只有一个使能端为有效信号,这就要求某个与总线进行数据传输的三态门必须关断以后,另一个三态门才允许与总线进行数据传输。即从高阻态到高电平(或低电平)输出的转换时间,略大于从高电平(或低电平)到高阻态的转换时间。这样,控制系统给出的使能信号,使前一个电路进入高阻状态以后,后一个电路的输出信号才送到总线上,以避免两个不同的信号在总线上引起冲突。

3.3.3 CMOS 逻辑门电路的主要技术参数

逻辑门电路的制造商,通常会提供逻辑器件的数据手册。对于 54 或 74 系列的 CMOS 电路,只要型号最后的数字相同,它们的逻辑功能就一样,但是电气性能参数有所不同。手册中一般都会给出门电路的电压传输特性 v_I-v_O,输入和输出的高、低电压,噪声容限,传输延迟时间,功耗等。除传输特性外,其他各项技术参数分别介绍如下:

1. 输入和输出的高、低电平

前已讨论,数字电路中的高、低电压常用高、低电平来描述,并规定在正逻辑体制中,用

逻辑 1 和 0 分别表示高、低电平。当逻辑电路的输入电压在一定范围内变化时，输出电压并不会改变，因此逻辑 1 或 0 对应一定的电压范围。不同系列的集成电路，输入和输出为逻辑 1 或 0 所对应的电压范围也不同。制造商的数据手册中一般都给出四种逻辑电平参数：输入低电平的上限值 $V_{IL(max)}$、输入高电平的下限值 $V_{IH(min)}$、输出低电平的上限值 $V_{OL(max)}$ 和输出高电平的下限值 $V_{OH(min)}$。

表 3.3.2 列出了几种 CMOS 系列非门在典型工作电压时的输入高、低电压值以及在规定输出电流 I_O 条件下的输出电压值。

表 3.3.2 几种 CMOS 系列非门的输入和输出电压值及输入噪声容限

参数/单位	类型				
	4 000 ($V_{DD}=5$ V, $I_O=1$ mA)	74HC ($V_{DD}=5$ V, $I_O=0.02$ mA)	74HCT ($V_{DD}=5$ V, $I_O=0.02$ mA)	74LVC ($V_{DD}=3.3$ V, $I_O=0.1$ mA)	74AUC ($V_{DD}=1.8$ V, $I_O=0.1$ mA)
$V_{IL(max)}$/V	1.0	1.5	0.8	0.8	0.6
$V_{OL(max)}$/V	0.05	0.1	0.1	0.2	0.2
$V_{IH(min)}$/V	4.0	3.5	2.0	2.0	1.2
$V_{OH(min)}$/V	4.95	4.9	4.9	3.1	1.7
高电平噪声容限(V_{NH}/V)	0.95	1.4	2.9	1.1	0.5
低电平噪声容限(V_{NL}/V)	0.95	1.4	0.7	0.6	0.4

2. 噪声容限

噪声容限反映了门电路的抗干扰能力。二值数字逻辑电路的优点在于它的输入信号允许一定的容差。在数字系统中，各逻辑电路之间的连线可能会受到各种噪声的干扰，如信号传输引起的噪声，信号的高低电平转换引起的噪声，或者邻近开关信号所引起的随机脉冲的噪声。这些噪声会叠加在工作信号上，只要高电平信号叠加噪声后不低于输入高电平最小值或低电平信号叠加噪声后不高于输入低电平最大值，则输出逻辑状态不会受影响。通常将这个最大噪声幅度称为噪声容限。电路的噪声容限越大，其抗干扰能力越强。

图 3.3.13 所示为噪声容限定义的示意图。前一级驱动门电路的输出，就是后一级负载门电路的输入。则输入高电平的噪声容限

$$V_{NH}=V_{OH(min)}-V_{IH(min)} \tag{3.3.3}$$

V_{NH} 反映了驱动门输出高电平时，容许叠加在其上的负向噪声电压的最大值。类似地，输入低电平的噪声容限

$$V_{NL}=V_{IL(max)}-V_{OL(max)} \tag{3.3.4}$$

V_{NL} 反映了驱动门输出低电平时，容许叠加在其上的正向噪声电压的最大值。

根据 74HC 系列 CMOS 集成电路在 5 V 典型工作电压时的参数($I_O=0.02$ mA)，求得其输入高、低电平的噪声容限分别为：

高电平的噪声容限 $V_{NH}=V_{OH(min)}-V_{IH(min)}=4.9\ V-3.5\ V=1.4\ V$

低电平的噪声容限 $V_{NL}=V_{IL(max)}-V_{OL(max)}=1.5\ V-0.1\ V=1.4\ V$

表 3.3.2 也列出其他几种 CMOS 系列的噪声容限。

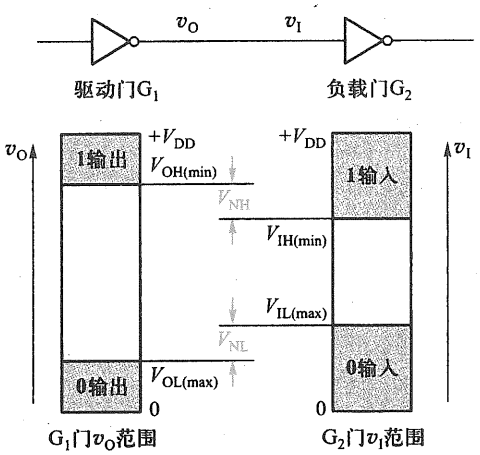


图 3.3.13 输入噪声容限示意图

3. 传输延迟时间

传输延迟时间是表征门电路开关速度的参数，它说明门电路在输入脉冲波形的作用下，输出波形相对于输入波形延迟了多长时间，其数值与电源电压 V_{DD} 及负载电容的大小有关。

当非门电路的输入端加入一脉冲波形，其相应的输出波形如图 3.3.14 所示。通常输出波形下降沿、上升沿的中点与输入波形对应沿中点之间的时间间隔，分别用 t_{pLH} 和 t_{pHL} 表示，由于 CMOS 门电路输出级的互补对称性，其 t_{pLH} 和 t_{pHL} 相等。

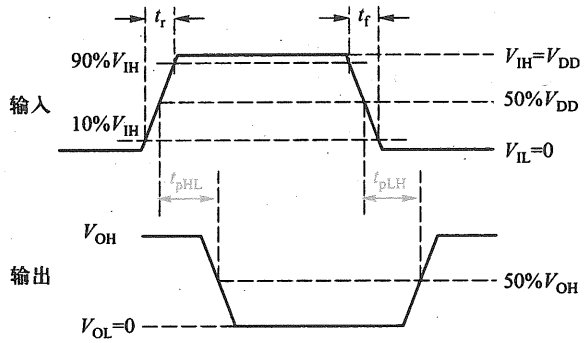


图 3.3.14 门电路传输延迟波形图

有时也采用平均传输延迟时间这一参数，即 $t_{pd}=(t_{pLH}+t_{pHL})/2$。例如，CMOS 与非门 74HC00 在 5 V 典型工作电压时 $t_{pLH}=t_{pHL}=7\ ns$，$t_{pd}=(7+7)\ ns/2=7\ ns$。在图 3.3.14 中还标出了上升时间 t_r 和下降时间 t_f。

图 3.3.15 所示为几种 CMOS 集成电路的传输延迟时间 t_{pd} 与电源电压 V_{DD} 的关系曲线。对于同一系列，当电源电压增加时，传输延迟时间减少，可提高工作速度。由图可见，74AHC

系列的速度达到了 74HC 系列的两倍，而低电压 74LVC、74ALVC 和超低电压 74AUC 系列的电源电压更低，传输延迟时间更短，工作速度更快。

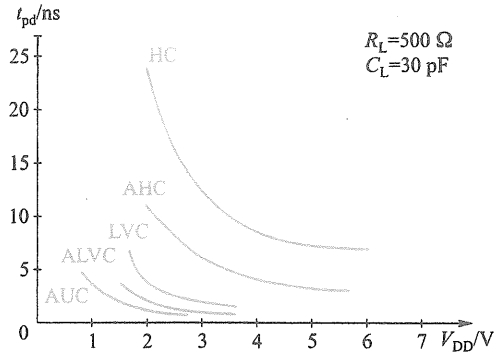


图 3.3.15 几种 CMOS 系列传输延迟时间与电源电压关系

4. 功耗

功耗是门电路重要参数之一。功耗有静态和动态之分。所谓静态功耗是指电路输出没有状态转换时的功耗。静态时，CMOS 电路的电流非常小，使得静态功耗非常低，所以 CMOS 电路广泛应用于要求功耗较低或电池供电的设备，如便携计算机、手机和掌上电脑等。这些设备在没有输入信号时，功耗非常低。

CMOS 电路在输出发生状态转换时的功耗称为动态功耗。它主要由两部分组成。一部分是电路输出状态转换瞬间 MOS 管的导通功耗。由图 3.2.13 所示的 CMOS 反相器电压和电流传输特性可知，当输出电压由高到低或由低到高变化过程中，在短时间内，NMOS 管和 PMOS 管均导通，从而导致有较大的电流从电源 V_{DD} 经导通的 NMOS 管和 PMOS 管流入地。这部分功耗可由下式表示：

$$P_T = C_{PD} V_{DD}^2 f$$

式中，f 为输出信号的转换频率；V_{DD} 为供电电源；C_{PD} 称为功耗电容（power dissipation capacitance），它不是一个实际电容，而是用来计算输出端在高、低电平转换时，输出电流动态特性的等效参数，与电源电压和工作频率有关，可以在数据手册中查到。

动态功耗的另一部分是因为 CMOS 门的负载通常是电容性的，当输出由高电平到低电平，或者由低电平到高电平转换时，会对电容进行充、放电，这一过程将增加电路的损耗。这部分动态功耗为

$$P_L = C_L V_{DD}^2 f$$

式中，C_L 为负载电容。由此得到 CMOS 电路总的动态功耗为

$$P_D = (C_{PD} + C_L) V_{DD}^2 f \tag{3.3.5}$$

从式(3.3.5)可见，CMOS 动态功耗正比于转换频率和电源电压的平方。当工作频率增加时，CMOS 门的动态功耗会线性增加。一个典型的 CMOS 门电路的静态功耗为 0.01 mW。当工作频率达到 1 MHz 时，功耗增加到 0.5 mW。当频率为 10 MHz 时，功耗为 5 mW。可见，其功耗主要取决于动态功耗。在设计 CMOS 电路时，为降低功耗，可选用低电源电压器件，如 3.3 V 的 74LVC 系列、1.8 V 的 74AUC 系列或超低功耗 74AUP 系列。

5. 延时-功耗积

从上述对传输延迟时间和功耗的讨论可知，若增加电源电压，电路的工作速度变快，但功耗也随之增加。理想的数字电路或系统，既要速度高，又要功耗低。在工程实践中要实现这种理想情况是较难的。高速数字电路往往要以较大的功耗为代价。一种衡量这种性能的综合性指标称为延时-功耗积，用符号 DP 表示，单位为 J(焦[耳])，即

$$DP = t_{pd} P_D$$

式中 $t_{pd} = (t_{pLH} + t_{pHL})/2$；$P_D$ 为门电路的功耗，由式(3.3.5)进行计算。

一个逻辑门电路的 DP 值越小，表明它的特性越接近理想情况。表 3.3.3 所示为不同 CMOS 系列非门在典型工作电压下，负载电容为 15 pF，工作频率为 10 MHz 时的 DP 值，其中功耗电容 C_{PD} 和传输延迟时间 t_{pd} 可在数据手册中查到。由此可见，74AHC 系列的性能优于 74HC 系列，而 74LVC 系列和 74AUC 系列比前两者性能更好。

表 3.3.3 几种 CMOS 系列非门的 DP 性能比较

参数/单位	系列			
	74HC04 (V_{DD} = 5 V)	74AHC04 (V_{DD} = 5 V)	74LVC04 (V_{DD} = 3.3 V)	74AUC04 (V_{DD} = 1.8 V)
功耗电容 C_{PD}/pF	21	12	8	17
传输延迟时间 t_{pd}/ns (C_L = 15 pF)	6	3.8	2.5	0.8
功耗 P_D/mW (10 MHz)	9	6.8	2.5	1
延时-功耗积 DP/pJ	54	25.84	6.25	0.8

6. 扇入与扇出数

门电路的扇入数取决于它的输入端的个数，例如一个 3 输入端的与非门，其扇入数 $N_I = 3$。

门电路的扇出数是指其在正常工作情况下，所能带同类门电路的最大数目。扇出数的计算则稍复杂些，要考虑两种情况，一种是拉电流负载，另一种是灌电流负载。

(1) 拉电流工作情况

图 3.3.16(a)所示为拉电流负载的情况。当驱动门的输出端为高电平时，将有电流 I_{OH} 从驱动门拉出而流入负载门，负载门的输入电流为 I_{IH}。当负载门的个数增加时，总的拉电流将增加，会引起输出高电平的降低。但不得低于输出高电平的下限值，这就限制了负载门的个数。这样，输出为高电平时的扇出数可表示为

$$N_{OH} = \frac{I_{OH}(\text{驱动门})}{I_{IH}(\text{负载门})} \tag{3.3.6}$$

(2) 灌电流工作情况

图 3.3.16(b)所示为灌电流负载的情况。当驱动门的输出端为低电平时，电流 I_{OL} 流入驱动门，它是负载门输入端电流 I_{IL} 之和。当负载门的个数增加时，总的灌电流 I_{OL} 将增加，同时也将引起输出低电平 V_{OL} 的升高。在保证不超过输出低电平的上限值时，驱动门所能驱动同类门的个数由下式决定：

$$N_{OL} = \frac{I_{OL}(\text{驱动门})}{I_{IL}(\text{负载门})} \tag{3.3.7}$$

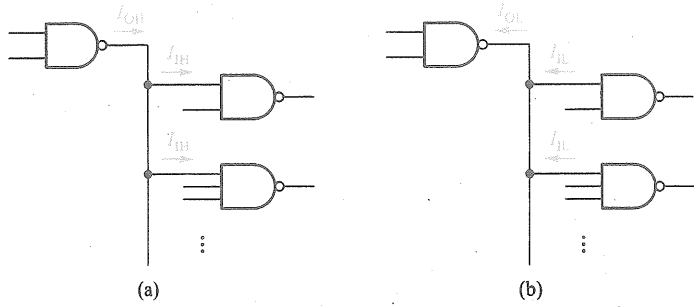


图 3.3.16 扇出数的计算

(a) 拉电流负载 (b) 灌电流负载

这里考虑每个负载门只有一个输入端与驱动门相接，如果每个负载门有两个以上的输入端接入驱动门，对于 CMOS 负载门而言，扇出数实为输入端数目。

如果驱动门的负载大于其扇出能力，低电平输出的数值将高于 $V_{OL(max)}$；高电平输出的数值低于 $V_{OH(min)}$，电路不能正常工作。

一般逻辑器件的数据手册中，并不给出扇出数，而需计算或用实验的方法求得，并注意在设计时留有余地，以保证数字电路或系统能正常地运行。在实际的工程设计中，如果 $N_{OL} \neq N_{OH}$，则取二者中的最小值。

为便于比较，表 3.3.4 给出了几种 CMOS 系列 2 输入与非门的输入和输出电压及电流参数的典型值。大多数 CMOS 器件有两类负载规格参数。一类对应于“CMOS 负载”，即输出端与其他 CMOS 门相连，此时输出端只有很小的电流，因此，高电平输出值接近 V_{DD}，而低电平输出值接近 0 V。另一类对应“TTL 负载”，即输出端与 TTL 门或 LED 等负载相连，此时输出端有较大的电流。

表 3.3.4 几种 CMOS 系列 2 输入与非门的电压与电流参数

参数/单位		系列						
		74HC00	74HCT00	74AHC00	74AHCT00	74LVC00	74ALVC00	74AUC00
$I_{IH(max)}$/μA		1	1	1	1	5	5	5
$I_{IL(max)}$/μA		−1	−1	−1	−1	−5	−5	−5
$I_{OH(max)}$/mA	CMOS 负载	−0.02	−0.02	−0.05	−0.05	−0.1	−0.1	−0.1
	TTL 负载	−4	−4	−8	−8	−24	−24	−9
$I_{OL(max)}$/mA	CMOS 负载	0.02	0.02	0.05	0.05	0.1	0.1	0.1
	TTL 负载	4	4	8	8	24	24	9
$V_{IH(min)}$/V		3.5	2	3.5	2	2	2	1.7
$V_{IL(max)}$/V		1.5	0.8	1.5	0.8	0.8	0.8	0.7
$V_{OH(min)}$/V	CMOS 负载	4.9	4.9	4.9	4.9	2.8	2.8	2.2
	TTL 负载	4.4	4.4	4.4	4.4	2.2	2	1.8

续表

参数/单位		系列						
		74HC00	74HCT00	74AHC00	74AHCT00	74LVC00	74ALVC00	74AUC00
$V_{OL(max)}$/V	CMOS 负载	0.1	0.1	0.1	0.1	0.2	0.2	0.2
	TTL 负载	0.33	0.33	0.44	0.44	0.55	0.55	0.6

注：表中的参数参考 Texas Instruments 公司在互联网站上提供的集成电路产品数据，以及文献[2]。74HC/HCT 和 74AHC/AHCT 是 $V_{DD}=5$ V 时的参数，74LVC/ALVC 是 $V_{DD}=3$ V 时的参数，74AUC 是 $V_{DD}=2.7$ V 时的参数。更详细的参数，可查阅有关器件的数据手册。

例如，根据表 3.3.4 可知 HC 系列 CMOS 门电路参数：当高电平输出为 4.9 V 时，输出电流 $I_{OH}=-20$ μA；低电平输出 0.1 V 时，$I_{OL}=20$ μA，输入电流 $I_{IH}=1$ μA，$I_{IL}=-1$ μA。数值前的负号表示电流从器件流出，否则电流流入器件，计算时只取绝对值。所以根据式(3.3.6)和式(3.3.7)计算出 $N_{OH}=N_{OL}=20$，即最多可接同类电路的输入端数为 20 个。

如果允许其高电平输出降至 4.4 V，并且低电平为 0.33 V，则 I_{OH} 和 I_{OL} 分别为 −4 mA 和 4 mA，此时计算出的扇出数为 4 000，实际不可能达到这么大的数。

以上 N_{OH} 和 N_{OL} 的计算公式没有考虑电容性负载。当 CMOS 门驱动同类门，并且输出状态产生由高到低或由低到高变化瞬间，由图 3.2.13(b)所示的 CMOS 反相器电流传输特性可知，负载门的电流很大，会对负载门输入端的杂散电容进行充放电，从而影响门电路的开关速度。因此，工作频率增加时，应减少扇出数。

另外，不同门电路的参数可能与表 3.3.4 给出的典型值不同，因此当分析实际问题时，必须查阅制造商提供的数据表。

CMOS 器件有许多不同系列产品，各系列产品的参数也有很多。对于设计者，比较重要的参数是速度和功耗。

复习思考题

3.3.1 为什么要在 CMOS 逻辑门电路输入、输出端加保护和缓冲电路？

3.3.2 CMOS 逻辑门的输入端分别通过 1 kΩ 和 100 kΩ 的电阻接地时，其输入电压是多少？

3.3.3 普通的两个 CMOS 逻辑门电路的输出端能否连接在一起，为什么？

3.3.4 漏极开路门和三态输出门的特点是什么，它们各用在什么场合？

3.3.5 如何确定漏极开路门的上拉电阻？它对开关速度有无影响？

3.3.6 当增加电源电压时，CMOS 电路的传输延迟时间和功耗如何变化？当增加工作频率时，功耗如何变化？

3.3.7 CMOS 电路的功耗电容 C_{PD} 的物理意义是什么？

3.4 类 NMOS 和 BiCMOS 逻辑门电路

3.4.1 类 NMOS 门电路

MOS 数字集成电路的发展经历了由 PMOS、NMOS 到 CMOS 的过程。PMOS 管是以空穴

为导电载流子，而 NMOS 管以电子为导电载流子，由于空穴的迁移率比电子低，因此，NMOS 电路的工作速度比 PMOS 电路快。CMOS 电路有静态功耗低、抗干扰能力强等诸多优点，但是 CMOS 门电路每增加一个输入端就要增加一个 NMOS 管和一个 PMOS 管，而且为获得同样的导通电阻和电流，PMOS 管所需的芯片面积更大。为减少电路中 PMOS 管的数目，节省芯片面积，在对性能要求不太高时，仍然采用 NMOS 电路。

NMOS 逻辑门电路全部由 N 沟道 MOS 管构成。以 NMOS 反相器为例，其工作管为增强型 MOS 管，早期负载管采用增强型 MOS 管，如图 3.4.1(a) 所示。电路开关速度慢、噪声容限小、静态功耗高。后来采用耗尽型负载管，如图 3.4.1(b) 所示。虽然耗尽型负载电路的性能不够理想，但某些特殊应用中仍然使用耗尽型负载管的 NMOS 电路。

类 NMOS(pseudo NMOS) 电路采用 PMOS 增强型负载管，栅极接地使负载管处于常通状态。类 NMOS 电路的一个优点是与 CMOS 电路相匹配。

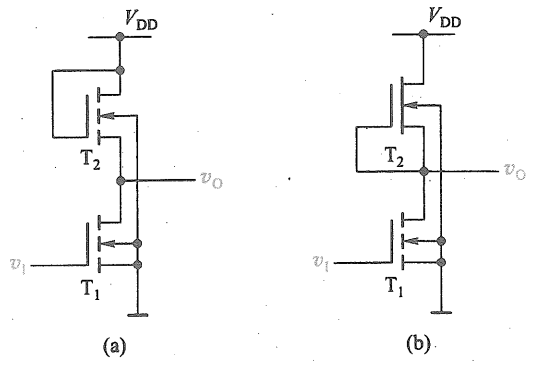


图 3.4.1 NMOS 反相器

(a) 增强型负载 (b) 耗尽型负载

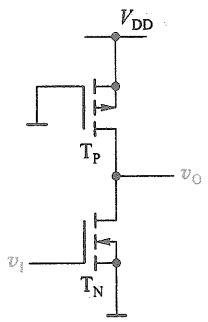


图 3.4.2 类 NMOS 反相器

1. 类 NMOS 反相器

类 NMOS 反相器电路结构如图 3.4.2 所示，其中工作管为 NMOS 管，负载管为 PMOS 管。PMOS 管的栅极接地，所以始终处于导通状态。

当输入 $v_I=0$ 时，NMOS 管截止，PMOS 管处于可变电阻区，$v_{DS2}\approx 0$，输出 $v_O\approx +V_{DD}$，此时电路的电流几乎为 0，静态功耗为 0。

当输入 $v_I=V_{DD}$ 时，NMOS 管和 PMOS 管都导通，NMOS 管工作在可变电阻区，PMOS 管工作在恒流区，输出 v_O 应为低电平，其电压值由 NMOS 管和 PMOS 管的导通电阻值分压比决定。通常在制造工艺上使 NMOS 管的导通电阻远小于 PMOS 管的导通电阻，使输出为低电平。此时的静态功耗不为 0。因此，类 NMOS 反相器多用于输出通常为高电平的电路。

2. 类 NMOS 与非门和或非门

图 3.4.3 为类 NMOS 与非门电路，其中 T_{N1}、T_{N2}、T_{N3} 和 T_{N4} 为工作管，它们串联；T_P 为负载管。当输入 A、B、C 或 D 中任一个为低电平时，与它对应的 NMOS 管截止，输出为高电平；仅当 A、B、C、D 全为高电平时，所有工作管都导通，输出才为低电平。可见电路具有与非功能，即

$$L=\overline{ABCD}$$

图 3.4.4 为类 NMOS 或非门电路，其中工作管 T_{N1}、T_{N2}、T_{N3}和 T_{N4}并联，T_P为负载管。当输入 A、B、C、D 中任一个为高电平时，与它对应的 NMOS 管导通，输出为低电平；仅当 A、B、C、D 全为低电平时，所有工作管都截止，输出才为高电平。电路具有或非功能，即

$$L=\overline{A+B+C+D}$$

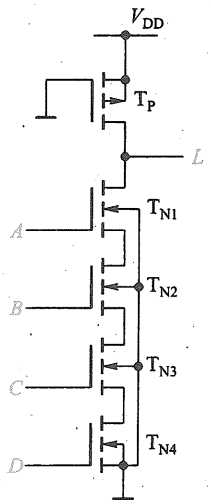


图 3.4.3 类 NMOS 与非门电路

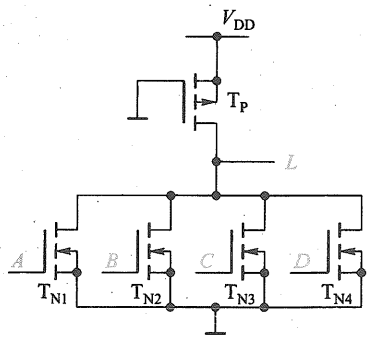


图 3.4.4 类 NMOS 或非门电路

值得注意的是，当增加与非门输入端数目时，串联的工作管数也随之增加。当输入全为高电平时，各管的导通电阻串联，使低电平输出电压升高，可能会破坏正常逻辑功能。而或非门的工作管是并联的，增加 NMOS 管的数目不会影响低电平输出电压的稳定，因而类 NMOS 电路多以或非门为基础，构成各种功能的逻辑电路。

与 CMOS 电路相比，类 NMOS 电路减少了 PMOS 管数目，输入端数目越多，减少的 PMOS 管数目也越多。例如 4 输入的 CMOS 与非门需要 8 个 MOS 管（4 个 NMOS，4 个 PMOS），而类 NMOS 只需要 5 个 MOS 管。因此，类 NMOS 与非门和或非门（也称为 NAND 结构和 NOR 结构）在大规模的只读存储器和存储器的地址译码器中得到应用。

3.4.2 BiCMOS 门电路

BiCMOS 门电路兼有 CMOS 的高密度、低功耗和双极性的高速大驱动能力。BiCMOS 产品有 BCT（BiCMOS bus-interface technology）和 ABT（advanced BiCMOS technology）系列，低电压供电的 LVT（low-voltage BiCMOS technology）系列和速度性能更好的 ALB（advanced low-voltage BiCMOS technology）系列等。

基本的 BiCMOS 反相器电路结构如图 3.4.5 所示。其中，MOS 管用符号 M 表示，BJT 用 T 表示。M_P、M_N、M_1和 M_2组成输入级，T_1和 T_2构成推拉式输出级。

输入信号 v_I为低电平时，M_N和 M_1截止，M_P导通，则 v_{B1}为高电平，使 M_2和 T_1导通，同时 v_{B2}为低电平，使 T_2截止（BJT 管的工作原理可参考 3.5.1 节），输出 v_O为高电平。

在 v_I由低电平向高电平跳变的过程中，M_P由导通向截止转换，M_1由截止向导通转变，则

v_{B1}由高电平向低电平转变，使 M_2和 T_1趋向截止。同时当 v_I升高到 $v_I > V_{TN} + V_{BE2}$时，M_N导通，在 T_1完全截止前，由 T_1通过 M_N向 T_2提供基极电流，使 T_2导通。随着 v_I到达标准高电平后，T_1可靠截止，则输出 v_O变为低电平。需要注意，由于 $v_{B2} \geqslant 0.5$ V，即使忽略 M_N的导通压降（$v_{DS}=0$），v_O的低电平也无法为 0 V。如果希望 v_O低电平接近地电位，可以采取一些改进措施①。

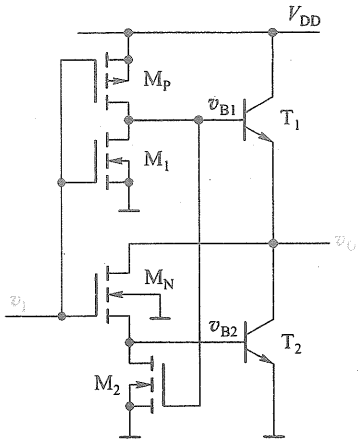


图 3.4.5 基本的 BiCMOS 反相器电路

当输出端接有比较大的容性负载时，双极型三极管输出级能为其提供足够大的充电电流。同理，已充电的电容负载也能迅速地通过 T_2放电。

电路中 M_1和 M_2的作用是加快 T_1和 T_2由饱和导通翻转到截止的过程，使 T_1和 T_2的基区存储电荷通过 M_1和 M_2释放。当 v_I为高电平时，M_1导通，T_1基区的存储电荷迅速消散。同理，当 v_I为低电平时，M_2导通，T_2基区的存储电荷通过 M_2而消散。从而，提高了门电路的开关速度。

根据前述的 CMOS 门电路的结构和工作原理，同样可以用 BiCMOS 技术实现与非门和或非门，具体电路见习题 3.4.3 和习题 3.4.4。

复习思考题

3.4.1 为什么类 NMOS 门电路多以或非门作为基本门电路？

3.4.2 为什么 BiCMOS 电路的开关速度比较快？

3.5 TTL 逻辑门电路

3.5.1 BJT 的开关特性

1. BJT 的开关作用

由 BJT 构成的开关电路如图 3.5.1(a)所示，图中 BJT 为 NPN 型硅管。图 3.5.1(b)为共射极输出特性。

当输入 $v_I = V_{IL} = 0$ V 时，BJT 的发射结为零偏（$v_{BE}=0$），集电结为反向偏置（$v_{BC}<0$），只有很小的漏电流流过 PN 结，故 $i_B \approx 0$，$i_C \approx 0$，$v_O = V_{CE} \approx V_{CC}$，对应于图 3.5.1(b)中的 A 点。这时集电极回路中的 c、e 极之间近似于开路，相当于开关断开一样，输出为高电平。BJT 的这种工作状态称为截止②。

当输入 $v_I = V_{IH} = V_{CC}$时，调节 R_b，使 $i_B = V_{CC}/(\beta R_c)$，则 BJT 工作在图 3.5.1(b)中的 B 点，集电极电流 i_C已接近最大值 V_{CC}/R_c，称为集电极饱和电流 $I_{CS} \approx V_{CC}/R_c$，对应的基极电流称为基极临界饱和电流 $I_{BS} \approx V_{CC}/(\beta R_c)$。如果再增加 i_B，则饱和程度加深，但 i_C基本上保持在

① Seder, Smith. Microelectronic Circuits[M].7th ed. New York: Oxford University Press, 2015.

② 当 v_{BE}小于开启电压（门槛电压）时，BJT 管即已进入截止区。

I_{CS}不再增加，集电极、发射极之间的电压 v_{CE} 约为 0.2～0.3 V，该电压称为 BJT 的饱和压降 V_{CES}，它也基本上不随 i_B 增加而改变。由于 V_{CES} 很小，集电极回路中的 c、e 极之间近似于短路，相当于开关闭合一样，输出为低电平。

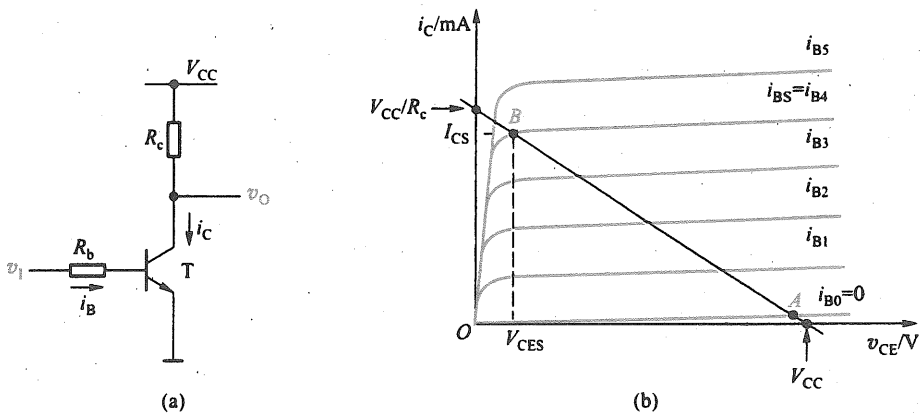


图 3.5.1 BJT 管开关电路及输出特性

(a) BJT 管开关电路 (b) BJT 管输出特性

2. BJT 的开关特性

BJT 的开关过程是其在饱和与截止两种状态之间的相互转换，也是内部电荷“建立”和“消散”的过程。因此，需要一定的时间才能完成。

当图 3.5.1(a)所示开关电路的输入端加入一个数字脉冲信号，输入信号的上升沿使 BJT 从截止到饱和变化，需要时间建立基区电荷以形成饱和电流。输入信号的下降沿使 BJT 从饱和到截止变化，需要时间消散基区存储的电荷。因此，输出电压 v_O 的变化滞后于输入电压 v_I 的变化。

3.5.2 TTL 反相器的基本电路

由于基本 BJT 反相器的动态性能不理想，为改善其动态性能，增加若干元器件构成三极管-三极管逻辑电路，即 TTL 反相器的基本电路，如图 3.5.2 所示。该电路由三部分组成，T_1 组成电路的输入级，用来提高工作速度。T_3、T_4 和二极管 D 组成输出级，以提高开关速度和带负载能力。由 T_2 组成的中间级作为输出级的驱动电路，将 T_2 的单端输入信号 v_{I2} 转换为互补的双端输出信号 v_{I3} 和 v_{I4}，以驱动 T_3 和 T_4。

当输入 $v_I=V_{IL}=0.2$ V 时，T_1 的发射结导通，其基极电压为 $v_{B1}=V_{IL}+V_{BE1}=0.9$ V，该电压作用于 T_1 的集电结和 T_2、T_3 的发射结上，所以 T_2、T_3 都截止，v_{I4} 为高电平，T_4 和 D 导通，输出为高电平。当忽略 R_{c2} 上的压降时，则 $v_O=V_{OH}\approx V_{CC}-V_{BE4}-V_D=3.6$ V。

当输入 $v_I=V_{IH}=3.6$ V 时，V_{CC} 通过 R_{b1} 和 T_1 的集电结向 T_2、T_3 提供基极电流，使 T_2、T_3 饱和导通，此时 $v_{B1}=V_{BC1}+V_{BE2}+V_{BE3}=2.1$ V，使 T_1 的发射结反向偏置，而集电结正向偏置。所以 T_1 处于发射结和集电结倒置的状态。由于 T_2 和 T_3 饱和，使 $v_{I4}=V_{CES2}+V_{BE3}=0.9$ V。该电压作用于 T_4 的发射结和二极管 D 两个 PN 结上，无法使它们导通。T_4 和 D 截止，且 T_3 饱和导通，使输出为低电平，$v_O=v_{C3}=V_{CES3}=0.2$ V。

上述电路实现了反相器的逻辑关系。

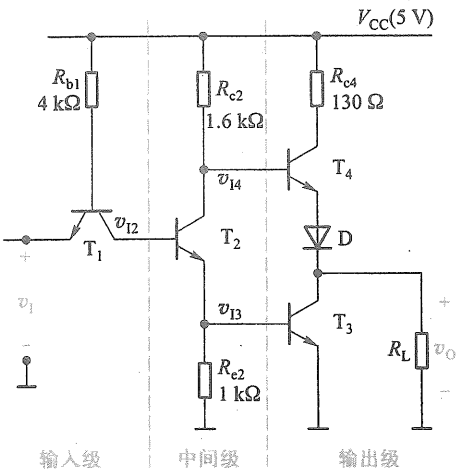


图 3.5.2 TTL 反相器的基本电路

3.5.3 抗饱和 TTL 门电路

上述 TTL 反相器中 BJT 导通时工作在深度饱和状态，这是产生传输延迟时间的主要原因，为此推出了各种系列的改进电路。早期的肖特基系列是 74S，之后推出低功耗 74LS 系列。后来改进工艺生产出 74AS 和 74ALS 系列，以及更快速的 74F 系列。改进效果比较明显的是抗饱和 TTL 电路。这种电路采用肖特基势垒二极管(Schottky-barrier-diode，SBD)的钳位作用来限制 BJT 导通时的饱和深度。

1. 肖特基势垒二极管工作原理

BJT 工作在饱和时，发射结和集电结都处在正向偏置，集电结正向偏置电压越大，则表明饱和程度越深。为了限制 BJT 的饱和深度，在 BJT 的基极和集电极并联一个肖特基二极管，如图 3.5.3(a)所示。通常把它们看成一个器件，并用图 3.5.3(b)所示符号表示。

肖特基势垒二极管的工作特点如下：

(1) 它和 PN 结一样，同样具有单向导电性。

(2) 它的导通阈值电压较低，约为 0.4～0.5 V，比普通硅 PN 结约低 0.2～0.3 V。

(3) 势垒二极管是多数载流子参与导电，没有少数载流子的积累，因而从正向导通到反向截止，没有内部电荷的建立和消散过程，使转换速度加快。

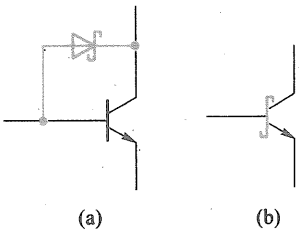


图 3.5.3 带有肖特基二极管钳位的 BJT
(a) 电路结构形式 (b) 符号

当 BJT 集电结正向偏置时，SBD 先导通，对流向基极的电流分流，以避免 BJT 基极电流过大导致深度饱和，因而这种电路是抗饱和电路，它能使电路的开关时间大为缩短。

2. 低功耗肖特基 TTL 与非门

低功耗肖特基 TTL 系列逻辑门电路中，有反相器、与门、与非门、或门、或非门等。图

3.5.4 所示为 74LS 系列 2 输入**与非**门(74LS00)的电路结构。与基本的 TTL 反相器电路相比,作了若干改进。

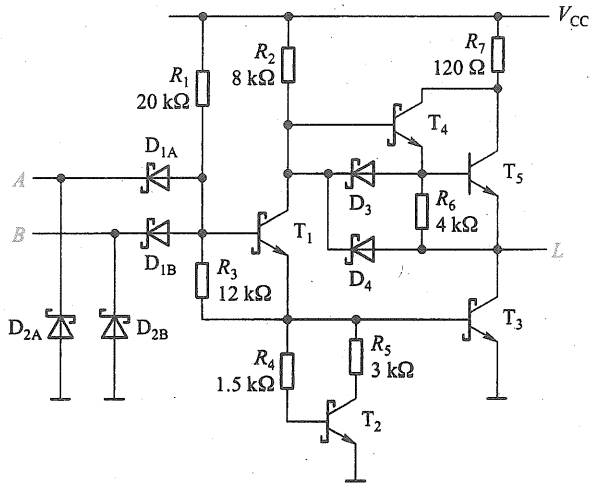


图 3.5.4 74LS00 中的一个与非门电路

首先是除 T_5外,其余的 BJT 均采用 SBD 钳位,以达到明显的抗饱和效果。其次输入级用 SBD 代替 BJT 管,由于 SBD 没有电荷存储效应,有利于提高工作速度。

肖特基 TTL 门电路还从以下三点对基本 TTL 电路(图 3.5.2)的性能做了改进:

(1) 由 T_4和 T_5所组成的复合管代替了基本 TTL 电路中的二极管 D 和 T_4,当输出由低电平向高电平过渡时,由于复合管电路的电流增益很大,输出电阻很小,从而减小了电路对负载电容的充电时间。

(2) 电路输入端所加的 SBD D_{2A}和 D_{2B},用来减小门电路之间的连线而引起的杂散信号,并防止输入信号反向过冲使 T_1电流过大而损坏。

(3) 由 T_2与 R_4、R_5组成的有源电路代替了基本 TTL 电路中的 R_{e2}(1 kΩ)。当电器输入端 A、B 所加信号使 T_1 的基极由低电平转向高电平时,T_1由截止变为导通,由于 T_2的基极回路串接了电阻 R_4,T_2的导通滞后于 T_3,使 T_1以较大的电流驱动 T_3,从而加快了 T_3的导通过程。随后,T_2开始导通,将对 T_3的基极电流产生分流作用,减轻了 T_3的饱和程度,当电路再次翻转时,T_3能很快地截止。因而,有源电阻缩短了门电路的转换时间,使其电压传输特性得到改善。

当输入 A、B 中有一个为低电平时,T_1、T_2、T_3截止,T_4和 T_5导通,输出为高电平。当输入 A、B 均为高电平时,T_1、T_2、T_3导通,T_4和 T_5截止,输出为低电平。电路实现**与非**功能。

与 CMOS 逻辑门电路类似,TTL 门电路也有另外两种不同输出结构形式的电路,集电极开路(open collector,OC)门和三态输出门。OC 门是将 TTL 门电路输出级的集电极开路,只有外接上拉电阻电路才能正常工作。外接电阻的计算与 OD 门类似。TTL 三态门也是在普通门电路的基础上,增加控制电路构成的,电路的输出状态有高电平、低电平和高阻三种状态。OC 门和三态门的 TTL 具体电路这里不再赘述。

复习思考题

3.5.1 在数字电路中，BJT 管作为开关使用时，工作在其输出特性曲线的什么区？

3.5.2 抗饱和 TTL 电路为什么可以提高开关速度？

3.6 逻辑描述中的几个问题

3.6.1 正负逻辑问题

1. 正负逻辑的规定

在数字电路中，可以采用两种不同的逻辑体制表示电路输入和输出的高、低电平。在前面讨论时，将高电平用逻辑 1 表示，低电平用逻辑 0 表示，这种表示方法称为正逻辑体制。如果将高电平用逻辑 0 表示，低电平用逻辑 1 表示，则这种表示方法称为负逻辑体制。

对于同一电路的输入与输出关系的描述，可以采用正逻辑，也可以采用负逻辑。正逻辑和负逻辑两种体制不牵涉逻辑电路本身的结构问题，但根据所选正负逻辑的不同，即使同一电路也具有不同的逻辑功能。例如某个逻辑门电路的输入和输出电平如表 3.6.1 所示，其中 H 和 L 分别表示高、低电平。如果采用正逻辑体制，令 H 为 1，L 为 0，得到如表 3.6.2 所示的真值表，它表示与非逻辑关系 $L=\overline{A\cdot B}$。如果采用负逻辑体制，令 H 为 0，L 为 1，得到如表 3.6.3 所示的真值表，它表示或非逻辑关系 $L=\overline{A+B}$。因此，正逻辑的与非门等效于负逻辑的或非门。正逻辑和负逻辑只是看问题的角度或分析问题的方法不同而已，问题的实质是不变的，即电路输入与输出的电平关系始终是不变的。本书如无特殊说明，一律采用正逻辑，即规定高电平为逻辑 1，低电平为逻辑 0。

表 3.6.1 某电路输入与输出电平表

A	B	L
L	L	H
L	H	H
H	L	H
H	H	L

表 3.6.2 正与非门真值表

A	B	L
0	0	1
0	1	1
1	0	1
1	1	0

表 3.6.3 负或非门真值表

A	B	L
1	1	0
1	0	0
0	1	0
0	0	1

2. 正负逻辑的等效变换

工程实践中，电路描述一般采用正逻辑体制，负逻辑体制用得比较少。上述分析可知，正逻辑的与非对应负逻辑的或非；同理，正逻辑的或非与负逻辑的与非相对应。因此，如果需要，可以按下列方式进行两种逻辑体制的互换：

与非⇔或非

与⇔或

非⇔非

3.6.2 基本逻辑门的等效符号及其应用

1. 基本逻辑门的等效符号

利用摩根定理对基本逻辑运算进行变换，可以得到不同形式的表达式。例如与非逻辑运算的表达式可以写成

$$L=\overline{AB}=\overline{A}+\overline{B}$$

由此，可以得到与非门的等效符号如图 3.6.1 所示。输入端的小圆圈表示先对信号作非运算。

对于或非运算的逻辑表达式，可以写成

$$L=\overline{A+B}=\overline{A}\cdot\overline{B}$$

所以得到其等效符号如图3.6.2 所示。

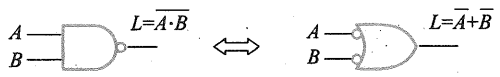


图 3.6.1 与非门及其等效符号

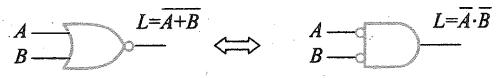


图 3.6.2 或非门及其等效符号

同理，利用摩根定理对与门和或门的逻辑表达式进行变换，可以得到它们的等效符号分别如图 3.6.3 和图 3.6.4 所示。

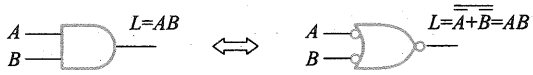


图 3.6.3 与门及其等效符号

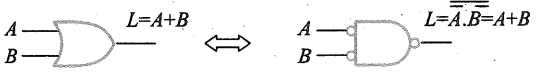


图 3.6.4 或门及其等效符号

上述各图所示的逻辑符号及其等效符号，是在同一逻辑体制下，用两种不同的方式描述同一逻辑运算。因此，不能将等效符号看成是负逻辑体制或者负逻辑表示方法。本书采用正逻辑体制，所以对于输入和输出均是高电平为 1，低电平为 0。可以用真值表验证各逻辑符号及其等效符号是等价的。

2. 逻辑门等效符号的应用

由于与门是由与非门和非门级联构成的，或门是由或非门和非门构成的。所以与非门和或非门比与门和或门电路结构简单，速度快。利用逻辑门等效符号对逻辑电路进行变换，很容易简化电路，减少实现电路的门的种类，提高工作速度。

图 3.6.5(a)所示电路由两级组成，第一级是两个与门，第二级是一个或门。

利用 $\overline{\overline{X}}=X$，在图 3.6.5(a)中间连线的两端各加一个圆圈，相当于进行两次非运算，但并没有改变电路的功能，得到图 3.6.5(b)所示电路。然后将图 3.6.5(b)所示电路第二级与非门的等效符号用与非门符号代替，就可以得到图 3.6.5(c)所示的全部由与非门构成的电路。该电路的工作速度比图 3.6.5(a)所示电路快。

3. 逻辑门等效符号强调低电平有效

在 3.3.2 节介绍三态门时，就涉及有效电平的概念。三态门的使能控制信号可以是高电平有效，也可以是低电平有效。对于高电平使能的三态门，当使能端信号为 1 时，电路处

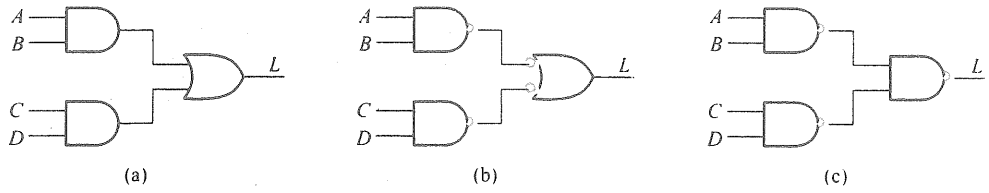


图 3.6.5 或门及其等效符号

(a)逻辑电路 (b)逻辑电路的等效变换 (c)用与非门替代等效符号

于正常逻辑工作状态;对于低电平使能的三态门,当使能端信号为 0 时,电路正常工作。有效电平的概念不止限于使能端信号。在实际电路,特别是大规模集成芯片中,任何输入或者输出信号都有可能是高电平有效,或者是低电平有效。所谓低电平有效,是指当信号为低电平时,电路完成规定的操作;而高电平有效,是指信号为高电平时,电路完成规定的操作。

图 3.6.6 所示是一个可以控制数据传输的电路。其中集成芯片 IC 的使能端$\overline{EN}$要求低电平有效,电路的两个控制信号分别是请求信号 RE 和允许信号$\overline{AL}$。图中,G_2门是输入、输出均为低有效的与门。根据图 3.6.4 可知,G_2门实际是或门的等效符号。这里之所以采用等效符号是为了强调低电平有效,以便于理解实际电路中,信号 RE、信号$\overline{AL}$以及 IC 芯片的使能信号$\overline{EN}$之间的逻辑关系。当 RE 为高电平信号,$\overline{AL}$为低电平信号时,G_2门的两个输入端均为有效信号,即低电平,则产生一个有效的输出信号,即$\overline{L}$为低电平,使$\overline{EN}$为低电平,允许 IC 传输数据。

信号名称$\overline{EN}$、$\overline{AL}$和$\overline{L}$上面的横线表示低电平有效,在进行逻辑运算时,作为一个整体符号。如果在运算过程中,变量上面的“-”号参与运算,则在画逻辑电路图或者验证真值表时,应将其还原为低有效符号。

需要注意的是,如果一根连线的两端都有圆圈,并且都包含有非运算的含义,可以用“圈圈相消”进行电路化简,如图 3.6.5 所示。但在图 3.6.6 中,G_2门的输出与使能端之间的连线两端也有圆圈,这两个圆圈不能抵消,因为集成芯片 IC 使能端的圆圈的功能在芯片内部,不能被化简掉。

如果要求信号 RE 和信号 AL 为高电平有效,而使能端$\overline{EN}$仍为低有效,可以采用如图 3.6.7(a)所示的控制电路。G_2门可以看成是输入为高电平有效,输出为低电平有效的与门。

如果要求信号$\overline{RE}$和信号$\overline{AL}$均为低电平有效,而使能端 EN 为高有效,则采用如图 3.6.7(b)所示的控制电路。G_2门可以看成是输入为低电平有效,输出为高电平有效的与门。根据图 3.6.2 可知,G_2门是或非门的等效符号。

同理,如果要求信号 RE 和信号 AL 均为高电平有效,使能信号 EN 也为高电平有效,则采用如图 3.6.7(c)所示的控制电路。G_2门为输入、输出均是高电平有效的与门。

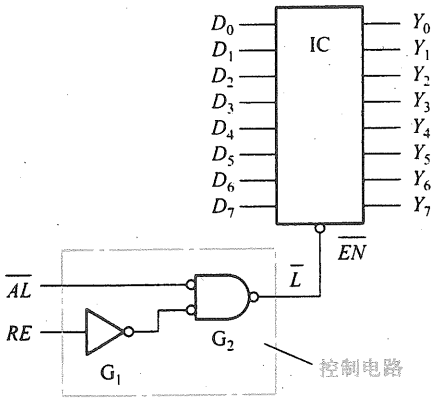


图 3.6.6 数据传输控制电路

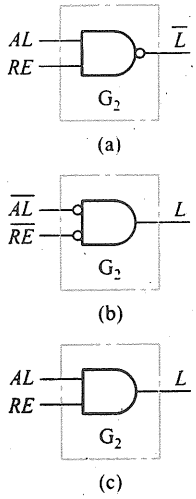


图 3.6.7 几种不同的控制电路

(a) 与非门实现 (b) 或非门实现 (c) 与门实现

复习思考题

3.6.1 列出正逻辑体系或非门和负逻辑体系与非门的真值表,并说明两者的等效关系。

3.6.2 为什么说逻辑符号及其等效符号不是正负逻辑关系?

3.6.3 对于小圆圈画在输入端或者画在输出端的非门,其逻辑运算结果是否相同,所表达的含义是否相同?

3.7 逻辑门电路使用中的几个实际问题

以上重点讨论了 CMOS 系列门电路,同时还介绍了其他几种逻辑门电路。在实际应用中,可以根据电源电压、传输延迟时间、功耗、噪声容限、带负载能力等要求来选择相应的门电路。有时需要混合使用不同系列的逻辑门电路,此时将遇到不同逻辑门电路之间的接口问题。另外,门电路与负载之间的匹配也是需要考虑的问题之一。下面对几个实际问题进行讨论。

3.7.1 各系列逻辑门电路之间的接口问题

CMOS 电路的动态功耗为 $P_D=(C_{PD}+C_L)V_{DD}^2f$。为了减小功耗,采用低电源电压器件。同时电路的集成度不断提高使晶体管尺寸越来越小,CMOS 的栅极与源极、栅极与漏极间的绝缘层也越来越薄,不足以承受 5 V 电源电压。JEDEC(joint electron device engineering council)固态技术协会选择 3.3 V±0.3 V、2.5 V±0.2 V、1.8 V±0.15 V、1.5 V±0.1 V、1.2 V±0.1 V和 1.0 V±0.1 V 等一系列电压作为各种低电压逻辑器件的供电电源,并规定了这些供电电源下输入和输出高、低逻辑电平的电压值。

图 3.7.1 给出了各个系列在给定电源电压下四个逻辑电平参数：输入低电平的上限值 $V_{IL(max)}$、输入高电平的下限值 $V_{IH(min)}$、输出低电平（$I_{OL}=4$ mA）的上限值 $V_{OL(max)}$、输出高电平（$I_{OH}=4$ mA）的下限值 $V_{OH(min)}$，便于进行逻辑电平的快速比较。为简单起见，图中省略了下标 min 和 max。

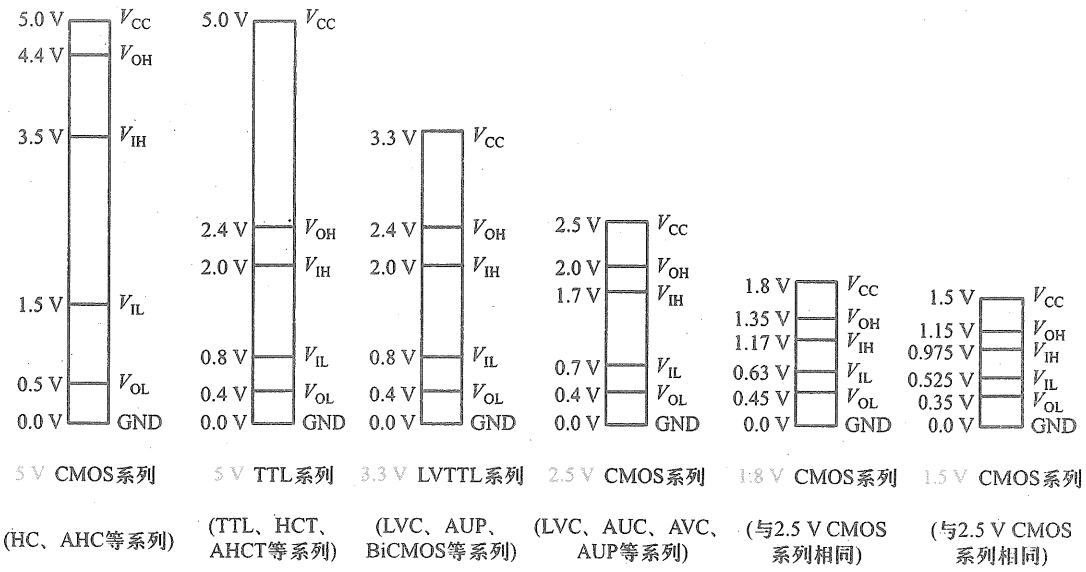


图 3.7.1 几种逻辑电平比较

在图 3.7.1 中，3.3 V LVTTL 是指 3.3 V CMOS 低电压器件所带 TTL 负载时的电压值。逻辑门的负载不同时，其输出高、低电平的下限值和上限值也不同。当其输出电流 I_O 低于 100 μA 时称为带 CMOS 负载。当 I_O 比较大时称为带 TTL 负载。

图 3.7.1 给出的是典型的逻辑电平值，不同系列电路的具体参数可能与典型值不同，分析具体的实际问题时，必须查阅制造商提供的数据表。

现在的很多数字电路或数字系统中，同时采用不同供电电压的逻辑器件，主要是为了降低成本，兼顾新老器件或设备的应用。不同系列逻辑器件的混合使用，需要考虑它们的匹配问题。

当一个系统中使用不同系列逻辑门电路时，由于它们的电压和电流参数各不相同，首先要考虑门电路的输入或输出电平是否超过数据手册规定的极值，即

$$V_{I(min)} \leqslant V_I \leqslant V_{I(max)} \tag{3.7.1}$$

$$V_{O(min)} \leqslant V_O \leqslant V_{O(max)} \tag{3.7.2}$$

如果不满足上述两个条件，有可能损坏芯片。

第二是逻辑电平兼容性问题，驱动器件的输出电压必须满足负载器件所要求的高电平或者低电平输入电压的范围。即

$$V_{OH(min)} \geqslant V_{IH(min)} \tag{3.7.3}$$

$$V_{OL(max)} \leqslant V_{IL(max)} \tag{3.7.4}$$

第三是逻辑门电路的扇出问题，即驱动器件必须能对负载器件提供足够的灌电流或者

拉电流。

灌电流情况下应满足 $$I_{OL(max)} \geqslant |I_{IL(total)}| \tag{3.7.5}$$

拉电流情况下应满足 $$|I_{OH(max)}| \geqslant I_{IH(total)} \tag{3.7.6}$$

其余如噪声容限、输入和输出电容以及开关速度等参数在某些设计中也必须予以考虑。下面分别就不同供电电压的逻辑电路之间的接口问题进行讨论。

1.5 V CMOS 电路与 3.3 V CMOS 电路的接口

两种供电电源的 CMOS 电路相连时，首先要考虑它们的输入输出电平是否满足式(3.7.1)和式(3.7.2)的要求。

(1) 输入电压的极值

有些逻辑门电路允许输入 V_I 超过电源电压 V_{DD}，而有些不允许。

例如：74HC 系列的最大输入 $V_{I(max)} = V_{DD} + 0.5$ V，即输入 V_I 不能超过电源电压 V_{DD}。图 3.7.2 所示为 74HC 系列的输入保护电路。当 $-0.5\text{V} \leqslant V_I \leqslant V_{DD} + 0.5$ V 时，D_1 和 D_2 均截止，电路正常工作。当 V_I 超出此范围时，二极管 D_1 或 D_2 导通，并且流过导通二极管的电流很大，电路不能正常工作。

74AHC 系列 $V_{I(max)} = 7$ V。当 $V_{DD} = 3.3$ V 时，允许输入电压 $V_I > 3.3$ V，只要小于 7 V 即可。74AHC 系列输入保护电路如图 3.7.3 所示。该电路去掉了输入到电源之间的二极管，因此允许 V_I 高于 V_{DD}，同时要求电路的 MOS 管能够承受足够高的电压。

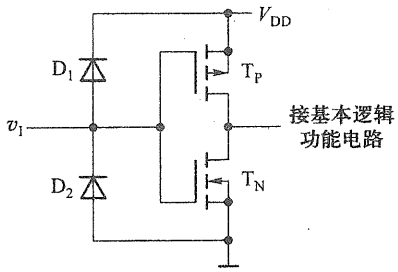


图 3.7.2 74HC 系列的输入电路

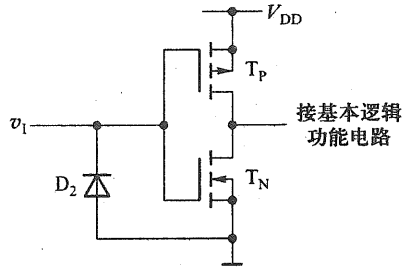


图 3.7.3 74AHC 系列的输入电路

输入电压的极小值 $V_{I(min)}$ 通常为 0 V。如果考虑输入保护二极管的作用时，$V_{I(min)} = -0.5$ V。

(2) 输出电压的极值

有些逻辑门电路允许输出 V_O 超过电源电压 V_{DD}，而有些则不允许。特别是三态输出的 5 V 和 3.3 V 两种逻辑电路接到同一个数据总线时，如果 5 V 逻辑电路输出高电平，而 3.3 V 逻辑电路处于高阻态。5 V 高电平输出信号通过总线也连接到 3.3 V 逻辑电路的输出端，此时需要考虑输出端允许的极限值。

例如：74HC 和 74AHC 系列电路的最大输出 $V_{O(max)} = V_{DD} + 0.5$ V，即输出 V_O 不能超过电源电压 V_{DD}。图 3.7.4 所示为 74HC/74AHC 系列三态输出电路，$V_{DD} = 3.3$ V。输出高阻态时，T_N 和 T_P 管均截止。当 5 V 电压加到输出端时，使二极管 D_1 正偏导通，T_P 的漏极与源极之间形成一个低阻通路，并且产生很大的电流，电路输出不是高组态了。

对于 74LVC 等系列，当 $V_{DD} = 3.3$ V 时，最大输出 $V_{O(max)} = 6.5$ V。即其输出电压 V_O 可以

高于 3.3 V，只要小于 6.5 V 即可。图 3.7.5 所示为带保护电路的 74LVC 系列的三态输出电路结构示意图。当输出端电压高于 V_{DD}+0.5 V 时，比较器使 S_1 开路，S_2 闭合，电流通路消失，其输出仍然保持高阻态，就能与 5 V 器件相接。

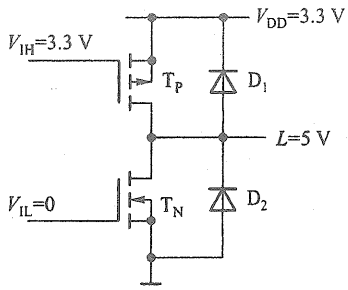


图 3.7.4 74HC 和 74AHC 系列三态输出电路

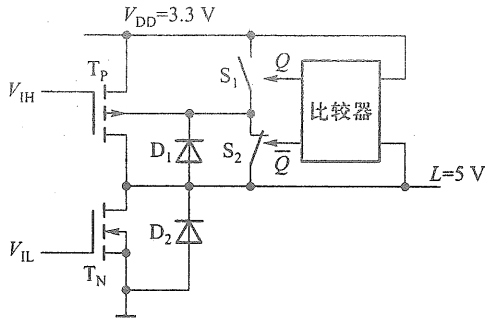


图 3.7.5 74LVC 系列的三态输出电路结构示意图

输出电压的极小值 $V_{O(min)}$ 通常为 0 V，如果考虑输入保护二极管的作用时，$V_{O(min)}$ = −0.5 V。

(3) 逻辑电平兼容性及输入输出电流匹配问题

① 5 V CMOS 系列驱动 3.3 V CMOS 系列

根据表 3.3.4 给出的数据，5 V 系列 74HC/AHC 的输出电压为 $V_{OH(min)}$ = 4.9 V，$V_{OL(max)}$ = 0.1 V，3.3V 系列 74LVC/ALVC 的输入电压为 $V_{IH(min)}$ = 2.0 V，$V_{IL(max)}$ = 0.8 V，满足式(3.7.3)和式(3.7.4)对逻辑电平的要求。

根据表 3.3.4 给出的电流参数可知，5 V CMOS 系列的高电平最大输出电流 $I_{OH(max)}$ 和低电平最大输出电流 $I_{OL(max)}$ 都在 20 μA 以上，3.3 V 系列的高、低电平最大输入电流 $I_{IH(max)}$ 和 $I_{IL(max)}$ 均为 5 μA，只要负载数目适当，即可满足式(3.7.5)和式(3.7.6)对输入输出电流的要求。

因此，5 V CMOS 系列可以直接驱动 3.3 V CMOS 系列。

② 3.3 V CMOS 系列驱动 5V CMOS 系列

根据表 3.3.4 给出的数据，3.3V 系列输出低电平电压为 $V_{OL(max)}$ = 0.2 V，5 V 系列输入低电平电压为 $V_{IL(max)}$ = 1.5 V，满足式(3.7.4)的要求。但是 3.3V 系列 $V_{OH(min)}$ = 2.8 V，低于 5 V 系列的 $V_{IH(min)}$ = 3.5 V，不满足式(3.7.3)的要求。为了解决这一矛盾，最简单的方法是在3.3 V CMOS 电路的输出端与+5 V 电源之间接一个上拉电阻 R_p，如图 3.7.6 所示。R_p 的取值可以用 OD 门外接上拉电阻的计算方法。因此有

$$V_{OH} = V_{DD} - R_p(I_{OZ} + I_{IH(total)}) \qquad (3.7.7)$$

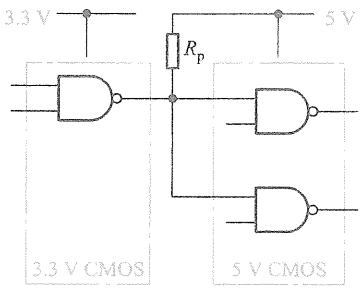


图 3.7.6 用上拉电阻提高 3.3 V CMOS 电路的输出高电平

I_{OZ} 为 3.3 V CMOS 门输出级 T_N 管截止时的漏电流，$I_{IH(total)}$ 为流入全部 5 V CMOS 负载电路的电流，这两个电流的数值都很小，如果 R_p 取值不太大，驱动门输出将被提高至接近 V_{DD}。但需注意，此时驱动门输出端必须能够承受提升的 V_{OH} 值。

除了用上拉电阻的方法外，还可以采用 OD 门或专门的逻辑电平转换器。

2. 低电压 CMOS 电路之间的接口

为满足不同系列逻辑电路之间的接口需求，特别是 2.5 V 以下逻辑电路之间的接口，通常使用专门的逻辑电平转换器，如图 3.7.7 所示。电平转换器有两个独立的电源 V_{DDA} 和 V_{DDB}，分别与两种系列逻辑电路的电源相接。

例如 74LVC1T45 是小逻辑封装（参看 3.7.5 节）的 1 位双向逻辑电平转换器，并具有驱动总线的三态输出结构，可以实现 5 V、3.3 V、2.5 V、1.8 V 或 1.5 V 逻辑电平之间的相互转换，电路如图 3.7.8 所示。当控制端 *DIR* 接 V_{DDA} 时，则 *A* 端信号传到 *B* 端，*DIR* 接地时，则 *B* 端信号传到 *A* 端。

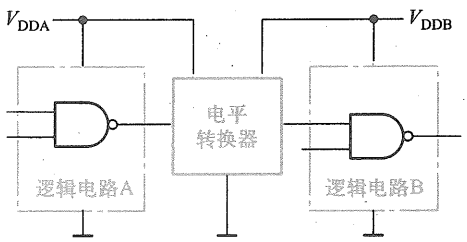


图 3.7.7 电平转换器用作接口

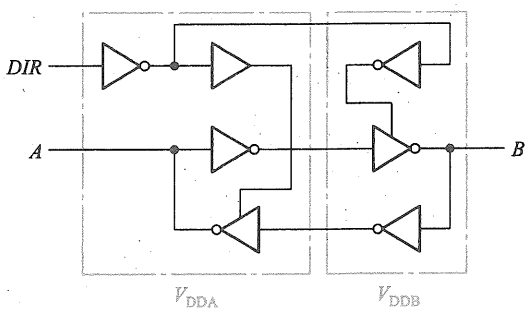


图 3.7.8 74LVC1T45 电平转换器内部电路图

如果同时需要多位逻辑电平转换器，可使用宽总线封装的芯片。例如 74ALVC164245 是 16 位双向逻辑电平转换器，具有驱动总线的三态输出结构，集成芯片内部是将 16 位电平转换器分成 2 组，每组有 8 位，组与组之间相互独立，其中一组中 1 位的结构如图 3.7.9 所示。74ALVC164245 功能表如表 3.7.1 所示。控制端 *DIR* 和 $\overline{OE}$ 接左侧 V_{DDA} 一边的电源或地，以控制信号的传输方向。它可以接收 5 V 或 3.3 V 供电电压的逻辑电平，输出 2.5 V 或 1.8 V 供电电压的逻辑电平。反之，也可以接收 2.5 V 或 1.8 V 供电电压的逻辑电平，输出 5 V 或 3.3 V 供电电压的逻辑电平。

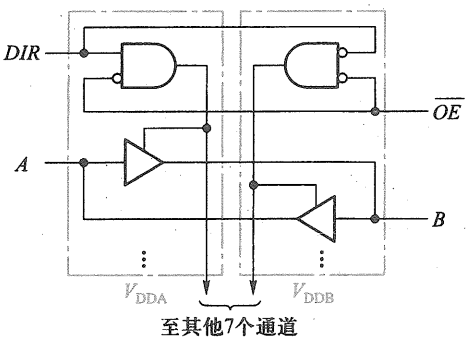


图 3.7.9 $\frac{1}{16}$ 74ALVC164245 电平转换器内部电路图

表 3.7.1 74ALVC164245 的功能表

输入		操作
$\overline{OE}$	DIR	
L	L	数据 B 传送到 A
L	H	数据 A 传送到 B
H	×	隔离

3.7.2 逻辑门电路驱动其他负载时的接口

1. 驱动显示器件

在数字电路中，往往需要用发光二极管来显示信息，如电源接通或者断开的指示、七段数码显示或图形符号显示等。

图 3.7.10 所示为反相器驱动一发光二极管 LED 电路。电路中串接了一限流电阻 R 以保护 LED。限流电阻的大小可分别按下面两种情况来计算。

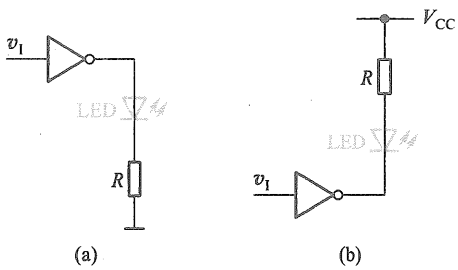


图 3.7.10 反相器驱动 LED 电路

对于图 3.7.10(a)所示电路，当门电路的输出为高电平时，LED 发光，则

$$R=\frac{V_{OH}-V_F}{I_D} \tag{3.7.8}$$

反之，图 3.7.10(b)所示电路，当门电路的输出为低电平时 LED 发光，故有

$$R=\frac{V_{CC}-V_F-V_{OL}}{I_D} \tag{3.7.9}$$

以上两式中，I_D为 LED 的电流，V_F为 LED 的正向压降，V_{OH}和 V_{OL}为门电路的输出高、低电平电压值。

对于图 3.7.10(a)，如果非门输出电流 $I_{OH}>I_D$，发光二极管会比预计的稍亮一些，但不会有什么危害。图 3.7.10(b)也类似。

如果需要的电流比较大，或驱动大电容负载时，可以加一级缓冲器。缓冲器中三极管的尺寸比一般逻辑门中三极管的尺寸大，可以提供大电流。

例 3.7.1 试用 74HC04 六个 CMOS 反相器中的一个作为接口电路，当门电路的输入为高电平时，使 LED 导通发光。已知 LED 正常发光需要几毫安的电流，并且导通时的压降 V_F为 1.6 V。

解：根据表 3.3.4 中 74HC 系列的数据，当 $V_{CC}=5$ V 时，$V_{OL}=0.33$ V，$I_{OL(max)}=4$ mA，因此 I_D取值不能超过 4 mA。采用的电路如图 3.7.10(b)所示，根据式(3.7.9)计算限流电阻的最小值为

$$R=\frac{(5-1.6-0.33)\text{ V}}{4\text{ mA}}=768\ \Omega$$

2. 驱动机电性负载

在工程实践中，往往会遇到用各种数字电路来控制机电性系统的功能，如控制电动机的转角和转速，继电器的接通与断开，流体系统中阀门的开通和关闭，自动生产线中的机械手多参数控制等。这些机电系统所需的工作电压和工作电流比较大，即使微型继电器的驱动电流也会在 10 mA 以上。要使这些机电系统正常工作，必须扩大驱动电路的输出电流以提高带负载能力，而且必要时要实现电平转移。

如果负载所需的电流不特别大，例如微型继电器，可以将两个反相器并联后的拉电流作为驱动电流，如图 3.7.11 所示。二极管 D 用来钳位电感产生的反电动势。即使封装在同一芯片内的两个反相器的参数也有差别，因此，并联后总的最大负载电流略小于单个门最大负载电流的两倍。

如果负载所需的电流比较大，达到几百毫安，则需要在数字电路的输出端与负载之间接入一个功率驱动器，称之为外围驱动器。

功率驱动器有多种类型，但电路结构一般都具有以下两个特点：一是采用集电极开路输出结构，其输出高电平几乎等于外加电压，通过调节外加电压来满足不同负载对高电平电压的要求。二是驱动电路的输出三极管具有较强的带负载能力，能提供较大的电流。具体外围驱动器的电路结构可查阅有关数据手册。

达林顿晶体管阵列 ULN2003A 作为驱动机电性负载的功率驱动器如图 3.7.12 所示，其内部有 7 组由达林顿管构成的 OC 反相器及二极管 D 组成的电路，这里只画了 1 组。若需更大的电流也可以将 2 组并联使用。ULN2003A 的输入与 CMOS 或 TTL 电路兼容，输出最大驱动电流为 500 mA，最高驱动电压为 50 V，输出端可直接用于驱动机电系统。

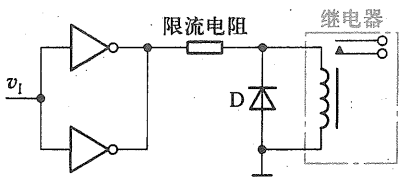


图 3.7.11 用并联逻辑门提高驱动能力

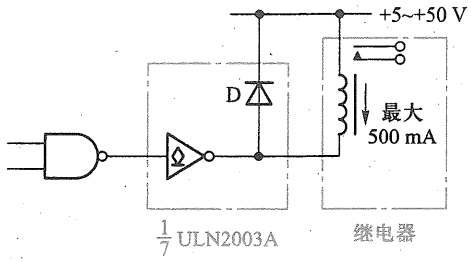


图 3.7.12 用功率驱动器提高驱动能力

3.7.3 抗干扰措施

利用 CMOS 逻辑门电路作具体的电路设计时，还应当注意下列几个实际问题。

1. 多余输入端的处理措施

CMOS 逻辑门电路在使用时，多余输入端绝对不能悬空。由于它的输入电阻很大，容易受到静电或工作区域工频电磁场引入电荷的影响，从而破坏电路的正常工作状态。

对多余输入端的处理以不改变电路工作状态及稳定可靠为原则，如图 3.7.13 所示。一是将**与门或者与非门**的多余输入端通过 1~10 kΩ 限流电阻接电源，**或门或者或非门**的多余输入端通过 1~10 kΩ 限流电阻接地。最好不要省去限流电阻，因为有些逻辑门在状态转换时，输入电流有可能超过限定数值，或者使得测试或调试更加困难。

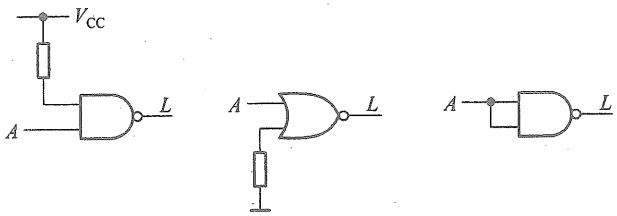


图 3.7.13 多余输入端的处理电路

工作频率比较低时，可以将多余输入端与其他输入端并接在一起。对于高速电路的设计，输入端并接会增加等效的电容性负载，而使信号的传输速度下降，最好采用前两种方法。

FPGA 等大规模集成芯片的输入端内部有可编程配置的上拉或下拉电阻，多余输入端不需要在外部进行处理。

2. 去耦合滤波电容

数字电路或数字系统往往由多片逻辑电路构成。它们由一公共的直流电源供电。这种电源是非理想的，一般由整流稳压电路供电，具有一定的内阻抗。当数字电路在高、低状态之间交替变换时，会产生较大的脉冲电流或尖峰电流。当它们流经公共的内阻抗时，必将产生相互影响，甚至使逻辑功能发生错乱。一种常用的处理方法是采用去耦合滤波电容，用 10~100 μF 的大电容器接在直流电源与地之间，滤除干扰信号。除此以外，对于每一集成芯片的电源与地之间接一个 0.1 μF 的电容器以滤除开关噪声。

3. 接地和安装工艺

正确的接地技术对于降低电路噪声是很重要的。方法是将电源地与信号地分开，先将信号地汇集在一点，然后将二者用最短的导线连在一起，以避免含有多种脉冲波形（含尖峰电流）的大电流引到某数字器件的输入端而破坏系统正常的逻辑功能。

此外，当系统中同时有模拟和数字两种器件时，同样需将二者的地分别连在一起，然后再选用一个合适共同点接地，以避免二者之间的影响。必要时，也可设计模拟和数字两块电路板，各备直流电源，然后将二者的地恰当地连接在一起。

在印刷电路板的设计或安装中，要注意连线尽可能短，以减少接线电容产生寄生反馈而引起的寄生振荡。这方面更详细的介绍，可参阅有关文献。某些典型电路应用设计也可参考集成数字电路的数据手册。

3.7.4 差分信号传输

差分信号传输（简称差分传输）是一种抗干扰能力强的信号传输技术。电视机、计算机、数字音响等设备的 HDMI（high-definition multimedia interface）接口及 USB 接口的数据传输均采用差分传输。低电压差分传输已应用到 FPGA、ASIC 等芯片内部。

传统的信号传输采用一根信号线和一根地线，信号线上传输高电平或低电平电压值。若叠加干扰信号后电压值改变，有可能导致逻辑电平发生变化。

差分信号传输是在两根线上都传输信号，两个信号的大小相等、相位相反。差分信号发送与接收系统结构示意图如图 3.7.14 所示。

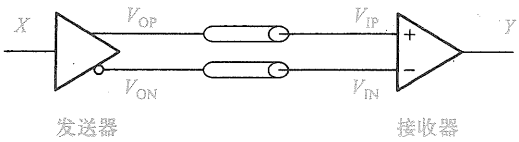


图 3.7.14 差分信号传输示意图

原始数字信号在输入端经过缓冲器和反相器,分别输出两个大小相等、相位相反的信号到两根传输线上。在电路板上,两根传输线必须等长、等宽、紧密靠近且在同一层面。当干扰信号同时作用时,叠加在两根传输线上的干扰信号(共模信号)大小相等、相位也相同。在接收端用差分放大器来检测差分信号,两个输入端信号相减后,干扰信号被抵消。因此,差分传输具有很强的抗干扰能力。

当 $X=0$ 时,V_{OP}为低电平,V_{ON}为高电平。经过传输线到接收端,$V_{IP}-V_{IN}<0$ 时,$Y=0$。

当 $X=1$ 时,V_{OP}高为电平,V_{ON}为低电平。在接收端,$V_{IP}-V_{IN}>0$ 时,$Y=1$。

3.7.5 CMOS 电路中的小尺寸逻辑和宽总线系列

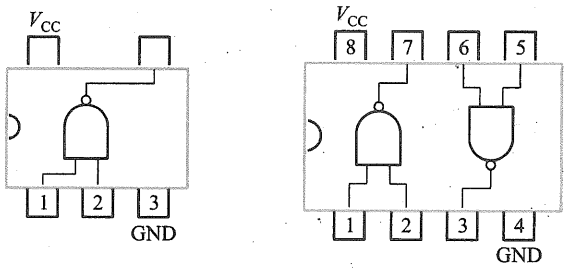
小尺寸逻辑(little logic)系列比传统通用逻辑芯片的体积小很多。这种芯片不是用来构成电路或系统,而是用来修改或完善大规模集成芯片之间连线或外围电路的连线。

另外,微处理器和计算机的进一步发展,要求性能更优越的总线驱动器件,出现了宽总线(widebus)系列。实际应用中,各种高速服务器、通信和网络设备也需要不同规格的小尺寸逻辑或宽总线逻辑器件,以满足它们的灵活性要求。因此,小尺寸逻辑和宽总线电路成为通用型逻辑器件的主流器件,但比传统的通用集成电路系列的品种少得多。

1. 小尺寸逻辑电路

小尺寸逻辑器件是将 1 个、2 个或 3 个门封装在一个芯片中构成的超小规模集成电路。

以 2 输入**与非**门为例,传统的通用集成电路采用 TSSOP(thin shrink small outline package)封装技术,将 4 个门封装在一个芯片中,面积为 33.66 mm^2。小尺寸逻辑电路是将 1 个或 2 个门分别采用 Nano Star 和 Micro QFN(quad flat non-leaded package)封装,面积分别减少了 97.3%和 95.7%,如图 3.7.15 所示。



Nano Star封装
面积为0.9 mm^2

Micro QFN封装
面积为1.4 mm^2

图 3.7.15 小尺寸逻辑封装的 2 输入与非门

采用小尺寸逻辑电路可以减小体积,提高工作速度。例如,某电路需要插入一个 2 输入**与非**门,当使用传统的通用集成芯片时,就会有 3 个**与非**门闲置不用造成浪费。即使需要多个门,如果需要分布在较远的位置,会造成布线困难。小尺寸逻辑芯片不仅大大减少了面积,而且可以降低布线对电路工作性能的影响。

单门、双门和三门系列分别以 1G、2G 和 3G 表示。逻辑电平转换电路的小尺寸逻辑器件用 1T 或 2T 分别表示以 1 位或 2 位传输。CMOS 通用集成电路 AHC/AHCT 系列、LVC 系列、AUC 系列和 AUP 系列都有小尺寸逻辑产品。对于 2 输入与非门，传统 AUC 系列的型号为 74AUC00，小尺寸逻辑单门封装的型号为 74AUC1G00，双门封装的型号为 74AUC2G00。74LVC1T45 是小尺寸逻辑封装的 1 位逻辑电平转换器。

例 3.7.2 可配置多功能门(configurable multiple-function gate)电路 74AUP1G97，如图 3.7.16 所示(输入端的 3 个非门具有施密特特性，施密特电路在第 9 章介绍，这里可将其视为普通非门进行逻辑功能分析)。通过简单的外部连线就可以实现表 3.7.2 所示的多种功能，试写出不同输入组合时的输出 L 的表达式。

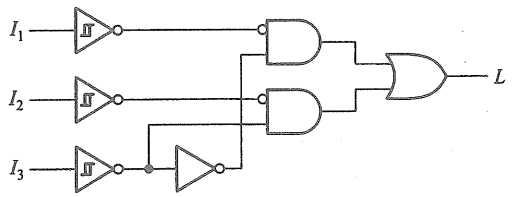


图 3.7.16 例 3.7.2 的小尺寸逻辑 74AUP1G97 内部电路

解：根据电路结构得出其输出与输入的表达式为

$$L=I_1\overline{I_3}+I_2\overline{I_3} \tag{3.7.10}$$

将每一组输入代入式(3.7.10)，化简得：

(1) $L=AC+B\overline{C}$，2 选 1 数据选择器，C 为控制端，A、B 为数据输入。数据选择器将在第 4 章介绍。

(2) $L=AB$，与运算。

(3) $L=A+\overline{C}$，A 与 $\overline{C}$ 进行或运算。

(4) $L=A+\overline{C}=\overline{\overline{A}C}$，$\overline{A}$ 与 C 进行与非运算。

(5) $L=B\overline{C}$，B 与 $\overline{C}$ 进行与运算。

(6) $L=\overline{\overline{B}+C}$，$\overline{B}$ 与 C 进行或非运算。

(7) $L=A+B$，或逻辑。

(8) $L=\overline{C}$，非运算。

(9) $L=A$，缓冲器。

表 3.7.2 74AUP1G97 实现的逻辑功能

序号	输入连接方式			功能
	I_1	I_2	I_3	
1	A	B	C	2 选 1 数据选择器
2	A	B	B	2 输入与门
3	A	$+V_{DD}$	C	带一个反相输入的 2 输入或门

续表

序号	输入连接方式			功能
	I_1	I_2	I_3	
4	A	$+V_{DD}$	C	带一个反相输入的 2 输入与非门
5	地	B	C	带一个反相输入的 2 输入与门
6	地	B	C	带一个反相输入的 2 输入或非门
7	A	B	A	2 输入或门
8	地	$+V_{DD}$	C	反相器
9	A	地	$+V_{DD}$	同相缓冲器

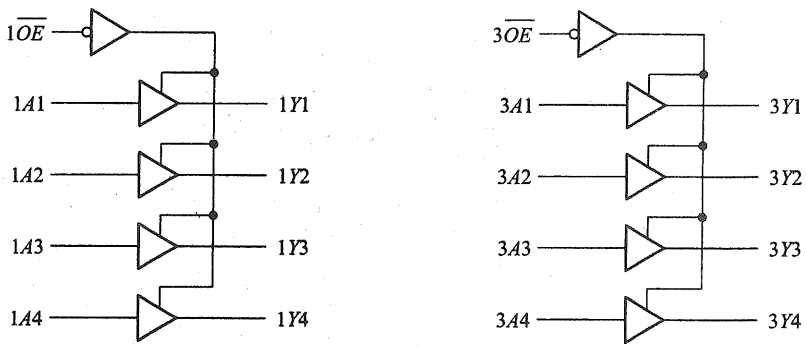
2. 宽总线电路

宽总线是指将多个相同的单元电路封装在一起，以减少体积、改善电路性能，满足数据总线传输的需求。有单向和双向总线驱动之分。宽总线驱动器即是将多个三态输出缓冲器（或反相器）集成在一个芯片中。有 8 位、16 位、32 位等集成电路芯片。

例如 16 位总线驱动器 74AUC16244 集成芯片内部有 16 个三态输出缓冲器，分成 4 组。每组由 4 个高电平使能的三态缓冲器和一个输入低有效的使能控制门构成。组与组之间相互独立，如图 3.7.17 所示。74AUC16244 的功能表如表 3.7.3 所示。使用时可根据需要连接成 16 位、两组 8 位或其他形式，作为总线的数据接收或传输、存储器地址驱动、时钟驱动等。有些总线驱动电路还带有逻辑电平转换功能，可以作为两种工作电源的接口电路。

表 3.7.3 74AUC16244 的功能表

使能$\overline{OE}$	输入 A	输出 Y
L	H	H
L	L	L
H	×	高阻



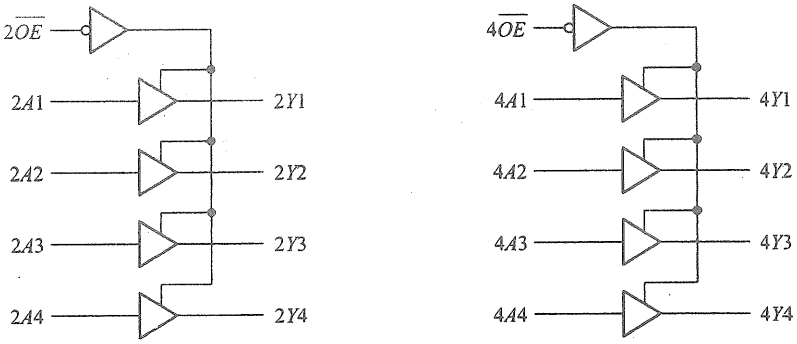


图 3.7.17 宽总线 74AUC16244 逻辑图

复习思考题

3.7.1 不同系列逻辑电路相互连接时，需要满足什么条件？

3.7.2 当 3.3 V 和 5 V 两种 CMOS 门电路连到同一数据总线时，需要注意什么问题？

3.7.3 3.3 V CMOS 电路驱动 5 V CMOS 电路时，如何解决高电平参数不兼容问题？

3.7.4 当负载所需的电流比较大时，如何增加驱动电流？

3.7.5 为什么 CMOS 电路的多余输入端绝对不能悬空？

3.7.6 什么是通用型集成电路？什么是专用型集成电路？

3.7.7 差分信号传输的特点是什么？

3.7.8 小尺寸逻辑和宽总线电路的特点各是什么？

3.8 用 Verilog HDL 描述 CMOS 门电路

Verilog 语言具有对 MOS 三极管级进行设计的能力，这种用 Verilog 语言对 MOS 管构成的数字开关逻辑电路进行的描述，被称为开关级建模①。

随着电路复杂性的增加（上百万的三极管）以及先进 CAD 工具的出现，设计者通常更愿意在逻辑门级、数据流级和行为级等较高抽象层次上进行数字逻辑的设计和仿真。仅在少数情况下，设计者可能会选择三极管作为设计的底层子模块，这是 Verilog 语言提供的最低层次描述。Verilog 中 MOS 三极管模型仅被当作导通或截止的开关，没有模拟器件的特性。本节仅讨论开关级建模的基本原理②。

3.8.1 CMOS 门电路的 Verilog 建模

为了对数字开关逻辑电路建模，Verilog 提供了 10 多个内置的基本开关元件。关键词 **nmos**、**pmos** 分别定义了最基本的 NMOS 管和 PMOS 管模型，实例引用时按照下列格式说明它的三个端口信号。

```
nmos N1(漏极,源极,控制栅极);
```

① 本节介绍的开关级模型可以使用 Verilog_XL、VeriLogger Pro 等仿真器进行仿真。

② 关于开关级建模的详细内容，可参考文献[10]中第 10 章 Switch Level Modeling。

pmos P1(漏极,源极,控制栅极);

对 NMOS 元件,如果控制栅极信号为 1,则 NMOS 开关导通,信号能够从管子的源极传输到漏极,如果控制栅极信号为 0,则输出呈现高阻值 z。类似地,如果控制栅极信号为 0 时,PMOS 开关导通,否则,PMOS 开关的输出呈现高阻值 z。由于 **nmos**、**pmos** 是基本元件,故实例名 N1、P1 可以省略。

关键词 **rnmos**、**rpmos** 分别是 NMOS 管和 PMOS 管另一种模型,前面的字母 r 说明 MOS 管的源极(输入端)和漏极(输出端)之间存在着电阻,当信号从 MOS 管的输入传输至输出时,信号的幅度会衰减,它们的用法与 **nmos**、**pmos** 元件相同。

Verilog 语言中用关键词 **supply1**、**supply0** 分别定义了电源线和地线。**supply1** 与电路图中的 V_{DD} 等效,在整个仿真期间将线网置逻辑高电平 **1**;**supply0** 与电路图中的地线或 V_{SS} 等效,在整个仿真期间将线网置逻辑低电平 **0**。它们的用法如下:

supply1 Vdd;

supply0 GND;

例 3.8.1 根据图 3.2.17 所示电路,用 Verilog 中开关级建模方法对 CMOS 与非门电路进行描述。

解:在 2 输入 CMOS 与非门中,两个 PMOS 管并联,且源极都与电源 Vdd 连接,漏极都与输出 L 相连,两管的栅极分别与输入 A、B 相连;两个 NMOS 管串联,有一个公共节点 W1,第一个 NMOS 管的漏极与输出 L 相连,第二个 NMOS 管的源极与地线 GND 相连,两个 NMOS 管的栅极分别与输入 A、B 相连。电路的描述代码如图 3.8.1 所示。

```
module myNAND2(L,A,B);          //IEEE 1364-1995 语法
    input A,B;          //输入端口声明
    output L;           //输出端口声明
    supply1  Vdd;
    supply0  GND;
    wire W1;            //将两个 NMOS 管之间的连接点定义为 W1
    pmos (L,Vdd,A);     //PMOS 管的源极与 Vdd 相连
    pmos (L,Vdd,B);     //两个 PMOS 管并行连接
    nmos (L,W1,A);      //两个 NMOS 管串行连接
    nmos (W1,GND,B);    //NMOS 管的源极与地相连
endmodule
```

图 3.8.1 图 3.2.17 电路的 Verilog 模型

3.8.2 CMOS 传输门电路的 Verilog 建模

Verilog 语言中用关键词 **cmos** 定义了基本传输门元件模型,它有一个输出端、一个输入端和两个控制端,如图 3.2.20(a)所示,习惯上也称之为 cmos 开关。其用法如下:

cmos C1(输出信号,输入信号,T_N管控制信号,T_P管控制信号);

通常 T_N 管控制信号和 T_P 管控制信号彼此是互补的,当 T_N 管控制信号为 1,T_P 管控制信号为 0 时,CMOS 开关导通;如果 T_N 管控制信号为 0,T_P 管控制信号为 1 时,CMOS 开关的输出呈现出高阻抗值 z。CMOS 开关的电源和地通常与 MOS 管的衬底相连,故实例引用 **cmos**

元件时，不需要考虑电源与地的连接问题。实例名 C1 可以省略。

关键词 **rcmos** 定义了传输门元件另一种模型，字母 r 说明传输门元件的输入端和输出端之间存在着电阻，当信号通过时会造成幅度衰减。注意，Verilog 语言中定义的两种传输门元件是单向传输信号的，与前面介绍的实际传输门元件存在一定差异。

例 3.8.2　根据图 3.2.21 所示电路，用 Verilog 中开关级建模方法对由 CMOS 传输门构成的异或门电路进行描述。

解　电路的描述代码如图 3.8.2 所示。程序两次实例引用在开关级自定义的低层次子模块 inverter，其实例名为 V1、V2（注意，实例引用低层次子模块时，实例名不能省略），完成反相的功能。还实例引用了两个 **cmos** 开关元件，它是 Verilog 内置的基本元件，故实例名可以被省略。

```
//传输门构成异或门模块(参考图 3.2.21)
module myXOR2(A,B,L);
    input A,B;
    output L;
    wire Anot,Bnot;        //模块内部节点
    inverter V1(Anot,A); //实例引用子模块
    inverter V2(Bnot,B);
    cmos (L,Anot,B,Bnot);
    cmos (L,A,Bnot,B);
endmodule
```

```
//CMOS 反相器子模块(参考图 3.2.12)
module inverter(Vo,Vi);
    input Vi;
    output Vo;
    supply1  Vdd;
    supply0  GND;
    pmos (Vo,Vdd,Vi);   //实例引用
    nmos (Vo,GND,Vi);
endmodule
```

图 3.8.2　图 3.2.21 电路的 Verilog 模型

小　结

- 逻辑门电路是组成各种数字电路的基本单元电路。按照构成门电路元器件的不同，分为 MOS 型、双极型和混合型。MOS 型集成逻辑门有 CMOS、NMOS、PMOS。双极型集成逻辑门主要有 TTL，混合型有 BiCMOS 等。不论哪一种逻辑门电路，作为开关器件的 MOS 管或 BJT 管起关键作用。影响它们开关速度的主要因素是器件内部各电极之间的结电容。
- CMOS 电路的优点是集成度高，功耗低，扇出数大，噪声容限大，开关速度快，是目前广泛使用的集成电路。工艺技术的改进使其向着低电压、超低电压和低功耗器件的方向发展。
- CMOS 电路的输入和输出具有保护电路和缓冲电路，因此，同一系列不同逻辑功能的电路，具有相同的特性和统一的参数。54 或 74 系列的逻辑电路，只要型号最后的数字相同，它们的逻辑功能就一样，但是电气性能参数有所不同。使用时要注意查阅具体的参数。
- 从输出端的结构看，CMOS 电路有普通缓冲器输出、漏极开路输出或三态输出电路。漏极开路门的输出端并接可以实现线与。三态输出用于数据总线连接，任何时刻只能有一个门的使能端为有效，其余三态门处于高阻状态。
- 逻辑门电路的主要技术参数有输入和输出高低电平的极值、噪声容限、传输延迟时间、功耗、延时-功耗积、扇入与扇出数等。

• 类 NMOS 逻辑门电路结构简单，几何尺寸小，易于集成化，在某些特殊应用中更具优势。BiCMOS 电路结合了 MOS 电路和 TTL 电路的优势，其开关速度较快，功耗亦较低。

• CMOS 系列已取代 TTL 系列成为数字电路和系统的首选。BiCMOS 电路主要用于通信领域。在某些特殊场合可能会用到 TTL。

• 在逻辑体制中有正、负逻辑的规定，本书主要采用正逻辑体制。逻辑门的等效符号常用于简化电路的分析和设计。

• 在逻辑门电路的实际应用中，有可能遇到不同系列门电路之间、门电路与负载之间的接口技术问题，以及抗干扰工艺问题。数字电路设计工作者应能正确分析与解决这些问题。现在广泛使用的差分信号传输是一种抗干扰能力强的信号传输技术。

• Verilog 中 MOS 三极管模型仅被当作导通或截止的开关，没有模拟器件特性。用 Verilog 可以描述 MOS 管构成逻辑门电路，即开关级建模。

习　题

3.1 逻辑门电路简介

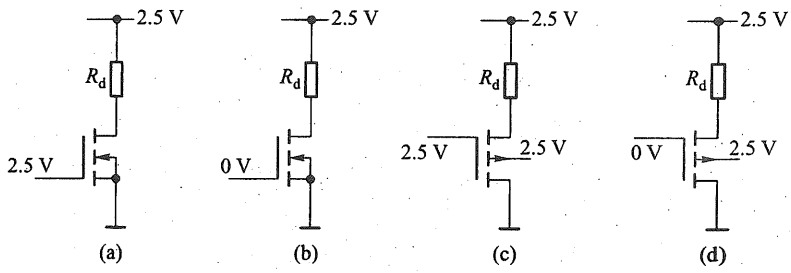
3.1.1　按照制造门电路三极管的不同，集成门电路分为哪几种类型？各种类型的代表是什么？

3.1.2　为什么要发展低电压和超低电压 CMOS 器件？

3.1.3　数字逻辑变量可以取什么值？三极管在数字电路中工作在什么状态？

3.2 基本 CMOS 逻辑门电路

3.2.1　已知图题 3.2.1 所示各 MOS 管的 $|V_T|=0.5$ V，$R_d=2$ kΩ，导通时 $I_D=0.5$ mA，试分别确定它们的工作状态（导通或截止）。



图题 3.2.1

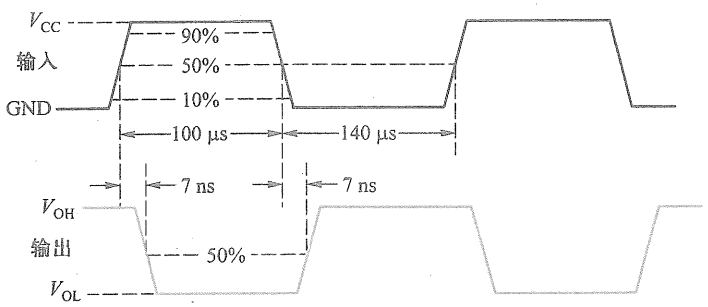
3.2.2　一个反相器的输入和输出波形如图题 3.2.2 所示，试确定：

（1）输入信号的周期和频率；

（2）输出由高电平变为低电平的传输延迟时间 t_{pHL} 和由低电平变为高电平的传输延迟时间 t_{pLH}。

3.2.3　为什么说 74LVC 系列 CMOS 与非门在 3.3 V 电源工作时，输入端在以下四种接法下都属于逻辑 0（74LVC 系列输入、输出电压值参考表 3.3.2）：

（1）输入端接地；



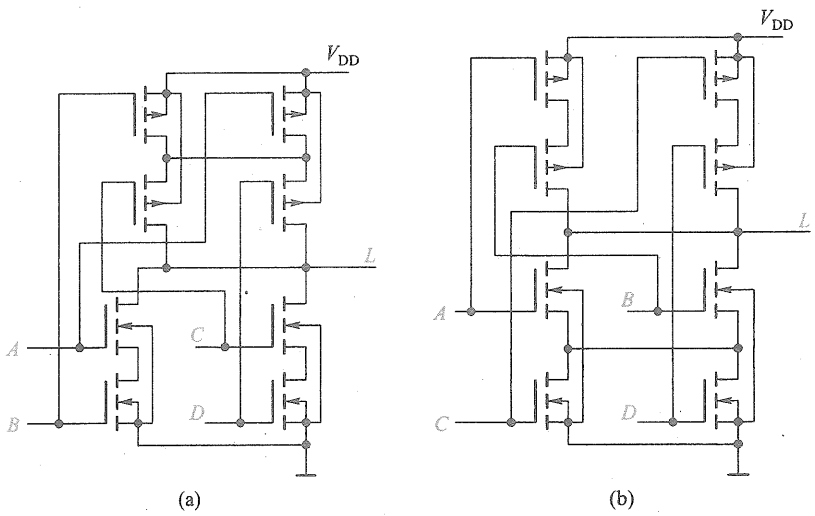
图题 3.2.2

(2) 输入端接低于 0.8 V 的电源;

(3) 输入端接同类与非门的输出低电平 0.2 V;

(4) 输入端到地之间接 10 kΩ 的电阻。

3.2.4 试分析图题 3.2.4 所示的电路,写出 L 的逻辑表达式,画出由与门、或门、非门逻辑符号构成的电路图。



图题 3.2.4

3.2.5 试分析图题 3.2.5 所示电路,写出 L 的逻辑表达式。

3.2.6 试分析图题 3.2.6 所示的电路,写出 L 的逻辑表达式,说明它实现什么逻辑功能。

3.2.7 CMOS 集成芯片 4007 中包含两个互补对和一个反相器,如图题 3.2.7 所示,试分别连接:

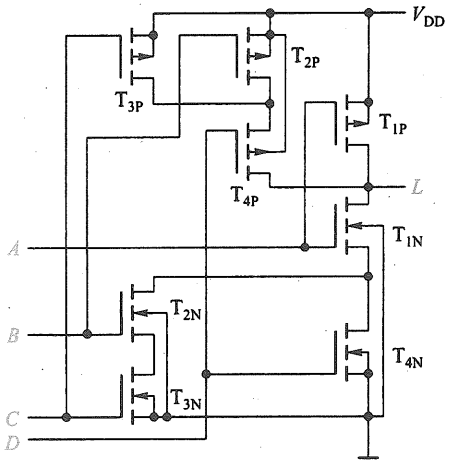
(1) 三个反相器;

(2) 3 输入端或非门;

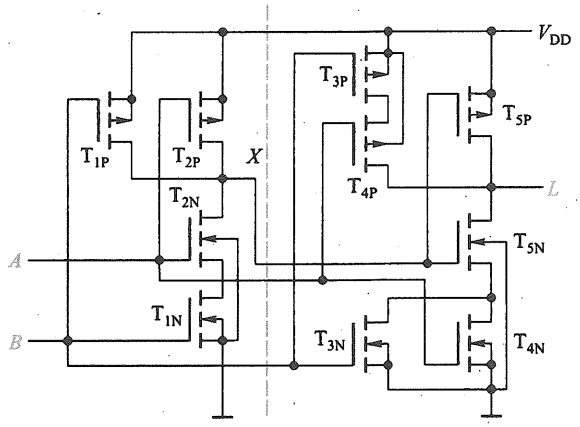
(3) 3 输入端与非门;

(4) 或与非门[$L=\overline{C(A+B)}$];

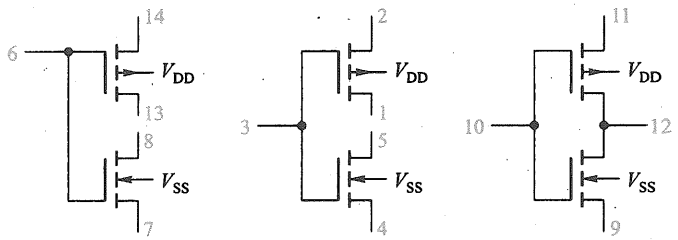
(5) 传输门(一个非门控制两个传输门分时传送)。



图题 3.2.5



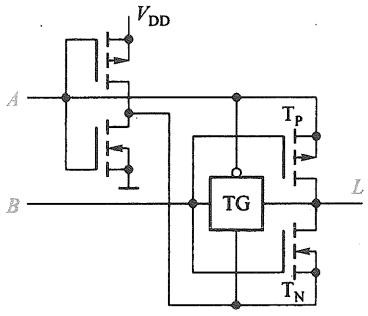
图题 3.2.6



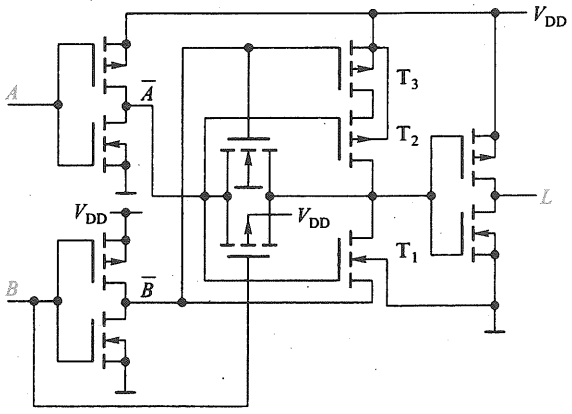
图题 3.2.7

3.2.8 试分析图题 3.2.8 所示 CMOS 门电路，写出 L 的逻辑表达式，说明它实现什么逻辑功能。

3.2.9 试分析图题 3.2.9 所示 CMOS 门电路，写出 L 的逻辑表达式，说明它实现什么逻辑功能。



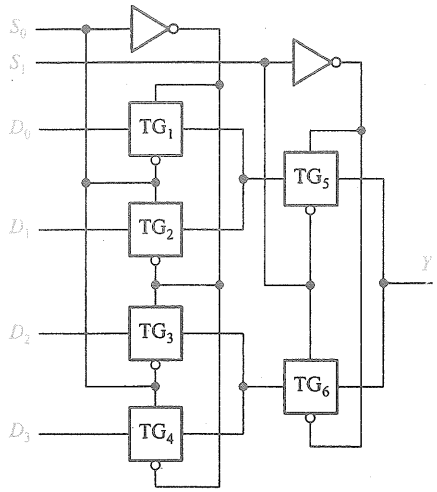
图题 3.2.8



图题 3.2.9

3.2.10 试用两个传输门和两个反相器实现同或逻辑功能，并画出逻辑电路。

3.2.11 由 CMOS 传输门构成的电路如图题 3.2.11 所示，S_1、S_0为控制端，D_0、D_1、D_2、D_3为输入，Y 为输出，试列出其功能表，并写出输出逻辑函数表达式。



图题 3.2.11

3.3 CMOS 逻辑门的其他输出结构及参数

3.3.1 在漏极开路门电路中，输出上拉电阻值的大小对逻辑门有什么影响？

3.3.2 试判断下列哪些 CMOS 门可以将输出端并接使用：

(1) 普通的互补输出；

(2) 漏极开路输出；

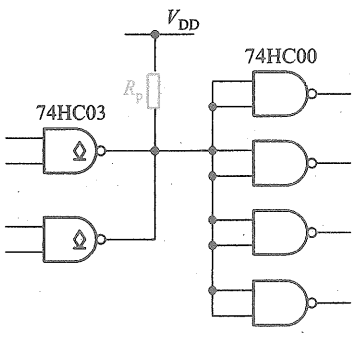
(3) 三态输出。

3.3.3 试用普通反相器和 OD 输出反相器实现下列表达式，并画出逻辑电路图。

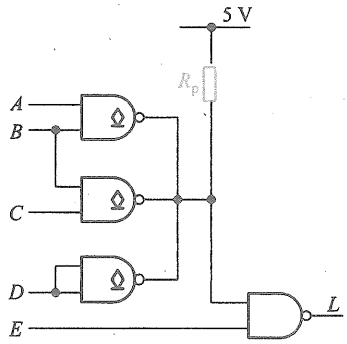
(1) $L=A\overline{BC}$；

(2) $L=AB\overline{CD}$。

3.3.4 用 74HC03 中 2 个漏极开路与非门及 74HC00 中的 4 个与非门构成的电路如图题 3.3.4 所示。试确定上拉电阻 R_p 的取值范围。已知 $V_{DD}=5$ V，OD 门输出低电平 $V_{OL(max)}=0.33$ V 时的输出电流 $I_{OL(max)}=4$ mA，输出高电平 $V_{OH(min)}=4.4$ V 时的漏电流 $I_{OZ}=5$ μA。负载门高电平和低电平输入电流最大值 $I_{IH(max)}=I_{IL(max)}=1$ μA。



图题 3.3.4



图题 3.3.5

3.3.5 由 CMOS 漏极开路门及与非门组成的电路如图题 3.3.5 所示。已知驱动门输出低电

平 $V_{OL(max)}$ = 0.33 V 时的最大输出电流 $I_{OL(max)}$ = 4 mA，输出高电平 $V_{OH(min)}$ = 4.4 V 时的漏电流 I_{OZ} = 5 μA，负载门高电平和低电平输入电流均为 1 μA。

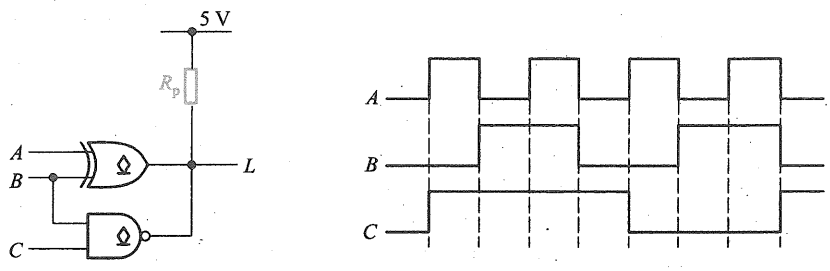
(1) 试写出输出与输入的逻辑关系式；

(2) 计算 $R_{p(min)}$ 和 $R_{p(max)}$。

3.3.6 由 OD 异或门和 OD 与非门构成的电路及输入电压波形如图题 3.3.6 所示。已知输出低电平 $V_{OL(max)}$ = 0.33 V 时的最大输出电流 $I_{OL(max)}$ = 4 mA，输出高电平 $V_{OH(min)}$ = 4.4 V 时的漏电流 I_{OZ} = 5 μA。

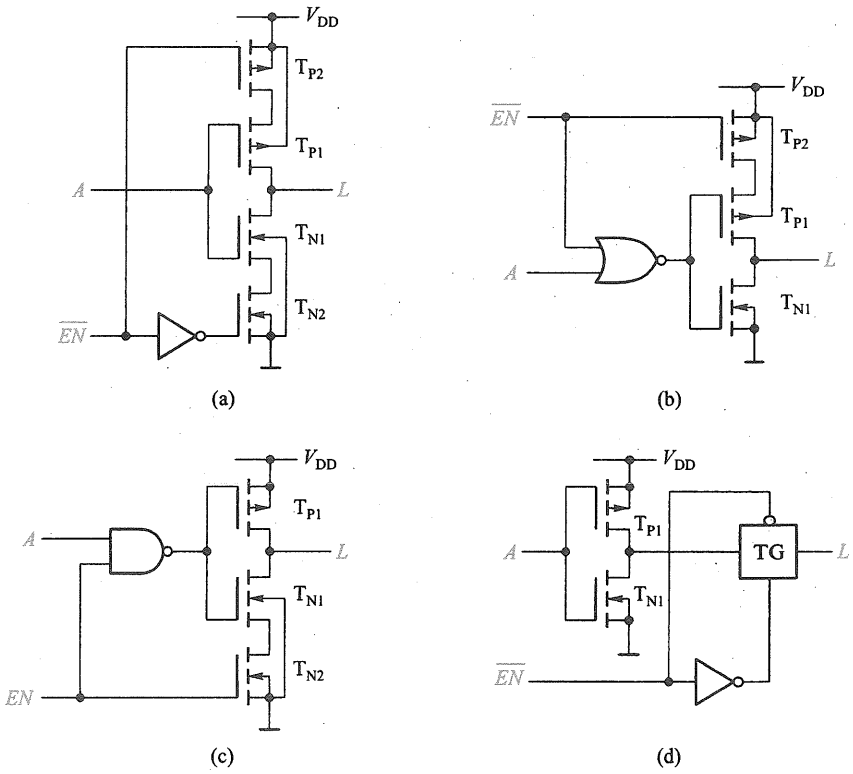
(1) 试写出输出与输入的逻辑关系式，画出输出电压波形；

(2) 计算 $R_{p(min)}$ 和 $R_{p(max)}$。



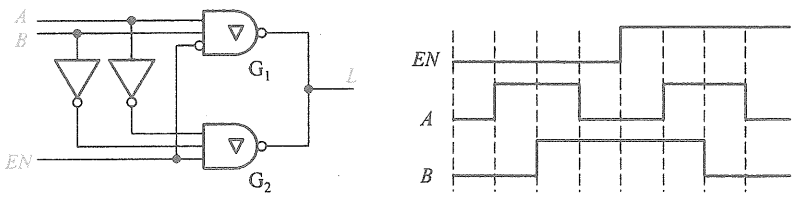
图题 3.3.6

3.3.7 试分析图题 3.3.7 所示的 CMOS 电路，说明它们的逻辑功能。



图题 3.3.7

3.3.8 CMOS 电路及输入波形如图题 3.3.8 所示，画出输出 L 的波形。



(a) (b)

图题 3.3.8

3.3.9 图题 3.3.9 表示三态门用于总线传输的示意图，图中 n 个三态门的输出连接到数据传输总线，D_1、D_2、…、D_n 为数据输入端，EN_1、EN_2、…、EN_n 为使能信号输入端。试问：

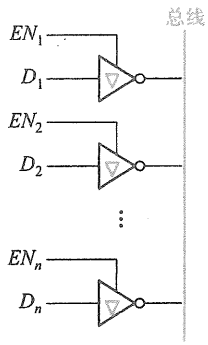
(1) 如何控制 EN 信号，以便数据 D_1、D_2、…、D_n 能通过该总线进行正常传输。

(2) EN 信号能否有两个或两个以上同时有效？如果出现两个或两个以上有效，可能发生什么情况？

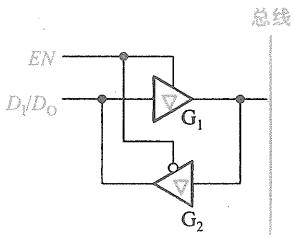
(3) 如果所有 EN 信号均无效，总线处在什么状态？

(4) 如果三态门的开通比断开要快，可能发生什么情况？

3.3.10 三态门与总线的连接方式如图题 3.3.10 所示，试分析电路的逻辑功能。



图题 3.3.9



图题 3.3.10

3.3.11 根据表题 3.3.11 所列的三种逻辑门电路的技术参数，试选择一种最适合工作在高噪声环境下的门电路。

表题 3.3.11 逻辑门电路的技术参数表

	$V_{OH(min)}$/V	$V_{OL(max)}$/V	$V_{IH(min)}$/V	$V_{IL(max)}$/V
逻辑门 A	2.4	0.4	2	0.8
逻辑门 B	3.5	0.2	2.5	0.6
逻辑门 C	4.2	0.2	3.2	0.8

3.3.12 CMOS 反相器的负载电容 $C_L = 100$ pF，功耗电容 $C_{PD} = 15$ pF，电源电压 $V_{DD} = 3.3$ V，输入矩形波的频率为 1 MHz，试计算反相器的动态功耗。

3.3.13 当 74HC 系列 CMOS 电路的电源电压由 5 V 降至 3.3 V，其功耗下降了百分之多少？

3.3.14 根据表题 3.3.14 所列的三种逻辑门电路的技术参数，计算出它们的延时-功耗积，并指出哪一种逻辑门的性能最好。

表题 3.3.14 逻辑门电路的技术参数表

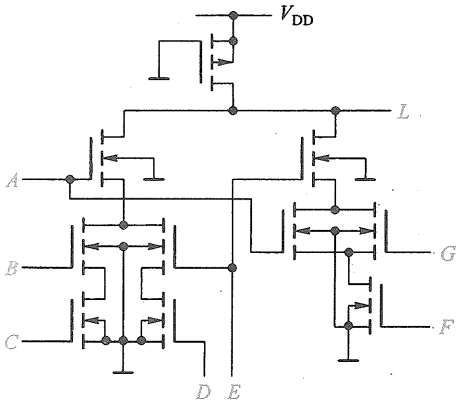
	t_{pLH}/ns	t_{pHL}/ns	P_D/mW
逻辑门 A	1	1.2	16
逻辑门 B	5	6	8
逻辑门 C	10	10	1

3.3.15 根据表 3.3.4 所示的电流值，求 74LVC 门驱动同类门的扇出数。

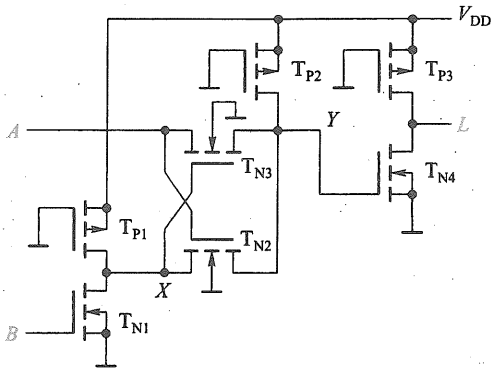
3.4 类 NMOS 和 BiCMOS 逻辑门电路

3.4.1 写出图题 3.4.1 所示电路的逻辑表达式。

3.4.2 写出图题 3.4.2 所示电路的逻辑表达式。



图题 3.4.1



图题 3.4.2

3.4.3 分析图题 3.4.3 所示电路的工作原理，写出 L 的逻辑表达式。

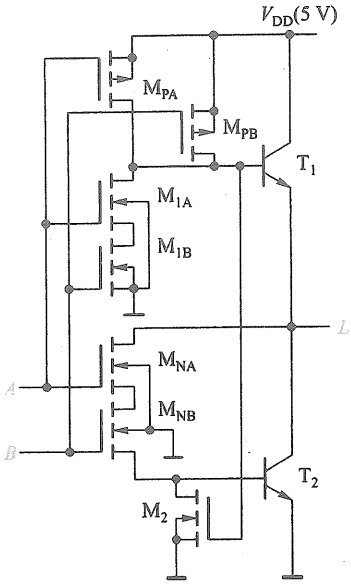
3.4.4 分析图题 3.4.4 所示电路的工作原理，写出 L 的逻辑表达式。

3.5 TTL 逻辑门电路

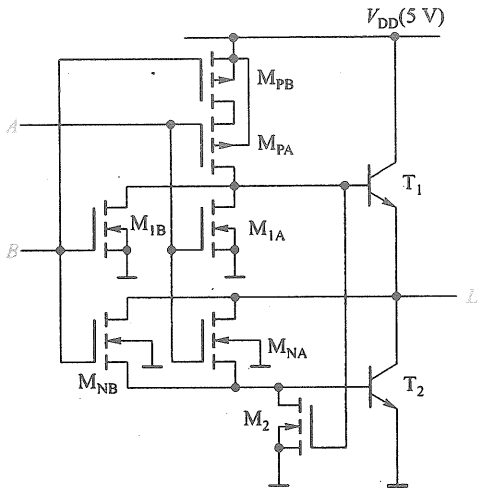
3.5.1 由 BJT 构成的反相器如图题 3.5.1 所示，$V_{CC} = 5$ V，$V_{BE} = 0.7$ V，$\beta = 100$。当输入 v_I 为 5 V 时，BJT 工作在饱和区，有 $I_C \leqslant \beta I_B$，输出为 0.2 V，试计算 R_b/R_c 的最大比值。

3.5.2 为什么说 TTL 非门的输入端在以下四种接法下，都属于逻辑 1。已知 TTL 门电路的输出和输入高电平的标准电压值为：$V_{OH} = 2.7$ V，$V_{IH} = 2$ V。

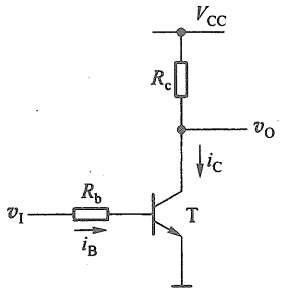
(1) 输入端悬空；



图题 3.4.3



图题 3.4.4



图题 3.5.1

(2) 输入端接高于 2 V 的电源;

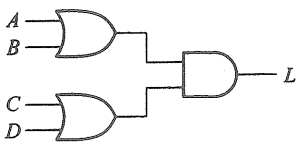
(3) 输入端接同类门的输出高电平 3.6 V;

(4) 输入端到地之间接 10 kΩ 的电阻。

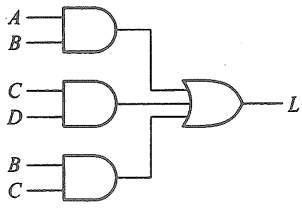
3.6 逻辑描述中的几个问题

3.6.1 试对图题 3.6.1 所示电路的逻辑门进行变换,使其可以用单一的或非门实现。

3.6.2 电路如图题 3.6.2 所示,试用与非门实现。



图题 3.6.1



图题 3.6.2

3.7 逻辑门电路使用中的几个实际问题

3.7.1 当供电电源不同的两种门电路相互连接时，要考虑哪些电压和电流参数？这些参数应满足怎样的关系？

3.7.2 若只考虑逻辑电平的兼容性，根据表 3.3.4 所示的参数值，试确定：

(1) 当用 5 V 供电的 74AHC00 驱动 3 V 供电的 74LVC00 时，是否需要接口，计算其扇出数；

(2) 当用 74LVC00 驱动 74AHC00 时，是否需要接口，若需要，选择 74LVC/T45 并画出它们的连接示意图。

3.7.3 根据图 3.7.1 所示的逻辑电平值，试计算：

(1) 3.3 V CMOS 系列驱动 2.5 V CMOS 系列的高、低电平噪声容限；

(2) 2.5 V CMOS 系列驱动 3.3 V CMOS 系列的高、低电平噪声容限。

(假设 2.5 V CMOS 系列输入输出均能承受 3.3 V 电压)。

3.7.4 根据图 3.7.1 所示的逻辑电平值，试计算：

(1) 3.3 V CMOS 系列驱动 1.8 V CMOS 系列的高、低电平噪声容限；

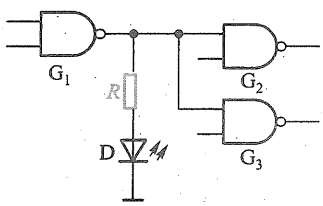
(2) 2.5 V CMOS 系列驱动 1.8 V CMOS 系列的高、低电平噪声容限。

(假设 1.8 V CMOS 系列输入可承受 3.3 V 电压)。

3.7.5 复习一下 CMOS 门的输出电路。若 CMOS 的输出级超载时，电路会出现什么现象？用什么仪器进行判断？

3.7.6 由 CMOS 非门 74AHC04 驱动发光二极管电路如图 3.7.10(b) 所示，试确定上拉电阻 R 的值。已知 74AHC04 的参数为 $V_{OL(max)}=0.44\ V$，$I_{OL(max)}=8\ mA$。发光二极管的参数为 $V_F=2.5\ V$，$I_D=4.5\ mA$。

3.7.7 在图题 3.7.7 所示的电路中，CMOS 与非门 74AC00 的参数 $V_{OH(min)}=4.4\ V$，$I_{OH(max)}=-24\ mA$，$V_{IH(min)}=3.5\ V$，$I_{IH(max)}=1\ \mu A$。发光二极管参数为 $V_F=1.6\ V$，$I_D=10\ mA$，试确定：



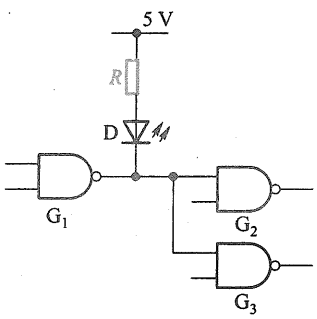
图题 3.7.7

(1) 限流电阻 R 的值；

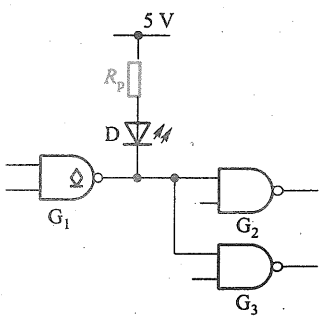
(2) 当 $R=0$ 时，会出现什么现象？

3.7.8 在图题 3.7.8 所示的电路中，CMOS 与非门 74AC00 的参数 $V_{OL(max)}=0.37\ V$，$I_{OL(max)}=24\ mA$，$V_{IL(max)}=1.5\ V$，$I_{IL(max)}=-1\ \mu A$。发光二极管参数为 $V_F=1.6\ V$，$I_D=10\ mA$，试确定：

(1) 限流电阻 R 的值；



图题 3.7.8

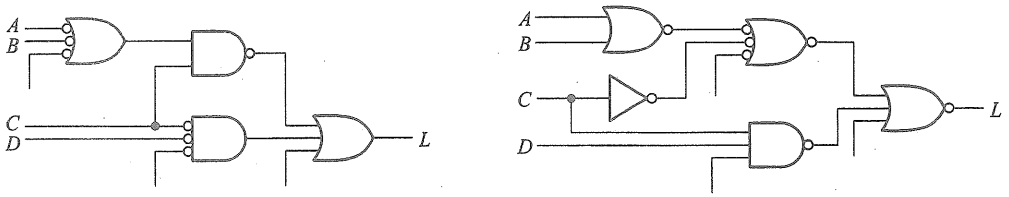


图题 3.7.9

(2) 当 $R=0$ 时,会出现什么现象?

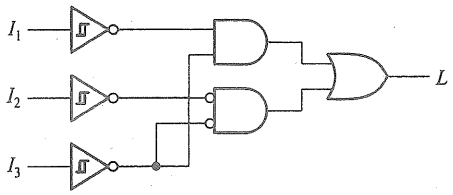
3.7.9 由 CMOS 漏极开路 OD 门及与非门组成的电路如图题 3.7.9 所示。试计算 $R_{p(min)}$ 和 $R_{p(max)}$。已知 OD 门输出低电平 $V_{OL(max)}=0.33$ V 时的最大输出电流 $I_{OL(max)}=4$ mA,输出高电平 $V_{OH(min)}=4.4$ V 时的漏电流 $I_{OZ}=5$ μA,负载门高电平和低电平输入电流均为 1 μA。发光二极管参数为 $V_F=1.6$ V,$I_D=4$ mA。

3.7.10 由 CMOS 门构成的电路如图题 3.7.10 所示,试对多余输入端进行处理,并写出 L 的最简与-或表达式。



图题 3.7.10

3.7.11 可配置多功能门电路 74LVC1G57 的内部结构如图题 3.7.11 所示(输入端的 3 个非门具有施密特特性,施密特电路在第 9 章介绍,这里可将其视为普通非门进行逻辑功能分析)。根据需要将 I_1、I_2、I_3 接输入变量、地或 V_{CC} 就可以实现不同逻辑功能,试确定输入端的连接方式以便分别实现:(1) $L=AB$;(2) $L=\overline{A+B}$;(3) $L=AB+\bar{A}\bar{B}$。



图题 3.7.11

3.8 用 Verilog HDL 描述 CMOS 门电路

3.8.1 试用 Verilog 提供的基本开关元件对图 3.2.18 所示的或非门电路进行描述。

3.8.2 试用 Verilog 提供的基本开关元件对图 3.2.19 所示的异或门电路进行描述。

文档 3.2:
第 3 章部分
习题答案

4 组合逻辑电路

引言

文档 4.1：
第 4 章思维导图

数字系统中常用的各种数字部件，就其结构和工作原理而言可分为两大类，即组合逻辑电路和时序逻辑电路。本章首先介绍组合逻辑电路的定义、分析方法和设计方法，并阐述竞争-冒险产生的原因及消除方法。然后讨论常用的几种组合逻辑电路模块的功能及基本应用，它们包括编码器和译码器、数据选择器和数据分配器、数值比较器、算术运算单元等。最后介绍组合逻辑电路的 Verilog HDL 描述以及用可编程逻辑器件 PLD 的实现方法。

4.1 组合逻辑电路的分析

4.1.1 组合逻辑电路的定义

对于一个逻辑电路，其输出状态在任何时刻只取决于同一时刻的输入状态，而与电路原来的状态无关，这种电路被定义为组合逻辑电路。组合逻辑电路的一般框图如图 4.1.1 所示，其输出与输入之间的逻辑关系可用如下的逻辑函数来描述，即

图 4.1.1 组合逻辑电路的一般框图

$$L_i = f(A_1, A_2, \cdots, A_n) \quad (i = 1, 2, \cdots, m) \tag{4.1.1}$$

式中，$A_1, A_2, \cdots, A_n$ 为输入变量。

组合逻辑电路的结构具有如下的特点：

（1）输出、输入之间没有反馈延迟通路；

（2）电路中不含具有记忆功能的元件。

4.1.2 组合逻辑电路的分析方法

分析组合逻辑电路的目的是，对于一个给定的逻辑电路，确定其逻辑功能。分析组合逻辑电路的步骤大致如下：

（1）根据逻辑电路，从输入到输出，写出各级逻辑函数表达式，直到写出输出信号与输

入信号的逻辑函数表达式；

(2) 将各逻辑函数表达式化简和变换，以得到最简单的表达式；

(3) 根据简化后的逻辑表达式列出真值表；

(4) 根据真值表和简化后的逻辑表达式对逻辑电路进行分析，最后确定其功能。

下面举例说明组合逻辑电路的分析方法。

例 4.1.1 已知逻辑电路如图 4.1.2 所示，分析该电路的功能。

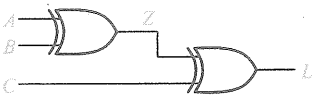


图 4.1.2 例 4.1.1 的逻辑电路

解：(1) 根据逻辑电路可写出输出端的逻辑函数表达式，为方便起见，电路中标出了中间变量 Z。电路由两个异或门构成，因此

$$Z=A\oplus B$$

$$L=Z\oplus C=(A\oplus B)\oplus C$$

该表达式无须化简和变换。

(2) 列写真值表。将 3 个输入变量的 8 种可能的组合一一列出。分别将每一组变量的取值代入逻辑函数表达式，然后算出中间变量 Z 值和输出 L 值，填入表中，如表 4.1.1 所示。

表 4.1.1 例 4.1.1 的真值表

输入			输出(含中间变量)	
A	B	C	Z	L
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

(3) 确定逻辑功能。分析真值表后可知，当 A、B、C 三个输入变量的取值中有奇数个 1 时，L 为 1，否则 L 为 0。所以该电路称为奇校验电路，用于检查输入的 3 位二进制数据的奇偶性。

如果在上述电路的输出端再加一级反相器，当输入电路的二进制数据中含有偶数个 1 时，输出为 1，则称此电路为偶校验电路。

波形图可以比较直观地反映输入与输出之间的逻辑函数关系，对于比较简单的组合逻辑电路，也可用画波形图的方法进行分析。为了避免出错，通常是根据输入波形的变化分段，然后逐段画出输出波形，例如，第一段的输入信号 A、B、C 的值均为 0，代入表达式中得出 Z 的值，然后得出 L 的值，如图 4.1.3 所示。最后根据波形图确定输入和输出的逻辑功能。

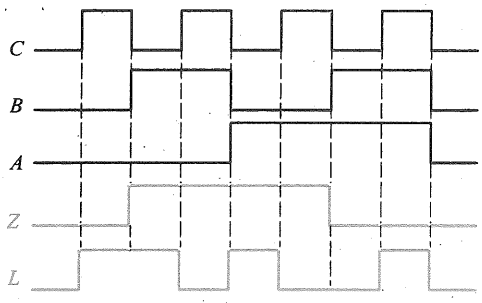


图 4.1.3 例 4.1.1 的波形分析图

奇偶校验电路是一种检测数据传输是否正确的最简单方法。通常在被传输的数据前面或者后面增加一位校验位。数据传输前，校验位被设置为 0 或 1 以满足：奇校验时，含校验位的待传数据中 1 的个数为奇数；偶校验时，含校验位的待传数据中 1 的个数为偶数。传输后，在接收端通过检测 1 的个数，便可判断传输是否有错。因此，奇偶校验有奇偶校验位产生电路和校验电路两部分组成。具体电路设计参见习题 4.2.4。

例如，原始数据为 **1011000**，如果采用奇校验则为 **1011000 [0]**；如果采用偶校验则为 **1011000 [1]**（方括号内为校验位）。奇偶校验只能检测出奇数个错误，而且不能确定哪一位出错。发生错误时只能扔掉全部数据，重新传输。

例 4.1.2 试分析图 4.1.4 所示组合逻辑电路的逻辑功能。

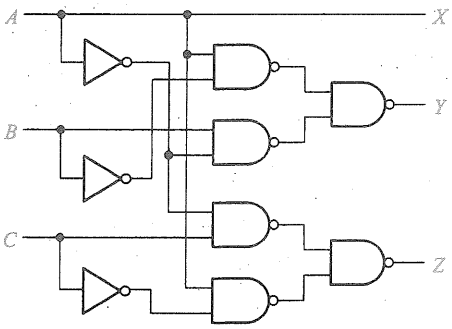


图 4.1.4 例 4.1.2 的逻辑电路

解：（1）根据逻辑电路可写出各输出端的逻辑表达式，并进行化简和变换。

$$X = A$$

$$Y = \overline{\overline{A\overline{B}} \cdot \overline{\overline{A}B}} = A\overline{B} + \overline{A}B$$

$$Z = \overline{\overline{A\overline{C}} \cdot \overline{\overline{A}C}} = A\overline{C} + \overline{A}C$$

（2）列写真值表，如表 4.1.2 所示。

表 4.1.2 例 4.1.2 的真值表

输入			输出		
A	*B*	*C*	*X*	*Y*	*Z*
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	0	1
1	1	1	1	0	0

(3) 确定逻辑功能。分析真值表可知,输出最高位 X 与输入最高位 A 相等。当 A 为 0 时,输出 Y、Z 分别与所对应的输入 B、C 相同;而当 A 为 1 时,输出 Y、Z 分别是输入 B、C 取反。这个电路逻辑功能是对输入的二进制码求反码。最高位为符号位,0 表示正数,1 表示负数,正数的反码与原码相同;负数的数值部分是在原码的基础上逐位求反。

复习思考题

4.1.1 什么是组合逻辑电路?

4.1.2 列出分析组合逻辑电路的步骤。

4.1.3 组合逻辑电路都能用波形分析法进行分析吗?

4.2 组合逻辑电路的设计

组合逻辑电路的设计与分析过程相反,对于提出的实际逻辑问题,得出满足这一逻辑问题的逻辑电路。电路设计的首要任务是满足功能要求,其次是优化,也称为最小化,即用指定的器件实现逻辑函数时,力求成本低并且工作速度快。前面介绍的用代数法和卡诺图法来化简逻辑函数,是为了获得最简的逻辑表达式。根据最简表达式获得最低成本电路。

电路的实现可以采用各种门电路,也可以利用数据选择器等基本模块,或者可编程逻辑器件。因此,逻辑函数的化简也要结合所选用的器件进行,有时还需要一定的变换,以便满足优化实现的要求。这一节主要介绍用门电路实现组合逻辑电路。

4.2.1 组合逻辑电路的设计过程

组合逻辑电路的设计步骤大致如下:

(1) 明确实际问题的逻辑功能。许多实际设计要求是用文字描述的,因此,需要确定实际问题的逻辑功能,并确定输入、输出变量数及表示符号;

(2) 根据对电路逻辑功能的要求,列出真值表;

(3) 由真值表写出逻辑表达式;

(4) 简化和变换逻辑表达式,从而画出逻辑图。

下面举例说明设计组合逻辑电路的方法和步骤。

例 4.2.1 某办公楼空气净化设备是根据三台空气质量检测仪的检测结果进行控制。试设计一个逻辑电路,当有两台或以上检测结果超标时,要求其向控制系统输出高电平,以启动空气净化设备。

解:(1) 明确逻辑功能。

设变量 A、B、C 分别表示三台空气质量检测仪的检测结果,并规定检测结果超标时为 1,没有超标时为 0。变量 L 表示向控制系统输出的信号,有两台或以上检测结果超标时,L 为 1,否则为 0。

根据题意列出真值表,如表 4.2.1 所示。

表 4.2.1 例 4.2.1 的真值表

输入			输出
A	B	C	L
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

(2) 根据真值表写出输出逻辑表达式。

根据 1.6 节介绍的方法,逻辑变量之间是与的关系,而输出状态之间的组合则是或的关系。对于输入或输出变量,凡取 1 值的用原变量表示,取 0 值用反变量表示。则

$$L=\bar{A}BC+A\bar{B}C+AB\bar{C}+ABC \tag{4.2.1}$$

(3) 简化逻辑表达式。

用公式法或卡诺图化简法对式(4.2.1)化简,得

$$L=AB+AC+BC \tag{4.2.2}$$

式(4.2.2)为最简与-或式,用与门和或门实现两级与-或结构的最简电路如图 4.2.1 所示。

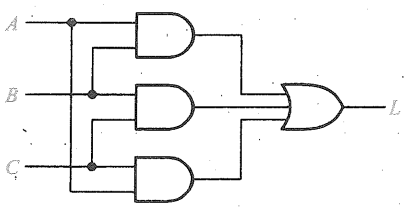


图 4.2.1 例 4.2.1 逻辑图

在不同的数字系统中,可能采用不同的码制对信息进行编码和处理。如果在采用不同码制的两个数字系统之间进行信息传输,则需要一个码转换电路。

例 4.2.2 试设计一个码转换电路,将 4 位格雷码转换为自然二进制码。可以采用任何逻辑门电路来实现。

解:(1) 明确逻辑功能,列出真值表。

设电路的 4 个输入变量为 G_3、G_2、G_1 和 G_0,4 个输出变量为 B_3、B_2、B_1 和 B_0。当输入格雷码按照从十进制数 0 到 15 递增排序时,对应输出的自然二进制码如表 4.2.2 所示。

(2) 根据表 4.2.2 画出各输出函数的卡诺图,如图 4.2.2 所示。表 4.2.2 中的输入是格雷码,填写卡诺图时要注意顺序。由卡诺图写出各输出逻辑表达式,并化简和变换,如式(4.2.3)所示。

表 4.2.2 例 4.2.2 的真值表

十进制数	输入				输出			
	G_3	G_2	G_1	G_0	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	1	0	0	1	0
3	0	0	1	0	0	0	1	1
4	0	1	1	0	0	1	0	0
5	0	1	1	1	0	1	0	1
6	0	1	0	1	0	1	1	0
7	0	1	0	0	0	1	1	1
8	1	1	0	0	1	0	0	0
9	1	1	0	1	1	0	0	1
10	1	1	1	1	1	0	1	0
11	1	1	1	0	1	0	1	1
12	1	0	1	0	1	1	0	0
13	1	0	1	1	1	1	0	1
14	1	0	0	1	1	1	1	0
15	1	0	0	0	1	1	1	1

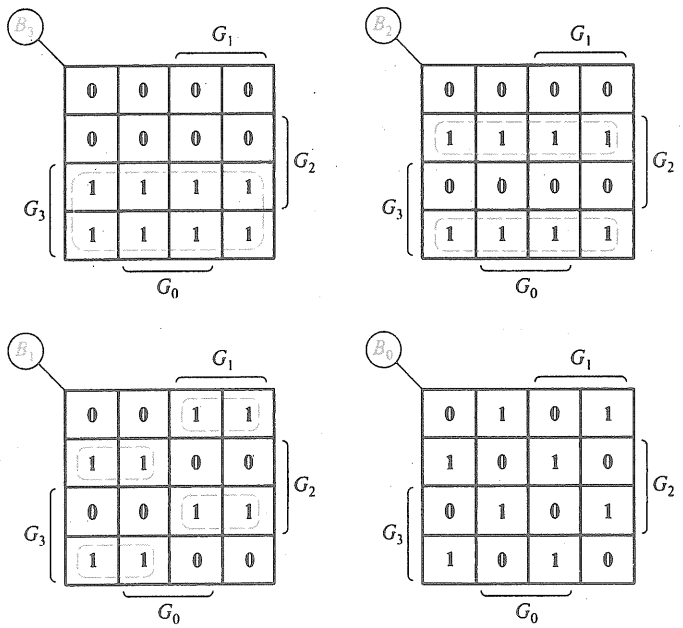


图 4.2.2 例 4.2.2 的卡诺图

$$
\begin{cases}
B_3 = G_3 \\
B_2 = G_3\overline{G}_2 + \overline{G}_3 G_2 \\
\quad = G_3 \oplus G_2 \\
\quad = B_3 \oplus G_2 \\
B_1 = G_3\overline{G}_2\overline{G}_1 + \overline{G}_3 G_2\overline{G}_1 + G_3 G_2 G_1 + \overline{G}_3\overline{G}_2 G_1 \\
\quad = (G_3\overline{G}_2 + \overline{G}_3 G_2)\overline{G}_1 + \overline{(G_3\overline{G}_2 + \overline{G}_3 G_2)}\, G_1 \\
\quad = G_3 \oplus G_2 \oplus G_1 \\
\quad = B_2 \oplus G_1 \\
B_0 = \overline{G}_3\overline{G}_2\overline{G}_1 G_0 + \overline{G}_3\overline{G}_2 G_1\overline{G}_0 + \overline{G}_3 G_2\overline{G}_1\overline{G}_0 + \overline{G}_3 G_2 G_1 G_0 + \\
\quad G_3 G_2\overline{G}_1 G_0 + G_3 G_2 G_1\overline{G}_0 + G_3\overline{G}_2\overline{G}_1\overline{G}_0 + G_3\overline{G}_2 G_1 G_0 \\
\quad = G_3 \oplus G_2 \oplus G_1 \oplus G_0 \\
\quad = B_1 \oplus G_0
\end{cases}
\tag{4.2.3}
$$

因此，n 位格雷码 $G_{n-1}G_{n-2}G_{n-3}\cdots G_0$ 转换为 n 位二进制码 $B_{n-1}B_{n-2}B_{n-3}\cdots B_0$ 的公式为

$$
\begin{cases}
B_{n-1} = G_{n-1} \\
B_{n-2} = B_{n-1} \oplus G_{n-2} \\
B_{n-3} = B_{n-2} \oplus G_{n-3} \\
\quad \vdots \\
B_0 = B_1 \oplus G_0
\end{cases}
\tag{4.2.4}
$$

互换表 4.2.2 中的输入和输出，以同样的方法，也可以得到 n 位二进制码 $B_{n-1}B_{n-2}B_{n-3}\cdots B_0$ 转换为 n 位格雷码 $G_{n-1}G_{n-2}G_{n-3}\cdots G_0$ 的公式为

$$
\begin{cases}
G_{n-1} = B_{n-1} \\
G_{n-2} = B_{n-1} \oplus B_{n-2} \\
G_{n-3} = B_{n-2} \oplus B_{n-3} \\
\quad \vdots \\
G_0 = B_1 \oplus B_0
\end{cases}
\tag{4.2.5}
$$

（3）根据式(4.2.3)的逻辑表达式，可画出逻辑图如图 4.2.3 所示。

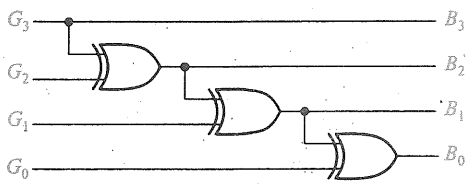


图 4.2.3 例 4.2.2 的逻辑图

从以上逻辑表达式和逻辑图可以看出，用异或门代替与门和或门能使逻辑电路比较简单。在化简和变换逻辑表达式时，注意综合考虑，使各式中的相同乘积项尽可能多，使某些输出作为另一些逻辑门的输入。例如，利用 B_2 作为 B_1 的一个输入，B_1 又作为 B_0 的一个输入，这样可以减少门电路的数目，降低实现电路的成本。

4.2.2 组合逻辑电路的优化实现

逻辑函数的优化实现的主要目的是用指定芯片中特定资源实现逻辑函数，使电路的成

本最低。逻辑电路的成本是由电路中总的逻辑门数量加上所有逻辑门的输入端数来表示。

前面介绍的组合逻辑电路的设计过程中，对满足功能要求的逻辑函数进行代数法或卡诺图法化简，通常得到最简与-或式。可以用与门和或门构成最简的两级与-或结构电路。但实际电路设计不仅要满足功能要求，还要确保能够用指定的芯片实现。因此要对逻辑函数的最简与-或式进行变换，变换成能够满足限定条件的形式。

但当变量数比较多时，手工化简变得很困难，故由 EDA 工具实现优化。EDA 工具的优化策略和实现过程庞大而复杂，由计算机在后台完成。作为入门，这一节简单介绍 EDA 工具实现优化综合时用到的一些技术。

1. 单输出电路

以基本门电路作为可用资源，用两级与-或结构实现最简函数

$$L = AB + CD \tag{4.2.6}$$

得到如图 4.2.4(a)所示电路。如果要求用与非门实现该逻辑电路，需要对式(4.2.6)所示逻辑函数式进行变换，用两次求反的方法得到

$$L = AB + CD = \overline{\overline{AB} \cdot \overline{CD}} \tag{4.2.7}$$

由此可画出逻辑图，如图 4.2.4(b)所示。相同输入端的与非门相对与门或者或门而言，所用三极管少，速度快，实际中多选择使用与非门。同样，或非门相对于或门和与门所用三极管也少，速度更快。

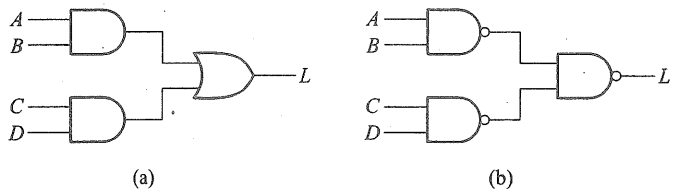


图 4.2.4 以门电路为资源优化实现逻辑函数

(a) 与门和或门结构 (b) 与非门结构

2. 多输出电路

在实际的数字系统中，通常都有多个输出，化简时需要将多个输出函数作为整体考虑，使各式中的相同乘积项尽可能多，以减少门的个数，降低成本。对于下面两个逻辑函数

$$L_1 = AB + \overline{A}C + BD$$

$$L_2 = AB + \overline{A}C + \overline{A}BD$$

如果直接分别实现这两个逻辑函数，需要 6 个与门(5 个 2 输入，1 个 3 输入)和两个 3 输入或门(不考虑非门)，共 19 个输入端，成本为 27。如果考虑共享相同乘积项则只需要 4 个与门和两个或门，共 15 个输入端，成本为 21，逻辑电路如图 4.2.5 所示。

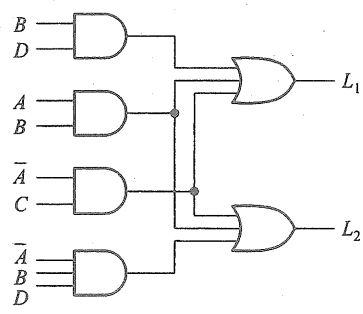


图 4.2.5 共享相同乘积项的逻辑电路

3. 多级逻辑电路

逻辑门电路的优化实现除了要考虑降低成本外，还必须保证能够实现。前面介绍的单输出或多

输出都可以用两级与-或结构实现。当输入变量数增加时，逻辑门输入端的数目将增多。如果所需逻辑门输入端数超过物理实现的可能(参见 3.2.3 节)，则需要变换逻辑表达式，以减少逻辑门的扇入数。变换方法可以是提取公因子或函数分解等。

(1) 提取公因子

对于逻辑函数

$$L=ABCD+AB\overline{C}E+ABD\overline{F}$$

用与-或两级电路实现上述函数需要 3 个 4 输入与门和一个 3 输入或门，共 15 个输入端，成本是 19，如图 4.2.6 所示。如果限定逻辑门的扇入数为 3，需要变换逻辑表达式，提取公因子得

$$L=AB(CD+\overline{C}E+D\overline{F})$$

实现上式逻辑门的最大扇入数为 3，满足限定条件。虽然电路的级数增加到 3 级，但电路的成本减少到 17，如图 4.2.7 所示。

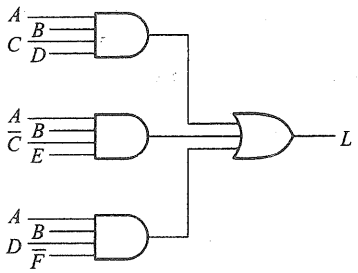


图 4.2.6 提取公因子前的与-或两级电路

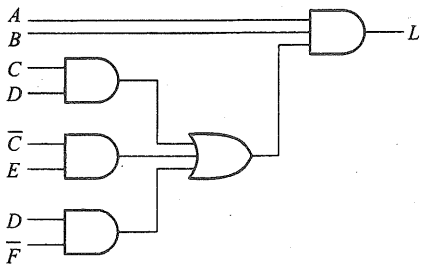


图 4.2.7 提取公因子后的 3 级电路

(2) 函数分解

对于多变量逻辑函数，减少扇入数除了提取公因子外，还可以用函数分解方法减少子函数的变量数目。

对于逻辑函数

$$L=\overline{A}BC+A\overline{B}C+ABD+\overline{A}\,\overline{B}D$$

用与-或两级电路实现上述函数需要 4 个 3 输入与门和一个 4 输入或门，共 16 个输入端，成本为 21，如图 4.2.8 所示。如果限定逻辑门的扇入数为 3，需要对上式进行变换，函数分解得到

$$\begin{aligned}L&=(\overline{A}B+A\overline{B})C+(AB+\overline{A}\,\overline{B})D\\&=(\overline{A}B+A\overline{B})C+\overline{(\overline{A}B+A\overline{B})D}\end{aligned}$$

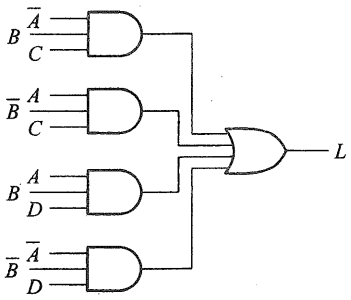


图 4.2.8 函数分解前的与-或两级电路

实现上述逻辑门的最大扇入数为 2，满足限定条件，并且成本减少到 20，但电路级数增加到 5 级，如图 4.2.9 所示。

提取公因子或函数分解可以将与-或两级电路转换为多级电路，减少逻辑门的扇入数，以满足设计要求。但电路级数的增加可能会使总延时增加。因此变换后电路性能的好坏需综合评定。

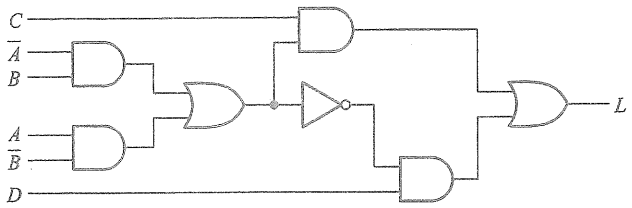


图 4.2.9 函数分解后的 5 级电路

复习思考题

4.2.1 列出设计组合逻辑电路的步骤。

4.2.2 为什么说在组合逻辑电路设计中正确列出真值表是最关键的一步?

4.2.3 什么是逻辑函数的优化实现?这里介绍的优化实现的策略有哪些?

4.3 组合逻辑电路中的竞争-冒险

前面进行组合逻辑电路的分析和设计时,都没有考虑逻辑门的延迟时间对电路产生的影响,并且认为电路的输入和输出均处于稳定的逻辑电平。实际上,信号经过逻辑门电路都需要一定的时间。由于不同路径上门的数目不同,导致信号的传输时间不同。或者传输路径上门的数目相同,而各个门延迟时间有差异,也会造成信号的传输时间不同。因此,电路在信号电平变化瞬间,可能与稳态下的逻辑功能不一致,产生错误输出,这种现象就是电路中的竞争-冒险。

4.3.1 产生竞争-冒险的原因

下面通过两个简单电路的工作情况,说明产生竞争-冒险的原因。图 4.3.1(a)所示的与门在稳态情况下,当 $A=0,B=1$ 或者 $A=1,B=0$ 时,输出 L 始终为 0。如果信号 A、B 的状态从 $A=0,B=1$ 同时翻转为 $A=1,B=0$,则 L 能保持 0 不变。但是,由于前级门电路的延迟差异或其他原因,致使 B 从 1 变为 0 的时刻,滞后于 A 从 0 变为 1 的时刻,即在很短的时间间隔内,A、B 同时为 1,则与门的输出端就会出现一个高电平窄脉冲(干扰脉冲),如图 4.3.1(b)所示,图中考虑了与门的延迟。

同理,图 4.3.2(a)所示的或门在稳态情况下,当 $A=0,B=1$ 或者 $A=1,B=0$ 时,输出 L 始终为 1。而当 A 从 0 变为 1 时刻,滞后于 B 从 1 到 0 的时刻,则在很短的时间间隔内,或门的两个输入端均为 0,使输出出现一个低电平窄脉冲,如图 4.3.2(b)所示。

下面进一步分析组合逻辑电路产生的竞争-冒险。图 4.3.3(a)所示的逻辑电路中,它的输出逻辑表达式为 $L=AC+B\overline{C}$。由此式可知,当 A 和 B 都为 1 时,表达式简化成两个互补信号相加,即 $L=C+\overline{C}$,此时,该电路可能存在竞争-冒险。由图 4.3.3(b)所示的波形图可以看出,在 C 由 1 变 0 时,$\overline{C}$ 由 0 变 1 有一延迟时间,G_2 和 G_3 的输出 AC 和 $B\overline{C}$ 分别相对于 C 和 $\overline{C}$ 均有延迟,AC 和 $B\overline{C}$ 经过 G_4 的延迟而使输出出现一负跳变的窄脉冲。

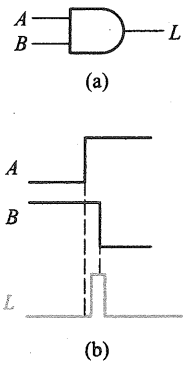


图 4.3.1 产生正跳变脉冲的竞争-冒险
(a) 逻辑电路 (b) 工作波形

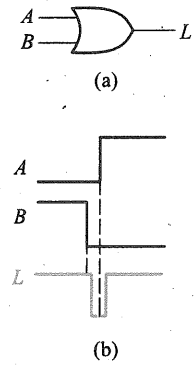


图 4.3.2 产生负跳变脉冲的竞争-冒险
(a) 逻辑电路 (b) 工作波形

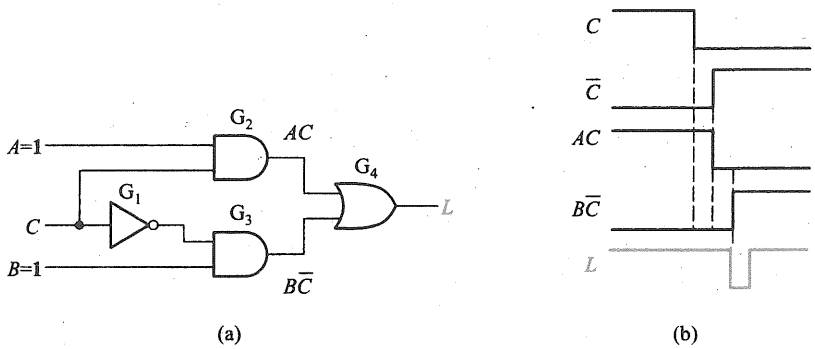


图 4.3.3 组合逻辑电路的竞争-冒险
(a) 逻辑电路 (b) 工作波形

综上所述,在一定条件下,当一个逻辑门的两个输入端的信号同时向相反方向变化,而变化的时间有差异的现象,称为竞争。两个输入端可以是不同变量所产生的信号,但其取值的变化方向是相反的,参见图 4.3.1 和图 4.3.2 中的 AB 及 $A+B$。也可以是在一定条件下,门电路输出端的逻辑表达式简化成两个互补信号相乘或者相加,即 $L=A\cdot\overline{A}$ 或者 $L=A+\overline{A}$,如图 4.3.3 所示。由竞争而可能产生输出干扰脉冲的现象称为冒险。

在考虑延迟的条件下,若与门的两个输入 A 和 $\overline{A}$,其中一个先从 0 变 1 时,则 $A\cdot\overline{A}$ 会向其非稳定值1变化,此时会产生冒险;若或门的两个输入 A 和 $\overline{A}$,其中一个先从 1 变 0 时,则 $A+\overline{A}$ 会向其非稳定值 0 变化,也会产生冒险。两者之间存在对偶关系。

值得注意的是,有竞争现象时不一定都会产生干扰脉冲,如图 4.3.2(a)中,如果 B 从 1 变为 0 时刻滞后 A 的变化,则输出不会产生冒险。

4.3.2 消去竞争-冒险的方法

针对上述分析,可以采取以下措施来消去竞争-冒险现象。

1. 发现并消去互补项

例如,函数式 $F=(A+B)(\overline{A}+C)$,在 $B=C=0$ 时,$F=A\cdot\overline{A}$。若直接根据这个逻辑表达式组

成逻辑电路，如图 4.3.4(a)所示，则可能出现竞争-冒险。如将该式变换为 $F=A\overline{A}+AC+\overline{A}B+BC=AC+\overline{A}B+BC$，这里已将 $A\overline{A}$ 消掉。根据这个表达式组成逻辑电路，如图 4.3.4(b)所示，就不会出现竞争-冒险。

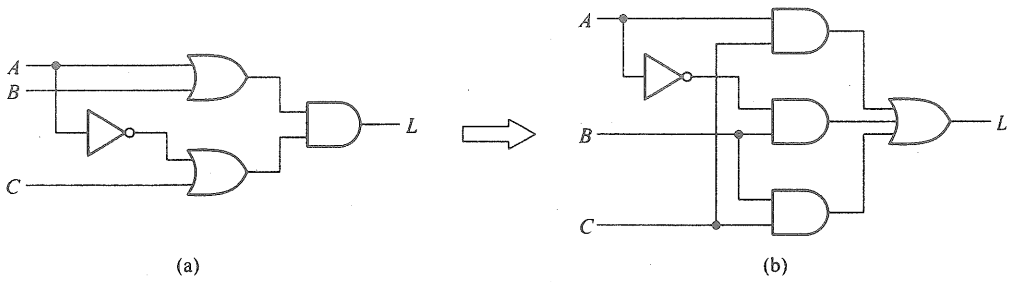


图 4.3.4 逻辑表达式的不同实现方法

(a) 存在竞争-冒险的电路 (b) 无竞争-冒险的电路

2. 增加乘积项以避免互补项相加

对于图 4.3.3(a)所示的逻辑电路的输出 $L=AC+B\overline{C}$，其卡诺图如图 4.3.5 所示，两个相邻包围圈①、②之间存在相邻最小项 m_6 和 m_7，当 $A=B=1$ 时，使得 $L=C+\overline{C}$。如果增加一个包围圈 AB，使输出变为 $L=AC+B\overline{C}+AB$，则 $A=B=1$ 时，$L=C+\overline{C}+1$，不会只出现互补项相加的情况。而此时电路中，G_5 输出为 1，使 G_4 输出亦为 1，这就消除了 C 的状态变化对输出状态的影响，从而消去了竞争-冒险，如图 4.3.6 所示。

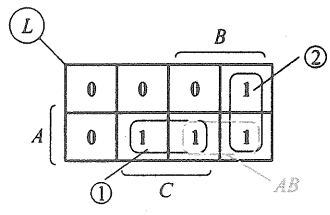


图 4.3.5 增加了乘积项 AB 的卡诺图

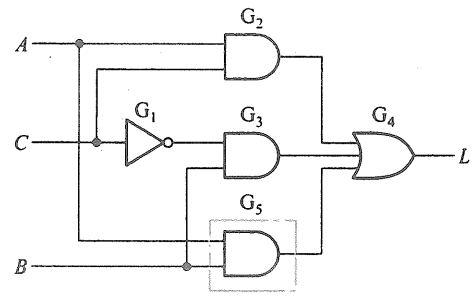


图 4.3.6 增加了乘积项 AB 的逻辑电路

3. 输出端并联电容器

对于工作速度不高的逻辑门构成的电路，为了消去竞争-冒险产生的干扰窄脉冲，可以在输出端并联一滤波电容，其容量为 4~20 pF 之间。若在图 4.3.3(a)所示电路的输出端并联电容 C，其输出端等效电路如图 4.3.7(a)所示，R_o 是逻辑门电路的输出电阻。当 $A=B=1$，C 的波形与图 4.3.3 相同的情况下，得到如图 4.3.7(b)所示的波形图。显然，电容对输出窄脉冲起到平波的作用，消除输出端出现的逻辑错误，但同时也使输出波形上升沿或下降沿变得缓慢。

以上介绍了产生竞争-冒险的原因和克服竞争-冒险的方法。现代数字电路或数字系统的分析与设计，可以借助计算机进行时序仿真，检查电路是否存在竞争-冒险现象。仿真时，由于逻辑门电路的传输延迟时间是采用软件设定的标准值或设计者自行设定的值，与电

路的实际工作情况有差异，最终要在实验中检查验证。因此，要能很好地解决这一问题，还必须在实践中积累和总结经验。

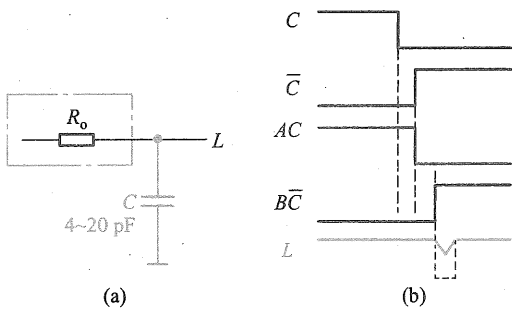


图 4.3.7 并联电容器消去竞争-冒险

(a) 图 4.3.3(a)的输出等效电路 (b) 波形图

复习思考题

4.3.1 什么是组合逻辑电路中的竞争-冒险？

4.3.2 列出三种消去组合逻辑电路竞争-冒险的方法。

4.4 常用组合逻辑电路模块

在大规模可编程器件普遍应用之前，许多具有特定功能的常用组合逻辑电路被制成了单个的中规模集成芯片，为构建数字系统提供了极大方便。在现代数字系统设计中，这种中规模芯片已经被淘汰，但它们作为基本模块在系统设计中仍被广泛采用。本节将从功能模块而非集成芯片的角度介绍这些电路。这些模块包括编码器、译码器、数据选择器、数据分配器、数值比较器、算术运算单元等。

4.4.1 编码器

编码和译码问题在日常生活中经常遇到。例如，你办理移动电话时，通信公司给你的电话设定一个号码，叫作编码。显然，这个特定的号码与你的姓名是等同的，任何人拨打你的移动电话号码，都能够找到你，这叫作译码。

1. 编码器的定义与功能

数字系统中存储或处理的信息，常常是用二进制码表示的。用一个二进制代码表示特定含义的信息称为编码。具有编码功能的逻辑电路称为编码器。图 4.4.1 所示为二进制编码器结构图，它有 n 位二进制码输出，与 2^n 个输入相对应。编码器有普通编码器和优先编码器之分。

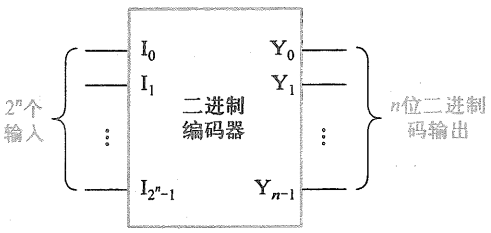


图 4.4.1 二进制编码器结构图

普通编码器任何时刻只允许一个输入信号有效，否则将产生错误输出。优先编码器允许多个输入信号同时有效，输出是对优先级别高的输

入信号进行编码。

2. 普通编码器

4 线-2 线普通编码器功能表如表 4.4.1 所示，根据功能表可以设计出编码器电路。4 个输入 $I_0 \sim I_3$ 为高电平有效，输出是 2 位二进制代码 Y_1Y_0。任何时刻 $I_0 \sim I_3$ 中只能有一个取值为 1，并且有一组对应的二进制代码输出。除表中列出 4 个输入变量的 4 种取值组合有效外，其余 12 种组合所对应的输出均应为 0。由真值表可以得到如下逻辑表达式：

$$\begin{aligned} Y_1 &= \bar{I}_0\bar{I}_1I_2\bar{I}_3 + \bar{I}_0\bar{I}_1\bar{I}_2I_3 \\ Y_0 &= \bar{I}_0I_1\bar{I}_2\bar{I}_3 + \bar{I}_0\bar{I}_1\bar{I}_2I_3 \end{aligned} \tag{4.4.1}$$

在任何时刻只有一个输入为 1 的前提下，可以进一步简化式(4.4.1)，将表 4.4.1 中没有列出的 12 种组合所对应的输出看作无关项，此时可得

$$\begin{aligned} Y_1 &= I_2 + I_3 \\ Y_0 &= I_1 + I_3 \end{aligned} \tag{4.4.2}$$

式(4.4.2)是考虑了无关项的简化结果，比式(4.4.1)简单。根据式(4.4.2)的逻辑表达式画出逻辑图如图 4.4.2 所示。

表 4.4.1 4 线-2 线编码器功能表

输入				输出	
I_0	I_1	I_2	I_3	Y_1	Y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

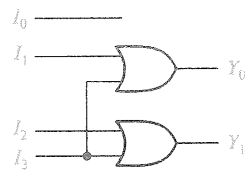


图 4.4.2 4 线-2 线编码器逻辑图

应当特别注意，这一电路实现正常编码时，对输入信号有严格的限制，即任何时刻 $I_0 \sim I_3$ 中只能并且必须有一个取值为 1。例如，当 I_1、I_2 同时为 1 时，输出出现错误编码 $Y_1Y_0 = 11$。因为正常编码时，输出 11 表示 I_3 为 1。

上述编码器还存在一个问题，当电路所有的输入为 0 时，输出 Y_1、Y_0 为 0。而当 I_0 为 1 时，输出 Y_1、Y_0 也全为 0，即输入条件不同而输出代码相同。这两种情况在实际中必须加以区分，解决的方法将在下面例题中介绍。

例 4.4.1 键盘输入逻辑电路就是由编码器组成的。图 4.4.3 是用十个按键和门电路组成的 8421BCD 码编码器，其功能表如表 4.4.2 所示，十个按键 $\bar{S}_0 \sim \bar{S}_9$ 分别对应十进制数 0~9，编码器的输出为 $ABCD$ 和 GS，试分析该电路的工作原理及 GS 的作用。

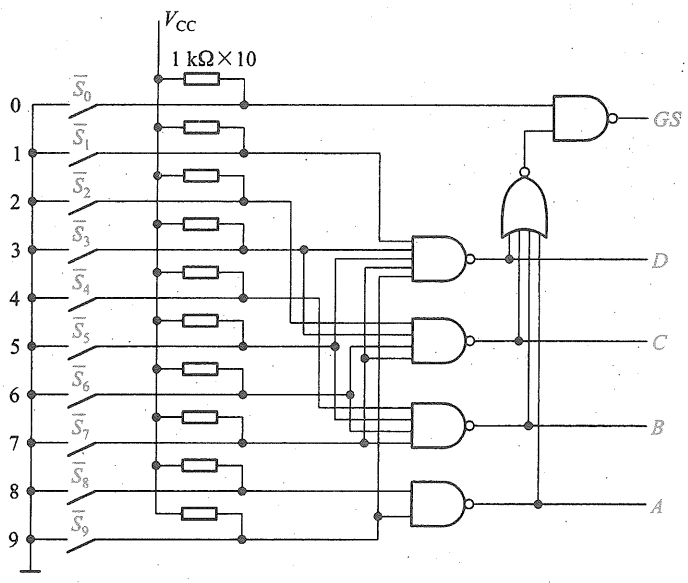


图 4.4.3 用十个按键和门电路组成的 8421BCD 码编码器

解：对功能表和逻辑电路进行分析，可得知：① 该编码器为输入低电平有效，因此用带非号的变量表示；② 在按下 $\overline{S}_0 \sim \overline{S}_9$ 中任意一个键时，即输入信号中有一个为低电平时，*ABCD* 输出信号为该键的 BCD 编码，同时 $GS=1$，表示有信号输入。而只有 $\overline{S}_0 \sim \overline{S}_9$ 均为高电平时 $GS=0$，表示无信号输入，此时的输出代码 0000 为无效代码。由此解决了前面 4 线-2 线编码器存在的问题，即输入条件不同而输出代码相同的问题。

表 4.4.2 十个按键 8421BCD 码编码器功能表

输入										输出				
$\overline{S}_9$	$\overline{S}_8$	$\overline{S}_7$	$\overline{S}_6$	$\overline{S}_5$	$\overline{S}_4$	$\overline{S}_3$	$\overline{S}_2$	$\overline{S}_1$	$\overline{S}_0$	*A*	*B*	*C*	*D*	*GS*
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0
1	1	1	1	1	1	1	1	1	0	0	0	0	0	1
1	1	1	1	1	1	1	1	0	1	0	0	0	1	1
1	1	1	1	1	1	1	0	1	1	0	0	1	0	1
1	1	1	1	1	1	0	1	1	1	0	0	1	1	1
1	1	1	1	1	0	1	1	1	1	0	1	0	0	1
1	1	1	1	0	1	1	1	1	1	0	1	0	1	1
1	1	1	0	1	1	1	1	1	1	0	1	1	0	1
1	1	0	1	1	1	1	1	1	1	0	1	1	1	1
1	0	1	1	1	1	1	1	1	1	1	0	0	0	1
0	1	1	1	1	1	1	1	1	1	1	0	0	1	1

在实际应用中，经常会遇到两个以上的输入同时为有效信号的情况。因此，必须根据轻重缓急，事先规定好这些输入编码的先后次序，即优先级别。识别这类请求信号的优先级别并进行编码的逻辑电路称为优先编码器。

3. 优先编码器

8 线-3 线优先编码器功能如表 4.4.3 所示。

表 4.4.3 8 线-3 线优先编码器功能表

输入									输出				
EI	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0	GS	EO
0	×	×	×	×	×	×	×	×	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	1
1	1	×	×	×	×	×	×	×	1	1	1	1	0
1	0	1	×	×	×	×	×	×	1	1	0	1	0
1	0	0	1	×	×	×	×	×	1	0	1	1	0
1	0	0	0	1	×	×	×	×	1	0	0	1	0
1	0	0	0	0	1	×	×	×	0	1	1	1	0
1	0	0	0	0	0	1	×	×	0	1	0	1	0
1	0	0	0	0	0	0	1	×	0	0	1	1	0
1	0	0	0	0	0	0	0	1	0	0	0	1	0

该编码器有 8 个信号输入端 $I_7 \sim I_0$，3 位二进制码输出端 $Y_2Y_1Y_0$，输入和输出均以高电平作为有效电平。从功能表可以判断输入 $I_7 \sim I_0$ 的优先级别。例如，当 I_7 为 1 时，无论其他输入是否为有效输入电平，输出 $Y_2Y_1Y_0$ 为 111 是对 I_7 进行的编码，说明 I_7 的优先级别最高。只有当其 $I_7 \sim I_1$ 均为 0，而 I_0 为 1 时，输出 $Y_2Y_1Y_0$ 为 000 是对 I_0 进行的编码，说明 I_0 的优先级别最低。因此，8 个输入信号的优先级别由高到低的顺序依次为 $I_7, I_6, \cdots, I_0$。

此外，为便于多个编码器级联扩展，还设置了高电平有效的输入使能端 EI 和输出使能端 EO，以及优先编码工作状态标志 GS。

当 $EI=1$ 时，编码器工作；而当 $EI=0$ 时，禁止编码器工作，此时不论 8 个输入端为何种状态，3 个输出端均为低电平，且 GS 和 EO 均为低电平。

只有在 EI 为 1，且所有输入端都为 0 时，EO 输出为 1，它可与另一个相同编码器的 EI 连接，以便组成更多输入端的优先编码器。

GS 的功能是，当 EI 为 1，且至少有一个输入端有高电平信号输入时，GS 为 1，表明编码器输出有效编码，否则 GS 为 0。由此可以区分当电路所有输入端均无高电平输入，或者只有 I_0 输入端有高电平时，$Y_2Y_1Y_0$ 均为 000 的情况。

根据功能表推导出各输出端的逻辑表达式为

$$\begin{cases} Y_2 = EI(I_7 + I_6 + I_5 + I_4) \\ Y_1 = EI(I_7 + I_6 + \bar{I}_5\bar{I}_4 I_3 + \bar{I}_5\bar{I}_4 I_2) \\ Y_0 = EI(I_7 + \bar{I}_6 I_5 + \bar{I}_6\bar{I}_4 I_3 + \bar{I}_6\bar{I}_4\bar{I}_2 I_1) \\ EO = EI(\bar{I}_7\bar{I}_6\bar{I}_5\bar{I}_4\bar{I}_3\bar{I}_2\bar{I}_1\bar{I}_0) \\ GS = EI(I_7 + I_6 + I_5 + I_4 + I_3 + I_2 + I_1 + I_0) \end{cases} \tag{4.4.3}$$

由式(4.4.3)可以画出逻辑图(此处省略),逻辑符号如图 4.4.4 所示。

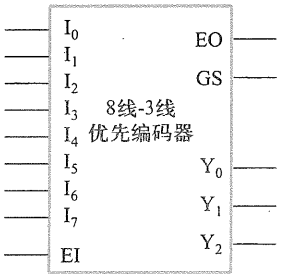


图 4.4.4 8 线-3 线优先编码器的逻辑符号

例 4.4.2 用 8 线-3 线优先编码器组成 16 线-4 线编码器,其逻辑图如图 4.4.5 所示,试分析其工作原理。

教学视频 4.1:
编码器的扩展

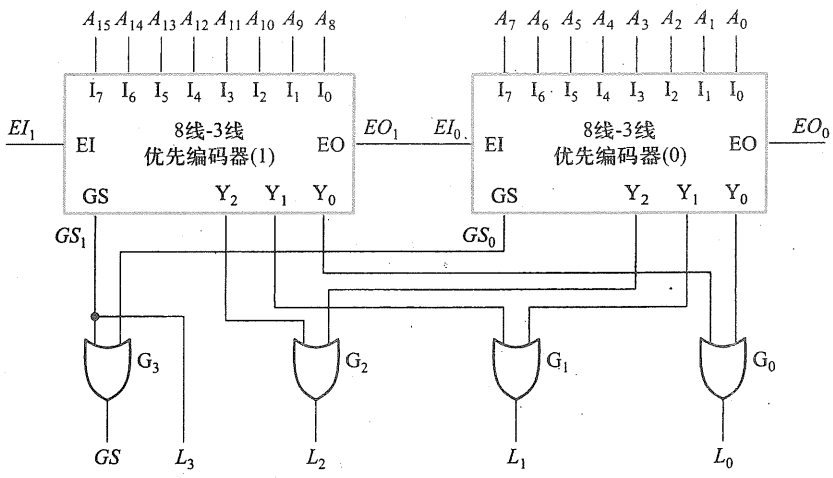


图 4.4.5 例 4.4.2 的逻辑图

解:根据 8 线-3 线优先编码器的功能表,对逻辑图进行分析得出:

(1) 当 $EI_1 = 0$ 时,编码器(1)禁止编码,其输出端 $Y_2\ Y_1\ Y_0$ 为 000,而且 GS_1、EO_1 均为 0。同时 EO_1 使 $EI_0 = 0$,编码器(0)也禁止编码,其输出端及 GS_0、EO_0 均为 0。由电路图可知 $GS = GS_1 + GS_0 = 0$,表示此时电路输出端的代码 $L_3\ L_2\ L_1\ L_0 = 0000$ 是非编码输出。

(2) 当 $EI_1 = 1$ 时,编码器(1)允许编码,若 $A_{15} \sim A_8$ 均无有效电平输入,则 $EO_1 = 1$,使 $EI_0 = 1$,从而允许编码器(0)编码,因此编码器(1)的优先级高于编码器(0)。

由于 $A_{15}\sim A_8$ 没有有效电平输入，所以编码器(1)的输出均为 0，使 4 个或门都打开，L_2、L_1、L_0 取决于编码器(0)的输出，而 $L_3=GS_1$ 总是等于 0，所以输出代码在 0000～0111 之间变化。若只有 A_0 有高电平输入，输出为 0000，若 A_7 及其他输入同时有高电平输入，则输出为 0111。A_0 的优先级别最低。

(3) 当 $EI_1=1$ 且 $A_{15}\sim A_8$ 中至少有一个为高电平输入时，$EO_1=0$，使 $EI_0=0$，编码器(0)禁止编码，其输出均为 0，此时 $L_3=GS_1=1$，L_2、L_1、L_0 取决于编码器(1)的输出，输出代码在 1000～1111 之间变化，并且 A_{15} 的优先级别最高。

电路实现了 16 位输入的优先编码，优先级别从 A_{15} 至 A_0 依次递减。

复习思考题

4.4.1.1 普通编码器和优先编码器的区别是什么？

4.4.1.2 图 4.4.4 所示 8 线-3 线优先编码器的输入、输出信号是高电平有效还是低电平有效？

4.4.1.3 说明图 4.4.4 所示 8 线-3 线优先编码器的输入信号 EI 和输出信号 GS、EO 的作用。

4.4.2 译码器/数据分配器

在数字系统中，经常需要将一种代码转换为另一种代码，以满足特定的需要，完成这种功能的电路称为码转换电路。译码器和编码器都是码转换电路。

1. 译码器的定义与功能

译码是编码的逆过程，它的功能是将具有特定含义的二进制码转换成对应的输出信号，具有译码功能的逻辑电路称为译码器。

译码器可分为两种类型，一种是将一系列代码转换成与之一一对应的有效信号。这种译码器可称为二进制译码器或唯一地址译码器，常用于存储器中单元地址的译码，即将每一个地址代码转换成一个有效信号，从而选中对应的单元。另一种是将一种代码转换成另一种代码，所以也称为代码变换器。

图 4.4.6 表示二进制译码器的一般框图，它具有 n 个输入端、2^n 个输出端和一个使能输入端。在使能输入端为有效电平时，对应每一组输入代码，只有其中一个输出端为有效电平，其余输出端则为相反电平。输出信号可以是高电平有效，也可以是低电平有效。

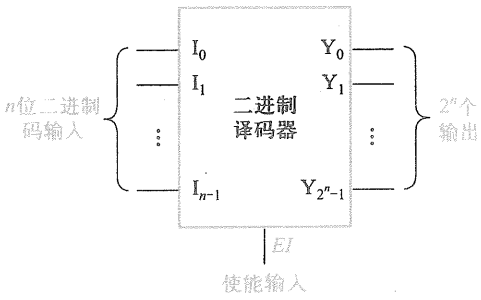


图 4.4.6 二进制译码器框图

2. 二进制译码器

(1) 2 线-4 线译码器

2 线-4 线译码器有 2 个输入变量 A_1、A_0，共有 4 种不同状态组合，因而有 4 个输出信号 $\overline{Y}_0\sim\overline{Y}_3$。当输出为低电平有效时，真值表如表 4.4.4 所示。

另外 $\overline{E}$ 为使能控制端，当 $\overline{E}$ 为 1 时，无论 A_1、A_0 为何种状态，输出全为 1，译码器处于非工作状态，没有有效信号输出。而当 $\overline{E}$ 为 0 时，对应于 A_1、A_0 的一种输入状态，其中只有一个输出端为 0，其余各输出端均为 1。例如，$A_1A_0=00$ 时，$\overline{Y}_0$ 为 0，$\overline{Y}_1\sim\overline{Y}_3$ 均为 1。由此可见，

表 4.4.4 2线-4线译码器真值表

输入			输出			
$\overline{E}$	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$
1	×	×	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

译码器是通过输出端的逻辑电平来识别不同的代码的。

根据真值表可写出各输出端的逻辑表达式为

$$\begin{cases} \overline{Y}_0 = \overline{\overline{E}\,\overline{A}_1\overline{A}_0} \\ \overline{Y}_1 = \overline{\overline{E}\,\overline{A}_1 A_0} \\ \overline{Y}_2 = \overline{\overline{E} A_1 \overline{A}_0} \\ \overline{Y}_3 = \overline{\overline{E} A_1 A_0} \end{cases} \tag{4.4.4}$$

根据逻辑表达式画出逻辑图如图 4.4.7(a)所示，对于这种特定功能模块，常用图 4.4.7(b)所示逻辑符号表示。

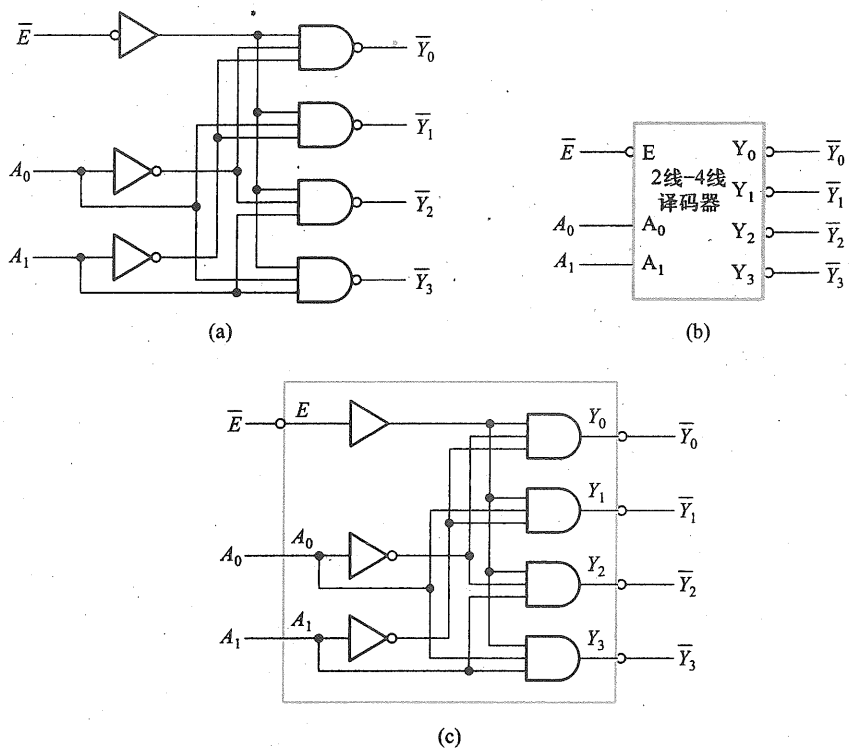


图 4.4.7 2线-4线译码器

(a) 逻辑图 (b) 逻辑符号 (c) 逻辑符号结构

逻辑符号说明 图 4.4.7(c)框内画出逻辑符号的内部结构，框外部的 $\overline{E}$、$\overline{Y}_0 \sim \overline{Y}_3$ 作为变量符号，表示外部输入或输出信号名称，字母上面的“-”号说明该输入或输出是低电平有效。框内部的输入、输出变量表示其内部的逻辑关系，全部为高电平有效。当输入或输出为低电平有效时，符号框外部逻辑变量 $\overline{E}$、$\overline{Y}_0 \sim \overline{Y}_3$ 的逻辑状态与符号框内相应的 E、$Y_0 \sim Y_3$ 的逻辑状态相反。在推导表达式的过程中，如果低有效的输入或输出变量上面的“-”号参与运算，则在画逻辑图或验证真值表时，注意将其还原为低有效符号。

(2) 3 线-8 线译码器

一种 3 个使能控制端的 3 线-8 线译码器的功能表如表 4.4.5 所示。图 4.4.8 是它的逻辑符号。该译码器有 3 位二进制输入 A_2、A_1、A_0，它们共有 8 种组合状态，即可译出 8 个输出信号 $\overline{Y}_0 \sim \overline{Y}_7$，输出为低电平有效。此外，还设置了 3 个使能输入端 E_3、$\overline{E}_2$ 和 $\overline{E}_1$，为扩展电路的功能提供了方便。这里设 $E = E_3 \overline{E}_2 \overline{E}_1$，由功能表可写出：

$$\begin{cases} \overline{Y}_0 = \overline{E \overline{A}_2 \overline{A}_1 \overline{A}_0} \\ \overline{Y}_1 = \overline{E \overline{A}_2 \overline{A}_1 A_0} \\ \overline{Y}_2 = \overline{E \overline{A}_2 A_1 \overline{A}_0} \\ \overline{Y}_3 = \overline{E \overline{A}_2 A_1 A_0} \\ \overline{Y}_4 = \overline{E A_2 \overline{A}_1 \overline{A}_0} \\ \overline{Y}_5 = \overline{E A_2 \overline{A}_1 A_0} \\ \overline{Y}_6 = \overline{E A_2 A_1 \overline{A}_0} \\ \overline{Y}_7 = \overline{E A_2 A_1 A_0} \end{cases} \tag{4.4.5}$$

表 4.4.5 3 线-8 线译码器功能表

输入						输出							
E_3	$\overline{E}_2$	$\overline{E}_1$	A_2	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
×	1	×	×	×	×	1	1	1	1	1	1	1	1
×	×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	0	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0

当 $E_3=1$，且 $\overline{E}_2=\overline{E}_1=0$ 时，$E=1$，代入式(4.4.5) 可得

$$\overline{Y}_0=\overline{m}_0,\overline{Y}_1=\overline{m}_1,\overline{Y}_2=\overline{m}_2,\overline{Y}_3=\overline{m}_3,\overline{Y}_4=\overline{m}_4,\overline{Y}_5=\overline{m}_5,\overline{Y}_6=\overline{m}_6,\overline{Y}_7=\overline{m}_7$$

译码器的输出包含了输入 A_2、A_1、A_0 组成的所有最小项。

功能表 4.4.5 中的前 3 行，是在使能输入信号控制下的禁止译码状态。

74HC138 是 3 线-8 线 CMOS 集成译码器，其功能表和逻辑符号分别如表 4.4.5 与图 4.4.8所示。74HC139 是双 2 线-4 线 CMOS 集成译码器，其内部封装了两个独立的与图 4.4.7(b)相同的译码器。

利用 3 线-8 线译码器可以构成 4 线-16 线、5 线-32 线或 6 线-64 线译码器。

教学视频 4.2：74HC138 扩展成 4 线-16 线

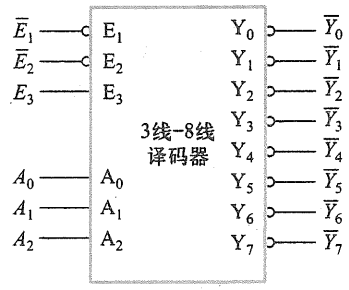


图 4.4.8 3 线-8 线译码器逻辑符号

例 4.4.3 试用四片 3 线-8 线译码器(74HC138)和一片 2 线-4 线译码器(74HC139)构成 5 线-32 线译码器，输入为 5 位二进制码 $B_4B_3B_2B_1B_0$，对应输出 $\overline{L}_0\sim\overline{L}_{31}$ 为低电平有效信号。

解：首先列出 5 线-32 线译码器的真值表如表 4.4.6 所示。

表 4.4.6 5 线-32 线译码器真值表

输入					输出										
B_4	B_3	B_2	B_1	B_0	$\overline{L}_0$	$\overline{L}_1$	$\overline{L}_2$	$\overline{L}_3$	$\overline{L}_4$	…	$\overline{L}_{27}$	$\overline{L}_{28}$	$\overline{L}_{29}$	$\overline{L}_{30}$	$\overline{L}_{31}$
0	0	0	0	0	0	1	1	1	1	…	1	1	1	1	1
0	0	0	0	1	1	0	1	1	1	…	1	1	1	1	1
0	0	0	1	0	1	1	0	1	1	…	1	1	1	1	1
0	0	0	1	1	1	1	1	0	1	…	1	1	1	1	1
0	0	1	0	0	1	1	1	1	0	…	1	1	1	1	1
		⋮								⋮					
1	1	0	1	1	1	1	1	1	1	…	0	1	1	1	1
1	1	1	0	0	1	1	1	1	1	…	1	0	1	1	1
1	1	1	0	1	1	1	1	1	1	…	1	1	0	1	1
1	1	1	1	0	1	1	1	1	1	…	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	…	1	1	1	1	0

从表中可以看出，当 $B_4B_3=00$，而 $B_2B_1B_0$ 从 000 变化到 111 时，对应 $\overline{L}_0\sim\overline{L}_7$ 中有一个输出为 0，其余输出全为 1，因此四片 3 线-8 线译码器(74HC138)中，设置片 (0)为译码状态，其余三片为禁止译码状态，对应的输出 $\overline{L}_8\sim\overline{L}_{31}$ 全为 1。

以此类推，当 $B_4B_3=01$，$B_2B_1B_0$ 从 000 变化到 111 时 $\overline{L}_8\sim\overline{L}_{15}$ 分别输出有效信号，此时设置片(1)为译码状态。当 $B_4B_3=10$ 和 11 时，分别设置片(2)和片(3)为译码状态。因此，将 5 位二进制码的低 3 位 $B_2B_1B_0$ 分别与四片 3 线-8 线译码器的 3 个地址输入端 $A_2A_1A_0$ 并接在一起。

高位 B_4B_3 有 4 种状态的组合，因此接入 2 线-4 线译码器(74HC139)的两个地址输入端 A_1A_0 进行第一级译码，2 线-4 线译码器的 4 个低有效输出信号分别接入四片 3 线-8 线译码器的低使能输入端，使它们在 B_4B_3 的控制下轮流工作在译码状态，即四片 3 线-8 线译码器进行第二级译码。这样就得到由两级译码构成的 5 线-32 线译码器，逻辑图如图 4.4.9 所示。

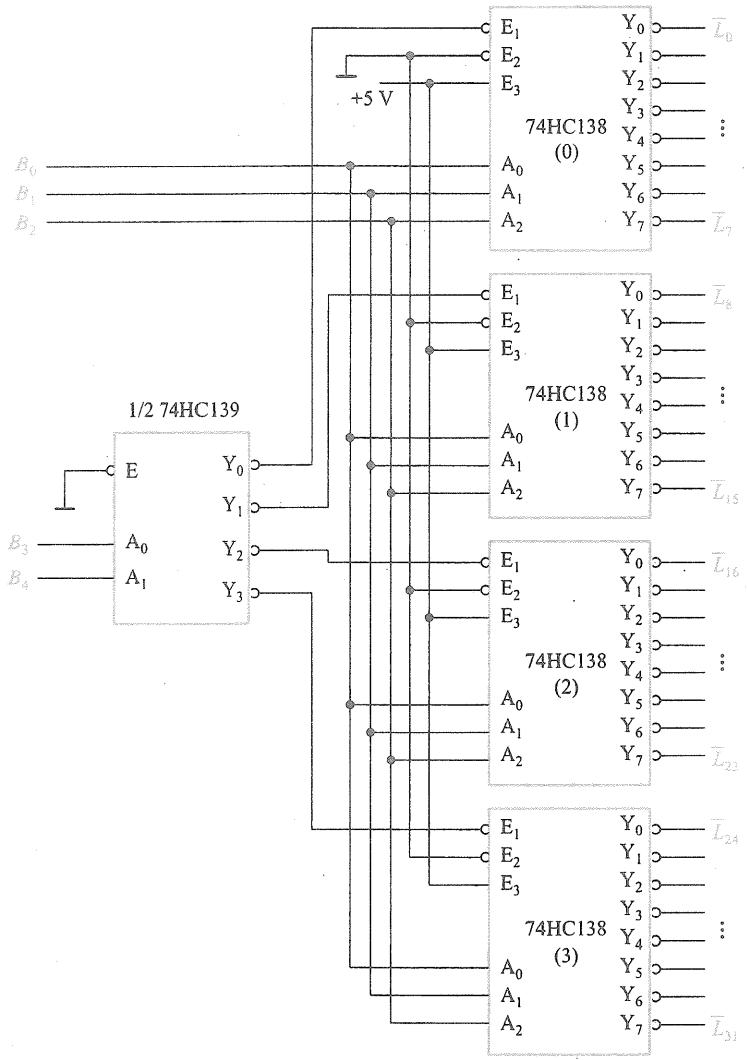


图 4.4.9 例 4.4.3 的逻辑图

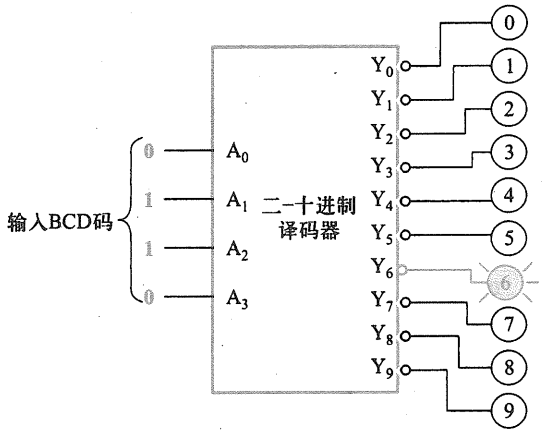


图 4.4.11 二-十进制译码器应用电路

4. 七段显示译码器

在数字测量仪表和各种数字系统中，都需要将数字量直观地显示出来，数字显示电路通常由译码驱动器和显示器等部分组成。数码显示器就是用来显示数字、文字或符号的器件。七段式数字显示器在日常生活中广泛使用。图 4.4.12 表示七段式数字显示器利用不同发光段的组合，显示 0~9 阿拉伯数字。有些数码显示器增加了一段，作为小数点。

七段式数字显示器，也称为七段数码管。常见的七段显示器有发光二极管和液晶显示器两种，这里主要介绍前者。发光二极管构成的七段显示器有共阴极和共阳极两种，如图 4.4.13所示。共阴极电路中，八个发光二极管的阴极连在一起接低电平，需要某一段发光，就将相应二极管的阳极接高电平。共阳极显示器的驱动则刚好相反。

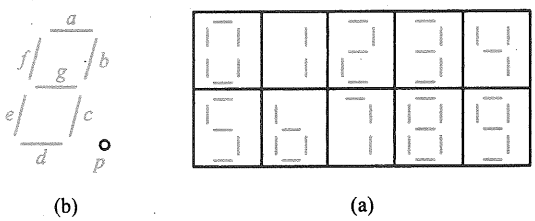


图 4.4.12 七段数字显示器发光段组合图

(a) 分段布置图 (b) 段组合图

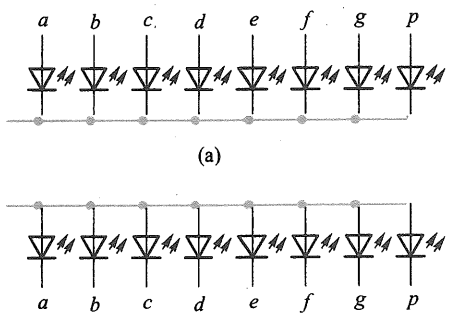


图 4.4.13 二极管显示器等效电路

(a) 共阴极电路 (b) 共阳极电路

为了使数码管能显示十进制数，必须将十进制数的代码经译码器译出，然后经驱动器点亮对应的段。例如，对于 8421BCD 码的 **0011** 状态，对应的十进制数为 3，则译码驱动器应点亮 a、b、c、d、g 各段。译码器的功能就是将 8421BCD 码转换为段的驱动信号。

常用的七段显示译码器有两类：一类译码器输出高电平有效信号，用来驱动共阴极显示器；另一类输出低电平有效信号，以驱动共阳极显示器。下面介绍输出高电平有效的七段显

示译码器(74HC4511)。

七段显示译码器功能表如表4.4.8所示。当输入$D_3D_2D_1D_0$为8421BCD码时,输出高电平有效,驱动共阴极显示器。当输入为1010~1111六个状态时,输出全为低电平,显示器无显示。该显示译码器设有三个辅助控制端LE、$\overline{BL}$、$\overline{LT}$,以增强器件的功能。

① 灯测试输入$\overline{LT}$

当$\overline{LT}=0$时,无论其他输入端是什么状态,所有输出$a\sim g$均为1,显示器显示字形8。该输入端常用于检查译码器本身及显示器各段的好坏。

表4.4.8 七段显示译码器(74HC4511)功能表

十进制或功能	输入							输出							字形
	LE	$\overline{BL}$	$\overline{LT}$	D_3	D_2	D_1	D_0	a	b	c	d	e	f	g	
0	0	1	1	0	0	0	0	1	1	1	1	1	1	0	0
1	0	1	1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	1	1	0	0	1	0	1	1	0	1	1	0	1	2
3	0	1	1	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	1	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	1	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	1	0	0	0	1	1	1	1	1	6
7	0	1	1	0	1	1	1	1	1	1	0	0	0	0	7
8	0	1	1	1	0	0	0	1	1	1	1	1	1	1	8
9	0	1	1	1	0	0	1	1	1	1	1	0	1	1	9
10	0	1	1	1	0	1	0	0	0	0	0	0	0	0	熄灭
11	0	1	1	1	0	1	1	0	0	0	0	0	0	0	熄灭
12	0	1	1	1	1	0	0	0	0	0	0	0	0	0	熄灭
13	0	1	1	1	1	0	1	0	0	0	0	0	0	0	熄灭
14	0	1	1	1	1	1	0	0	0	0	0	0	0	0	熄灭
15	0	1	1	1	1	1	1	0	0	0	0	0	0	0	熄灭
灯测试	×	×	0	×	×	×	×	1	1	1	1	1	1	1	8
灭灯	×	0	1	×	×	×	×	0	0	0	0	0	0	0	熄灭
锁存	1	1	1	×	×	×	×				*				*

* 此时输出状态取决于LE由0跳变为1时BCD码的输入。

② 灭灯输入$\overline{BL}$

当$\overline{BL}=0$,并且$\overline{LT}=1$时,无论其他输入端是什么电平,所有输出$a\sim g$均为0,所以字形熄灭。该输入端用于将不必要显示的零熄灭,例如一个6位数字023.050,将首、尾多余的0熄灭,则显示为23.05,使显示结果更加清楚,参考习题4.4.10电路图。

③ 锁存使能输入LE

在$\overline{BL}=\overline{LT}=1$的条件下,当$LE=0$时,锁存器不工作,译码器的输出随输入码的变化而变化;当LE由0跳变为1时,输入码被锁存,输出只取决于锁存器的内容,不再随输入的变化而变化。有关锁存器的内容将在第五章介绍。

七段显示译码器(74HC4511)与七段显示器的连接方式如图4.4.14所示。

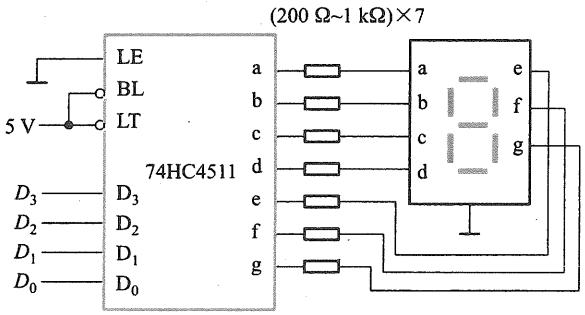


图4.4.14 七段显示译码器与显示器的连接

例4.4.5 由2线-4线译码器、显示译码器(74HC4511)和4个七段显示器构成的4位动态显示电路及时序图如图4.4.15所示,试分析其工作原理。

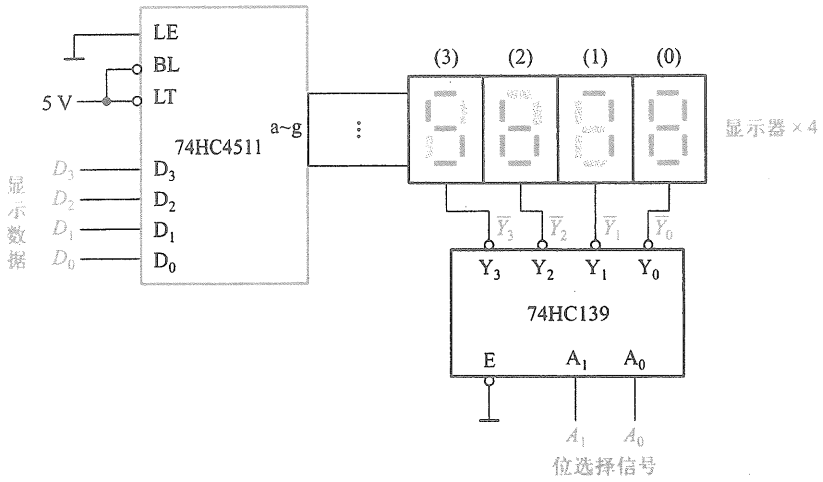
解:如果每位显示器配一个显示译码器驱动,当显示器的位数比较多时,电路的复杂程度会增加。采用动态扫描方式,可以减少显示译码器数目,简化电路连线。

图中4个显示器对应的$a\sim g$段分别接在一起,并与显示译码器的输出$a\sim g$分别相连。将要显示的BCD码表示的十进制数据组分别送到显示译码器的输入端$D_3D_2D_1D_0$。2线-4线译码器的选择信号A_1A_0控制输出$\overline{Y_0}\sim\overline{Y_3}$依次产生低电平,使4个显示器轮流显示。位选择信号$A_1A_0$必须与显示的数据$D_3D_2D_1D_0$同步变化。例如要显示4位数5678,当$D_3D_2D_1D_0=1000$,$A_1A_0=00$时,$\overline{Y_0}=0$,(0)位显示器显示8。当$D_3D_2D_1D_0=0111$,$A_1A_0=01$时,$\overline{Y_1}=0$,(1)位显示器显示7,以此类推,对应的显示数据和被点亮的显示器如图(b)所示。4个显示器刷新的周期为T,每个显示器被点亮的时间为$T/4$。虽然4个显示器是轮流被点亮的,但当刷新频率达到一定数值(如40 Hz,即$T=1/40\ \text{Hz}=0.025\ \text{s}$)时,由于人眼的视觉暂留现象,会看到显示器同时在显示5678,不会有闪烁感,动态显示的效果和静态显示是一样的。

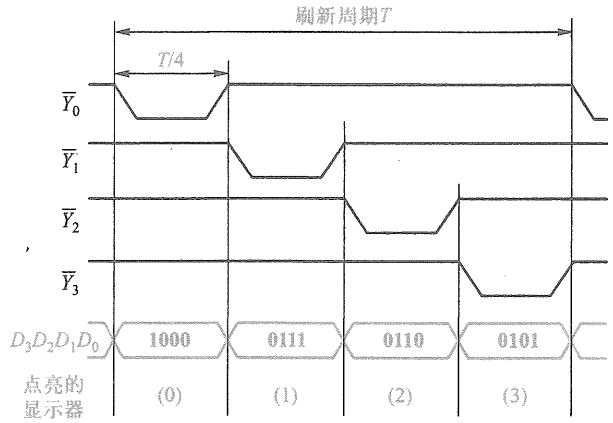
通常人的视觉暂留时间为0.05 s,所以要求刷新周期$T\leqslant 0.05$ s,那么在T和驱动电流不变的情况下,显示器个数越多,一个刷新周期内同一显示器的点亮时间就越短,视觉亮度就会降低。

5. 数据分配器

数据分配是将公共数据线上的数据根据需要送到不同的通道上去,实现数据分配功能的逻辑电路称为**数据分配器**。它的作用相当于多个输出的单刀多掷开关,其示意图如图



(a)



(b)

图 4.4.15 例 4.4.5 的动态显示电路及时序图

(a)动态显示电路 (b)时序图

4.4.16 所示。

数据分配器可以用带使能端的二进制译码器实现。如用 3 线-8 线译码器可以把 1 个数据信号分配到 8 个不同的通道上去。用 3 线-8 线译码器作为数据分配器的逻辑原理图如图 4.4.17 所示。将 $\overline{E}_2$ 接低电平，E_3 作为使能端，$\overline{E}_1$ 作为数据输入，A_2、A_1 和 A_0 作为选择通道地址输入。例如，当 $E_3=1$，$A_2A_1A_0=010$ 时，由式(4.4.5)可得

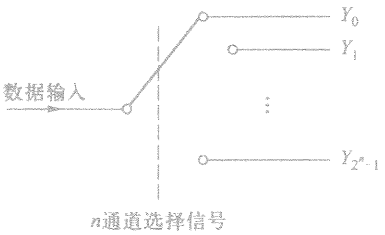


图 4.4.16 数据分配器示意图

$$\overline{Y}_2=\overline{(E_3\cdot\overline{E}_2\cdot\overline{E}_1)\cdot\overline{A}_2\cdot A_1\cdot\overline{A}_0}=\overline{E}_1$$

而其余输出端均为高电平。因此，当地址 $A_2A_1A_0=010$ 时，只有输出端 $\overline{Y}_2$ 得到与输入相同的数据波形。改变 $A_2A_1A_0$ 的取值可以将数据送到不同的输出端。

数据分配器的用途比较多，比如用它将一台 PC 机与多台外部设备连接，将计算机的数

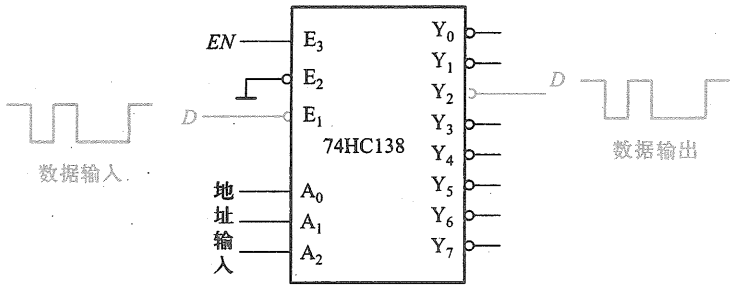


图 4.4.17 用 3 线-8 线译码器作为数据分配器

据分送到外部设备中。它还可以与计数器结合组成脉冲分配器，用它与数据选择器连接组成分时数据传送系统。

例 4.4.6 试用门电路设计一个具有低电平使能控制的 1 线-4 线数据分配器，使能信号无效时，电路所有的输出为高阻态。当通道选择信号将 1 路输入信号连接到其中 1 路输出端时，其他输出端为高阻状态。

解：(1) 明确逻辑功能，列出功能表。

根据题意可知，电路有一个数据输入端 In，一个低有效的使能端 $\overline{E}$，两个通道选择输入端 S_1、S_0，以及 4 个输出端 $Y_3 \sim Y_0$。每个输出端有 3 种状态(0、1、z)，因此电路的输出级必须由 4 个三态门 G_3、G_2、G_1、G_0 组成。三态门的工作状态由其控制信号决定，亦即电路的设计重点是如何产生这些三态门的控制信号。设 C_3、C_2、C_1、C_0 分别为 4 个三态门的控制信号。它们由使能端 $\overline{E}$、通道选择输入端 S_1、S_0 共同作用产生。电路的功能表如表4.4.9所示。

(2) 根据功能表写出逻辑表达式。

根据表 4.4.9 所示的逻辑关系，可以写出每个三态门控制端的表达式。

表 4.4.9 例 4.4.6 的 1 线-4 线数据分配器功能表

输入			三态门控制信号				输出			
$\overline{E}$	S_1	S_0	C_3	C_2	C_1	C_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	1	z	z	z	In
0	0	1	0	0	1	0	z	z	In	z
0	1	0	0	1	0	0	z	In	z	z
0	1	1	1	0	0	0	In	z	z	z
1	x	x	0	0	0	0	z	z	z	z

例如，当 $\overline{E}=0, S_1=S_0=0$ 时，$C_0=1$，门 G_0 工作，使得输出 $Y_0=In$。控制端 C_0 的逻辑表达式为：$C_0=\overline{\overline{E}\cdot\overline{S_1}\cdot\overline{S_0}}$。其他控制端 $C_3=C_2=C_1=0$，对应的三态门输出高阻态。

以此类推，写出其他三态门控制信号的逻辑表达式为

$$C_1=\overline{\overline{E}\cdot\overline{S_1}\cdot S_0},\quad C_2=\overline{\overline{E}\cdot S_1\cdot\overline{S_0}},\quad C_3=\overline{\overline{E}\cdot S_1\cdot S_0}$$

(3) 画逻辑电路。

由以上表达式可画出产生 $C_0 \sim C_3$ 的逻辑电路，由它们分别控制 4 个三态门。三态门的

输入端均接输入信号 In，完整的逻辑电路如图 4.4.18 所示。注意，$\overline{E}$ 表示低电平有效，看做完整的变量符号。

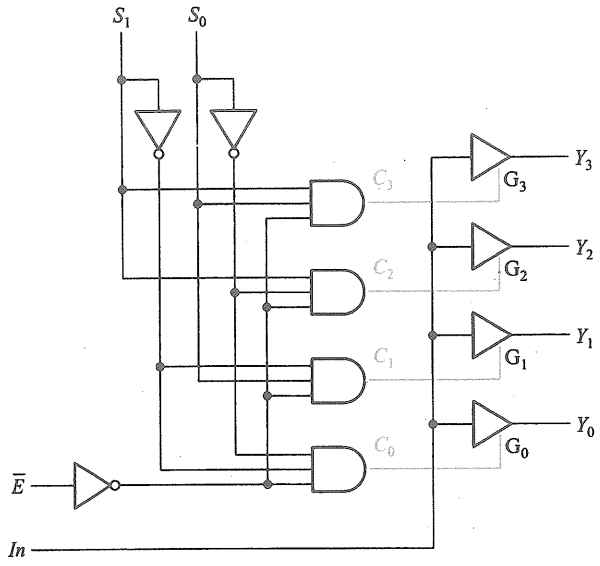


图 4.4.18 例 4.4.6 的逻辑电路

复习思考题

4.4.2.1 什么是译码？什么是二进制译码器或唯一地址译码器？

4.4.2.2 发光二极管构成的七段显示器有哪两种，七段显示译码器有哪两类？

4.4.2.3 3 线-8 线译码器作为数据分配器时，对于 $\overline{E}_1$、$\overline{E}_2$、E_3 的设置办法，除了图4.4.17所示外，你还有别的什么办法？

4.4.3 数据选择器

1. 数据选择器的定义与功能

数据选择是指经过选择，把多路数据中的某一路数据传送到公共数据线上，实现数据选择功能的逻辑电路称为数据选择器。它的作用相当于多个输入的单刀多掷开关，其示意图如图 4.4.19 所示。

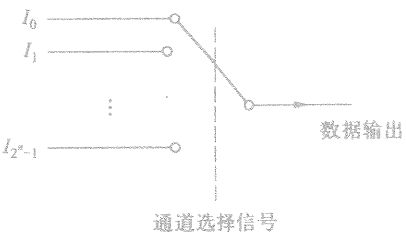


图 4.4.19 数据选择器示意图

在 3.2.4 节已经介绍了由传输门构成的 2 选 1 数据选择器。这里介绍将 2 选 1 数据选择器作为基本模块，构成更大规模的电路。图 4.4.20 所示为与门和或门构成的 2 选 1 数据选择器电路及逻辑符号，该符号常在大规模逻辑电路中使用。选择输入端（又称为地址输入端）S 决定输出 Y 等于 D_0 还是 D_1，真值表如表 4.4.10 所示。输出的逻辑函数式为

$$Y=\overline{S}D_0+SD_1$$

表 4.4.10 2 选 1 数据选择器真值表

选择输入 S	输出 Y
0	D_0
1	D_1

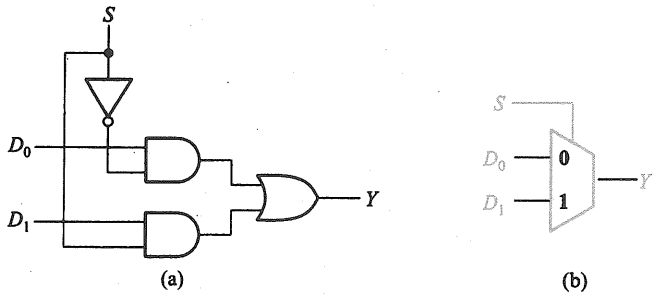


图 4.4.20 2 选 1 数据选择器

(a) 逻辑电路 (b) 逻辑符号

可以用 3 个 2 选 1 数据选择器构成 4 选 1 数据选择器，如图 4.4.21(a)所示。它是两级电路，第 1 级两个数据选择器分别实现 $Y_0=\overline{S}_0D_0+S_0D_1$ 和 $Y_1=\overline{S}_0D_2+S_0D_3$。第 2 级实现 $Y=\overline{S}_1Y_0+S_1Y_1$，将 Y_0 和 Y_1 代入有

$$\begin{aligned}Y&=\overline{S}_1Y_0+S_1Y_1\\&=\overline{S}_1(\overline{S}_0D_0+S_0D_1)+S_1(\overline{S}_0D_2+S_0D_3)\\&=\overline{S}_1\overline{S}_0D_0+\overline{S}_1S_0D_1+S_1\overline{S}_0D_2+S_1S_0D_3\end{aligned}$$

图 4.4.21(b)所示为其逻辑符号，其真值表如表 4.4.11 所示。

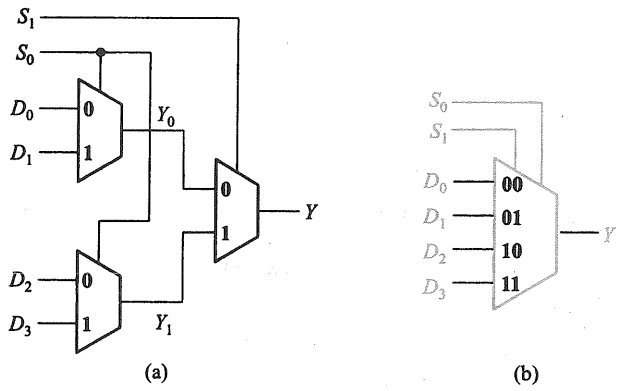


图 4.4.21 4 选 1 数据选择器逻辑图

(a) 由 2 选 1 数据选择器构成的 4 选 1 数据选择器 (b) 逻辑符号

表 4.4.11 4 选 1 数据选择器真值表

选择输入		输出
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

同样原理，可以构成更多输入通道的数据选择器。被选数据源越多，所需选择输入端的位数也越多，若选择输入端为 n，可选输入通道数为 2^n。

译码器与数据选择器都具有选择功能，因此可以用二进制译码器构成数据选择器。图 4.4.22 所示是由译码器、与门和或门构成的 4 选 1 数据选择器。译码器的输入作为通道选择端，其输出分别控制各个与门，以便选择 D_0、D_1、D_2 或 D_3 中的一个数据传送到输出端。

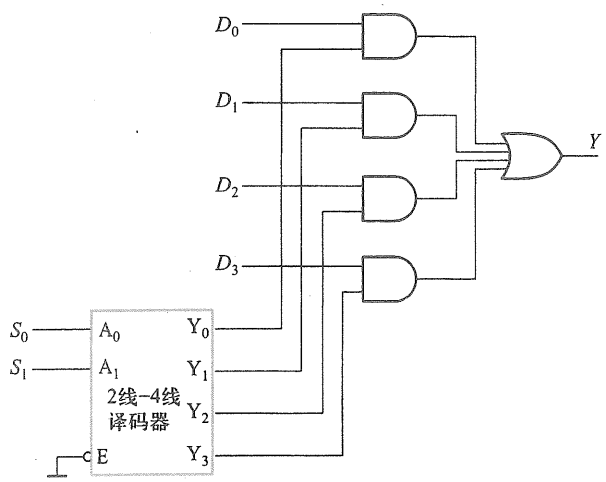


图 4.4.22 由译码器构成的 4 选 1 数据选择器

数据选择器的应用非常广泛，并且是构成 FPGA 器件内部查找表（LUT）的基本单元。在数字系统中，用于移位运算的移位器由数据选择器构成。常用的数据选择器模块有 2 选 1 数据选择器、4 选 1 数据选择器、8 选 1 数据选择器、16 选 1 数据选择器等。还有一些数据选择器具有三态输出功能，除了正常的 0 或 1 输出之外，当使能输入端为无效信号时，输出为高阻状态。利用这一特点，可以将多个数据选择器的输出端线与连在一起，共用一根数据传输线，而不会出现相互干扰问题。

2. 用数据选择器实现逻辑函数

（1）简单逻辑函数的实现

以 4 选 1 数据选择器为例，它有 2 个选择输入 S_1、S_0 和 4 个数据输入 D_0、D_1、D_2 和 D_3，其输出与输入的关系式可以写成

$$Y=\overline{S}_1\ \overline{S}_0D_0+\overline{S}_1S_0D_1+S_1\ \overline{S}_0D_2+S_1S_0D_3=\sum_{i=0}^{3}m_iD_i$$

式中，m_i 是选择输入端 S_1、S_0 构成的最小项。数据输入作为控制信号，当 $D_i=1$ 时，其对应的最小项 m_i 在表达式中出现，当 $D_i=0$ 时，对应的最小项就不出现。利用这一点将函数变换成最小项表达式，函数的变量接选择输入端，就可以实现组合逻辑函数。

例 4.4.7 试用数据选择器实现下列逻辑函数：

① 用 4 选 1 数据选择器实现 $L_0=\overline{A}B+A\ \overline{B}$；

② 用 4 选 1 数据选择器实现 $L_1=A\ \overline{C}+BC$；

③ 用 2 选 1 数据选择器和必要的逻辑门实现 $L_1=A\ \overline{C}+BC$。

解：① 把所给的函数式变换成最小项表达式

$$L_0=\overline{A}B\cdot 1+A\overline{B}\cdot 1=m_1D_1+m_2D_2$$

变量 A、B 分别接 4 选 1 数据选择器的两个选择端 S_1 和 S_0。L_0 中出现的最小项 m_1、m_2 对应的数据输入端 D_1、D_2 都应该等于 1，而没有出现的最小项 m_0、m_3 对应的数据输入端 D_0、D_3 都应该等于 0。由此可画出逻辑图如图 4.4.23(a)所示。

② 根据 L_1 的函数式列出真值表如表 4.4.12 所示。将变量 A、B 接入 4 选 1 数据选择器选择输入端 S_1 和 S_0。将变量 C 分配在数据输入端。从表中可以看出输出 L_1 与变量 C 的关系。当 $AB=00$ 时选通 D_0，而此时 $L_1=0$，所以数据端 D_0 接 0；当 $AB=01$ 时选通 D_1，由真值表得此时 $L_1=C$，即 D_1 应接 C；当 AB 为 10 和 11 时，D_2 和 D_3 分别接 $\overline{C}$ 和 1。因此得到逻辑电路如图 4.4.23(b)所示。

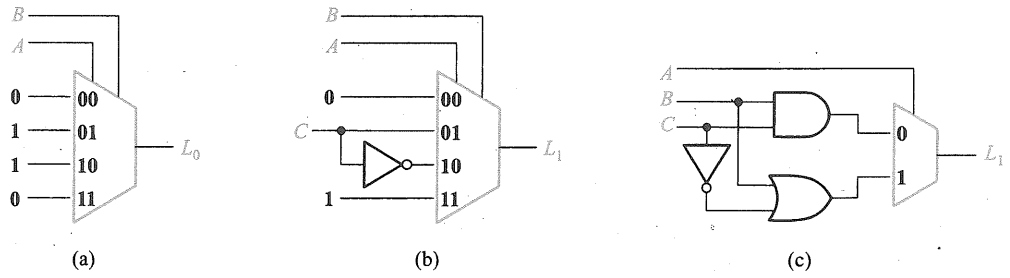


图 4.4.23 用数据选择器实现逻辑函数

(a) 用 4 选 1 数据选择器实现 L_0 (b) 用 4 选 1 数据选择器实现 L_1 (c) 用 2 选 1 数据选择器实现 L_1

③ L_1 的真值表如表 4.4.13 所示。2 选 1 数据选择器只有一个选择输入端，将变量 A 接选择输入端。根据表中 L_1 和 B、C 的关系，当 $A=0$ 时，可以求出 $L_1=BC$，即数据端 $D_0=BC$。同理求出 $D_1=B+\overline{C}$。将变量 B、C 用逻辑门组合后接入数据端，如图 4.4.23(c)所示，这样可以实现变量数更多的逻辑函数，而且这种方法实现的电路比②中使用的逻辑门总数少(包含选择器里面的门)。

由此可知，用一个具有 n 位选择输入的数据选择器，实现变量数不大于 $n+1$ 的逻辑函数时，每一个数据输入端可以接 0、1、单变量或它的非。当变量数大于 $n+1$ 时，可以用多个数据选择器扩展使用，也可以附加其他门电路后连接到数据输入端。

表 4.4.12 例 4.4.7②的真值表

输入			输出	
A	B	C	L_1	
0	0	0	0	$L_1=0$
0	0	1	0	
0	1	0	0	$L_1=C$
0	1	1	1	
1	0	0	1	$L_1=C$
1	0	1	0	
1	1	0	1	$L_1=1$
1	1	1	1	

表 4.4.13 例 4.4.7③的真值表

输入			输出	
A	B	C	L_1	
0	0	0	0	$L_1=BC$
0	0	1	0	
0	1	0	0	
0	1	1	1	
1	0	0	1	$L_1=B+\overline{C}$
1	0	1	0	
1	1	0	1	
1	1	1	1	

(2) 采用香农展开定理分解逻辑函数后实现

前面例题都是通过真值表得到数据选择器数据输入端的表达式，当变量数比较多时，就不容易得出结果，这时可以采用香农展开定理对逻辑函数进行分解，得出数据端的逻辑表达式。

2.1.2 节介绍了香农展开定理，即任何一个逻辑函数 $f(X_1,X_2,\cdots,X_n)$ 可以分解为

$$f(X_1,X_2,\cdots,X_n)=\overline{X_1}\cdot f(0,X_2,\cdots,X_n)+X_1\cdot f(1,X_2,\cdots,X_n)$$

例如，将上述例题 4.4.7 中逻辑函数 $L_1=A\overline{C}+BC$ 用香农展开定理，对 A 展开得到

$$\begin{aligned}L_1&=\overline{A}(0\cdot\overline{C}+BC)+A(1\cdot\overline{C}+BC)\\&=\overline{A}BC+A(\overline{C}+BC)\\&=\overline{A}BC+A(B+\overline{C})\end{aligned}$$

与例题 4.4.7 中推导的表达式完全相同。

如果对 B 展开，则得到

$$\begin{aligned} L_1 &= \overline{B}(A\overline{C}+\mathbf{0}\cdot C)+B(A\overline{C}+\mathbf{1}\cdot C) \\ &= \overline{B}(A\overline{C})+B(A+C) \end{aligned}$$

如果对 C 展开，则得到

$$\begin{aligned} L_1 &= \overline{C}(A\cdot\overline{\mathbf{0}}+B\cdot\mathbf{0})+C(A\cdot\overline{\mathbf{1}}+B\cdot\mathbf{1}) \\ &= \overline{C}(A)+C(B) \end{aligned}$$

实现对 A 和 B 展开结果所用成本相同，实现对 C 展开结果的成本更低。实际应该选择成本最低的展开式实现。

香农展开定理可以推广到多个变量展开式，例如将函数 $f(X_1,X_2,\cdots,X_n)$ 对 X_1 和 X_2 展开可得

$$\begin{aligned} f(X_1,X_2,\cdots,X_n) = {} & \overline{X}_1\overline{X}_2\cdot f(\mathbf{0},\mathbf{0},X_3,\cdots,X_n)+\overline{X}_1X_2\cdot f(\mathbf{0},\mathbf{1},X_3,\cdots,X_n)+ \\ & X_1\overline{X}_2\cdot f(\mathbf{1},\mathbf{0},X_3,\cdots,X_n)+X_1X_2\cdot f(\mathbf{1},\mathbf{1},X_3,\cdots,X_n) \end{aligned}$$

例 4.4.8 试用香农展开定理分解逻辑函数 $L=AB+BC+AC$：

① 用 2 选 1 数据选择器和必要的逻辑门实现；

② 只用 2 选 1 数据选择器实现。

解：① 将逻辑函数式用香农展开定理对 A 展开得

$$\begin{aligned} L &= \overline{A}(\mathbf{0}\cdot B+\mathbf{0}\cdot C+BC)+A(\mathbf{1}\cdot B+\mathbf{1}\cdot C+BC) \\ &= \overline{A}(BC)+A(B+C) \end{aligned}$$

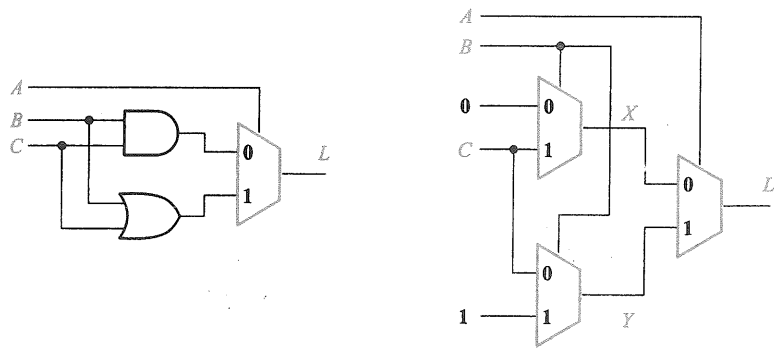
将变量 A 接数据选择端，数据端 $D_0=BC$，$D_1=B+C$。将变量 B、C 用逻辑门组合后接入数据端，如图 4.4.24(a)所示。

② 只用 2 选 1 数据选择器实现时，在上述 L 表达式中设 $X=BC$，$Y=B+C$，则 $L=\overline{A}X+AY$。对 X、Y 表达式继续用香农展开定理分解，则

$$X=\overline{B}(\mathbf{0})+B(C)$$

$$Y=\overline{B}(C)+B(\mathbf{1})$$

根据上述 X、Y 及 L 的表达式得出对应的电路如图 4.4.24(b)所示。



(a) (b)

图 4.4.24 用香农展开定理分解逻辑函数后实现

3. 用数据选择器构成查找表 LUT

构成 FPGA 基本单元的逻辑块主要是查找表 LUT。LUT 实质是一个小规模的存储器，以真值表的形式实现给定的逻辑函数。3 输入 LUT 的结构及逻辑符号如图 4.4.25 所示，由 7 个 2 选 1 数据选择器组成 3 级电路，第 1 级的数据输入端接存储单元（存储单元在第 7 章介绍），每个存储单元可以存放一个逻辑值 0 或 1，由编程决定存 0 还是 1。用数据选择器扩展方法可以得到 4 输入的 LUT，这时输入端需要 16 个存储单元。当输入端增加时，LUT 使用的数据选择器呈几何增长。FPGA 中 LUT 的输入端数一般为 4 个，有的达到 6 个。

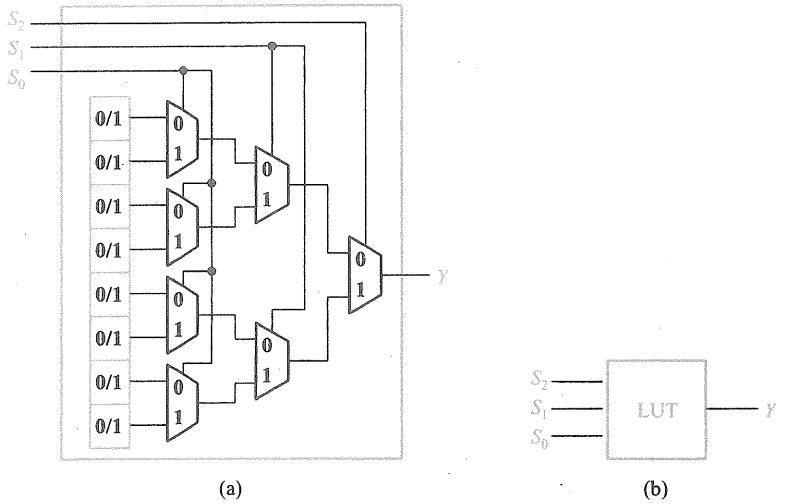


图 4.4.25 3 输入 LUT 结构图及符号

（a）结构图 （b）符号

图 4.4.26 所示为 3 输入 LUT 实现例 4.4.7 中的函数 $L_1=A\overline{C}+BC$。根据表 4.4.12 所示真值表中 L_1 的取值，对存储单元编程，即将 L_1 的值写入 LUT 对应的存储单元。输入变量 A、B、C 接入选择端，根据它们的取值从 8 个存储单元中选择一个作为输出。

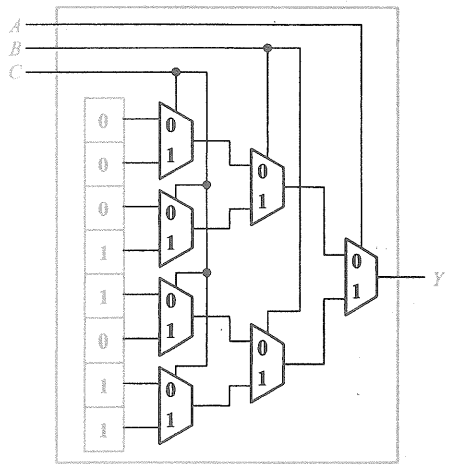


图 4.4.26 用 LUT 实现给定的函数 L_1

用 FPGA 实现大规模逻辑电路时，变量数比较多，首先要对复杂逻辑函数进行函数分解变换，以满足单个 LUT 对变量数的要求，然后通过多个 LUT 扩展实现复杂的逻辑函数。

4. 用数据选择器构成移位器

在计算机等数字系统中除了算术运算和逻辑运算，通常还需要移位运算。在第 6 章介绍的移位寄存器的移位是在时钟脉冲控制下进行的，一个时钟脉冲只能移动一位。但在运算电路中常需要高速移位，这时便可采用组合逻辑电路构成的移位器。

（1）基本移位器

由数据选择器构成的 4 位基本移位器如图 4.4.27 所示。选择变量 S_1S_0 控制移位器的操作类型。当 $S_1S_0=\mathbf{00}$ 时，输入数据 D_3、D_2、D_1、D_0 通过各数据选择器直接输出，$Y_3Y_2Y_1Y_0=D_3D_2D_1D_0$。当 $S_1S_0=\mathbf{01}$ 时，$Y_3Y_2Y_1Y_0=I_RD_3D_2D_1$，将数据右移一位输出，$I_R$ 的值移入最左边。当 $S_1S_0=\mathbf{10}$ 时，$Y_3Y_2Y_1Y_0=D_2D_1D_0I_L$。将数据左移一位输出，$I_L$ 的值移入最右边。其中 $S_1S_0=\mathbf{11}$ 没有使用。4 位基本移位器的功能表如表 4.4.14 所示。

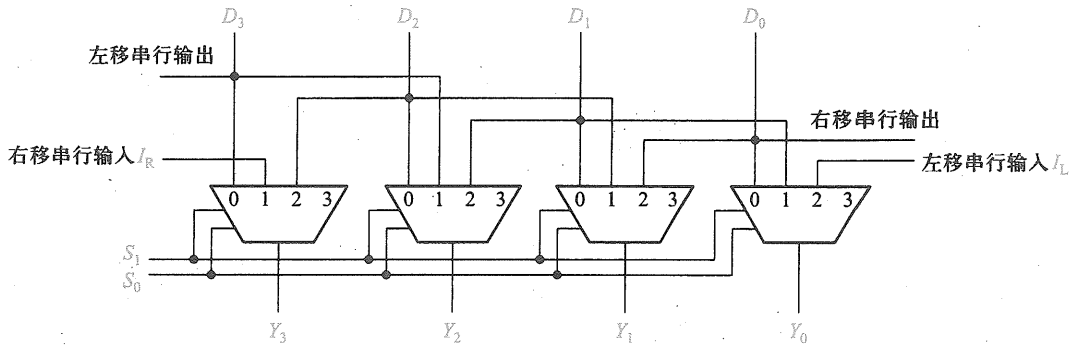


图 4.4.27 4 位基本移位器

表 4.4.14 4 位基本移位器的功能表

选择		输出				功能
S_1	S_0	Y_3	Y_2	Y_1	Y_0	
0	**0**	D_3	D_2	D_1	D_0	直接输出
0	**1**	I_R	D_3	D_2	D_1	右移一位
1	**0**	D_2	D_1	D_0	I_L	左移一位

（2）桶形移位器

能将数据移动多位，移出位被放置在右边（左移时）或左边（右移时），且能进行循环移位，具备这种功能的电路称为桶形移位器（barrel shifter）。

一个 4 位桶形移位器如图 4.4.28 所示。它由 4 个具有相同选择信号 S_1S_0 的 4 选 1 数据选择器构成。当 $S_1S_0=\mathbf{00}$ 时，不移位，输入数据直接输出 $Y_3Y_2Y_1Y_0=D_3D_2D_1D_0$。当 $S_1S_0=\mathbf{01}$ 时，$Y_3=D_0$，$Y_2=D_3$，$Y_1=D_2$，$Y_0=D_1$，输入数据循环右移一位。当 $S_1S_0=\mathbf{10}$ 时，$Y_3Y_2Y_1Y_0=D_1D_0D_3D_2$，输入数据循环右移二位。当 $S_1S_0=\mathbf{11}$ 时，$Y_3Y_2Y_1Y_0=D_2D_1D_0D_3$，输入数据循环右

移三位。其功能表如表 4.4.15 所示。如果要构成 n 位桶形移位器，就需要 n 个 n 选 1 数据选择器，最多可移动 $n-1$ 位。

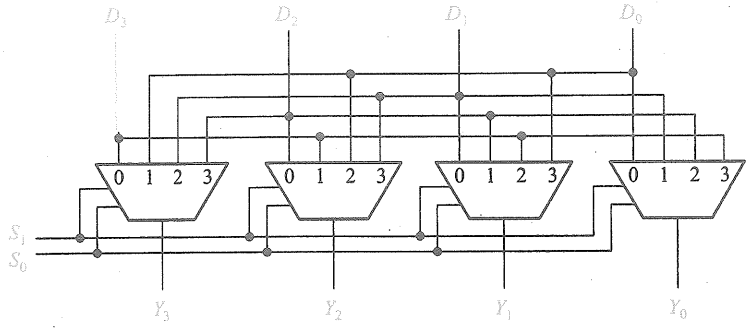


图 4.4.28 4 位桶形移位器

注意，这种桶形移位器也可看作循环左移的移位器。例如，循环右移三位与循环左移一位的结果相同（见表 4.4.15 功能栏括号中的描述）。

表 4.4.15 4 位桶形移位器的功能表

选择		输出				功能
S_1	S_0	Y_3	Y_2	Y_1	Y_0	
0	0	D_3	D_2	D_1	D_0	直接输出
0	1	D_0	D_3	D_2	D_1	循环右移一位（循环左移三位）
1	0	D_1	D_0	D_3	D_2	循环右移二位（循环左移二位）
1	1	D_2	D_1	D_0	D_3	循环右移三位（循环左移一位）

5. 在信号传输系统的应用

数据选择器又称为多路复用器，数据分配器又称为多路分接器（或解复用器）。由多路复用器、多路分配器和总线连接构成的信号传输系统示意图如图 4.4.29 所示。该系统将多个数据信号中的一路信号连接到总线，经过远距离传送后，由多路分配器再将总线上的数据分配到被选中的多个目的地之一。这种信息传输的基本原理在通信系统、计算机网络系统以及计算机内部各功能部件之间的信息传送等都有广泛的应用。

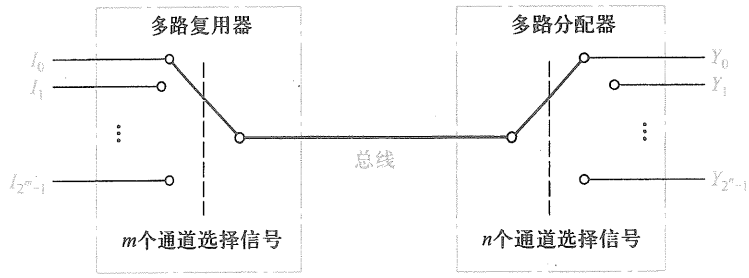


图 4.4.29 多路复用器、多路分配器、总线构成的信号传输系统示意图

复习思考题

4.4.3.1 试用十六进制数的方式写出 16 选 1 的数据选择器的各地址码。

4.4.3.2 构成 16 选 1 数据选择器需要多少个 2 选 1 数据选择器，电路有几级？试画出逻辑电路。

4.4.3.3 试说明 FPGA 中查找表 LUT 实现逻辑函数的原理。

4.4.4 数值比较器

1. 数值比较器的定义及功能

在数字系统中，特别是在计算机中常需要对两个数的大小进行比较。数值比较器就是对两个二进制数 A、B 进行比较的逻辑电路，比较结果有 $A>B$、$A<B$ 以及 $A=B$ 三种情况。

（1）1 位数值比较器

1 位数值比较器是多位比较器的基础。当 A 和 B 都是 1 位二进制数时，它们只能取 0 或 1 两种值，由此可写出 1 位数值比较器的真值表，如表 4.4.16 所示。

表 4.4.16 一位数值比较器真值表

输入		输出		
A	B	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

由真值表得到如下逻辑表达式：

$$\begin{cases} F_{A>B}=A\overline{B} \\ F_{A<B}=\overline{A}B \\ F_{A=B}=\overline{A}\,\overline{B}+AB \end{cases} \tag{4.4.7}$$

由以上逻辑表达式可画出图 4.4.30 所示的逻辑电路。

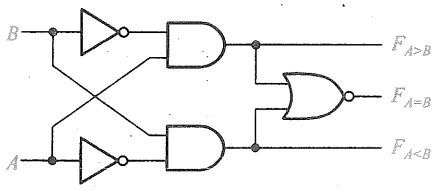


图 4.4.30 1 位数值比较器逻辑图

（2）两位数值比较器

两位二进制数分别为 A_1A_0 和 B_1B_0，用 $F_{A>B}$、$F_{A<B}$ 和 $F_{A=B}$ 表示比较结果。当高位（A_1、B_1）不相等时，无须比较低位（A_0、B_0），高位比较的结果就是两个数的比较结果。当高位相等时，两数的比较结果由低位比较的结果决定。利用 1 位数值的比较结果，可以列出简化的功能表，如表 4.4.17 所示。

表 4.4.17 两位数值比较器功能表

输入		输出		
A_1 B_1	A_0 B_0	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
$A_1>B_1$	×	1	0	0
$A_1<B_1$	×	0	1	0
$A_1=B_1$	$A_0>B_0$	1	0	0
$A_1=B_1$	$A_0<B_0$	0	1	0
$A_1=B_1$	$A_0=B_0$	0	0	1

由表 4.4.17 可以写出如下逻辑表达式：

$$\begin{cases} F_{A>B}=F_{A_1>B_1}+F_{A_1=B_1}\cdot F_{A_0>B_0} \\ F_{A<B}=F_{A_1<B_1}+F_{A_1=B_1}\cdot F_{A_0<B_0} \\ F_{A=B}=F_{A_1=B_1}\cdot F_{A_0=B_0} \end{cases} \tag{4.4.8}$$

根据上式画出逻辑图，如图 4.4.31 所示。电路利用了 1 位数值比较器的输出作为中间结果。它所依据的原理是，如果两位数 A_1A_0 和 B_1B_0 的高位不相等，则高位比较结果就是两数比较结果，与低位无关。这时，高位输出 $F_{A_1=B_1}=0$，使与门 G_1、G_2、G_3 均封锁，而或门都打开，低位比较结果不能影响或门，高位比较结果则从或门直接输出。如果高位相等，即 $F_{A_1=B_1}=1$，使与门 G_1、G_2、G_3 均打开，同时由 $F_{A_1>B_1}=0$ 和 $F_{A_1<B_1}=0$ 作用，或门也打开，低位的比较结果直接送达输出端，即低位的比较结果决定两数的大、小或者相等。

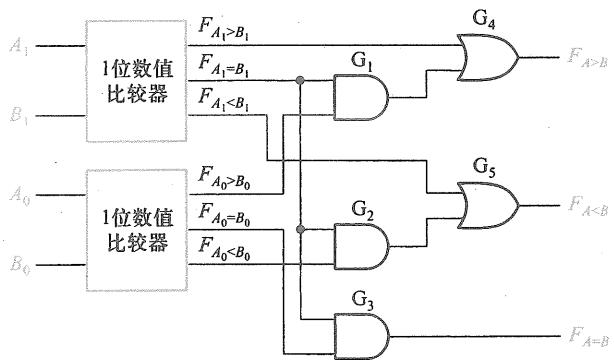


图 4.4.31 两位数值比较器逻辑图

用以上方法可以构成更多位的数值比较器。

2. 4 位数值比较器模块及扩展

常用的比较器模块有 4 位数值比较器、8 位数值比较器等。

（1）4 位数值比较器

4 位数值比较器是对两个 4 位二进制数 $A_3A_2A_1A_0$ 与 $B_3B_2B_1B_0$ 进行比较，比较原理与 2 位比较器相同。先比较最高位 A_3 和 B_3，如果它们不相等，则该位的比较结果作为两数的比较结果。若最高位 $A_3=B_3$，则再比较次高位 A_2 和 B_2，以此类推。显然，如果两数相等，那么，必须将比较进行到最低位才能得到结果。可以得出：

$$\begin{cases} F_{A>B}=F_{A_3>B_3}+F_{A_3=B_3}F_{A_2>B_2}+F_{A_3=B_3}F_{A_2=B_2}F_{A_1>B_1}+F_{A_3=B_3}F_{A_2=B_2}F_{A_1=B_1}F_{A_0>B_0} \\ F_{A<B}=F_{A_3<B_3}+F_{A_3=B_3}F_{A_2<B_2}+F_{A_3=B_3}F_{A_2=B_2}F_{A_1<B_1}+F_{A_3=B_3}F_{A_2=B_2}F_{A_1=B_1}F_{A_0<B_0} \\ F_{A=B}=F_{A_3=B_3}F_{A_2=B_2}F_{A_1=B_1}F_{A_0=B_0} \end{cases} \tag{4.4.9}$$

74HC85 是 4 位 CMOS 集成数值比较器，除了有 8 个数据输入端、3 个输出端外，还有 3 个扩展输入端 $I_{A>B}$、$I_{A<B}$ 和 $I_{A=B}$，可以与低位片的输出连接，以便组成位数更多的数值比较器。若相比较的 4 位数不等时，直接输出比较结果；如果相等时，输出由扩展端的输入决定。若仅对 4 位数比较时，应对 $I_{A>B}$、$I_{A<B}$、$I_{A=B}$ 进行适当处理，即 $I_{A>B}=I_{A<B}=0$，$I_{A=B}=1$。

（2）数值比较器的扩展

数值比较器的扩展有串联和并联两种方式。图 4.4.32 表示 4 个 4 位数值比较器串联成为一个 13 位的数值比较器，低位比较器的输出 $F_{A>B}$ 和 $F_{A<B}$ 分别接高位比较器的 A_0 和 B_0。如果最高位比较器的前 3 位不相等，则直接得到结果；若高位片的前 3 位相等，则需要看相邻低位片的比较结果。以此类推，直到比较到最低位比较器才能得到结果。显然，串联比较速度较慢。

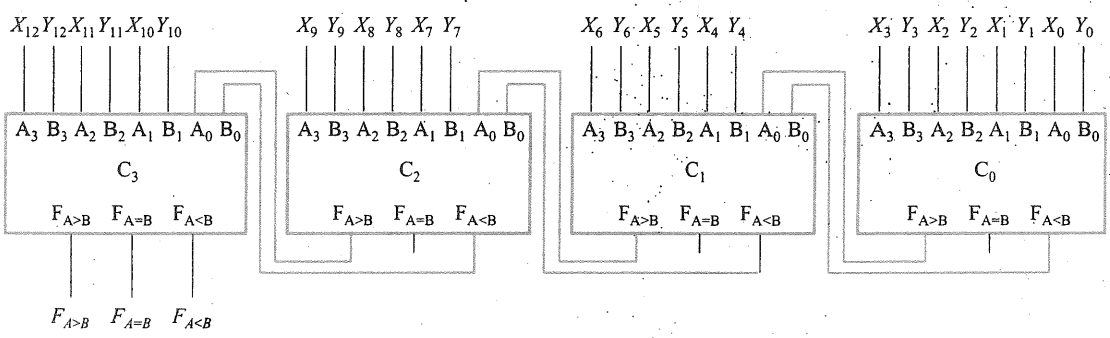


图 4.4.32 串联方式扩展数值比较器的位数

当位数较多且要满足一定的速度要求时，可以采取并联方式。图 4.4.33 表示 16 位并联数值比较器的原理示意图。由图可以看出这里采用两级比较方法，将 16 位按高低位次序分成四组，每组 4 位，各组的比较是并行进行的。每组的比较结果再经第二级 4 位比较器比较后得出结果。显然，从数据输入到稳定输出只需 2 倍的 4 位比较器延迟时间，若用串联方式，则 13 位的数值比较器从输入到稳定输出需要约四倍的 4 位比较器的延迟时间。

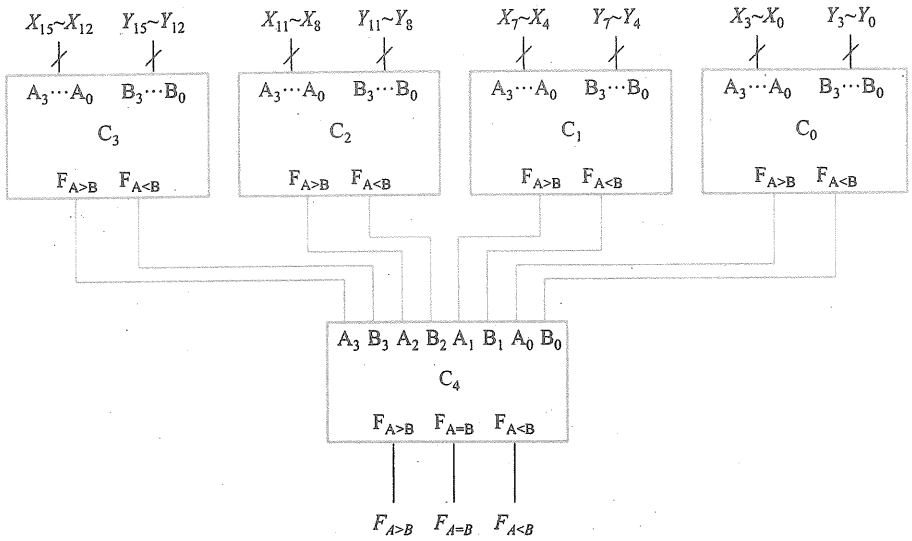


图 4.4.33 并联方式扩展数值比较器的位数

复习思考题

4.4.4.1 采用图 4.4.32 所示串联方式比较两个 8 位数需要几片 4 位数值比较器，如何连接，多余输入端如何处理？

4.4.4.2 采用并联方式比较两个 32 位数需要几片 4 位数值比较器，如何连接？

4.4.5 算术运算电路

算术运算是数字系统的基本功能，算术运算电路更是计算机中不可缺少的组成单元。在第 1 章介绍了二进制数的算术运算，下面介绍实现加法运算和减法运算的逻辑电路。

1. 半加器和全加器

(1) 半加器

半加器和全加器是算术运算电路中的基本单元，它们是完成 1 位二进制数相加的一种组合逻辑电路。

只考虑了两个加数本身，而没有考虑低位进位的加法运算，称为半加。实现半加运算的逻辑电路称为半加器。两个 1 位二进制的半加运算真值表如表 4.4.18 所示，其中 A、B 是两个加数，S 表示和数，C 表示进位数。由真值表可得逻辑表达式

表 4.4.18 半加器真值表

输入		输出	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$\begin{cases} S=\overline{A}B+A\overline{B} \\ C=AB \end{cases} \tag{4.4.10}$$

由此可画出由**异或门**和**与门**组成的半加器，如图 4.4.34(a)所示，图 4.4.34(b)是半加器的逻辑符号。

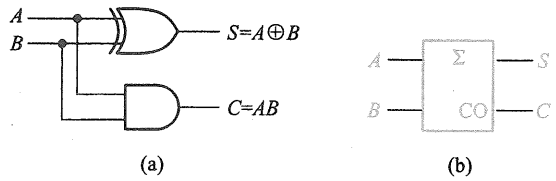


图 4.4.34 半加器

(a) 逻辑图 (b) 半加器符号

(2) 全加器

完成被加数、加数和来自低位的进位信号相加，并根据求和结果给出该位进位信号的运算称为全加。实现全加运算的电路称为全加器。

根据全加器的功能，可列出它的真值表，如表 4.4.19 所示。其中 A 和 B 分别是被加数及加数，C_i 为低位进位数，S 为本位和数(称为全加和)以及 C_o为向高位的进位数。为了求出 S 和 C_o的逻辑表达式，首先分别画出 S 和 C_o的卡诺图，如图 4.4.35 所示，其中 C_o的包围圈是为了便于利用 $A\oplus B$ 结果，得出下列表达式：

$$\begin{cases} S=\overline{A}\,\overline{B}C_i+\overline{A}B\overline{C}_i+A\overline{B}\,\overline{C}_i+ABC_i \\ \quad =A\oplus B\oplus C_i \\ C_o=AB+A\overline{B}C_i+\overline{A}BC_i \\ \quad =AB+(A\oplus B)C_i \end{cases} \tag{4.4.11}$$

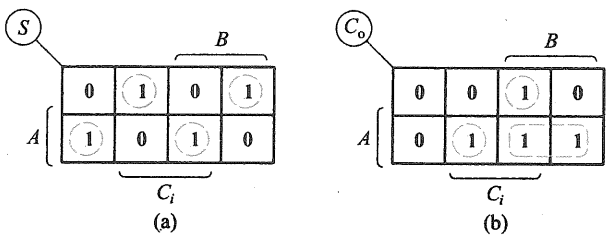


图 4.4.35 全加器的 S 和 C_o卡诺图

(a) S 的卡诺图 (b) C_o的卡诺图

由式(4.4.11)可以画出 1 位全加器的逻辑图，如图 4.4.36(a)所示，它是由两个半加器和一个**或门**构成的，图 4.4.36(b)是它的逻辑符号。

2. 多位数加法器

(1) 串行进位加法器

若有多位数相加，则可采用并行相加串行进位的方式来完成。例如，有两个 4 位二进制数 $A_3A_2A_1A_0$ 和 $B_3B_2B_1B_0$ 相加，可以采用 4 个全加器构成 4 位数加法器，其原理图如图 4.4.37

表 4.4.19 全加器真值表

输入			输出	
A	B	C_i	S	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

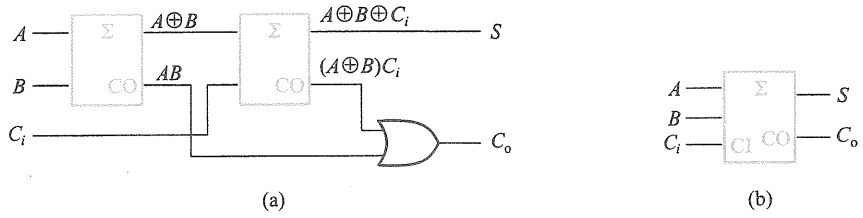


图 4.4.36 全加器
(a) 逻辑图 (b) 逻辑符号

所示。将低位的进位输出信号接到高位的进位输入端,因此,任 1 位的加法运算必须在低 1 位的运算完成之后才能进行,这种进位方式称为串行进位。这种加法器的逻辑电路比较简单,但它的运算速度不高。设门的延迟时间均为 t_{pd},一位全加器的延时是 $3t_{pd}$,4 位串行全加器的延时是 12 t_{pd}。为提高速度,可以采用超前进位等方式。

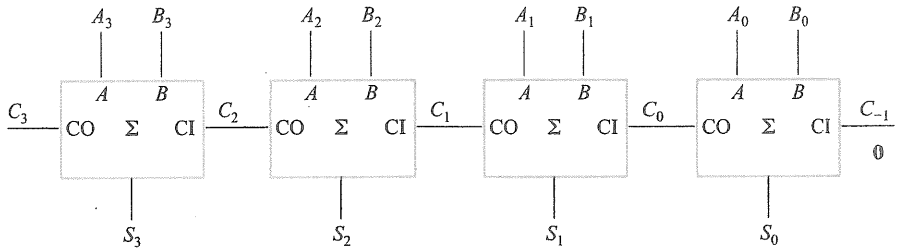


图 4.4.37 4 位串行进位加法器

(2) 超前进位加法器

由于串行进位加法器的速度受到进位信号的限制,人们又设计了一种多位数超前进位加法逻辑电路,使每位的进位只由被加数和加数决定,而与低位的进位无关。下面介绍超前进位的概念。

由式(4.4.11),并考虑多位数值相加时,全加器的和数 S_i 和进位 C_i 的逻辑表达式

教学视频 4.3：超前进位加法器

$$S_i = A_i \oplus B_i \oplus C_{i-1} \tag{4.4.12}$$

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1} \tag{4.4.13}$$

定义两个中间变量 G_i 和 P_i

$$G_i = A_i B_i \tag{4.4.14}$$

$$P_i = A_i \oplus B_i \tag{4.4.15}$$

当 $A_i = B_i = 1$ 时，$G_i = 1$，由式(4.4.13)得 $C_i = 1$，即产生进位，所以 G_i 称为产生变量。若 $P_i = 1$，则 $A_i B_i = 0$，由式(4.4.13)得 $C_i = C_{i-1}$，即 $P_i = 1$ 时，低位的进位能传送到高位的进位输出端，故 P_i 称为传输变量。这两个变量都与进位信号无关。

将式(4.4.14)和式(4.4.15)代入式(4.4.12)和式(4.4.13)，得

$$S_i = P_i \oplus C_{i-1} \tag{4.4.16}$$

$$C_i = G_i + P_i C_{i-1} \tag{4.4.17}$$

由式(4.4.17)得各位进位信号的逻辑表达式如下：

$$C_0 = G_0 + P_0 C_{-1} \tag{4.4.18a}$$

$$C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0 + P_1 P_0 C_{-1} \tag{4.4.18b}$$

$$C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{-1} \tag{4.4.18c}$$

$$C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{-1} \tag{4.4.18d}$$

由式(4.4.18)可知，因为进位信号只与变量 G_i、P_i 和 C_{-1} 有关，而 C_{-1} 是向最低位的进位信号，其值为 0，所以各位的进位信号都只与两个加数有关，它们是可以并行产生的。用与门和或门即可实现式(4.4.18)所表示的超前进位产生电路，如图 4.4.38 所示。根据超前进

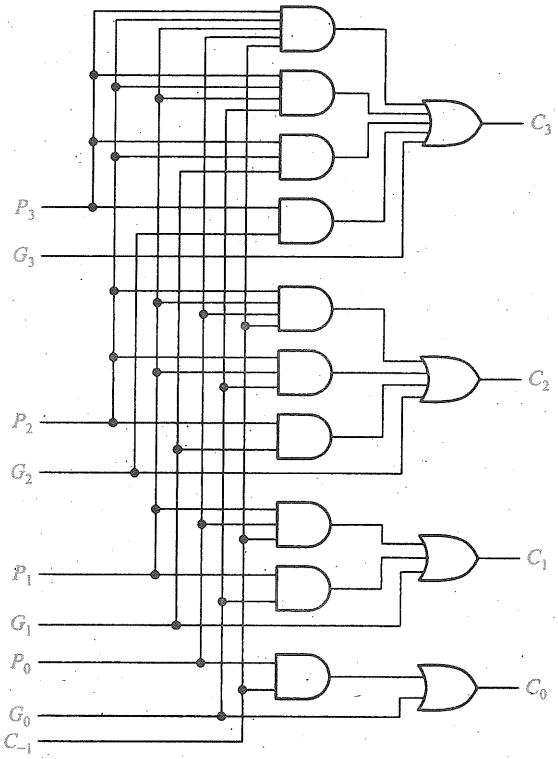


图 4.4.38 超前进位产生电路

位概念构成的 4 位加法器的结构示意图如图 4.4.39 所示。从输入加数、被加数到产生 C_0~C_3，需要 3 级门延迟，再经过一级异或门便得到结果。整个电路的延时为 $4t_{pd}$。当扩展更多位时，电路的延迟时间几乎不再增加。

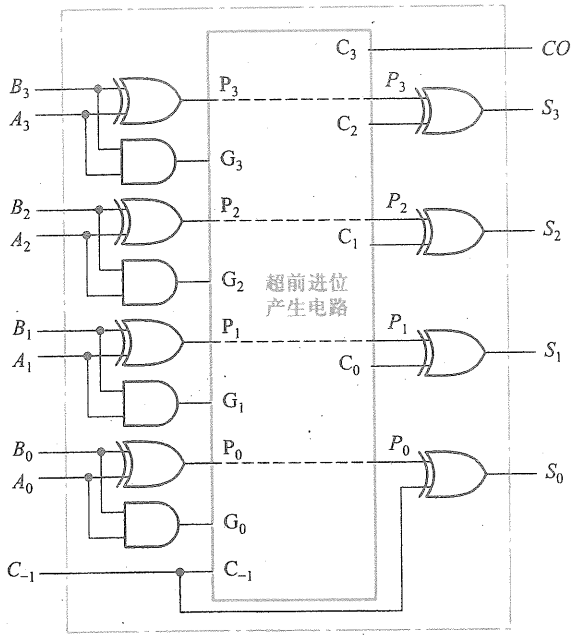


图 4.4.39 4 位超前进位加法器结构示意图

(3) 加法器的层次化扩展

超前进位加法器大大提高了运算速度，但是随着加法器位数的增加，超前进位逻辑电路越来越复杂。为降低复杂度，可以采用分层次设计方法。以 16 位加法电路为例，将其分成 4 个 4 位加法器模块，每个模块内部采用超前进位方式，而模块之间采用串行进位，如图 4.4.40所示。一个模块的延时为 $4t_{pd}$，则 4 个模块串行连接电路的总延时为 16 t_{pd}。当模块数量为 n 时，总延时为 $4nt_{pd}$。

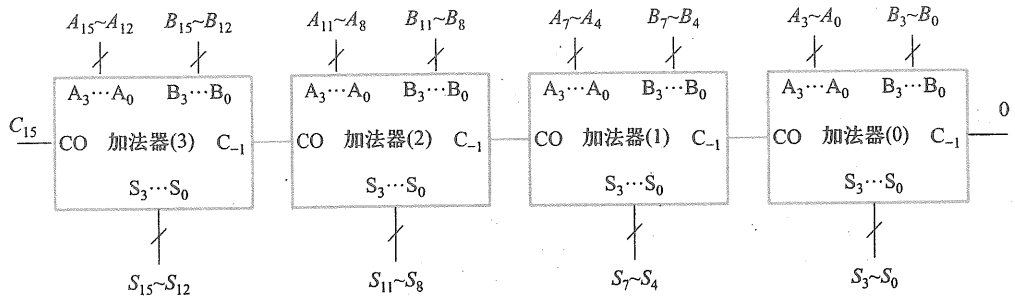


图 4.4.40 层次化串行进位扩展连接方式

为了进一步提高速度，可以采用 2 级超前进位方式。模块内部是第一级超前进位，在 4 个模块之间采用第二级超前进位连接方式，使得模块之间的进位信号也能快速产生，如图

4.4.41所示。

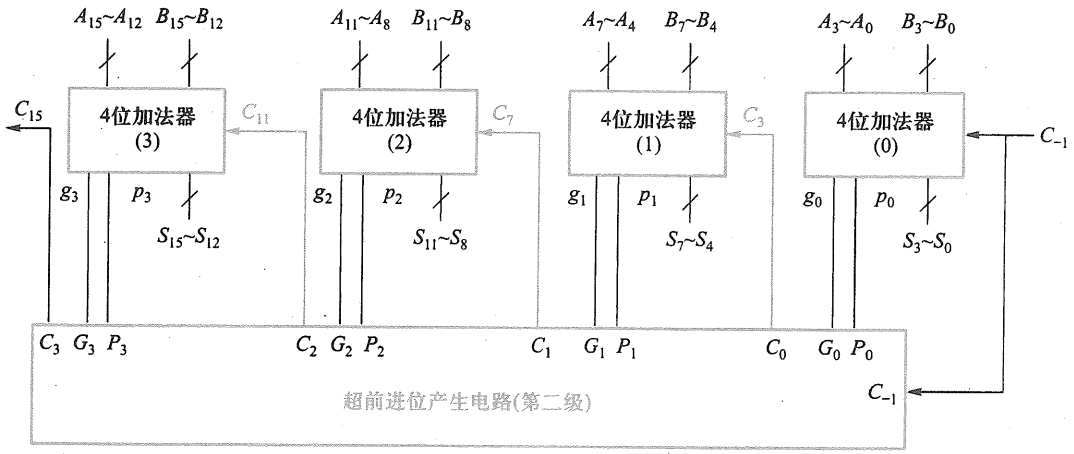


图 4.4.41 层次化超前进位扩展连接方式

令加法器模块(0)~(4)的产生变量和传输变量分别为 g_0、p_0,g_1、p_1,g_2、p_2和 g_3、p_3,并且令式(4.4.18d)中

$$p_0 = P_3P_2P_1P_0$$

$$g_0 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

因此,式(4.4.18d)可以写成

$$C_3 = G_3 + P_3C_2 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{-1}$$

$$= g_0 + p_0C_{-1}$$

同样,加法器模块(1)(2)和(3)的进位信号分别为

$$C_7 = g_1 + p_1C_3 = g_1 + p_1(g_0 + p_0C_{-1})$$

$$C_{11} = g_2 + p_2g_1 + p_2p_1g_0 + p_2p_1p_0C_{-1}$$

$$C_{15} = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0 + p_3p_2p_1p_0C_{-1}$$

将上面进位表达式与式(4.4.18)比较可知,第二级超前进位产生电路就是图 4.4.38 所示电路。

从输入加数、被加数到产生 g_i、p_i 需要三级门延时(即从加数、被加数产生 G_i、P_i 需要一级门延时,从 G_i、P_i 产生 g_i、p_i 需要两级门延时),再经过第二级超前进位的两级门(进位产生电路的**与**门和**或**门)的延时才能产生进位信号 C_3、C_7、C_{11}和 C_{15}。从进位信号 C_3、C_7、C_{11}分别输入片(1)(2)(3),到各模块内部进位信号的建立,需要再加上两级门延时,产生每一个和还需一个**异或**门的延时。因此,完成 16 位数总共需要 8 级门的延时,即 $8t_{pd}$。当位数增加时,延迟时间几乎不再增加。

3. 减法运算

由第 1 章介绍的二进制数算术运算可知,可以将减法运算转换成补码的加法运算。如果前面介绍的加法器既能实现加法运算,又可用于减法运算,就可以简化数字系统结构。

若 n 位二进制的原码为 $N_{原}$,则与它相对应的补码为

$$N_{补} = 2^n - N_{原} \tag{4.4.19}$$

补码与反码的关系式为

$$N_{补}=N_{反}+1 \tag{4.4.20}$$

设两个数 A、B 相减，利用式(4.4.19)和式(4.4.20)可得

$$A-B=A+B_{补}-2^n=A+B_{反}+1-2^n \tag{4.4.21}$$

式(4.4.21)表明，A 减 B 可由 A 加 B 的补码并减 2^n 完成。4 位减法运算电路如图 4.4.42(a)所示，具体原理如下。

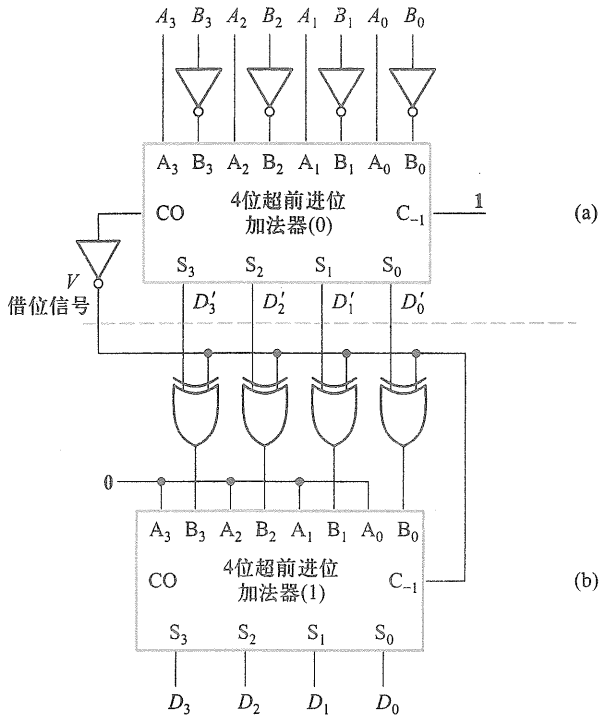


图 4.4.42 输出为原码的 4 位减法运算逻辑图

(a) 4 位减法运算逻辑图 (b) 输出求补逻辑图

由 4 个反相器将 B 的各位求反，并将进位输入端 C_{-1} 接逻辑 1 以实现加 1，由此求得 B 的补码。加法器(0)相加的结果为($A+B_{反}+1$)。

由于 $2^n=(10000)_2$，因此相加结果与 2^n 相减需要由加法器进位输出信号来完成。当进位输出信号为 1 时，它与 2^n 的差为 0；当进位输出信号为 0 时，它与 2^n 的差值为 1，同时还应发出借位信号。因此，只要将进位信号反相即实现了减 2^n 的运算，反相器的输出 V 为 1 时需要借位，故 V 也可作为借位信号。下面分两种情况分析图 4.4.42(a)所示电路实现减法运算的过程。

(1) $A-B\geqslant 0$ 的情况。设 $A=0101$，$B=0001$。

电路求补相加过程如下：

```
    0 1 0 1   (A)
    1 1 1 0   (B反)
+           1   (加 1)
  1 0 1 0 0
    ↓
    0 0 1 0 0   (进位反相)
(借位)——↑
```

直接作减法演算，则有

$$\begin{array}{r} 0\ 1\ 0\ 1 \\ -\ 0\ 0\ 0\ 1 \\ \hline 0\ 1\ 0\ 0 \end{array}$$

比较两种运算结果，它们完全相同。在 $A-B\geqslant 0$ 时，所得的差值就是差的原码，借位信号为 **0**。

(2) $A-B<0$ 的情况。设 $A=0001$，$B=0101$。

电路求补相加过程如下：

```
    0 0 0 1   (A)
    1 0 1 0   (B反)
+           1   (加 1)
  0 1 1 0 0
    1 1 1 0 0   (进位反相)
(借位)——↑
```

直接作减法演算，则有

$$\begin{array}{r} 0\ 0\ 0\ 1 \\ -\ 0\ 1\ 0\ 1 \\ \hline -\ 0\ 1\ 0\ 0 \end{array}$$

(符号)——↑

比较两种运算结果可知，前者正好是后者的绝对值的补码，即 $A-B<0$ 时，电路运算结果使借位信号 V 为 **1**，表示差值为负数，且为补码。而第(1)种情况 $A-B\geqslant 0$ 时，V 为 **0** 表示差为正数，且为原码。若要求差值以原码形式输出，则第(2)种情况还需进行变换，将补码再求补得原码。

求补逻辑电路如图 4.4.42(b)所示，它和图 4.4.42(a) 共同组成输出为原码的完整的 4 位减法运算电路。由图 4.4.42(a)所得的差值输入到异或门的一个输入端，而另一端输入端由借位信号 V 控制。当 $V=1$ 时，$D_3\sim D_0$ 反相，并与 $C_{-1}=1$ 相加，实现求补运算；$V=0$ 时，$D_3\sim D_0$ 不反相，加法器也不实现加 **1** 运算，维持原码。

复习思考题

4.4.5.1 什么是半加器？什么是全加器？

4.4.5.2 超前进位加法器和串行进位加法器的区别是什么？

4.4.5.3 简要说明由加补码完成减法运算的原理。

4.5 组合逻辑的可编程电路实现

早期通用型的小规模门电路和中规模组合逻辑集成器件的逻辑功能是固定的，所包含的逻辑门数量较少。当构造一个大型复杂的数字系统时，过多的器件可能导致功耗高、占用空间大和系统可靠性差等问题。利用可编程逻辑器件（PLD）设计数字系统，可以较好地解决以上问题。PLD 是一种可以由用户定义和设置逻辑功能的器件。该类器件具有逻辑功能实现灵活、集成度高、处理速度快和可靠性高等特点。

我们知道，任何组合逻辑函数都可以表示为与-或表达式，因此通过对与、或阵列编程就可以实现所需的组合逻辑功能。本节主要讨论以可编程的与、或阵列为基础的组合逻辑电路实现原理。

4.5.1 PLD 的电路表示、编程技术及分类

1. PLD 的电路表示

由于 PLD 的与阵列和或阵列规模很大，为便于表达，其逻辑图采用一种与传统画法不同的简化表示方法，该方法的各种符号及含义如下。

（1）基本门电路的表示方式

PLD 中基本门电路符号如图 4.5.1 所示。图 4.5.1（a）和图 4.5.1（b）分别为与门 $L_1=ABC$ 和或门 $L_2=A+B+C$。由于 $L_3=A\cdot\overline{A}\cdot B\cdot\overline{B}=0$，图 4.5.1（c）给出了与门输出恒等于 0 的简化画法。图 4.5.1（d）中与门的所有输入项均不接通，保持“悬浮”的 1 状态，即 $L_4=1$。图 4.5.1（e）为具有互补输出的缓冲器。图 4.5.1（f）为三态输出缓冲器。

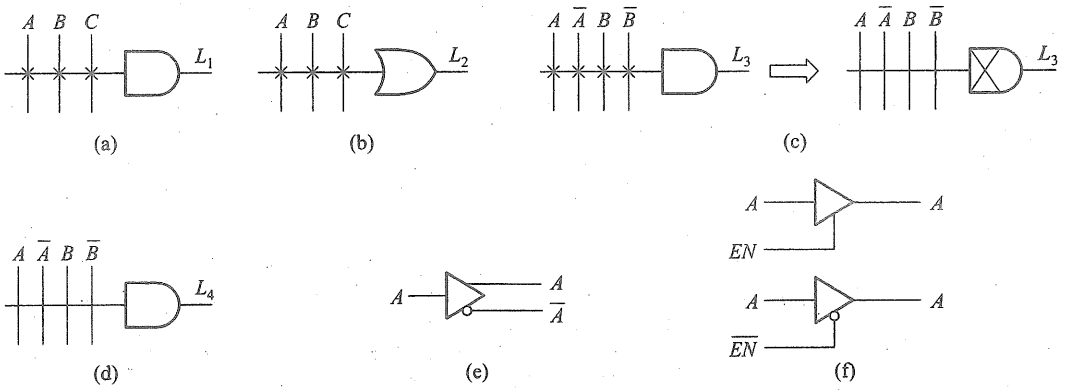


图 4.5.1 可编程器件中基本门电路的符号

（a）与门 （b）或门 （c）输出恒等于 0 的与门

（d）输出为 1 的状态 （e）互补输出的缓冲器 （f）三态输出缓冲器

（2）连接方式

由图 4.5.1 看出，与门和或门的输入由水平线和垂直线的交叉连接来表示。每个交叉点称为“单元”，单元的连接方式共有三种情况，如图 4.5.2 所示：

① 硬线连接：用“ • ”表示固定连接，不可以编程改变。

② 可编程“接通”单元：用“×”表示用户可编程连接。

③ 可编程“断开”单元：交叉点上没有任何标记表示两条线没有连接，称为被编程擦除单元。

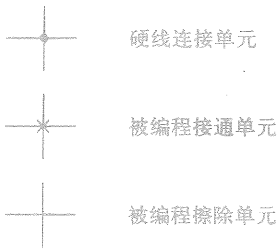


图 4.5.2 PLD 连接符号

(3) 编程连接技术

在 PLD 中采用的可编程连接单元主要有熔丝型、反熔丝型和浮栅 MOS 管型，如图 4.5.3 所示。

在熔丝型 PLD 中，没有编程时，单元由金属熔丝连通，如图 4.5.3(a)所示。在编程时，用比工作电流大许多的电流，将不需要连通的熔丝烧断。因为熔丝单元会占用大的芯片面积，这类器件的集成度受到限制。在门阵列中表示“单元”连接状况的阵列图，称为熔丝图，并且这个名称被沿用下来。

在反熔丝型 PLD 中，可编程连接单元的结构如图 4.5.3(b)所示。在多晶硅层和扩散层两个导体之间，有一层很薄的绝缘介质层，未编程时呈现高阻抗，相当于断开状态。编程时利用高压将介质层击穿，使上下两层导电材料连在一起，呈现导通状态。反熔丝单元占用芯片面积小，成本低，可靠性高。

熔丝型和反熔丝型 PLD 都只能进行一次编程，一旦编程后就不能再修改。

在浮栅 MOS 管型 PLD 中，单元的连接采用一种浮栅 MOS 管，如图 4.5.3(c)所示。这种 MOS 管在绝缘层中还有第 2 个栅极，由于与其他部分绝缘，所以称为浮栅。浮栅上没有电荷的 MOS 管，与普通 N 沟道增强型 MOS 管一样，当栅极加正常逻辑高电平时，MOS 管处于导通状态，否则截止。而经编程向浮栅上注入电子后，栅极在正常高电平作用下，MOS 管仍然处于截止状态，亦即无论栅极加正常高电平还是低电平，MOS 管始终相当于断开。如果以某种方式释放浮栅上的电子，也可以使 MOS 管恢复正常。浮栅 MOS 管是可以多次编程的连接单元。

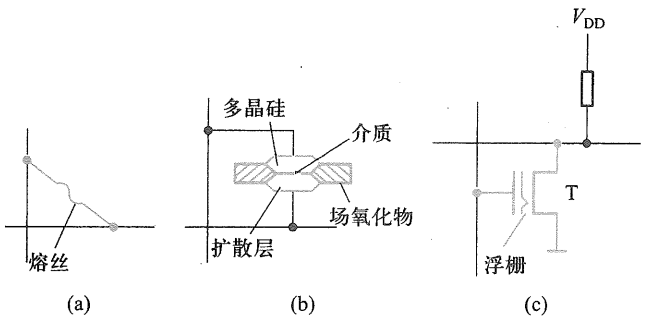


图 4.5.3 PLD 中编程连接单元结构

(a) 熔丝型 (b) 反熔丝型 (c) 浮栅 MOS 管型

(4) 浮栅 MOS 管编程原理简介

浮栅 MOS 管还可细分为叠栅注入 MOS 管(stacked-gate injection metal oxide semiconductor，SIMOS 管)、浮栅隧道氧化层 MOS 管(floating-gate tuned oxide，FLOTOX 管)和快闪(flash)叠栅 MOS 管。不同的浮栅 MOS 管连接的 PLD，编程信息的擦除方法也不同。

SIMOS 管连接的 PLD，采用紫外光照射擦除；FLOTOX 管和 Flash MOS 管，采用电擦除方法。

① SIMOS 管

SIMOS 管的结构如图 4.5.4(a)所示。它是一个增强型 NMOS 管，有两个重叠的栅极，控制栅 g_c 和浮栅 g_f。浮栅被绝缘的 SiO_2 包围着。编程处理前，浮栅上没有电荷，与普通 MOS 管一样。当控制栅加正常工作的高电平时，SIMOS 管处于导通状态，此时的阈值电压为 V_{T1}，转移特性如图 4.5.4(b)所示。编程时漏源间加几十伏的正电压，漏极与衬底间的 PN 结产生雪崩击穿，若同时在控制栅加正脉冲电压，则雪崩产生的高能电子，在栅极电场的作用下，穿过 SiO_2 层注入浮栅上。编程电压撤除后，因浮栅被绝缘层包围，注入的电子无放电通路，可以长期保留，此时 SIMOS 管的阈值电压升高到 V_{T2}，特性曲线右移如图 4.5.4(b)所示。因此，控制栅加正常逻辑高电平也不能达到其阈值电压，SIMOS 管始终截止，相当于断开一样。

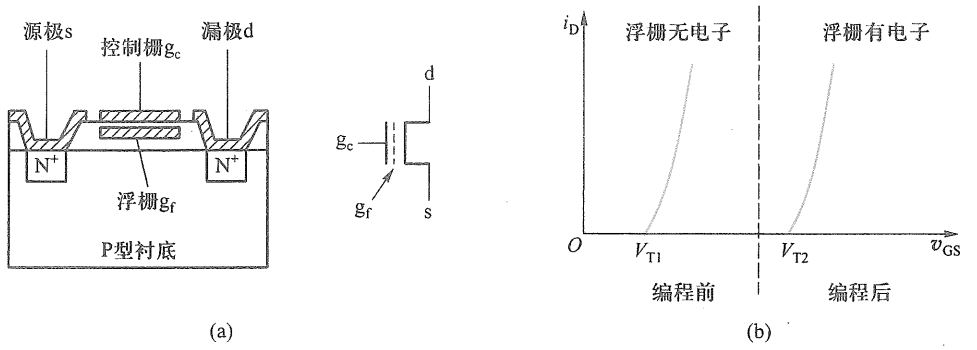


图 4.5.4 叠栅注入(SIMOS)管

(a) 结构 (b)浮栅上累积电子与开启电压的关系

擦除的方法是用紫外线灯照射器件 20 分钟，则 SiO_2 层中将产生电子-空穴对，为浮栅上的电子提供泄放通道，使之放电，SIMOS 管恢复到编程前的状态。

② FLOTOX 管开关

FLOTOX 管的结构及符号如图 4.5.5 所示。其浮栅与漏区间的氧化层很薄，称为隧道区。当隧道区的电场强度足够强时，漏区与浮栅之间便出现导电隧道，在电场的作用下，电子通过隧道形成电流，这种现象称为隧道效应。编程时在控制栅与漏极间加高的电压，隧道区产生强电场，吸引漏区的电子通过隧道到达浮栅。此时，在正常逻辑电平下，FLOTOX 管始终关断。

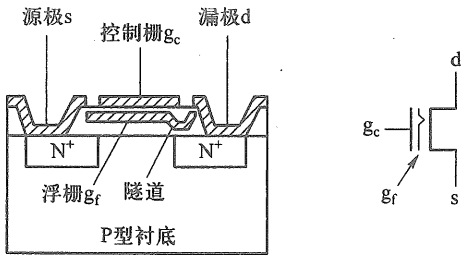


图 4.5.5 FLOTOX 管的结构及符号

如果需要擦除编程信息，将 FLOTOX 管漏极与控制栅所加高电压极性反过来，则浮栅上的电子在电场的作用下通过隧道回到漏区，FLOTOX 管恢复到编程前的状态，从而擦除了编程信息。FLOTOX 管的编程和擦除无须改变电路连接，称为在线编程，并且编程和擦除同时进行，每次编程是以新的信息代替原来的信息。

③ Flash MOS 管

Flash MOS 管的结构及符号如图 4.5.6 所示。它与 SIMOS 管结构类似。两者最大的区别在于 Flash MOS 管的浮栅到 P 型衬底间的氧化绝缘层比 SIMOS 管的更薄。因此,它可通过浮栅与源极之间超薄氧化层的电子隧道效应进行擦除,而无须紫外线或 X 射线照射。

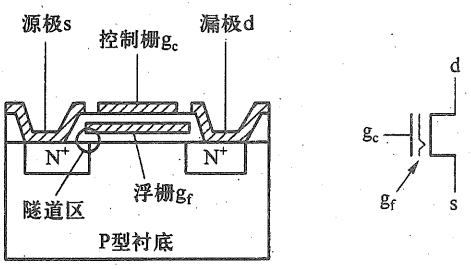


图 4.5.6 Flash MOS 管的结构及符号

对 Flash MOS 管编程时,漏极接正电压(6 V),源极接地,同时在控制栅加 12 V 正脉冲电压,向浮栅注入电子。编程后,Flash MOS 管在正常逻辑电平条件下相当于断开。

Flash MOS 管擦除时,在源极加 12 V 正脉冲电压,控制栅接地,即可利用隧道效应使浮栅放电而擦除编程信息。Flash MOS 管既有 SIMOS 管结构简单、工作可靠的优点,又有 Flotox 管隧道效应带来的速度快、操作简单的特点,因而被广泛使用。

2. PLD 的分类

PLD 的分类方法有多种。按照 PLD 门电路的集成度,可以分为低密度和高密度器件,1 000 门以下为低密度,例如可编程只读存储器(programmable read only memory,PROM)、可编程逻辑阵列(programmable logic array,PLA)、可编程阵列逻辑(programmable array logic,PAL)和通用阵列逻辑(generic array logic,GAL)等;1 000 门以上的为高密度,如 CPLD、FPGA 等。

按照 PLD 的结构体系,可分为简单 PLD(如 PAL、GAL),复杂可编程逻辑器件 CPLD 和现场可编程门阵列 FPGA。CPLD 和 FPGA 将在第 8 章介绍。

按照逻辑函数实现方式,可分为基于**与**、**或**阵列的 PLD(如 CPLD)和基于查找表 LUT 的 PLD(如 FPGA)。

按照 PLD 中的**与**、**或**阵列是否可编程分为三种,分别如图 4.5.7(a)、图 4.5.7(b)和图 4.5.7(c)所示,PROM 的**与**阵列固定,**或**阵列可编程;PLA 的**与**阵列、**或**阵列均可编程;PAL 和 GAL 等的**与**阵列可编程,**或**阵列固定。

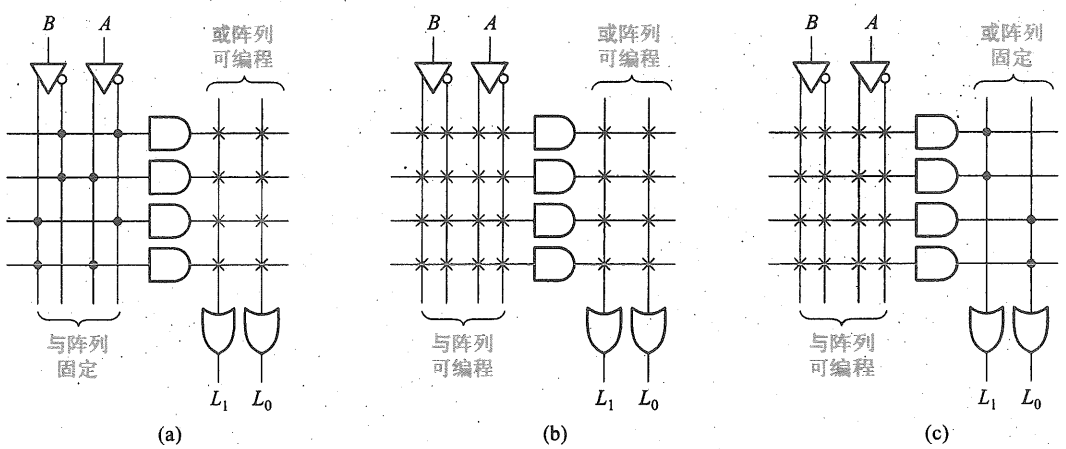


图 4.5.7 PLD 的分类

(a) PROM 的基本电路结构 (b) PLA 的基本电路结构 (c) PAL 的基本电路结构

4.5.2 组合逻辑电路的 PLD 实现

任何组合逻辑关系都可以变换成与-或表达式，因此通过 PLD 的与、或阵列可以实现任何一个逻辑函数。

从图 4.5.7(a)可以看出，PROM 的与阵列是将输入变量的全部最小项译出来了，如果用它来实现逻辑函数，往往只用到一部分最小项，芯片的利用率不高，因此很少作为 PLD 器件使用。

1. 可编程逻辑阵列 PLA

PLA 就是为解决 PROM 实现逻辑函数时芯片利用率不高的问题而设计的。由于它的与、或阵列均可编程，所以将逻辑函数化简后再实现，可以有效地提高芯片的利用率。

例 4.5.1　由 PLA 构成的逻辑电路如图 4.5.8 所示，试写出该电路的逻辑表达式，并确定其逻辑功能。

解：(1) 由图 4.5.8 可知，该电路有 7 个与项，根据或阵列得到输出逻辑表达式：

$$L_0=\overline{A}\,\overline{B}C+\overline{A}B\overline{C}+A\overline{B}\,\overline{C}+ABC$$

$$L_1=AB+AC+BC$$

(2) 列出真值表如表 4.5.1 所示。

(3) 由真值表看出该电路实现全加器的功能，A、B、C 分别为加数、被加数和低位进位数。L_0 为和数，L_1 为向高位的进位数。

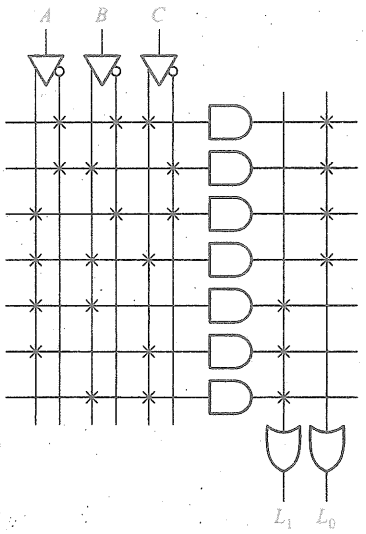


图 4.5.8　例 4.5.1 的 PLA 电路

表 4.5.1　例 4.5.1 电路的真值表

输入			输出	
A	B	C	L_1	L_0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

2. 可编程阵列逻辑 PAL

由于 PAL 只有与阵列可编程，因此编程相对简单。图 4.5.9 所示为一简单的 PAL 结构

图,它有 4 组 10×3 位的可编程与阵列,4 个输入信号和 1 个输出反馈信号产生 10 个与阵列的输入变量。每 3 个乘积项构成一组固定或阵列,共有 4 组输出。

PAL 与阵列所有交叉点都由熔丝连通(图 4.5.9 中所有交叉点上省略了“×”),编程时保留有用的熔丝,断开无用的熔丝,就得到所需的电路。

例 4.5.2 用图 4.5.9 所示 PAL 实现下列逻辑函数

$$\begin{cases} L_0 = AB\overline{C} + \overline{A}\,\overline{B}C\overline{D} \\ L_1 = A\overline{B}C + A\overline{C}\,\overline{D} + BC\overline{D} \\ L_2 = \overline{A}\,\overline{B}C + A\overline{B}\,\overline{C} \\ L_3 = AB\overline{C} + \overline{A}\,\overline{B}C\overline{D} + B\overline{C}D + AC\overline{D} \end{cases}$$

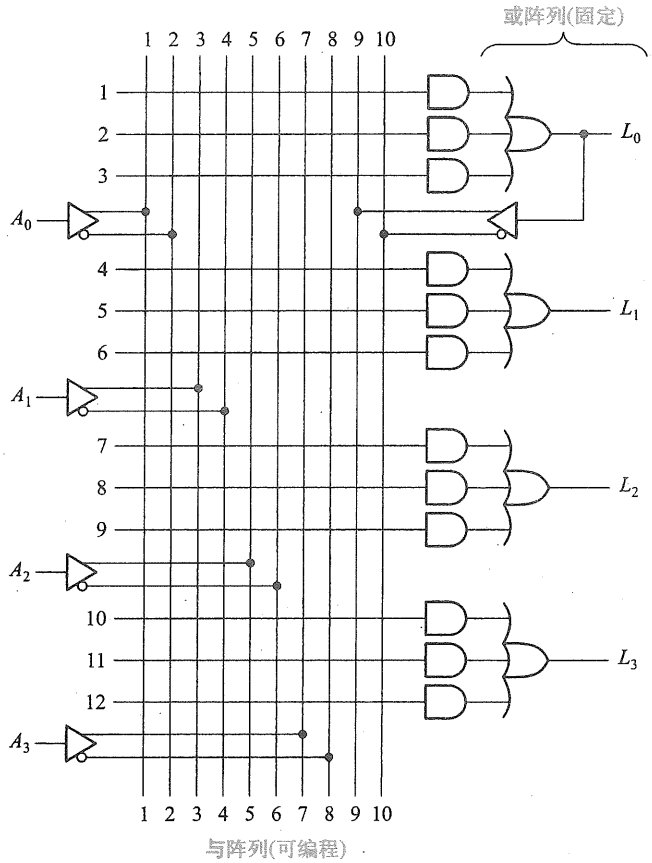


图 4.5.9 PAL 的基本电路结构图

解:这组逻辑函数均为简化的逻辑表达式,有 4 个输入变量,4 个输出。固定或阵列的每一个输出包含 3 个乘积项,而 $L_0 \sim L_2$ 三个表达式各包含 3 个以下乘积项,满足输出端对乘积项数目的要求,可以直接编程实现。

L_3 表达式包含 4 个乘积项,不能直接编程实现,但其前两项正好为 L_0,即

$$L_3 = AB\overline{C} + \overline{A}\,\overline{B}C\overline{D} + B\overline{C}D + AC\overline{D} = L_0 + B\overline{C}D + AC\overline{D}$$

因此,将 L_0 反馈到输入端作为 L_3 的输入就可以实现。编程后的逻辑图如图 4.5.10

所示。

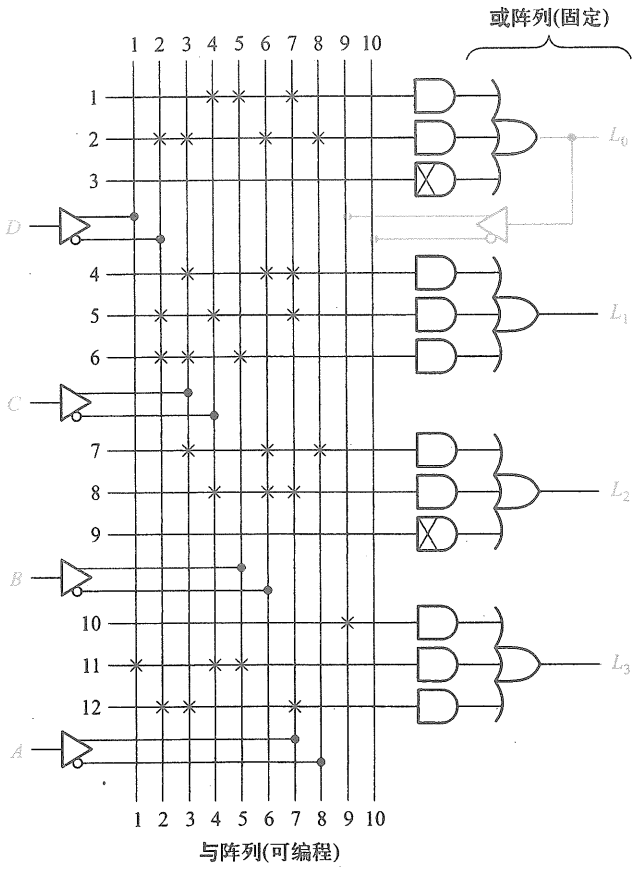


图 4.5.10 例 4.5.2 中编程后的 PAL 电路

由于大规模可编程器件的发展与应用,无论是 PLA、PAL 还是 GAL,都如同中规模集成器件一样,已逐渐退出历史舞台。这一节内容主要是为了便于读者理解可编程器件实现组合逻辑的基本原理。

复习思考题

4.5.1 PLD 编程单元的连接方式有哪几种?

4.5.2 PLD 连接单元主要有哪几种类型?

4.5.3 浮栅 MOS 管编程信息的擦除方法主要有哪些?

4.6 用 Verilog HDL 描述组合逻辑电路

2.5 节介绍了 Verilog 模块的基本结构以及 Verilog 的基本知识,并学习过门级建模和数据流建模方式,本节将进一步讨论组合逻辑电路的行为级建模以及分模块、分层次的结构化建模方法。行为级建模用于描述一个电路输入、输出的行为(即逻辑功能),而不是怎样去实现硬件电路。

4.6.1 组合逻辑电路的行为级建模

行为级建模一般使用 **always** 结构和过程赋值语句、条件语句(**if-else**)、多路分支语句(**case-endcase**)和 **for** 循环语句等。

1. 条件语句

条件语句就是根据条件表达式的真假,确定下一步进行的运算。Verilog 语言中有 3 种形式的 **if** 语句,一般用法如下:

```
if (condition_expr) true_statement;
```

或

```
if (condition_expr) true_statement;
else fale_ statement;
```

或

```
if (condition_expr1) true_statement1;
else if (condition_expr2) true_statement2;
else if (condition_expr3) true_statement3;
.....
else default_statement;
```

if 后面的条件表达式一般为逻辑表达式或关系表达式。执行 **if** 语句时,首先计算表达式的值,若结果为 **0**、x 或 z,按"假"处理;若结果为 **1**,按"真"处理,并执行相应的语句。

注意,在第三种形式中,从第一个条件表达式 condition_expr1 开始依次进行判断,直到最后一个条件表达式被判断完毕,如果所有的表达式都不成立,才会执行 **else** 后面的语句。这种判断上的先后次序,本身隐含着一种优先级关系,在使用时应予注意。

例 4.6.1 使用 **if-else** 语句对表 4.4.11 所示 4 选 1 数据选择器的行为(功能)进行描述。

解:4 选 1 数据选择器的代码如图 4.6.1 所示,模块的名称为 mux4to1。选择信号 S1、S0 使用一个 2 位的向量 **S** 来表示,它的 4 个可能值用 2 位二进制数来表示;输入端口 D3、D2、D1、D0 使用一个 4 位的向量 **D** 来表示;输出端口 Y 为 **reg** 变量。

该代码使用 **if-else** 语句来描述数据选择器的逻辑功能,它没有涉及实现选择器的具体电路结构。因此,这种描述方式被称为行为级建模。行为级建模的标识是 **always**[①] 结构,"@"符号后面圆括号中的部分,称为**敏感列表**(sensitivity list)。对组合逻辑电路来说,所有的输入都是敏感信号,应该写在圆括号内。

对 Verilog 进行仿真时,当敏感列表中的一个或多个信号的值改变时,仿真器将按顺序执行一次 **always** 下面由 **begin-end** 所包围的语句,执行完最后一条语句后,暂停执行,直到敏感列表中的信号再次发生变化。

对 Verilog 代码进行电路综合时,敏感列表直观地告诉 Verilog 编译器哪些信号会直接影响 **always** 块所产生的输出。

---

① 实际上,**always** 是一个 Verilog 语言结构,但习惯上使用术语"**always** 语句或者 **always** 块"来表述。

典型的 Verilog 模块可以包含几个 always 块，每个 always 块代表电路模型的一个部分。

注意，过程语句只能给变量赋值，在图 4.6.1 所示代码中通过关键词 reg 将输出变量 Y 声明成变量数据类型。

```
//文件名: mux4to1.v
//Verilog-2001/2005 语法
module   mux4to1    (
  input  D0, D1, D2, D3, //输入端口
  input [1:0] S,
  output reg  Y   //输出端口及数据类型
);
always @(D,S) //电路功能描述
  begin
    if (S == 2'b00)       Y = D0;
    else if (S== 2'b01)   Y = D1;
    else if (S== 2'b10)   Y = D2;
    else                  Y = D3;
  end
endmodule
```

图 4.6.1 数据选择器的 Verilog 代码（一）

2. 多路分支语句

case 语句是一种多分支条件选择语句，一般形式如下：

```
case (case_expr)
    item_expr1 : statement1;
    item _expr2 : statement2;
        ......
    default : default_statement;
endcase
```

执行时，首先计算 case_expr 的值，然后依次与各分支项中表达式的值进行比较，如果 case_expr 的值与 item_expr1 的值相等，就执行语句 statement1，以此类推，如果 case_expr 的值与所有列出来的分支项的值都不相等，则执行语句 default_statement。

注意：(1) 每个分支项中的语句可以是单条语句，也可以是多条语句。如果是多条语句，必须在多条语句的最前面写上关键词 **begin**，在这些语句的最后写上关键词 **end**，这样多条语句就成了一个整体，称之为顺序语句块。在分支语句中，可以嵌入 **if-else** 语句，也可以嵌入另一条 **case** 语句。

(2) 每个分支项表达式的值必须各不相同，一旦判断到与某分支项的值相同并执行相应语句后，case 语句的执行便结束了。

(3) 如果某几个连续排列的分支执行同一条语句，则这几个分支项表达式之间可以用逗号分隔，将语句写在这几个分支项表达式的最后一个中。

例 4.6.2 用 case 语句描述 4 选 1 数据选择器的行为，其功能表是在表 4.4.11 的基础上增加一个使能输入端 En，当 En = 0 时允许数据选择器工作，当 En = 1 时禁止数据选择器工作，输出为 0。

解：可以混合使用 **if-else** 和 **case** 语句对带使能输入端的 4 选 1 数据选择器进行描述，其代码如图 4.6.2 所示。首先使用 **if-else** 语句对使能输入端 En 进行判断，然后使用 **case** 语句对输入的选择条件进行比较。2 位选择输入向量 S 的 4 个可能值可以用十进制数来表示。

```
module mux4to1_bh   (
    input En,
    input  D0, D1, D2, D3
    input [1:0] S,
    output reg Y
);
    always @(D, S, En)
    begin
        if (En==1)   Y = 0;
        else      //当 En=0,实现选择功能
            case (S)
                2'd0: Y = D0;
                2'd1: Y = D1;
                2'd2: Y = D2;
                2'd3: Y = D3;
            endcase
        end
endmodule
```

图 4.6.2 数据选择器的 Verilog 代码（二）

例 4.6.3 根据功能表 4.4.8,对共阴极七段显示译码器的行为进行描述。

解: 描述真值表时,使用 **case** 语句比较方便,其代码如图 4.6.3 所示。程序中用位拼接运算符将输出端口 a、b、c、d、e、f、g 拼接起来成为一个 7 位向量,将 4 位 BCD 码输入 D_3、D_2、D_1、D_0 拼接成一个 4 位向量。**case** 语句的分支项按顺序排列,最后一个分支项 **default** 把其他非 8421BCD 码输入的情况设置成全零输出,可以省去所有无用输入码的显示。

注意,**case** 语句中列出的各个条件是不存在优先权差别的。

```
module seg7_decoder(                    //Verilog-2001/2005 语法
    input LE,BL,LT,D3,D2,D1,D0,
    output   reg   a,b,c,d,e,f,g
);
always @(*)        //Verilog-2001 规定可以用*号代表电路所有输入信号
begin
    if(LT==0)              {a,b,c,d,e,f,g}=7'b111_1111; //让显示器 7 段都发光,显示 8
    else if(BL==0)         {a,b,c,d,e,f,g}=7'b000_0000; //让显示器 7 段都熄灭
    else if(LE==1)         {a,b,c,d,e,f,g}={a,b,c,d,e,f,g}; //锁存显示
    else
      case ({D3,D2,D1,D0})                  //实现显示译码器的功能
              4'd0: {a,b,c,d,e,f,g}=7'b111_1110;   //7e
              4'd1: {a,b,c,d,e,f,g}=7'b011_0000;   //30
              4'd2: {a,b,c,d,e,f,g}=7'b110_1101;   //6d
              4'd3: {a,b,c,d,e,f,g}=7'b111_1001;   //79
              4'd4: {a,b,c,d,e,f,g}=7'b011_0011;   //33
              4'd5: {a,b,c,d,e,f,g}=7'b101_1011;   //5b
              4'd6: {a,b,c,d,e,f,g}=7'b001_1111;   //1f
              4'd7: {a,b,c,d,e,f,g}=7'b111_0000;   //70
              4'd8: {a,b,c,d,e,f,g}=7'b111_1111;   //7f
              4'd9: {a,b,c,d,e,f,g}=7'b111_1011;   //7b
            default: {a,b,c,d,e,f,g}=7'b000_0000; //非 8421BCD 码输入时,不显示
      endcase
    end
endmodule
```

图 4.6.3 共阴极七段显示译码器的 Verilog 代码

例 4.6.4 使用 **case** 语句描述表 4.4.9 所示 1 路-4 路数据分配器的功能(行为)。

解: 该例给出了 **case** 语句的嵌套使用情况,在使能信号 E = **1** 的分支语句中,嵌入另一条 **case** 语句对选择条件({S1,S0})进行比较,从而确定电路的输出,其代码如图 4.6.4 所示。

在 Verilog 语言中,**case** 语句还有另外两种形式,用关键词 **casex** 和 **casez** 表示,以便处理表达式值中含有逻辑值 x 和 z 的情况。**casez** 将分支项表达式和 case_expr 控制表达式中出现 z 的位当作无关位来处理,在分支项表达式中所有为 z 的位也可以用? 来表示。而 **casex** 则认为表达式中所有的 z 和 x 都为无关位。下面举例进行说明。

```
module Demux1_to_4 (
    output reg Y0,Y1,Y2,Y3,
    input In,
    input S1,S0,E
    );
always@(S1, S0, In , E)
    case(E)
      1'b1:case({S1,S0})
            2'b00: begin  Y0 = In;   Y1 = 1'bz;  Y2 = 1'bz;  Y3 = 1'bz;  end
            2'b01: begin  Y0 = 1'bz; Y1 = In;    Y2 = 1'bz;  Y3 = 1'bz;  end
            2'b10: begin  Y0 = 1'bz; Y1 = 1'bz;  Y2 = In;    Y3 = 1'bz;  end
            2'b11: begin  Y0 = 1'bz; Y1 = 1'bz;  Y2 = 1'bz;  Y3 = In;    end
          endcase
      default: begin  Y0 = 1'bz;  Y1 = 1'bz;  Y2 = 1'bz;  Y3 = 1'bz; end
    endcase
endmodule
```

图 4.6.4 数据分配器的 Verilog 代码

例 4.6.5 根据功能表 4.4.3,描述 8 线-3 线优先编码器的逻辑功能。

解：下面是两种不同的建模方式。在图 4.6.5(a)所示代码中,首先使用 casex 语句描述

```
module encoder8to3_bh(
  input EI,
  input [7:0] I,
  output reg [2:0] Y,
  output reg GS,EO
  );
always @(EI, I)
begin
if (EI==0) begin Y = 3'd0; GS=0; EO = 0;end
else   //当 EI=1 时,实现优先编码功能
  begin
      GS=1; EO = 0;
      casex (I)      // x 表示任意值
        8'b1xxx_xxxx: Y = 3'd7;
        8'b01xx_xxxx: Y = 3'd6;
        8'b001x_xxxx: Y = 3'd5;
        8'b0001_xxxx: Y = 3'd4;
        8'b0000_1xxx: Y = 3'd3;
        8'b0000_01xx: Y = 3'd2;
        8'b0000_001x: Y = 3'd1;
        8'b0000_0001: Y = 3'd0;
      default: begin Y = 3'd0; GS=0;EO=1;end
    endcase
  end
end
endmodule
```

(a)

```
module encoder8to3_bh(
  input EI,
  input [7:0] I,
  output reg [2:0] Y,
  output reg GS,EO
);
always @(EI, I)    //另一种描述方式
begin
  if (EI==0) begin Y = 3'd0; GS=0; EO = 0;end
  else
  begin
        GS=1; EO = 0;
        if(I[7]) Y = 3'd7; else
        if(I[6]) Y = 3'd6; else
        if(I[5]) Y = 3'd5; else
        if(I[4]) Y = 3'd4; else
        if(I[3]) Y = 3'd3; else
        if(I[2]) Y = 3'd2; else
        if(I[1]) Y = 3'd1; else
        if(I[0]) Y = 3'd0;
        else begin Y = 3'd0;
            GS=0;
            EO = 1;end
    end
  end
endmodule
```

(b)

图 4.6.5 优先编码器的 Verilog 代码

了带使能控制端优先级编码器。当 EI=1 时，才对 **casex** 语句中的条件项进行比较。当至少有一个输入数据为 1 时，编码器的输出 GS 必然为 1 且 EO 必然为 0，在 **casex** 语句执行之前，这两个输出信号即被设置，避免在每个分支语句中对 GS 和 EO 进行赋值。若 8 个分支表达式的值没有一个与 I 的值匹配，将会执行 **default** 分支后面语句，将输出设置成默认值。

注意，**casex** 在比较表达式时，忽略了表达式各个位中的 **x**，根据比较的先后次序，这样 **casex** 中先比较语句的优先级就高于后比较的语句，使得 **casex** 存在隐含的优先级。

在图 4.6.5(b)所示代码中，则使用嵌套的 **if-else** 语句来描述优先编码器的行为。注意，**if-else** 语句本身隐含着优先级关系。

3. for 循环语句

for 语句的一般形式如下：

```
for (initial_assignment;condition;step_assignment)  statement;
```

initial_assignment 为循环变量的初始值，condition 给出了循环的条件，只要条件为真，就执行过程赋值语句 statement，当循环条件不成立时，循环结束，执行 **for** 后面的语句。step_assignment 为循环变量的步长，每次迭代后，循环变量将增加或减少一个步长。

注意：(1) 循环变量必须为 **integer**(整数)型变量。

(2) 过程语句 statement 可以是过程赋值语句、条件语句和多路分支语句。也可以是另一条循环语句，即循环语句可以嵌套。

例 4.6.6 用 Verilog 描述一个具有使能输入端的 3 线-8 线译码器的功能，要求输出为低电平有效。

解： 3 线-8 线译码器的代码如图 4.6.6 所示，其功能参考表 4.4.5，但只有一个高电平有效的使能信号 En。当 En = 1 时，针对循环变量(k = 0、1、…、7)的变化，重复执行 8 次 **if-else** 语句。每一次迭代实现一个不同的子电路，例如 A = 0 时，设置 Y[0] = 0；A = 1 时，设置 Y[1] = 0，…，其余的以此类推。

```
module decoder3to8_bh(
  input [2:0] A,
  input  En,
  outputreg  [7:0]Y
);
  integer k;                    //声明一个整型变量 k

  always @(A, En)
  begin
    Y =8'b1111_1111;  //设置输出默认值
    for(k = 0; k <= 7; k = k+1)
      if ((En==1) && (A== k) )
          Y[k] = 0;    //当 En=1,根据 A 译码
      else
          Y[k] = 1;    //处理使能或输入无效
  end
endmodule
```

图 4.6.6 译码器的 Verilog 代码

例 4.6.7 试用 **for** 循环语句对 *n* 位串行加法器的逻辑功能进行描述。

解： 加法器的代码如图 4.6.7 所示。它使用 **for** 语句描述了一个通用的串行进位电路，在 **parameter** 语句中将参数 n 的初值设置为 32，所以此例代码实现的是 32 位加法器。但在实例引用该模块时，可以改变 n 值[①]来重新设置加法器的位宽。

① 改变参数的具体方法见本节例 4.6.11。

```
module addern_bh   #( parameter n=32)
 (   input [n-1:0] A,B,
     input   Cin,
     output reg [n-1:0]SUM,
     output reg Cout
);
      reg [n:0]C;   //声明中间变量
      integer k;    //声明一个整型变量 k
always @( A,B,Cin)
begin
  C[0]=Cin;
  for(k = 0; k < n; k = k+1)       //begin…end 之间语句循环 n 次
    begin
      SUM[k]= A[k] ^ B[k] ^ C[k];   //求和
      C[k+1]= (A[k] & B[k]) | (A[k] & C[k]) | (B[k] & C[k]); //进位
    end
  Cout = C[n];    //输出进位信号
end
endmodule
```

图 4.6.7 串行加法器的 Verilog 描述

for 循环体由 begin 和 end 之间的两条语句组成,第一条语句描述了第 k 级加法器所对应的求和函数,第二条语句描述了进位函数。其中 k 为循环次数,它的取值范围是 0 到 n−1,每经过一次循环,k 的值增加 1。

注意,在 Verilog 中,没有 C 语言中的“++”和“--”运算符,因此 k++和 k--是错误的。

例 4.6.8 描述表 4.4.10 所示 2 选 1 数据选择器的功能。

解:描述 2 选 1 数据选择器的方法很多,例 2.5.1 介绍过门级描述方式和数据流描述方式。这里介绍用条件运算符进行建模的两种方法。条件运算符可以用在连续赋值语句中,也可以用在 always 结构的过程语句中。其代码如图 4.6.8 所示。

```
//数据流描述方式
module mux2to1   #(parameter n=1)
( input   S,
  input [n-1:0]   D1,D0,
  output [n-1:0] Y
);
  assign Y = (S==0) ? D0 : D1;

endmodule
```

(a)

```
//行为描述方式
module mux2to1   #(parameter n=1)
(    input   S,
     input [n-1:0] D1,D0,
     output reg [n-1:0] Y
);
     always   @( D1,D0,S )
          Y = (S==0) ? D0 : D1;
endmodule
```

(b)

图 4.6.8 数据选择器的 Verilog 代码

可见，数据流建模也是根据电路的逻辑功能进行描述，不需要考虑电路组成以及元件之间的连接，也是属于电路行为描述范畴。下面再举一例，说明数据流建模方法。

例 4.6.9 用数据流方式描述表 4.4.4 所示 2 线-4 线译码器的逻辑功能。

解：2 线-4 线译码器的代码如图 4.6.9 所示。使用连续赋值语句描述其逻辑功能，并且 4 个逻辑表达式可以共用一条 **assign** 语句，但各表达式之间必须用逗号隔开，最后一个表达式用分号结束。

```
module decoder_df(
    input A1,A0,E,
    output [3:0]  Y
);
// 多条赋值语句
  assign Y[0] = ~(~A1 & ~A0 & ~E), //逗号分隔
         Y[1] = ~(~A1 & A0 & ~E),
         Y[2] = ~(A1 & ~A0 & ~E),
         Y[3] = ~(A1 & A0 & ~E);    //分号结尾
endmodule
```

图 4.6.9 2 线-4 线译码器的 Verilog 代码

也可以将每一个表达式单独写成一条 **assign** 语句，此时每条语句后面必须用分号结尾。

4.6.2 分模块、分层次的电路设计

在电路设计中，可以将两个或多个模块组合起来描述电路逻辑功能，通常称之为分模块、分层次的电路设计，自上而下(Top-down)和自下而上(Bottom-up)是两种常用的设计方法。在自上而下设计中，先定义顶层模块，然后再定义顶层模块中用到的子模块。而在自下而上设计中，底层的各个子模块首先被确定下来，然后将这些子模块组合起来构成顶层模块。

以图 4.4.37 所示的加法器电路为例，4 位串行进位全加器是一个顶层电路模块，它由 4 个 1 位全加器的子模块构成，而每个全加器又可以由两个半加器和一个或门构成，因此有 3 个层次如图 4.6.10 所示。采用自上而下的设计方法，首先定义 4 位串行进位全加器顶层模块，然后定义 1 位的全加器，最后定义底层的半加器子模块。如果用自下而上的设计方法，则设计次序相反，下面举例说明。

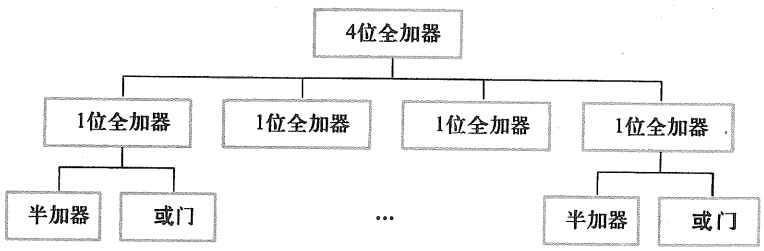


图 4.6.10 4 位全加器的层次结构框图

例 4.6.10 使用自下而上的方法描述图 4.6.10 所示三层次结构的 4 位全加器。

解：建立 4 位全加器模型需要经过下列 3 个步骤：

(1) 建立模块名为 halfadder 的 1 位半加器子模块，其代码如图 4.6.11 所示。其中引用基本门级元件 xor、and。综合后的电路与图 4.4.34(a)相同。

(2) 建立模块名为 fulladder 的 1 位全加器子模块，其代码如图 4.6.12 所示。这里引用两个半加器模块 halfadder 和一个基本或门元件(or)。综合后的电路与图 4.4.36 相同。

```
//一位半加器 (参考图 4.4.34)
module halfadder (
   input A,B,
   output S,C
);
   //实例引用门级元件
   xor (S,A,B);
   and (C,A,B);

endmodule
```

图 4.6.11 一位半加器的 Verilog 代码

```
//一位全加器 (参考图 4.4.36)
module fulladder (
   output Sum,Co,
    input A,B,Ci
);
   wire S1,D1,D2;        //内部节点信号
   halfadder HA1 (.B(B),.S(S1),.C(D1),.A(A));
   halfadder HA2 (.A(S1),.B(Ci),.S(Sum),.C(D2));
   or    (Co,D2,D1);
endmodule
```

图 4.6.12 一位全加器的 Verilog 代码

(3) 实例引用 4 个 1 位的全加器模块 fulladder 构成 4 位全加器的顶层模块，其代码如图 4.6.13 所示。综合后的电路与图 4.4.37 相同。

```
//四位全加器 (参考图 4.4.37)
module _4bit_adder (S,C3,A,B,C_1);
      input [3:0] A,B;
      input C_1;
      output [3:0] S;
      output C3;
      wire C0,C1,C2;   //声明子模块内部连接线
      fulladder U0_FA (S[0],C0,A[0],B[0],C_1);
      fulladder U1_FA (S[1],C1,A[1],B[1],C0);
      fulladder U2_FA (S[2],C2,A[2],B[2],C1);
      fulladder U3_FA (S[3],C3,A[3],B[3],C2);
endmodule
```

图 4.6.13 4 位全加器的 Verilog 代码

可见，当一个模块被其他模块实例引用时，就形成了层次化结构。这种层次表明了引用模块与被引用模块之间的关系，引用模块称为父模块，被引用的模块称为子模块，即包含子模块的模块是父模块。如果一个模块是实例引用其他子模块构成的，则该模块的描述就称为结构描述。

模块实例引用语句的格式如下：

module_name instance_name(port_associations) ;

其中，module_name 为子模块名称，instance_name 为实例引用名(或称为实例名)，port_associations 为父模块与子模块之间端口信号的关联方式，通常有名称关联法和位置关联法。

父模块引用子模块时，通过模块名完成实例引用，且实例名不能省略。例如，在图 4.6.12所示的一位全加器模块中，两次实例引用子模块 halfadder，其实例名分别是 HA1 和 HA2(不能省略)，而引用基本门级元件 or 时，则可以省略实例名。注意，实例名在父模块中必须是唯一的。

下面说明父、子模块端口关联问题。当端口较多时，建议使用名称关联的方式。图 4.6.12中采用了这种方式，其中第 2 次引用子模块 halfadder 的语句如图 4.6.14 所示，带有圆

点的名称(如.S、.C 等)是子模块中使用的端口名称,也称为形式名称,写在圆括号内的名称(如 Sum、D2 等)是父模块中使用的新名称,也称为实际名称。用这种方式实例引用子模块时,直接通过名称建立模块端口的连接关系,不需要考虑端口的排列次序,端口连接清楚,不容易犯粗心的错误。

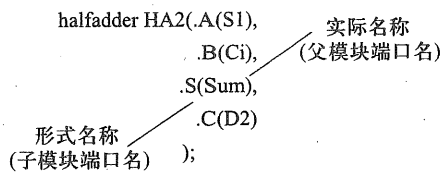


图 4.6.14 模块端口的名称关联法

当子模块的端口较少时,可以使用位置(端口排列次序)对应关联的方式。在图 4.6.13 中采用了这种方式,其中第一次引用 1 位全加器语句为

```
fulladder U0_FA (S[0],C0,A[0],B[0],C_1)
```

其中,父模块信号 S[0]、C0、A[0]、B[0]、C_1 与子模块 fulladder 的端口 Sum Co、A、B、Ci 依次连接。父模块引用子模块时可以使用一套新端口,也可以使用同名的旧端口,但必须注意端口的排列次序。

另外,端口关联时允许某些端口不连接,方法是:让不需要连接的端口位置为空白即可(但逗号不能省略)。在进行逻辑综合时,未连接的输入端口值被设置为高阻态(z),未连接的输出端口表示该端口没有被使用。

在顶层模块_4bit_adder 中,每一个 fulladder 后面的端口名称隐含地表示了这些子模块是如何相互连接在一起的。例如,子模块 U0_FA 中的进位输出 C0 被连接到子模块 U1_FA 中作为进位输入信号。

关于模块引用的几点注意事项:

(1) 模块只能以实例引用的方式嵌套在其他模块内,嵌套的层次是没有限制的,但不能在一个模块内部使用关键词 **module** 和 **endmodule** 去定义另一个模块,也不能以循环方式嵌套模块,即不能在 **always** 语句内部引用子模块。

(2) 实例引用的子模块可以是一个设计好的 Verilog 文件(即一个设计块),也可以是 FPGA 元件库中一个元件或嵌入式元件功能块,或者是用别的 HDL 语言(如 VHDL、AHDL 等)设计的元件,还可以是 IP(intellectual property,知识产权)核模块。

(3) 在一条实例引用子模块的语句中,不能混合使用位置关联和名称关联建立端口之间的连接。

例 4.6.11 试用 **parameter** 语句描述一个 4 位加法器子模块。然后在一个顶层模块中实例引用两次该模块,描述一个 8 位的加法器和一个 16 位的加法器。

解: 带参数的 4 位加法器子模块代码如图 4.6.15 所示。用一条连续赋值语句描述其逻辑功能,加号"+"指明是二进制数的加法,由于 A 和 B 都是 4 位的,而 Ci 为 1 位,所以运算结果可能为 5 位,用{Co,Sum}拼接起来表示。

顶层模块的代码如图 4.6.16 所示。由于子模块 adder 中参数 n 的初值为 4,所以顶层模块 adder_hier 实例引用子模块 adder 时,要将参数 n 定义成所需的 8 和 16。可以在顶层模块使用 **defparam** 语句。语句

```
defparam U1.n = 8;
```

将 adder 实例 U1 中 n 的值重新设置为 8。类似地,语句

```
defparam U2.n = 16;
```

将实例 U2 中 n 的值重新设置为 16。

```
//带参数的 4 位加法器子模块
module adder #(parameter n=4 )
 (
  input [n-1:0] A,B,
  input Ci,
  output [n-1:0] Sum,
  output Co
);
  assign {Co,Sum} = A + B + Ci;

endmodule
```

图 4.6.15 带参数的 4 位加法器

```
module adder_hier (
  input [7:0] A1,B1,
  input [15:0] A2,B2,
  input Cin1,Cin2 ,
  output [7:0] Sum1,
  output [15:0] Sum2,
  output Cout1,Cout2
);

  adder  U1 (A1,B1, Cin1, Sum1, Cout1);
  defparam U1.n = 8;
  adder  U2 (A2,B2, Cin2, Sum2, Cout2);
  defparam U2.n = 16;
endmodule
```

图 4.6.16 设置参数的一个例子

另一种定义 n 的方式是在顶层模块中直接使用 Verilog 运算符#,方法是在子模块名后面的括号中写上新的参数值[这里用#(8)和#(16)],其代码如图 4.6.17 所示。

图 4.6.18 给出了图 4.6.17 中代码的另一种版本。这里,仍然使用#运算符,但通过#(.n(8))和 #(.n(16))明确地包含了参数名称 n,同时,还使用名称关联方式明确地给出了子模块端口名称。虽然图 4.6.18 中的代码较长,但表述更清楚,是一种值得推荐的编写代码风格。

```
module adder_hier (
  input [7:0] A1,B1,
  input [15:0] A2,B2,
  input Cin1,Cin2,
  output [7:0] Sum1,
  output [15:0] Sum2,
  output Cout1,Cout2
);
    //按位置顺序关联端口
  adder #( 8) U1 (A1,B1, Cin1,
        Sum1, Cout1);

  adder #( 16 ) U2 (A2,B2, Cin2,
        Sum2, Cout2);

endmodule
```

图 4.6.17 采用运算符#定义参数

```
module adder_hier (
  input [7:0] A1,B1,     input [15:0] A2,B2,
  input Cin1,Cin2,       output [7:0] Sum1,
  output [15:0] Sum2,  output Cout1,Cout2
);
  adder #( .n(8)) U1(
        .A(A1),
        .B(B1),
        .Ci(Ci1),
        .Sum( Sum1),
        .Co( Cout1)
  );
  adder #( .n(16) ) U2(
        .A(A2),
        .B(B2),
        .Ci(Cin2),
        .Sum(Sum2),
        .Co(Cout2)
  );
endmodule
```

图 4.6.18 按名称关联端口及参数

小 结

- 根据电路的结构和工作特点，将数字电路分为两大类，即组合逻辑电路和时序逻辑电路。
- 组合逻辑电路的特点是，其输出状态在任何时刻只决定于同一时刻的输入状态。它可以由逻辑门电路或可编程器件(PLD)等组成。组合逻辑电路在形式和功能上种类繁多，但其分析方法和设计方法具有共同特点。因此，学习的重点是掌握一般的分析方法和设计方法。
- 分析组合逻辑电路的目的是确定已知电路的逻辑功能，其步骤大致是：写出各输出端的逻辑表达式→化简和变换逻辑表达式→列出真值表→确定功能。
- 设计组合逻辑电路的目的是根据提出的实际问题，设计出逻辑电路。设计步骤大致是：明确逻辑功能→列出真值表→写出逻辑表达式→逻辑化简和变换→画出逻辑图。
- 组合逻辑电路优化实现的技术包括：多输出的化简变换，用提取公因子或函数分解来减少扇入数。
- 电路在信号电平变化的瞬间经常会产生竞争-冒险现象，在电路设计过程中要采取措施避免产生竞争-冒险。
- 常用的组合逻辑电路模块包括编码器、译码器、数据选择器、数值比较器、加法器和算术运算单元等。这些组合逻辑电路除了具有其基本功能外，通常还具有输入使能、输出使能、输入扩展、输出扩展功能，使其功能更加灵活，便于构成较复杂的逻辑系统。在用可编程逻辑器件设计逻辑系统时，这些常用的组合逻辑电路经常被当作基本功能模块调用。
- 可编程逻辑器件(PLD)由用户定义和设置逻辑功能，可以实现各种组合逻辑电路。其特点是结构灵活、集成度高、速度快和可靠性高等。
- 在电路设计中，可以将两个或多个模块组合起来描述电路逻辑功能，通常称之为分模块、分层次的电路设计。如果一个模块是实例引用其他子模块构成的，则该模块的描述就称为**结构描述**。
- 用 Verilog 对组合逻辑电路建模时有三种不同的描述风格，即行为级建模、数据流级建模和结构级建模，其中数据流级建模方式也属于行为建模的范畴，而结构级建模方式通常用于规模较大的复杂逻辑电路。

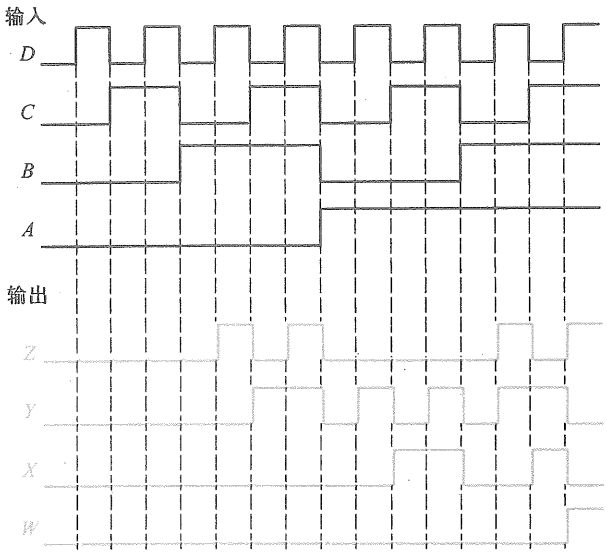
习 题

4.1 组合逻辑电路的分析

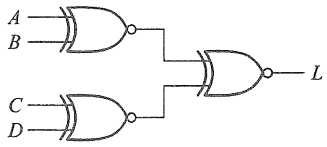
4.1.1 组合逻辑电路的输入波形(从高位到低位依次为 *ABCD*)及输出波形(从高位到低位依次为 *WXYZ*)如图题 4.1.1 所示，试分析电路的逻辑功能。

4.1.2 试分析图题 4.1.2 所示逻辑电路的功能。

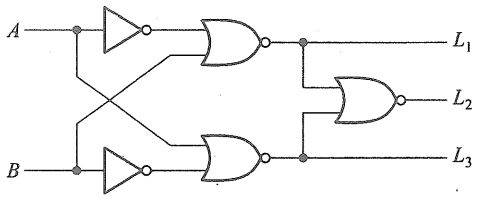
4.1.3 逻辑电路如图题 4.1.3 所示，试分析其逻辑功能。



图题 4.1.1



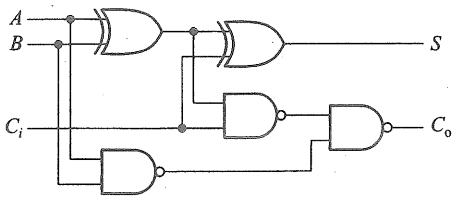
图题 4.1.2



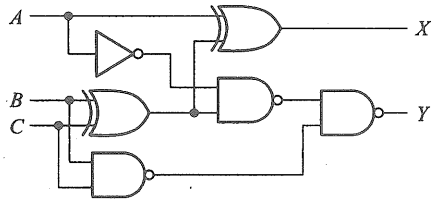
图题 4.1.3

4.1.4 试分析图题 4.1.4 所示逻辑电路的功能。

4.1.5 分析图题 4.1.5 所示逻辑电路的功能。

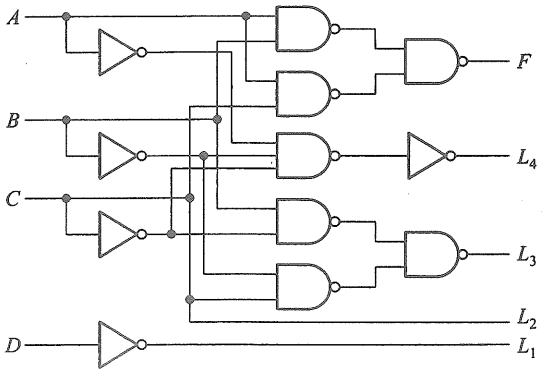


图题 4.1.4



图题 4.1.5

4.1.6 分析图题 4.1.6 所示逻辑电路的功能。



图题 4.1.6

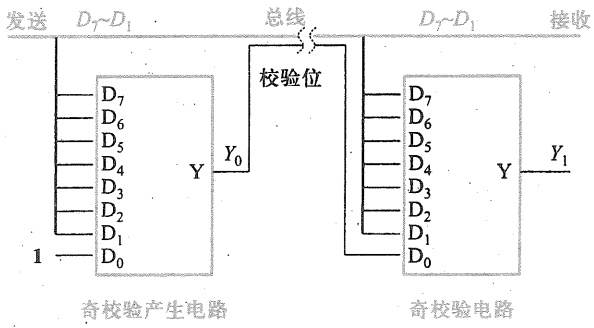
4.2 组合逻辑电路的设计

4.2.1 设计一个 4 位二进制数表示的十进制数为素数的检测电路，当输入为素数时，输出为 1，否则输出为 0。写出其最小项逻辑表达式，并化简成最简与或表达式。

4.2.2 试用 2 输入与非门设计一个 3 输入的组合逻辑电路。当输入的二进制码小于 3 时，输出为 0；输入大于等于 3 时，输出为 1。

4.2.3 试设计一个 4 位的偶校验器，即当 4 位数中有偶数个 1（或没有 1）时输出为 1，否则输出为 0。采用异或门及非门实现。

4.2.4 图题 4.2.4 所示为奇校验数据传送电路示意图。发送端的 7 位数据 $D_7 \sim D_1$ 与奇校验产生电路生成的校验位 Y_0 一起通过总线传送。在接收端，奇校验电路检测 $D_7 \sim D_1$ 及 D_0 以判断传送数据的正确性。为使两部分电路的输入端数目相等，奇校验产生电路增加了 D_0 端。试用异或门及非门设计：



图题 4.2.4

（1）奇校验产生电路和奇校验电路。如果要传输 ASCII 码的“O”或者“K”，则 Y_0 和 Y_1 各为何值？

（2）偶校验产生电路和偶校验电路，要求偶校验产生电路的 $D_0=0$。如果要传输 ASCII 码的“O”或者“K”，则 Y_0 和 Y_1 又为何值？

4.2.5 试设计一个具有控制端 C 及 4 个输入 $A_3A_2A_1A_0$、4 个输出 $L_3L_2L_1L_0$ 的逻辑电路。当控制信号 $C=0$ 时，输出状态与输入状态相反；$C=1$ 时，输出状态与输入状态相同。可以采用任何逻辑功能的门电路来实现。

4.2.6 试设计一可逆的 4 位码转换电路。当控制信号 $C=1$ 时，它将二进制码转换为格雷码；$C=0$ 时，它将格雷码转换为二进制码。可以采用任何门电路来实现。

4.2.7 试设计一组合逻辑电路，能够对输入的 4 位二进制数进行求反加 1 的运算。可以采用任何门电路来实现。

4.2.8 房间照明灯有主控和辅控两个开关控制，当主控开关闭合，或者辅控开关闭合并且门打开时灯亮，要求用与非门实现控制电路。

4.2.9 某足球评委会由一位教练和三位球迷组成，对裁判员的判罚进行表决。当满足以下条件时表示同意：有三人或三人以上同意，或者有两人同意，但其中一人是教练。试用 2 输入与非门设计该表决电路。

4.2.10 某雷达站有三部雷达 A、B、C，其中 A 和 B 功率消耗相等，C 的功率是 A 的两倍。这些雷达由两台发电机 X 和 Y 供电，发电机 X 的最大输出功率等于雷达 A 的功率消耗，发电机 Y

的最大输出功率是 X 的 3 倍。要求设计一个逻辑电路，能够根据各雷达的启动和关闭信号，以最节约电能的方式启、停发电机。可以采用任何门电路来实现。

4.2.11 某火车站有特快、直快和慢车三种类型的客运列车进出，试用 2 输入与非门和反相器设计一个指示列车等待进站的逻辑电路，3 个指示灯一、二、三号分别对应特快、直快和慢车。列车的优先级别依次为特快、直快和慢车，要求当特快列车请求进站时，无论其他两种列车是否请求进站，一号灯亮。当特快没有请求，直快请求进站时，无论慢车是否请求，二号灯亮。当特快和直快均没有请求，而慢车有请求时，三号灯亮。

4.2.12 会议室顶灯分别由安装在 4 扇门旁边的 4 个开关控制，设计一个控制电路，要求改变任意一个开关的状态都能打开或关闭顶灯。可以采用任何门电路来实现。

4.2.13 某化工厂有五种原料，编号为 1 到 5，在使用时必须遵守以下规则：

(1) 用第 3 号时必须使用第 1 号。

(2) 第 2 号和第 4 号必须同时使用。

(3) 第 2 号和第 5 号不能同时使用。

试设计一个逻辑电路，能在违反上述任何一项规定时给出高电平指示信号。写出逻辑表达式。

4.2.14 设计一个电路，能实现表题 4.2.14 所示的逻辑功能，可以采用任何门电路来实现。化简变换时考虑多输出函数的公共乘积项，以减少门的数目。

表题 4.2.14 真 值 表

A	B	C	L_1	L_2
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	1	0
1	1	1	0	0

4.2.15 设计一 2 位二进制数相加的逻辑电路，可以用任何门电路实现。化简变换时考虑多输出函数的公共乘积项，以减少门的数目。提示：

$$\begin{array}{cccc} & & A_1 & A_0 \\ + & & B_1 & B_0 \\ \hline & C_1 & S_1 & S_0 \end{array}$$

A_1、A_0 和 B_1、B_0 分别为被加数和加数，S_1、S_0 为相加的和，C_1 为进位位。

4.3 组合逻辑电路中的竞争-冒险

4.3.1 判断下列逻辑函数是否有可能产生竞争-冒险，如果可能应如何消除。

(1) $L_1(A,B,C,D) = \sum m(5,7,13,15)$

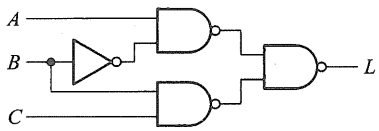
(2) $L_2(A,B,C,D) = \sum m(5,7,8,9,10,11,13,15)$

(3) $L_3(A,B,C,D)=\sum m(0,2,4,6,8,10,12,14)$

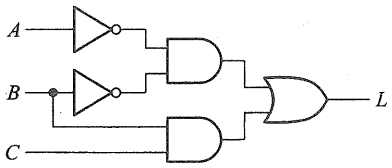
(4) $L_4(A,B,C,D)=\sum m(0,2,4,6,12,13,14,15)$

4.3.2 判断图题 4.3.2 所示电路是否会产生竞争-冒险。

4.3.3 判断图题 4.3.3 所示电路在什么条件下产生竞争-冒险,怎样修改电路能消除竞争-冒险?



图题 4.3.2



图题 4.3.3

4.3.4 画出下列逻辑函数的逻辑图,电路在什么条件下产生竞争-冒险,怎样修改电路能消除竞争-冒险?

$$L(A,B,C)=(A+\overline{B})(B+C)$$

4.4 常用组合逻辑电路模块

4.4.1 图 4.4.4 所示优先编码器的输入端 $I_1=I_3=I_5=1$,其余输入端均为 0,试确定其输出 $Y_2Y_1Y_0$。

4.4.2 试用**与非**门和**非**门设计一 4 输入的优先编码器,要求输入、输出及工作状态标志均为高电平有效。列出真值表,画出逻辑图。

4.4.3 10 线-4 线优先编码器的功能表如表题 4.4.3 所示,试用该优先编码器模块和适当的门构成输入为低电平有效的 $\overline{I}_0\sim\overline{I}_9$,输出为高电平有效的 8421BCD 码,并具有编码输出标志的编码器。

表题 4.4.3 10 线-4 线优先编码器功能表

输入									输出			
$\overline{I}_1$	$\overline{I}_2$	$\overline{I}_3$	$\overline{I}_4$	$\overline{I}_5$	$\overline{I}_6$	$\overline{I}_7$	$\overline{I}_8$	$\overline{I}_9$	$\overline{Y}_3$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$
1	1	1	1	1	1	1	1	1	1	1	1	1
×	×	×	×	×	×	×	×	0	0	1	1	0
×	×	×	×	×	×	×	0	1	0	1	1	1
×	×	×	×	×	×	0	1	1	1	0	0	0
×	×	×	×	×	0	1	1	1	1	0	0	1
×	×	×	×	0	1	1	1	1	1	0	1	0
×	×	×	0	1	1	1	1	1	1	0	1	1
×	×	0	1	1	1	1	1	1	1	1	0	0
×	0	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0

4.4.4 试用表题 4.4.3 所示的编码器模块设计键盘编码电路，十个按键分别对应十进制数 0~9，编码器的输出为高有效的 8421BCD 码。要求按键 9 的优先级别最高，其余依次降低，并且有工作状态标志，以说明没有按键按下和按键 0 按下两种情况。

4.4.5 试用两片 74HC138 构成 4 线-16 线译码器，输入为 4 位二进制码 $B_3B_2B_1B_0$，输出 $\overline{L}_0$~$\overline{L}_{15}$ 为低电平有效信号。

4.4.6 译码器的功能表如表题 4.4.6 所示，试用 74HC138 实现该译码器。

表题 4.4.6 真 值 表

选择输入				译码输出									
D	C	B	A	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	1	0	1	1
1	0	1	1	1	1	1	1	1	1	1	1	0	1
1	1	0	0	1	1	1	1	1	1	1	1	1	0

4.4.7 在图 4.4.10 所示的一个 20 位地址总线和 8 位数据总线的计算机系统中，扩展的 8 个外围设备为 8 个存储芯片。每个存储芯片有 13 根地址线并与计算机地址总线的低 13 位连接。地址总线中剩余的 7 根地址线作为译码器的控制信号和选择信号。试确定每一个存储芯片对应在计算机的地址范围，并用十六进制数表示。

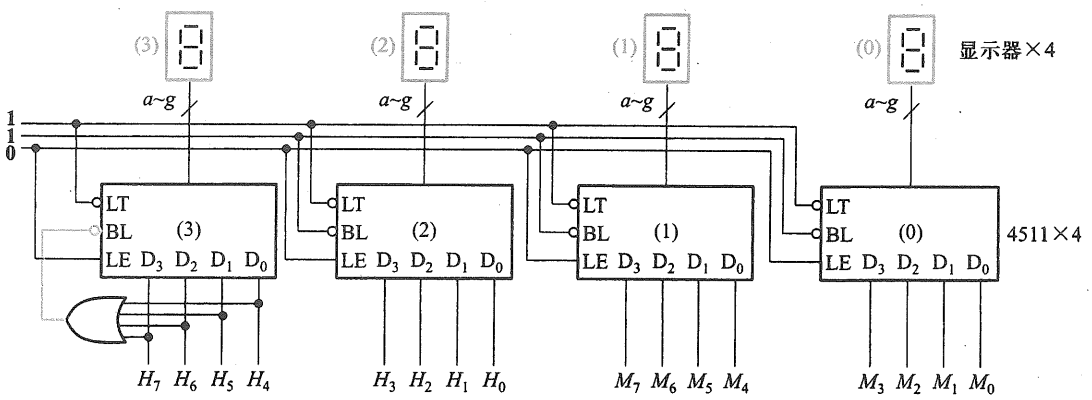
4.4.8 用译码器 74HC138 和适当的逻辑门实现函数 $F=\overline{A}\,\overline{B}\,\overline{C}+A\,\overline{B}\,\overline{C}+AB\overline{C}+ABC$。

4.4.9 试用译码器 74HC138 和必要的与非门，设计一个乘法器电路，实现两位二进制数相乘，并输出结果。

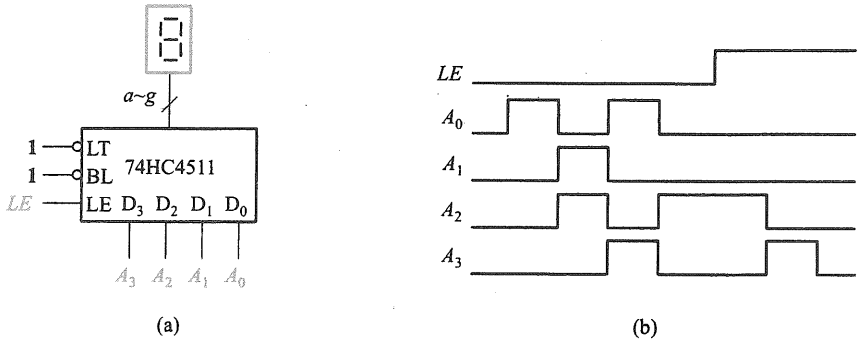
4.4.10 由 74HC4511 构成 24 小时及分钟的译码电路如图题 4.4.10 所示，试分析小时高位是否具有零熄灭功能。

4.4.11 七段显示译码电路如图题 4.4.11(a)所示，对应图题 4.4.11(b)所示输入波形，试问显示器显示的字符序列是什么？

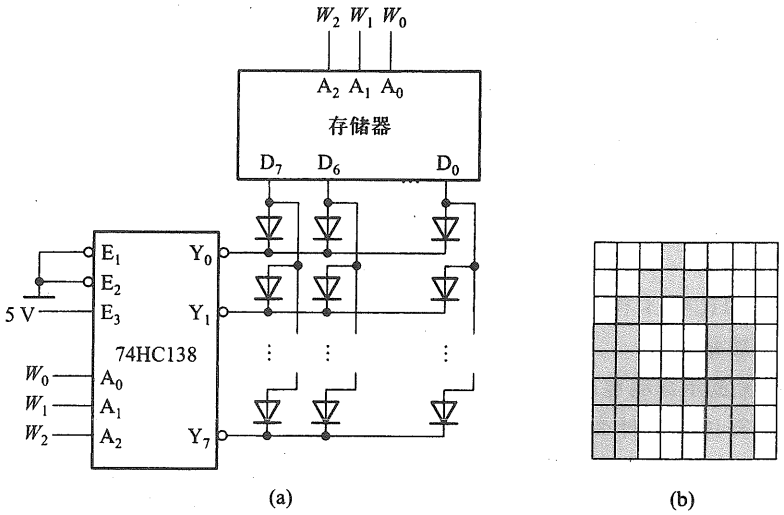
4.4.12 图题 4.4.12 所示为 8×8 个 LED 阵列显示示意图。3 线-8 线译码器控制逐行扫描，从上到下每次显示一行。存储阵列共有 8×8 个存储单元，每个单元存放 1 位显示的数据，需要显示的点存 1，否则存 0。地址线 $W_2W_1W_0$ 从 000 到 111 变化时，每次将一组 8 个数据送到输出端，控制发光二极管，需要发光的二极管接 1，否则接 0。如要显示的字形如图题 4.4.12(b)所示，试写出存储器存放的数据。若人的视觉暂留时间为 0.05 s，在满足 LED 阵列图像稳定不闪烁的情况下，试计算地址变换的最低频率。



图题 4.4.10



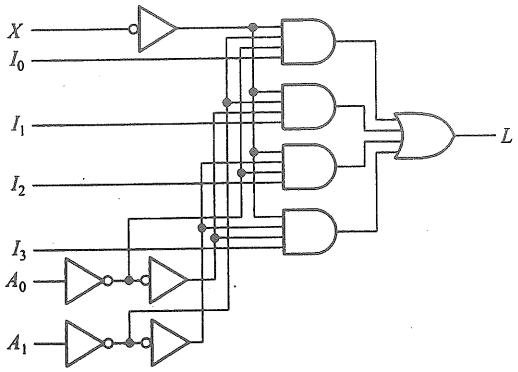
图题 4.4.11



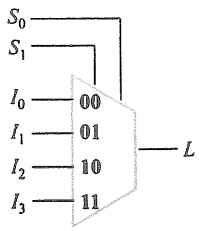
图题 4.4.12

4.4.13 数据选择器如图题 4.4.13 所示，并行输入数据 $I_3I_2I_1I_0=\mathbf{1010}$，控制端 $X=\mathbf{0}$，A_1A_0 的态序分别为 **00**，**01**，**10**，**11**，试画出输出端 L 的波形。

4.4.14 数据选择器如图题 4.4.14 所示，当 $I_3=0, I_2=I_1=I_0=1$ 时，有 $L=\overline{S}_1+S_1\overline{S}_0$ 的关系，证明该逻辑表达式的正确性。



图题 4.4.13



图题 4.4.14

4.4.15 应用图题 4.4.14 所示的电路产生逻辑函数 $F=S_1+S_0$。

4.4.16 设计一 4 选 1 数据选择器。数据输入是 I_0、I_1、I_2、I_3，数据输出是 Y，4 个控制信号为 S_3、S_2、S_1、S_0。要求只当 $S_i=1$ 时，I_i 与 Y 接通，且由另一控制信号 E 作为该选择器的使能信号。

(1) 画出由反相器、两输入与门和或门实现的逻辑电路。

(2) 选择一合适的三态门作为输出级。

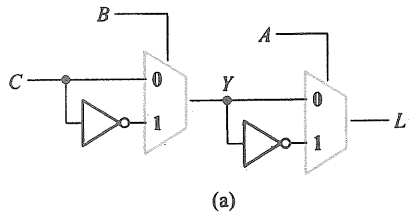
4.4.17 用 32 选 1 数据选择器选择数据，若选择的输入数据为 D_{10}、D_{17}、D_{18}、D_{27}、D_{31}，试依次写出对应的地址码。

4.4.18 由数据选择器构成的电路如图题 4.4.18 所示，试写出输出端的逻辑函数式。

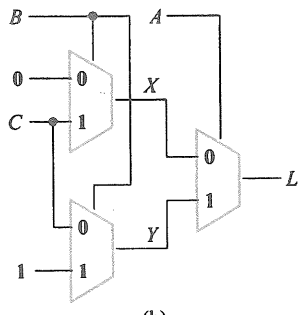
4.4.19 由 2 选 1 数据选择器构成的电路如图题 4.4.19 所示，其数据输入端和选择输入端可接 0、1 或输入变量，试用该电路实现下列逻辑函数。

(1) $L(A,B)=AB+C$

(2) $L(A,B,C)=AC+BC$

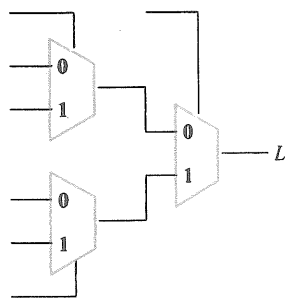


(a)



(b)

图题 4.4.18



图题 4.4.19

4.4.20 试用 4 选 1 数据选择器产生下列逻辑函数。

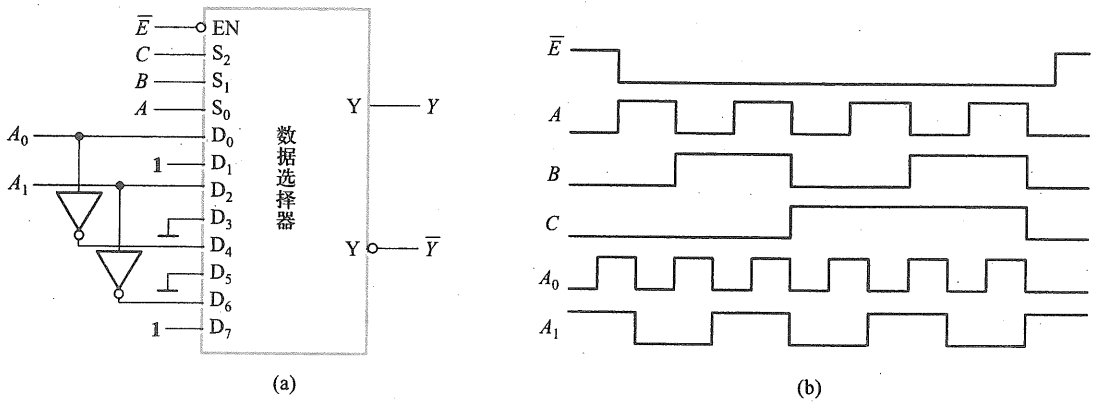
(1) $L(A,B)=\overline{A}\,\overline{B}+AB$

(2) $L(A,B,C)=\sum m(1,2,6,7)$

4.4.21 用香农展开定理分解逻辑函数 $L=\overline{A}\,\overline{C}+\overline{A}B+B\,\overline{C}$，试用一个 2 选 1 数据选择器和必要的门电路实现。

4.4.22 用香农展开定理分解逻辑函数 $L=A\,\overline{C}+\overline{A}C+B\,\overline{C}+A\,\overline{B}$，只用 2 选 1 数据选择器实现，输入可以是 **0**、**1**、原变量或非变量，画出逻辑图。

4.4.23 具有低使能控制的 8 选 1 数据选择器（74HC151，$\overline{E}=1$ 时，$Y=0$）构成的电路和各输入端的输入波形如图题 4.4.23 所示，画出输出端 Y 的波形。



图题 4.4.23

4.4.24 应用具有低使能控制的 8 选 1 数据选择器（74HC151）实现下列逻辑函数。

（1）$L=A\,\overline{B}\,\overline{C}+A\,\overline{B}C+\overline{A}\,\overline{B}C$

（2）$L=(A\odot B)\odot C$

4.4.25 用 2 选 1 数据选择器构建一个 4 输入查找表 LUT，画出电路结构图。

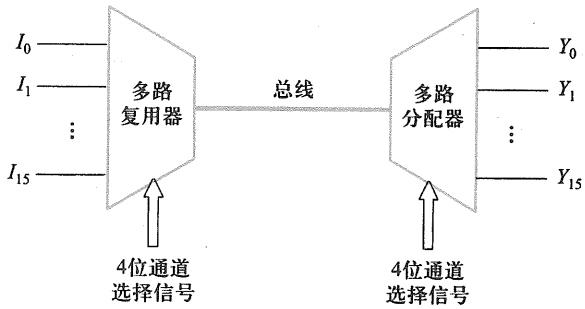
4.4.26 试用 3 输入 LUT 实现下列逻辑函数，画出电路结构，标明输入端存储单元的数值。

（1）$L=\overline{A}\,\overline{B}\,\overline{C}+A\,\overline{B}\,\overline{C}+ABC$

（2）$L=A\oplus B\oplus C$

4.4.27 试用 2 选 1 数据选择器设计一个将 4 位二进制 $D_3D_2D_1D_0$ 向左移动 1 位的电路，并画出逻辑图。令电路的输出为 W 及 $Y_3Y_2Y_1Y_0$，当 $S=1$ 时，$W=D_3$，$Y_3=D_2$，$Y_2=D_1$，$Y_1=D_0$，$Y_0=0$。当 $S=0$ 时，$W=0$，$Y_3Y_2Y_1Y_0=D_3D_2D_1D_0$。

4.4.28 应用具有低电平使能控制的 8 选 1 数据选择器和表 4.4.5 描述的 3 线-8 线译码器模块设计一个数据传输电路，其功能是在 4 位通道选择信号的控制下，能将 16 个输入数据中的任何一个传送到 16 个输出端中相对应的一个输出端，其逻辑电路示意图如图题 4.4.28 所示。



图题 4.4.28

4.4.29 试用三个 3 输入端与门和一个或门及两个非门实现"$A>B$"的比较电路，A 和 B 均为两位二进制数。

4.4.30 试用五个 2 输入端或门、一个与门和两个非门实现"$A>B$"的比较电路，A 和 B 均为两位二进制数。

4.4.31 试设计一个八位相同数值比较器，当两数相等时，输出 $L=1$，否则 $L=0$。

4.4.32 试用 4 位数值比较器设计一个 8421BCD 码有效性测试电路，当输入为 8421BCD 码时，输出为 1，否则为 0。

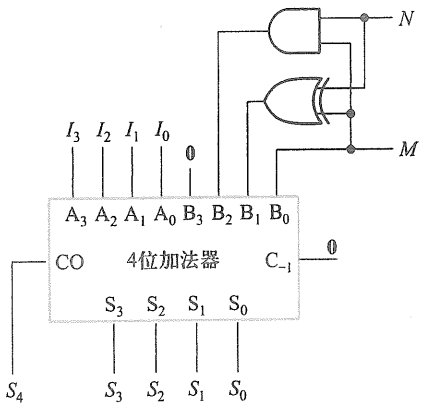
4.4.33 试用 4 位数值比较器和必要的逻辑门设计一个余 3 码有效性测试电路，当输入为余 3 码时，输出为 1，否则为 0。

4.4.34 试用反相器、与门和或门设计 1 位二进制全加器。

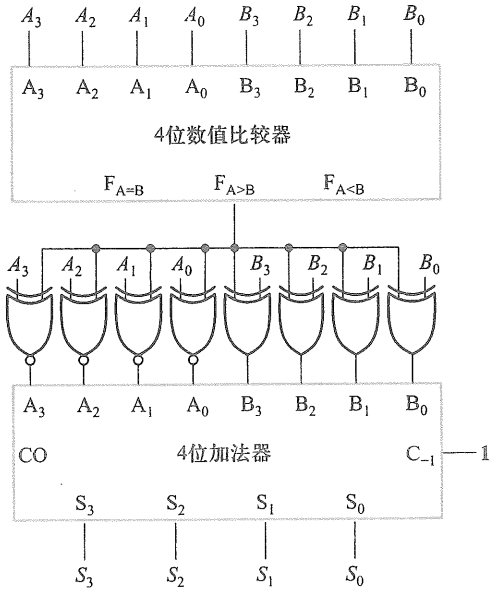
4.4.35 仿照半加器和全加器的设计方法，试设计一半减器和一全减器，所用的门电路由自己选定。

4.4.36 由 4 位加法器构成的逻辑电路如图题 4.4.36 所示，M 和 N 为控制端，试分析该电路的功能。

4.4.37 逻辑电路如图题 4.4.37 所示，试分析该电路的功能。



图题 4.4.36

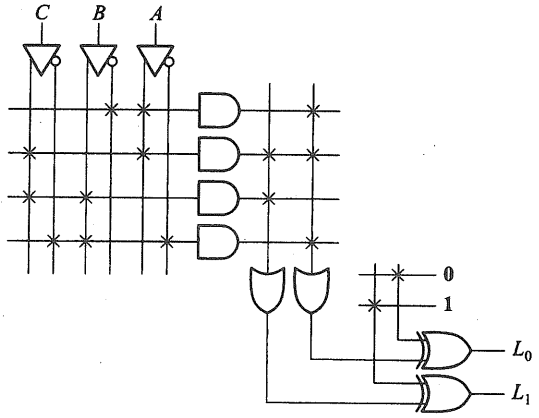


图题 4.4.37

4.4.38 试用图 4.4.39 所示的 4 位加法器构成一个 12 位二进制加法器，画出连接图。此加法器能否用来构成超前进位的级连方式，为什么？

4.5 组合逻辑的可编程电路实现

4.5.1 一个可编程逻辑阵列 PLA 电路如图题 4.5.1 所示。试写出输出逻辑函数表达式。



图题 4.5.1

4.5.2 试用可编程逻辑阵列 PLA 实现下列逻辑函数，并考虑尽量减少乘积项数目。

$$L_0(A,B,C) = \sum(0,1,2,4)$$

$$L_1(A,B,C) = \sum(0,5,6,7)$$

4.5.3 试用图 4.5.9 所示的可编程阵列逻辑 PAL，实现表题 4.5.3 所示真值表给出的逻辑关系。

表题 4.5.3

A	B	C	L_1	L_2	L_3	L_4
0	0	0	0	1	0	0
0	0	1	1	1	1	1
0	1	0	1	0	1	1
0	1	1	0	1	0	1
1	0	0	1	0	1	0
1	0	1	0	0	0	1
1	1	0	1	1	1	0
1	1	1	0	1	1	1

4.5.4 试用图 4.5.9 所示的可编程阵列逻辑 PAL，实现码转换电路，输入为 4 位 8421BCD 码，输出为余 3 码。

4.6 用 Verilog HDL 描述组合逻辑电路

4.6.1 根据下面的 Verilog 描述，画出数字电路的逻辑图，写出逻辑电路的输出表达式。

(1) **module** Circuit_A (**input** [1:0] A,B, **output** AgtB,AltB,AeqB);

nor(AgtB,AltB,AeqB);

or (AltB,w1,w2,w3);

and (AeqB,w4,w5),

(w1,w6,B[1]),

```
        (w2,w6,w7,B[0]),
        (w3,w7,B[1],B[0]);
    not (w6,A[1]),
        (w7,A[0]);
    xnor ( w4,A[1],B[1]);
    xnor ( w5,A[0],B[0]);
endmodule
```

(2)
```
module Circuit_B ( input  A,B,  output Y1,Y2);
    assign Y1 = A & B;
    or (Y2,A ,B);
endmodule
```

4.6.2 阅读下面的 Verilog 程序,说明模块 Comparator 完成的逻辑功能。

```
module Comparator #(parameter n)  (input [n-1:0] A,B,  output GT,EQ,LT);
    assign GT = (A > B);
    assign EQ = (A == B);
    assign LT = (A < B);
endmodule
```

4.6.3 阅读下面的 Verilog 程序,说明模块 ALU 完成的逻辑功能。

```
module ALU #(parameter n = 16)
  (input [n-1:0] A,B,input [1:0] F,  output reg [n-1:0] Z,output reg Cout );
  always @ ( * ) begin
    case (F)
        2'b00:   {Cout,Z} = A+B;
        2'b01:   {Cout,Z} = A-B;
        2'b10:   {Cout,Z} = A&B;
        default: {Cout,Z} = A|B;
    endcase
  end
endmodule
```

4.6.4 图 4.2.3 是一个码制变换器,将输入的格雷码转换成二进制码输出,试用 Verilog 数据流方式描述该码制变换器的功能。然后用仿真软件进行逻辑功能仿真,并给出仿真波形。

4.6.5 阅读下面的 Verilog 程序,说明模块 ATest 完成的功能,并用逻辑门画出逻辑电路图。

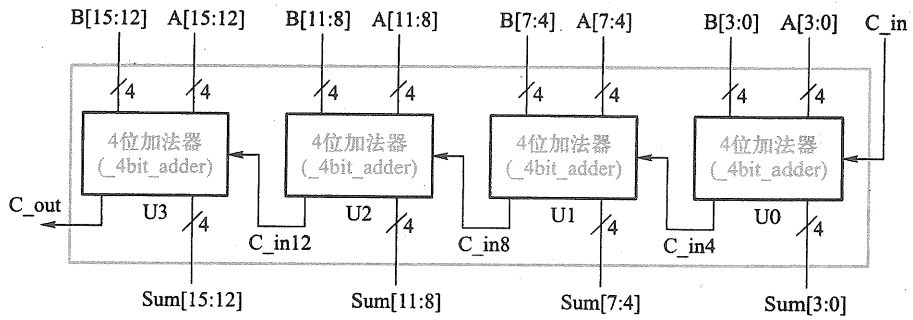
```
module ATest(output [3:0] Y,  input [3:0] A,B,C,D  input [1:0] S);
  assign Y[0] =(~S[1] & ~S[0] & A[0]) | (~S[1] & S[0] & A[1])
              | ( S[1] & ~S[0] & A[2]) | ( S[1] & S[0] & A[3]);
  assign Y[1] =(~S[1] & ~S[0] & B[0]) | (~S[1] & S[0] & B[1])
              | ( S[1] & ~S[0] & B[2]) | ( S[3] & S[2] & B[3]);
  assign Y[2] =(~S[1] & ~S[0] & C[0]) | (~S[1] & S[0] & C[1])
              | ( S[1] & ~S[0] & C[2]) | ( S[1] & S[0] & C[3]);
```

```
assign Y[3] = (~S[1] & ~S[0] & D[0]) | (~S[1] & S[0] & D[1])
            | (S[1] & ~S[0] & D[2]) | (S[3] & S[2] & D[3]);
endmodule
```

4.6.6 说明下列 Verilog 程序所描述电路的功能，并画出逻辑图。

```
module Circuit_A ( input [1:0] A,B, input S,E, output [1:0]Y );
    assign Y = E ? (S ? A : B) : 'bz ;
endmodule
```

4.6.7 一个 16 位的加法器的组成框图如图题 4.6.7 所示，它由 4 个 4 位的加法器级联而成，每个单元所产生的进位从最低位开始逐级传递至下一级的进位输入端。要求直接引用例 4.6.10 中的 4 位加法器模块_4bit_adder 对 16 位加法器的行为进行描述，然后用仿真软件对整个电路进行逻辑功能仿真，并给出仿真波形。



图题 4.6.7

4.6.8 根据图 4.4.5 所示的逻辑图，使用分模块、分层次设计方法，对 16 线-4 线优先编码器的行为进行描述。要求如下：

(1) 根据功能表 4.4.3 写出 8 线-3 线优先编码器行为级描述，并用仿真软件对该模块进行逻辑功能仿真，并给出仿真波形。

(2) 调用上面设计的编码器子模块和基本门级元件，完成 16 线-4 线优先编码器的建模。

(3) 用仿真软件对整个电路进行逻辑功能仿真，并给出仿真波形。

4.6.9 试根据图 4.4.30 和图 4.4.31 所示的数值比较器逻辑图，使用分模块、分层次设计方法，对两位数值比较器的行为进行描述。要求如下：

(1) 根据图 4.4.30 对 1 位数值比较器的行为进行描述，并用仿真软件对该模块进行逻辑功能仿真，并给出仿真波形。

(2) 根据图 4.4.31，调用上面设计的 1 位比较器模块和基本门级元件，完成两位数值比较器的建模。

(3) 用仿真软件对整个电路进行逻辑功能仿真，并给出仿真波形。

4.6.10 下面是用分层次方法设计的 4 位串行全加器程序。设计者首先完成了 1 位全加器(模块名为_1bitAdder)的建模和仿真，结果是正确的；然后在顶层调用 4 个 1 位全加器子模块组合成为 4 位全加器(模块名为_4bitAdder)，结果编译未能通过，试参照图 4.4.37 所示组成框图分析下列程序中存在的错误，并进行改正。

```
module _4bitAdder( A,B,Cin,Sum,Cout );
```

```
    input [3:0] A,B;
    input Cin;
    output [3:0] Sum;
    output Cout;
    reg Cout;
    reg [4:0] temp;
always @ (A or B or Cin)
  begin
    temp[0] = Cin;
    _1bitAdder u0(A[0],B[0],temp[0],Sum[0],temp[1]);
    _1bitAdder u1(A[1],B[1],temp[1],Sum[1],temp[2]);
    _1bitAdder u2(A[2],B[2],temp[2],Sum[2],temp[3]);
    _1bitAdder u3(A[3],B[3],temp[3],Sum[3],temp[4]);
    Cout = temp[4];
  end
endmodule

module _1bitAdder ( A,B,Ci,Sum,Co );  //此模块正确
    input A,B,Ci;
    output Sum,Co;
    assign Sum = A^B^Ci;
    assign Co = (A & B)|(B & Ci)|(A & Ci);
endmodule
```

文档 4.2：
第 4 章部分
习题答案

5 锁存器和触发器

引言

文档 5.1：第 5 章思维导图

大多数数字系统中，除了需要具有逻辑运算和算术运算功能的组合逻辑电路外，还需要具有存储功能的电路。组合电路与存储电路结合构成时序逻辑电路，简称时序电路。本章将讨论实现存储功能的两种逻辑单元电路：锁存器和触发器，着重讨论它们的工作原理与电路结构，以及所实现的不同逻辑功能。此外，本章还将讨论用 Verilog HDL 描述锁存器与触发器的方法。

5.1 基本双稳态电路

将两个非门 G_1 和 G_2 交叉连接，则构成最基本的双稳态电路，如图 5.1.1 所示。它有两个输出端 Q 和 $\overline{Q}$。

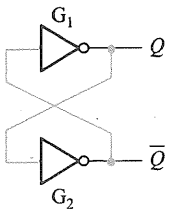


图 5.1.1 最基本的双稳态电路

分析其逻辑关系可知，若 $Q=0$，经非门 G_2 反相，则 $\overline{Q}=1$。$\overline{Q}$ 反馈到 G_1 输入端，又保证了 $Q=0$。因而电路能保持在 $Q=0$、$\overline{Q}=1$ 的状态不变，形成第一种稳定状态。反之，若 $Q=1$、$\overline{Q}=0$，则形成第二种稳定状态。在两种稳定状态中，输出端 Q 和 $\overline{Q}$ 总是逻辑互补的。

将 Q 的状态定义为电路的状态，$Q=0$ 时电路为 0 状态，$Q=1$ 时电路为 1 状态。其中任意一种逻辑状态都能长期保持下去。因此，该电路具有存储 1 位二进制数据的功能。

功能如图 5.1.1 所示电路，具有 0、1 两种逻辑状态，一旦进入其中一种状态，就能长期保持不变的单元电路，称为**双稳态存储电路**，简称**双稳态电路**。本章所讨论的锁存器和触发器均属于双稳态电路。

但图 5.1.1 所示双稳态电路没有输入端，因此无法改变或控制它的状态。在接通电源后，它随机进入 0 状态或 1 状态，并且始终保持这一状态，因而不能作为存储电路使用。但是，该电路是各种锁存器、触发器等存储单元的基础。

复习思考题

5.1.1 为什么图 5.1.1 所示的电路能长期保持状态不变？

5.2 SR锁存器

锁存器(Latch)是一种对脉冲电平敏感的双稳态电路,它具有0和1两个稳定状态,一旦状态被确定,就能自行保持,直到有外部输入脉冲电平作用,才有可能改变状态。这种特性可以用于存储1位二进制数据。本节首先讨论*SR*锁存器(set-reset latch)。

5.2.1 基本SR锁存器

1. 用或非门构成的基本SR锁存器

将图5.1.1中双稳态电路的非门换成或非门,则构成图5.2.1(a)所示的基本*SR*锁存器,它是一种具有最简单控制功能的双稳态电路。图中,S和R是两个输入端,Q和$\overline{Q}$是两个输出端。如果将Q的状态规定为锁存器的状态,则$Q=0$,$\overline{Q}=1$为锁存器的0状态或复位状态,$Q=1$,$\overline{Q}=0$则是锁存器的1状态或置位状态。S端称为直接置1端或置位端,R端称为直接置0端或复位端。下面根据S、R的4种输入状态组合来分析它的工作原理。

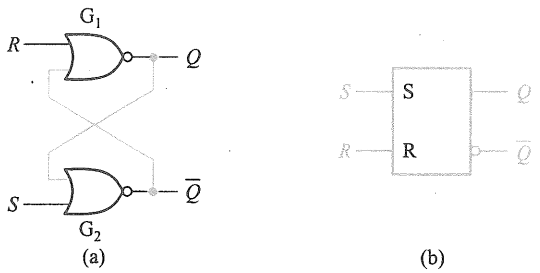


图5.2.1 用或非门构成基本*SR*锁存器

(a) 逻辑电路 (b) 逻辑符号

① $S=R=0$

或非门是对高电平敏感的,此时S、R均为低电平,对或非门不起作用,电路保持原来的状态不变。

② $S=0$、$R=1$

对于或非门而言,$S=0$不会影响G_2的输出状态。而$R=1$的高电平触发信号作用于G_1使得$Q=0$,即锁存器置0。该信号再反馈到G_2输入端,与$S=0$一起决定了$\overline{Q}=1$。$\overline{Q}=1$反馈到G_1的另一个输入端,此时即使$R=1$信号消失(即回到0),锁存器输出仍能保持0状态。

③ $S=1$、$R=0$

电路是对称的,$S=1$首先使$\overline{Q}=0$。它反馈到G_1输入端,与$R=0$一起决定了$Q=1$,锁存器置1。该信号再反馈回G_2输入端,使$\overline{Q}$保持0,即使$S=1$信号消失,同样可使锁存器的1状态得以保持。

教学视频 5.1：SR 锁存器的不确定状态

④ $S=R=1$

无论 Q 和$\overline{Q}$原来是什么状态，$S=R=1$ 将强制 $Q=\overline{Q}=0$，锁存器处在既非 1，又非 0 的非定义状态。由于 G_1、G_2 两门的延迟时间总会有微小差别①，若 G_1 的延迟时间稍短，在 S 和 R 同时跳变到 0 时，Q 端会先跳变为 1，迫使$\overline{Q}=0$；反之，若 G_2 延迟时间稍短，锁存器则进入 0 状态。所以，无法预知电路稳定在哪个状态。为保证锁存器始终工作于定义状态，输入信号应遵守 $SR=0$ 的约束条件，也就是说不允许 $S=R=1$。

由上述分析可得基本 SR 锁存器的功能表，如表 5.2.1 所示。

表 5.2.1 用或非门构成基本 SR 锁存器的功能表

S	R	Q	$\overline{Q}$	功能
0	0	不变	不变	保持
0	1	0	1	置 0
1	0	1	0	置 1
1	1	0	0	非定义状态

图 5.2.1(b)所示为基本 SR 锁存器的逻辑符号，S 和 R 分别为置位端和复位端，Q 和$\overline{Q}$为互补的两个输出端，其中$\overline{Q}$输出锁存器的非状态，所以用小圆圈示之。这样，仅从抽象的逻辑符号也可以理解基本 SR 锁存器各输入、输出信号之间的逻辑关系。

基本 SR 锁存器具有存储单元最基本的数据保持、置 0 和置 1 逻辑功能，因此，实际应用中可以作为存储单元。

基本 SR 锁存器的典型工作波形如图 5.2.2 所示。依照输入波形的变化划分时间段，在每个时间段中，由 S、R 的取值，对照功能表 5.2.1，就可以确定 Q 和$\overline{Q}$的状态。

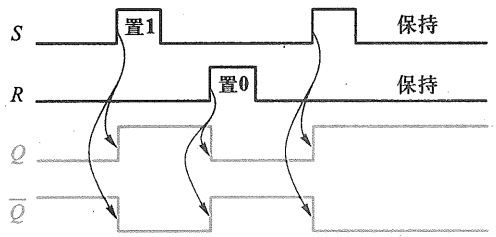


图 5.2.2 基本 SR 锁存器的典型工作波形

例 5.2.1 设图 5.2.1(a)所示基本 SR 锁存器的初始状态为 1，其 S、R 端输入波形如图 5.2.3 中虚线上面所示，试画出对应的 Q 和$\overline{Q}$波形。

解：画出 Q 和$\overline{Q}$端的波形如图 5.2.3 中虚线下面所示。

需要注意，虽然图 5.2.3 中①、②两处输入信号不满足 SR 锁存器的约束条件，出现 $S=$

① 如果两个或非门的延迟时间完全相等，当 S 和 R 同时回到 0，则锁存器在 $Q=\overline{Q}=0$ 和 $Q=\overline{Q}=1$ 之间反复振荡，并一直持续下去。

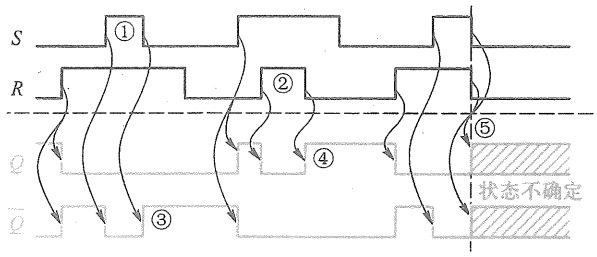


图 5.2.3 例 5.2.1 的波形

$R=1$ 使 $Q=\overline{Q}=0$ 的情况，但是，如果 S 和 R 的 1 电平没有同时撤销，此后的输出状态仍然是可以确定的，如图中③、④所示。而在⑤处，由于 S 和 R 的高电平同时撤销，即 S 和 R 同时回到 0，所以之后锁存器的状态将无法确定，从而失去对它的控制。在波形图中，这种不确定性用阴影表示。在实际应用中必须避免出现这种情况。

2. 由与非门构成的基本 SR 锁存器

基本 SR 锁存器也可以用与非门构成，其逻辑原理图和逻辑符号如图 5.2.4 所示。由于与非门输入端对低电平有效，因此，电路的两个输入端以低电平作为有效信号，并分别以 $\overline{S}$、$\overline{R}$ 表示置 1 输入端和置 0 输入端。图 5.2.4(a)中的两个与非门用其等效符号替代，输入端的小圆圈表示低电平有效，如图 5.2.4(b)所示。为了区别，有时也称这种锁存器为基本 $\overline{S}\,\overline{R}$ 锁存器，逻辑符号如图 5.2.4(c)所示。

由图可分析出当 $\overline{S}$、$\overline{R}$ 为不同输入状态组合时锁存器的状态，如表 5.2.2 所示。当输入为 $\overline{S}=\overline{R}=0$ 时，$Q=1$、$\overline{Q}=1$，该锁存器处于非定义状态，因此工作时应当受到 $\overline{S}+\overline{R}=1$ 的条件约束。

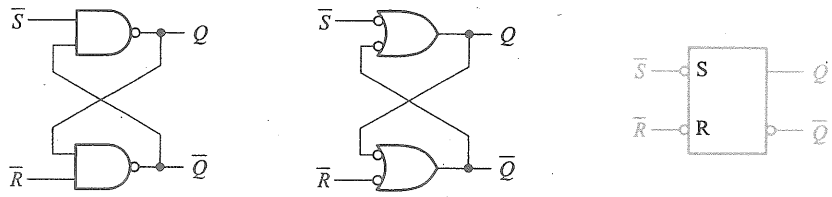


图 5.2.4 用与非门构成的基本 SR 锁存器

(a) 逻辑电路 (b) 等效符号逻辑电路 (c) 逻辑符号

表 5.2.2 用与非门构成基本 SR 锁存器的功能表

$\overline{S}$	$\overline{R}$	Q	$\overline{Q}$	功能
1	1	不变	不变	保持
1	0	0	1	置 0
0	1	1	0	置 1
0	0	1	1	非定义状态

3. 基本 *SR* 锁存器的应用

基本 *SR* 锁存器在数字电路或数字系统中都有应用。可以用于某些特定标志的设置，当某种预设逻辑条件具备时，电路可以通过 *S* 端将基本 *SR* 锁存器置 **1**，标志预设事件已经发生。例如，用锁存器的输出端 *Q* 控制报警器，当发生意外时传感器产生一个高电平，即 $S=1$ 使得 $Q=1$ 接通报警器，报警器发出报警信号。此后即使传感器恢复到 $S=0$ 状态，报警器仍然发出报警信号，直到手动置 $R=1$ 使 $Q=0$ 关闭报警器。

SR 锁存器的另一个典型应用实例是消除机械开关抖动引起的输出电压反复跳变。

数字电路或数字系统中，输入的高、低电平常常由机械开关电路（例如按键、拨动开关、继电器等）产生。由于机械开关接通或断开瞬间的弹性震颤，触点会在短时间内多次接通和断开，出现如图 5.2.5 所示的“抖动”现象，使 v_O 的逻辑电平在 **0** 和 **1** 之间反复跳变，导致输出错误逻辑电平。机械开关触点震颤的延续时间因开关结构、几何形状和尺寸以及材料的差异而不同，从数毫秒到上百毫秒不等。在设计数字系统时，可以采用硬件方法（如本例）或软件方法来消除开关电路的抖动。

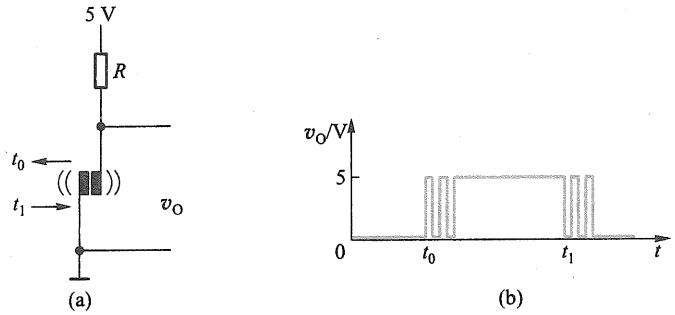


图 5.2.5 机械开关的“抖动”现象

（a）机械开关电路（在 t_0 时断开，t_1 时接通） （b）实际输出波形

消除机械开关抖动现象的一种硬件电路如图 5.2.6（a）所示。它利用基本 *SR* 锁存器的存储作用消除因开关触点振动引起的输出电压反复跳变现象，称为去抖动电路。图 5.2.6（b）中横虚线上面是图 5.2.6（a）所示电路中 $\overline{S}$ 和 $\overline{R}$ 端的波形，表示了单刀双掷开关 S 由 B 拨向 A，然后又拨回 B 的过程。

初始时，开关 S 的动触点与 B 点接通，锁存器的状态为 **0**。在开关 S 拨向 A，动触点脱离 B 点瞬间产生的抖动，并不影响锁存器的状态。在动触点悬空瞬间，$\overline{S}=\overline{R}=1$，*Q* 仍维持为 **0**。当它第一次触碰 A 点时，便使 $\overline{S}=0$，*Q* 端状态立即翻转为 **1**。此后，即使触点抖动，使 $\overline{S}$ 端再次出现高、低电平的跳变，也不会改变 $Q=1$ 的状态。由于电路是对称的，开关由 A 拨向 B 与前述的情况类似。于是得到 *Q* 端波形，如图 5.2.6（b）中横虚线下面所示。可以看到，在开关每次变化时，锁存器只翻转一次，不存在抖动波形。图 5.2.6（a）所示去抖动电路特别适用于需要对机械开关状态进行计数的场合，它可以避免开关触点抖动造成的误计数。

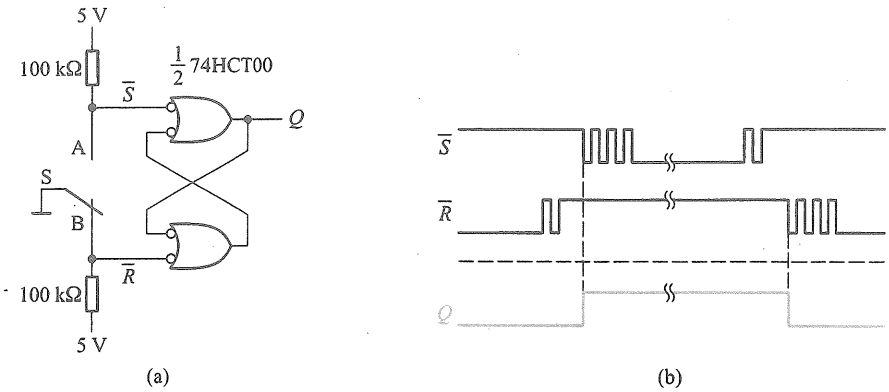


图 5.2.6 用基本 *SR* 锁存器构成机械开关去抖动电路

(a) 电路 (b) 波形图

5.2.2 门控 SR 锁存器

1. 门控 SR 锁存器的逻辑功能

前面所讨论的基本 *SR* 锁存器的输出状态是由输入信号 *S* 或 *R* 直接控制的，当它们之一有效时，输出状态立刻变化。但实际应用中，希望输入信号在规定的时刻起作用。这时可以增加一个控制端。当控制端有效时，*S* 或 *R* 才能影响输出状态。而当控制端无效时，即使 *S* 或 *R* 有效，输出状态也不改变。

在基本 *SR* 锁存器输入端增加一对与门和一个控制端，就构成了门控 *SR* 锁存器，如图 5.2.7(a)所示，其中 *E* 为控制端或使能端。

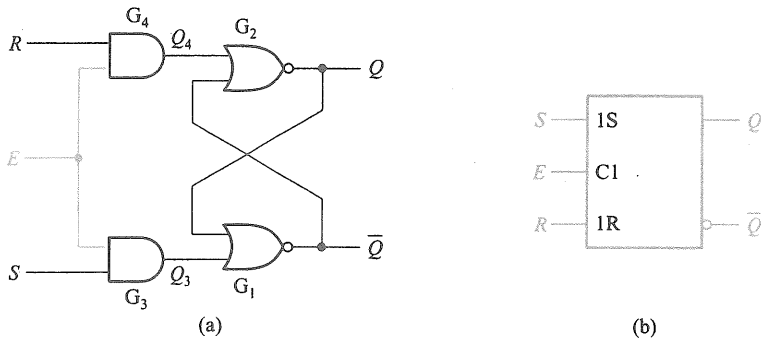


图 5.2.7 门控 *SR* 锁存器

(a) 逻辑电路 (b) 逻辑符号

从图 5.2.7(a)可以看出，当 $E=0$ 时，$Q_3=Q_4=0$，*S*、*R* 端的逻辑状态不会影响到锁存器的状态。当 $E=1$ 时，*S*、*R* 端的信号被传送到基本 *SR* 锁存器的输入端，从而可确定 *Q* 和 $\overline{Q}$ 端的状态，其功能如表 5.2.3 所示。若 $E=1$ 时输入信号 $S=R=1$，则 $Q=\overline{Q}=0$，锁存器将处于非定义的逻辑状态。当 *E* 恢复为 0 时，由于 Q_3、Q_4 同时回到 0，将不能确定锁存器的状态。因此，应用这种锁存器必须更严格地遵守 $SR=0$ 的约束条件。由于约束条件的限制，很少单独

使用门控 SR 锁存器。但是，在许多中、大规模集成电路中，常用这种锁存器构成触发器或存储器。所以，SR 锁存器仍是重要的基本逻辑单元。

表 5.2.3 门控 SR 锁存器的功能表

E	S	R	Q	$\overline{Q}$	功能
0	×	×	不变	不变	保持
1	0	0	不变	不变	保持
1	0	1	0	1	置 0
1	1	0	1	0	置 1
1	1	1	0	0	非定义状态

图 5.2.7(b)所示是门控 SR 锁存器的逻辑符号。其方框内用 C1 和 1R、1S 表达内部逻辑之间的关联关系。C 表示控制信号，其后缀标识序号“1”表示它对所有以“1”为前缀的输入信号起控制作用，即这里的 1R、1S 均受 C1 的控制。两个输出端 Q 和 $\overline{Q}$ 与图 5.2.1(b)所示基本 SR 锁存器的含义相同。

例 5.2.2 图 5.2.7(a)所示门控 SR 锁存器的 E、S、R 的波形如图 5.2.8 中横虚线上面所示，设锁存器的初始状态为 $Q=0$，$\overline{Q}=1$，试画出 Q_3、Q_4、Q 和 $\overline{Q}$ 的波形。

解： 从图 5.2.7(a)的逻辑电路图得 $Q_3=SE$，$Q_4=RE$。于是，可根据 E、S 和 R 的波形画出 Q_3 和 Q_4 的波形。图 5.2.7(a)中 G_1、G_2 构成基本 SR 锁存器，再根据表 5.2.1 即可画出 Q 和 $\overline{Q}$ 的波形。全部波形如图 5.2.8 所示。

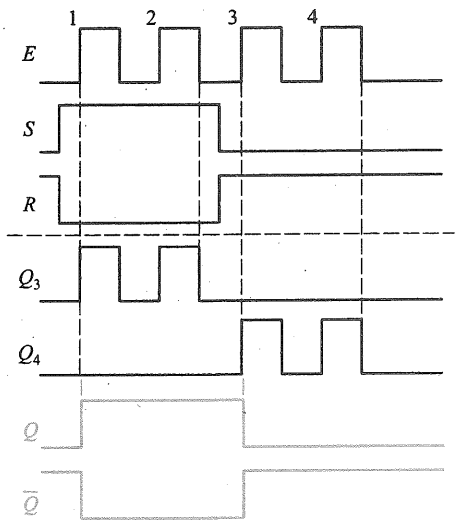


图 5.2.8 例 5.2.3 的波形图

如果只要求画出 Q 和 $\overline{Q}$ 的波形，可以根据输入 E、S、R 的波形，直接查表 5.2.3 判断输出状态，画出 Q 和 $\overline{Q}$。

2. 用与非门构成的门控 SR 锁存器

前面介绍了在或非门交叉耦合的基础上实现门控 SR 锁存器，也可以用**与非门**构成门控 SR 锁存器，电路如图 5.2.9 所示。其功能表与表 5.2.3 的前面 4 行相同，最后一行的 Q_1 和 $\overline{Q}$ 均为 **1**。由于该电路中增加的控制门是**与非门**，所以输入 S、R 与图 5.2.7(a)的位置相反。

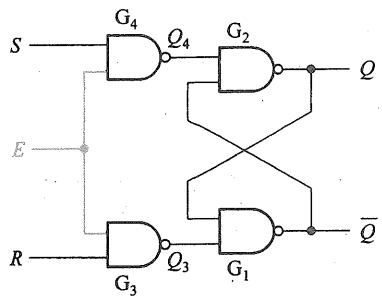


图 5.2.9 与非门构成门控 SR 锁存器

复习思考题

5.2.1 图 5.2.1(a)所示基本 *SR* 锁存器和图 5.2.7(a)所示门控 *SR* 锁存器在电路结构、数据锁存动作上有什么区别？为什么它们在工作中均需遵循 $SR=0$ 的约束条件？

5.2.2 门控 *SR* 锁存器电路在使能信号 $E=1$ 期间，如果输入信号 S、R 发生多次变化，其输出会怎样？

5.3 D锁存器

5.3.1 D锁存器的电路结构

与 *SR* 锁存器不同，*D* 锁存器在工作中不存在非定义状态，因而得到广泛应用。目前，CMOS 集成电路主要采用传输门控 *D* 锁存器和逻辑门控 *D* 锁存器两种电路结构形式，特别是前者电路结构简单、在芯片中占用面积小而更受青睐。

1. 传输门控 D 锁存器

在图 5.1.1 的双稳态电路中插入两个传输门 TG_1 和 TG_2，则可构成如图 5.3.1(a)所示的传输门控 *D* 锁存器，图 5.3.1(b)所示是它的逻辑符号。

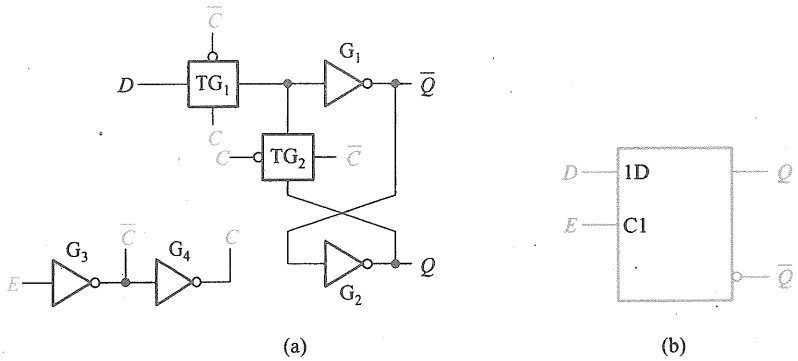


图 5.3.1 传输门控 *D* 锁存器
(a) 逻辑电路 (b) 逻辑符号

D 锁存器有两个输入端：使能端 *E* 和数据输入端 *D*。

当 $E=1$ 时，$\overline{C}=0$，$C=1$，TG_1 导通，TG_2 断开，如图 5.3.2(a)所示。输入数据 *D* 经 G_1、G_2 两个非门，使 $Q=D$，$\overline{Q}=\overline{D}$。显然，这时 *Q* 端跟随输入信号 *D* 的变化。

当 $E=0$ 时，$\overline{C}=1$，$C=0$，TG_1 断开，TG_2 导通，如图 5.3.2(b)所示，其原理与图 5.1.1 所示双稳态电路相同。由于 G_1、G_2 输入端存在的分布电容对逻辑电平有短暂的保持作用，所以在两个传输门状态转换瞬间并不影响电路的输出状态。之后，电路将被锁定在 *E* 由 1 变 0 前瞬间 *D* 的状态，在 $E=0$ 的条件下可保持锁存器状态不变，使 1 位二进制数据得以存储。表 5.3.1 概括了 *D* 锁存器的功能。由于这种锁存器在 $E=1$ 时 *Q* 端可跟随 *D* 端的逻辑状态变化，故又称为透明锁存器。

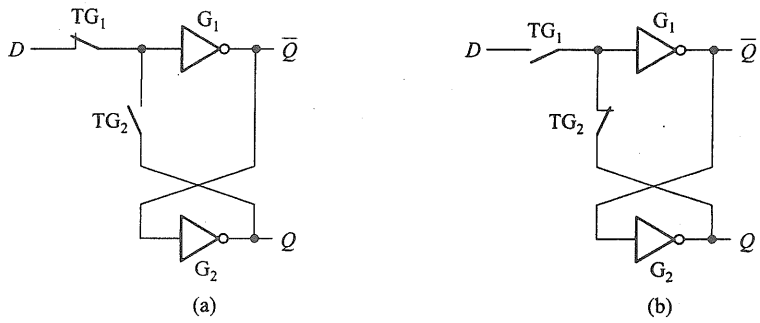


图 5.3.2 传输门的开关状态

(a) $E=1$ 时的状态 (b) $E=0$ 时的状态

表 5.3.1 D锁存器的功能表

E	D	Q	$\overline{Q}$	功能
0	×	不变	不变	保持
1	0	0	1	置0
1	1	1	0	置1

2. 逻辑门控D锁存器

图 5.3.3 所示为逻辑门控 D 锁存器的逻辑电路，它在门控 SR 锁存器的 S 和 R 输入端之间连接了一个非门 G_5，从而保证了 $SR=0$ 的约束条件，消除了可能出现的非定义状态。读者可仿照前述方法自行分析，并用表 5.3.1 来验证图 5.3.3 所示电路的逻辑功能。由于它的逻辑功能与传输门控 D 锁存器完全相同，所以逻辑符号亦相同。

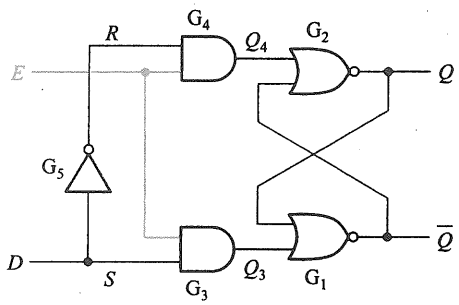


图 5.3.3 逻辑门控 D 锁存器的逻辑电路

例 5.3.1 设图 5.3.1(a)所示电路的初始状态为 $Q=0$，输入信号 D、E 的波形如图 5.3.4 中横虚线上面所示，画出 Q 和 $\overline{Q}$ 的输出波形。

解：根据表 5.3.1 或图 5.3.2(a)、(b)可知，当 $E=1$ 时，Q 端波形跟随 D 端变化，当 E 跳变为 0 时，锁存器保持此前瞬间 D 的状态，可以画出 Q 和 $\overline{Q}$ 波形如图 5.3.4 中横虚线下面所示。由此可以看出，门控 D 锁存器的输出受到输入高、低电平的控制，因此，锁存器是电平敏感型的。

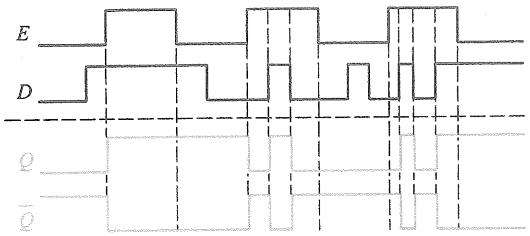


图 5.3.4 例 5.3.1 的波形图

5.3.2 D锁存器集成电路

集成 CMOS 的 D 锁存器有多种，图 5.3.5 为中规模集成的 CMOS 八 D 锁存器 74HC/HCT373 的内部逻辑图，其核心电路是 8 个如图 5.3.1(a)所示的传输门控 D 锁存器。8 个锁存器共用同一对互补的传输门控制信号 C 和 $\overline{C}$，这对信号又由锁存使能信号 LE 驱动。当 LE 为高电平时，允许各 D 锁存器的输出跟随相应输入信号的变化；LE 为低电平时则保持状态不变。8 个 D 锁存器输出端都带有三态门，当输出三态门使能信号 $\overline{OE}$ 为低电平时，三态门有效，输出锁存的信号；当 $\overline{OE}$ 为高电平时，输出处于高阻状态。这种三态输出电路，不仅提高了对负载的驱动能力，而且对锁存器与输出负载起到隔离作用，避免因负载变化而影响锁存器的动态特性；更重要的是使 74HC/HCT373 可以方便地应用于微处理器或计算机的总线传输电路。

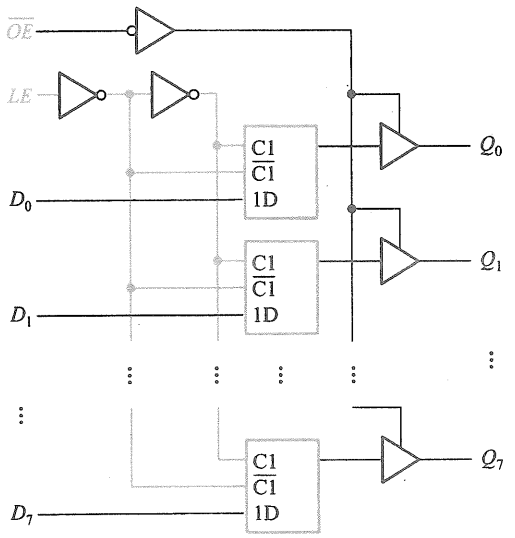


图 5.3.5 74HC/HCT373 八 D 锁存器的内部逻辑图

根据 LE 和 $\overline{OE}$ 的不同逻辑电平，74HC/HCT373 可分为三种工作模式：① 使能和读锁存器(传送模式)；② 锁存和读锁存器；③ 禁止输出。表 5.3.2 所示为其功能表。

表 5.3.2 74HC/HCT373 的功能表

工作模式	输入			内部锁存器状态	输出
	$\overline{OE}$	LE	D_N		Q_N
使能和读锁存器（传送模式）	L	H	L	L	L
	L	H	H	H	H
锁存和读锁存器	L	L	L*	L	L
	L	L	H*	H	H
禁止输出	H	×	×	×	高阻

注：D_N和 Q_N的下标表示第 N 位锁存器数据。L* 和 H* 表示使能信号 LE 的电平由高变低之前瞬间 D_N的电平。

除此之外，还有 16 位、20 位、32 位等“宽总线”D 锁存器集成电路产品，以适用于不同总线宽度的计算机系统。例如锁存器 74LVC32373A 芯片中，集成了 32 个三态输出的 D 锁存器，它们分为 4 组，每一组的逻辑功能都相当于一个 74HC373，各组间既可同步工作，又可独立运行，异步工作。这种结构给芯片的总线应用带来很大的灵活性。

另外，“小逻辑”锁存器 74LVC1G373 的内部只封装了一路 D 锁存器，其逻辑功能只相当于 74HC373 中的一个通道。

5.3.3 D 锁存器的动态特性

图 5.3.6 所示是 D 锁存器的时序图，对于传输门控和逻辑门控两种电路结构的 D 锁存器都是适用的，只是具体参数值有所差异。下面对各参数进行说明。

传输延迟时间 t_{pd}

t_{pd}是输出信号对输入信号的响应延迟时间，对于 D 锁存器则是指 D 信号和 E 信号共同作用后，Q（或 $\overline{Q}$）端响应的延迟时间。图 5.3.6 中所示 $t_{pLH(D-Q)}$是输出 Q 从低电平到高电平对 D 信号的延迟时间，$t_{pHL(E-Q)}$则是 Q 从高电平到低电平对 E 信号的延迟时间。根据不同的输入状态，还存在图中没有显示的 $t_{pHL(D-Q)}$ 和 $t_{pLH(E-Q)}$。对于 CMOS 集成电路，因为输出信号对各输入信号的延迟相差不多，有时统一以 t_{pHL} 和 t_{pLH}表达，更经常的是取平均传输延迟时间：$t_{pd}=(t_{pLH}+t_{pHL})/2$。

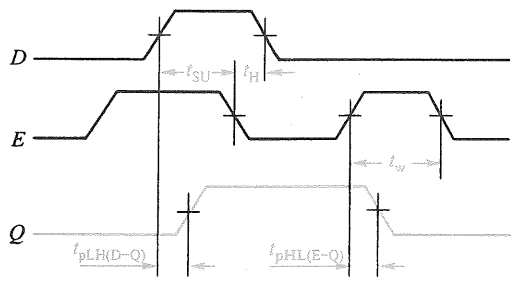


图 5.3.6 D 锁存器的时序图

建立时间 t_{SU}

信号 D 的逻辑电平必须在使能信号 E 的有效电平结束之前建立起来，才能保证正确地锁存。t_{SU}表示在 E 的有效电平结束前 D 信号保持稳定的最短时间。

保持时间 t_H

在 E 的有效电平结束后，D 信号不允许立即撤除，否则不能确保数据的锁存。t_H表示在

t_{SU} 的有效电平结束后 D 信号电平需要继续保持的最短时间。

脉冲宽度 t_w。

为保证 D 信号正确传送到 Q 和 $\overline{Q}$，要求 E 信号的脉冲宽度不小于 t_w。

上述 t_{SU}、t_H 和 t_w 是对输入信号的时间要求。如果电路运行中达不到要求，则会分别出现如图 5.3.7 所示的情况，可能导致 D 锁存器不确定的逻辑输出。

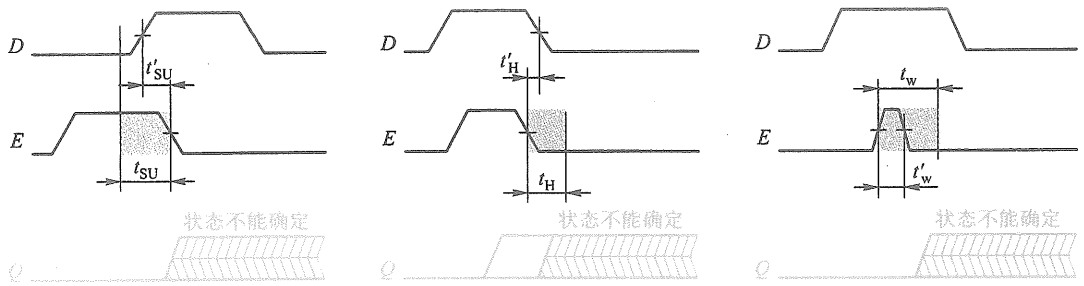


图 5.3.7 t_{SU}、t_H 和 t_w 达不到要求时可能导致 D 锁存器不确定的逻辑输出

(a) $t'_{SU}<t_{SU}$ (b) $t'_H<t_H$ (c) $t'_w<t_w$

在制造商提供的数据手册中，都会给出这些时间的明确限度，表 5.3.3 示出环境温度在 −40 ℃ ~ +85 ℃ 范围的几种典型的 D 锁存器定时参数。

表 5.3.3 几种典型的 D 锁存器定时参数

型号	功能	工作电压 V_{CC}/V	t_{pd}/ns *				t_{SU}/ns	t_H/ns	t_w/ns
			D-Q		LE-Q		最小	最小	最小
			最小	最大	最小	最大			
74HCT373	八 D 锁存器	4.5		38		44	13	12	20
74LVC32373A	三十二 D 锁存器	3.3±0.3	1.6	4.2	2.1	4.6	1.7	1.2	3.3
74LVC1G373	单 D 锁存器	3.3±0.3	1	5.4	1	5.5	1.5	1.5	3

* 测试条件：Q 端负载电容 C_L = 50 pF。

在设计工作中，对电路的动态特性必须予以充分重视，特别是电路工作在接近定时极限的高频条件下，对上述时间关系更要注意留有充分余地。否则，电路在长期工作时会发生原因难以查明的偶发性逻辑错误，或因环境条件改变（如温度变化）而出现工作不稳定的情况。

复习思考题

5.3.1 传输门控和逻辑门控 D 锁存器在电路结构和工作原理上有何不同？

5.3.2 D 锁存器工作时对使能信号 E 和输入信号 D 在定时上有什么要求？输出信号 Q 的变化对输入信号 D、使能信号 E 有何定时特性？

5.4 D触发器

本节讨论另一种对脉冲边沿敏感的双稳态电路。如前所述，*D* 锁存器在使能信号 *E* 为逻辑 1 期间更新状态，在图 5.4.1(a)所示的波形图中以加粗部分表示这个敏感时段。图 5.4.1(b)所示为低电平敏感，在低电平期间，它的输出会随输入信号变化。而很多时序电路要求存储电路只对时钟信号的上升沿或下降沿敏感，而在其他时刻保持状态不变，例如 6.5 节中将要讨论的移位寄存器和计数器。这种对时钟脉冲边沿敏感的状态更新称为触发，具有触发工作特性的存储单元称为触发器。电路结构不同的触发器对时钟脉冲敏感的边沿可能不同，分为上升沿触发和下降沿触发。本书以 *CP*(clock pulse)命名上升沿触发的时钟信号，触发边沿如图 5.4.1(c)波形中的箭头所示；以 $\overline{CP}$ 命名下降沿触发的时钟信号，触发边沿如图 5.4.1(d)中箭头所示。

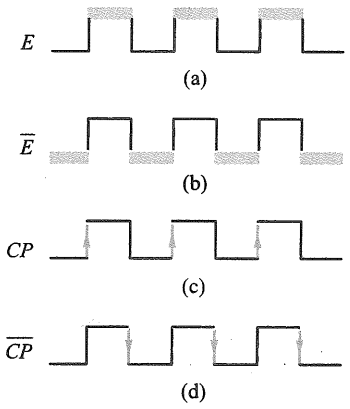


图 5.4.1 锁存器对使能信号的响应和触发器对时钟信号的响应

(a) 对脉冲高电平敏感 (b) 对脉冲低电平敏感 (c) 对脉冲上升沿敏感 (d) 对脉冲下降沿敏感

在 Verilog HDL 中，对脉冲电平敏感的锁存器和脉冲边沿敏感的触发器的描述语句是不同的，这一点将在 5.6 节中说明。正因为如此，这里要特别强调锁存器与触发器在概念上的差异。

目前应用的触发器电路结构主要有：主从触发器、维持阻塞触发器和利用传输延迟的触发器。CMOS 主从结构的 *D* 触发器在芯片上占用的面积最小，逻辑设计方法也较简单，在大规模 CMOS 集成电路，特别是可编程逻辑器件(如 CPLD、FPGA)和专用集成电路(ASIC)中得到普遍应用，因而在目前的工程实践中也会更多地面对这种 *D* 触发器。下面重点讨论 CMOS 主从 *D* 触发器。

5.4.1 主从D触发器

将两个如图 5.3.1(a)所示 *D* 锁存器级联，便构成典型的 CMOS 主从 *D* 触发器，如图 5.4.2(a)所示。图中左边的锁存器称为主锁存器，右边的称为从锁存器。主锁存器与从锁存器的使能信号相位相反，利用两个锁存器的交互锁存，可实现存储数据和输入信号之间的隔离。

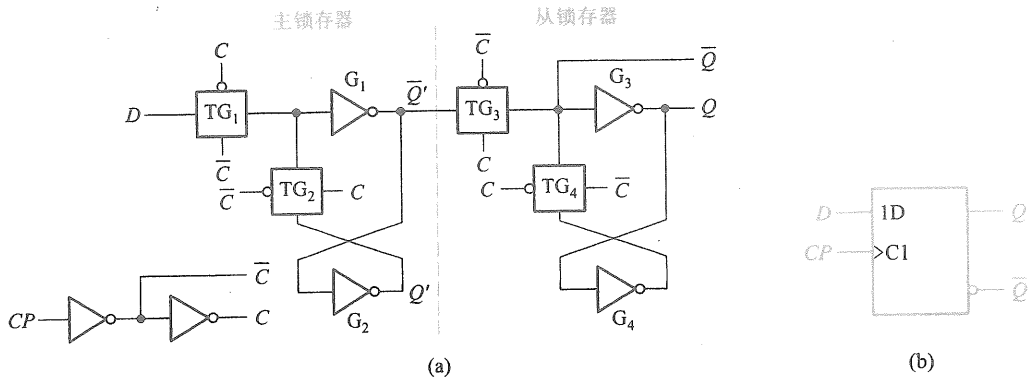


图 5.4.2 CMOS 主从 D 触发器

(a) 逻辑电路 (b) 逻辑符号

主从 D 触发器工作过程分为以下两个节拍。

① 当时钟信号 $CP=0$ 时，$\overline{C}=1$，$C=0$，使 TG_1 导通，TG_2 断开，D 端输入信号进入主锁存器，这时 Q' 跟随 D 端的状态变化，使 $Q'=D$。例如，D 为 1 时，经 TG_1 传到 G_1 的输入端，使 $\overline{Q'}=0$，$Q'=1$。同时由于 TG_3 断开，切断了从锁存器与主锁存器之间的联系，而 TG_4 导通，使 G_3 的输入端和 G_4 的输出端经 TG_4 连通，构成最基本的双稳态电路，使从锁存器维持原来的状态，即触发器的输出状态不变。

② 当 CP 由 0 跳变到 1 后，$\overline{C}=0$，$C=1$，使 TG_1 断开，从而切断了 D 端与主锁存器的联系，同时 TG_2 导通，将 G_1 的输入端和 G_2 的输出端连通，主锁存器锁存 CP 跳变前 D 端的数据。这时，TG_3 导通，TG_4 断开，$\overline{Q}'$ 端信号传送到 Q 端。若 $Q'=1$，$\overline{Q}'=0$，经 TG_3 传送给 G_3 的输入端，于是 $\overline{Q}=0$，$Q=1$。

可见，从锁存器在工作中是跟随主锁存器的状态变化的，触发器因之冠名主从(master-slave)。它的状态转换发生在 CP 信号上升沿到来后的瞬间，输出状态由 CP 信号上升沿到达前瞬间的数据信号 D 所决定，从功能上考虑称为 D 触发器。如果以 Q^{n+1} 表示 CP 信号上升沿到达后触发器的状态，则 D 触发器的特性可以用下式来表达

$$Q^{n+1}=D \tag{5.4.1}$$

该式称为 D 触发器的特性方程。它反映了触发器在时钟信号作用后的状态与此前输入信号 D 的关系。

上升沿触发的 D 触发器逻辑符号如图 5.4.2(b)所示，方框内侧的“>”符号表示电路对 CP 信号的脉冲边沿敏感。如果是下降沿触发，则在 CP 输入端加一个小圆圈表示。

小逻辑系列的 D 触发器 74LVC1G79 的内部逻辑图就是在图 5.4.2 基础上，在 Q 端增加一个缓冲器构成的，其逻辑已达到最简。

5.4.2 具有清零和置数输入的主从 D 触发器

触发器在接通电源后，会随机进入 0 状态或 1 状态。在实际应用中，需要对触发器设置初始状态，进行清零或置数(也称预置)，因此，增加了清零和置数输入端。清零或置数有同

步和异步之分。异步是指当清零或置数端的信号有效时，可以立即将触发器置 0 或置 1，而与 CP 信号和 D 输入信号无关，又称为直接清零或直接置数。同步是指清零或置数端的信号在 CP 有效沿到来时才起作用。

教学视频 5.2：主从 D 触发器的直接置 1 与直接置 0 分析

为了在图 5.4.2 所示电路中引入直接置 0 端 $\overline{R}_D$ 和直接置 1 端 $\overline{S}_D$，将 4 个非门换成或非门，低电平有效的 $\overline{R}_D$ 或 $\overline{S}_D$ 经非门缓冲后，分别送入主锁存器和从锁存器，如图 5.4.3(a)所示。

当 $CP=1$ 时，TG_1、TG_4 断开而 TG_2、TG_3 导通，或非门 G_1 和 G_2 构成基本 SR 主锁存器，$\overline{R}_D$ 或 $\overline{S}_D$ 信号先将主锁存器的输出 Q' 和 $\overline{Q'}$ 置 0 或置 1，进而传送到 Q 和 $\overline{Q}$ 端。

当 $CP=0$ 时，TG_3 断开、TG_4 导通，或非门 G_3 和 G_4 构成基本 SR 从锁存器。$\overline{R}_D$ 或 $\overline{S}_D$ 信号直接将从锁存器的输出置 0 或置 1。

所以，无论 $CP=1$ 或 $CP=0$，$\overline{R}_D$ 或 $\overline{S}_D$ 信号可以直接将触发器置 0 或置 1，显然，$\overline{R}_D$ 和 $\overline{S}_D$ 对触发器的状态有优先控制权。只有当 $\overline{S}_D=\overline{R}_D=1$ 时，触发器才能被 CP 上升沿触发，按 D 端逻辑值更新状态。

具有清零和置数输入的 D 触发器逻辑符号如图 5.4.3(b)所示，通过在输入端 $\overline{R}_D$ 或 $\overline{S}_D$ 增加一个小圆圈表示低电平有效。

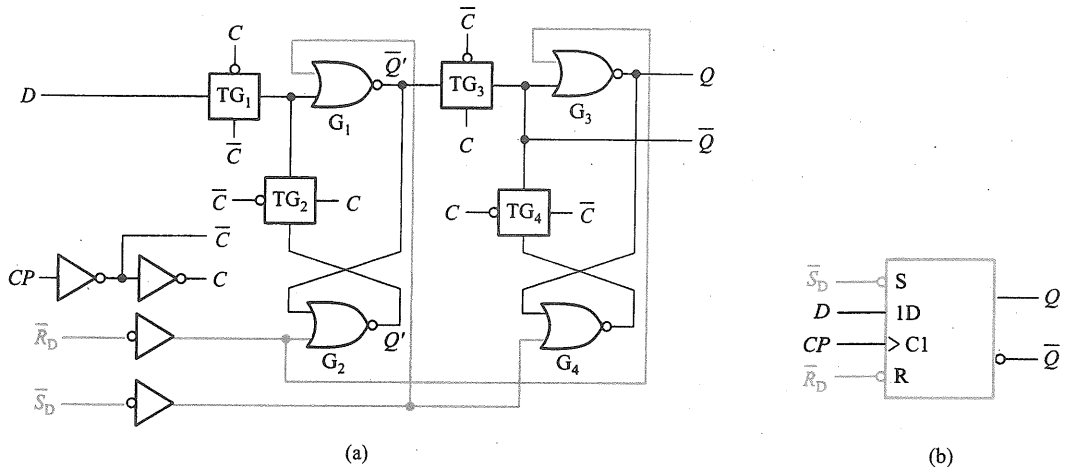


图 5.4.3 具有清零和预置输入的 D 触发器逻辑图

(a) 逻辑电路 (b) 逻辑符号

D 触发器的功能表如表 5.4.1 所示。表的上半部分是 $\overline{S}_D$、$\overline{R}_D$ 与输出信号的关系。其中，当 $\overline{S}_D$ 和 $\overline{R}_D$ 均为低电平时，输出 Q 和 $\overline{Q}$ 均为高电平，若 $\overline{S}_D$、$\overline{R}_D$ 同时恢复高电平，则不能确定触发器此后的状态，因而仍然需要遵守 $\overline{S}_D+\overline{R}_D=1$ 的约束条件。表 5.4.1 的下半部分是在 $\overline{S}_D$、$\overline{R}_D$ 均为高电平时的动态功能表。符号“↑”表示 CP 脉冲上升沿触发，Q^{n+1} 和 $\overline{Q^{n+1}}$ 分别为 CP 脉冲上升沿到达后 Q 和 $\overline{Q}$ 端的状态。

CMOS 集成 D 触发器 74HC/HCT74，是在图 5.4.3 所示电路的输入和输出端增加了缓冲电路，以提高电路的稳定性和驱动能力，并且将两个 D 触发器集成在一个芯片中，其逻辑符

号如图 5.4.4 所示。两个 D 触发器之间相互独立，C1 和 C2 分别关联控制着 1D 和 2D。有些制造商的集成 D 触发器是用与非门取代图 5.4.3(a)中的或非门构成。

表 5.4.1 D触发器的功能表

输入				输出	
$\overline{S}_D$	$\overline{R}_D$	CP	D	Q	$\overline{Q}$
L	H	×	×	H	L
H	L	×	×	L	H
L	L	×	×	H	H
$\overline{S}_D$	$\overline{R}_D$	CP	D	Q^{n+1}	$\overline{Q^{n+1}}$
H	H	↑	L*	L	H
H	H	↑	H*	H	L

L* 和 H* 表示 CP 脉冲上升沿到来之前瞬间的电平。

例 5.4.1　由 D 触发器和数据选择器组成的电路如图 5.4.5(a)所示。若 CP、$\overline{CR}$、$\overline{LD}$和 A 端的波形如图 5.4.5(b)横虚线上面所示。试画出 Q 的波形，并说明电路的功能。设触发器的初始状态为 $Q=0$。

解：电路中触发器的状态更新是在 CP 上升沿到达时刻，由 D 端的状态确定。根据数据选择器$\overline{CR}$、$\overline{LD}$的不同取值，将数据输入端的 0、1 或 A 的值送到 D 端。

在 CP 上升沿到达时，若$\overline{CR}=\overline{LD}=1$，$D=A$，则 $Q=A$。若$\overline{CR}=0$时，无论$\overline{LD}$为 0 或 1，$D=0$，则 $Q=0$。若$\overline{CR}=1$，$\overline{LD}$为 0 时，$D=1$，则 $Q=1$。由此画出如图 5.4.5(a)中 Q 端的波形。

由上述分析可知，$\overline{CR}$ 具有清零功能，$\overline{LD}$ 具有置数功能。清零$\overline{CR}$和置数$\overline{LD}$信号的作用受信号 CP 控制，并且是低电平有效。因此，电路具有低电平同步清零和同步置数的 D 触发器功能，并且同步清零的优先级别最高，功能如表 5.4.2 所示。

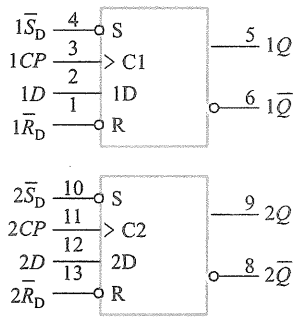
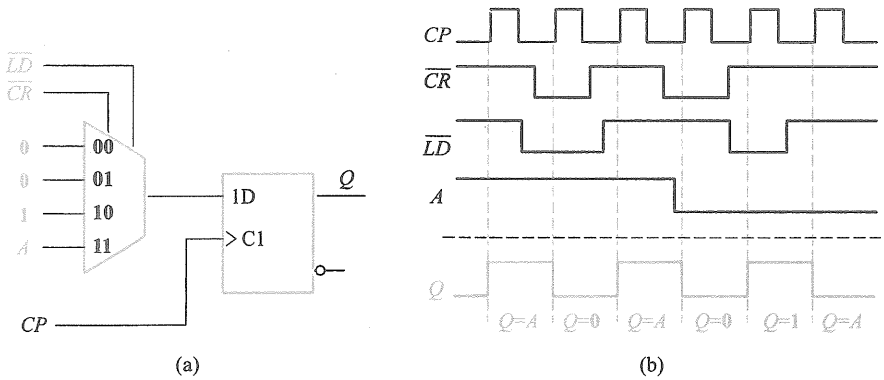


图 5.4.4　74HC/HCT74 的逻辑符号



(a)　(b)

图 5.4.5　例 5.4.1 的电路图及波形图

(a) 逻辑电路　(b) 波形图

表 5.4.2 例 5.4.1 电路的功能表

CP	$\overline{CR}$	$\overline{LD}$	A	Q	功能
↑	0	×	×	0	同步清零
↑	1	0	×	1	同步置数
↑	1	1	0	0	$Q=A$
↑	1	1	1	1	$Q=A$

5.4.3 具有使能控制的主从 D 触发器

在大规模集成电路中，许多触发器的 CP 接在同一个时钟源上，它们的状态更新是同步的，但有时希望时钟有效沿到来时，其中一些触发器的状态保持不变，因此需要增加一个使能控制输入 EN。

若将 EN 和时钟源信号经过与门接入触发器的 CP，便可通过 EN 控制触发器是否被触发。但由于与门存在延迟，将导致与那些没有使能控制的触发器的状态更新不同步。因此，实际是通过数据选择器将使能端加到触发器的 D 输入端，如图 5.4.6 所示。

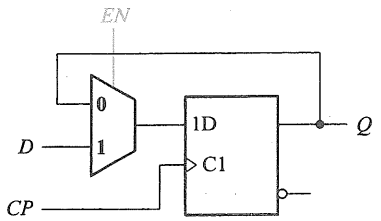


图 5.4.6 具有使能控制输入的 D 触发器

当 $EN=0$ 时，触发器处于保持状态。当 $EN=1$ 时，触发器输出等于 D。

5.4.4 其他结构的 D 触发器

除了常见的主从结构的触发器，还有维持阻塞触发器和利用门电路传输延迟的触发器。它们也可以完成与主从触发器相同的功能。这里主要介绍维持阻塞触发器。

维持阻塞结构的 D 触发器逻辑电路如图 5.4.7 所示。该触发器由 6 个与非门构成，其中，G_1、G_2、G_3 和 G_4 响应外部输入信号 D 和时钟信号 CP，所产生的 $\overline{S}$ 和 $\overline{R}$ 信号控制由 G_5、G_6 构成的基本 $\overline{S}\,\overline{R}$ 锁存器的状态，也就是整个触发器的状态。下面分析其工作原理。

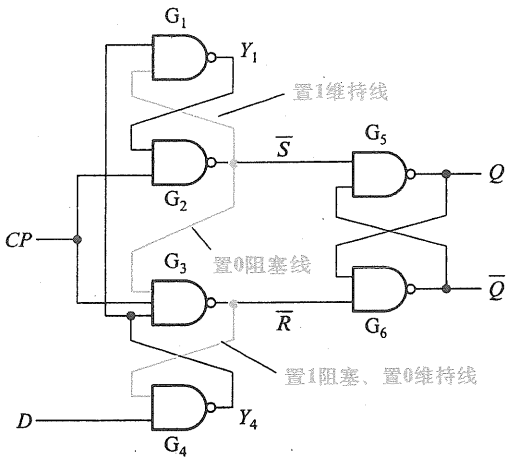


图 5.4.7 维持阻塞 D 触发器的逻辑电路

① 当 $CP=0$ 时，与非门 G_2 和 G_3 被低电平封锁，它们的输出 $\overline{S}=\overline{R}=1$，使 G_5、G_6 组成的输出锁存器处于保持状态，触发器的输出 Q 和 $\overline{Q}$ 不变。同时，$\overline{S}$ 和 $\overline{R}$ 的高电平分别反馈到 G_1 和 G_4 输入端，将这两个门打开，使 $Y_4=\overline{D}$，$Y_1=\overline{Y}_4=D$，D 信号进入触发器，为触发器状态更新做好准备。

② 当 CP 由 0 跳变到 1 后瞬间，G_2 和 G_3 打开，$\overline{S}$ 和 $\overline{R}$ 分别由 Y_1 和 Y_4 的状态所决定，即 $\overline{S}=$

$\overline{Y_1}=\overline{D^*}$，$\overline{R}=\overline{Y_4}=D^*$。这里，$D^*$表示 CP 脉冲上升沿到来之前 D 的逻辑值，因此，D 信号必须满足建立时间要求。可以看出，$\overline{S}$和$\overline{R}$的逻辑值始终是互补的，即二者中仅有一个为 0。如果 $D^*=1$，则$\overline{S}=\overline{D^*}=0$，$\overline{R}=D^*=1$，将基本$\overline{S}\,\overline{R}$锁存器置 1，即触发器状态更新为 $Q^{n+1}=1$。如果 $D^*=0$，则$\overline{S}=1$，$\overline{R}=0$，将输出状态更新为 $Q^{n+1}=0$。因此，$Q^{n+1}=D$。

当 CP 由 0 跳变到 1 使 $Q^{n+1}=1$，即$\overline{S}=0$，$\overline{R}=1$ 时。随后的 $CP=1$ 期间，$\overline{S}=0$ 将 G_1 和 G_3 封锁。其中，$\overline{S}$至 G_1 的反馈线使 $Y_1=1$，起到维持$\overline{S}=0$ 的作用，从而维持了触发器的 1 状态，故称之为置 1 维持线。虽然 D 信号在 $CP=1$ 期间可能由 1 跳变为 0，使 $Y_4=1$，但$\overline{S}=0$ 至 G_3 的反馈线阻止其状态的改变，使$\overline{R}=1$ 保持不变，从而阻塞了 D 端输入的置 0 信号。故称该反馈线为置 0 阻塞线。

反之，当 CP 由 0 跳变到 1 使 $Q^{n+1}=0$，即$\overline{S}=1$，$\overline{R}=0$ 时，随后的 $CP=1$ 期间，$\overline{R}$至 G_4 的反馈线为 0，将 G_4 封锁，使 $Y_4=1$，既阻塞了 $D=1$ 信号进入触发器的路径，又通过$\overline{R}=\overline{Y_4\cdot\overline{S}\cdot CP}=0$，将触发器维持在 0 状态。故这根反馈线称为置 1 阻塞、置 0 维持线。

待 CP 高电平结束，电路又回到最初描述的 $CP=0$ 时的状态。可以看出，触发器只在 CP 由 0 跳变到 1 瞬间更新状态，其余时间均处于保持不变的状态。

虽然维持阻塞 D 触发器的电路结构与前述主从 D 触发器完全不同，但这两个电路所实现的逻辑功能是完全相同的，都是在 CP 上升沿到来后瞬间转换输出状态，将输入信号 D 传递到 Q 端并保持下去。因此，它们使用同一逻辑符号和特性方程[即式(5.4.1)]来表达，并具有十分相似的动态特性。具有直接清零和直接置数端的维持阻塞 D 触发器电路参见习题 5.4.8。

5.4.5 D触发器的动态特性

上升沿触发的 D 触发器的时序图如图 5.4.8 所示，它描述了 D 触发器的动态特性，反映了输出对时钟信号响应的延迟时间，以及对输入信号和时钟信号的定时要求。下面具体说明各参数的含义。

传输延迟时间 t_{pd}

时钟脉冲 CP 上升沿至输出端新状态稳定建立起来的时间定义为 D 触发器的传输延迟时间。图 5.4.8 所示波形中，t_{pLH}是输出 Q 从低电平到高电平的延迟时间，t_{pHL}则是 Q 从高电平到低电平的延迟时间。应用中更多地采用它们的平均值，即平均传输延迟时间 $t_{pd}=(t_{pLH}+t_{pHL})/2$。

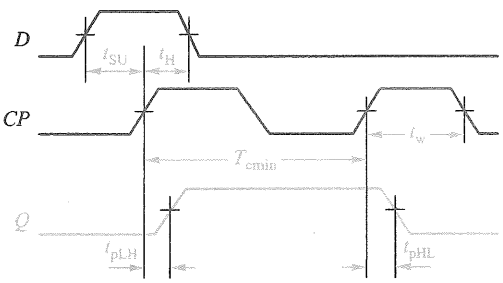


图 5.4.8 D触发器时序图

建立时间 t_{SU}

输入信号 D 在时钟信号 CP 有效沿到来之前必须保持稳定的最短时间称为建立时间 t_{SU}。由于输入信号 D 的变化会引起触发器内部电路逻辑电平的一系列变化，为保证相关电路建立起稳定的状态，以使触发器状态得到正确的转换，必须满足建立时间 t_{SU}的要求。

保持时间 t_H

输入信号 D 在时钟信号 CP 有效沿到来之后必须保持稳定的最短时间称为保持时间 t_H，即在 CP 的有效沿之后，输入信号 D 仍需保持不变，才能保证将 D 可靠地传送到 Q 和$\overline{Q}$

端。t_{SU}和t_H的值与集成电路制造工艺有关。由于技术的进步,已有多种触发器把保持时间几乎降到0。该特性在高速移位寄存器或计数器中是十分重要的。

触发脉冲宽度 t_w

数据手册里规定了时钟脉冲宽度的最短时间t_w,以保证内部门电路有足够的时间实现正确的翻转。

最高时钟频率 f_{cmax}

触发器可靠工作时允许时钟脉冲频率达到的最大值$f_{cmax}=1/T_{cmin}$。因为*CP*无论在高电平还是在低电平期间,触发器内部都要完成一系列动作,存在一定的时间延迟,所以对*CP*最高工作频率有一个限制。

对于特定型号的*D*触发器,上述动态特性参数在制造商的数据手册中都会给出。表5.4.3所示为环境温度在-40 ℃~+85 ℃范围的几种典型的*D*触发器定时参数。

表5.4.3 几种典型的*D*触发器定时参数

型号	功能	工作电压V_{CC}/V	t_{pd}/ns*		t_{SU}/ns	t_H/ns	t_w/ns	f_{cmax}/MHz
			最小	最大	最小	最小	最小	最高
74HC74	双*D*触发器	4.5		44	25	0	20	25
74HC374	八*D*触发器	4.5		45	25	5	20	24
74AUC1G74	单*D*触发器	2.5±0.2	0.8	1.2	0.4	0.3	1	275
74LVC1G79	单*D*触发器	3.3±0.3	1	4	1.3	1	2.5	160
74AUC1G79	单*D*触发器	2.5±0.2	0.3	1.3	0.5	0.1	1.7	275

* 测试条件:表中74HC系列逻辑电路的*Q*端负载电容为$C_L=50$ pF,其余逻辑系列为$C_L=15$ pF。

复习思考题

5.4.1 触发器有哪几种常见的电路结构?试归纳它们的工作原理和动作特点。

5.4.2 图5.4.2所示*D*触发器要求输入信号*D*与时钟信号*CP*有什么样的定时关系?输出信号*Q*和$\overline{Q}$与*CP*有什么样的定时关系?

5.4.3 触发器的异步清零(或置数)与同步清零(或置数)有什么不同?

5.4.4 为什么有些触发器需要设置使能端?

5.5 触发器的逻辑功能

在5.4节中介绍了两种不同结构的*D*触发器,本节将进一步讨论触发器的逻辑功能。我们将触发器在时钟触发沿到来之前的状态称为现态,之后的状态称为次态。所谓触发器的逻辑功能,是指以输入信号和现态为变量,以次态为函数的逻辑关系,可以用特性表、特性方程或状态转换图来描述这种关系。

按照逻辑功能的不同特点,通常将触发器分为*D*触发器、*JK*触发器、*T*触发器和*SR*触发器等几种不同类型。上升沿触发的各个触发器的逻辑符号如图5.5.1所示,方框内分别

标明了时钟信号与各输入信号的控制关联。

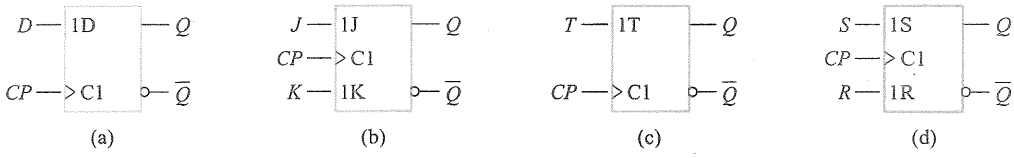


图 5.5.1 上升沿触发的不同触发器的逻辑符号

(a) D 触发器 (b) JK 触发器 (c) T 触发器 (d) SR 触发器

需要指出的是,逻辑功能和电路结构是两个不同的概念。某一种逻辑功能的触发器可以用不同的电路结构来实现,如前述两种不同电路结构而功能完全相同的 D 触发器。同时,以某一种基本电路结构为基础,也可以构成不同逻辑功能的触发器,例如 5.5.5 节将要讨论的将 D 触发器转换为其他逻辑功能的触发器。在本节讨论触发器的逻辑功能时,可以暂时不考虑其内部的电路结构。

5.5.1 D触发器

1. 特性表

以输入信号和触发器的现态为变量,以次态为函数,描述它们之间逻辑关系的真值表称为触发器的特性表。D 触发器的特性表如表 5.5.1 所示,表中列出了触发器的输入信号 D 和现态 Q^n 在每种组合情况下对应的次态 Q^{n+1}。

表 5.5.1 D触发器的特性表

D	Q^n	Q^{n+1}	功能
0	0	0	次态与 D 相同
0	1	0	
1	0	1	
1	1	1	

2. 特性方程

触发器的逻辑功能也可以用逻辑表达式来描述,称为触发器的特性方程。根据表 5.5.1 可以列出 D 触发器的特性方程:

$$Q^{n+1}=D \tag{5.5.1}$$

该式与 5.4 节中从触发器电路结构导出的式(5.4.1)完全相同。

3. 状态转换图

状态转换图是以图形方式描述触发器状态之间的转换及输入条件。根据 D 触发器的特性表可以画出状态转换图,如图 5.5.2 所示。

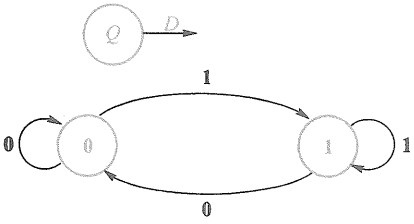


图 5.5.2 D 触发器的状态转换图

图中，每个状态以一个圆圈表示。箭头线表示状态转换方向，箭头线的起点为触发器的现态 Q^n，箭头指向相应的次态 Q^{n+1}。在箭头线旁边标明状态转换的输入变量取值，即输入信号 D 的逻辑值。分别用包围 0 和 1 的两个圆圈表示触发器的两个状态；4 根箭头线分别对应特性表中的 4 行。例如表 5.5.1 中第一行 $Q^n=0, Q^{n+1}=0, D=0$，则箭头线从 0 状态又回到 0 状态，旁边标注的输入值为 0。

由特性表、特性方程或状态转换图均可看出，当 $D=0$ 时，D 触发器的下一状态将被置 0（$Q^{n+1}=0$）；当 $D=1$ 时，将被置 1（$Q^{n+1}=1$）。在时钟脉冲的两个触发沿之间，触发器状态保持不变，即存储 1 位二进制数据。

5.5.2 JK触发器

1. 特性表

JK 触发器的特性表如表 5.5.2 所示，其中列出了触发器的输入信号 J、K 和现态 Q^n 在不同组合条件下次态 Q^{n+1} 的值。

由表看出：

当 $J=K=0$ 时，*JK* 触发器状态保持不变（$Q^{n+1}=Q^n$）。

当 $J=0, K=1$ 时，触发器的下一状态将被置 0（$Q^{n+1}=0$）。

当 $J=1, K=0$ 时，触发器的下一状态将被置 1（$Q^{n+1}=1$）。

当 $J=K=1$ 时，触发器的下一状态将翻转（$Q^{n+1}=\overline{Q^n}$）。

表 5.5.2 JK触发器的特性表

J	K	Q^n	Q^{n+1}	功能
0	0	0	0	保持不变
0	0	1	1	
0	1	0	0	置 0
0	1	1	0	
1	0	0	1	置 1
1	0	1	1	
1	1	0	1	翻转
1	1	1	0	

2. 特性方程

根据表 5.5.2 可以得出 *JK* 触发器次态的逻辑表达式，经化简可得其特性方程如下：

$$Q^{n+1}=J\overline{Q^n}+\overline{K}Q^n \tag{5.5.2}$$

3. 状态转换图

JK 触发器的状态转换图如图 5.5.3 所示，它可从表 5.5.2 导出。与 D 触发器状态转换图

的差别是它有两个输入变量，所以每根箭头线旁都标有两个逻辑值，分别为 J、K 的值。可以注意到，在每一个转换方向上，J、K 中总有一个是无关变量。例如，表 5.5.2 的第 5 行和第 7 行，$Q^n=0$ 转换为 $Q^{n+1}=1$，条件是 $J=1$，而 K 既可以取 0，也可以取 1，故状态转换图中的转换条件以 1×表示。所以，状态转换图中的 4 根方向线实际对应表中 8 行。

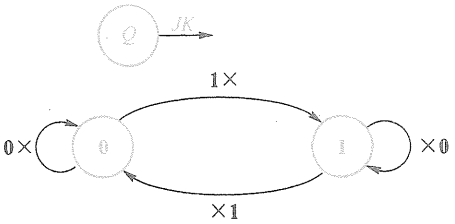


图 5.5.3 JK 触发器的状态转换图

在所有逻辑类型的触发器中，JK 触发器具有最强的逻辑功能，在外部 J、K 信号控制下，它能执行保持、置 0、置 1 和翻转四种操作，并可用简单的附加电路转换为其他功能的触发器。

例 5.5.1 设下降沿触发的 JK 触发器时钟脉冲 $\overline{CP}$ 和 J、K 信号的波形如图 5.5.4 中横虚线上面所示，试画出输出端 Q 的波形。设触发器的初始状态为 0。

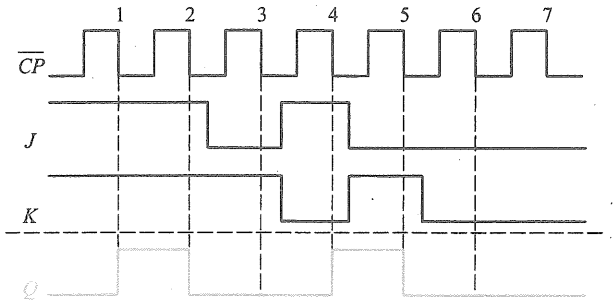


图 5.5.4 例 5.5.1 的波形图

解：根据 JK 触发器的特性表、特性方程或状态转换图都可画出 Q 端波形，如图 5.5.4 中横虚线下面所示。

从图 5.5.4 可以看出，在第 1、2 个 $\overline{CP}$ 脉冲作用期间，J、K 均为 1，每输入一个脉冲，Q 端的状态就改变一次，即触发器翻转一次。触发器的这种工作状态称为计数状态：由触发器翻转的次数可以计算出时钟脉冲的个数。同时，Q 端的方波频率是时钟脉冲频率的二分之一。若以 CP（或 $\overline{CP}$）为输入信号，Q 为输出信号，则一个触发器可作为二分频电路，两个触发器级联可获得四分频，其余类推。

5.5.3 T 触发器

在某些应用中，需要对上述计数功能进行控制：当控制信号 $T=1$ 时，每来一个 CP（或 $\overline{CP}$）脉冲，它的状态翻转一次；而当 $T=0$ 时，则不对 CP（或 $\overline{CP}$）信号作出响应而保持状态不变。具备这种逻辑功能的触发器称为 T（toggle）触发器。

1. 特性表

T 触发器的特性表如表 5.5.3 所示。

表 5.5.3 T触发器的特性表

T	Q^n	Q^{n+1}	功能
0	0	0	保持
0	1	1	
1	0	1	翻转
1	1	0	

2. 特性方程

根据表 5.5.3 可以写出 T 触发器的特性方程：

$$Q^{n+1}=T\overline{Q^n}+\overline{T}Q^n \tag{5.5.3}$$

3. 状态转换图

T 触发器的状态转换图如图 5.5.5 所示。

由此可知，T 触发器的功能是：$T=1$ 时为翻转状态，$Q^{n+1}=\overline{Q^n}$；$T=0$ 时为保持状态，$Q^{n+1}=Q^n$。

比较式(5.5.3)和式(5.5.2)，如果令 $J=K=T$，则两式等效。事实上只要将 JK 触发器的 J、K 端连接在一起作为 T 输入端，就可实现 T 触发器的功能，因此，在小规模集成触发器产品中没有专门的 T 触发器，如果有需要，可用其他功能的触发器转换。

4. T′触发器

当 T 触发器的 T 输入端固定接高电平时(即 $T\equiv1$)，则式(5.5.3)变为

$$Q^{n+1}=\overline{Q^n} \tag{5.5.4}$$

也就是说，时钟脉冲每作用一次，触发器翻转一次。这种特定的 T 触发器常在集成电路内部逻辑图中出现，其输入只有时钟信号，有时称为 T'触发器。上升沿触发的 T'触发器逻辑符号如图 5.5.6 所示。

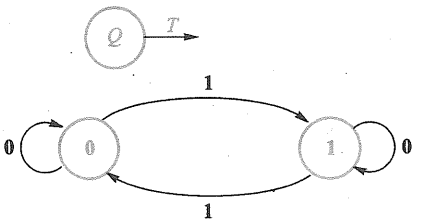


图 5.5.5 T 触发器的状态转换图

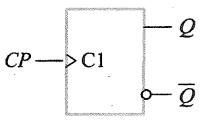


图 5.5.6 T'触发器的逻辑符号

5.5.4 SR触发器

仅有置位、复位功能的触发器称为 SR 触发器，它的特性表如表 5.5.4 所示。从表中可以看出，$S=R=1$ 时，触发器的次态是不能确定的，如果出现这种情况，触发器将失去控制。因此，SR 触发器的使用必须遵循 $SR=0$ 的约束条件。从特性表可导出表达次态与现态、输入信号关系的表达式

$$\begin{cases} Q^{n+1}=S\overline{R}\,\overline{Q^n}+S\overline{R}Q^n+\overline{S}\,\overline{R}Q^n=S\overline{R}+\overline{S}\,\overline{R}Q^n \\ SR=0(\text{约束条件}) \end{cases}$$

由于约束条件限制，当 $S=1$ 时，$R=0$，$Q^{n+1}=S$；当 $S=0$ 时，$Q^{n+1}=\overline{R}Q^n$。上式可进一步化简，于是得到特性方程

$$\begin{cases} Q^{n+1}=S+\overline{R}Q^n \\ SR=0 \quad (\text{约束条件}) \end{cases} \tag{5.5.5}$$

表 5.5.4 SR触发器的特性表

S	R	Q^n	Q^{n+1}	功能
0	0	0	0	保持
0	0	1	1	
0	1	0	0	复位
0	1	1	0	
1	0	0	1	置位
1	0	1	1	
1	1	0	不确定	禁用
1	1	1	不确定	

从特性表也可以导出状态转换图，如图 5.5.7 所示。

事实上，制造商罕有专门的 SR 触发器产品提供，它主要出现于集成电路的内部结构，需要单独使用 SR 触发器时，可以 JK 触发器直接代用。比较图 5.5.3 和图 5.5.7 的状态转换图，令 $J=S$，$K=R$，便可用 JK 触发器实现 SR 触发器的全部有效功能。

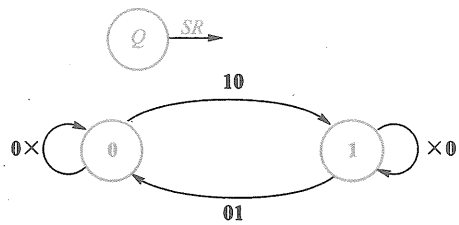


图 5.5.7 SR 触发器的状态转换图

5.5.5 D触发器逻辑功能的转换

D 触发器是一个通用的存储元件，在大规模集成电路中应用广泛，是可编程逻辑器件内部触发器的基础。在 D 触发器的输入端增加一些简单的门电路，就可以实现其他类型触发器的逻辑功能。

1. D触发器构成JK触发器

比较 D 触发器和 JK 触发器的特性方程，即式(5.5.1)和式(5.5.2)，可以令

$$Q^{n+1}=D=J\overline{Q^n}+\overline{K}Q^n \tag{5.5.6}$$

按上式，可得电路如图 5.5.8 所示，电路特性符合 JK 触发器的特性方程，从而能实现 JK

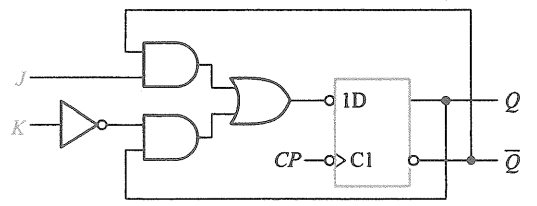


图 5.5.8 用 D 触发器实现 JK 触发器的逻辑功能

触发器的所有功能。有些 CMOS 主从结构的集成 *JK* 触发器就是采用类似方式实现的。

2. *D* 触发器构成 *T* 触发器

用构成 *JK* 触发器相同的方法，令

$$Q^{n+1}=D=T\overline{Q^n}+\overline{T}Q^n=T\oplus Q^n=T\odot\overline{Q^n} \tag{5.5.7}$$

只需在 *D* 输入端前增加一个异或门或者同或门即可实现，于是得到如图 5.5.9(a)、(b)所示的两种 *T* 触发器逻辑电路。

3. *D* 触发器构成 *T'* 触发器

比较式(5.5.1)和式(5.5.4)可得 $Q^{n+1}=D=\overline{Q^n}$，于是，画出用 *D* 触发器构成的 *T'* 触发器如图 5.5.10 所示。

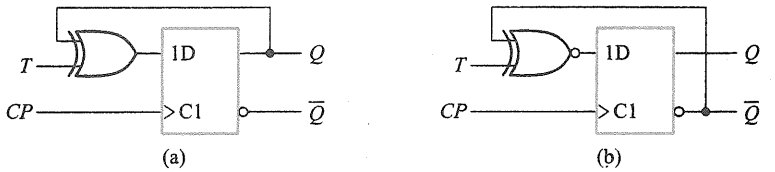


图 5.5.9 用 *D* 触发器实现 *T* 触发器的逻辑功能
(a) 用异或门实现 (b) 用同或门实现

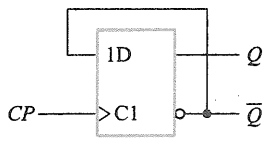


图 5.5.10 用 *D* 触发器实现 *T'* 触发器的逻辑功能

复习思考题

5.5.1 触发器的逻辑功能和电路结构之间有什么关系？

5.5.2 写出 *D* 触发器、*JK* 触发器、*T* 触发器和 *SR* 触发器的特性方程，并画出它们的状态图。

5.5.3 将 *D* 触发器的 *D* 与 $\overline{Q}$ 相连，对于给定的 *CP* 画出 *Q* 的波形，说明两者之间的频率关系。

5.6 用 Verilog HDL 描述锁存器和触发器

4.6 节介绍了组合逻辑电路的行为级建模，本节将讨论时序电路的基本部件——锁存器和触发器的建模，首先介绍有关的基础知识，然后以实例进行说明。

5.6.1 时序逻辑电路建模基础

在 Verilog 中，行为级描述主要使用由关键词 **initial** 或 **always** 定义的两种结构类型的语句。一个模块的内部可以包含一个或多个 **initial** 或 **always** 语句块，仿真时这些语句块同时并行执行，即与它们在模块内部排列的顺序无关，都从仿真的 0 时刻开始执行。

initial(初始化)主要在测试模块中使用，通常用来描述激励信号，仿真时仅执行一次。在硬件电路的行为描述中，有时为了仿真的需要，也可以用 **initial** 语句给寄存器变量赋初值。**initial** 语句主要是面向仿真的过程语句，逻辑综合工具不支持 **initial** 语句，这里不详细讨论。

always 本身是一个无限循环语句，即不停地循环执行其内部的过程语句，直到仿真过程结束。但用它来描述硬件电路的逻辑功能时，通常在 **always** 后面紧跟着循环的控制条

件，所以 always 语句的一般用法如下：

```
always @(敏感信号列表)
begin
    块内部局部变量的定义;
    过程赋值语句;
end
```

这里，“敏感信号列表[①]”是 Verilog-2001 中的说法，在 Verilog-1995 中称为“事件控制表达式”，即等待确定的事件发生(信号的变化)或某一特定的条件变为“真”，它是执行后面过程赋值语句的条件。“过程赋值语句”左边的变量必须被定义成 reg 数据类型，右边变量可以是任意数据类型。begin 和 end 将多条过程赋值语句包围起来，组成一个顺序语句块，块内的语句按照排列顺序依次执行，最后一条语句执行完后，执行挂起，然后 always 语句处于等待状态，等待下一个事件的发生。注意，当 begin 和 end 之间只有一条语句，且没有定义局部变量时，则关键词 begin 和 end 可以被省略。

在 Verilog 中，将逻辑电路中的敏感信号分为两种类型：电平敏感信号和边沿触发信号。在组合电路中，输入信号的变化直接导致输出信号的变化；时序电路中的锁存器输出在使能信号有效电平(高电平或低电平)内亦随输入电平而变化，如图 5.4.1(a)、(b)所示波形。这种对输入信号电平变化的响应称为电平敏感信号。

例如，例 4.6.1 中对 4 选 1 数据选择器的描述语句

```
always @(D,S)
```

说明 S 或 D 中任意一个信号的电平发生变化(即有电平敏感事件发生)，后面的过程赋值语句将会执行一次。

而触发器状态的变化仅仅发生在时钟脉冲的上升沿或下降沿，如图 5.4.1(c)、(d)所示波形。Verilog 中分别用关键词 posedge(上升沿)和 negedge(下降沿)进行说明，这就是边沿敏感信号。例如，语句

```
always @(posedge CP or negedge CR)//Verilog-1995 语法
```

也可以用逗号进行分隔多个变量。例如

```
always @(posedge CP,negedge CR)  //Verilog-2001/2005 语法
```

说明在时钟信号 CP 的上升沿到来或在清零信号 CR 跳变为低电平时，执行后面的过程语句。注意，不能省略关键词 negedge，因为敏感信号列表中不能同时包含电平敏感信号和边沿敏感信号。

always 块内部的过程赋值语句有两种类型：阻塞型赋值语句(blocking assignment statement)和非阻塞型赋值语句(non-blocking assignment statement)。它们分别使用不同的赋值符号，在阻塞型赋值语句中使用“=”(阻塞赋值符)，在非阻塞型赋值语句中使用“<=”(非阻塞赋值符)。在由 begin…end 构成的串行语句块中，多条阻塞型赋值语句按照它们在块中排列的顺序依次执行。即前一条语句没有完成赋值之前，则不能执行后面的语句，换言之，前面的语句阻塞了后面语句的执行。例如，下面两条阻塞型赋值语句的执行过程是：首

① 是 sensitivity list 的中文翻译。

先执行第一条语句，将 A 的值赋给 B，接着执行第二条语句，将 B 的值（等于 A 值）加 1，并赋给 C，执行完后，C 的值等于 A+1。如果 A=0，B=1，则执行完语句块后 B=0，C=1。

```
begin
    B=A;
    C=B+1;
end
```

在由 **begin…end** 构成的串行语句块中，多条非阻塞型赋值语句是并行执行的。首先计算各条语句右边表达式的值，并将各个值分别存入暂存器，在语句块结束之前将暂存器的值同时赋给左边变量。例如，执行完下面两条非阻塞型赋值语句后：C 的值等于 B 的原始值（而不是 A 的值）加 1。如果 A=0，B=1，则执行完语句块后 B=0，C=2。

```
begin
    B<=A;
    C<=B+1;
end
```

综上所述，阻塞型赋值语句和非阻塞型赋值语句的主要区别是完成赋值操作的时间不同，前者的赋值操作是立即执行的，即执行后一句时，前一句的赋值已经完成；而后者的赋值操作要到顺序块内部的多条非阻塞型赋值语句运算结束时，才同时并行完成赋值操作，一旦赋值操作完成，语句块的执行也就结束了。

注意，在可综合的电路设计中，一个语句块的内部只允许出现唯一一种类型的赋值语句，而不允许阻塞型赋值语句和非阻塞型赋值语句二者同时出现。在时序电路的设计中，建议采用非阻塞型赋值语句。

5.6.2 锁存器和触发器的 Verilog 建模实例

本节给出一些锁存器和触发器的行为级描述实例。

例 5.6.1 描述表 5.3.1 所示门控 *D* 锁存器的行为。

解： *D* 锁存器是一种对脉冲电平敏感的双稳态电路，其输出 Q 受到 E 和 D 电平变化的影响，所以在 **always** 语句中@后是电平敏感信号 E 和 D，如果 E 或者 D 发生变化，将执行一次后面的 **if** 语句。当 E 为 1 时，输入 D 的变化传送到输出 Q；否则，输出 Q 保持不变。其代码如图 5.6.1 所示。

```
module D_latch (Q, QN, D, E);//Verilog-1995 语法
  input D, E;
  output reg Q;
  output QN;

  assign QN = ~Q ;//连续赋值语句
  always @(E or D)
    if (E) Q <= D;   // if (E)与 if (E==1)等价

endmodule
```

图 5.6.1 *D* 锁存器的 Verilog 代码

注意，代码没有给出 **else** 语句，即没有说明 E=0 时会执行什么操作，这种情况下，Verilog 综合器会用锁存器来实现电路，以便输出端保持原来的值不变。

例 5.6.2 分析图 5.6.2 所示 Verilog 模块，说明它所完成的逻辑功能。

解： 图 5.6.2(a)模块描述的是基本 *D* 触发器的行为。该模块的输出信号为 Q，输入信

号为 D 和 CP。在 always 语句@符号之后使用了边沿触发事件，即 posedge CP，则语句Q<=D 仅在 CP 上升沿期间将 D 的值赋给 Q，而在其他任何时间，Q 保持状态不变。

图 5.6.2(b)模块描述的是具有同步清零功能的 *D* 触发器的行为，即清零信号 Rd 在 CP 脉冲上升沿到来后才起作用。if-else 语句首先判断 Rd 信号，如果 Rd 为逻辑 0，则将输出 Q 清零；否则，将输入 D 传给输出 Q。显然，在该语句块中，清零信号 Rd 具有较高的优先权，只有 Rd=1 时，才有可能执行 Q<=D 语句。

```
module DFF (
  output reg Q,
  input D,
  input CP
);
  always @(posedge CP)
    Q<= D;
endmodule
```

(a)

```
module sync_rst_DFF (
  output reg Q,
  input D, CP, Rd
);
  always @(posedge CP)
    if (~Rd) Q <= 1'b0;
    else Q <= D;
endmodule
```

(b)

图 5.6.2 两种 *D* 触发器的 Verilog 代码

例 5.6.3 分析图 5.6.3 所示 Verilog 模块，说明它所完成的逻辑功能。

```
module async_set_rst_DFF (
  output reg Q, QN,
  input D, CP, Sd, Rd
);
always @(posedge CP, negedge Sd, negedge Rd)
  begin
    if (~Sd || ~Rd)
        if (~Sd ) begin   Q <= 1'b1; QN <= 1'b0;   end
        else begin   Q <= 1'b0; QN <= 1'b1;   end
    else
        begin   Q <= D;   QN <= ~D;   end
    end
endmodule
```

图 5.6.3 含异步清零端和异步置数端的 *D* 触发器

解：这个模块描述的是具有异步直接置 1、置 0 功能的 *D* 触发器。在 always 语句的@符号之后列出了 3 个敏感信号，其中必须有一个时钟事件，其他为异步事件。

这个模块在输入信号 CP 的上升沿到来，Sd 或者 Rd 跳变为低电平时，后面的 if-else 语句就会被执行一次。negedge Sd 和 negedge Rd 是两个异步事件，它需要与 if(~Sd || ~Rd)语句相匹配。如果条件成立，先执行 if(~Sd)这条语句，判断 Sd 是否为逻辑 0，如果 Sd 为 0，则 Q 置 1，QN 置 0；否则将执行第一个 else 后面的两条语句，将 Q 置 0，QN 置 1；

如果 Sd 和 Rd 均不为 0，当时钟 CP 上升沿到来时，将执行最后面的 else 语句，Q 等于 D，QN 等于~D。从语句执行的顺序来看，如果直接置 1、置 0 和时钟上升沿三个事件同时发

生，则置 1 事件的优先级别最高，其次是置 0 事件，时钟上升沿事件的优先级别最低。

例 5.6.4 分析图 5.6.4 中的 Verilog 模块，说明阻塞赋值语句和非阻塞赋值语句之间的区别。

```
module Circuit_A (
  output reg f, g,
  input A, B, C, CP
);
always @(posedge CP)
begin
  f = A & B;
  g = f | C;
end
endmodule
```

```
module Circuit_B (
  output reg f, g,
  input A, B, C, CP
);
always @(posedge CP)
begin
  f <= A & B;
  g <= f | C;
end
endmodule
```

图 5.6.4 使用阻塞赋值和非阻塞赋值的 Verilog 代码

解：因为模块声明输出信号 f 和 g 是寄存器型变量，逻辑综合后会用 *D* 触发器来存储它们的值。模块 Circuit_A 在 **always** 语句@ 符号之后的敏感信号表中使用了 **posedge** CP，所以 f 和 g 都是 *D* 触发器的输出。由于使用了阻塞赋值，f=A&B 产生的新 f 值，会在 g=f|C 中使用，即 g=f|C 与 g=(A&B)|C 是等效的。综合后得到的电路如图 5.6.5 所示。

模块 Circuit_B 使用了非阻塞赋值语句，进入 **always** 语句后，使用各个变量的原始值计算两个表达式右边的值并存入暂存器，在 **always** 语句结束时 f 和 g 的值被同时更新。即计算 g<=f|C 时使用了 f 更新之前的值，即或门的一个输入端应该连接到触发器的输出端 f。综合后得到的电路如图 5.6.6 所示。

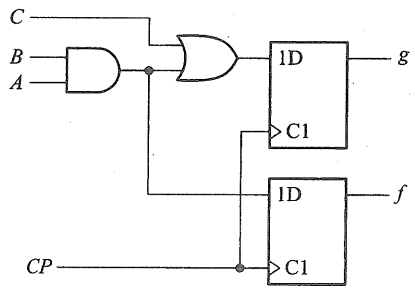


图 5.6.5 模块 Circuit_A 的电路

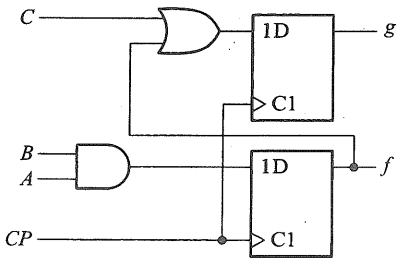


图 5.6.6 模块 Circuit_B 的电路

假设分别将两个模块中给 f 和 g 赋值的两条语句的次序颠倒一下，那么会综合成什么样的电路呢？

对模块 Circuit_A 来说，由于使用阻塞赋值，**begin-end** 之间的语句是顺序执行的，颠倒语句次序后，首先执行 g=f|C，此时 f 的值就是进入 **always** 语句时的值，即 CP 上升沿到来之前的值，所以综合后生成的电路为图 5.6.6 所示。对模块 Circuit_B 来说，由于用的是非阻塞赋值，**begin-end** 之间的语句是并行执行的，所以颠倒语句次序不影响综合后生成的电路。

可见，阻塞赋值对语句的顺序具有依赖性，用它对时序电路建模存在一定的不确定性，所以，对时序逻辑电路建模时，建议使用非阻塞赋值语句。

复习思考题

5.6.1 在 Verilog 中，**initial** 块和 **always** 块的主要区别是什么？

5.6.2 在 **always** 语句中对电平敏感事件和边沿敏感事件的描述有何不同？

5.6.3 阻塞型赋值和非阻塞型赋值有何区别？

小 结

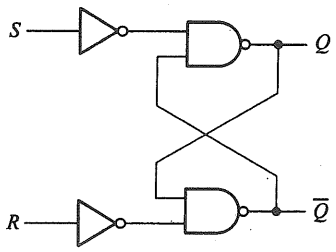
- 锁存器和触发器是具有记忆功能的基本逻辑单元，它们都有两个稳定状态，从而可以存储 1 位二进制数值。置 0、置 1 和数据保持是存储单元最基本的功能。
- 锁存器是对脉冲电平敏感的存储电路。基本 *SR* 锁存器的输出状态由输入电平直接控制。门控 *SR* 锁存器在使能信号有效的条件下由输入信号决定其状态。
- *D* 锁存器的电路简单，特别是传输门控 *D* 锁存器在中、大规模 CMOS 集成电路中应用较多，是构成触发器的基础电路。在使能信号有效时，*D* 锁存器输出跟随输入信号 *D* 而变化。
- 触发器是对时钟脉冲边沿敏感的存储电路，它们在时钟脉冲的上升沿或下降沿作用下更新状态。目前流行的触发器电路主要有主从和维持阻塞电路结构，它们的工作原理各不相同。
- 触发器按逻辑功能分类有 *D* 触发器、*JK* 触发器、*T* 触发器和 *SR* 触发器等几种类型。它们的功能可用特性表、特性方程和状态转换图来描述。
- 触发器的电路结构与逻辑功能没有必然联系。例如，*D* 触发器既有主从结构的，也有维持阻塞结构的。每一种逻辑功能的触发器都可以通过适当增加外部连线和门电路转换为其他逻辑功能的触发器。
- 可以用 Verilog 语言对锁存器和触发器的行为特性进行建模。Verilog 语言提供了两种赋值符：阻塞型赋值和非阻塞型赋值。对组合逻辑电路建模时，建议使用阻塞型赋值；对时序电路建模时，建议使用非阻塞型赋值。
- 触发器状态的变化仅仅发生在时钟脉冲的上升沿或下降沿，在 Verilog 中分别用关键词 **posedge**（上升沿）和 **negedge**（下降沿）进行说明。

习 题

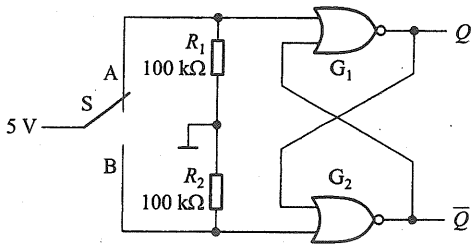
5.2 *SR* 锁存器

5.2.1 分析图题 5.2.1 所示电路的功能，列出功能表。

5.2.2 用 CMOS 电路 74HCT02 或非门构成消除机械开关抖动影响的电路如图题 5.2.2 所示，试画出在开关 S 由位置 A 到 B 时 Q 和 $\overline{Q}$ 端的波形。



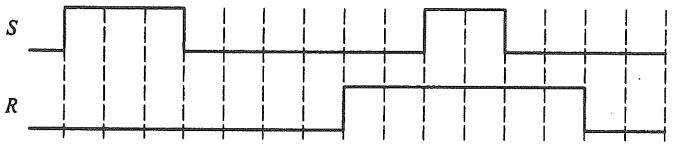
图题 5.2.1



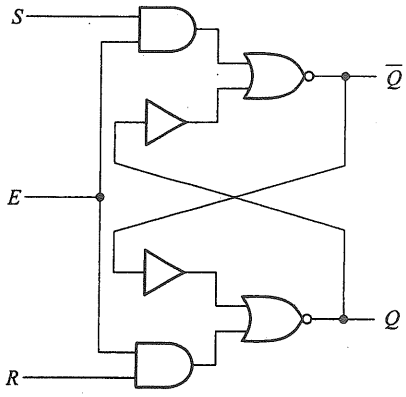
图题 5.2.2

5.2.3 *SR* 锁存器如图 5.2.1(a)所示，其初始状态为 $Q=0$、$\overline{Q}=1$，图中两个或非门的传输延迟时间均为 10 ns。*SR* 锁存器的输入波形如图题 5.2.3 所示，虚线表示的时间间隔为 10 ns，画出考虑或非门传输延迟时间的输出波形。假定输入和输出信号的上升和下降时间均为 0。

5.2.4 由与或非门组成的 *SR* 锁存器如图题 5.2.4 所示(图中缓冲器的作用是为了减少传输到或非门两个输入端信号的延迟时间的差值)，试分析其工作原理并列出功能表。



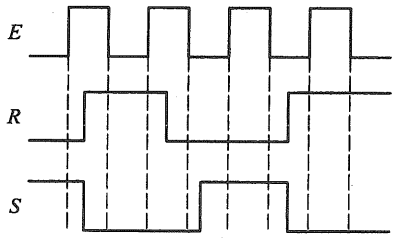
图题 5.2.3



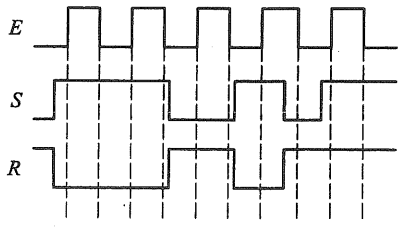
图题 5.2.4

5.2.5 图题 5.2.4 所示锁存器的 *E*、*R*、*S* 端的输入信号波形如图题 5.2.5 所示，试画出 *Q* 和 $\overline{Q}$ 端的波形。设初始状态 $Q=0$。

5.2.6 若图 5.2.7(a)所示电路的初始状态为 $Q=1$，*E*、*S*、*R* 端的输入信号如图题 5.2.6 所示，试画出相应 *Q* 和 $\overline{Q}$ 端的波形。



图题 5.2.5



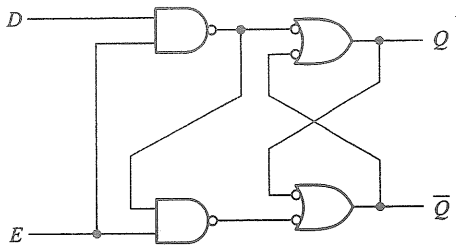
图题 5.2.6

5.3 D锁存器

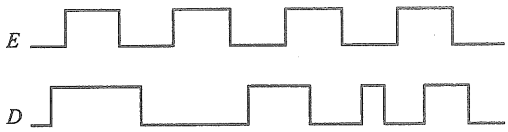
5.3.1 列出图题 5.3.1 所示电路的功能表，并与表 5.3.1 比较，证明该电路的逻辑功能与图

5.3.1(a)和图 5.3.3 所示的锁存器相同。与图 5.3.3 所示电路比较,图题 5.3.1 所示电路有何优点?

5.3.2 图题 5.3.1 所示电路的输入端 E、D 的波形如图题 5.3.2 所示,试画出输出 Q 的波形。设电路的初始状态为 0。



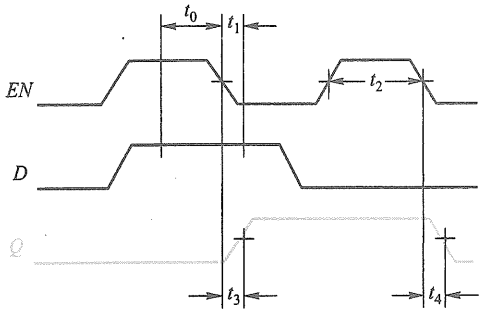
图题 5.3.1



图题 5.3.2

5.3.3 试用 1 片八 D 锁存器 74HC373 设计一个能锁存两组 BCD 码信号的锁存电路。假定三态输出使能端 $\overline{OE}=0$,锁存器原输出为 $Q_7Q_6Q_5Q_4=1001(9_D)$ 和 $Q_3Q_2Q_1Q_0=0100(4_D)$,输入为 $D_7D_6D_5D_4=1001(9_D)$ 和 $D_3D_2D_1D_0=0101(5_D)$,画出锁存器锁存新数据前、后使能端 LE 应输入的波形和相应 Q_0 的波形。74HC373 的内部逻辑图请参见图 5.3.5。

5.3.4 D 锁存器时序图如图题 5.3.4 所示,试说明各段是指什么时间参数。



图题 5.3.4

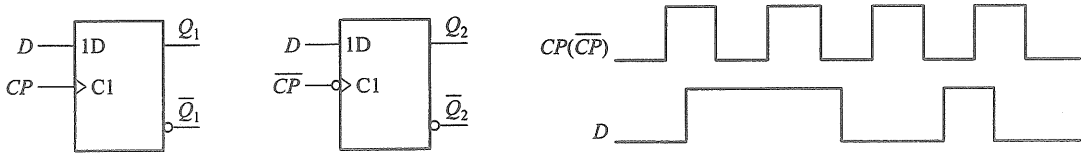
5.3.5 根据表 5.3.3 所示的参数回答下列问题:

(1) 74HCT373 的使能信号 E 的脉冲宽度为 15 ns,是否能够正常工作?

(2) 74LVC1G373 的 E 为有效状态并且 $Q=0$ 时,当 D 由低变高后,Q 至少需要经过多长时间变为高电平?

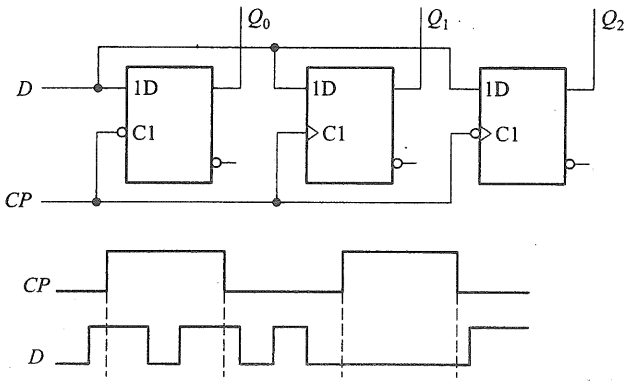
5.4 D触发器

5.4.1 上升沿触发和下降沿触发的 D 触发器逻辑符号及时钟信号 CP($\overline{CP}$)和输入信号 D 的波形如图题 5.4.1 所示。分别画出它们的 Q 端波形。设触发器的初始状态为 0。



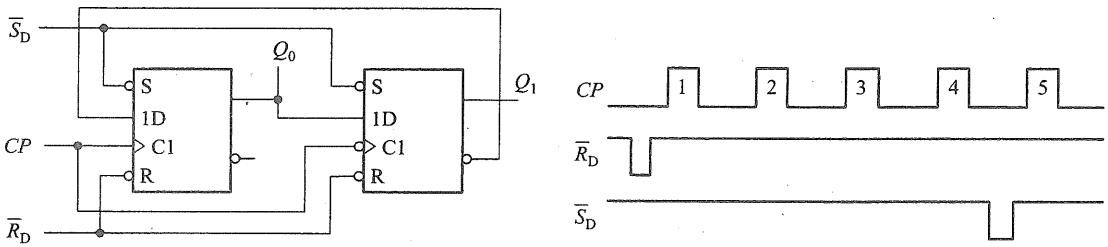
图题 5.4.1

5.4.2 包含 D 锁存器和 D 触发器的电路，以及输入信号 D、控制信号 CP 的波形如图题5.4.2所示，试画出输出端 Q_0、Q_1、Q_2 的波形，设锁存器和触发器的初始状态均为 0。



图题 5.4.2

5.4.3 由 D 触发器构成的电路，以及时钟脉冲 CP、直接清零 $\overline{R}_D$ 和直接置数 $\overline{S}_D$ 的波形如图题 5.4.3 所示，试画出输出端 Q_0 和 Q_1 的波形。



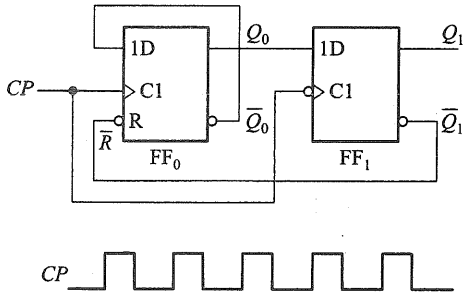
图题 5.4.3

5.4.4 由 D 触发器构成的电路如图题 5.4.4 所示，$\overline{R}$ 为异步清零端。画出在时钟脉冲 CP 作用下，输出端 Q_0 和 Q_1 的波形。设触发器的初始状态均为 0。

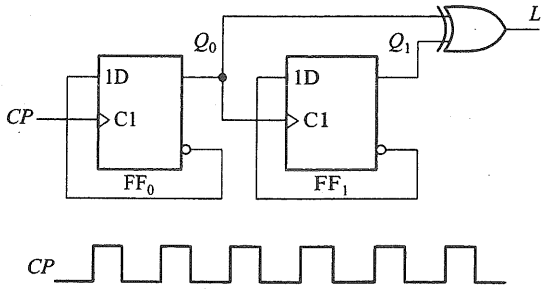
5.4.5 由 D 触发器构成的电路如图题 5.4.5 所示。设触发器的初始状态均为 0。

(1) 画出在时钟脉冲 CP 作用下，输出端 Q_0、Q_1 和 L 的波形。

(2) 如果 CP 的频率为 100 MHz，试确定 Q_0、Q_1 和 L 的频率。

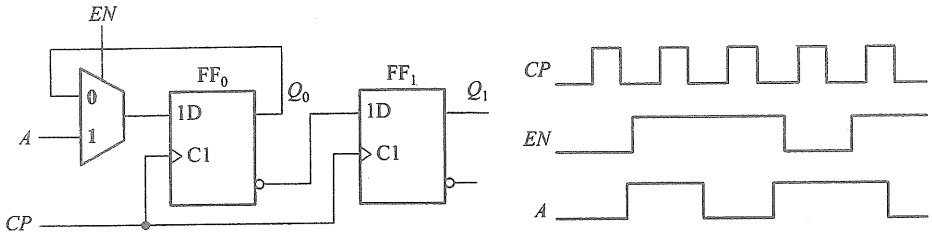


图题 5.4.4



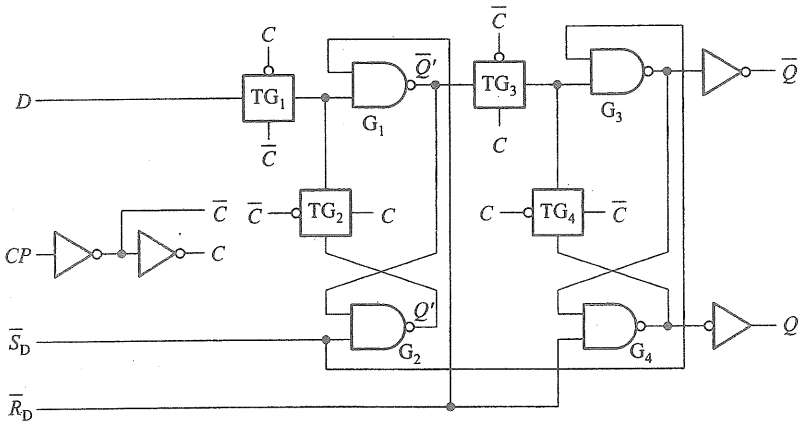
图题 5.4.5

5.4.6 由 D 触发器和数据选择器构成的电路，以及各个输入信号 EN、A 和时钟脉冲 CP 的波形如图题 5.4.6 所示。试画出输出 Q_0 与 Q_1 的波形。设触发器的初始状态均为 0。



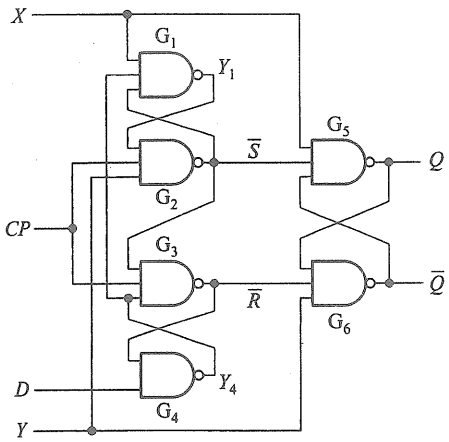
图题 5.4.6

5.4.7 触发器的逻辑电路如图题 5.4.7 所示，确定其属于何种电路结构的触发器并分析工作原理，列出功能表。



图题 5.4.7

5.4.8 维持阻塞 D 触发器电路如图题 5.4.8 所示，试分析输入端 X 和 Y 的作用。



图题 5.4.8

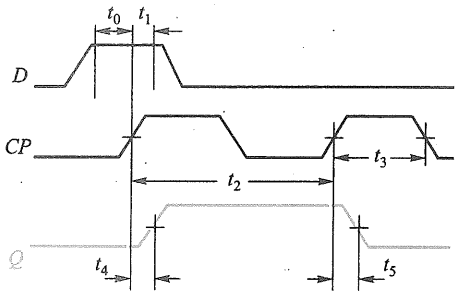
5.4.9 根据表 5.4.3 所示的参数，判断上升沿触发的 74HC74 在下列情况下是否能够正常工作。

（1）当 CP 上升沿到来之前，D 信号保持 10 ns 时间不变。

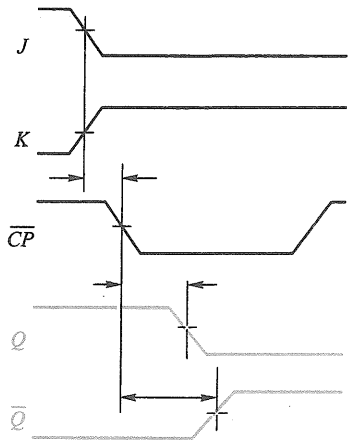
（2）当 CP 上升沿到来之后，D 信号保持 3 ns 时间不变。

5.4.10 D 触发器时序图如图题 5.4.10 所示，试说明各段是指什么时间参数。

5.4.11 某下降沿触发的 JK 触发器的动态特性参数为：$t_{SU}=5$ ns，$t_{pHL}=10$ ns，$t_{pLH}=15$ ns。试在图题 5.4.11 所示的时序图中标出各时间参数的位置。



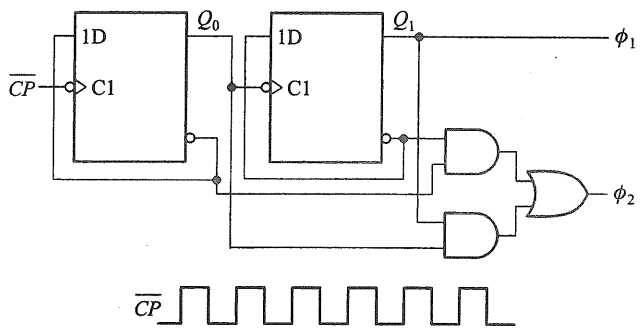
图题 5.4.10



图题 5.4.11

5.5 触发器的逻辑功能

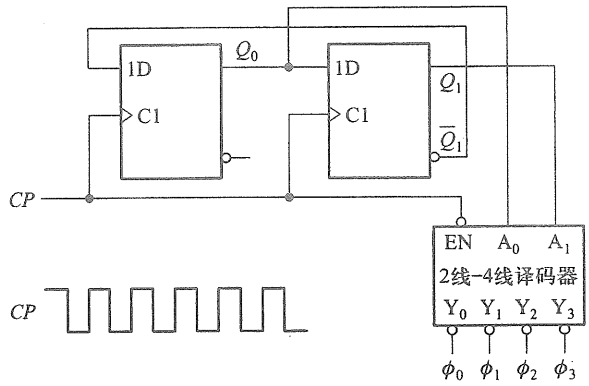
5.5.1 两相脉冲产生电路如图题 5.5.1 所示，试画出在 $\overline{CP}$ 作用下 ϕ_1、ϕ_2 的波形，并说明 ϕ_1 和 ϕ_2 的时间关系。各触发器的初始状态为 **0**。



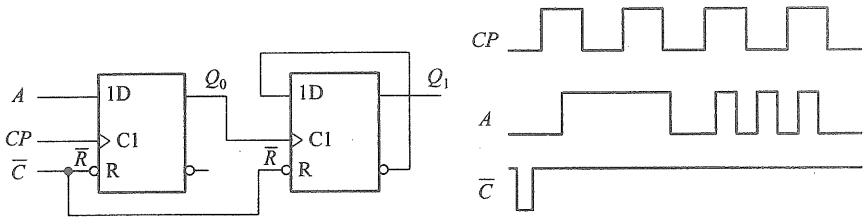
图题 5.5.1

5.5.2 逻辑电路如图题 5.5.2 所示，初始状态为 $Q_0=Q_1=0$，试画出在 CP 作用下，ϕ_0、ϕ_1、ϕ_2 和 ϕ_3 的波形。

5.5.3 逻辑电路和各输入信号波形如图题 5.5.3 所示，$\overline{R}$ 为异步清零端。画出两触发器 Q 端的波形。设触发器的初始状态均为 **0**。

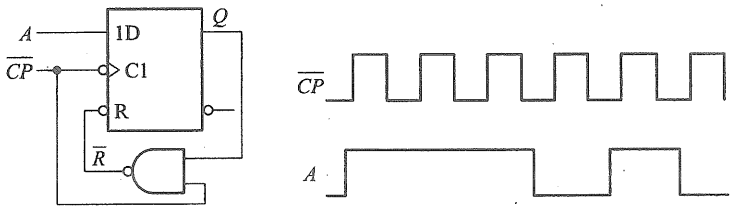


图题 5.5.2



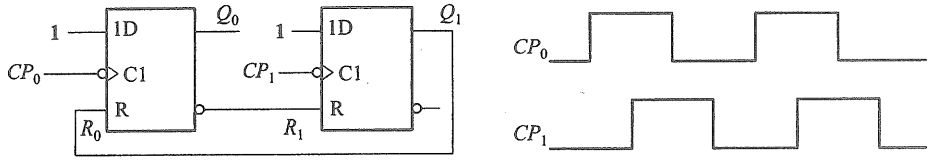
图题 5.5.3

5.5.4 逻辑电路如图题 5.5.4 所示，$\overline{R}$ 为异步清零端，已知$\overline{CP}$和 A 的波形，画出触发器 Q 端的波形，设触发器的初始状态为 0。



图题 5.5.4

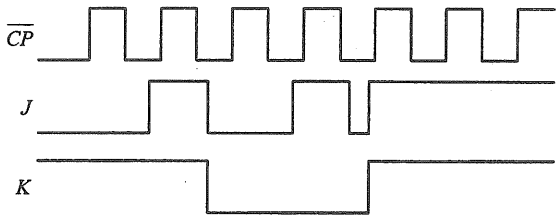
5.5.5 逻辑电路和输入信号波形如图题 5.5.5 所示，R 为异步清零端，画出各触发器 Q 端的波形。设触发器的初始状态均为 0。



图题 5.5.5

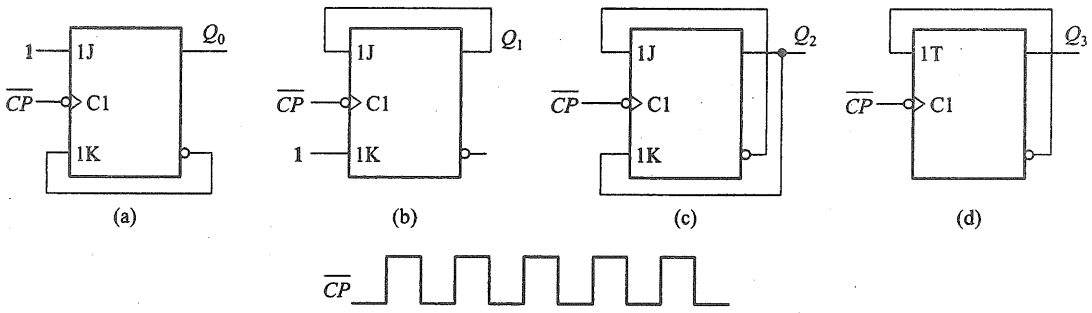
5.5.6 试用卡诺图化简表 5.5.2 表达的逻辑关系，并将结果与式(5.5.2)核对。

5.5.7 设下降沿触发的 JK 触发器初始状态为 **0**，$\overline{CP}$、J、K 信号如图题 5.5.7 所示，试画出触发器 Q 端的输出波形。



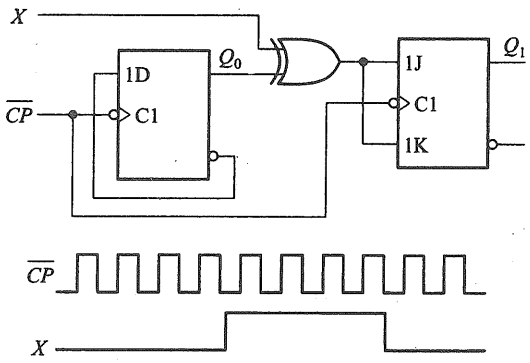
图题 5.5.7

5.5.8 电路如图题 5.5.8 所示，画出在 $\overline{CP}$ 脉冲作用下 Q 端波形。设各触发器的初始状态为 **0**。



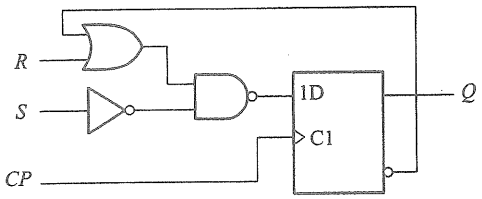
图题 5.5.8

5.5.9 逻辑电路如图题 5.5.9 所示，已知 $\overline{CP}$ 和 X 的波形，试画出 Q_0 和 Q_1 的波形。触发器的初始状态均为 **0**。



图题 5.5.9

5.5.10 SR 触发器是一个具有置 **1** 和清 **0** 功能的触发器，由 D 触发器和逻辑门构成的 SR 触发器如图题 5.5.10 所示，试列出特性表，求特性方程。



图题 5.5.10

5.5.11 试用 T 触发器和必要的门电路实现 D 触发器的逻辑功能。

5.6 用 Verilog HDL 描述锁存器和触发器

5.6.1 试说明下列程序所完成的逻辑功能，并画出它的逻辑图。

```
module d_latch_rst(Rd,control,D,Q);
  input Rd,control,D;
  output reg Q;
  always @(Rd or control or D)
    if(~Rd)Q<=1'b0;
    else if(control)
        Q<=D;
endmodule
```

5.6.2 试用 Verilog 行为建模方式描述一个下降沿触发的 D 触发器，要求具有异步置零功能，即置零信号变为低电平时，将触发器的输出置零。

5.6.3 阅读下列两个程序，画出它们的逻辑图。

(1)

```
module DFF1(Qa,Qb,D,CP);
  input D,CP;
  output reg Qa,Qb;
  always @(posedge CP)
  begin
    Qa=D;
    Qb=Qa;
  end
endmodule
```

(2)

```
module DFF2(Qa,Qb,D,CP);
  input D,CP;
  output reg Qa,Qb;
  always @(posedge CP)
  begin
    Qa<=D;
    Qb<=Qa;
  end
endmodule
```

5.6.4 阅读下列程序，说明它所完成的逻辑功能。

```
module JK_FF(
    output reg Q,
    output Qnot,
    input J,K,CP
    );
    assign Qnot=~Q;//连续赋值
```

```
    always @ ( negedge CP )
    begin
      case( { J,K } )
        2'b00 : Q <= Q ;
        2'b01 : Q <= 1'b0 ;
        2'b10 : Q <= 1'b1 ;
        2'b11 : Q <= ~ Q ;
      endcase
    end
endmodule
```

文档 5.2：
第 5 章部分
习题答案

6 时序逻辑电路

引言

本章将在组合逻辑电路和锁存器、触发器的基础上讨论时序逻辑电路(简称时序电路)。时序电路中除具有逻辑运算功能的组合电路外,还必须有能够记忆电路状态的存储单元或延迟单元,这些存储或延迟逻辑单元主要由锁存器或触发器组成。

本章首先介绍时序电路的基本概念,然后讨论其分析与设计方法。其中,重点侧重于同步时序电路的分析与设计。在明确基本概念和基本方法之后,介绍逻辑设计中常用的典型时序电路模块,最后通过实例介绍用 Verilog 描述时序电路的方法。

文档 6.1:
第 6 章思维导图

6.1 时序逻辑电路的基本概念

日常生活中,时序逻辑的实例并不鲜见,例如楼房电梯的控制便是一个典型的时序逻辑问题。电梯的控制电路需要根据电梯内和各楼层入口处的按键信号,以及电梯当前的状态来决定电梯的升降,同时将电梯当前所处楼层信号输出到电梯内外。这里,按键信号和所到达的楼层信号是时序逻辑的“输入信号”,升降信号和到达楼层的显示则是其“输出信号”。显然,控制电路中必须具有存储单元,以记忆当前电梯所在的楼层。可以定义电梯目前所处楼层为现在的状态,简称“现态”,将要到达的楼层为下一个状态,简称“次态”,楼层的变换即为“状态转换”。电梯的升降不仅取决于当前各按键的输入信号,而且取决于电梯运转的历史状态。例如,电梯正在从低楼层向高楼层上升,若这时电梯内已有人按下更高楼层的按键,或更高的楼层有人召唤,电梯则应向高一层转换而暂时忽略电梯内要求下楼的按键输入或低楼层的召唤信号。这种确定电梯状态如何转换的信号称为“激励信号”。

诸如上例中的输入信号、输出信号、激励信号以及现态、次态及其状态转换的关系是时序电路研究的主要内容。

6.1.1 时序逻辑电路的基本结构与分类

1. 时序电路的基本结构

时序电路的基本结构如图 6.1.1 所示,它由完成逻辑运算的组合电路和具有记忆作用

的存储电路两部分构成，其中，存储电路由触发器或锁存器组成。为了方便，图中各组逻辑变量均以向量形式表示，其中，$I=(I_1,I_2,\cdots,I_i)$为输入信号，$O=(O_1,O_2,\cdots,O_j)$为输出信号，$E=(E_1,E_2,\cdots,E_k)$为驱动存储电路转换为下一状态的激励信号，而$S=(S_1,S_2,\cdots,S_m)$为存储电路状态，称为状态信号，亦称为状态变量，它表示时序电路当前的状态，简称现态。状态变量S被反馈到组合电路的输入端，与输入信号I一起决定时序电路的输出信号O，并产生对存储电路的激励信号E，从而确定电路的下一状态，即次态。于是，上述4组变量间的逻辑关系可用下列三个向量函数形式的方程来表达：

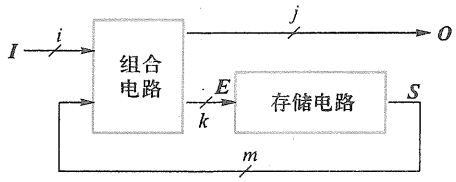


图 6.1.1 时序电路的基本结构

$$E=f(I,S) \tag{6.1.1}$$

$$S^{n+1}=g(E,S^n) \tag{6.1.2}$$

$$O=h(I,S) \tag{6.1.3}$$

式(6.1.1)表达了激励信号与输入信号、状态变量的关系，称为时序电路的激励方程。式(6.1.2)表达了存储电路从现态到次态的转换，故称为状态转换方程。其中，S^n和S^{n+1}分别表示存储电路的现态和次态。注意，为了突出现态到次态的转换关系，本书在写状态转换方程时现态变量S也标出上标n。式(6.1.3)表达了时序电路的输出信号与输入信号、状态变量的关系，称为输出方程。上述三个向量函数形式的方程分别对应于表达时序电路的三个基本方程组：激励方程组、状态转换方程组和输出方程组。

如上所述，时序电路是状态依赖的，故又称为状态机(state machine，SM)。本章将只限于讨论有限数量的存储单元构成的状态机，因而其状态数也是有限的，称为有限状态机(finite state machine，FSM)。

由上述基本结构和方程可见，时序电路具有以下主要特征：

① 时序电路由组合电路和存储电路组成。

② 时序电路的状态与时间因素相关，即时序电路在任一时刻的状态变量不仅是输入信号的函数，而且还是电路以前状态的函数，并由当前输入变量和状态决定电路的下一状态。

③ 时序电路的输出信号由输入信号和电路状态共同决定。

2. 异步与同步时序电路

时序电路可分为异步时序电路和同步时序电路两大类。

若电路中触发器的时钟输入端没有连接在统一的时钟脉冲上，或电路中没有时钟脉冲(如SR锁存器构成的时序电路)，从而，电路中各存储单元的状态更新不是同时发生的，则这种电路称为异步时序电路。根据电路是对脉冲边沿敏感还是对电平敏感，异步时序电路又分为脉冲异步时序电路(由触发器构成)和电平异步时序电路(由锁存器构成)两种。

与异步时序电路不同，同步时序电路中存储电路状态的转换是在同一时钟脉冲源的同一边沿作用下同步动作的，它也称作时钟同步状态机(clocked synchronous SM)，其结构如图6.1.2所示。同步时序电路的存储电路一般用触发器实现，所有触发器的时钟输入端都接在同一个时钟脉冲源上，其状态更新是在同一时刻。在非时钟有效沿期间，触发器处于保持状态。因此，根据输入信号及时钟作用下各个触发器的状态变化规律就可以对电路进行分析

设计。所以，目前结构较复杂、运行速度较高的时序电路广泛采用同步方式来实现。

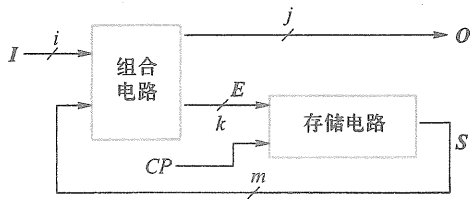


图 6.1.2 同步时序电路的结构

3. 米利型和穆尔型时序电路

米利(Mealy)型时序电路的结构如图 6.1.3 所示，是将图 6.1.2 中的组合电路拆解成激励组合电路和输出组合电路两部分。它的输出信号 O 是状态变量 S 和输入信号 I 二者的函数，即 $O=h(I,S)$。电路在非时钟有效沿期间，输入信号变化时，输出信号可能会发生变化，即不能保证输出信号始终与时钟同步。

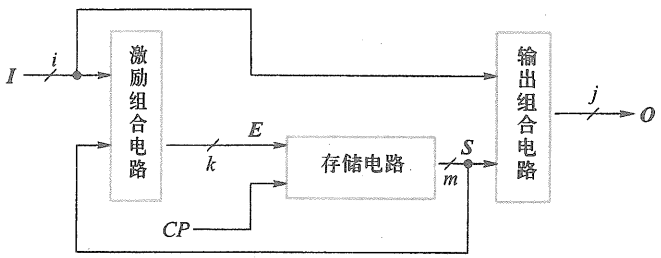


图 6.1.3 米利型时序电路的结构

穆尔(Moor)型时序电路是米利型时序电路的一种特例，它的输出信号 O 仅仅是状态变量 S 的函数，即 $O=h(S)$，其结构如图 6.1.4 所示。穆尔型时序电路的输出信号只取决于各触发器的状态，与输入信号没有直接联系。

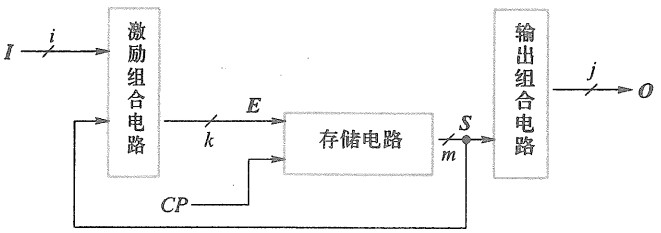


图 6.1.4 穆尔型时序电路的结构

在现代高速时序电路设计中，一般尽量采用穆尔型时序电路结构，以利于后续高速电路的同步。在米利型时序电路的输出端增加一级存储电路，构成“流水线输出”形式，是将其转化为穆尔型电路的最简单方法，电路结构如图 6.1.5 所示。需要注意的是，流水线存储电路将把输出信号延迟一个时钟周期。虽然电路增加了一些逻辑元件，但它的输出信号同步特性更好，具有更好的稳定性和抗干扰性能。

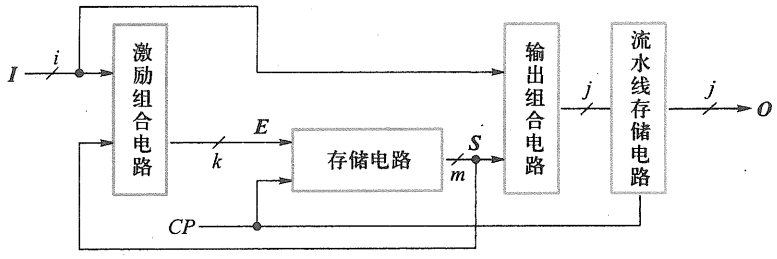


图 6.1.5 流水线输出的米利型时序电路的结构

6.1.2 时序逻辑电路功能的表达

从理论上讲，有了激励方程组、状态转换方程组和输出方程组，就可以唯一确定时序电路的功能。但很多时候，仅从这三组方程并不容易判断其逻辑功能，在设计时序电路时，也很难根据给出的逻辑需求直接写出这三组方程。因此，还需要借助能够直观反映电路状态变化全过程的**状态转换表**和**状态转换图**。三组方程、状态转换表和状态转换图之间可以相互转换，根据其中任意一种表达方式，都可以画出**时序图**。换言之，时序电路的功能可用逻辑方程组、状态转换表、状态转换图和时序图等形式来表达，也可以用 HDL 语言描述。下面通过实例介绍时序电路逻辑功能的前四种表达方法，使用 Verilog HDL 描述时序电路的方法将在 6.7 节讨论。

图 6.1.6 所示电路由组合电路和存储电路两部分组成。其中，存储电路由共用时钟信号 CP 的两个 D 触发器 FF_1、FF_0构成，因此是同步时序电路。组合电路包括激励电路和输出电路两部分。A 为输入信号，X、Y 为输出信号。D_1和 D_0为触发器的激励信号，Q_1、Q_0为电路

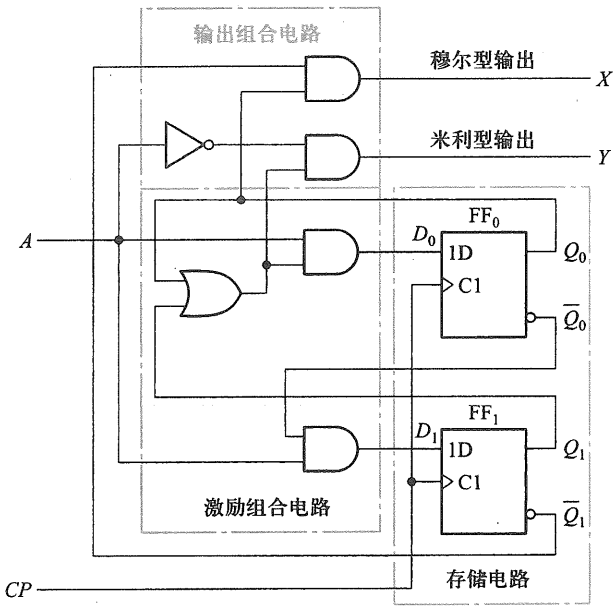


图 6.1.6 同步时序电路一例

的状态变量。从图中可以看出，输出信号 Y 是状态变量 Q_1、Q_0和输入信号 A 的函数，所以，Y 是米利型输出，而输出信号 X 只由状态变量 Q_1、Q_0决定，所以 X 是穆尔型输出。

1. 逻辑方程组

（1）激励方程组

根据图 6.1.6 可写出两个 D 触发器的激励方程组

$$D_0=(Q_1+Q_0)A \tag{6.1.4}$$

$$D_1=\overline{Q_0}A \tag{6.1.5}$$

（2）状态转换方程组

将式(6.1.4)和式(6.1.5)分别代入 D 触发器的特性方程$Q^{n+1}=D$，于是，得到状态转换方程组

$$Q_0^{n+1}=(Q_1^n+Q_0^n)A \tag{6.1.6}$$

$$Q_1^{n+1}=\overline{Q_0^n}A \tag{6.1.7}$$

以上两式表明，触发器的次态Q_0^{n+1}和Q_1^{n+1}是输入变量 A 和触发器现态Q_0^n、Q_1^n的函数。

（3）输出方程组

根据图 6.1.6 的输出组合电路得到输出方程组

$$X=\overline{Q_1Q_0} \tag{6.1.8}$$

$$Y=(Q_1+Q_0)\overline{A} \tag{6.1.9}$$

可见，X 是穆尔型输出，Y 是米利型输出。注意，输出方程组中的触发器状态均为现态，即$Q_1=Q_1^n$，$Q_0=Q_0^n$。

上述三组方程中，激励方程组和输出方程组表达了其中组合电路的特性，而状态转换方程组则表达了存储电路从现态到次态的状态转换特性。

2. 状态转换表

与组合电路分析方法类似，根据逻辑表达式(6.1.6)～式(6.1.9)可以列出真值表如表 6.1.1 所示。Q_1^n、Q_0^n和 A 为输入变量，Q_1^{n+1}、Q_0^{n+1}和 X、Y 为输出变量。由于该真值表反映了触发器从现态到次态的转换，故称为状态转换真值表。

表 6.1.1 图 6.1.6 电路的状态转换真值表

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	X	Y
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	0	0	1
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	0	1	0	0

使用时序图时需要注意，有时它并不完全表达出电路状态转换的全部过程，而是根据需要仅画出部分典型的波形图，如图 6.1.8 中就没有表达出当状态 Q_1Q_0为 **11** 而输入 A 为 **0** 时的状态转换波形和输出波形。

复习思考题

6.1.1 时序电路由哪几部分组成？它和组合电路在逻辑功能和结构上有什么区别？

6.1.2 异步时序电路与同步时序电路有哪些不同的特性？

6.1.3 时序电路的逻辑功能表达方法有哪几种？哪种可直接从逻辑图得出？哪几种方法可直接表示出时序电路状态转换的全部过程？哪种方法可观察到电路的输入信号、触发器状态和输出信号的波形并在实验和 HDL 仿真时应用？

6.1.4 米利型和穆尔型时序电路的输出特性有何不同？米利型输出和穆尔型输出在状态转换表和状态转换图中的表达有何不同？

6.2 同步时序逻辑电路的分析

同步时序电路的分析实际上是一个读图、识图的过程：按照给定的时序电路，通过分析其状态和输出信号在输入变量和时钟作用下的转换规律，理解其逻辑功能和工作特性。下面首先介绍分析同步时序电路的一般步骤，然后通过例题加深对分析方法的理解。

6.2.1 分析同步时序逻辑电路的一般步骤

（1）根据给定的同步时序电路，写出下列逻辑方程组：

① 写出每个触发器的激励方程，得到激励方程组；

② 将激励方程代入相应触发器的特性方程，得到各触发器的状态转换方程，获得状态转换方程组；

③ 写出每个输出变量的输出方程，组成输出方程组。

（2）根据状态转换方程组和输出方程组，列出电路的状态转换表，画出状态转换图和时序图。

（3）确定电路的逻辑功能，必要的话，可用文字详细描述。

上述步骤是分析同步时序电路的一般化过程，实际分析中可根据具体情况调整，最终达到明确电路的功能即可。

6.2.2 同步时序逻辑电路分析举例

例 6.2.1 分析图 6.2.1 所示同步时序电路的逻辑功能。

解：这是一个由两个 T 触发器和两个与门组成的米利型同步时序电路，分析如下。

（1）根据电路列出三个方程组

① 激励方程组

$$T_0 = A$$

$$T_1 = AQ_0$$

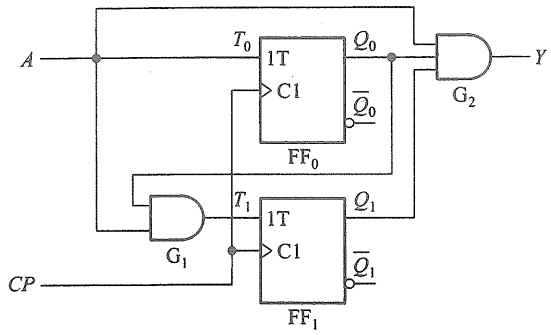


图 6.2.1 例 6.2.1 的逻辑电路图

② 状态转换方程组

将上述激励方程代入 T 触发器的特性方程 $Q^{n+1}=T\oplus Q^n=T\overline{Q^n}+\overline{T}Q^n$，即得状态转换方程组

$$Q_0^{n+1}=A\oplus Q_0^n$$

$$Q_1^{n+1}=(AQ_0^n)\oplus Q_1^n$$

③ 输出方程组

$$Y=AQ_1Q_0$$

(2) 列出状态转换表

首先在状态转换表中列出电路所有可能出现的现态，即$Q_1^nQ_0^n$的 4 种取值：00、01、10、11，以及输入变量的所有取值：$A=0$ 和 $A=1$。然后将现态和输入逻辑值一一代入上述状态转换方程组和输出方程组，分别求出次态和输出逻辑值。例如，将$Q_1^n=Q_0^n=A=0$ 分别代入两个状态转换方程，得到$Q_1^{n+1}=0$ 和$Q_0^{n+1}=0$；同时代入输出方程，得 $Y=0$。于是在表中对应$Q_1^nQ_0^n=00$的行与对应 $A=0$ 的列的交叉位置填入 00/0。其余以此类推，最后列出的状态转换表如表 6.2.1 所示。

表 6.2.1 例 6.2.1 的状态转换表

$Q_1^nQ_0^n$	$Q_1^{n+1}Q_0^{n+1}/Y$	
	$A=0$	$A=1$
0 0	0 0/0	0 1/0
0 1	0 1/0	1 0/0
1 0	1 0/0	1 1/0
1 1	1 1/0	0 0/1

(3) 画出状态转换图

由状态转换表即可画出状态转换图，如图 6.2.2 所示。

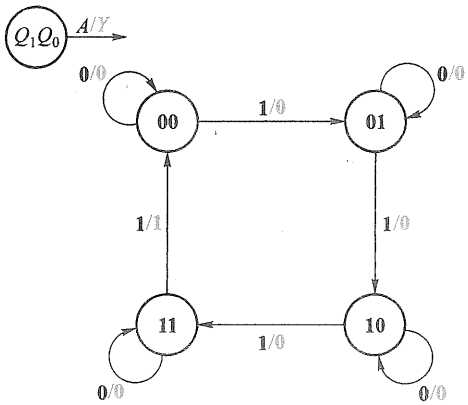


图 6.2.2 例 6.2.1 的状态转换图

(4) 画出时序图

设电路的初始状态为 $Q_1Q_0=00$,根据状态转换表和状态转换图,以及输入信号波形,可画出在一系列 CP 脉冲作用下电路的时序图,如图 6.2.3 所示。

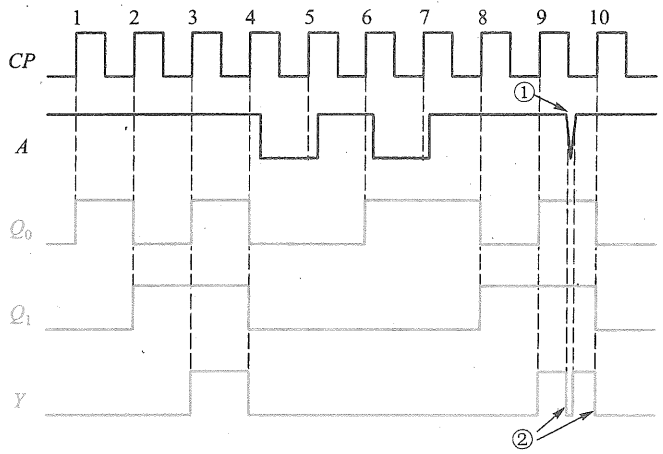


图 6.2.3 例 6.2.1 的时序图

(5) 逻辑功能分析

观察状态转换图和时序图可知,图 6.2.1 的电路是一个由信号 A 控制的可控二进制计数器,CP 为计数脉冲。当 $A=0$ 时,电路处于保持状态;当 $A=1$ 时,在 CP 上升沿到来后电路状态值加 1,当计数到 11 状态,且 $A=1$,Y 输出 1,电路状态将在下一个 CP 上升沿回到 00。输出信号 Y 的下降沿可用于触发进位操作。

该电路亦可作为序列信号检测器,用来检测同步脉冲信号序列 A 中 1 的个数,一旦检测到四个 1 状态(这四个 1 状态可以不连续),电路输出 Y 则出现一次从 1 到 0 的跳变。

观察图 6.2.3 的时序图,在第 9 个和第 10 个 CP 脉冲之间,由于外界干扰,造成输入信号 A 出现短时间的 0 电平“毛刺”,如图中箭头①所示,使 Y 发生变化,将产生两次进位触发脉冲沿,如图 6.2.3 中箭头②所示。如果删除图 6.2.1 中 A 和与门 G_2 输入之间的连线,将电

路转化为穆尔型，则输出信号 Y 仅取决于电路的状态，可以避免输入信号 A 对 Y 的直接影响。从而提高了电路的抗干扰性能。当然，修改后的电路逻辑功能也将稍有变化。

例 6.2.2　分析图 6.2.4 所示的同步时序电路。

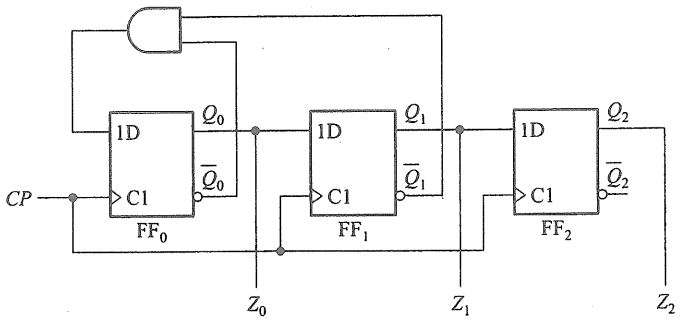


图 6.2.4　例 6.2.2 的逻辑电路图

解：由图 6.2.4 可见，该同步时序电路没有输入信号，输出为三个触发器的状态，是穆尔型时序电路。

（1）根据电路列出逻辑方程组

D 触发器的特性方程为 $Q^{n+1}=D$，根据逻辑电路可以直接列出状态转换方程组。

① 状态转换方程组

$$Q_0^{n+1}=D_0=\overline{Q_1^n}\,\overline{Q_0^n}$$

$$Q_1^{n+1}=D_1=Q_0^n$$

$$Q_2^{n+1}=D_2=Q_1^n$$

② 输出方程组

$$Z_0=Q_0$$

$$Z_1=Q_1$$

$$Z_2=Q_2$$

（2）列出状态转换表

电路中没有输入信号，同时，输出 Z_2、Z_1、Z_0 就是各触发器的状态，不需要单列输出栏。状态转换表可简化为表 6.2.2 所示形式。

表 6.2.2　例 6.2.2 的状态转换表

$Q_2^nQ_1^nQ_0^n$	$Q_2^{n+1}Q_1^{n+1}Q_0^{n+1}$
0 0 0	0 0 1
0 0 1	0 1 0
0 1 0	1 0 0
0 1 1	1 1 0
1 0 0	0 0 1

续表

$Q_2^nQ_1^nQ_0^n$	$Q_2^{n+1}Q_1^{n+1}Q_0^{n+1}$
101	010
110	100
111	110

(3) 画出状态转换图

根据状态转换表可画出电路的状态转换图如图 6.2.5 所示。由图可见,001、010、100 三个状态形成闭合回路,并按照箭头方向循环变化。这三个状态称为**有效状态**,其余的五个状态则称为**无效状态**。如果电路处于任何一个无效状态,经过若干时钟脉冲后,都可以自动进入有效状态,则称其具有**自校正能力**。因此,该电路是具有自校正能力的同步时序电路。

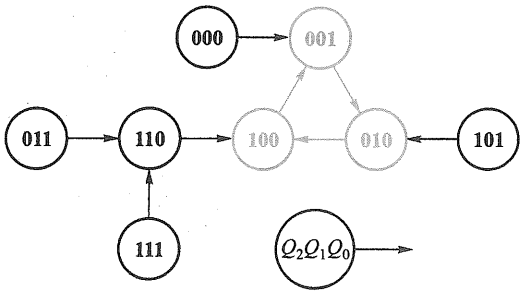


图 6.2.5 例 6.2.2 的状态转换图

(4) 画出时序图

设电路的初始状态为 $Q_2Q_1Q_0=000$,根据状态转换表或状态转换图,可画出时序图如图 6.2.6 所示。

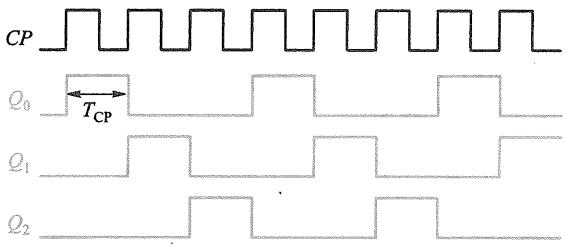


图 6.2.6 例 6.2.2 的时序图

(5) 逻辑功能分析

由状态转换图可见,电路的有效状态是三位循环码。从时序图可以看出,电路正常工作时,各触发器的 Q 端轮流出现一个脉冲信号,其宽度为一个 CP 周期,即 $1T_{CP}$,循环周期为 $3T_{CP}$,可以看作在 CP 脉冲作用下,电路把宽度为 $1T_{CP}$ 的脉冲依次分配给 Q_0、Q_1、Q_2 三个输出端,因此,电路的功能为脉冲分配器或节拍脉冲产生器。

例 6.2.3 分析图 6.2.7 所示同步时序电路。

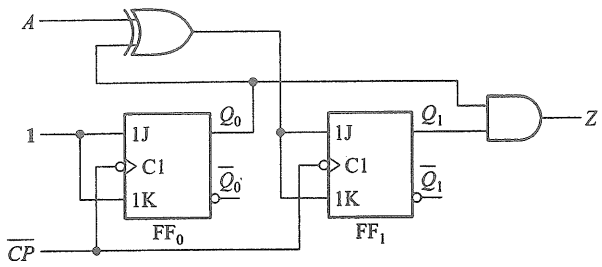


图 6.2.7 例 6.2.3 的逻辑电路图

解：这是一个由两个下降沿触发的 *JK* 触发器、一个**异或**门及一个**与**门组成的穆尔型同步时序电路。

(1) 根据电路列出三个方程组

① 激励方程组

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = A \oplus Q_0$$

② 状态转换方程组

将激励方程代入 *JK* 触发器的特性方程，得到状态转换方程组

$$Q_0^{n+1} = J_0\overline{Q_0^n} + \overline{K}_0 Q_0^n = \overline{Q_0^n}$$

$$\begin{aligned} Q_1^{n+1} &= J_1\overline{Q_1^n} + \overline{K}_1 Q_1^n \\ &= (A \oplus Q_0^n)\overline{Q_1^n} + \overline{A \oplus Q_0^n} Q_1^n \\ &= A \oplus Q_0^n \oplus Q_1^n \end{aligned}$$

③ 输出方程组

$$Z = Q_1 Q_0$$

(2) 列出状态转换表

根据状态转换方程组和输出方程组可以列出状态转换表如表 6.2.3 所示。

表 6.2.3 例 6.2.3 的状态转换表

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1}$		Z
	$A=0$	$A=1$	
0 0	0 1	1 1	0
0 1	1 0	0 0	0
1 0	1 1	0 1	0
1 1	0 0	1 0	1

(3) 画出状态转换图

根据状态转换表可以画出状态转换图如图 6.2.8 所示。

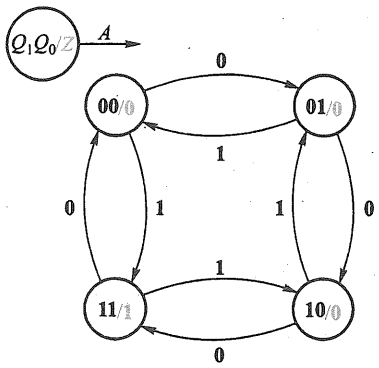


图 6.2.8 例 6.2.3 的状态转换图

(4) 画出时序图

设电路的初始状态为 $Q_1Q_0=00$,根据状态转换表、状态转换图和输入信号波形,可画出在一系列$\overline{CP}$脉冲作用下的时序图如图 6.2.9 所示。

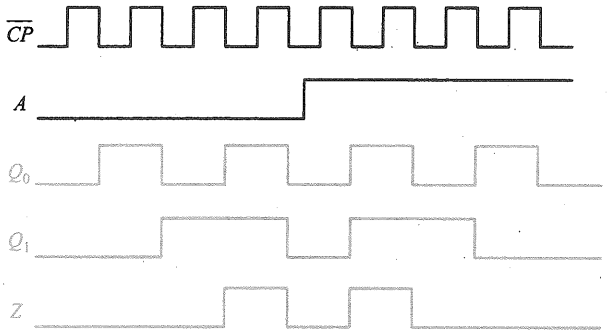


图 6.2.9 例 6.2.3 电路的时序图

(5) 逻辑功能分析

由状态转换图可以看出,图 6.2.7 所示电路是一个可逆二进制计数器。当 $A=0$ 时,进行递增计数,在$\overline{CP}$信号作用下,Q_1Q_0按照 00→01→10→11 变化,相当于加 1 并计数,且 4 个$\overline{CP}$循环一次。当 $A=1$ 时,进行递减计数,依次为 11→10→01→00。Z 端在 Q_1Q_0为 11 时输出 1。在递增计数时,Z 信号的下降沿可以触发进位操作;在递减计数时,Z 信号的上升沿可触发借位操作。有关计数器的详细内容将在 6.5.2 节讨论。

复习思考题

6.2.1 同步时序电路的分析过程可分为哪几个步骤?

6.2.2 在图 6.2.9 中,为什么说在递增计数时,Z 信号的下降沿可以触发进位操作,而在递减计数时 Z 信号的上升沿可触发借位操作?

6.3 同步时序逻辑电路的设计

时序电路设计又称为时序电路综合,其任务是根据给定的逻辑功能需求,选择适当的逻

辑器件，设计出符合要求的时序电路。本节讨论的用触发器及门电路设计同步时序电路的方法是时序电路设计的基础，也是用 Verilog HDL 描述时序电路及使用可编程逻辑器件实现时序电路的基础。

6.3.1 设计同步时序逻辑电路的一般步骤

设计同步时序电路的一般过程如图 6.3.1 所示。

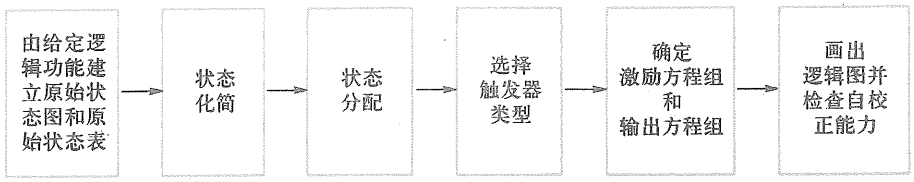


图 6.3.1 同步时序电路的设计过程

下面对设计过程中的主要步骤加以说明。

1. 由给定的逻辑功能建立原始状态转换图和原始状态转换表

通常，所要设计的时序电路的逻辑功能是通过文字、图形或波形图来描述的，首先对实际问题进行分析，把它们变换成状态转换图和状态转换表。此时的状态转换图和状态转换表分别称为原始状态转换图和原始状态转换表。具体做法是：

(1) 分析电路的逻辑问题描述，确定输入变量和输出变量的数目和名称。一般时钟脉冲 CP(或 $\overline{CP}$)不作为输入变量。

(2) 找出所有可能的状态以及状态转换之间的关系和输入条件。定义电路每个状态的名称及含义。从设定的初始状态开始，根据输入条件确定转换方向(次态)和输出，直到画出从每一个状态出发向下一个状态的转换和输出，便完成了原始状态转换图的建立。

(3) 根据原始状态转换图建立原始状态转换表。

2. 状态化简

原始状态转换图或原始状态转换表很可能隐含多余的状态，去除多余状态的过程称为状态化简，其目的是减少电路中触发器及门电路的数量。状态化简建立在等价状态的基础上：如果两个状态在相同输入下的次态及输出完全相同，则这两个状态称为等价状态。将两个等价状态合并成一个状态不会改变输入-输出关系。

3. 状态分配

为每个状态指定一个特定的二进制代码，称为状态分配或状态编码。根据状态数确定触发器的数目。N 个触发器有 2^N 个状态。若电路的状态数为 M，则应满足

$$2^{N-1}<M\leqslant 2^N \tag{6.3.1}$$

从 2^N 个状态中取 M 个状态组合可能有多种不同方案，随着 N 值的增大，编码方案的数目会急剧增多①。编码方案选择得当，设计出的电路结构相对简单、可靠。采用自然二进制

① 研究证明，从 N 位编码中取 M 个状态，其可能的状态分配方案数目为 $S=\dfrac{2^N!}{(2^N-M)!}$。例如，$N=3$，$M=5$，其可能的编码方案总数达 6 720 之多！

数编码有时可以简化设计过程。而采用格雷码，可降低竞争-冒险的产生，同时在状态数相同情况下，触发器的翻转次数少，动态功耗低。

完成状态分配后，就可将状态转换表中的状态名替换为状态编码。

4. 选择触发器类型

逻辑功能不同的触发器，其驱动方程不同，所设计出的电路也不同，要根据需要选择触发器类型。在可编程逻辑器件中采用 D 触发器来实现时序电路设计，而且 D 触发器也非常容易构成其他逻辑功能的触发器，从而满足一些特殊要求。

5. 确定激励方程组和输出方程组

根据状态转换表，用卡诺图或其他方式对逻辑函数进行化简，可求得电路的激励方程组和输出方程组。这两个方程组决定了同步时序电路的组合电路部分。

6. 画出逻辑图，并检查自校正能力

根据激励方程组和输出方程组，可画出逻辑电路图。

有些电路会有无效状态，当电路上电后有可能陷入这些无效状态而不能退出，因此，设计的最后一步应检查电路是否能进入有效状态，即是否具有自校正能力。如果不能自校正，则需修改设计。

有些时序电路要求必须从指定的初始状态开始工作。这时，应利用触发器的直接置 0、置 1 功能，将电路置为初始有效状态。图 6.3.2(a)所示为一低电平手动复位信号产生电路，当按键开关按下，$\overline{RESET}$产生低电平，使电路进入初始状态。按键抬起，$\overline{RESET}$为高电平，不影响电路正常工作。图 6.3.2(b)是一种上电自动复位电路，它利用电容两端电压不能突变的原理，在上电之初复位信号$\overline{RESET}$为低电平，将系统预置为初始状态。随着电源 V_{CC} 通过电阻 R 对电容 C 充电，使 C 两端电压逐渐升高，经过一段时间之后，$\overline{RESET}$信号变为高电平，使系统脱离复位状态而进入正常工作状态。

需要说明的是，上述步骤是设计同步时序电路的一般化过程，实际设计中并不是每一步都必须执行，可根据具体情况进行调整。

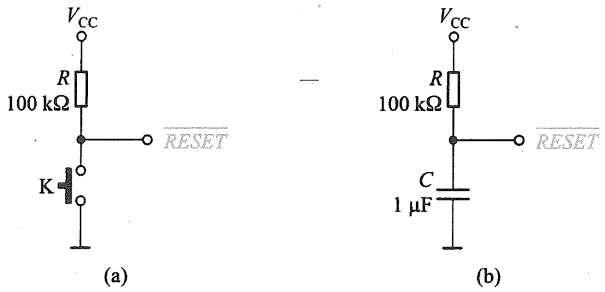


图 6.3.2 复位电路

(a) 手动复位电路 (b) 上电自动复位电路

6.3.2 同步时序逻辑电路设计举例

例 6.3.1 用 D 触发器设计一个同步递增 8421BCD 计数器。

解： 计数器实际上是对时钟脉冲进行计数，在时钟脉冲作用下，计数器的状态按照

8421BCD 编码值加 1，从 0000 到 1001 共有 10 个有效状态。由于电路的状态数、状态转换关系及状态编码等都是明确的，因此可以简化设计步骤。

（1）列出状态转换表

10 个状态的计数器共需要 4 个 D 触发器构成。由于 D 触发器功能较简单，可将激励信号同时列入状态转换表中，如表 6.3.1 所示。这是一个穆尔型时序电路，输出变量即状态编码。

表 6.3.1 同步 8421BCD 计数器的状态转换表

Q_3^n	Q_2^n	Q_1^n	Q_0^n	$Q_3^{n+1}(D_3)$	$Q_2^{n+1}(D_2)$	$Q_1^{n+1}(D_1)$	$Q_0^{n+1}(D_0)$
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

（2）确定激励方程组

按表 6.3.1 可画出各触发器激励信号的卡诺图如图 6.3.3 所示。

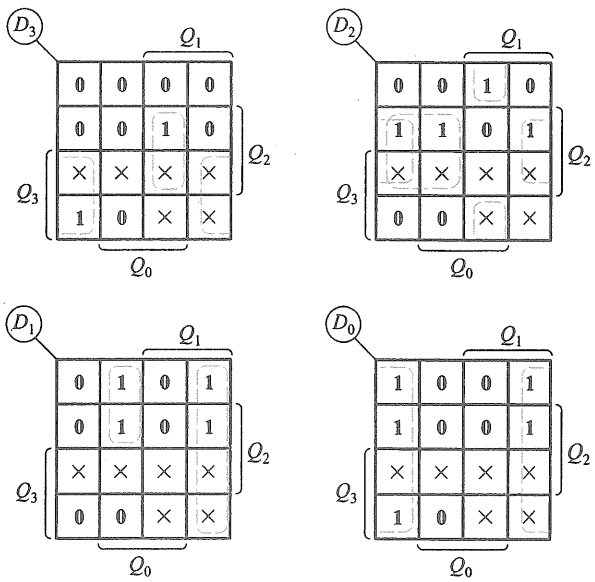


图 6.3.3 例 6.3.1 的卡诺图

4 个触发器输出从 **0000** 到 **1111** 有 16 个组合状态，其中 **1010**～**1111** 的 6 个状态在 8421BCD 中是无效状态，在图 6.3.3 的卡诺图中以无关项×表示。于是，得到激励方程组（在本例中也是状态转换方程组）

$$Q_3^{n+1}=D_3=Q_3^n\overline{Q_0^n}+Q_2^nQ_1^nQ_0^n$$

$$Q_2^{n+1}=D_2=Q_2^n\overline{Q_1^n}+Q_2^n\overline{Q_0^n}+\overline{Q_2^n}Q_1^nQ_0^n$$

$$Q_1^{n+1}=D_1=Q_1^n\overline{Q_0^n}+\overline{Q_3^n}\,\overline{Q_1^n}Q_0^n$$

$$Q_0^{n+1}=D_0=\overline{Q_0^n}$$

（3）画出逻辑图

根据激励方程组可画出逻辑图如图 6.3.4 所示。图中，各触发器的直接置 **0** 端为低电平有效，电路工作时，$\overline{RESET}$输入端应保持为高电平。

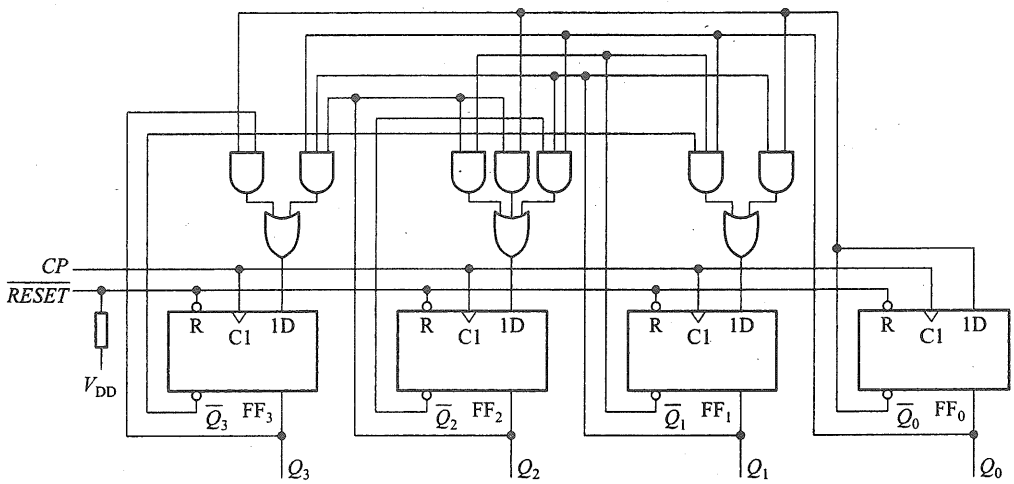


图 6.3.4 例 6.3.1 的逻辑电路

（4）检查自校正能力

将电路的 6 个无效状态 **1010**、**1011**、**1100**、**1101**、**1110** 和 **1111** 作为现态，代入电路的状态转换方程组而求其次态。如果还没有进入有效状态，再以新的状态作为现态求下一个次态，以此类推，看最终能否进入有效状态。结果证明，这 6 个状态在一或两个时钟周期后全部都能进入有效循环状态。

（5）画出状态转换图

图 6.3.4 所示电路的完全状态转换图如图 6.3.5 所示。

如果要求电路必须从 **0000** 开始计数，则可将前述复位电路连接在$\overline{RESET}$输入端。$\overline{RESET}$产生低电平脉冲，强制电路进入 **0000** 的初始状态，待$\overline{RESET}$=**1** 后再开始计数。

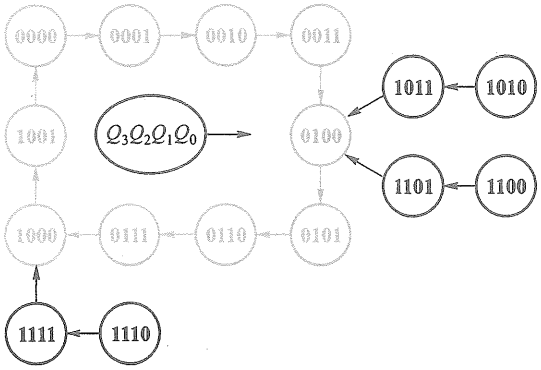


图 6.3.5 图 6.3.4 所示电路的完全状态转换图

例 6.3.2 试设计一序列编码检测器，当检测到输入信号出现 110 序列编码（按自左至右的顺序）时，电路输出为 1，否则输出为 0。要求用同步时序电路实现。

解：（1）由给定的逻辑功能建立原始状态转换图和原始状态转换表

从给定的逻辑功能可知，电路有一个输入信号 A 和一个输出信号 Y，电路功能是对输入信号 A 的编码序列进行检测，当检测到 A 连续输入 110 序列时，输出为 1，否则输出为 0。

设电路的初始状态为 a，如图 6.3.6 所示。在此状态下，电路输出 $Y=0$，这时可能的输入有 $A=0$ 和 $A=1$ 两种情况。若 $A=0$，电路应保持在状态 a 不变；若 $A=1$，则转向状态 b，表示电路收到一个 1。当在状态 b 时，若输入 $A=0$，则表明连续输入编码为 10，不是 110，则应回到初始状态 a，重新开始检测；若 $A=1$，则进入状态 c，表示已连续收到两个 1。在状态 c 时，若 $A=0$，表明已收到序列编码 110，则输出 $Y=1$，并进入状态 d；若 $A=1$，则收到的编码为 111，应保持在状态 c 并且输出仍为 0。在状态 d，若输入 $A=0$，则应回到状态 a，重新开始检测；若 $A=1$，电路应转向状态 b，表示在收到 110 之后又重新收到一个 1，已进入下一轮检测；在 d 状态下，无论 A 为何值，输出 Y 均为 0。根据上述分析，可以得出如图 6.3.6 所示的原始状态转换图和表 6.3.2 的原始状态转换表。

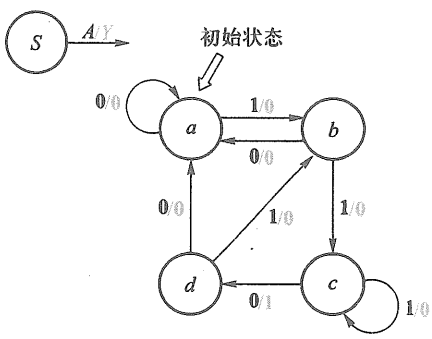


图 6.3.6 例 6.3.2 的原始状态转换图

表 6.3.2 例 6.3.2 的原始状态转换表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$a/0$	$c/0$
c	$d/1$	$c/0$
d	$a/0$	$b/0$

(2) 状态化简

观察表 6.3.2 中 a 和 d 两行可以看出,在 $A=0$ 和 $A=1$ 时,分别具有相同的次态 a、b 及相同的输出 0,因此,a 和 d 是等价状态,可以合并。这里选择去除 d 状态,并将其他行中的次态 d 改为 a。于是,得到化简后的状态转换表如表 6.3.3 所示,状态转换图亦可作相应化简。从实际物理意义看也不难理解这种化简:当进入 c 状态后,电路已连续接收到两个 1,这时输入若为 0,则意味着已接收到编码 110,下一步电路应回到初始状态 a,以准备新的一轮检测,原始状态转换表中的 d 状态显然是多余的(米利型电路)。

表 6.3.3 例 6.3.2 化简后的状态转换表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$a/0$	$c/0$
c	$a/1$	$c/0$

(3) 状态分配

化简后有三个状态,可以用两位二进制代码组合(00,01,10,11)中的任意三个代码表示,用两个触发器组成电路。

这里采取 00→01→11→00 的变化顺序,于是,令 $a=00$,$b=01$,$c=11$,状态分配后得到的状态转换图如图 6.3.7 所示。

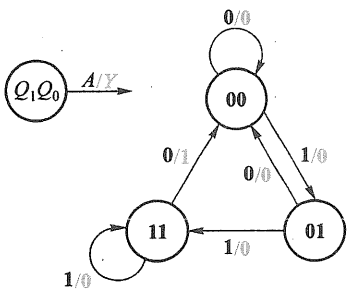


图 6.3.7 例 6.3.2 状态分配后的状态转换图

(4) 选择触发器类型

这里选用 JK 触发器实现。

(5) 确定激励方程组和输出方程组

用 JK 触发器设计时序电路时,电路的激励方程需要间接导出。根据表 5.5.2 所示 JK 触发器的特性表,以给定的状态转换为条件,列出所需要的输入信号,得到 JK 触发器的**激励表**,如表 6.3.4 所示。表中的×表示其逻辑值与该行的状态转换无关。

表 6.3.4 JK 触发器的激励表

Q^n	Q^{n+1}	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0

根据图 6.3.7 和表 6.3.4 可以列出状态转换真值表及两个触发器所要求的激励信号如表 6.3.5 所示。据此，分别画出两个触发器的输入 J、K 和电路输出 Y 的卡诺图，如图 6.3.8 所示。图中，不使用的状态均以无关项×填入。化简后得到激励方程组

$$J_1 = AQ_0 \qquad K_1 = \overline{A}$$

$$J_0 = A \qquad K_0 = \overline{A}$$

和输出方程

$$Y = \overline{A}Q_1$$

表 6.3.5 例 6.3.2 的状态转换真值表及激励信号

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	Y	激励信号			
						J_1	K_1	J_0	K_0
0	0	0	0	0	0	0	×	0	×
0	0	1	0	1	0	0	×	1	×
0	1	0	0	0	0	0	×	×	1
0	1	1	1	1	0	1	×	×	0
1	1	0	0	0	1	×	1	×	1
1	1	1	1	1	0	×	0	×	0

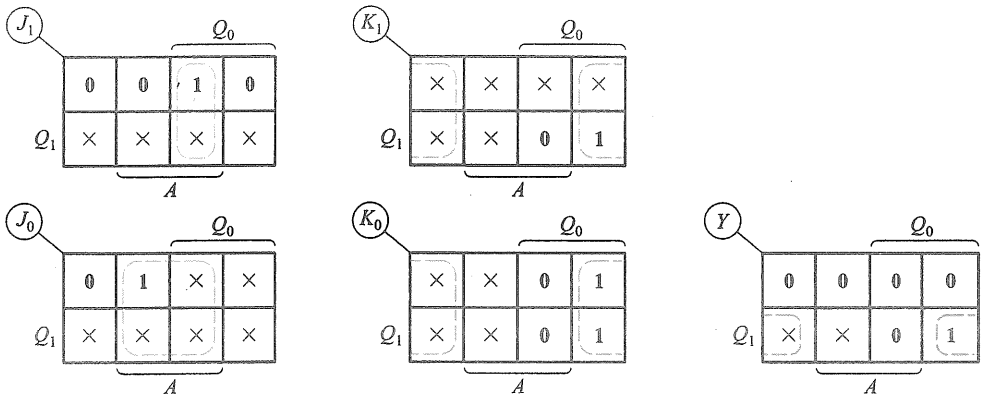


图 6.3.8 激励信号及输出信号的卡诺图

(6) 画出逻辑图

根据激励方程组和输出方程画出逻辑图，如图 6.3.9 所示。

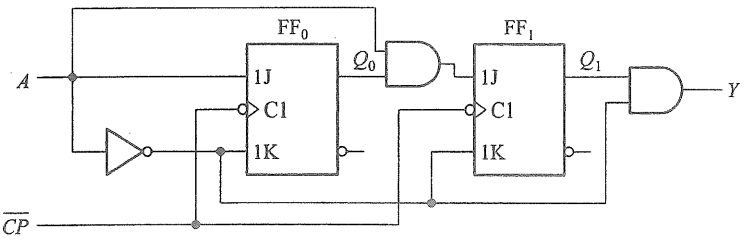


图 6.3.9 例 6.3.2 的逻辑图

(7) 检查自校正能力

当电路进入无效状态 10 后,由激励方程组和输出方程可知,若 $A=0$,则次态为 00;若 $A=1$,则次态为 11,电路能自动进入有效序列。

如果发现所设计的电路不能自校正,则应修改设计。通常取消利用无关项的化简都可以使问题得到解决。

例 6.3.3 试用 D 触发器设计一同步时序电路,实现图 6.3.10 所示原始状态转换图的逻辑功能。

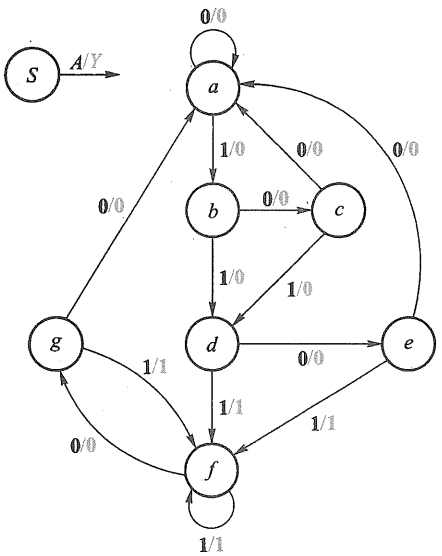


图 6.3.10 例 6.3.3 的原始状态转换图

解:(1) 列出原始状态转换表

根据图 6.3.10 列出原始状态转换表如表 6.3.6 所示。

表 6.3.6 例 6.3.3 的原始状态转换表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$c/0$	$d/0$
c	$a/0$	$d/0$
d	$e/0$	$f/1$
e	$a/0$	$f/1$
f	$g/0$	$f/1$
g	$a/0$	$f/1$

(2) 状态化简

观察表 6.3.6 发现,状态 e、g 是等价状态,可以合并。表 6.3.7 示出第一步化简的结果:

将状态 g 一行删除，并用状态 e 替换表 6.3.6 中“S^{n+1}/Y”栏中的状态 g。

表 6.3.7 表 6.3.6 的第一步化简

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$c/0$	$d/0$
c	$a/0$	$d/0$
d	$e/0$	$f/1$
e	$a/0$	$f/1$
f	$e/0$	$f/1$

再观察表 6.3.7，又出现状态 d 和 f 是等价的，状态 f 亦可去除，代之以 d。于是，得到表 6.3.8 所示状态转换表。检查该表，已不存在等价状态，因此是最简状态转换表。根据表 6.3.8 画出的状态转换图如图 6.3.11 所示。

表 6.3.8 例 6.3.3 的最简状态转换表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$c/0$	$d/0$
c	$a/0$	$d/0$
d	$e/0$	$d/1$
e	$a/0$	$d/1$

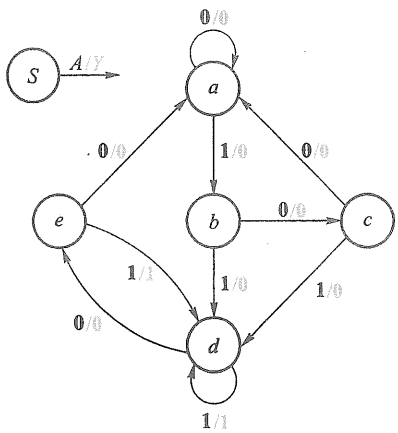


图 6.3.11 化简后的状态转换图

上述状态化简过程将原有的 7 个状态化简为 5 个。

(3) 状态分配

表 6.3.8 中列出 5 个状态,取二进制计数序列的前 5 个连续编码,如表 6.3.9 中的状态分配方案 1。

表 6.3.9 例 6.3.3 的三种状态分配方案

状态	方案 1 自然二进制码	方案 2 格雷码	方案 3 "一对一"
a	0 0 0	0 0 0	0 0 0 0 1
b	0 0 1	0 0 1	0 0 0 1 0
c	0 1 0	0 1 1	0 0 1 0 0
d	0 1 1	0 1 0	0 1 0 0 0
e	1 0 0	1 1 0	1 0 0 0 0

表 6.3.9 中所列方案 2 为格雷码方案。如果状态转换图是简单地从 a 到 e,那么它从一个状态转换到下一状态仅有一个触发器改变状态,使用这种方案将降低电路发生竞争-冒险的可能,提高电路的可靠性。但本例的状态转换并不是简单的单循环序列。

表 6.3.9 中的方案 3 为"一对一"编码方案①。虽然这将使用较多的触发器,但它的编码方式非常简单,可有效地简化组合电路,并换得工作可靠性和工作速度的提高。在大规模可编程逻辑器件,如 FPGA 中,触发器数量较多而门逻辑相对较少,"一对一"的编码方案有时反而更有利于提高器件资源的利用率。

本例按表 6.3.9 中方案 1 分配状态,得到的状态转换表如表 6.3.10 所示。

表 6.3.10 例 6.3.3 的状态转换表

$Q_2^nQ_1^nQ_0^n$	$Q_2^{n+1}Q_1^{n+1}Q_0^{n+1}/Y$	
	$A=0$	$A=1$
0 0 0	0 0 0/0	0 0 1/0
0 0 1	0 1 0/0	0 1 1/0
0 1 0	0 0 0/0	0 1 1/0
0 1 1	1 0 0/0	0 1 1/1
1 0 0	0 0 0/0	0 1 1/1

(4) 确定激励方程组和输出方程组

该电路需要用 3 个 D 触发器实现,它们的输出分别为 Q_2、Q_1、Q_0。于是,由表 6.3.10 可得状态转换真值表如表 6.3.11 所示。

① 有些国外文献称之为"one-hot encoding",国内某些文献直译为"1 位热码编码"。

由于 D 触发器的特性方程为 $Q^{n+1}=D$，所以根据表 6.3.11，可以画出 D_2、D_1、D_0和 Y 的卡诺图如图 6.3.12 所示，未应用的状态可作为无关项×填入。

表 6.3.11 例 6.3.3 的状态转换真值表

Q_2^n	Q_1^n	Q_0^n	A	$Q_2^{n+1}(D_2)$	$Q_1^{n+1}(D_1)$	$Q_0^{n+1}(D_0)$	Y
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	1	0	0
0	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
0	1	1	1	0	1	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	1	1	1

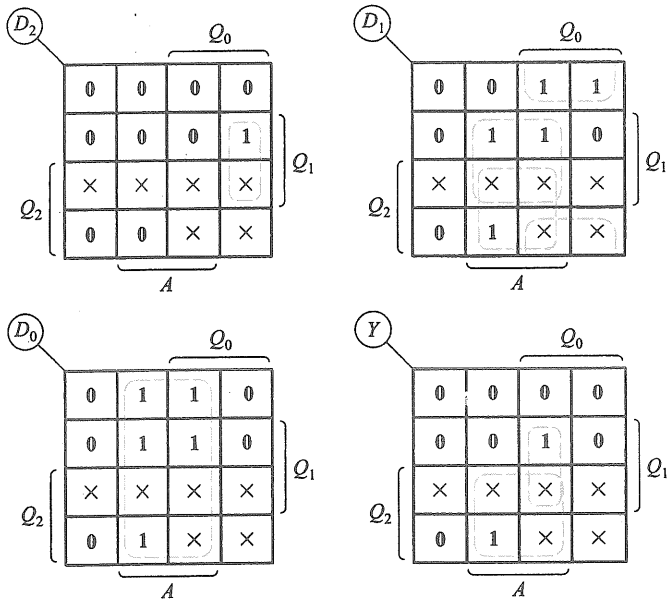


图 6.3.12 表 6.3.11 的卡诺图

经卡诺图化简，得到三个触发器的激励方程组

$$D_2=Q_1^nQ_0^n\overline{A}$$

$$D_1=\overline{Q_1^nQ_0^n}+Q_1^nA+Q_2^nA$$

$$D_0=A$$

和输出方程

$$Y=Q_1Q_0A+Q_2A$$

经过几次实践之后，实际上可不必列状态转换真值表而直接从状态转换表得到卡诺图。

（5）画出逻辑图

根据激励方程组和输出方程，可以画出图 6.3.13 所示的逻辑图。电路中引出了直接复位端$\overline{RESET}$，可外接复位电路，强制电路在开启上电时从状态 000 开始工作。

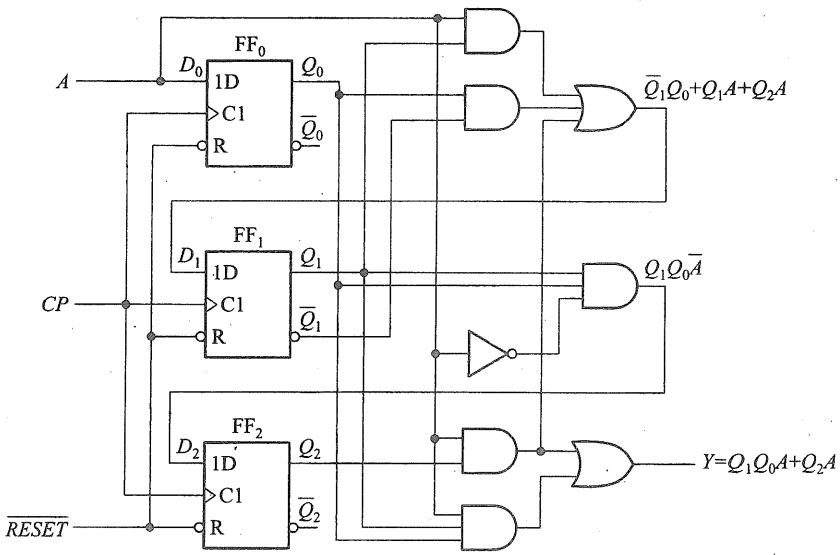


图 6.3.13 例 6.3.3 的逻辑图

（6）检查自校正能力

将 3 个无效状态 101、110 和 111 分别作为现态，与可能出现的所有输入信号一起分别代入电路的状态转换方程组而求其次态。结果证明都能进入有效状态。于是，可进一步画出完全状态转换图如图 6.3.14 所示。

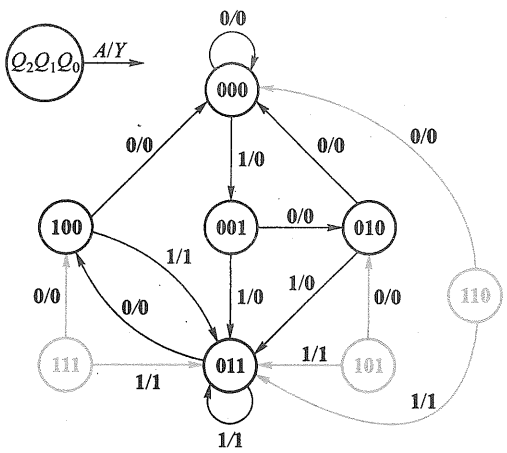


图 6.3.14 例 6.3.3 的完全状态转换图

6.3.3 同步时序逻辑电路中的时钟偏移

一般的时序电路都含有多个触发器，特别是规模较大的时序电路（如 PLD 中），其触发器的数量将会更多。同步时序电路要求时钟脉冲必须同时连接到每个触发器的时钟输入端。当电路规模较大时，由于实际电路制板时触发器的分布位置和布线等限制，每个触发器的时钟信号线的长度可能存在较大差异。另外，当触发器数量较多时，由于公共时钟信号驱动电路的负载能力有限（即扇出的限制），经常需要在某些位置增加缓冲器来提高时钟信号线的驱动能力，由此造成时钟脉冲传输到不同触发器的延迟时间不同。这种从同一时钟源出发的时钟脉冲，通过不同路径到达每个触发器的时间不同而产生的偏差称为时钟偏移。随着工艺水平的提高，触发器的传输延迟时间大大缩短（为 1～2 ns，参见表 5.4.3），很有可能出现小于时钟偏移的情况，此时，将导致同步时序电路出现错误动作。

例如某同步时序逻辑电路中的 2 个触发器电路如图 6.3.15（a）所示，触发器 FF_0 的输出 Q_0 作为触发器 FF_1 的输入，且假设它们的连线很短，可以忽略其延迟。另外，假设受元器件位置及布线等的限制，由 CP 到 FF_1 的时钟 CP_1 的传输路径很长，有较大延迟，即时钟偏移较大。在不考虑时钟偏移时，电路按照设计初衷正常工作，第 1 个 CP 脉冲有效沿到来时，输入信号 A 的状态存入 FF_0。第 2 个 CP 脉冲到来时，A 的状态才传递到 FF_1，其工作波形应如图 6.3.15（b）所示（设触发器初态 $Q_1Q_0=00$）。但是，实际电路存在时钟偏移，如果时钟偏移量超过触发器 FF_0 的传输延迟时间 t_{pd}，即 FF_1 的时钟 CP_1 在 Q_0 翻转后才到达，如图 6.3.15（c）

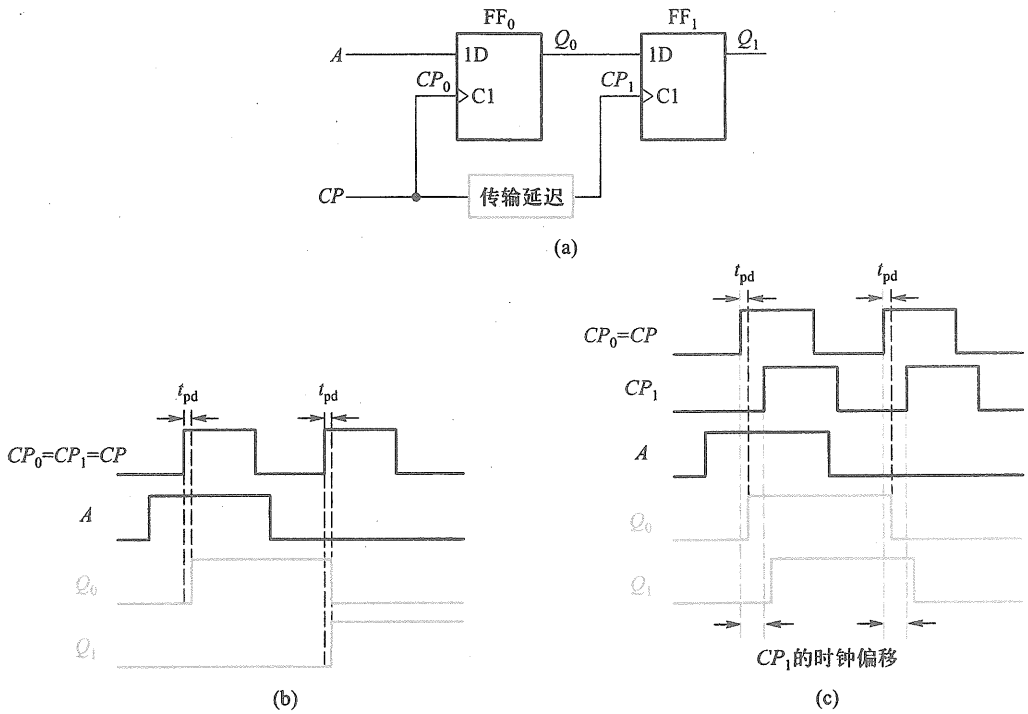


图 6.3.15 某同步时序逻辑电路工作情况

（a）逻辑电路 （b）无时钟偏移时的时序图 （c）由时钟偏移引起的错误时序图

所示，则 Q_1 的波形将与正常工作时（图 6.3.15(b)）不同，即 A 的状态在第 1 个 CP 脉冲到来时就已经传递到了 FF_1，出现了逻辑错误。

由此看出，严重的时钟偏移很有可能造成同步时序电路的工作错误。为了消除图 6.3.15(a)中时钟偏移带来的影响，一方面应尽可能缩短 CP_1 的传输路径，减小其延迟；另一方面，也可以在 CP_0 的传输路径上添加缓冲器，延长其延迟，从而减小两触发器的时钟偏移。

另外，不同的时钟传输路径上的负载（特别是容性负载）不平衡时，将导致不同路径上时钟脉冲的上升沿和下降沿的斜率不同，也会引起时钟偏移。因此，设计时钟布线网络时，还需要考虑各传输路径上负载的平衡问题。

通过以上分析，引起时钟偏移的主要原因可归纳为：

（1）各触发器时钟传输路径的长度不同；

（2）各触发器时钟传输路径上经过的缓冲器的数量不同；

（3）各触发器时钟传输路径上的负载不平衡。

在设计大规模的同步时序逻辑电路时，为了减少时钟偏移对电路正常工作的影响，应尽可能减小各触发器的时钟偏移。图 6.3.16 是一种可以有效减小时钟偏移的 H 型时钟布线网络。该方案的设计思路是使同步时钟信号传输到每个触发器 CP 端的路径尽可能相同。为简洁起见，图中未画出完整的触发器，而仅以 FF_i 替代。考虑到系统时钟 CP 扇出数的限制，网络中添加了 4 个缓冲器，每个缓冲器驱动 8 个触发器负载。当然，在实际电路设计时，可根据触发器 CP 端的负载效应和缓冲器的驱动能力来安排缓冲器的位置和数量。

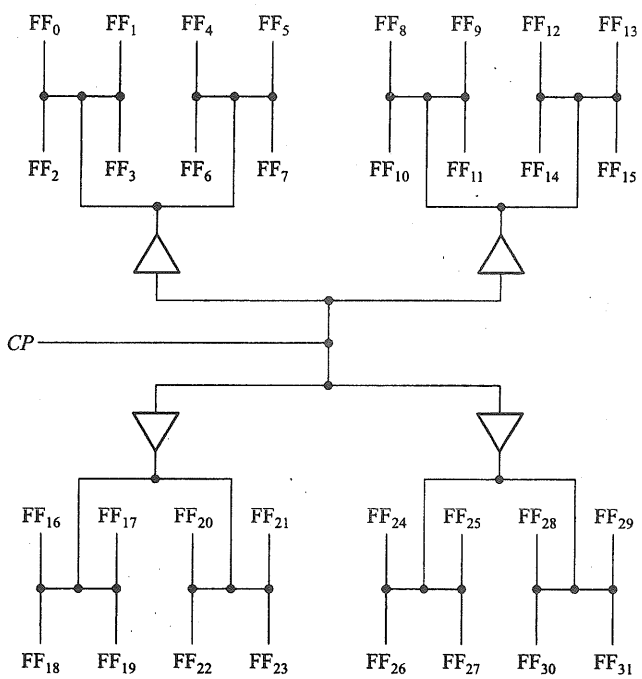


图 6.3.16 H 型时钟布线网络

复习思考题

6.3.1 同步时序电路的设计过程可分为哪几个步骤?

6.3.2 什么是原始状态转换图和原始状态转换表?怎样建立原始状态转换图和原始状态转换表?

6.3.3 什么是等价状态?若状态 a 与 b 等价,b 又与 c 等价,那么状态 a 与 c 等价吗?

6.3.4 同步时序电路中触发器的数目与状态数目有何关系?与状态分配又有何关系?

6.3.5 激励方程组和输出方程组是如何确定的?它们决定了同步时序电路中的哪部分电路?

6.3.6 如何检查同步时序电路的自校正能力?如果要求电路从特定状态开始工作,应如何处理?

6.3.7 造成时钟偏移的原因是什么?一般如何解决?

6.4 异步时序逻辑电路的分析

本节主要讨论用触发器构成的脉冲异步时序电路的分析方法。异步时序电路与同步时序电路的主要区别在于电路中没有统一的时钟脉冲,因而各存储电路不是同时更新状态。在分析脉冲异步时序电路时必须注意以下几点。

(1) 分析状态转换时必须考虑各触发器的时钟信号作用情况

异步时序电路中,由于各个触发器只有在其 CP_n(或 $\overline{CP}_n$,下标 n 表示电路中第 n 个触发器)端的信号有效时,才有可能改变状态。因此,在分析状态转换时,首先应根据给定的电路列出每个触发器的时钟信号方程,再确定其 CP_n(或 $\overline{CP}_n$)端是否有时钟信号的作用,并用变量 cp_n 表示:有作用,则令 $cp_n=1$;否则 $cp_n=0$。对于上升沿触发的触发器,当其 CP_n 端的信号由 0 变 1 时 $cp_n=1$;反之,对下降沿触发的触发器,则在 $\overline{CP}_n$ 信号由 1 变 0 时 $cp_n=1$。然后再根据激励信号确定那些 $cp_n=1$ 的触发器的次态,$cp_n=0$ 的触发器处于保持状态。这里,cp_n 不是一个逻辑变量,但参与各状态转换方程的逻辑运算。

(2) 每一次状态转换必须从输入信号所能触发的第一个触发器开始逐级确定

分析异步时序电路时,首先确定输入信号作用的第一个触发器的状态变化,并根据其值确定下一个触发器的时钟信号和状态变化,以此类推逐级分析,直到最后一个触发器。待全部触发器的次态确定后,才能最终确定电路的次态,填入状态转换表或状态转换图。

(3) 每一次状态转换都有一定的时间延迟

同步时序电路的所有触发器几乎是同时更新状态的,但异步时序电路各个触发器之间的状态更新存在一定的延迟,也就是说,从现态 S^n 到次态 S^{n+1} 的转换过程中有一段各触发器分别延迟更新状态的"不稳定"时间。在此期间,整个电路的状态是不确定的。只有当全部触发器的状态更新完毕,电路才进入新的"稳定"状态,即次态 S^{n+1}。因此,异步时序电路的输入信号(包括时钟信号)必须等待电路进入稳定状态之后才允许发生改变,否则电路会处在不确定的状态。由于上述延迟时间的存在,异步时序电路的工作速度要慢于相同逻辑功能的同步时序电路。

下面以两个实例来说明异步时序电路的分析过程。

例 6.4.1 分析图 6.4.1 所示逻辑电路。

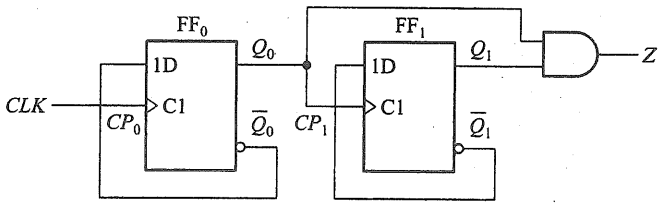


图 6.4.1 例 6.4.1 的逻辑电路图

解：在图 6.4.1 的电路中，两触发器 FF_0 和 FF_1 的 CP_0 和 CP_1 未共用时钟信号，故属于异步时序电路。

(1) 列出各逻辑方程组

① 时钟信号方程组

根据逻辑图列出各触发器的时钟信号表达式

$$CP_0 = CLK$$

$$CP_1 = Q_0$$

② 激励方程组

$$D_0 = \overline{Q}_0$$

$$D_1 = \overline{Q}_1$$

③ 状态转换方程组

这时需要考虑各触发器时钟信号 CP_n 的作用：只有 $cp_n = 1$ 发生后，相应触发器才可能转换状态，当 $cp_n = 0$，即 $\overline{cp}_n = 1$ 时，触发器应保持原态。因此，将 cp_n 引入触发器的特性方程得到如下的状态转换方程组：

$$Q_0^{n+1} = D_0 cp_0 + Q_0^n \overline{cp_0} = \overline{Q_0^n} cp_0 + Q_0^n \overline{cp_0} \tag{6.4.1}$$

$$Q_1^{n+1} = D_1 cp_1 + Q_1^n \overline{cp_1} = \overline{Q_1^n} cp_1 + Q_1^n \overline{cp_1} \tag{6.4.2}$$

④ 输出方程组

$$Z = Q_1 Q_0$$

(2) 列出状态转换表

列状态转换表的方法与同步时序电路分析过程基本相似，只是还应注意各触发器是否存在 $cp_n = 1$，因此，可在状态转换表中增加 cp_0、cp_1 两列。对应于 CLK 的每一个上升沿，$cp_0 = 1$，将 cp_0 和 FF_0 的现态 Q_0^n 代入式(6.4.1)，从而得到 Q_0^{n+1}。因为只有当 Q_0 由 0 跳变到 1，才有 $cp_1 = 1$，而当 Q_0 由 1 变到 0 时，$cp_1 = 0$。所以在每一次 Q_0 的状态转换后，应首先根据 FF_1 的 CP_1 表达式确定 cp_1，然后将其现态 Q_1^n 和相应的 cp_1 代入式(6.4.2)，求得其次态 Q_1^{n+1}。如果从 $Q_1^n Q_0^n$ 为 00 开始推导，可以得到如表 6.4.1 所示的状态转换表。

表 6.4.1 例 6.4.1 的状态转换表

Q_1^n	Q_0^n	cp_1	cp_0	Q_1^{n+1}	Q_0^{n+1}	Z
0	0	1	1	1	1	0
0	1	0	1	0	0	0
1	0	1	1	0	1	0
1	1	0	1	1	0	1

(3) 画出状态转换图和时序图

由表 6.4.1 所示的状态转换表可画出状态转换图，如图 6.4.2 所示。

根据状态转换图和触发器的传输延迟时间 t_{pLH} 和 t_{pHL}，可以画出时序图，如图 6.4.3 所示。可以看出，由于两个触发器状态翻转存在延迟，电路存在短暂的不确定状态，如果使用 74HCT74 双 D 触发器实现图 6.4.1 所示的电路，这段时间大约在 40 ns 左右。这种不确定的状态有时不容易判定，常常给电路设计和调试带来困难。

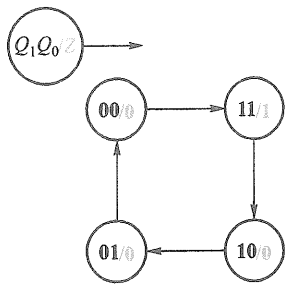


图 6.4.2 例 6.4.1 的状态转换图

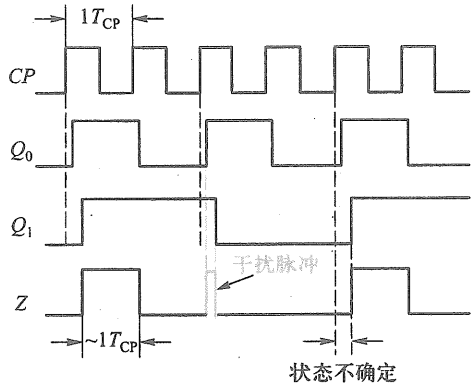


图 6.4.3 例 6.4.1 的时序图

(4) 逻辑功能分析

由状态转换图和时序图可知，该电路是一个异步二进制递减计数器，Z 信号的上升沿可触发借位操作。也可把它看作为一个序列信号发生器，输出序列脉冲信号 Z 的重复周期为 $4T_{CP}$，脉宽约为 $1T_{CP}$。

例 6.4.2 分析图 6.4.4 所示逻辑电路。

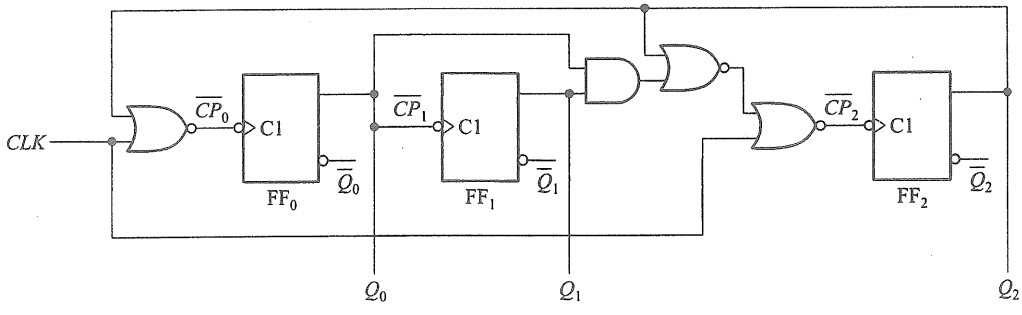


图 6.4.4 例 6.4.2 的逻辑电路图

解：这是由 3 个下降沿触发的 T' 触发器构成的异步时序电路。当触发器的时钟输入端 $\overline{CP}_n$ 出现从 1 到 0 的跳变，其状态就会翻转一次。下面按步骤进行分析。

（1）列出各逻辑方程组

① 时钟信号方程组

根据逻辑图列出各触发器时钟信号的逻辑表达式

$$\overline{CP}_0 = \overline{Q_2 + CLK} = Q_2 \overline{CLK} \tag{6.4.3}$$

$$\overline{CP}_1 = Q_0 \tag{6.4.4}$$

$$\overline{CP}_2 = \overline{\overline{Q_0 Q_1 + Q_2} + CLK} = (Q_0 Q_1 + Q_2)\overline{CLK} \tag{6.4.5}$$

② 状态转换方程组

将 cp_n 引入 T' 触发器的特性方程 $Q_0^{n+1} = \overline{Q_0^n}$ 得到状态转换方程

$$Q_0^{n+1} = \overline{Q_0^n} cp_0 + Q_0^n \overline{cp_0} \tag{6.4.6}$$

$$Q_1^{n+1} = \overline{Q_1^n} cp_1 + Q_1^n \overline{cp_1} \tag{6.4.7}$$

$$Q_2^{n+1} = \overline{Q_2^n} cp_2 + Q_2^n \overline{cp_2} \tag{6.4.8}$$

注意：此例中每当 $\overline{CP}_n$ 发生由 1 到 0 的跳变时 $cp_n = 1$。

③ 输出方程组

即三个触发器的输出信号 Q_2、Q_1、Q_0。

（2）列出状态转换表

从现态 $Q_2 = Q_1 = Q_0 = 0$ 开始列状态转换表。从 CLK 触发的第一个触发器 FF_0 开始推导其次态。首先确定 cp_0：根据式(6.4.3)，由于 $Q_2 = 0$，当 CLK 有 0→1 跳变将使 $\overline{CP}_0$ 产生 1→0 的跳变，所以 $cp_0 = 1$。然后将 cp_0 和现态 $Q_0^n = 0$ 代入式(6.4.6)，得到 $Q_0^{n+1} = 1$。类似地，根据式(6.4.5)，由于 $Q_2 = Q_1 = Q_0 = 0$，$\overline{CP}_2$ 为 0，此时 CLK 的任何变化都不会使 $\overline{CP}_2$ 产生下降沿，故 $cp_2 = 0$，FF_2 不会改变状态，$Q_2^{n+1} = 0$。这时，再根据式(6.4.4)确定 cp_1：因为 Q_0 是 0→1 跳变，所以 $cp_1 = 0$，Q_1 也将保持原状。CLK 信号第一个上升沿到来后，电路状态改变为 001。以此类推，可得电路的状态转换表，如表 6.4.2 所示。

表 6.4.2 例 6.4.2 的状态转换表

Q_2^n	Q_1^n	Q_0^n	cp_2	cp_1	cp_0	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	1	0	1	0
0	1	0	0	0	1	0	1	1
0	1	1	1	1	1	1	0	0
1	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0	1
1	1	0	1	0	0	0	1	0
1	1	1	1	0	0	0	1	1

(3) 画出状态转换图

由表 6.4.2 所示的状态转换表可画出状态转换图,如图 6.4.5 所示。该图表明,当电路处于循环外的状态(101、110 和 111 三个状态)时,在 *CLK* 信号出现第一个上升沿后,电路便能进入有效循环状态。

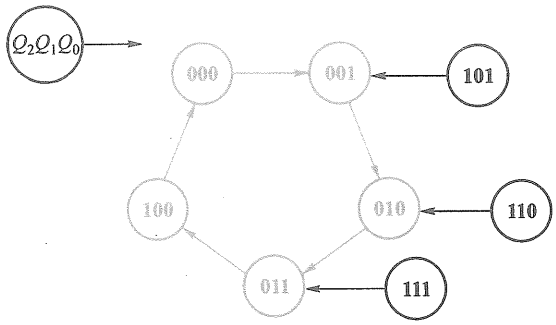


图 6.4.5 例 6.4.2 的状态转换图

(4) 逻辑功能分析

由状态转换图可知,图 6.4.4 所示电路是一个具有 000~100 五个连续二进制编码状态的异步计数器。

复习思考题

6.4.1 异步时序电路的分析与同步时序电路的分析主要有哪些不同之处?

6.4.2 分析异步时序电路时,为什么需要列出触发器的时钟信号方程组?

6.4.3 为什么在分析异步时序电路的状态转换时,必须按信号作用的顺序对每个触发器进行逐个推导,才能确定电路的次态?

6.5 常用时序逻辑电路模块

与组合逻辑电路类似,在大规模可编程器件普遍应用之前,许多具有特定功能的常用时序逻辑电路被制成了单个的中规模集成芯片,为构建数字系统提供了极大方便。在现代数字系统设计中,这种中规模芯片已经被淘汰,但它们作为基本模块在系统设计中仍被广泛采用。本节主要从功能模块而非集成芯片的角度介绍这些电路。这些模块包括寄存器、移位寄存器和计数器等。

6.5.1 寄存器和移位寄存器

1. 寄存器

寄存器是数字系统中用来存储二进制数据的逻辑部件。1 个触发器可存储 1 位二进制数据,存储 N 位二进制数据的寄存器需要用 N 个触发器组成。

由 8 个触发器构成的 8 位寄存器的逻辑图如图 6.5.1 所示。图中,$D_7 \sim D_0$是 8 位数据输入端,在 *CP* 脉冲上升沿作用下,$D_7 \sim D_0$端的数据同时存入相应的触发器,是一种极为简单

的同步时序电路。当输出使能控制信号$\overline{OE}=0$时，触发器存储的数据通过三态门输出端$Q_7 \sim Q_0$并行输出。表 6.5.1 是典型的中规模集成 8 位寄存器 74HC/HCT374 的功能表。

从存储数据的角度看，5.3.2 小节介绍的八 D 锁存器 74HC/HCT373 与此处的 8 位寄存器 74HC/HCT374 具有类似的逻辑功能。两者的区别在于，前者是电平敏感电路，而后者是脉冲边沿敏感电路。如果数据更新出现在控制信号（锁存器的 LE）有效之后，则只能使用锁存器，但它不能保证各个锁存器输出状态完全同步更新；如果输入数据的更新是在控制信号（寄存器的 CP）有效沿之前，并要求输出状态同时更新，则可选择寄存器。一般来说，寄存器比锁存器具有更好的同步性能和抗干扰性能。

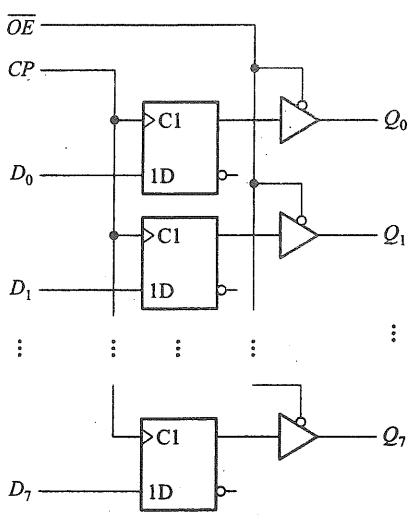


图 6.5.1 带输出缓冲器的 8 位寄存器

表 6.5.1 74HC/HCT374 的功能表

工作模式	输入			内部触发器	输出
	$\overline{OE}$	CP	D_N	Q_N^{n+1}	$Q_0 \sim Q_7$
存入和读出数据	L	↑	L*	L	相应内部触发器的状态
	L	↑	H*	H	
存入数据，禁止输出	H	↑	L*	L	高阻
	H	↑	H*	H	高阻

注：D_N和Q_N^{n+1}的下标表示第 N 位触发器。L* 和 H* 表示 CP 脉冲上升沿之前瞬间 D_N的电平。

与“宽总线”锁存器类似，寄存器也有 16 位、18 位、20 位和 32 位的“宽总线”集成电路。为了应用的灵活方便，一般都把其中的 D 触发器分为若干组，各组间既可同步工作，又可异步工作。另外，也有 1~2 个触发器的“小逻辑”寄存器产品。

2. 移位寄存器

（1）基本的移位寄存器

将若干个触发器级联则构成基本的移位寄存器。4 位移位寄存器如图 6.5.2 所示，各触发器的激励方程为 $D_0=D_{SI}$、$D_1=Q_0$、$D_2=Q_1$、$D_3=Q_2$。将串行数据 $D_3'D_2'D_1'D_0'$从高位（D_3'）至低位（D_0'）按时钟脉冲 CP 间隔依次送到 D_{SI}端，经过第一个 CP 后，$Q_0=D_3'$。经过第二个 CP 后，$Q_1=D_3'$，$Q_0=D_2'$。以此类推，经过 4 个 CP 后，寄存器输出 $Q_3Q_2Q_1Q_0=D_3'D_2'D_1'D_0'$。该移位寄存器的功能表如表 6.5.2 所示（×表示不确定状态）。图 6.5.3 中画出了 $D_3'D_2'D_1'D_0'=1101$（D_{SI}在输入 **1101** 之后全部输入 **0**）在移位寄存器中移位的波形，经过 4 个 CP 后，$Q_3Q_2Q_1Q_0=1101$，即串行输入数据转换为并行输出数据。在第 7 个 CP 后，数据从输出端 D_{SO}（即 Q_3端）全部移出，即从 D_{SO}可得到 **1101** 的串行输出。

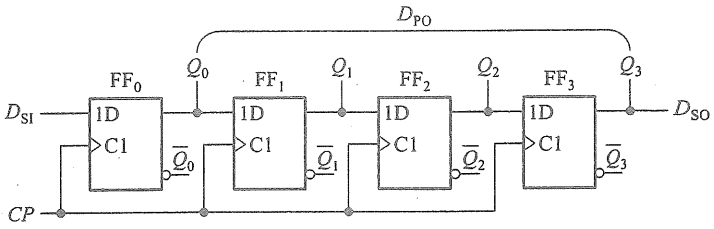


图 6.5.2 用 D 触发器构成的 4 位移位寄存器

表 6.5.2 图 6.5.2 电路的功能表

CP	Q_0	Q_1	Q_2	Q_3
第一个 CP 脉冲之前	×	×	×	×
1	D_3	×	×	×
2	D_2	D_3	×	×
3	D_1	D_2	D_3	×
4	D_0	D_1	D_2	D_3

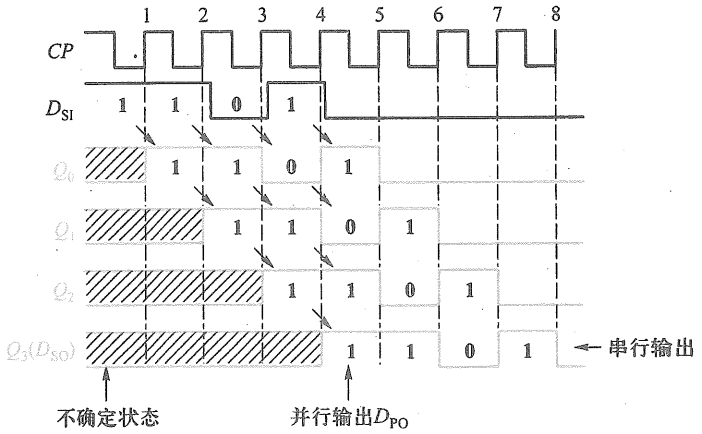


图 6.5.3 图 6.5.2 电路的时序图

一般来说，N 位移位寄存器要由 N 个触发器构成，需要 $N \cdot T_{CP}$ 时长来完成串行到并行的数据转换。这里，T_{CP} 为时钟周期。

从上述操作可以看出，移位寄存器只能用脉冲边沿敏感的触发器，而不能用电平敏感的锁存器来构成，否则在时钟脉冲有效电平期间，锁存器的输出跟随输入变化的特性将使移位操作失去控制。显然，移位寄存器属于同步时序电路。

（2）多功能双向移位寄存器

① 工作原理

有时需要控制移位寄存器的数据流向，实现数据的双向移动。为区分两个移位方向，通常称其中一个方向为右移，另一个方向则为左移。

为了扩展逻辑功能和增加使用的灵活性，在双向移位基础上，还可以增加并行输入、并行输出等功能，构成多功能移位寄存器，其工作模式的简化示意图如图 6.5.4 所示。

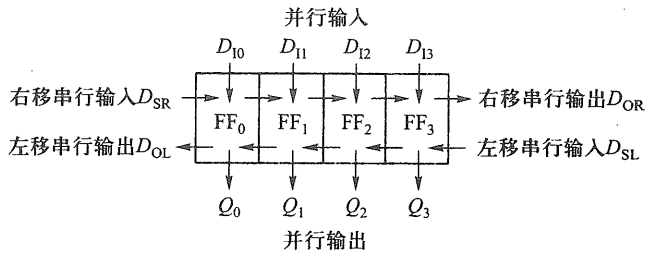


图 6.5.4 多功能移位寄存器工作模式简图

图 6.5.5 是实现数据保持、右移、左移、并行输入和并行输出的一种电路方案。图中 FF_m 表示 N 位移位寄存器中的第 m 位触发器，其输入端 D_m 连接一个 4 选 1 数据选择器的输出，由选择输入信号 S_1S_0 选择 D_m 的数据来源。当 $S_1S_0=\mathbf{00}$ 时，$D_m=Q_m$，次态 $Q_m^{n+1}=D_m=Q_m^n$，触发器处于保持状态；当 $S_1S_0=\mathbf{01}$ 时，$D_m=Q_{m-1}$，CP 上升沿到来时，$Q_m^{n+1}=Q_{m-1}^n$，而 $Q_{m+1}^{n+1}=Q_m^n$，从而实现右移功能；类似地，当 $S_1S_0=\mathbf{10}$ 时，$D_m=Q_{m+1}$，实现左移功能；而当 $S_1S_0=\mathbf{11}$ 时，$D_m=D_{Im}$，$Q_m^{n+1}=D_{Im}$，则完成并行数据的输入。上述四种功能概述于表 6.5.3 中。显然，任何时候寄存器的 N 位数据都可以由输出端 $Q_{N-1}\sim Q_0$ 并行输出。

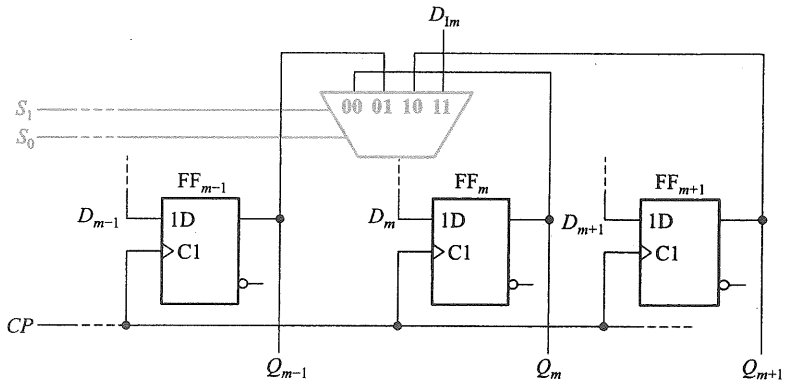


图 6.5.5 实现多种功能双向移位寄存器的一种方案

② 4 位双向移位寄存器模块

采用图 6.5.5 方案实现数据保持、右移、左移、并行输入和并行输出的一种多功能 4 位双向移位寄存器如图 6.5.6 所示。图中，D_{SR} 是右移串行数据输入端，D_{SL} 是左移串行数据输入端，$D_{I0}\sim D_{I3}$ 是并行数据输入端，$\overline{CR}$ 连接到每个触发器的直接置 **0** 端，可控制寄存器异步清零。其功能表如表 6.5.4 所示。表中内容：第 1 行表示寄存器异步清零操作，第 2 行为保持

表 6.5.3 图 6.5.5 的功能表

控制信号		功能
S_1	S_0	
0	0	保持
0	1	右移
1	0	左移
1	1	并行输入

状态，第 3、4 行为串行数据右移操作，第 5、6 行为串行数据左移操作，第 7 行是并行数据的同步置入操作。中规模集成芯片 74HC/HCT194 便具有这样的逻辑功能。

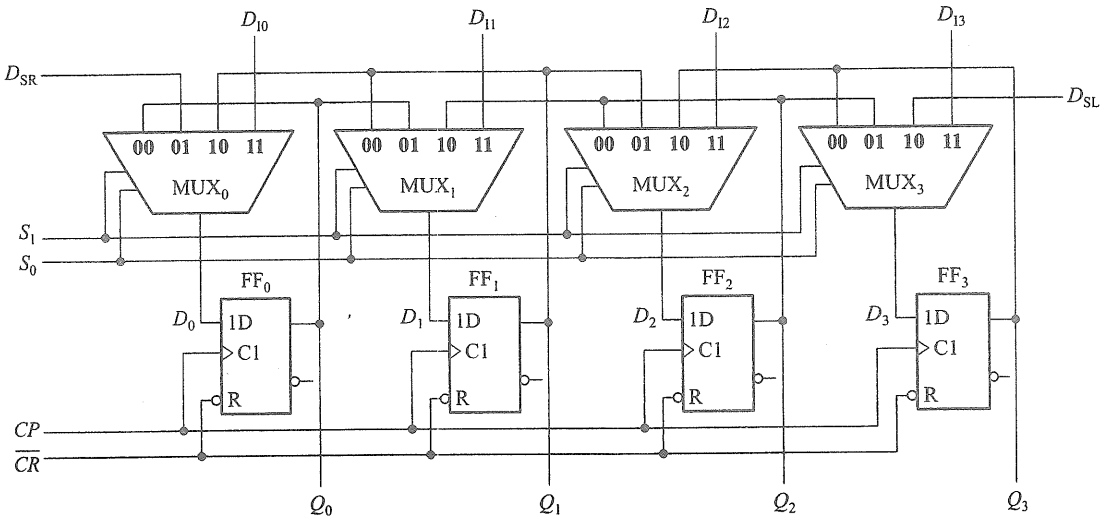


图 6.5.6 多功能 4 位双向移位寄存器的逻辑图

6.5.2 计数器

计数器是最常用的时序电路之一，它们不仅可用于对脉冲进行计数，还可用于分频、定时、产生节拍脉冲以及其他时序信号。计数器的种类繁多，按触发器动作分类，可分为同步计数器和异步计数器；按数值增减分类，可分为递增计数器、递减计数器和可逆计数器；按编码分类，又可分为二进制计数器、BCD 计数器、循环码计数器等。计数器运行时，从某一状态开始完整循环一次所经历的状态数称为计数器的模(modulo)，并用 M 表示。若某个计数器在 n 个状态下循环计数，通常则称之为模 n 计数器，或 $M=n$ 计数器。例如一个在 60 个不同状态中循环转换的计数器，就可称为模 60，或 $M=60$ 计数器。有时也把模 n 计数器称为 n 进制计数器。

表 6.5.4 多功能 4 位双向移位寄存器的功能表

输入										输出				功能
清零 $\overline{CR}$	控制信号		时钟 CP	串行输入		并行输入				Q_0^{n+1}	Q_1^{n+1}	Q_2^{n+1}	Q_3^{n+1}	
	S_1	S_0		右移 D_{SR}	左移 D_{SL}	D_{I0}	D_{I1}	D_{I2}	D_{I3}					
L	×	×	×	×	×	×	×	×	×	L	L	L	L	异步清零
H	L	L	×	×	×	×	×	×	×	Q_0^n	Q_1^n	Q_2^n	Q_3^n	保持
H	L	H	↑	L	×	×	×	×	×	L	Q_0^n	Q_1^n	Q_2^n	右移
H	L	H	↑	H	×	×	×	×	×	H	Q_0^n	Q_1^n	Q_2^n	右移
H	H	L	↑	×	L	×	×	×	×	Q_1^n	Q_2^n	Q_3^n	L	左移
H	H	L	↑	×	H	×	×	×	×	Q_1^n	Q_2^n	Q_3^n	H	左移
H	H	H	↑	×	×	D_{I0}^*	D_{I1}^*	D_{I2}^*	D_{I3}^*	D_{I0}	D_{I1}	D_{I2}	D_{I3}	同步并行置数

注：D_{IN}^* 表示 CP 脉冲上升沿之前瞬间 D_{IN} 的电平。

1. 二进制计数器

按照自然二进制数递增或递减编码的计数器称为二进制计数器（binary counter），N 位二进制计数器的模为 2^N，由 N 个触发器组成。

（1）异步二进制计数器

4 位异步二进制计数器如图 6.5.7 所示，它由 4 个下降沿触发的 T' 触发器组成。计数脉冲 $\overline{CP}$ 加至触发器 FF_0 的时钟脉冲输入端，每输入一个计数脉冲，FF_0 翻转一次。FF_1、FF_2 和 FF_3 都以前级触发器的 Q 端输出作为触发信号，当 Q_0 由 1 变 0 时，FF_1 翻转，其余类推。分析其工作过程，不难得到输出波形如图 6.5.8 所示。由图可见，从初态 0000（可由 $\overline{CR}$ 低电平脉冲使 4 个触发器全部置 0）开始，每输入一个计数脉冲，计数器的状态就按二进制编码值递增 1，输入第 16 个计数脉冲后，计数器又回到 0000 状态。显然，该计数器以 16 个 $\overline{CP}$ 脉冲构成一个计数周期，是模 16（$M=16$）递增计数器。其中，Q_0 的频率是 $\overline{CP}$ 的 1/2，即实现了 2 分频，Q_1 得到 $\overline{CP}$ 的 4 分频，以此类推，Q_2、Q_3 分别对 $\overline{CP}$ 进行了 8 分频和 16 分频，因而，计数器也可作为分频器使用。

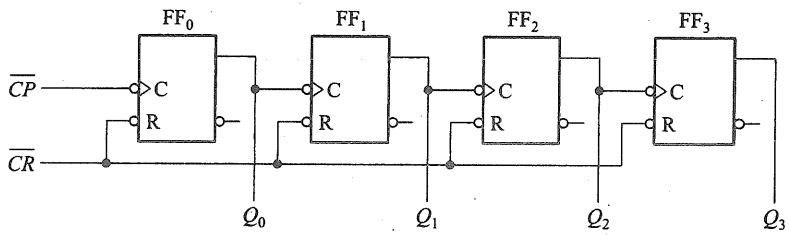


图 6.5.7 4 位异步二进制计数器逻辑图

异步二进制计数器的结构和工作原理均较简单，低位触发器的输出接高位触发的时钟输入，使触发器逐级翻转实现计数进位，故亦称之为纹波计数器（ripple counter）。图 6.5.8 中的虚线是考虑了触发器逐级翻转过程中平均传输延迟时间 t_{pd} 的波形。由于各触发器不在同一时间翻转，可能会出现瞬间错误输出。例如，当计数值从 0111 加 1 时，$Q_3Q_2Q_1Q_0$ 要先后经过 0110、0100、0000 几个状态，才最终翻转为 1000。

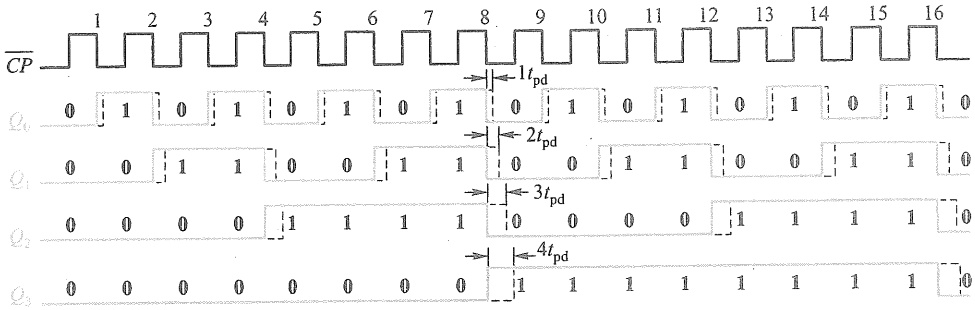


图 6.5.8 4 位异步二进制计数器时序图

另外，当计数脉冲频率很高时，$Q_3 \sim Q_0$ 甚至会出现编码输出分辨不清的情况。对于一个 N 位二进制异步计数器来说，从计数脉冲作用在第一个触发器开始，到第 N 个触发器翻转的延迟时间为 Nt_{pd}。为了保证正确地检出计数器的输出状态，必须满足 $T_{CP} \gg Nt_{pd}$ 的条件，其中，T_{CP} 为计数脉冲 $\overline{CP}$ 的周期。

（2）同步二进制计数器

为了提高计数速度，可采用同步计数器，即电路中所有触发器的时钟接在一起。当脉冲有效沿到来时，使应该翻转的触发器同时翻转，不存在异步计数器的纹波进位造成的延迟时间，所以可以有较高的计数速度，输出编码也不会发生纹波进位时的那种混乱。

① 工作原理

表 6.5.5 是 4 位二进制计数器的状态转换表[①]。观察该表递增计数一栏可以看出，Q_0 在每个计数脉冲作用后都要翻转一次；Q_1 只在 $Q_0=1$ 时接受计数脉冲作用而翻转；Q_2 则在 $Q_0=Q_1=1$ 时其次态翻转；Q_3 在 $Q_0=Q_1=Q_2=1$ 时其次态翻转。当用 T 触发器实现 4 位同步二进制递增计数器时，可根据上述激励条件，确定每个 T 触发器的激励方程为

$$\begin{cases} T_0 = 1 \\ T_1 = Q_0 \\ T_2 = Q_1Q_0 \\ T_3 = Q_2Q_1Q_0 \end{cases} \tag{6.5.1}$$

即除了触发器 FF_0 的 $T_0=1$ 外，其他触发器都是在比其低的各位均为 1 时准备好翻转条件。由此类推，可以扩展到更多的位数。当用 N 个 T 触发器构成 N 位同步二进制递增计数器时，各触发器的激励方程为

教学视频 6.1：同步二进制计数器

① 表 6.5.5 是时序电路状态转换表的简化形式，只给出在一系列时钟脉冲信号作用下电路状态的转换顺序，若以某行状态作为现态，下一行则为次态。

$$\begin{cases} T_0 = 1 \\ T_i = Q_{i-1}\, Q_{i-2} \cdots Q_1\, Q_0 = \prod_{j=0}^{i-1} Q_j \end{cases} \quad (i = 1, 2, \cdots, N-1)$$

由表 6.5.5 递减计数一栏可以看出，Q_0同样在每个计数脉冲沿到来时都要翻转一次；而其他位都是在比其低的各位均为 0 时准备好翻转条件，在次态完成翻转，实现二进制计数的递减借位。当用 T 触发器实现 4 位同步二进制递减计数器时，激励方程为

$$\begin{cases} T_0 = 1 \\ T_1 = \overline{Q}_0 \\ T_2 = \overline{Q}_1 \overline{Q}_0 \\ T_3 = \overline{Q}_2 \overline{Q}_1 \overline{Q}_0 \end{cases} \quad (6.5.2)$$

表 6.5.5 4 位二进制计数器的状态转换表

计数顺序	递增计数	递减计数
	$Q_3Q_2Q_1Q_0$	$Q_3Q_2Q_1Q_0$
0	0 0 0 0	0 0 0 0
1	0 0 0 1	1 1 1 1
2	0 0 1 0	1 1 1 0
3	0 0 1 1	1 1 0 1
4	0 1 0 0	1 1 0 0
5	0 1 0 1	1 0 1 1
6	0 1 1 0	1 0 1 0
7	0 1 1 1	1 0 0 1
8	1 0 0 0	1 0 0 0
9	1 0 0 1	0 1 1 1
10	1 0 1 0	0 1 1 0
11	1 0 1 1	0 1 0 1
12	1 1 0 0	0 1 0 0
13	1 1 0 1	0 0 1 1
14	1 1 1 0	0 0 1 0
15	1 1 1 1	0 0 0 1
16	0 0 0 0	0 0 0 0

对于 N 位同步二进制递减计数器，类似地有

$$\begin{cases} T_0 = 1 \\ T_i = \overline{Q}_{i-1}\, \overline{Q}_{i-2} \cdots \overline{Q}_1\, \overline{Q}_0 = \prod_{j=0}^{i-1} \overline{Q}_j \end{cases} \quad (i = 1, 2, \cdots, N-1)$$

根据式(6.5.1)和式(6.5.2)得出由 T 触发器构成的 4 位同步二进制计数器逻辑图如图 6.5.9 所示。其中,图(a)为递增计数器,图(b)为递减计数器。

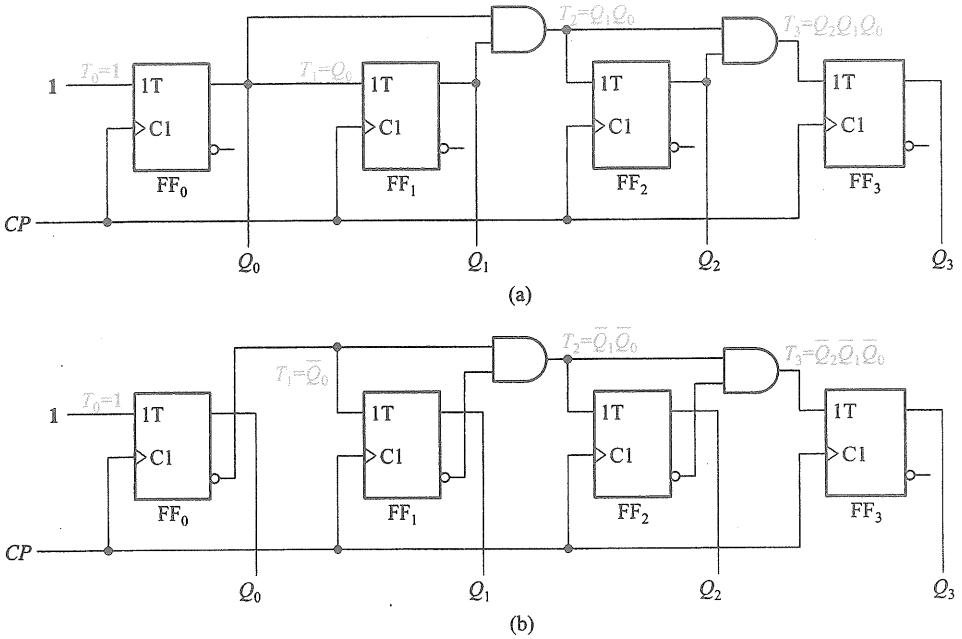


图 6.5.9 4 位同步二进制计数器

(a) 递增计数器 (b) 递减计数器

图 6.5.10 所示是图 6.5.9(a)所示计数器的时序图,其中虚线是考虑到触发器传输延迟时间 t_{pd} 的波形。由图 6.5.10 可知,在同步计数器中,由于计数脉冲 CP 同时作用于各触发器,所有触发器的状态更新几乎是同时进行的,都比计数脉冲沿的作用时间滞后一个 t_{pd}。因此,输出状态比异步二进制计数器稳定。对于计数功能相同、位数相同的计数器,同步计数器的工作速度一般高于异步计数器。

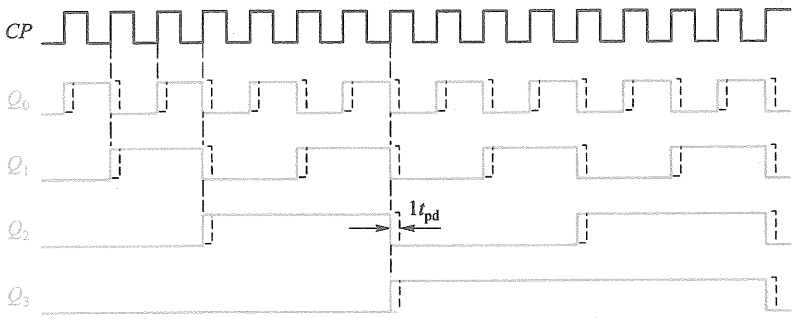


图 6.5.10 4 位同步二进制递增计数器时序图

当用 D 触发器实现二进制计数器时,可根据图 5.5.9(a)D 触发器与 T 触发器的关系,得到 D 触发器构成的 4 位二进制递增计数器如图 6.5.11 所示。其中,4 个点画线方框内为等

效的 T 触发器。

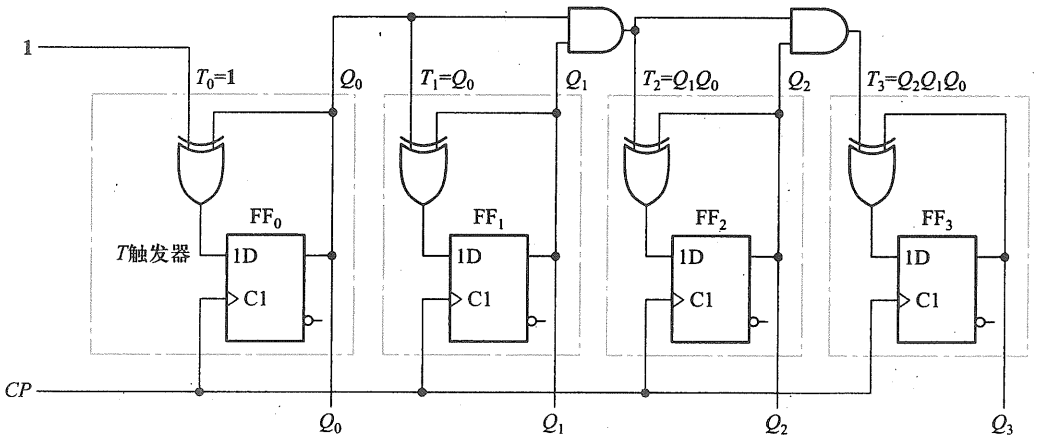


图 6.5.11 D 触发器构成的 4 位同步二进制递增计数器

② 同步计数器的进位方法

递增计数器的低位计满后激励高位触发器翻转的过程称为计数进位。图 6.5.11 所示电路的进位信号是通过一系列串行的逻辑门逐位向高位传递的。如果一个计数器中有 N 位触发器，将它们按二进制数位从低位向高位编号，最低位输出为 Q_0，那么，Q_0 的进位信号至少要经过 $N-2$ 个逻辑门的延迟，才可能到达最高位触发器的输入端。假设每个逻辑门的延迟时间相同，均为 t_{Gpd}，那么计数器中组合电路的最大延迟时间为 $t_{COMpd(max)}=(N-2)t_{Gpd}$。显然，$t_{COMpd(max)}$ 将随着计数器的位数增多而增大。当计数器位数较多时，会降低电路的运行质量，或被迫降低电路的工作频率。

通过使用扇入数更多的逻辑门实现进位信号的传输，可以减少进位信号串行逻辑门级数，从而缩短延迟时间，提高工作速度。通过这种方式构成的计数器电路如图 6.5.12 所示。与图 6.5.11 所示电路相比，到达高位触发器输入端的信号都只经过了一级门（G_1、G_2、G_3）的传输。而且位数增加，门的级数也不会变，只是门的扇入数要增加。虽然图 6.5.12 中各触发器的激励方程与前者相同，但后者在一次状态转换完成后，只需经过相当于一个逻辑门的传输延迟时间 t_{Gpd}，就可将激励信号传递到所有高位触发器的 T 输入端，等待下一个 CP 触发沿的到来。即无论计数器有多少位，都有 $t_{COMpd(max)}=t_{Gpd}$。因此，这种进位方式可以大大提高计数器的工作频率，位数越多，提高越明显。然而，随着计数器位数的增多，低位触发器的扇出和高位触发器激励电路的扇入会增多，例如，图 6.5.12 所示电路中 Q_0 目前的扇出数是 5，随着触发器的增加，其扇出数也增加；G_3 门目前的扇入数是 4，位数更高的触发器对应的门的扇入数会更大，所以这种电路的位数也是受限的。

图 6.5.12 中 CET 和 CEP 是计数使能端，也分别称为直接进位允许和片内进位允许信号。只有在它们都为 1 时，才允许计数器计数，否则计数器处于保持状态。$TC=Q_3Q_2Q_1Q_0\cdot CET$ 是进位输出信号。

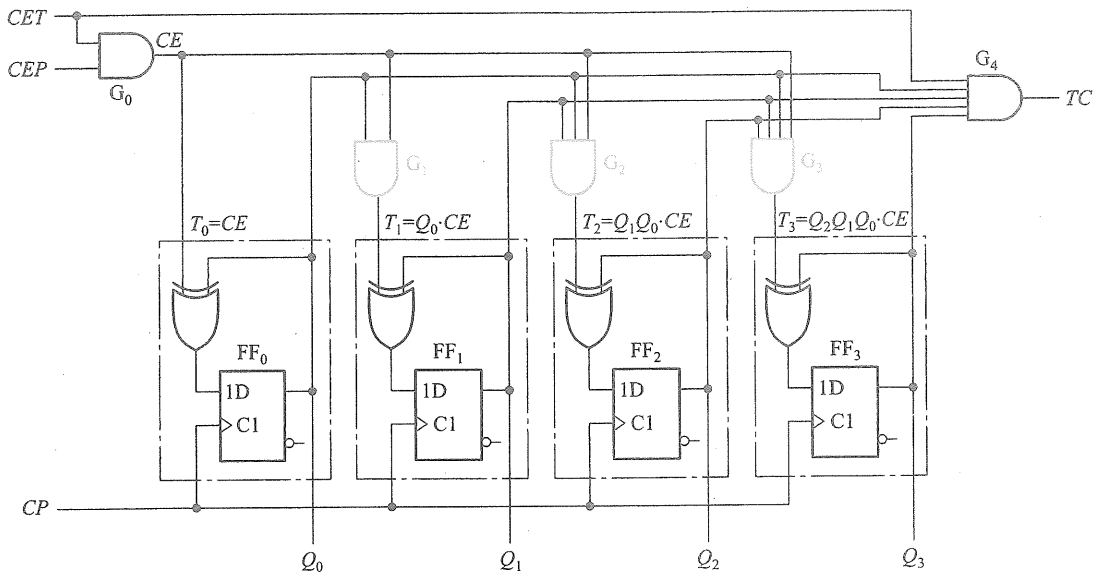


图 6.5.12 并行进位的同步 4 位二进制递增计数器

③ 异步清零和同步并行置数

图 6.5.13 所示电路是功能更为完备的 4 位同步二进制计数器的逻辑图。

在许多应用中，需要计数器从零状态开始工作，为此，在图 6.5.13 所示电路中增加了直接清零输入端$\overline{CR}$，与 4 个触发器的置 0 端直接相连。当$\overline{CR}$为低电平时，无论其他输入端是何状态(包括时钟信号 CP)，使所有触发器置 0，因此，$\overline{CR}=0$ 的优先级最高，其他各种操作都是在$\overline{CR}=1$ 的条件下才能执行。

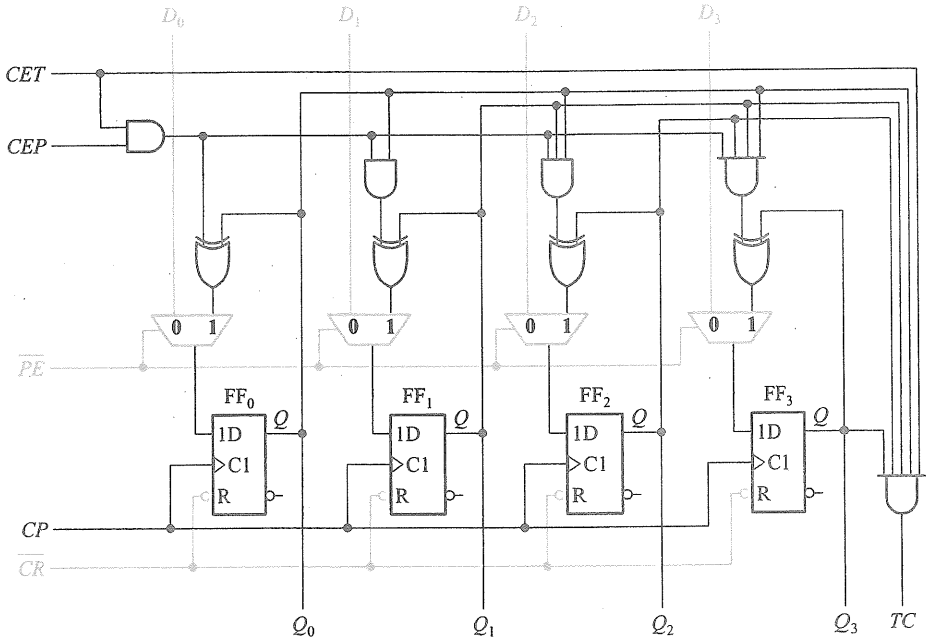


图 6.5.13 具有清零和并行置数功能的 4 位同步二进制递增计数器

计数器应用时，有时需要将计数器预置为某种状态，然后再从该状态开始计数。图 6.5.13 所示电路是在图 6.5.12 中每个 D 触发器的输入端插入一个 2 选 1 数据选择器，当置数控制端 $\overline{PE}=0$ 时，数据选择器将 $D_3D_2D_1D_0$ 数据送至各触发器的 D 输入端，在下一个 CP 沿到来时，被并行置入各触发器。这种在时钟脉冲作用下的并行置数称为同步置数。而当 $\overline{PE}=1$ 时，将异或门的输出信号送至各触发器 D 输入端，计数器恢复正常计数功能，于是，可在预置数据的基础上继续递增计数。

由图 6.5.13 看出，$\overline{PE}=0$ 具有次高优先级，仅低于 $\overline{CR}=0$，计数和保持状态的操作都要求 $\overline{PE}=1$。

④ 典型集成同步二进制计数器

74LVC161 是一种典型的高性能、低功耗 CMOS 4 位同步二进制递增计数器，其逻辑图与图 6.5.13 相同。它的时钟最高工作频率可达 200 MHz。图 6.5.14 所示为 74LVC161 的引脚功能图，功能表如表 6.5.6 所示。

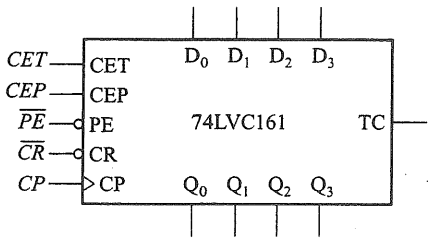


图 6.5.14 74LVC161 引脚功能图

表 6.5.6 74LVC161 的功能表

输入									输出				
清零 $\overline{CR}$	预置 $\overline{PE}$	使能		时钟 CP	预置数据输入				Q_3	Q_2	Q_1	Q_0	进位 TC
		CEP	CET		D_3	D_2	D_1	D_0					
L	×	×	×	×	×	×	×	×	L	L	L	L	L
H	L	×	×	↑	D_3^*	D_2^*	D_1^*	D_0^*	D_3	D_2	D_1	D_0	#
H	H	L	×	×	×	×	×	×	保	持			#
H	H	×	L	×	×	×	×	×	保	持			L
H	H	H	H	↑	×	×	×	×	计	数			#

注：D_N^* 表示 CP 脉冲上升沿之前瞬间 D_N 的电平。#表示只有当 $Q_3Q_2Q_1Q_0 \cdot CET=1$（正逻辑体系）时，TC 输出为高电平，其余均为低电平。

74LVC161 时序图如图 6.5.15 所示。图中，当清零信号 $\overline{CR}=0$ 时，各触发器置 0。当 $\overline{CR}=1$ 时，若 $\overline{PE}=0$，在下一个时钟脉冲上升沿到来后，$Q_3Q_2Q_1Q_0=D_3D_2D_1D_0=1100$。在 $\overline{CR}=\overline{PE}=1$ 的条件下，若 $CEP=CET=1$，则电路处于计数状态。当 $CEP \cdot CET=0$，停止计数，并保持在 $Q_3Q_2Q_1Q_0=0010$ 状态。只有当 $Q_3Q_2Q_1Q_0=1111$ 且 $CET=1$，进位信号 $TC=1$，其余时间均为 0。

与 74LVC161 工作原理和功能相似的集成计数器还有 74LVC163，其差别仅在于清零方式不同，后者为同步清零，即清零是在 CP 信号控制下完成的。

2. 其他模数的计数器

除了模为 2^N 的二进制计数器外，工程上还经常需要其他模数的计数器，可以用 6.3 节学习的设计方法设计这些计数器。这里主要介绍用已有的集成计数器（计数器模块）构成任

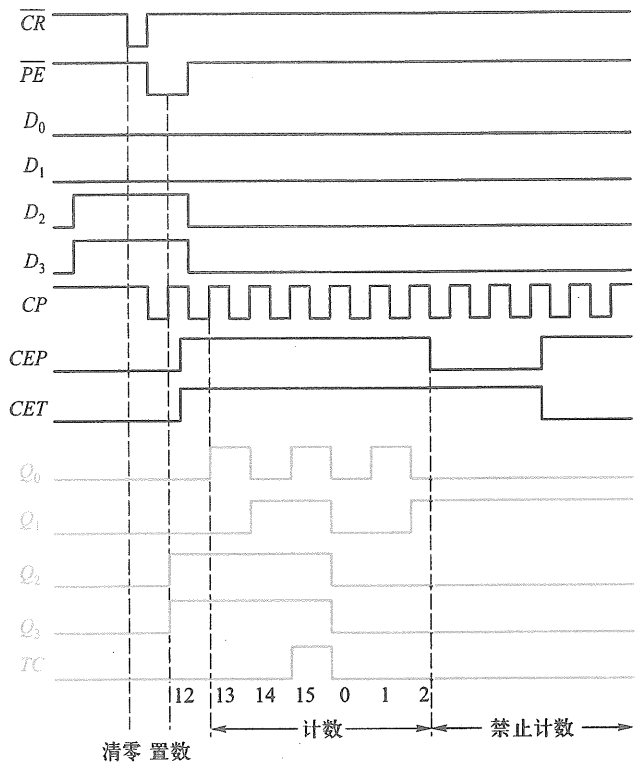


图 6.5.15 74LVC161 的典型时序图

意模数计数器的方法。

用模 M 的集成计数器构成模 N 计数器时，如果 $N<M$ 则只需一个模 M 集成计数器；如果 $N>M$，则要用多个模 M 计数器来构成。下面结合例题分别介绍这两种情况的实现方法。

(1) $N<M$ 的情况

例 6.5.1 用 74LVC161 构成模 9 递增计数器。

解：模 9 计数器有 9 个状态，而 74LVC161 有 16 个状态。因此属于 $N<M$ 的情况。如果设法跳过多余的 7 个状态，则可实现模 9 计数器。通常用两种方法实现，即反馈清零法和反馈置数法。

① 反馈清零法

反馈清零法适用于有清零输入端的计数器。74LVC161 具有异步清零功能，在计数过程中，无论其输出处于哪一状态，只要 $\overline{CR}=0$，输出会立即从所处状态回到 0000 状态。清零信号消失后（$\overline{CR}=1$），74LVC161 又从 0000 状态开始重新计数。

利用 74LVC161 异步清零功能实现的模 9 计数器如图 6.5.16(a)所示。图(b)是其主循环状态转换图。由图可知，74LVC161 从 0000 状态开始计数，当第九个 CP 脉冲上升沿到达时，输出 $Q_3Q_2Q_1Q_0=1001$，通过一个与非门译码后，使 $\overline{CR}=0$，立即使 $Q_3Q_2Q_1Q_0$ 返回到 0000 状态。此刻，产生清零信号的条件已消失，$\overline{CR}=1$，74LVC161 重新从 0000 状态开始新的计数周期。这样就跳过了 1001～1111 七个状态，构成模 9 计数器。需要说明，电路是在进入

1001 状态后才被置成 0000 的，因此在主循环状态转换图中用虚线圈表示 1001 状态。虽然该状态存在的时间很短，但是，如果 Q_0 端通过某种方式另接有其他时序电路，则应充分考虑 $Q_0=1$ 的窄脉冲是否会引起错误的逻辑输出，例如触发器被误触发等故障。

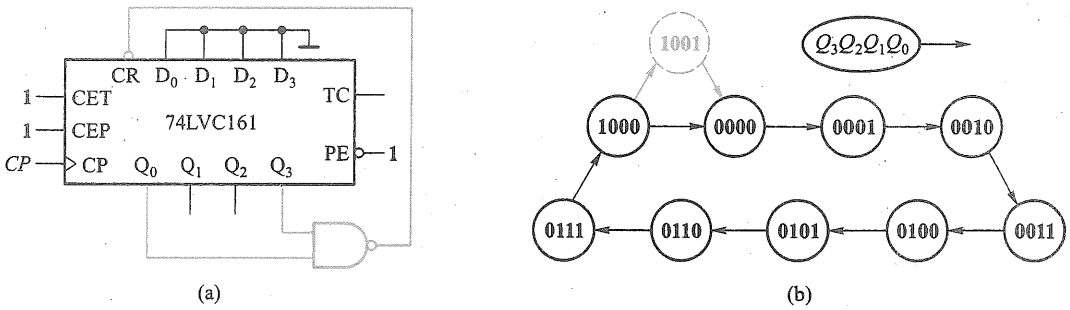


图 6.5.16 用反馈清零法将 74LVC161 接成模 9 计数器

（a）电路图 （b）主循环状态转换图

具有同步清零功能的模 M 集成计数器也可用反馈清零法构成模 N 计数器。这里不再举例，读者可自行分析两者的差异。

② 反馈置数法

反馈置数法适用于具有预置数功能的集成计数器。对于具有同步预置功能的计数器，在计数过程中的任何一个状态通过译码，产生反馈信号至预置控制端，在下一个 CP 脉冲作用下，将数据输入端 D_3、D_2、D_1、D_0 的值置入计数器。预置控制信号消失后，计数器就从被置入的状态开始重新计数。

借助 74LVC161 同步预置功能，采用反馈置数法构成模 9 递增计数器的电路如图 6.5.17（a）所示。其将输出 $Q_3Q_2Q_1Q_0=1000$ 的状态经译码产生预置信号 0 反馈至 $\overline{PE}$ 端，在下一个 CP 脉冲上升沿到达时置入 0000 状态。图 6.5.17（b）是图（a）电路的主循环状态转换图。它是取 0000～1111 的 16 种状态中前 9 个状态构成模 9 计数器。也可以取后 9 个状态 0111～1111 构成模 9 计数器如图 6.5.18 所示。当计数到 1111 状态时 $TC=Q_3\cdot Q_2\cdot Q_1\cdot Q_0\cdot CET=1$，$\overline{PE}=0$，在第 9 个 CP 脉冲作用后，$Q_3Q_2Q_1Q_0$ 被置成 0111 状态，同时使 $TC=0$，$\overline{PE}=1$。新的计数周期又从 0111 开始。

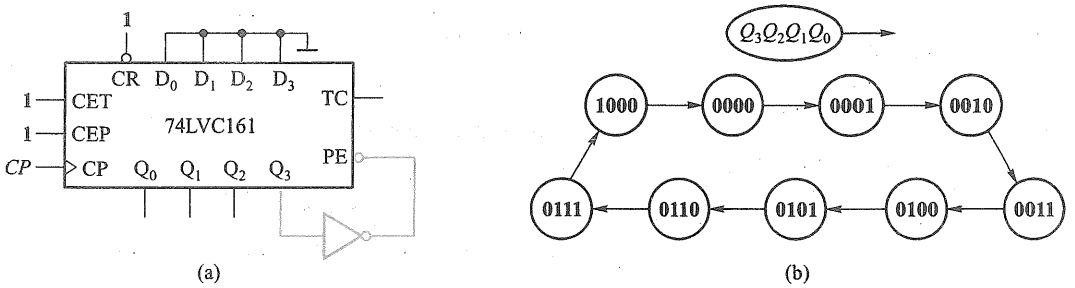


图 6.5.17 用反馈置数法将 74LVC161 接成模 9 计数器

（a）电路图 （b）主循环状态转换图

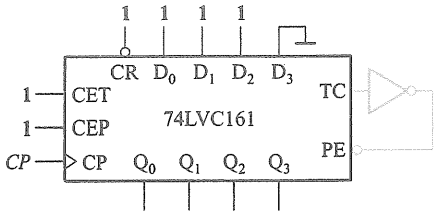


图 6.5.18 另一种反馈置数法的电路

(2) $N>M$ 的情况

当 $N>M$ 时,需要将多个模 M 的计数器级联构成 N 进制计数器。级联的方式有同步级联和异步级联两种方法。下面以 4 位二进制计数器 74LVC161 构成的 8 位二进制计数器电路为例说明。

① 同步级联

采用同步级联方法构成的电路如图 6.5.19 所示。由于它们共用同一个时钟,所以是同步工作方式。其中低位片(0)的进位信号连接高位片(1)的计数使能端,控制高位片的工作状态。每当片(0)计满一个循环 TC 输出 1 时,控制片(1)计数一次;当片(0)的 TC 为 0 时,片(1)处于保持状态,所以电路的模数为 $N=16\times16=256$。

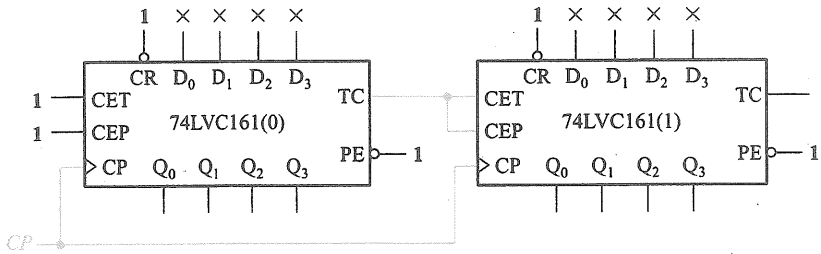


图 6.5.19 同步级联方式

② 异步级联

异步级联方式不再共用时钟,而是用片(0)的进位信号作为片(1)的时钟输入,电路构成方式如图 6.5.20 所示。当片(0)计数到 1111 时,TC 为高电平,经反相器变为低电平。下一个 CP 到来时,片(0)返回到 0000,同时 TC 回 0 产生一个下降沿,经反相后相当于给片(1)的 CP 输入端一个上升沿,触发片(1)加 1 计数。所以电路的模数同样是 $N=16\times16=256$。

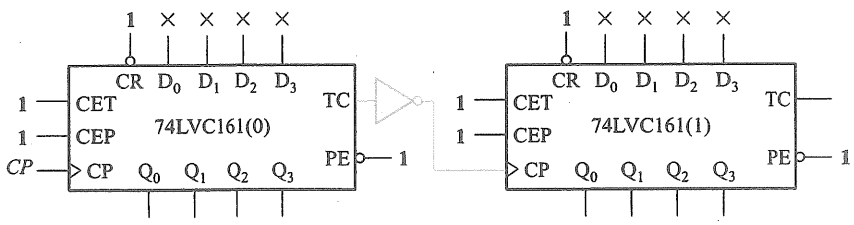


图 6.5.20 异步级联方式

级联后的电路可以采用整体反馈清零法或反馈置数法构成模小于 256 的计数器，其原理与单片的构成方法类似。读者可自行分析或设计。

*(3) 多片 74LVC161 级联时 *CET* 和 *CEP* 的灵活运用

当用多片 74LVC161 级联扩展为更多位的计数器时，通过合理连接 *CEP* 和 *CET*，可以充分发挥计数器的工作速度。图 6.5.21 示出按这种方案实现 16 位同步二进制计数的一个实例。4 个芯片从低位到高位分别以 $C_0 \sim C_3$ 命名。需要注意的是每一片中 *CET*、*CEP* 和 *TC* 的连接方法。对于 C_0 片，*CET* 与 *CEP* 相连，可作为整个计数器的使能输入 *CE*，当 *CE*=1 时允许计数。其他各片的 *CEP* 均接收 C_0 片计数器的进位信号 TC_0；C_1 片 *CET* 直接与计数允许输入相连，C_2、C_3 片的 *CET* 接收相邻低位片的进位信号。电路工作时，只有在 C_0 片发出进位信号，即 $TC_0=1$ 后，才允许所有高位片内部进行并行进位操作；高位各片的进位信息则通过各自的 G_4 直接上传。可见，C_0 片 Q_0（见图 6.5.12）的翻转信号，只需经过该片的 G_4 和 C_3 片的 G_0、G_3（见图 6.5.12）三个逻辑门的延迟，历时大约 $3t_{Gpd}$，就能抵达整个计数器的最高位触发器，也就是 C_3 片的 FF_3。进一步分析可以证明，电路中所有触发器的激励电路延迟都不会大于 $3t_{Gpd}$，即 $t_{COMpd(max)} \approx 3t_{Gpd}$。因而，这种并、串行结合的进位方法既具有较短的进位延迟，又同时兼顾了电路的复杂程度。

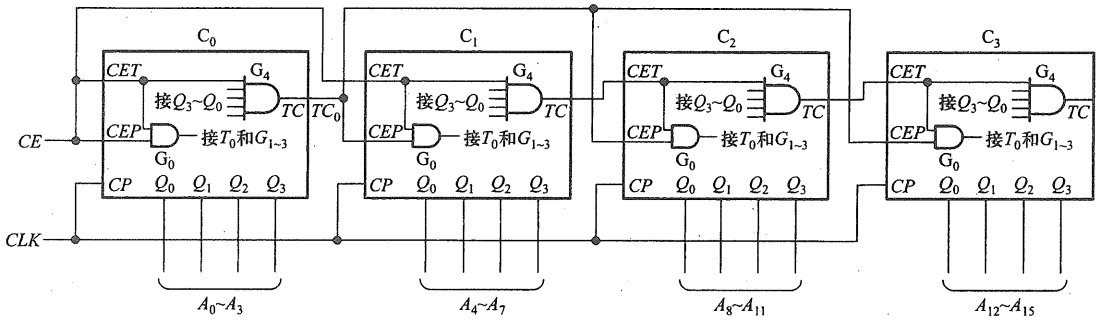


图 6.5.21 以图 6.5.12 所示电路实现的并行进位和串行进位结合的 16 位同步二进制计数器

图 6.5.21 所示的进位方法，对于设计大型同步计数器是具有普遍意义的，包括 PLD 和 ASIC 的应用。例如，在高速数据采集系统中，经常需要用计数器对地址单元数在 1k 以上的存储器进行地址扫描，以便将高速采集到的数据按顺序同步存入存储器。用于这种目的的二进制计数器，位数一般都超过 10 位，如果采用并、串行结合的进位方式，则可提高电路工作的可靠性，或充分发挥已有电路的潜力。

(4) 环形计数器

将移位寄存器最高位的输出接至最低位的输入，或将最低位的输出接至最高位的输入，便可实现所谓的环形计数器，也称为环形移位寄存器。

① 基本环形计数器

如果将图 6.5.2 中移位寄存器的 D_{SO}（Q_3）与 D_{SI} 相连，则构成环形计数器，如图 6.5.22(a)所示。若事先通过 $\overline{PE}$ 端施加低电平脉冲，将触发器置为 $Q_3Q_2Q_1Q_0=0001$，那么环形计数器有 4 个循环状态如图 6.5.22(b)所示，于是，电路构成模 4 计数器。图 6.5.22(c)所示为在 4 个 *CP* 脉冲作用下的波形。可以看出，这种计数器不必译码就能直接输出 4 个状态的

译码信号，亦不存在普通译码电路输出易出现的竞争-冒险现象。

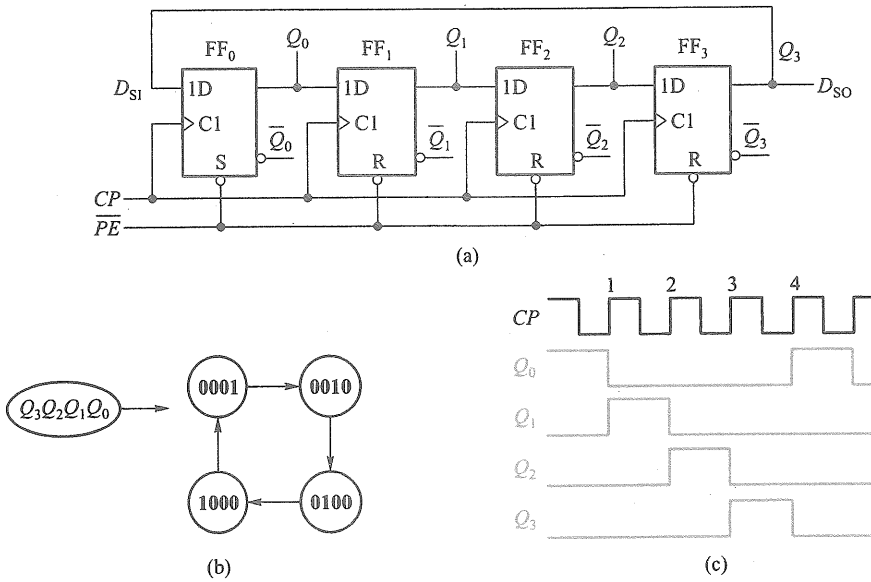


图 6.5.22 基本环形计数器的电路和状态变化

（a）模 4 环形计数器 （b）状态转换图 （c）波形图

② 扭环形计数器

上述基本环形计数器的状态利用率不高，4 个触发器只有 4 个计数状态。若将图 6.5.2 所示电路中的$\overline{Q}_3$与D_{SI}相连，则构成扭环形计数器，亦称为约翰逊计数器（Johnson counter），图 6.5.23（a）所示是 5 个触发器构成的模 10 扭环形计数器，图 6.5.23（b）所示是其状态转换图，可见电路的状态比环形计数器增加了一倍。表 6.5.7 列出了它的十个状态以及对各状态译码的逻辑表达式。可以看出，它的译码电路将十分简单。由于每一次时钟脉冲触发沿到来后只有一个触发器改变状态，所以译码输出不会出现竞争-冒险。图 6.5.23（a）中附加的逻辑门使电路能够自校正，请读者自行验证。

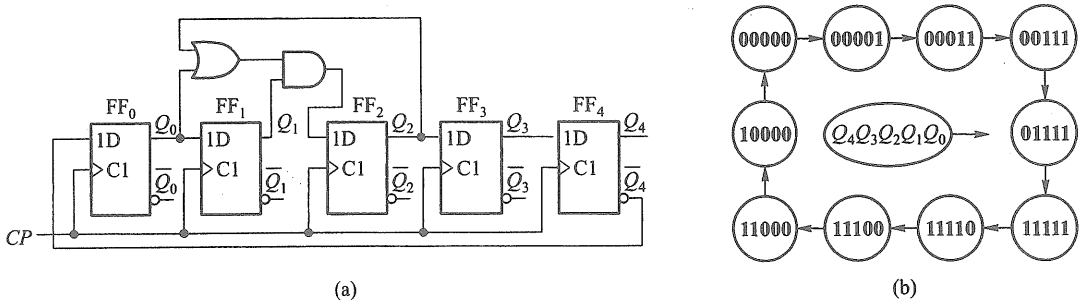


图 6.5.23 模 10 扭环形计数器

（a）逻辑图 （b）状态转换图

表 6.5.7 图 6.5.23 所示扭环形计数器的十个状态及其译码

状态编号	Q_4	Q_3	Q_2	Q_1	Q_0	状态译码
0	0	0	0	0	0	$\overline{Q}_4\overline{Q}_0$
1	0	0	0	0	1	$\overline{Q}_1Q_0$
2	0	0	0	1	1	$\overline{Q}_2Q_1$
3	0	0	1	1	1	$\overline{Q}_3Q_2$
4	0	1	1	1	1	$\overline{Q}_4Q_3$
5	1	1	1	1	1	Q_4Q_0
6	1	1	1	1	0	$Q_1\overline{Q}_0$
7	1	1	1	0	0	$Q_2\overline{Q}_1$
8	1	1	0	0	0	$Q_3\overline{Q}_2$
9	1	0	0	0	0	$Q_4\overline{Q}_3$

复习思考题

6.5.1 电平敏感的锁存器和脉冲边沿敏感的寄存器在数据寄存操作上有何不同？

6.5.2 用 D 锁存器能否构成移位寄存器或计数器？为什么？

6.5.3 如何用双向移位寄存器实现二进制数据的$\times 2^n$和$\div 2^n$运算？

6.5.4 同步和异步二进制计数器各有什么特点？在需要高速计数并对计数值译码时应采用哪种计数器？

6.5.5 电子钟表中的石英晶体振荡器一般采用 32.768 kHz 的振荡频率，需要使用多少位二进制计数器分频才能得到周期为 1 s 的秒信号？

6.5.6 如何用具有同步清零功能的二进制计数器采取反馈清零法来构成模 N 计数器？

6.5.7 试用 10 个二输入端与门为图 6.5.23(a)所示电路设计十个状态的译码电路，并说明为什么它们的输出不会出现竞争-冒险。

6.6 时序逻辑的可编程电路实现

4.5 节已经介绍了用 PLD 实现组合逻辑的原理，实际上，只要在 PLD 中再增加触发器和反馈连接，就可以实现时序逻辑了。本节通过例子加以说明。

例 6.6.1 对于例 6.3.2 改用含 D 触发器的可编程逻辑器件实现，试求激励方程和输出方程，并画出编程后的逻辑图。

解：由图 6.3.7 的状态转换图，可以列出状态转换表如表 6.6.1 所示。

表 6.6.1 例 6.3.2 的状态转换表

$Q_1^nQ_0^n$	$Q_1^{n+1}Q_0^{n+1}/Y$	
	$A=0$	$A=1$
0 0	0 0/0	0 1/0
0 1	0 0/0	1 1/0
1 1	0 0/1	1 1/0

由于 D 触发器特性方程为 $Q^{n+1}=D$，所以可根据表 6.6.1 列出 2 个 D 触发器激励信号和输出信号的卡诺图如图 6.6.1 所示。

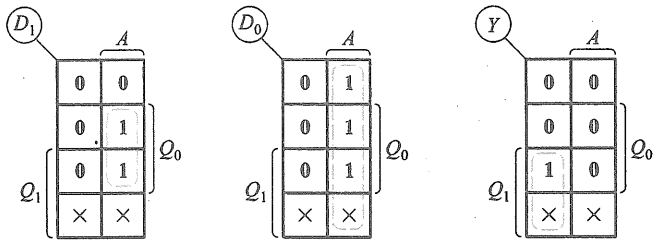


图 6.6.1 例 6.6.1 的卡诺图

得激励方程和输出方程

$$D_1 = Q_0A$$

$$D_0 = A$$

$$Y = Q_1\overline{A}$$

用含 D 触发器的 PLD 实现编程后的逻辑图如图 6.6.2 所示。其中 CLK 是 PLD 中所有触发器统一的时钟信号，OE 是寄存器型输出使能信号。

对 PLD 编程需要在开发系统（包括硬件编程器和软件）的控制下完成。其开发过程将在 8.3 节统一介绍。

复习思考题

6.6.1 用 PLD 实现组合逻辑与实现时序逻辑有何差别？

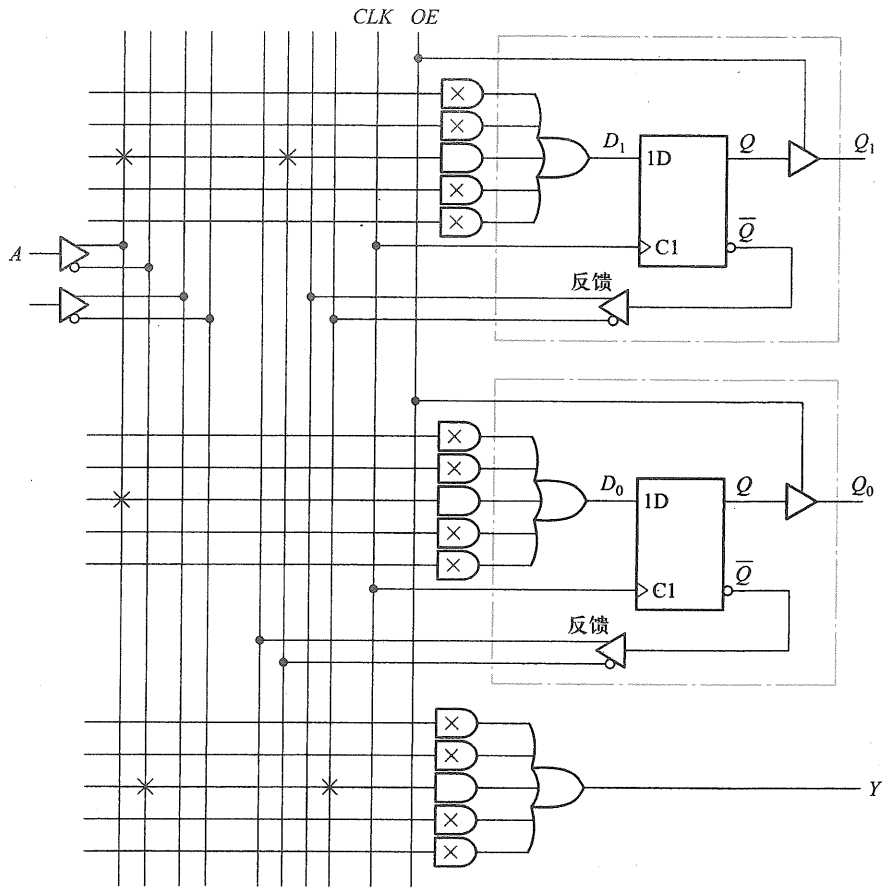


图 6.6.2 例 6.6.1 逻辑功能的 PLD 实现

6.7 用 Verilog HDL 描述时序逻辑电路

在 4.6 节和 5.6 节分别介绍了用 Verilog 描述组合电路和触发器的方法，本节将通过一些例题讨论时序逻辑电路的行为描述方法。

6.7.1 移位寄存器的 Verilog 建模

例 6.7.1 试用 Verilog 描述表 6.5.4 所示 4 位双向移位寄存器的行为(功能)。

解： 该模块能完成 5 种逻辑功能：异步清零、同步置数、左移、右移和保持原状态不变。功能与图 6.5.6 所示的 74HC/HCT194 类似，其代码如图 6.7.1 所示。在 **always** 结构中，使用 **if-else** 和 **case** 语句进行描述。

当 CR 信号跳变到低电平时，将寄存器输出异步清 **0**；当 CR = **1** 时，在 CP 作用下根据 **case** 语句中的 S1、S0 对 4 种功能进行选择。例如：

```
Q<={Q[1:3],Dsl};
```

是将左移串行输入 Dsl 和触发器输出 Q[1:3]拼接起来完成左移操作。在 CP 上升沿到来时,将 Dsl 传给 Q[3],并将 Q[1:3]传给 Q[0:2](即 Q[3]→Q[2],Q[2]→Q[1],Q[1]→Q[0])。

```
module shift74x194_beh (
    input S1,S0,
    input Dsl,Dsr,
    input CP,CR,
    input [0:3]D,
    output reg [0:3]Q
);
  always @(posedge CP, negedge CR)
    if(!CR)Q<=4'b0000;   //异步清零
    else
      case({S1,S0})
        2'b00:Q<=Q;        //输出保持不变
        2'b01:Q<={Dsr, Q[0:2] };   //右移
        2'b10:Q<={Q[1:3],Dsl};   //左移
        2'b11:Q<=D;          //并行置数
      endcase
  endmodule
```

图 6.7.1 移位寄存器的 Verilog 代码

6.7.2 计数器的 Verilog 建模

下面通过 3 个例题介绍同步二进制计数器、异步二进制计数器和非二进制计数器的 Verilog 建模。

例 6.7.2 试用 Verilog 描述一个带使能端和同步置数端的 4 位二进制可逆计数器的行为。

解:可逆计数器 Verilog 代码如图 6.7.2 所示。通过参数 n 设置二进制计数器的位数。当 Up_down=1 时,整型变量 direction 为 1,计数器为递增计数;Up_down=0 时,direction 为-1,计数器为递减计数。

例 6.7.3 试用 Verilog 描述一个有异步清零功能的 D 触发器子模块,再根据图 6.5.7 描述 4 位异步二进制计数器的行为。

解:异步计数器的结构化描述如图 6.7.3 所示。先写出有异步清零功能的 D 触发器子模块,在顶层模块 ripplecounter 中,通过 4 次调用子模块完成计数功能。

第 1 个触发器 FF0 的时钟信号接外部输入 CP 端,其输出 Q0 经反相(用~Q0 表示)后与 D 输入相连(在 FF0 中用~Q0 取代 D),构成 T'触发器(参见图 5.5.10)。第 2 个触发器 FF1 的时钟信号接前一个触发器的输出端(用 Q0 取代 CP),其输出 Q1 经反相后与 D 输入相连(在 FF1 中用~Q1 取代 D)。以此类推,构成 4 位异步二进制计数器。

```
module updowncount_beh #(parameter n=4)
(
  input Load,Up_down,En,CP,
  input [n-1:0]D,
  output reg [n-1:0]Q
  );
  integer direction;

always @(posedge CP)
begin
  if(Up_down)
    direction <= 1;
  else
    direction <= -1;
  if(Load)
    Q <= D;                  //Load=1,同步置数
  else if(En)
    Q <= Q+direction;   //加 1 或减 1
  else
       Q <= Q;             //输出保持不变
end
endmodule
```

图 6.7.2 可逆二进制计数器的 Verilog 代码

```
//D 触发器子模块
module D_FF(
   output reg Q,
   input D,CP,Rd
 );
always @(negedge CP, negedge Rd)
begin
   if(!Rd)   Q <= 0;   //异步清零
   else       Q <= D;
end
endmodule
// 4 位异步二进制计数器
module ripplecounter (
      output Q0,Q1,Q2,Q3,
      input CP,CR
);
   //实例化引用 D 触发器子模块
   D_FF FF0(Q0,~Q0,CP,CR);
   D_FF FF1(Q1,~Q1,Q0,CR);
   D_FF FF2(Q2,~Q2,Q1,CR);
   D_FF FF3(Q3,~Q3,Q2,CR);
endmodule
```

图 6.7.3 异步二进制计数器的 Verilog 代码

例 6.7.4 试用 Verilog 描述一个带有使能端和异步清零端的同步模 10 计数器。

解： Verilog 代码如图 6.7.4 所示。当清零信号 CR 跳变到低电平时，计数器输出异步置 0；否则，当 CR=1，且使能信号 CE=1 时，在 CP 上升沿作用下，若计数值小于 9，则计数器的值加 1；若计数值大于或者等于 9，计数器的输出被置 0。当 CR=1，但 CE=0 时，计数器处于保持状态。

```
module m10_counter(
   output reg [3:0]Q,
   input CE,CP,CR
);
always @(posedge CP,negedge CR)
begin
   if(!CR)Q=4'b0000;
   else if(CE)
        begin
          if (Q >= 4'b1001)      //判断计数值
                Q<=4'b0000;
          else     Q<=Q+1'b1;
        end
          else Q <= Q;              //输出保持不变
end
endmodule
```

图 6.7.4 同步模 10 计数器的 Verilog 代码

注意,电路的功能描述与具体的硬件电路结构是无关的。

6.7.3 状态转换图的 Verilog 建模

描述状态转换图的方法很多,下面通过两个例题进行说明。

例 6.7.5 试用 Verilog 语言描述图 6.7.5 所示状态转换图的行为。

解: Verilog 代码如图 6.7.6 所示。程序首先对电路的输入、输出、时钟以及清零信号进行声明,对保存电路状态值的触发器用标识符 state(中间节点)进行声明,并用参数定义语句 **parameter** 定义电路的四种状态,即 S0 = 2'b00、S1 = 2'b01、S2 = 2'b10 和 S3 = 2'b11。

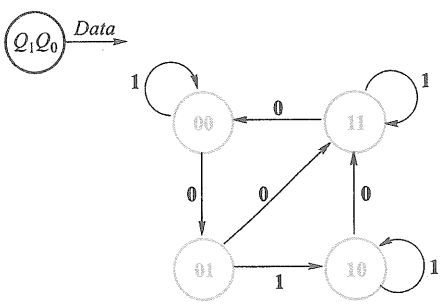


图 6.7.5 一个穆尔型时序电路的状态转换图

```
module MooreMachine(
    input Data,CP,CR,
    output [1:0] Q
);
  reg [1:0]state;      //两个变量记忆电路状态
  parameter   S0=2'b00,
              S1=2'b01,
              S2=2'b10,
              S3=2'b11;
always @(posedge CP,negedge CR)
  begin
    if(!CR) state <= S0;   //进入初态 S0
    else
      case(state)          //电路状态转换
        S0:  if(~Data) state<= S1;
        S1:  if(Data)  state <= S2;
             else      state <= S3;
        S2:  if(~Data) state<= S3;
        S3:  if(~Data) state<= S0;
      endcase
  end
  assign Q=state;   //电路的输出
endmodule
```

图 6.7.6 图 6.7.5 的 Verilog 代码

接着,用一个 **always** 语句描述电路的状态转换以及次态逻辑。异步复位信号 CR = 0 将电路置入初始状态 S0;当 CR = 1 且 CP 上升沿到来时,根据电路的现态和输入信号,确定下一个状态,这里用 **case** 语句的多个分支进行描述。

最后用 **assign** 语句描述了电路的输出,该电路的输出与电路现态相同。注意,在 CP 信号的两个上升沿之间,输入信号 Data 的变化对输出信号是没有影响的。

上例描述状态转换图的方法仅仅用了一条 **always** 语句,看起来非常简单,但电路结构却不够清晰,电路的现态和次态均用变量 state 表示,并且该方法对米利型时序电路不适用,因为米利型电路输出可能直接受到与时钟无任何关系的输入信号的作用。最好用两个 **always** 块对状态转换图建模,下面举例说明。

例 6.7.6 图 6.3.7 是用于检测连续输入序列 **110** 的状态转换图，试用 Verilog 描述该状态转换图所表示的行为。

解： 检测 **110** 序列的状态转换图的 Verilog 代码如图 6.7.7 所示。

```
module  Mealy_sequence_detector(
  input A,CP,CR,
  output reg Y
);
  reg [1:0]current_state,next_state; //中间变量
  parameter  S0=2'b00,S1=2'b01,S2=2'b11;
  always @(negedge CP,negedge CR)//时序电路
  begin
    if(~CR)   current_state<= S0;
    else      current_state<= next_state; //状态转换
  end
  always @(current_state,A)  //组合电路
  begin
    case(current_state)  //准备下一状态, 电路输出
      Y=0;
      S0:begin next_state=(A==1)?S1:S0;end
      S1:begin next_state=(A==1)?S2:S0;end
      S2:if(A==1)    begin next_state = S2;end
         else        begin Y=1;next_state = S0;end
      default:       begin next_state = S0; end
    endcase
  end
endmodule
```

图 6.7.7 检测 **110** 序列的 Verilog 代码

这里使用了两个并行执行的 **always** 块，通过公用变量（next_state）相互交换信息。第一个 **always** 块使用边沿触发信号描述电路的同步时序逻辑部分，第二个 **always** 块使用电平敏感信号描述组合逻辑部分。

第一个 **always** 块是将电路异步复位到初始状态 S0，并在 CP 的下降沿将 next_state 的值赋给 current_state。

第二个 **always** 块完成电路输出并为下一个状态做准备。现态 current_state 和输入数据 A 作为它的敏感信号，只要其中的任何一个信号发生变化，就会执行顺序语句块内部的 **case** 语句，完成下一个状态的准备和信号输出。由于 A 发生变化将直接影响输出 Y，因此是米利型电路。

综上所述，利用 Verilog 语言描述状态转换图主要包含四部分内容：

（1）利用参数定义语句 **parameter** 描述状态机中各个状态的名称，并指定状态编码。例如，对序列检测器的状态分配可以使用最简单的自然二进制码，其描述如下：

parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

或者，**parameter** [1:0] S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

注意，在语法上使用 S2 = 3 的形式定义状态也是正确的，但存储“3”这个整数至少要使

用 32 位寄存器，而存储 2'b11 只需要两位寄存器，所以例题中使用的定义方式更好一些。上面第二种方式的定义中，明确地指出使用两个状态触发器，对逻辑综合更为有利。

（2）使用 **always** 语句块描述时钟控制的同步时序逻辑部分，即完成电路状态的转换。

（3）使用 **always** 语句和 **case** 语句（也可以采用 **if-else** 等价语句）描述电路的组合逻辑部分，即现态和输入信号作为组合电路的输入，计算出组合电路的输出，作为下一个状态的值。

（4）在现态或/和输入信号条件下，描述状态机的输出逻辑。

6.7.4 数字钟的 Verilog 建模

计数器最常见的应用之一是数字钟——用数字显示时、分和秒的钟表。为了保证数字钟计时的准确性，通常采用石英晶体振荡器作为时钟脉冲源，石英振荡器的频率较高（例如 32 768 Hz），所以，需要分频到 1 Hz 作为 *CP* 信号。数字钟电路经常会用到分频器，下面先介绍分频器的设计。

例 6.7.7 试用 Verilog 设计一个分频电路，将 50 MHz 信号分频产生 1 Hz 的秒脉冲，要求输出信号的占空比为 50%。

解：设计一个模数为 25×10^6 的二进制递增计数器，其十进制数的计数范围是 0～24 999 999，每当计数器计到最大值时，输出信号翻转一次，则分频后的频率 = 50 MHz/(2×(24 999 999+1)) = 1 Hz，且占空比为 50%。其代码如图 6.7.8 所示。

```
module Divider50MHz   #(
        parameter CLK_Freq=50000000,   //50MHz 时钟输入
        parameter OUT_Freq=1,          //1 Hz 时钟输出
  ) (
    input        CR,CLK_50M,           //输入端口
    output reg   CLK_1HzOut            //输出端口及数据类型
  );
    reg [24:0] Count_DIV;              //内部节点
  always@(posedge CLK_50M, negedge CR)
  begin
    if(!CR)   begin
                CLK_1HzOut <= 0;       //输出被异步清零
                Count_DIV<= 0;         //分频器输出被异步清零
              end
    else begin
      if( Count_DIV< (CLK_Freq/(2*OUT_Freq)-1))   //计数值小于最大计数值
                Count_DIV<= Count_DIV+1'b1;       //分频计数器增 1 计数
      else begin
                Count_DIV<= 0;                    //分频器的输出被清零
                CLK_1HzOut <= ~CLK_1HzOut;        //输出信号取反
        end
      end
  end
endmodule
```

图 6.7.8 分频器的 Verilog 代码

例 6.7.8 采用分层次、分模块的方法，用 Verilog 语言设计具有时、分、秒计时的数字钟，按 24 小时制计时。要求：

（1）输出用 8421BCD 码表示时、分、秒信号，输入计时脉冲频率为 1 Hz。

（2）具有分、时校正功能，校正输入脉冲频率为 1 Hz。

解：（1）设计分析

数字钟的组成框图如图 6.7.9 所示，它由两个 60 进制计数器、1 个 24 进制计数器和两个 2 选 1 选择器共 5 个模块构成，时、分、秒 3 个同步计数器输出 8421BCD 码，并与译码显示电路相连。

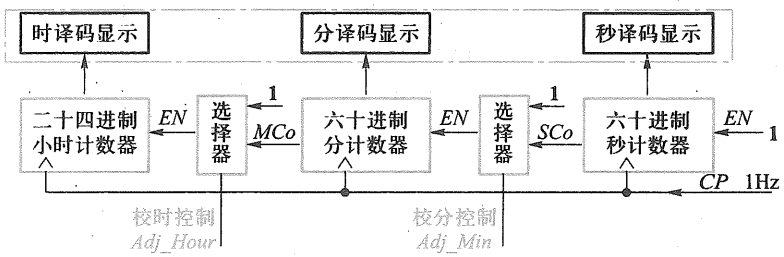


图 6.7.9 数字钟组成框图

两个选择器分别用于选择分钟和小时计数器的使能控制信号。正常情况下，分钟和小时计数器的工作是由低位来的进位信号控制的，当秒计数器计到 59 秒时，$SCo=1$ 使分计数器加 1，分和秒计数器计到 59 分 59 秒时，$MCo=1$ 使小时计数器加 1。校正小时或分钟的时候，通过控制 *Adj_Hour* 或 *Adj_Min* 使相应计数器的使能信号 $EN=1$，此时每来 1 个 *CP*，计数器加 1 计数，从而实现校正。

（2）逻辑设计

这里采用自顶向下的设计方法，首先划分顶层模块以及各个子模块的功能，确定它们之间的端口连接。然后用 Verilog 描述顶层模块及各个子模块的功能。

整个项目分为两个层次 4 个模块，其层次结构如图 6.7.10 所示。顶层模块（top_clock.v）通过实例引用底层的 3 个子模块完成数字钟的计时功能。底层有 3 个子模块，6 进制（counter6.v）、10 进制（counter10.v）和 24 进制（counter24.v），分和秒都引用 6 进制和 10 进制组成 60 进制计数器。实现上述功能的 Verilog HDL 代码如图 6.7.11 所示。

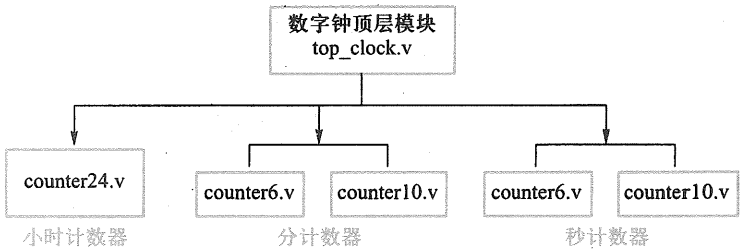


图 6.7.10 数字钟程序的层次结构图

```
//*************时钟顶层模块(top_clock.v)**********
module top_clock(
 input   CP, nCR, EN ,Adj_Min, Adj_Hour,
 output [7:0]Hour, Minute, Second
);
 supply1 Vdd;
 wire SecH_EN, MinL_EN, MinH_EN, Hour_EN;   //中间节点

 //60 进制秒计数器:调用 10 进制和 6 进制底层子模块构成
 counter10 U1(Second[3:0], nCR, EN, CP);          //秒计数器个位
 counter6   U2(Second[7:4], nCR, SecH_EN, CP);//秒计数器十位
      assign SecH_EN = (Second[3:0]==4'h9) & EN;

 //产生分钟计数器使能信号。Adj_Min=1,校正分钟;Adj_Min=0,分钟正常计时
 assign MinL_EN=Adj_Min?Vdd:(Second==8'h59);
 assign MinH_EN=(Adj_Min&&(Minute[3:0]==4'h9))||(Minute[3:0]==4'h9)&&(Second==8'h59);
 //60 进制分钟计数器:调用 10 进制和 6 进制底层模块构成
 counter10 U3(Minute[3:0], nCR, MinL_EN, CP);    //分计数器个位
 counter6   U4(Minute[7:4], nCR, MinH_EN, CP);   //分计数器十位

 //产生小时计数器使能信号。Adj_Hour=1,校正小时;Adj_Hour=0,小时正常计时
 assign Hour_EN=Adj_Hour ?Vdd : ((Minute==8'h59) && (Second==8'h59));
 //24 进制小时计数器:调用 24 进制底层模块构成
 counter24 U5(Hour[7:4], Hour[3:0], nCR, Hour_EN, CP);    //小时计数器
endmodule

//***************counter10.v(BCD:0~9)*************
module counter10( output reg [3:0]Q,   input nCR, EN, CP );
 always @(posedge CP,negedge nCR)
 begin
   if(~nCR)              Q<=4'b0000;        //nCR=0,异步清零
   else if(~EN)          Q<=Q;              //EN=0,暂停计数
   else if (Q==4'b1001)  Q<=4'b0000;
   else                  Q<=Q+1'b1;      //增 1 计数
 end
endmodule

//*****************counter6.v(BCD:0~5)**************
module counter6( output reg [3:0]Q,     input nCR, EN, CP );
```

```
always @(posedge CP,negedge nCR)
 begin
   if(~nCR)              Q<=4'b0000;       //nCR=0,异步清零
   else if(~EN)          Q<=Q;             //EN=0,暂停计数
   else if(Q==4'b0101)   Q<=4'b0000;
   else                  Q<=Q+1'b1;        //增 1 计数
 end
endmodule

//*************counter24.v(BCD:0~23)****************
module counter24( output reg [3:0]CntH, CntL,      input nCR, EN, CP );
always @(posedge CP, negedge nCR)
begin
   if(~nCR)          {CntH,CntL}<=8'h00;
   else if(~EN)      {CntH,CntL}<={CntH,CntL};
   else if ( (CntH>2) || (CntL>9) || ( (CntH==2) && (CntL>=3) ) )
                     {CntH,CntL}<=8'h00;//出错处理
   else if((CntH==2)&&(CntL<3))          //小时(20~23)的计数
               begin  CntH<=CntH;  CntL<=CntL+1'b1;end
   else if(CntL==9)  //小时十位计数
               begin  CntH<=CntH+1'b1;  CntL<=4'b0000;end
   else              //小时个位计数
               begin  CntH<=CntH;  CntL<=CntL+1'b1;end
end
endmodule
```

图 6.7.11 数字钟的 Verilog 代码

复习思考题

6.7.1 异步计数器和同步计数器的 Verilog 描述方式有何不同?

6.7.2 状态转换图的建模方法与前述计数器的建模有何区别?试用状态转换图的建模方法描述一个 4 位同步二进制递增计数器。

小 结

- 时序电路一般由组合电路和存储电路两部分构成。它们在任一时刻的输出不仅是当前输入信号的函数,而且还与电路原来的状态有关。时序电路可分为同步和异步两大类。逻辑方程组、状态转换表、状态转换图和时序图从不同方面表达了时序电路的逻辑功能,是分析和设计时序电路的主要工具。

- 同步时序电路是目前广泛应用的时序电路,是本章重点讨论的内容。同步时序电路的分析,首先按照给定电路列出各逻辑方程组、进而列出状态转换表、画出状态转换图和时序图,最后分析得到电路的逻辑功能。同步时序电路的设计,首先根据逻辑功能的需求,推导出原始状态转换图或原始状态转换表,有必要时需进行状态化简,继而对状态进行编码得到状态转换表,

然后根据状态转换表导出激励方程组和输出方程组，最后画出逻辑图完成设计任务。

• 时序电路的功能、结构和种类繁多。本章仅对寄存器和计数器等几种典型的时序电路模块进行了较详细的讨论。应用这些电路模块，能设计出各种不同功能的电子系统。这些模块的内部电路结构可作为设计其他逻辑电路的范例，也可作为初学者使用 HDL 以抽象方式描述时序电路时的实际映象。

• 用 Verilog 行为级描述方式可以对寄存器、移位寄存器、计数器及状态转换图等时序逻辑电路进行建模。

• 本章分别在 6.6 节和 6.7 节介绍了用可编程逻辑器件实现时序逻辑的基本原理和用 Verilog 描述时序电路的方法，它们是电子技术和计算机技术以及 EDA 技术高度发展的结果。这两节所讨论的方法和器件，尚需参考有关软件的详细文献资料并配合上机实验才能达到实际应用的程度。

习　题

6.1　时序逻辑电路的基本概念

6.1.1　已知一时序电路的状态转换表如表题 6.1.1 所示，A 为输入信号，试作出相应的状态转换图。

表题 6.1.1

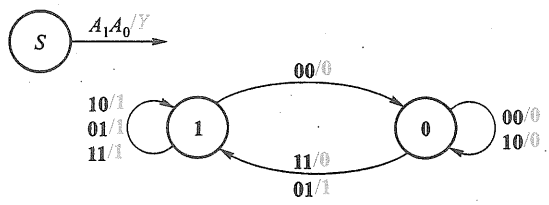
S^n	S^{n+1}/Z	
	$A=0$	$A=1$
a	$d/1$	$b/0$
b	$d/1$	$c/0$
c	$d/1$	$a/0$
d	$b/1$	$c/0$

6.1.2　已知状态转换表如表题 6.1.2 所示，输入为 A_1A_0，试作出相应的状态转换图。

表题 6.1.2

S^n	S^{n+1}/Z			
	$A_1A_0=00$	$A_1A_0=01$	$A_1A_0=10$	$A_1A_0=11$
S_0	$S_0/0$	$S_1/0$	$S_3/0$	$S_2/1$
S_1	$S_1/0$	$S_2/1$	$S_3/1$	$S_0/0$
S_2	$S_2/0$	$S_1/0$	$S_3/0$	$S_3/0$
S_3	$S_3/0$	$S_0/1$	$S_2/0$	$S_2/0$

6.1.3 已知状态转换图如图题 6.1.3 所示，试列出它的状态转换表。



图题 6.1.3

6.1.4 试画出 101 序列检测器的状态转换图，已知此检测器的输入序列、输出序列如下：

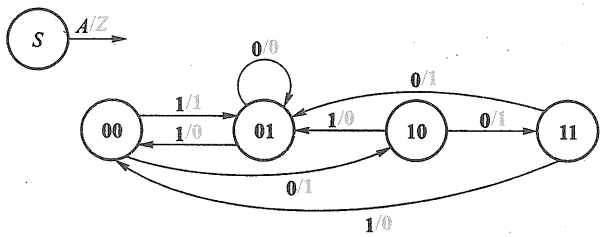
(1) 输入 A：0 1 0 1 0 1 1 0 1

输出 Z：0 0 0 1 0 1 0 0 1

(2) 输入 A：0 1 0 1 0 1 1 0 1 0

输出 Z：0 0 0 1 0 0 0 0 1 0

6.1.5 图题 6.1.5 所示是某时序电路的状态转换图，设电路的初始状态为 01，当序列 A = 100110 自左至右输入时，求该电路输出 Z 的序列。



图题 6.1.5

6.1.6 已知某时序电路的状态转换表如表题 6.1.6 所示，输入为 A，试画出它的状态转换图。如果电路的初始状态为 b，输入信号 A 依次是 0 1 0 1 1 1 1，试求其相应的输出。

表题 6.1.6

S^n	S^{n+1}/Z	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$a/1$	$d/1$
c	$b/1$	$e/1$
d	$d/0$	$c/0$
e	$b/1$	$a/1$

6.1.7 已知某同步时序电路含有两个上升沿触发的 D 触发器，其激励方程组为

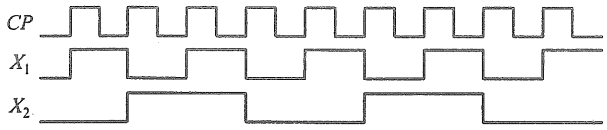
$$D_0 = X_2X_1 + X_1Q_0 + X_2Q_0$$

$$D_1 = X_2 \oplus X_1 \oplus Q_0$$

输出方程为

$$Z = Q_1$$

列出状态转换表，画出状态转换图，并分析其逻辑功能。若输入信号的波形如图题 6.1.7 所示，且电路的初始状态为 00，试画出 Q_1、Q_0的波形。

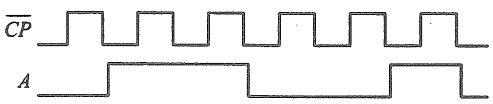


图题 6.1.7

6.1.8　已知状态转换表如表题 6.1.8 所示，若电路的初始状态为 $Q_1Q_0 = 00$，输入信号 A 的波形如图题 6.1.8 所示，输出信号为 Z，试画出 Q_1Q_0的波形（设触发器为下降沿触发）。

表题 6.1.8

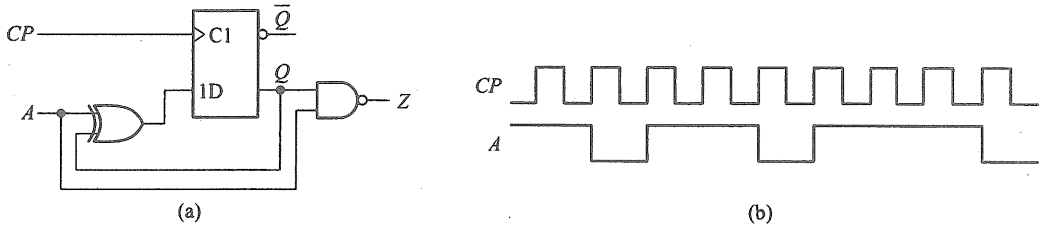
$Q_1^nQ_0^n$	$Q_1^{n+1}Q_0^{n+1}/Z$	
	$A=0$	$A=1$
0 0	0 1/1	1 1/1
0 1	1 0/0	1 0/0
1 0	1 0/0	1 1/0
1 1	0 1/1	0 0/1



图题 6.1.8

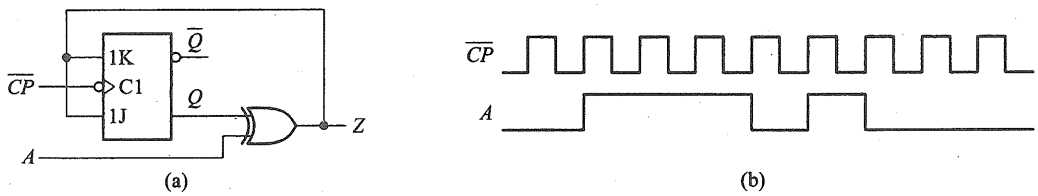
6.2　同步时序逻辑电路的分析

6.2.1　试分析图题 6.2.1（a）所示时序电路，列出状态转换表并画出状态转换图。设电路的初始状态为 0，试画出在图题 6.2.1（b）所示波形作用下，Q 和 Z 的波形图。



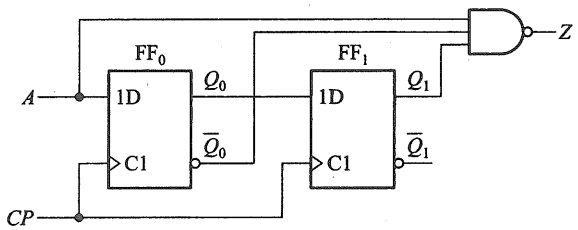
图题 6.2.1

6.2.2　试分析图题 6.2.2（a）所示时序电路，列出状态转换表并画出状态转换图。设电路的初始状态为 0，画出在图题 6.2.2（b）所示波形作用下，Q 和 Z 的波形图。



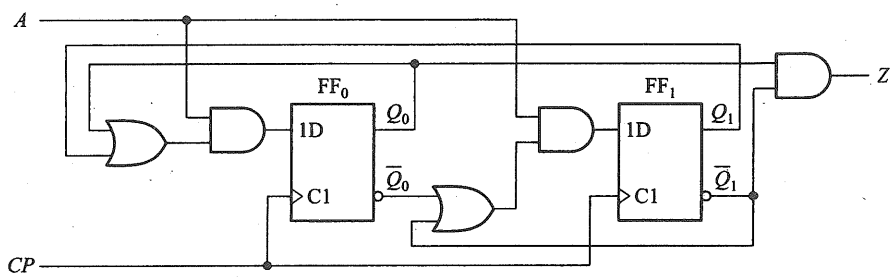
图题 6.2.2

6.2.3 试分析图题 6.2.3 所示同步时序电路，画出状态转换图。



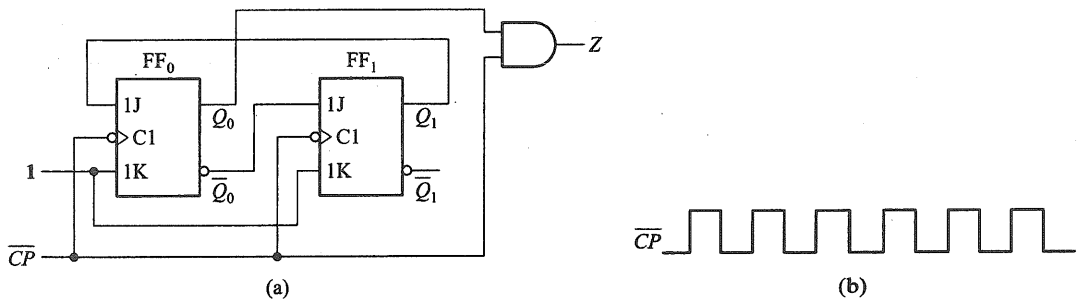
图题 6.2.3

6.2.4 试分析图题 6.2.4 所示电路，列出状态转换表，并画出状态转换图。当输入序列 A 为 **01011011101111111101**，其输出序列 Z 是什么？该电路可以检测 A 的何种输入序列？



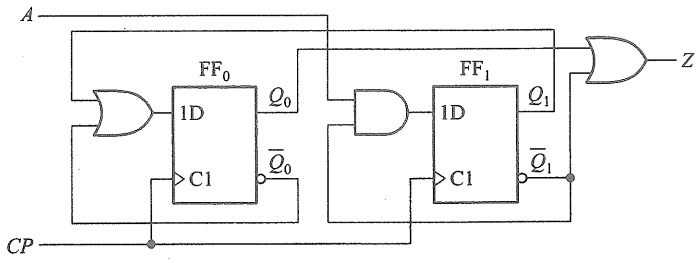
图题 6.2.4

6.2.5 试画出图题 6.2.5(a)所示同步时序电路的状态转换图，并画出对应于 $\overline{CP}$ 的 Q_1、Q_0 波形，以及输出 Z 的波形，设电路的初始状态为 **00**。



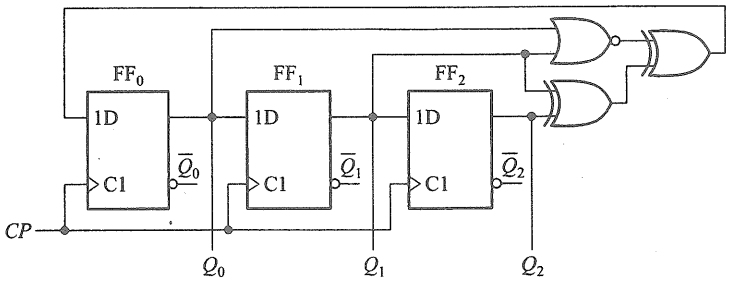
图题 6.2.5

6.2.6 试分析图题 6.2.6 所示同步时序电路，写出激励方程组、状态转换方程组和输出方程，列出状态转换表并画出状态转换图。



图题 6.2.6

6.2.7 试分析图题 6.2.7 所示同步时序电路，写出激励方程组、状态转换方程组，列出状态转换表并画出状态转换图。



图题 6.2.7

6.3 同步时序逻辑电路的设计

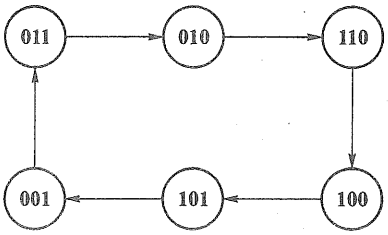
6.3.1 用 D 触发器设计一同步时序电路，其状态转换表如表题 6.3.1 所示。

表题 6.3.1

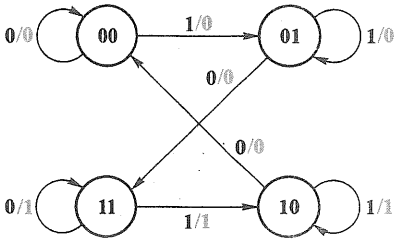
$Q_1^nQ_0^n$	$Q_1^{n+1}Q_0^{n+1}/Y$	
	$A=0$	$A=1$
00	01/0	11/0
01	10/0	00/0
10	11/0	01/0
11	00/1	10/1

6.3.2 某同步时序电路的状态转换图如图题 6.3.2 所示，试写出用 D 触发器设计时的最简激励方程组。不考虑自校正问题。

6.3.3 试用上升沿触发的 D 触发器设计一同步时序电路，其状态转换图如图题 6.3.3 所示，要求电路使用的门电路最少。

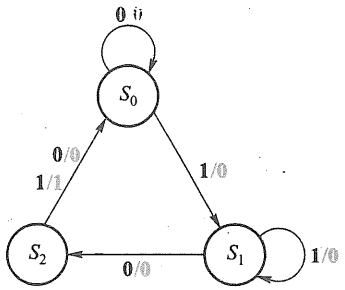


图题 6.3.2



图题 6.3.3

6.3.4 试用下降沿触发的 D 触发器设计一同步时序电路，其状态转换图如图题 6.3.4(a)所示，S_0、S_1、S_2的编码如图题 6.3.4(b)的卡诺图所示。



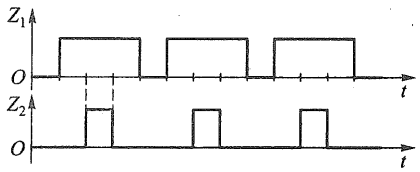
(a)

Q_1 \ Q_0		
	S_0	S_1
	S_2	×

(b)

图题 6.3.4

6.3.5 试用下降沿触发的 JK 触发器和最少的门电路，实现图题 6.3.5 所示的 Z_1和 Z_2输出波形。



图题 6.3.5

6.3.6 试用上升沿触发的 D 触发器设计一个 **1101** 序列检测器(序列可重复)，输入为串行编码序列，输出为检出信号。

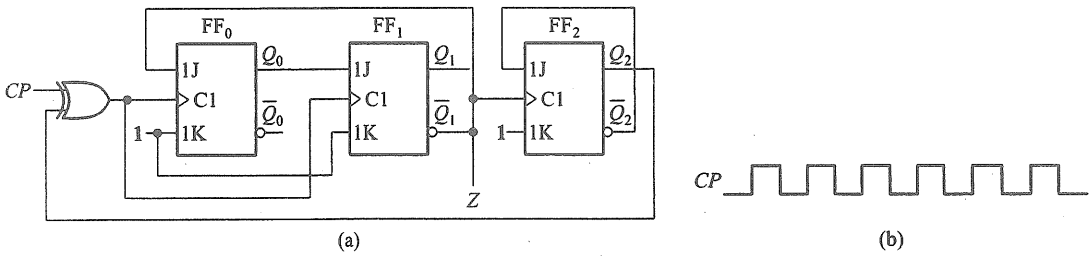
6.3.7 试用 D 触发器设计一同步时序电路，其状态转换表如表题 6.3.7 所示。

表题 6.3.7

S^n	S^{n+1}		Z
	$A=0$	$A=1$	
a	b	d	0
b	c	b	0
c	b	a	1
d	b	c	0

6.4 异步时序逻辑电路的分析

6.4.1 一时序电路如图题 6.4.1(a)所示，试画出在 CP 作用下，Q_0、Q_1、Q_2和 Z 端的波形，设各触发器的初始状态均为 0。



图题 6.4.1

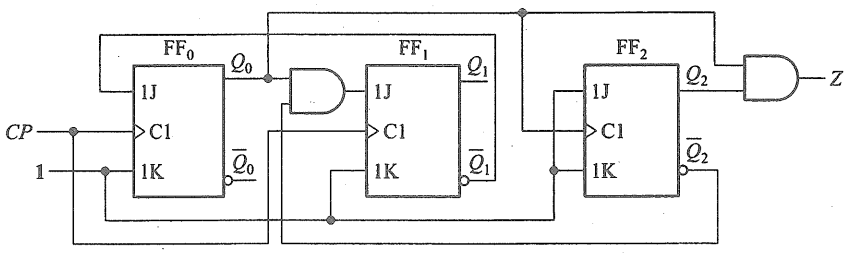
6.4.2 分析图题 6.4.2 所示时序电路[CP 脉冲同图题 6.4.1(b)]。

(1) 写出各触发器的 CP 信号方程和激励方程。

(2) 写出电路的状态转换方程组和输出方程。

(3) 列出状态转换表并画出状态转换图。

(4) 画出电路的时序图。

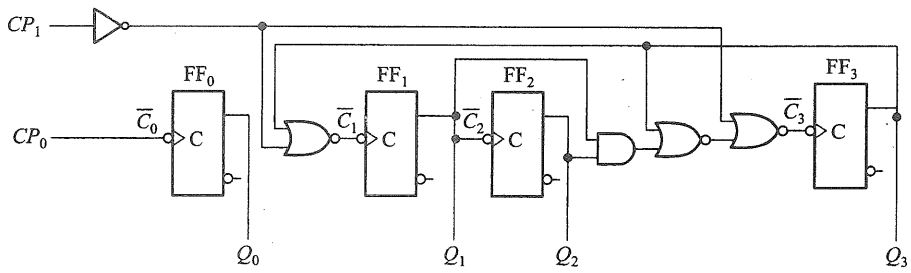


图题 6.4.2

6.4.3 由 T'触发器构成的异步 BCD 计数器如图题 6.4.3 所示。

(1) CP_0接计数脉冲信号，将 Q_0与CP_1相连，试列出状态转换表并画出状态转换图。

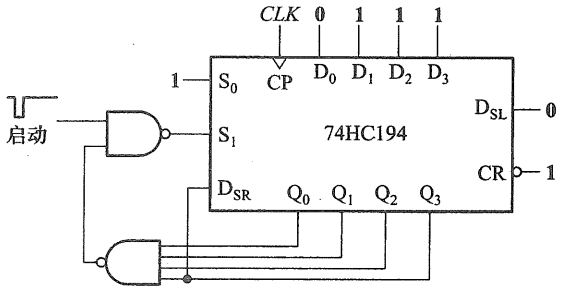
(2) CP_1接计数脉冲信号，将 Q_3与CP_0相连，再列状态转换表并画状态转换图。说明为什么该电路为 BCD 计数器？



图题 6.4.3

6.5 常用时序逻辑电路模块

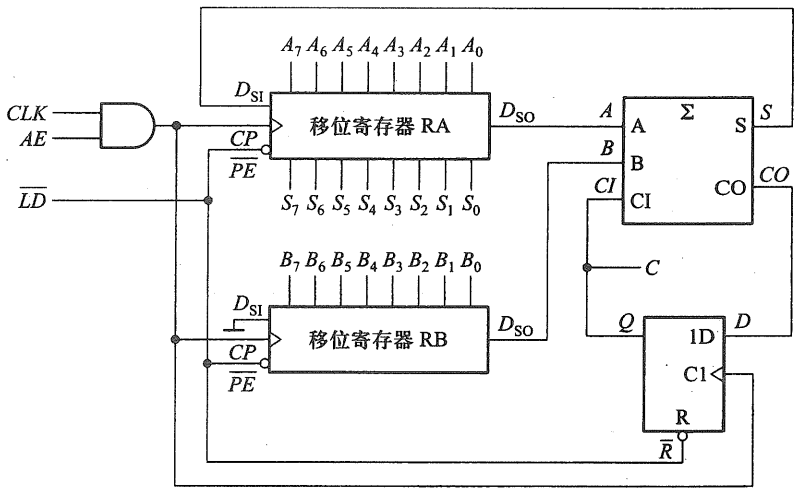
6.5.1 试画出图题 6.5.1 所示电路的输出($Q_3 \sim Q_0$)波形,并分析该电路的逻辑功能。



图题 6.5.1

6.5.2 试用两个多功能 4 位双向移位寄存器(74HC194)构成 8 位双向移位寄存器。

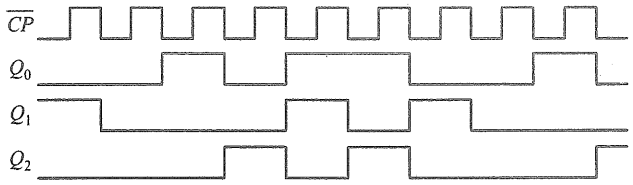
6.5.3 用一个全加器和一个 D 触发器及两个 8 位移位寄存器 RA、RB 构成的 8 位串行加法电路如图题 6.5.3 所示。图中,CLK 为时钟输入端;$\overline{LD}$ 为置数控制输入端,当 $\overline{LD}=0$ 时,8 位被加数 $A_7 \sim A_0$ 和 8 位加数 $B_7 \sim B_0$ 将分别进入移位寄存器 RA 和 RB;AE 为加运算控制端,当 $AE=1$



图题 6.5.3

时，进行串行加法运算，输入 8 个时钟脉冲后恢复为 0；$S_7 \sim S_0$ 为 8 位和输出端；C 为进位输出端。移位寄存器 RA、RB 的 CP 端为时钟输入端，$\overline{PE}$ 端为并行置数控制端，D_{SI} 和 D_{SO} 端分别为串行数据输入端、输出端。试分析电路的工作原理。

6.5.4 在某计数器的输出端观察到如图题 6.5.4 所示的波形，试确定该计数器的模。



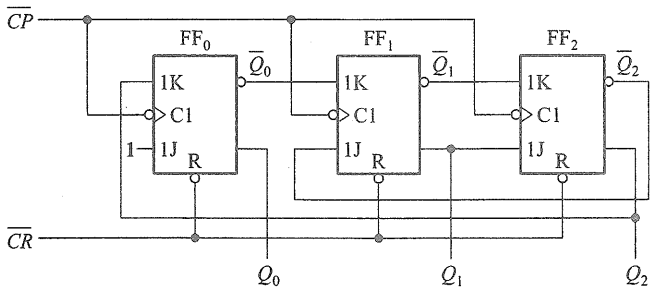
图题 6.5.4

6.5.5 试用下降沿触发的 JK 触发器组成 4 位异步二进制递减计数器，画出逻辑图。

6.5.6 试用下降沿触发的 D 触发器组成 4 位异步二进制递增计数器，画出逻辑图。

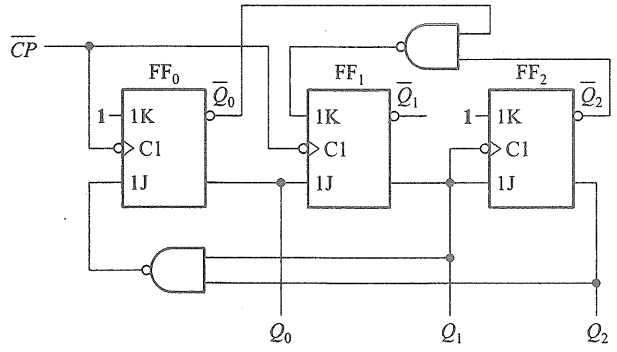
6.5.7 试用上升沿触发的 D 触发器及门电路组成 3 位同步二进制递增计数器，画出逻辑图。

6.5.8 试分析图题 6.5.8 所示计数器的模，画出各触发器输出端的波形图。



图题 6.5.8

6.5.9 计数器电路如图题 6.5.9 所示，试画出它的状态转换图，并确定它的模。

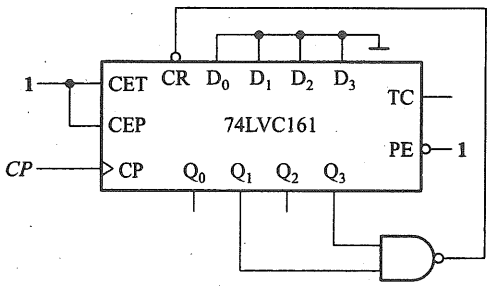


图题 6.5.9

6.5.10 试用上升沿触发的 D 触发器和门电路设计一个同步模 3 递减计数器。

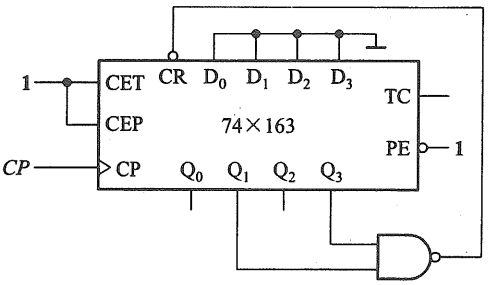
6.5.11 试用 JK 触发器设计一个同步模 6 递增计数器。

6.5.12 试分析图题 6.5.12 所示计数器，画出它的状态转换图，并确定它的模。



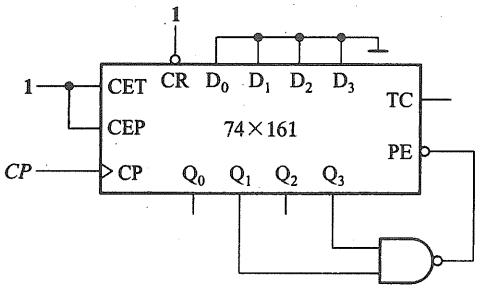
图题 6.5.12

6.5.13 试分析图题 6.5.13 所示计数器，画出它的状态转换图，并确定它的模。（74×163 是具有同步清零功能的 4 位同步二进制递增计数器，其他功能与 74×161 相同）



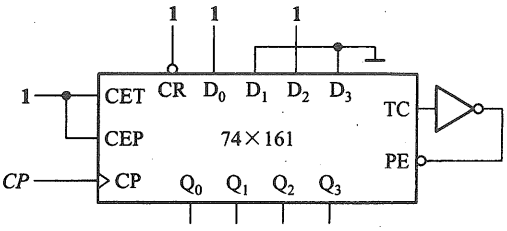
图题 6.5.13

6.5.14 试分析图题 6.5.14 所示计数器，画出它的状态转换图，并确定它的模。



图题 6.5.14

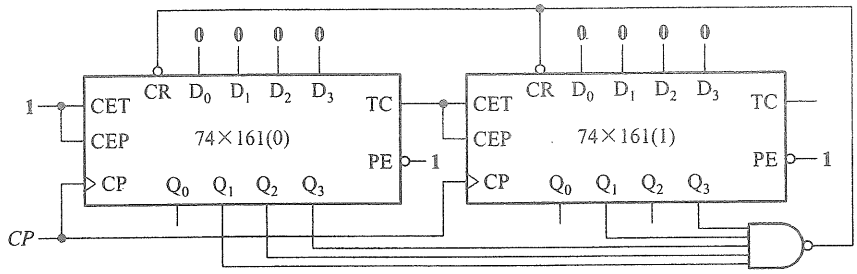
6.5.15 试分析图题 6.5.15 所示计数器，画出它的状态转换图，并确定它的模。



图题 6.5.15

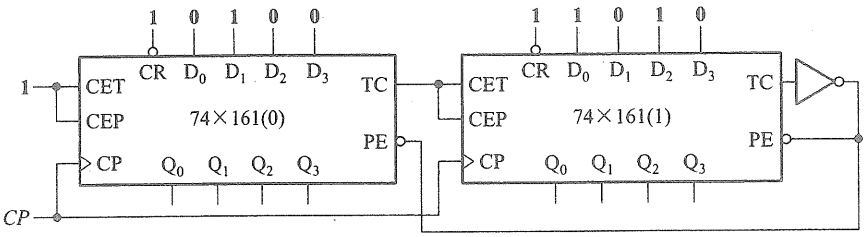
6.5.16 试用 74HC161 设计一个计数器，其计数序列以自然二进制数 1001～1111 顺序循环。

6.5.17 试分析图题 6.5.17 所示计数器，确定它的模。



图题 6.5.17

6.5.18 试分析图题 6.5.18 所示计数器，确定它的模。



图题 6.5.18

6.5.19 试用 74×161 构成同步模 24 计数器，要求采用两种不同的方法。

6.5.20 试用四个具有复位功能的 D 触发器设计一个扭环型计数器，用复位方式将计数器初始状态置为 $Q_3Q_2Q_1Q_0=0000$，并用八个 2 输入端与门对它的八个计数状态译码，画出逻辑电路图。

6.7 用 Verilog HDL 描述时序逻辑电路

6.7.1 根据图 6.5.2 所示的逻辑图，试用 Verilog 语言描述 4 位移位寄存器的功能。然后进行逻辑功能仿真，并给出仿真波形。

6.7.2 阅读下列两个模块程序，说明它们各自完成的功能。

(1)

```
module shiftn(Q,PData,SerialData,Load,CP);
    input SerialData;
    input CP,Load;
    input [n-1:0]PData;
    output reg [n-1:0]Q;
    parameter n=8;
    integer k;
always @(posedge CP)
begin
  if(Load) Q<=PData;
```

(2)

```
module counter_beh(
  input CEP,CET,PE,CP,CR,
  input [3:0]D,
  output TC,
  output reg [3:0]Q
);
  wire CE;
assign CE=CEP & CET;//CE=1 时,计数
assign TC=CET & PE &(Q==4'b1111);
always @(posedge CP,negedge CR)
```

```
    else begin
            for(k=0;k<n-1;k=k+1)
              Q[k]<=Q[k+1];
              Q[n-1]<=SerialData;
          end
end
endmodule
```

```
begin
    if(~CR)          Q<=4'b0000;
    else if(~PE)     Q<=D;
    else if(CE)      Q<=Q+1'b1;
    else             Q<=Q;
end
endmodule
```

6.7.3 阅读下列 Verilog HDL 程序，完成要求的任务。

(1) 根据图题 6.7.3 所示信号波形关系，在程序中三个位置的横线上将内容补充完整。

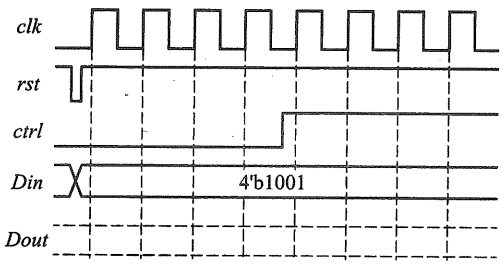
```
module test(clk,rst,ctrl,Din,Dout);
input clk,rst,ctrl;
input [3:0] Din;
output Dout;
reg [1:0] iCnt;

always @(posedge clk or _____) //第一个 always 语句块
   if(! rst)iCnt <= 2'b00;
   else if(! ctrl) iCnt <= iCnt +1'b1;
   else            iCnt <= iCnt -1'b1;

always @(iCnt or Din)          //第二个 always 语句块
        Dout = Din[iCnt];
```

(2) 第一个 **always** 语句块的功能是什么？第二个 **always** 语句块的功能是什么？

(3) 根据输入波形，画出输出信号 Dout 波形(注：水平虚线是高、低电平基准线)。



图题 6.7.3

6.7.4 下面是用 Verilog HDL 描述的电路模块代码，试按要求完成后面的任务。

```
module flip_flop(
    output reg Q,
    input A,CP,CR
);
```

```
module logic_c(output Y,output [1:0] Q,
               input X,CLK,CLR);
    wire [1:0] A;
    flip_flop ff0(Q[0],A[0],CLK,CLR);
```

```
always @ (negedge CP, negedge CR)
        if(~CR) Q <= 0;
        else if(A)
          Q <= ~Q;
endmodule
```

```
flip_flop ff1(Q[1],A[1],CLK,CLR);
assign A[1] = X ^ Q[0];
assign A[0] = 1'b1;
assign Y = Q[1] & Q[0];
endmodule
```

(1) 简述 flip_flop 模块功能。

(2) 如果将程序中的变量名 Q[1]、Q[0]分别用 Q_1、Q_0代替，写出 logic_c 电路中 Y 端的输出方程。

(3) 如果将程序中的变量名 Q[1]、Q[0]分别用 Q_1、Q_0代替，试在下面的括号中填写 logic_c 电路的状态转换方程 $Q_i^{n+1}(X,Q_1,Q_0)$ 的最小项表达式。

$$Q_1^{n+1}=\sum m(\qquad\qquad),Q_0^{n+1}=\sum m(\qquad\qquad)$$

(4) 列出 logic_c 电路的状态转换表，并画出其状态转换图。

(5) 简述 logic_c 电路的功能。

6.7.5 试用 Verilog 语言描述一个 4 位二进制可逆计数器的行为。要求如下：

(1) 电路具有 5 种功能，即异步清零、同步置数、递增计数、递减计数和保持原有状态不变。且要求计数器能输出进位信号和借位信号，即当计数器递增计数到最大值时，产生一个高电平有效的进位信号 Co；当计数器递减计数到最小值 0 时，产生一个高电平有效的借位信号 Bo。

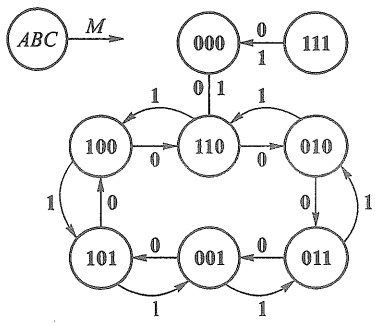
(2) 进行逻辑功能仿真，并给出仿真波形。

6.7.6 试用 Verilog 语言描述一个变模计数器，在 S 和 T 的控制下，实现同步模 5、模 8、模 10 和模 12 计数，其模数控制表如表题 6.7.6 所示，并要求具有异步清零和暂停计数的功能。然后进行逻辑功能仿真，并给出仿真波形。

表题 6.7.6 计数器的模数控制表

控制信号		模数
S	T	
0	0	模 5 计数
0	1	模 8 计数
1	0	模 10 计数
1	1	模 12 计数

6.7.7 某电路的状态转换图如图题 6.7.7 所示，图中，M 为控制变量，当 $M=0$ 时，电路按顺时针方向所指的状态进行转换；当 $M=1$ 时，则按逆时针方向进行状态转换。试用 Verilog 描述该电路的功能，并进行逻辑功能仿真，并给出仿真波形。



图题 6.7.7

6.7.8 设计一个序列检测器电路。功能是检测出串行输入数据 Data 中的 4 位二进制序列 **0101**(自左至右输入)，当检测到该序列时，输出 Out=**1**；没有检测到该序列时，输出 Out=**0**。要求：

(1) 给出电路的状态编码,画出状态转换图(注意考虑序列重叠的可能性,如**010101**,相当于出现两个**0101**序列)。

(2) 用*JK*触发器和门电路来设计此电路。

(3) 用Verilog的行为描述方式描述该电路的功能。

(4) 然后进行逻辑功能仿真,并给出仿真波形。

6.7.9 试用Verilog的行为描述方式写出数字钟的小时时间计数器程序。要求如下:

(1) 计数器的功能是从1开始计数到12,然后又从1开始,周而复始运行。计数器的输出为8421BCD码。

(2) 要求该计数器带有复位端CR和计数控制端EN。当CR为低电平时,计数器复位,其输出为1;当CR和EN均为高电平时,计数器处于计数状态;当CR为高电平但EN为低电平时,计数器暂停计数。

(3) 进行逻辑功能仿真,并给出仿真波形。

6.7.10 在例6.7.8的基础上,将计时输入脉冲频率改为1 kHz,要求增加以下功能:

(1) 具有仿电台整点报时功能。即每逢59分51秒、53秒、55秒、57秒输出500 Hz低音频信号,在59分59秒时输出1 kHz高音频信号,输出音频信号的持续时间均为1 s,高音频信号结束时,正好为整点。

(2) 增加闹钟功能,最长闹铃时间为1 min。闹钟的闹铃时刻可以任意设置(只要求对小时、分钟进行设置);闹铃信号为500 Hz和1 kHz的方波信号,两种频率的信号交替输出,且均持续1 s。设置一个停止闹铃的控制键,可以停止输出闹铃信号。

(3) 采用分层次、分模块的方法,用Verilog语言进行设计,并给出各模块的仿真波形图。

文档6.2:
第6章部分
习题答案

7 半导体存储器

引言

半导体存储器用来存储大量的二值数据，它是计算机等大型数字系统中不可缺少的组成部分，同时也广泛用于小型便携的数码产品中，如数码相机、手机等。按照集成度划分，半导体存储器属于大规模集成电路。

本章介绍半导体存储器的基本结构和工作原理，以及各类存储器的特点。用户实际使用时更需要关注存储器的读写控制时序和容量扩展等内容，本章对此也做了介绍。

文档 7.1
第 7 章思维导图

7.1 只读存储器

目前，半导体存储器基本上可以分为两大类，即只读存储器（read-only memory，ROM）和随机存取存储器（random access memory，RAM，又称为读写存储器）。本节先介绍 ROM。

ROM 在断电后，其中的数据不会消失，可以长久保存。根据是否允许用户对 ROM 写入数据，又可将 ROM 分为固定 ROM（或掩模 ROM）和可编程 ROM（programmable read-only memory，PROM）。PROM 又可分为一次可编程 ROM（PROM），光可擦除可编程 ROM（erasable programmable read-only memory，EPROM），电可擦除可编程 ROM（electrical erasable programmable read-only memory，E^2PROM）和闪存存储器（flash memory）。

ROM 常用于存放系统程序、数据表、字符代码等相对固定的数据。E^2PROM 和 Flash 则广泛用于移动存储设备中。

7.1.1 ROM 的基本结构

存储器由存储阵列、地址译码器和输出控制电路三部分组成，其结构如图 7.1.1 所示。

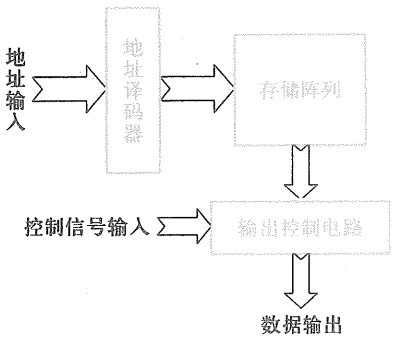


图 7.1.1 ROM 的基本结构框图

存储阵列由许多存储单元组成，每个存储单元存放 1 位二值数据。通常存储单元排列成矩阵形式，且按一定位数进行编组，每次读出一组数据，这里的组称为字。一个字中所含的位数称为字长，常以字数和字长的乘积表示存储器的容量（也称为密度）。为了区别各个不同

的字，给每个字赋予一个编号，称为地址。构成字的存储单元也称为地址单元。地址译码器将输入的地址代码译成相应的地址信号，从存储矩阵中选出相应的存储单元，并将其中的数据送到输出控制电路。地址单元的个数 N 与二进制地址码的位数 n 满足关系式 $N=2^n$。

输出控制电路一般都包含三态缓冲器，以便与系统的数据总线连接。当有数据读出时，可以有足够的能力驱动数据总线；没有数据输出时，输出高阻态不会影响数据总线。

图 7.1.2 是一个 ROM 的结构示意图，其中存储阵列由字线和位线交叉处的二极管构成。该存储器有 4 个地址单元(4 个字)，字长为 4 位，容量为 4×4 位。

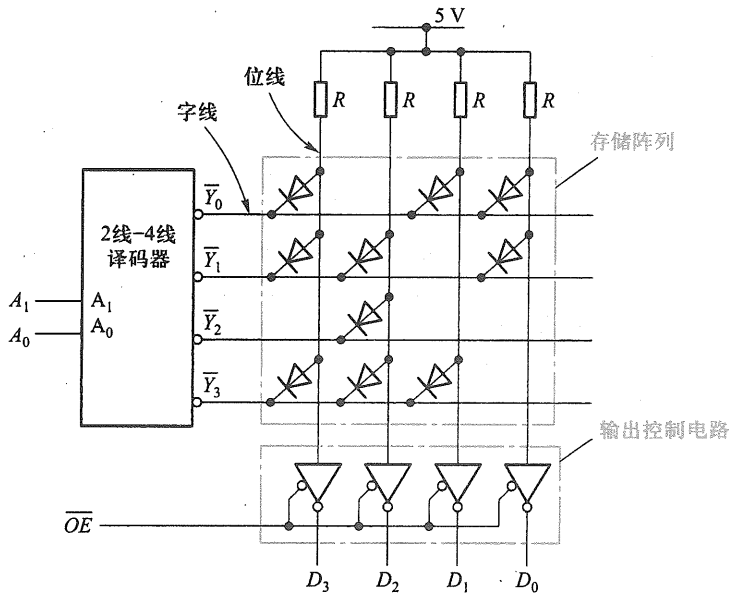


图 7.1.2 ROM 结构示意图

读操作时，如果给定的地址码为 $A_1A_0=\mathbf{01}$，译码器的 $\overline{Y}_0\sim\overline{Y}_3$ 中只有 $\overline{Y}_1$ 为低电平，则 $\overline{Y}_1$ 字线与所有位线交叉处的二极管导通，使相应的位线变为低电平，而交叉处没有二极管的位线仍保持高电平。此时，若输出使能控制信号 $\overline{OE}=\mathbf{0}$，则位线电平经缓冲器反相输出，使 $D_3D_2D_1D_0=\mathbf{1101}$。因此，ROM 属于组合电路，给定一组输入(地址)，便可得到一组输出(内容)。该 ROM 的 4 个地址内所存储的数据如表 7.1.1 所示。

表 7.1.1 图 7.1.2 ROM 存储的内容

地址 $A_1\ A_0$	内容 $D_3D_2D_1D_0$
0 0	1 0 1 1
0 1	1 1 0 1
1 0	0 1 0 0
1 1	1 1 1 0

以上分析可知，字线与位线交叉处相当于一个存储单元，此处若有二极管存在，则存储单元存有 1 值，否则为 0 值。

存储器的容量越大，意味着能存储的数据越多。例如，一个容量为 256×4 位的存储器，有 256 个字，字长为 4 位，总共有 1 024 个存储单元。存储容量较大时，字数通常采用 K、M 或 G 为单位。其中 1 K=2^{10}=1 024，1 M=2^{20}=1 024 K，1 G=2^{30}=1 024 M。

7.1.2 二维译码与存储阵列

如果采用图 7.1.2 中的译码方式，构造一个 2^8×1 位的 ROM，则需要 256 根译码输出选择线，译码电路规模显著增大，特别是大容量存储器，这种译码方式会使译码电路急剧膨胀。而采用行译码和列译码的二维译码结构，如图 7.1.3 所示，可以大大减少选择线数量，只需要行、列译码器各 16 根选择线，减小了译码电路的规模。

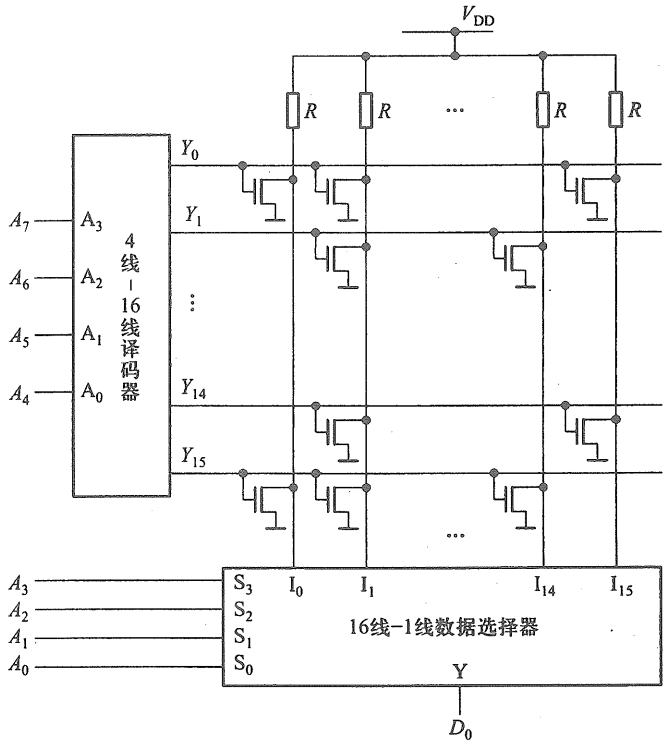


图 7.1.3 用 MOS 管构成存储单元的 ROM 结构示意图

图 7.1.3 中的存储阵列由行选择线和列选择线交叉处的 MOS 管[①]构成。当给定的地址码为$A_7A_6A_5A_4A_3A_2A_1A_0$=**00010001** 时，$A_7A_6A_5A_4$经译码器译码，输出 Y_1行线为高电平，则栅极与 Y_1相连的 MOS 管导通，使 I_1、I_{14}的位线变为低电平；而交叉处未连接 MOS 管的位线仍保持高电平，如 I_0、I_{15}的位线仍为高电平。而此时地址码的低 4 位 $A_3A_2A_1A_0$=**0001**，数据选择器选择 I_1位线输出，即 $D_0=I_1=$**0**。如果 Y_1行线和 I_1位线交叉处没有 MOS 管，$D_0=I_1=$**1**。—

① 为简明起见，本章 MOS 管采用简化符号。

般数据选择器的输出 D_0还需经反相缓冲器再输出。由此看出,4 线-16 线译码器实现行的选择,16 线-1 线数据选择器实现列的选择,从而完成行和列的译码。ROM 通过行和列交叉点上是否连有 MOS 管来存储 **0** 和 **1**。

由于二维译码需通过行、列共同作用才能选中地址单元,所以存储矩阵中水平线不再是字线,而称为行线或行选择线。

7.1.3 可编程 ROM

1. 掩模 ROM

由图 7.1.2 和图 7.1.3 看出,存储阵列可用二极管构成,也可用 MOS 管或 BJT 管构成。这类 ROM 也称为固定 ROM 或掩模 ROM,器件制造商根据用户提供的 ROM 存储内容,在制造时,利用掩模技术把数据写入存储器中(构建存储阵列中字线与位线交叉处二极管的有、无),一旦 ROM 制成,其存储的数据也就固定不变了。

2. 一次性编程的 PROM

一次可编程存储器 PROM 的存储阵列由 4.5.1 节介绍的带熔丝或反熔丝的半导体器件构成。出厂时,PROM 存储内容全为 **1**(或者全为 **0**),用户通过专用编程器,将要写入数据单元的熔丝烧断(反熔丝是接通)来改写存储的内容,只能烧写一次。

3. 可重复编程的 ROM

(1) EPROM

可擦除可编程存储器 EPROM 的存储阵列由 SIMOS 管构成,其数据写入需要通用或专用的编程器。EPROM 芯片的封装外壳装有透明的石英盖板,用紫外线或 X 射线照射 15~20 min,便可擦除其全部内容。擦除后可重新写入数据。如今大多数 PROM 实际上是不装透明石英盖板的 EPROM,因而无法擦除,只能写入一次,也称 OTP(one time programmable)EPROM。

(2) E^2PROM

电可擦除可编程存储器 E^2PROM 的存储器阵列由 Flotox MOS 管构成。E^2PROM 既具有 ROM 的非易失性,又具有写入功能。改写过程就是电擦除过程(在线擦除,即不需要将芯片从电路系统中取出。可重复擦写 1 万次以上),改写以字为单位进行。目前,大多数 E^2PROM 芯片内部都备有升压电路。因此,只需提供单电源供电,便可进行读、擦除/写操作,这为数字系统的设计和在线调试提供了极大方便。与 EPROM 相比,E^2PROM 的存储单元电路复杂,所以集成度低。

(3) Flash 存储器

Flash 的存储器阵列由 Flash MOS 管构成。目前,Flash 存储器主要分为 NOR 型和 NAND 型两大类。NOR 型存储阵列结构类似于图 7.1.3 所示的存储阵列,列线上所有 MOS 管都是并联接地的或非(NOR)结构,只是 MOS 管换作可编程的叠栅 MOS 管。与 E^2PROM 相比,由于 NOR Flash 是单管存储单元,所以集成度大大提高,而且采用按块擦除、按字写入的方式,写入速度快于 E^2PROM。

NAND 型 Flash 存储器的存储阵列如图 7.1.4 所示。存储阵列按组划分,组内每列 MOS 管串联后接在一根位线上,其中最后一个 MOS 管接地。这样该列 MOS 管与位线及上拉电

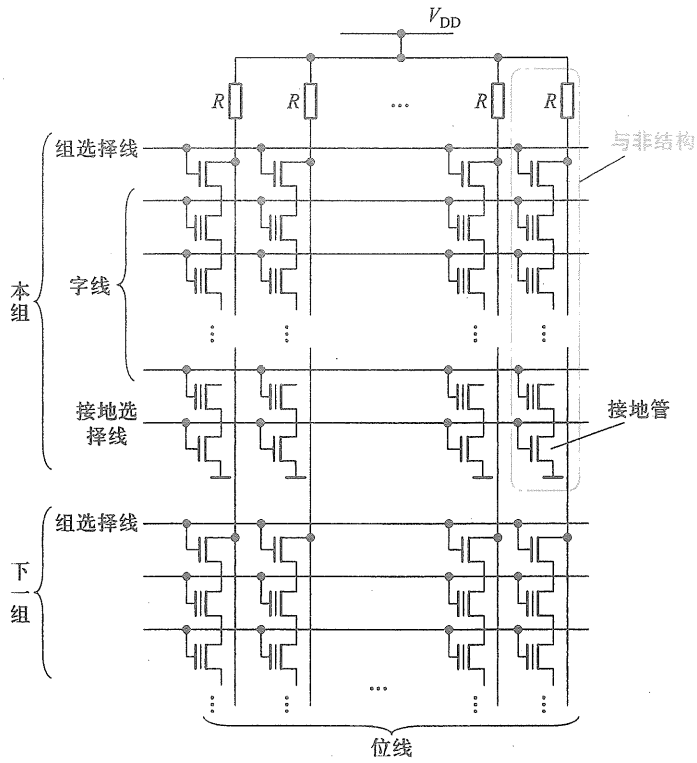


图 7.1.4 NAND 型 Flash 存储器的存储阵列示意图

阻就构成了与非(NAND)门。由于省去了大量 MOS 管的接地,从而节省了空间,集成度比 NOR 型 Flash 存储器更高。

当读取某一存储单元的内容时,首先选中相应的组,即组选择线和接地选择线均为高电平,被选中的字线也为高电平,对应 MOS 管的浮栅之前存储的数据决定该管是否导通,存 1 的 MOS 管浮栅充了电荷,因而不导通;存 0 的 MOS 管浮栅未充电荷而导通。而组内其他字线为更高的电平,使得这些 MOS 管的浮栅无论之前是否存储了电荷都可以导通。这样,位线电平仅取决于被选中的字线单元,存了 1 的输出高电平,存了 0 的输出低电平。

NOR Flash 的读写类似于 E^2PROM,可以按字读写,且大多采用并行接口,所以在计算机系统中常用于存放运行程序;而 NAND Flash 大多不能按字寻址,数据以块存取,类似于磁盘读写方式,且容量更大,通常采用串行接口,所以更多地用于数据文件存储,如计算机的固态硬盘(solid state disk,SSD)、数码相机的存储卡和 U 盘等。

上述可编程 ROM 的发展,实际上已经改变了 ROM 最初只读存储器的含义,其既有读功能,又有写功能。特别是 Flash 存储器具有的大容量、可读写、非易失性特点,使之广泛应用于各种数码产品中。为便于比较,将几种 ROM 性能列于表 7.1.2 中。

表 7.1.2 几种 ROM 性能比较

性能	Flash	ROM	EPROM	E^2PROM
非易失性	是	是	是	是
高密度	是	是	是	否
单管存储单元	是	是	是	否
在系统可写	是	否	否	是

7.1.4 ROM 实例

下面通过对 Flash 存储器 MX29F200C 的简介，使读者对实际 ROM 芯片有一个感性认识。

1. MX29F200C 简介

MX29F200C 是中国台湾旺宏电子股份有限公司①生产的 256K×8 位/128K×16 位的并行 NOR Flash。采用 5 V 电源工作，读取时间最短为 70 ns，读操作时工作电流小于40 mA。图 7.1.5 是它的框图和引脚图。图中 V_{CC}是工作电压，$\overline{OE}$是输出使能信号，$\overline{WE}$是读写控制信号，$\overline{CE}$为片选信号，$\overline{RESET}$是复位控制信号，$\overline{BYTE}$ 为字(16 位)/字节(8 位)模式选择输入端。NC 为空脚。其部分工作模式如表 7.1.3 所示。

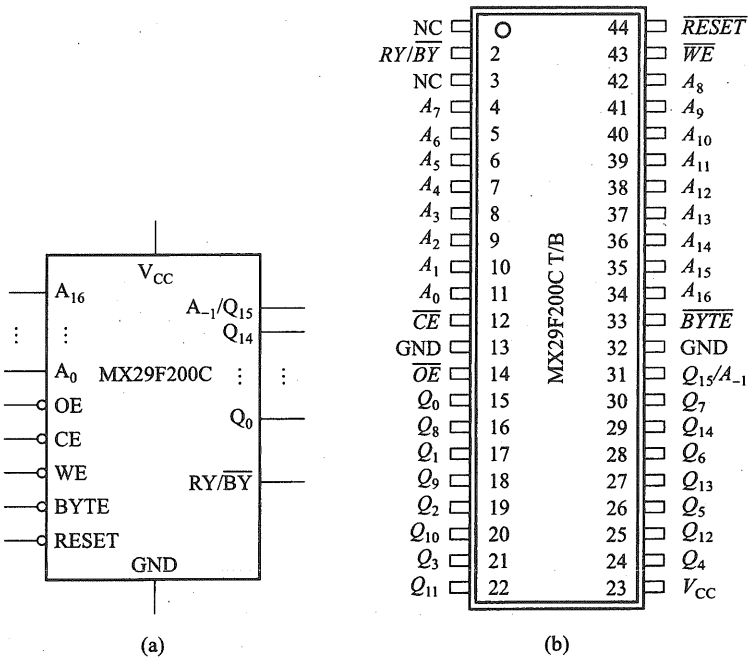


图 7.1.5 256K×8 位/128K×16 位的 NOR Flash MX29F200C
(a) 框图 (b) SOP 封装的引脚图

① 是全球非易失性存储器的重要制造商。

表 7.1.3 部分工作模式

工作模式	$\overline{RESET}$	$\overline{CE}$	$\overline{OE}$	$\overline{WE}$	$A_{16}\sim A_0A_{-1}$	$Q_7\sim Q_0$
读	H	L	L	H	A_i	数据输出
输出无效	H	L	H	H	×	高阻
等待	H	H	×	×	×	高阻
写	H	L	H	L	A_i	数据输入
复位	L	×	×	×	×	高阻

2. 读操作及时序图

为了保证存储器准确无误地工作，加到存储器的地址和控制信号必须遵守规定的时序要求，MX29F200C 的读时序如图 7.1.6 所示，阴影部分表示不确定状态。

读出过程操作如下（$\overline{WE}=1$）：

（1）将地址加到存储器的地址输入端。

（2）加入有效的片选信号$\overline{CE}$。

（3）使输出使能信号$\overline{OE}$有效，经过一定延时后，有效数据出现在数据线上。

（4）让片选信号$\overline{CE}$或输出使能信号$\overline{OE}$无效，经过一定延时后，数据线呈高阻态，本次读出结束。

图 7.1.6 中 T_{aa}为地址存取时间，表示从地址信号加到存储器上到数据输出端有稳定的数据输出所需要的时间。T_{ce}为片选存取时间，表示从片选信号$\overline{CE}$有效到数据稳定输出所需时间。T_{oe}为输出使能时间，表示从输出使能信号$\overline{OE}$有效到数据稳定输出所需时间。显然在进行读操作时，只有在地址、$\overline{CE}$和$\overline{OE}$均有效，且延迟时间均满足 T_{aa}、T_{ce}和 T_{oe}以后，被读单元的内容才能稳定地出现在数据输出端。而当地址失效后，数据输出端上的数据保持 T_{oh}后才失效。$\overline{CE}$或$\overline{OE}$失效后，经延迟 T_{df}（输出失效时间）后数据端呈高阻态。T_{rc}为读周期。

MX29F200C 延迟时间的一组典型值为：$T_{aamax}=70$ ns，$T_{cemax}=70$ ns，$T_{oemax}=30$ ns，$T_{ohmin}=0$ ns，$T_{dfmax}=20$ ns，$T_{rcmin}=70$ ns。

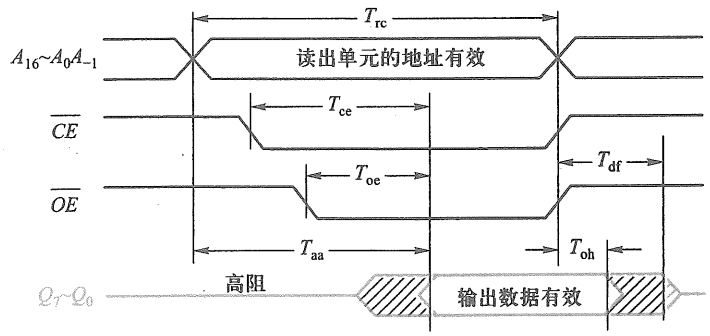


图 7.1.6 MX29F200C 的读操作时序图

MX29F200C 更详细的应用请参阅制造商的产品数据手册。

7.1.5 ROM 应用举例

ROM 常用于存放系统的运行程序或固定不变的数据。此外，由于 ROM 是一种组合逻辑电路，因此可以用它来实现各种组合逻辑函数，特别是多输入、多输出的逻辑函数。设计实现时，只需列出真值表，逻辑函数的输入作为地址，输出作为存储内容，将内容按地址写入 ROM 即可。下面举例说明 ROM 的简单应用。

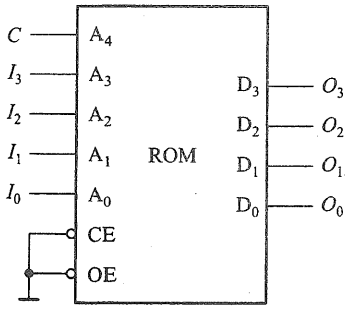


图 7.1.7 用 ROM 实现二进制码与格雷码相互转换

用 ROM 实现二进制码与格雷码相互转换的电路如图 7.1.7 所示。该电路需要用 ROM 的 5 根地址线和 4 根数据线。连接地址线最高位的 C 为转换方向控制位，待转换的代码由 $I_3\ I_2\ I_1\ I_0$ 输入，转换后的代码由 $O_3O_2O_1O_0$ 输出。当 $C=0$ 时，实现二进制码到格雷码的转换；而当 $C=1$ 时，实现格雷码到二进制码的转换。ROM 中的内容如表 7.1.4 所示（两种码的关系参见 1.4.2 节）。该 ROM 的容量至少为 $2^5\times4$ 位。

表 7.1.4 ROM 中的内容

C (A_4)	$I_3\ I_2\ I_1\ I_0$ ($A_3A_2A_1A_0$) 二进制码	$O_3O_2O_1O_0$ ($D_3D_2D_1D_0$) 格雷码	C (A_4)	$I_3\ I_2\ I_1\ I_0$ ($A_3A_2A_1A_0$) 格雷码	$O_3O_2O_1O_0$ ($D_3D_2D_1D_0$) 二进制码
0	0 0 0 0	0 0 0 0	1	0 0 0 0	0 0 0 0
0	0 0 0 1	0 0 0 1	1	0 0 0 1	0 0 0 1
0	0 0 1 0	0 0 1 1	1	0 0 1 0	0 0 1 1
0	0 0 1 1	0 0 1 0	1	0 0 1 1	0 0 1 0
0	0 1 0 0	0 1 1 0	1	0 1 0 0	0 1 1 1
0	0 1 0 1	0 1 1 1	1	0 1 0 1	0 1 1 0
0	0 1 1 0	0 1 0 1	1	0 1 1 0	0 1 0 0
0	0 1 1 1	0 1 0 0	1	0 1 1 1	0 1 0 1
0	1 0 0 0	1 1 0 0	1	1 0 0 0	1 1 1 1
0	1 0 0 1	1 1 0 1	1	1 0 0 1	1 1 1 0
0	1 0 1 0	1 1 1 1	1	1 0 1 0	1 1 0 0
0	1 0 1 1	1 1 1 0	1	1 0 1 1	1 1 0 1
0	1 1 0 0	1 0 1 0	1	1 1 0 0	1 0 0 0
0	1 1 0 1	1 0 1 1	1	1 1 0 1	1 0 0 1
0	1 1 1 0	1 0 0 1	1	1 1 1 0	1 0 1 1
0	1 1 1 1	1 0 0 0	1	1 1 1 1	1 0 1 0

另外，还可以用 ROM 实现两个 4 位二进制数的乘法运算。两个 4 位二进制数分别作为地址的高 4 位和低 4 位，它们的积需要 8 位。这样选用容量为 $2^8\times8$ 位的 ROM 便可实现该乘法运算，读者可以试着自行确定 ROM 中的内容。

随着集成电路技术的发展，ROM 的价格已变得相对低廉，加之用它实现逻辑函数非常简单易行，所以在某些情况下，用该法实现逻辑函数不失为一种有效的方法。实际上，8.1 节的现场可编程门阵列就是借鉴这种方法实现逻辑函数的。

复习思考题

7.1.1 什么是存储器的数据非易失性？

7.1.2 在存储器的结构中，什么叫“字”？什么叫“字长”？如何标注存储器的容量？

7.1.3 一个存储容量为 256×8 位的 ROM，其地址码应为多少位？

7.1.4 哪几种 ROM 具有多次擦除重写功能？哪种 ROM 的擦除过程就是数据写入过程？

7.1.5 用 ROM 实现无符号 16 位二进制数的加/减运算，要求有加/减模式控制、低位的进位输入以及进位输出。问：该 ROM 需要有多少根地址线？多少根数据线？其存储容量为多少？

7.2 随机存取存储器

RAM 是另一大类存储器，它与 ROM 的最大区别就是数据易失性，一旦失去电源供电，所存储的数据立即丢失。最大优点是可以随时快速地从其中任一指定地址读出（取出）或写入（存入）数据，而不管前一次存取的是哪一个位置，即“随机”存取。

RAM 又分为静态 RAM（static random access memory，SRAM）和动态 RAM（dynamic random access memory，DRAM）。SRAM 中的存储单元相当于一个锁存器，有 0、1 两个稳态；DRAM 则是利用电容器存储电荷来保存 0 或 1 的，因此需要定时对其进行刷新，否则随着时间推移，电容器中存储的电荷将逐渐消散。RAM 一般用在需要频繁读写数据的场合，如计算机系统中的数据缓存。下面首先介绍 SRAM。

7.2.1 SRAM

1. SRAM 的基本结构及输入输出

静态 RAM 常称为 SRAM，它的基本结构与 ROM 类似，由存储阵列、地址译码器和输入/输出控制电路三部分组成。SRAM 的一般结构框图如图 7.2.1 所示。

其中 $A_0\sim A_{n-1}$ 是 n 根地址线，$I/O_0\sim I/O_{m-1}$ 是 m 根双向数据线，其容量为 $2^n\times m$ 位。$\overline{OE}$ 为输出使能信号，$\overline{WE}$ 是写使能信号，$\overline{CE}$ 为片选信号。只有在 $\overline{CE}=0$ 时，RAM 才能进行正常读写操作，否则，三态缓冲器均为高阻，SRAM 不工作。为降低功耗，一般 SRAM 中都设计有电源控制电路（图中未画），当片选信号 $\overline{CE}$ 无效时，将降低 SRAM 内部的工作电压，使其处于微功耗状态。I/O 电路主要包含数据输入驱动电路和读出放大器，以使 SRAM 的内外电平能更好地匹配。SRAM 的工作模式如表 7.2.1 所示。

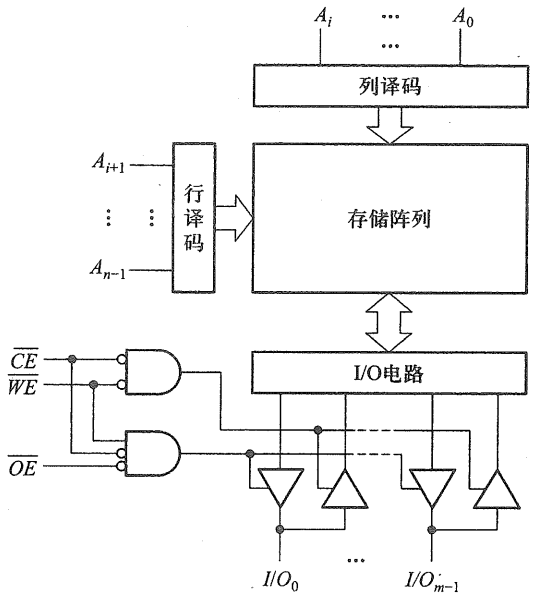


图 7.2.1 SRAM 的结构框图

表 7.2.1 SRAM 的工作模式

工作模式	$\overline{CE}$	$\overline{WE}$	$\overline{OE}$	$I/O_0 \sim I/O_{m-1}$
保持(微功耗)	1	×	×	高阻
读	0	1	0	数据输出
写	0	0	×	数据输入
输出无效	0	1	1	高阻

2. SRAM 存储单元

SRAM 与 ROM 最主要的差别是存储单元。SRAM 的存储单元是由锁存器构成的,因此它属于时序逻辑电路。图 7.2.2 画出了存储阵列中第 j 列、第 i 行的存储单元结构示意图。点画线框中为六管 SRAM 存储单元,其中 T_1 和 T_3、T_2 和 T_4 分别构成两个 CMOS 反相器,交叉连接它们的输出与输入便成为基本双稳态电路(锁存器),用来存储 1 位二值数据。X_i 为行译码器的输出,Y_j 为列译码器的输出。T_5、T_6 为本单元控制门,由行选择线 X_i 控制。$X_i=1$,T_5、T_6 导通,锁存器与位线接通;$X_i=0$,T_5、T_6 截止,锁存器与位线隔离。T_7、T_8 为一列存储单元公用的控制门,用于控制位线与数据线的连接状态,由列选择线 Y_j 控制。显然,当行选择线和列选择线均为高电平时,$T_5 \sim T_8$ 都导通,锁存器的输出才与数据线接通,该单元才能通过数据线传送数据。因此,存储单元能够进行读/写操作的条件是,与它相连的行、列选择线均须呈高电平。

由此可见,SRAM 中数据由锁存器记忆,只要不断电,数据就能永久保存。

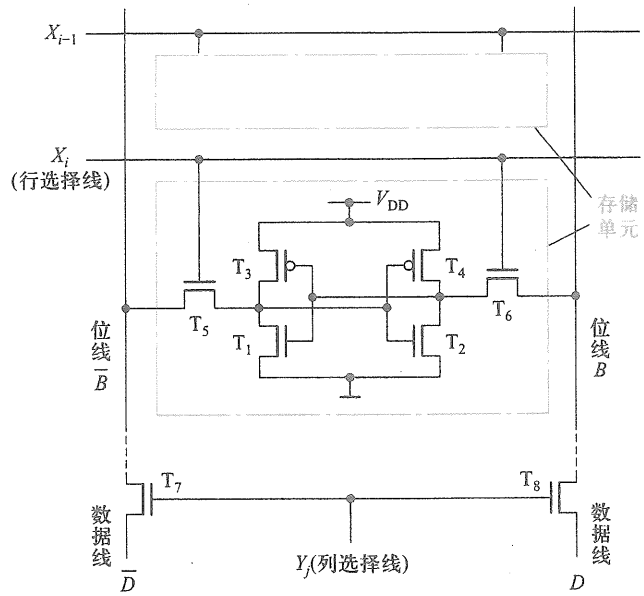


图 7.2.2 第 j 列、第 i 行的存储单元结构示意图

与 ROM 类似，RAM 的读写操作也需要遵循一定的时序，使用时请参阅器件数据手册，此处不再赘述。

3. 其他存取方式的存储器

(1), FIFO

一种称作先进先出(first-in first-out, FIFO)的存储器也得到广泛应用。所谓先进先出是指先存入的数据也先被读出，不能随机存取。意味着 FIFO 的读写地址是按一定顺序变化的。为了实现先进先出的读写功能，FIFO 的数据写入端口和数据读出端口是分开的，并且没有地址输入端口。读写数据的地址由 FIFO 内部的读地址指针计数器和写地址指针计数器来确定。写操作时，写控制信号有效，输入端口的数据写入到由写指针指向的地址单元中，然后写指针计数器加 1。读操作与此类似。该存储器必须先写入数据后，才能读出数据。当 FIFO 中的数据被读完后，表示“空”状态的输出信号有效。若写入 FIFO 中的数据没有被及时读出，可能会出现被写“满”的情况，此时表示“满”状态的输出信号有效。“空”和“满”状态信号为用户控制 FIFO 的读写提供了方便。

FIFO 存储器多用来做数据缓冲器，特别适合用于需要长时间、不间断、高速数据采集的缓冲器。

(2) 双口 SRAM

还有一种不同于 FIFO 的双口(dual-port) SRAM。这种存储器有两套完全独立完整的地址、数据和控制端口，共用同一个存储阵列。两个端口都能进行读写操作，而且能根据各自端口的地址随机存取。当出现两个端口同时访问相同的地址单元时，SRAM 内部仲裁电路将根据两个端口先后访问的微小时差，决定先完成哪个端口的访问，并输出状态信号告知另一端口将延迟其访问。双口 SRAM 为数据缓存提供了更便利的条件。

与 ROM 类似,SRAM 也有串行结构。由于串行 SRAM 芯片减少了引脚数目,所以可以做得非常小,通常用在体积要求非常小的系统中。

7.2.2 同步 SRAM

1. 同步 SRAM 的基本结构及工作原理简介

随着各种数据密集型应用(如互联网中的交换机、路由器、服务器,以及通信领域的无线基站和测试设备等)对速度要求的不断提高,RAM 也得到快速发展,同步 SRAM(synchronous SRAM,SSRAM)是在 SRAM 基础上发展起来的一种更适合高速存取的 RAM。同步 SRAM 的读写操作是在时钟脉冲节拍控制下完成的,因此,同步 SRAM 最明显的标志是有时钟脉冲输入端。为便于区别,上节介绍的 SRAM 也称为异步 SRAM(asynchronous SRAM,ASRAM)。

同步 SRAM 的基本结构如图 7.2.3 所示。为了能更好地实现同步操作,与 SRAM 相比,同步 SRAM 在电路内部增加了包括地址、数据、读写控制等各种输入信号的寄存器。除输出使能控制信号$\overline{OE}$外,所有输入均在时钟脉冲 CP 的上升沿被取样,存入寄存器,读写过程的延时等待均被限制在时钟作用间隔内。这种工作方式使其应用变得非常简便,用户只要围绕时钟的有效沿设计所有的输入信号即可。另外,可以对芯片内部的各种时延进行优化设计,使得同步 SRAM 的读写速度高于异步 SRAM。

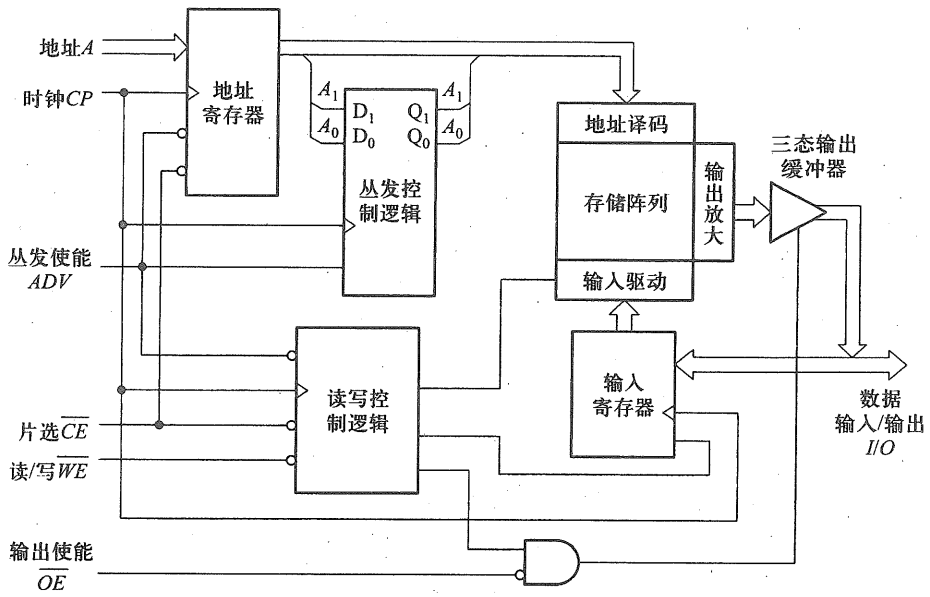


图 7.2.3 同步 SRAM 的基本结构

同步 SRAM 具有的另一个特点是丛发(burst,也译为突发)读写操作模式[①],由 ADV 控制。当 $ADV=1$ 时,地址寄存器不再接收外部新地址而保持上一次存入的地址,接下来访问

① 在同步的静态和动态 RAM 中均有丛发模式。

的地址低位由丛发控制逻辑中的计数器自行累加产生，这样便可以读/写多个连续地址的数据，而不占用地址总线。丛发计数器的位数通常有 1 位、2 位、3 位等，且可循环累加。

读操作时，在 *CP* 有效沿先将存储阵列读出的数据保存在输出寄存器中，在下一个 *CP* 有效沿被送出。

写操作时，在 *CP* 有效沿取样到写信号后，在下一个 *CP* 沿将外部数据锁存到输入寄存器，并在读写控制逻辑信号控制下，将数据写入存储阵列。

当 $ADV=0$ 时，为一般模式读写，此时地址寄存器中的地址不受丛发控制逻辑电路影响，直接送至地址译码电路，按外部给定的地址进行读/写。

目前，同步 SRAM 已广泛应用于各种同步工作的数字系统中，特别是与处理器一同工作的系统。如个人电脑中的超高速缓冲存储器(cache)。

2. 其他同步 SRAM

上述同步 SRAM 只在时钟的上升沿传输数据，并且共用读/写数据总线，读和写只能分时进行。这种同步 SRAM 也称为单倍数据传输率(single data rate，SDR)同步 SRAM。此后，各大 RAM 厂商又先后开发出双倍数据传输率(double data rate，DDR)SRAM 和四倍数据传输率(quad data rate，QDR)SRAM。

DDR 同步 SRAM 在每个时钟周期的上升沿和下降沿各传输一次数据，这样数据传输效率提高了一倍，但是读写仍不能同时进行。

QDR 同步 SRAM 进一步改进了结构，为读和写操作分别提供独立的接口，不但在每个时钟周期的上升沿和下降沿共传输两次数据，而且每次读写能够同时进行，避免了数据总线的争抢，使数据传输效率比同步 SRAM 提高了两倍。

对 DDR 和 QDR 某些性能进行改善后的产品称为 DDR Ⅱ 和 QDR Ⅱ 同步 SRAM。

表 7.2.2 中列出了几种 SRAM 产品的几个主要指标。

表 7.2.2 几种 SRAM 产品的主要指标

型号	类型	容量(密度)	字×位	存取时间 ns	时钟频率 MHz	工作电压	工作电流	厂家
CY7C199D	SRAM	16 Mbit	512K×32	10	–	3.3 V	150 mA	Cypress
IDT71V3556	SSRAM	4.5 Mbit	128K×36	5	200	3.3 V	400 mA	IDT
CY7C1440KV25	SSRAM	36 Mbit	1M×36	2.5	250	2.5 V	240 mA	Cypress
CY7C1612KV18	QDR Ⅱ	144 Mbit	8M×18	–	360	1.8 V	1 025 mA	Cypress

注：对于 DDR、DDR Ⅱ 和 QDR Ⅱ，工作电压指核心工作电压。

7.2.3 DRAM

1. DRAM 存储单元

图 7.2.2 所示的 SRAM 存储单元由 6 个 MOS 管构成，所用的管子数目多、功耗大，集成度受到限制，动态 RAM 克服了这些缺点。动态 RAM 也称为 DRAM，其存储单元由一个 MOS 管和一个容量较小的电容器构成，如图7.2.4 中点画线框内所示。它利用电容器的电

荷存储效应来存储数据 0 或 1。当电容 C 充有电荷、呈现高电压时，相当于存有 1 值，反之为 0 值。MOS 管 T 相当于一个受行选择线控制的开关。由于电路中漏电流的存在，电容器上存储的数据（电荷）不能长久保存，因此必须定期给电容补充电荷，以免存储数据丢失，这种操作称为刷新（refresh）或再生。一般的刷新间隔时间为 15.6 ms（或 7.8 ms），刷新时间小于 100 ns。可见刷新时间只占刷新周期的 0.64%，对正常读/写操作影响很小。

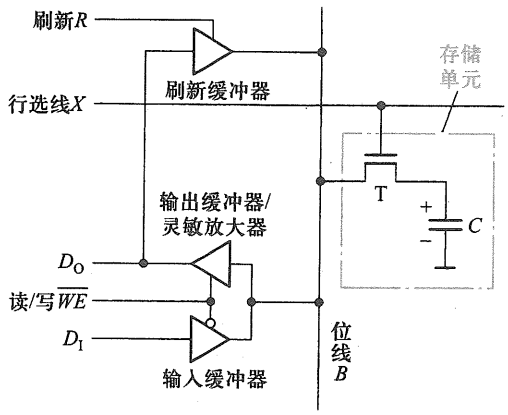


图 7.2.4 动态存储单元及基本操作原理

写操作时，行选线 X 为高电平，T 导通，电容器 C 与位线 B 连通。同时读写控制信号 $\overline{WE}$ 为低电平，输入缓冲器被选通，数据 D_I 经缓冲器和位线写入存储单元（外部输入/输出引脚上的数据经列选通电路送至 D_I。图中未画列选通电路）。如果 D_I 为 1，则向电容器充电，反之电容器放电。未选通的缓冲器呈高阻态。

读操作时，行选线 X 为高电平，T 导通，电容器 C 与位线 B 连通。此时读写控制 $\overline{WE}$ 为高电平，输出缓冲器/灵敏放大器被选通，C 中存储的数据通过位线和缓冲器由 D_O 输出（D_O 再经列选通电路送至最终的输出引脚）。由于读出时会消耗 C 中的电荷，存储的数据被破坏，故每次读出后，必须及时对读出单元刷新，即此时刷新控制 R 也为高电平，则读出的数据又经刷新缓冲器和位线对电容器 C 进行刷新。

除了读、写操作可以对存储单元进行刷新外，刷新操作也可以通过只选通行选择线来实现。例如，当行选线 X 为高电平，且 $\overline{WE}$ 亦为高电平时，C 上的数据经 T 到达位线 B，然后经输出缓冲器和刷新缓冲器对存储单元刷新，此时的刷新是整行刷新。由于未进行列选通，所以 D_O 的数据不会送至最终的输出引脚。实际上，刷新操作时输出缓冲器和刷新缓冲器环路构成一正反馈，如果位线为高电平，则将位线电平拉向更高；反之则使位线电平降得更低。

由于存储单元电容的容量很小，所以在位线容性负载较大时，C 中存储的电荷（C 存有 1 时）可能还未将位线拉至高电平时便耗尽了，由此出现读出错误。为避免出现这种情况，通常在读之前先将位线电平预置为高、低电平的中间值。这样，T 导通时，根据电容 C 存储的是 0 还是 1，会将位线拉向低电平或高电平。位线电平的这种变化经灵敏放大器放大，可以准确得到 C 所存储的逻辑值。

2. DRAM 的基本结构

由于 DRAM 的集成度很高，存储容量大，因此需要较多的地址线。为减少引线数目，DRAM 大都采用行、列地址分时送入的方法。例如，对于一个 1 M 字的存储器，有 2^{20} 个地址，即有 20 根地址线。采用行、列地址分时送入时，只需要 10 根地址线加少量控制线。DRAM 的基本结构如图 7.2.5 所示，其内部设有行、列两个地址寄存器。通常先将行地址（地址码高位部分）加到地址线 A 上，在行地址选通信号 $\overline{RAS}$（row address strobe）控制下送入行地址寄存器中，然后将列地址（地址码低位部分）加到地址线 A 上，在列地址选通信号 $\overline{CAS}$

(column address strobe)控制下送入列地址寄存器中。此外,DRAM 内部还设有刷新计数器和刷新控制及定时电路,由此可以自动产生行地址进行刷新。

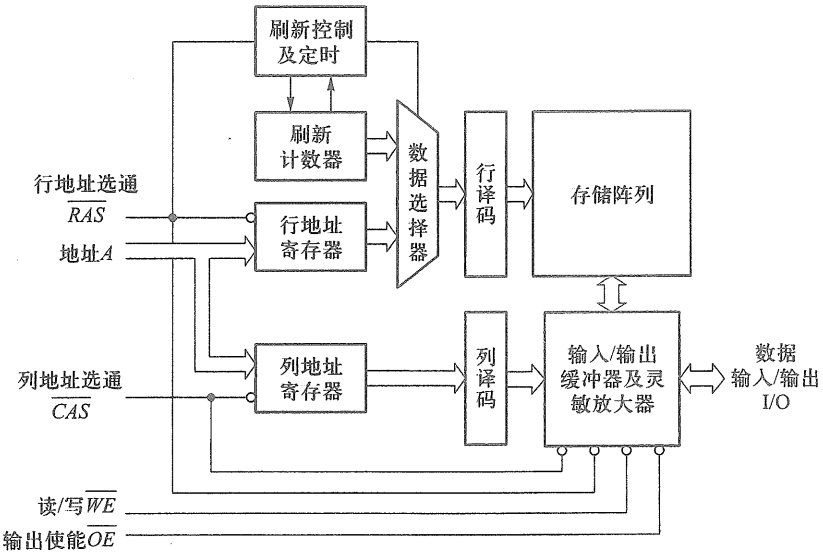


图 7.2.5 DRAM 的基本结构

因为需要定时刷新,所以 DRAM 的操作方式比 SRAM 要复杂些,除了正常的读写外,还有多种不同的刷新操作方式,具体请参阅所选用器件的数据手册,此处不再赘述。

与 SRAM 的发展类似,DRAM 也有同步 DRAM(synchronous DRAM,SDRAM)、DDR 同步 DRAM 和 QDR 同步 DRAM。由于 DRAM 的存储单元结构简单,其集成度远高于 SRAM,所以同等容量情况下,DRAM 更廉价。

目前,改进型 DDR Ⅱ(二代)和 DDR Ⅲ(三代)同步 DRAM 已成为个人电脑的主流内存。

7.2.4 存储容量的扩展

目前,尽管各种容量的存储器产品已经很丰富,且最大容量已达 32 Gbit 以上,用户能够方便地选择满足需要的芯片。但是,只用单个芯片不能满足存储容量要求的情况仍然存在,个人电脑中的内存条就是一个典型的例子,它由焊在一块印刷电路板上的多个芯片组成。如要构成容量为 8 G×32 位的存储器,就需要 8 片上述容量的芯片。此时,便涉及存储容量的扩展问题。

扩展存储容量的方法可以通过增加字长(位数)或字数来实现。

1. 字长(位数)的扩展

通常 RAM 芯片的字长为 1 位、4 位、8 位、16 位和 32 位等。当实际存储器系统的字长超过 RAM 芯片的字长时,需要对 RAM 实行位扩展。

可以用芯片的并联方式实现位扩展,即将 RAM 的地址线、读/写控制线和片选信号对应地并联在一起,而各个芯片的数据输入/输出端作为字的各个位线。如图 7.2.6 所示,用 4 个 4K×4 位 RAM 芯片可以扩展成 4K×16 位的存储系统。

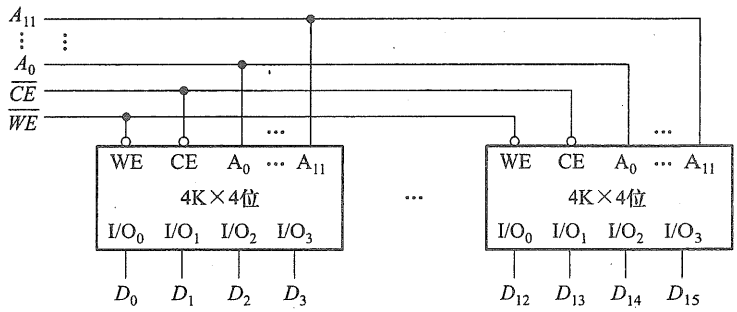


图 7.2.6 用 4K×4 位 RAM 芯片构成 4K×16 位的存储器系统

2. 字数的扩展

字数的扩展必然涉及地址码位数的增加，将增加的地址码作为高位，利用外加译码器译码后控制存储器芯片的片选使能输入端，就可以实现字扩展。例如，利用 2 线-4 线译码器将 4 个 8K×8 位的 RAM 芯片扩展为 32K×8 位的存储器系统。扩展方式如图 7.2.7 所示。图中，存储器扩展所要增加的地址线 A_{14}、A_{13} 与译码器的输入相连，译码器的输出 $\overline{Y}_0 \sim \overline{Y}_3$ 分别接至 4 片 RAM 的片选信号控制端 $\overline{CE}$。这样，当输入一个地址码（$A_{14} \sim A_0$）时，只有一片 RAM 被选中，从而实现了字的扩展。

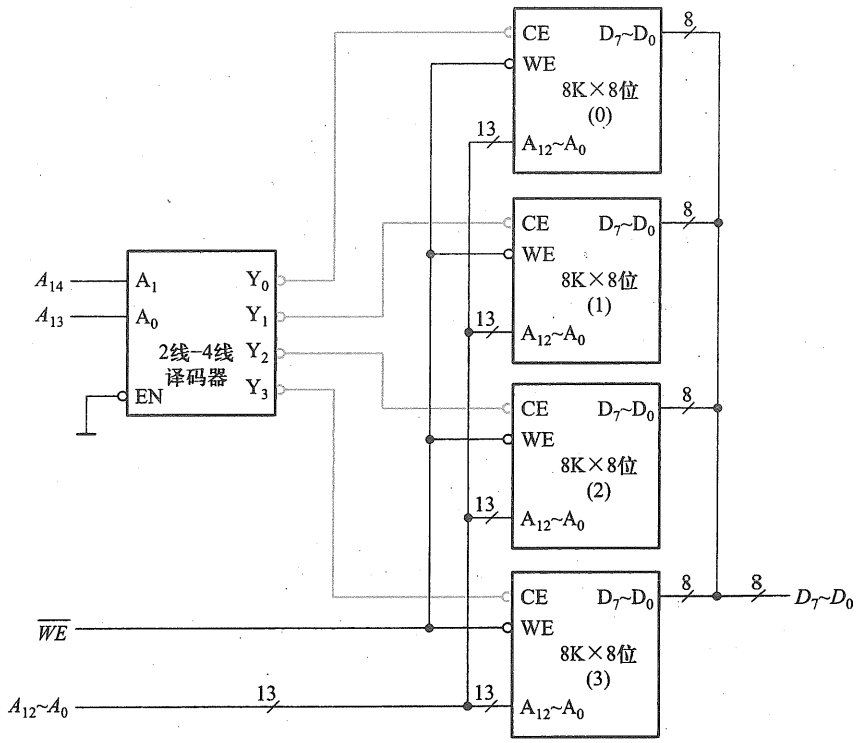


图 7.2.7 用 8K×8 位 RAM 芯片构成 32K×8 位的存储器系统

实际应用中，常将两种方法相互结合，以达到同时扩展字和位的要求。可见，无论需要多大容量的存储器系统，均可利用一定容量存储器芯片，通过位数和字数的扩展来构成。

7.2.5 RAM 应用举例

这里结合 LED 阵列动态显示电路，介绍 RAM 在其中的应用。

一种 LED 阵列动态显示原理电路如图 7.2.8 所示。显示屏由 8×32 个 LED 构成，行线和列线电平控制交叉点上 LED 的“亮”和“灭”。

当更新控制信号 $\overline{NEW}=1$ 时，电路处于动态显示工作状态。此时，数据更新控制电路直接将计数器的输出 $Q_4 \sim Q_0$ 通过 $B_4 \sim B_0$ 和 $Y_4 \sim Y_0$ 送至 RAM 的地址输入 $A_4 \sim A_0$，即 $A_4 \sim A_0 = Q_4 \sim Q_0$；而其输出 $OT_7 \sim OT_0$ 呈高阻态。RAM 则处于读操作状态，从由 $Q_4 \sim Q_0$ 决定的地址单元读出 8 位数据，经三态驱动器驱动 8 根行线。与此同时，计数器的输出 $Q_4 \sim Q_0$ 也通过 5 线-32 线译码器使显示屏的 1 根列线为低电平，这样，高电平行线与该低电平列线交叉点上的 LED 被点亮，而低电平行线与该列线交叉点上的 LED 则不亮，从而控制 1 列 LED 的显示。

例如，图 7.2.8 的显示屏要显示“CALL”时，计数器的 $Q_4 \sim Q_0 = 00000$ 时，需要从 RAM 中地址为 **00000** 的地址单元读出数据 **00111000**，控制 LED 阵列的行线 O_7、O_6、O_2、O_1、O_0 为低电平，而 O_5、O_4、O_3 为高电平；同时，$Q_4 \sim Q_0 = 00000$ 使译码器的 $\overline{Y}_0 = 0$，其他输出均为高电平，这样便点亮 LED 阵列 O_5、O_4、O_3 行与 $\overline{Y}_0$ 列交叉点上的 LED，$\overline{Y}_0$ 列上其他 LED 则是“灭”的，其余各列也都是“灭”的。类似地，在下一个 *CP* 到来时，计数器加 1 使 $Q_4 \sim Q_0 = 00001$，则 RAM 中 **00001** 地址单元的数据被读出，该数据需为 **01111100**，同时译码器的 $\overline{Y}_1 = 0$，控制 $\overline{Y}_1$ 列的显示，即该列的 O_6、O_5、O_4、O_3、O_2 行“亮”。以此类推，在 *CP* 作用下，显示屏自左向右进行逐列扫描。完成一屏扫描后，在下一个 *CP* 到来时，计数器 $Q_4 \sim Q_0$ 由 11111 变为 **00000**，开始下一屏扫描，如此循环往复。

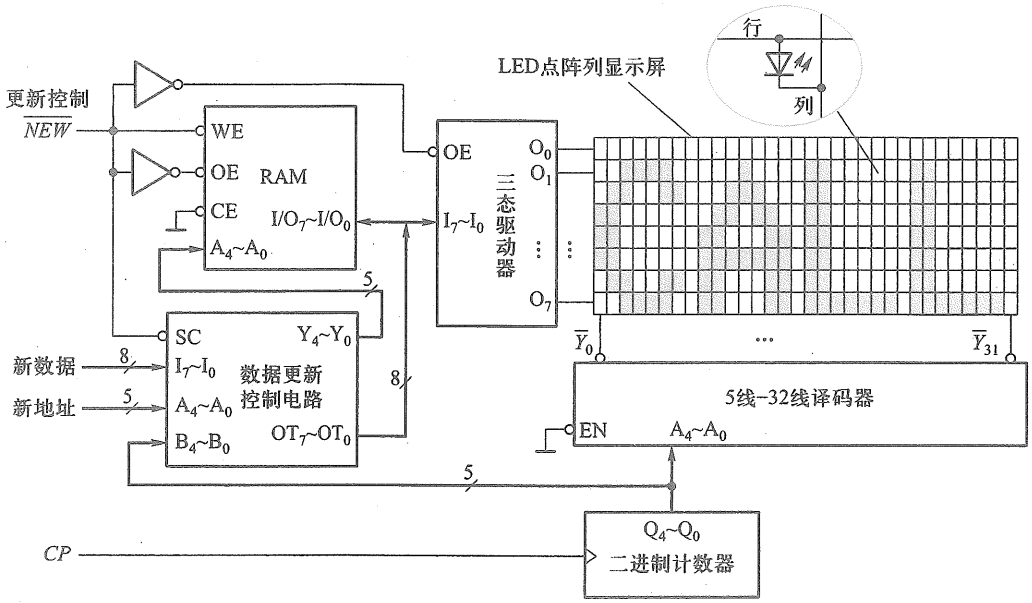


图 7.2.8 8×32 LED 点阵列动态显示原理

由上述 LED 阵列动态显示原理可知，显示屏上要显示的字符，以点阵方式存储在 RAM 中，每个点占用 1 个存储单元。需要发光的点存为 1，不发光的点存为 0。因此，8 行×32 列的显示阵列需要 32×8 位 RAM 存放一屏的数据。根据显示字符形状确定 RAM 中需要存储的点阵数据。如要显示“CALL”时，RAM 中的点阵数据应如表 7.2.3 所示。如果向 RAM 中写入不同的点阵数据，则可显示不同的字符。

表 7.2.3 显示“CALL”时 RAM 中的点阵数据

地址 $A_4A_3A_2A_1A_0$	内容 $D_7D_6D_5D_4D_3D_2D_1D_0$	地址 $A_4A_3A_2A_1A_0$	内容 $D_7D_6D_5D_4D_3D_2D_1D_0$
00000	00111000	10000	11111110
00001	01111100	10001	11111110
00010	11000110	10010	10000000
00011	10000010	10011	10000000
00100	10000010	10100	10000000
00101	11000110	10101	10000000
00110	01000100	10110	10000000
00111	00000000	10111	00000000
01000	11110000	11000	11111110
01001	11111000	11001	11111110
01010	00101100	11010	10000000
01011	00100110	11011	10000000
01100	00101100	11100	10000000
01101	11111000	11101	10000000
01110	11110000	11110	10000000
01111	00000000	11111	00000000

当$\overline{NEW}=0$时，电路处于 RAM 内容更新状态。此时，RAM 处于写操作模式，且三态驱动器输出为高阻态，而数据更新控制电路将外部新地址通过 $A_4\sim A_0$和 $Y_4\sim Y_0$送至 RAM 的地址输入端 $A_4\sim A_0$，同时将外部新数据通过 $I_7\sim I_0$和 $OT_7\sim OT_0$送至存储器的数据端口 $I/O_7\sim I/O_0$，并存入 RAM 中。

注意，RAM 的写入不需要与 *CP* 同步。只要能保证更新内容在一个 *CP* 周期内全部写入 RAM 中，就不会对动态显示造成影响。

若动态显示扫描频率为 31.25 Hz，即扫描周期为 32 ms，则扫描 32 列显示屏时每列持续

时间为 1 ms,即要求 *CP* 脉冲的周期为 1 ms。更新显示内容时,只要在 1 ms 内完成 RAM 中 32 个地址单元点阵数据的写入即可。

除了可以显示字符外,LED 点阵显示器还可以显示各种图案。只要向 RAM 中写入图案的点阵数据,LED 阵列便可显示出希望的图案。

复习思考题

7.2.1 DRAM 中存储的数据如果不进行周期性的刷新,其数据将会丢失;而 SRAM 中存储的数据无须刷新,只要电源不断电就可以永久保存,为什么?

7.2.2 一般情况下,DRAM 的集成度比 SRAM 的集成度高,为什么?

7.2.3 同步 SRAM 与异步 SRAM 的主要差别是什么?它有什么优点?

7.2.4 同步 SRAM 中从发模式读/写操作的特点是什么?

小 结

- 半导体存储器是现代数字系统特别是计算机中的重要组成部分,它可分为 ROM 和 RAM 两大类,属于 MOS 工艺制成的大规模集成电路。
- ROM 是一种非易失性的存储器,它存储的是固定数据,一般只能被读出。根据数据写入方式的不同,ROM 又可以分为固定 ROM 和可编程 ROM。可编程 ROM 又可以细分为 PROM、EPROM、E^2PROM 和 Flash 存储器等。特别是 E^2PROM 和 Flash 存储器可以进行电擦写,已兼有了 RAM 的特性。
- RAM 是一种时序逻辑电路,具有记忆功能。它存储的数据随电源断电而消失,因此是一种易失性的读写存储器。它包含有 SRAM 和 DRAM 两种类型,前者用锁存器记忆数据,后者靠 MOS 管栅极电容存储数据。因此,在不停电的情况下,SRAM 的数据可以长久保持,而 DRAM 则必须定期刷新。
- 无论是 SRAM 还是 DRAM,目前都有在时钟脉冲作用下工作的同步 RAM(SSRAM 和 SDRAM),而且同步 RAM 已成为主流存储器。在此基础上发展起来的 DDR、DDR Ⅱ 和 QDR 等 RAM 也已越来越多地应用于计算机内存、显存和通信设备中。

习 题

7.1 只读存储器

7.1.1 指出下列存储系统各具有多少个存储单元,至少需要几根地址线和数据线。

(1) 64K×1 (2) 256K×4 (3) 1M×1 (4) 128K×8

7.1.2 设存储器的起始地址为全 0,试指出下列存储系统的最高地址的十六进制地址码为多少。

(1) 2K×1 (2) 16K×4 (3) 256K×32

7.1.3 试确定用 ROM 实现下列逻辑函数时所需的容量:

(1) 实现两个 3 位二进制数相乘的乘法器。

(2) 将 8 位二进制数转换成十进制数(用 BCD 码表示)的转换电路。

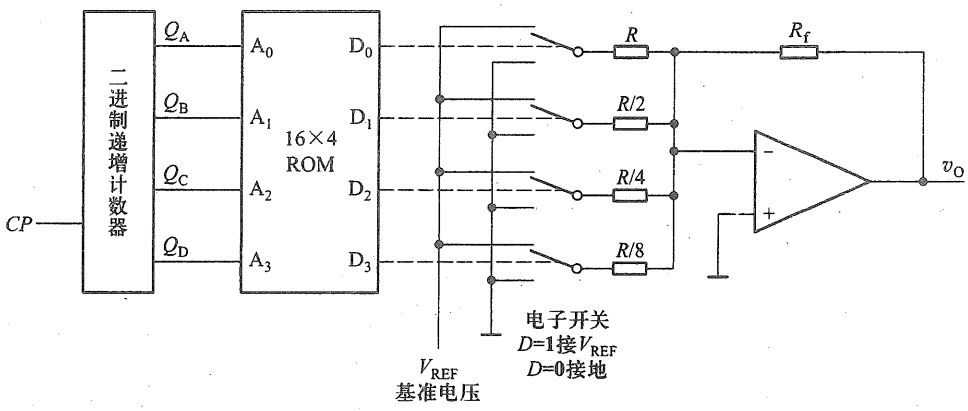
7.1.4 用一片 128×8 位的 ROM 实现各种码制之间的转换。要求用从第 0 个地址单元(全 0 地址码)开始的前 16 个地址单元中的 10 个,实现 8421BCD 码到余 3 码的转换;用从第 17 个地址单元开始接下来的 16 个单元中的 10 个,实现余 3 码到 8421BCD 码的转换。

(1) 列出 ROM 的地址与内容对应关系的真值表。

(2) 确定输入变量和输出变量与 ROM 地址线和数据线的对应关系。

(3) 简要说明将 8421BCD 码的 **0101** 转换成余 3 码和将余 3 码的 **1001** 转换成 8421BCD 码的过程。

7.1.5 利用 ROM 构成的任意波形发生器如图题 7.1.5 所示,改变 ROM 的内容,即可改变输出波形。当 ROM 的内容如表题 7.1.5 所示时,画出输出端随 *CP* 变化的波形。



图题 7.1.5

表题 7.1.5

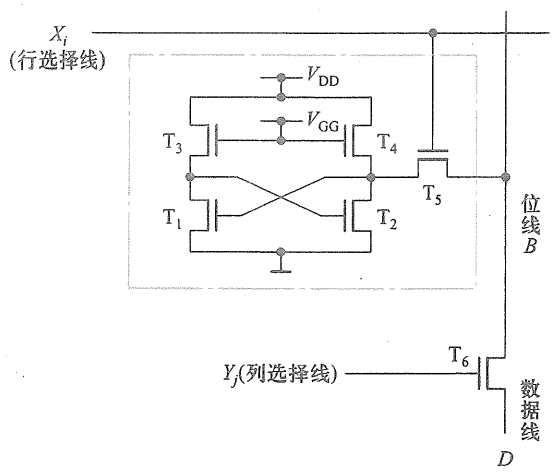
$A_3A_2A_1A_0$	$D_3D_2D_1D_0$	$A_3A_2A_1A_0$	$D_3D_2D_1D_0$
0000	0100	1000	0100
0001	0101	1001	0011
0010	0110	1010	0010
0011	0111	1011	0001
0100	1000	1100	0000
0101	0111	1101	0001
0110	0110	1110	0010
0111	0101	1111	0011

7.1.6 应用 74HC/38 和其他逻辑门设计一地址译码器,要求地址范围是十六进制 00~3F。

7.1.7 指出题 7.1.6 中对应十六进制地址码 07、0E、13、2C、3B 的输入。

7.2 随机存取存储器

7.2.1 一个 NMOS 存储单元如图题 7.2.1 所示，试分析其工作原理。



图题 7.2.1

7.2.2 设同步 SRAM 工作在丛发模式下，若数据写入的首地址为 0094H，问接下来的 3 个数据将被写入的存储单元的地址分别是多少？

7.2.3 一个有 1M×1 位的 DRAM，采用地址分时送入的方法，芯片应具有几根地址线？

7.2.4 用容量为 16K×1 位存储器芯片构成一个 32K×8 位的存储系统，总共需要多少根地址线？多少根数据线？多少个 16K×1 位的存储器芯片？

7.2.5 试用具有片选使能$\overline{CE}$、输出使能$\overline{OE}$、读/写控制$\overline{WE}$、容量为 8K×8 位的 SRAM 芯片和必要的逻辑门，设计一个 16K×16 位的存储器系统，试画出其逻辑图。

7.2.6 在图 7.2.8 的 LED 点阵列字符动态显示电路中：

(1) 若人的视觉暂留时间为 0.05 s，在满足 LED 阵列图像稳定不闪烁的情况下，试计算 *CP* 脉冲的最低工作频率。

(2) 若要在显示屏上显示“HELP”，试确定 RAM 中存放的点阵数据。

(3) 若 LED 阵列改为 16 行×128 列，则需要 RAM 的位数为多少？容量是多少？

文档 7.2：第 7 章部分习题答案

8 FPGA 和 CPLD

引言

文档 8.1：第 8 章思维导图

前面有关章节介绍了低密度(或简单)的可编程逻辑器件(PLD),如 PLA、PAL。与中、小规模逻辑集成器件相比,PLD 在集成度、功耗和系统可靠性等方面有显著的优势。但是对于规模更大、更复杂的数字系统,低密度 PLD 仍然不能满足需要。目前,采用高密度的 FPGA 和 CPLD 器件设计实现大型数字系统已成为普遍应用的方法。

由于实际的 FPGA 和 CPLD 内部电路规模庞大且非常复杂,所以本章仅从基本原理和一般性问题出发,简要介绍这两类器件的基本结构、逻辑功能编程实现原理和一般开发过程。以期读者对此有一定了解,并作为进一步学习和运用它们开发数字电路或系统的基础知识。

从 20 世纪 90 年代后期开始,FPGA 的集成度和速度得到了快速发展,特别是在集成度上与 CPLD 拉开了距离,FPGA 成了大规模 PLD 的代表。另外,FPGA 的应用领域也从通信、图像处理扩展到了人工智能、大数据分析、机器人等众多领域。因此,本章先介绍 FPGA,后介绍 CPLD。

8.1 现场可编程门阵列(FPGA)

FPGA 是一种通过编程来实现用户所需逻辑功能的大规模集成电路。根据编程机理的不同,FPGA 大致上分为两类,一类采用 SRAM 编程技术,另一类采用反熔丝技术(参见 4.5.1 节)。基于反熔丝技术的 FPGA 只能编程一次,这类 FPGA 主要用于定型产品和大批量情况。而基于 SRAM 技术的 FPGA 可以进行无限次编程,这里我们主要介绍基于 SRAM 技术的 FPGA。

下面首先介绍 FPGA 实现逻辑函数的基本原理,然后简单介绍 FPGA 的内部结构。

8.1.1 FPGA 中编程实现逻辑功能的基本原理

查找表(LUT)是 FPGA 实现逻辑函数的基本逻辑单元,由若干存储单元和数据选择器构成(参见 4.4.3 节)。2 输入 LUT 的结构如图 8.1.1 所示,用其实现 2 变量组合逻辑函数时,输入变量接选择端,并对接入数据输入端的 4 个 SRAM 存储单元 $M_0 \sim M_3$ 编程写入 **0** 或 **1** 即可。由此看出,只要编程改变 SRAM 单元 $M_0 \sim M_3$ 中的数据,就可以实现不同的逻辑函数。

该 LUT 也可以看作是一个 4×1 位的 SRAM,S_1S_0 为 2 位地址码输入。当 S_1S_0 取不同值

时,相当于输入一个地址,找出地址对应的 $M_0 \sim M_3$ 内容输出,查找表(LUT)因此得名。这一原理也与 7.1.5 节 ROM 实现组合逻辑函数的原理相同。

目前 FPGA 中大多使用 4~6 个输入、1 个输出的 LUT。当变量数超过一个 LUT 的输入数时,可以将多个 LUT 扩展连接以满足更多变量数的要求。下面以 2 变量 LUT 为例,说明在 FPGA 中如何扩展连接实现一个 3 变量的逻辑函数。

用一个小规模的 FPGA 实现逻辑函数 $F=F_1+F_2=AB+\overline{B}C$,该逻辑函数的真值表如表 8.1.1 所示。

首先用图 8.1.1 的 LUT 实现 F_1。将变量 A 和 B 分别连接 S_1 和 S_0,同时将真值表 8.1.1 中 F_1 的值分别存入 $M_0 \sim M_3$ 中,得到图 8.1.2,图(b)为符号表示,此 LUT 的输出便实现了 F_1。同理,实现 F_2 和 F。

表 8.1.1 逻辑函数真值表

$A\ B$	F_1
0 0	0
0 1	0
1 0	0
1 1	1

$B\ C$	F_2
0 0	0
0 1	1
1 0	0
1 1	0

$F_1\ F_2$	F
0 0	0
0 1	1
1 0	1
1 1	1

教学视频 8.1:FPGA 实现逻辑功能的基本原理

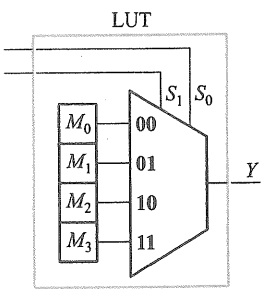


图 8.1.1 2 输入 LUT 结构图

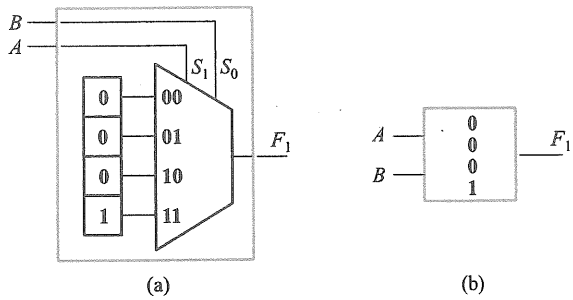


图 8.1.2 实现逻辑函数 F_1 的 LUT
(a) F_1 的 LUT 实现 (b) 符号表示

编程后 FPGA 中一部分逻辑块和连线资源的编程状态如图 8.1.3 所示。需要完成的编程任务有:① 将 I/O 引脚上的输入变量 A、B、C 通过可编程连线资源连接到相应 LUT 的输入端;② 将真值表中 F_1、F_2、F 的值写入相应 LUT 中的 SRAM 单元中;③ 将 F 通过可编程连线资源连接到 I/O 引脚上,作为逻辑函数的输出。

在 LUT 的基础上再增加触发器,便可实现时序逻辑的基本逻辑单元电路。FPGA 就是由更多输入端的 LUT 和触发器构成的这样的基本逻辑单元来实现各种复杂逻辑功能的。

由于 SRAM 具有数据易失性,即一旦断电,其原来写入的信息将消失。所以 FPGA 一般需要一个外部的 PROM(或其他非易失性存储器)保存编程数据。上电后,FPGA 首先从 PROM 中读入编程数据进行初始化,然后才开始正常工作。理论上 SRAM 中的数据可以进行无限次写入,所以,基于 SRAM 技术的 FPGA 可以进行无限次的编程。

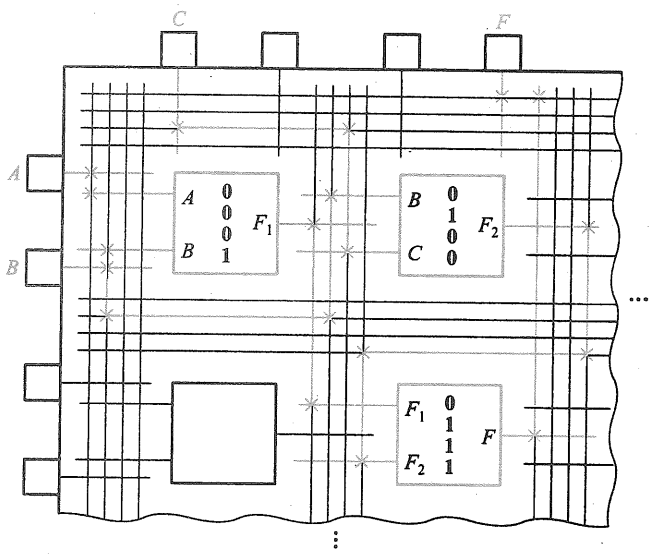


图 8.1.3 已被编程的 FPGA 的一部分

8.1.2 FPGA 的结构简介

目前,FPGA 产品种类较多,各制造商的产品也各不相同。这里仅从构成 FPGA 的基本原理出发,介绍 FPGA 的基本结构。在实际使用时,用户必须根据所选用的器件型号查阅相关的数据手册。

FPGA 的一般结构如图 8.1.4 所示。它至少包含三种最基本资源:可编程逻辑块、可编程连线资源和可编程 I/O 块。

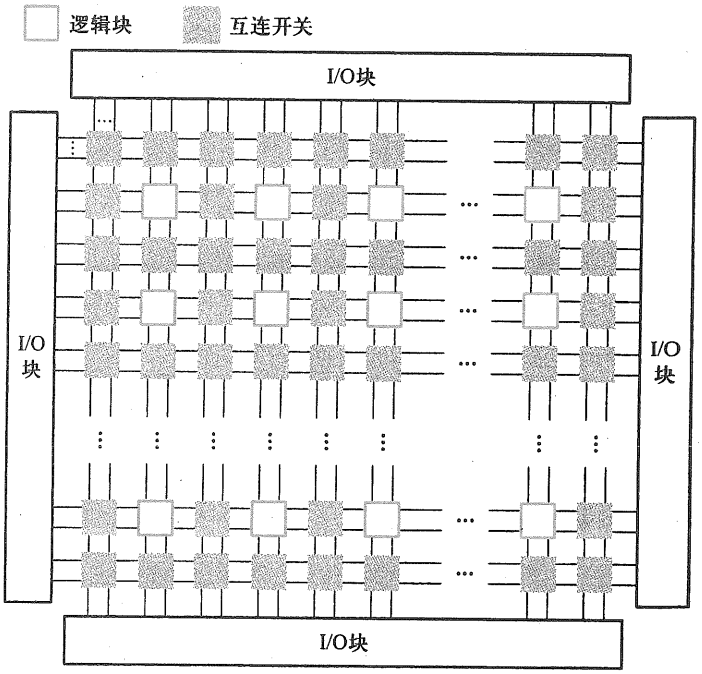


图 8.1.4 FPGA 结构示意图

FPGA 中的逻辑块排列成二维阵列。规模不同,FPGA 所含逻辑块的数量也不同。连线资源在逻辑块行和列之间纵横分布,它包括纵向和横向连线以及可编程开关(互连开关)。逻辑块用来实现所需要的逻辑功能;连线资源用来实现逻辑块与逻辑块之间、逻辑块与 I/O 块之间的连接;I/O 块是芯片外部引脚数据与内部数据进行交换的接口电路,通过编程可将 I/O 引脚设置成输入、输出和双向等不同的功能。

1. 可编程逻辑块

可编程逻辑块[①]是 FPGA 实现各种逻辑功能的基本单元。逻辑块中除了包含 LUT 外,还附加了触发器和一些可编程数据选择器和必要的逻辑门。

图 8.1.5 是 Xilinx 公司 Spartan-Ⅱ系列 FPGA 中逻辑块的主要逻辑电路原理图,其中数据选择器除了 CY、F5、F6 外均为可编程选择器(未画 SRAM 单元)。

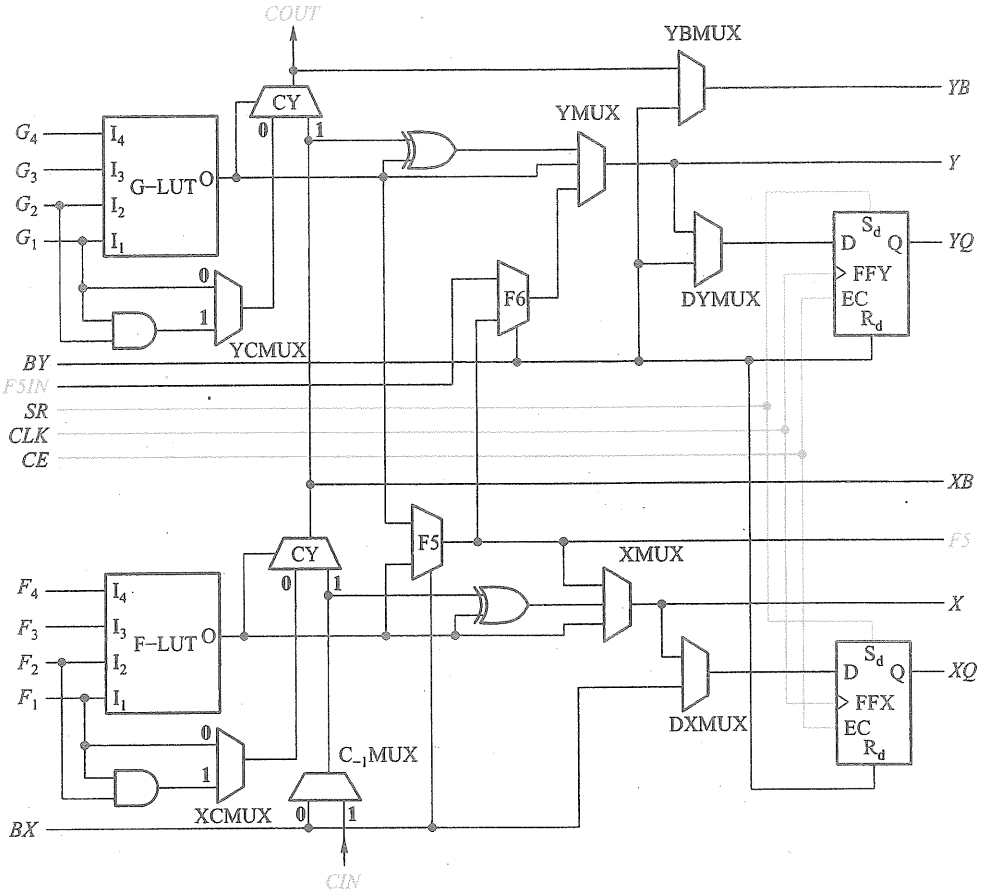


图 8.1.5 Spartan-Ⅱ系列逻辑块主要逻辑电路

① 各公司对 FPGA 中逻辑块结构有不同的称呼。例如,Xilinx 公司称其为 CLB(configurable logic block),以前的 Altera 公司称其为 LAB(logic array block)。

由图看出，逻辑块有 15 个输入端和 8 个输出端。$F_1 \sim F_4$和 $G_1 \sim G_4$为两组 4 变量逻辑函数输入端；*SR*、*CE* 和 *CLK* 为触发器的控制及时钟信号输入端；*F5IN* 为级联输入端，用于 LUT 的级联扩展；*BX*、*BY* 为旁路输入端，信号由此进入逻辑块后可以绕过 LUT 直接传至输出；*CIN* 为进位链输入端，当与其他逻辑块实现多位数加法运算时，*CIN* 可作为进位输入。8 个输出包括 2 个组合逻辑输出端 *X* 和 *Y*，2 个寄存器输出端 *XQ* 和 *YQ*，2 个算术运算进位端 *XB* 和 *YB*，1 个级联输出端 *F5* 和 1 个进位链输出端 *COUT*。

例如，用图 8.1.5 实现 2 位二进制递增计数器时，由表 8.1.2 所示的 2 位二进制递增计数器状态转换，可得 *D* 触发器的激励方程为

$$Q_1^{n+1} = D_1 = \overline{Q}_1 Q_0 + Q_1 \overline{Q}_0$$

$$Q_0^{n+1} = D_0 = \overline{Q}_0$$

表 8.1.2 2 位二进制递增计数器状态转换表

Q_1	Q_0	$Q_1^{n+1}(D_1)$	$Q_0^{n+1}(D_0)$
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

用 G-LUT 产生 D_1，仅需要用到 2 个输入变量，高位的 2 个输入接地。此时，G-LUT 相当于图 8.1.1 的 2 输入 LUT，所以仅需对其中 SRAM 单元的 $M_0 \sim M_3$编程。类似地，用 F-LUT 产生 D_0，仅用到 1 个输入变量，F-LUT 相当于 1 个输入的 LUT，所以仅需对它的 M_0和 M_1编程。图 8.1.6 就是对图 8.1.5 编程后，实现 2 位二进制递增计数器的结果。

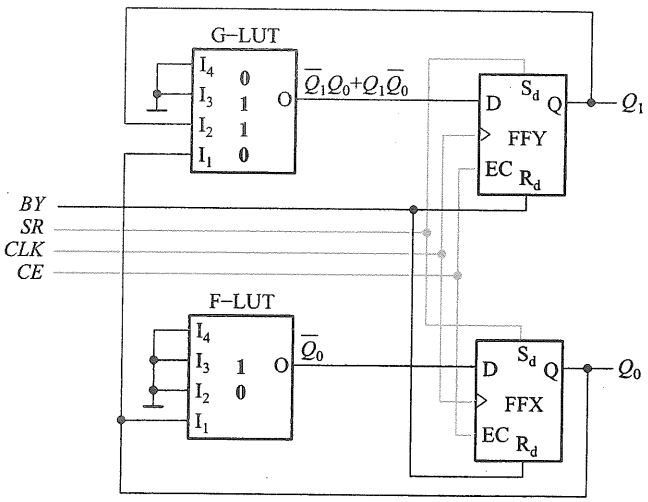


图 8.1.6 逻辑块编程后实现 2 位二进制计数器

由多个逻辑块便可构成更复杂的时序逻辑电路。

由于4输入的LUT相当于一个16×1位的SRAM,所以逻辑块也可用来做存储器使用,不过此时LUT中的内容不再是预先配置好的,而是在正常工作时可以随时读写的,而且LUT不能再作为逻辑函数产生器使用。

2. I/O块

I/O块是FPGA外部封装引脚和内部逻辑间的接口。I/O块由多个I/O单元组成,其简化原理图如图8.1.7所示。每个I/O单元对应一个封装引脚,并有许多I/O选项,可以通过配置存储单元进行选择,这些存储单元由带“M”的方框表示。通过配置,可将引脚分别定义为输入、输出和双向功能。

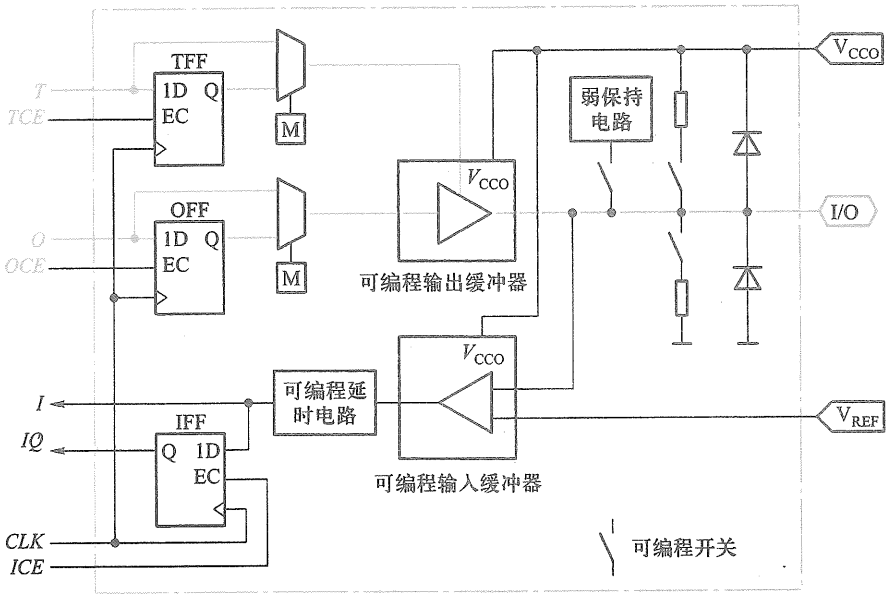


图8.1.7 简化的I/O单元原理框图

I/O单元中设计了两个电压输入端V_{CCO}和V_{REF},用来支持不同的I/O标准。V_{REF}为逻辑电平的参考电压,在执行某些I/O标准时,需要输入V_{REF}。

每一个I/O块共用一个V_{CCO}和V_{REF},但不同块的V_{CCO}和V_{REF}可以不同。只有电气标准相同的端口才能接在一起。这样FPGA内部核心区的逻辑电路在其核心电源(core power supply)下工作,位于四周的I/O块可以使用不同的工作电源,这样FPGA可以工作在由不同工作电源构成的复杂系统中。

I/O单元中有输入和输出两条信号通路。用作输出时,信号由O端进入I/O单元,经可编程数据选择器确定是直接送输出缓冲器还是经过触发器OFF寄存后再送输出缓冲器。输出缓冲器使能控制信号T可以直接控制输出缓冲器,也可以通过触发器TFF后再控制输出缓冲器。

当I/O引脚用作输入时,引脚上的输入信号经过输入缓冲器,可以直接由I进入内部逻辑电路,也可以经触发器IFF寄存后由IQ输入到内部逻辑电路中。没有用到的引脚被预置

为高阻态。可编程延时电路可以控制输入信号进入的时机，保证内部逻辑电路协调工作。其最短延迟时间为零。

3 个触发器均可编程配置为边沿触发或电平触发方式，它们共用一个时钟信号 *CLK*，但有各自的时钟使能控制信号 EC。通过它们可以实现同步输入/输出。

图 8.1.7 中两个钳位二极管具有瞬时过压保护和静电保护作用。上拉、下拉电阻和弱保持电路(weak-keeper circuit)可通过编程配置给 I/O 引脚。弱保持电路监视并跟踪 I/O 引脚输入电压的变化，当连至引脚总线上所有的驱动信号全部无效时，弱保持电路将维持在引脚最后一个状态的逻辑电平上，可以避免总线处于悬浮状态，消除总线抖动。

3. 可编程连线资源

可编程连线资源是 FPGA 不可或缺的组成部分，它包括纵向和横向连线以及可编程开关。图 8.1.4 中灰色方块表示的可编程开关(有时也称为可编程开关矩阵)有三种位置：处在和逻辑块相邻的灰色开关用来连接逻辑块和相邻逻辑块之间、逻辑块和连线之间的连接；而处在逻辑块间对角线位置上的开关则用于实现内部各连线之间的连接；与 I/O 块相邻的开关可以实现 I/O 块和逻辑块、I/O 块和内部连线之间的连接。

两种典型的互连开关结构如图 8.1.8 所示，当 NMOS 开关管栅极连接的 SRAM 单元 M 被编程为 1 时，开关管导通，否则开关管断开。图(a)通过对 M 编程，可以决定纵、横连线是否连通，而图(b)6 路互连开关通过编程，可以连通上、下、左、右任意方向，非常灵活。

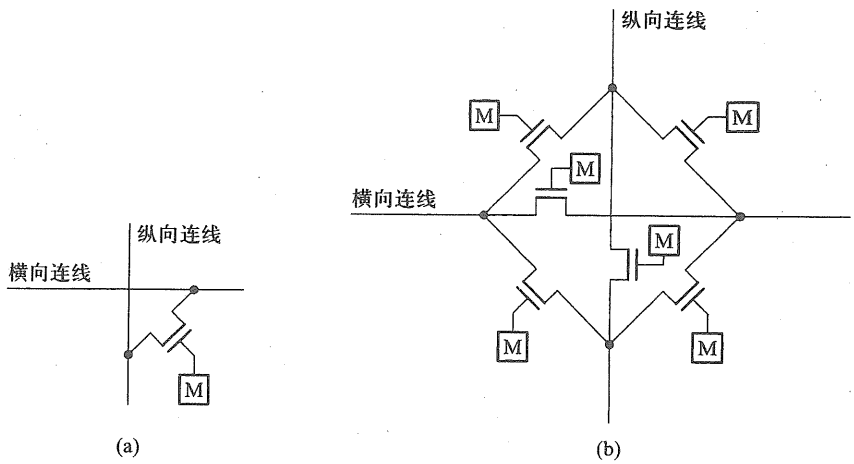


图 8.1.8 两种典型的可编程互连开关

(a) 纵横交叉互连开关 (b) 6 路互连开关

在实际的 FPGA 芯片中，除了上述通用连线资源外，通常还有一些特殊用途的连线资源，如时钟信号线和全局控制信号线等。

信号的传输延时是限制器件工作速度的根本原因。在 FPGA 的设计过程中，由软件进行优化，确定电路布局的位置和线路选择，以减小传输延迟时间，提高工作速度。

早期的 FPGA 主要通过可编程的布线连接含有查找表的逻辑块，用这种基本的结构就可以实现用户所需的逻辑电路。那时可在 FPGA 上实现的电路规模也比较小，FPGA 主要用来实现大规模系统中子模块间的接口电路、系统控制状态机等，这类简单的逻辑电路也被

称为胶连逻辑(glue logic)。

随着技术的进步,FPGA 的架构也在不断地发展。目前的 FPGA 产品已在芯片内部添加了许多重要的功能模块,包括:高速加法器和专用进位逻辑电路、由 LUT 构成的分布式小内存 RAM(distributed RAM,简称 DRAM)、块存储器(block RAM,简称 BRAM)、硬核微处理器(例如 Cortex-A9 核)、千兆的串行收发器、数字信号处理器模块、锁相环模块等。

Xilinx 和 Intel PSG[①] 是目前全球最大的两家 FPGA 供应商。近年来,国产 FPGA 也有了长足进步,深圳市紫光同创电子有限公司、上海安路信息科技有限公司、上海复旦微电子集团股份有限公司等企业都已经成功推出自主知识产权的 FPGA 产品。

复习思考题

8.1.1 LUT 实现各种组合逻辑功能的原理是什么?

8.1.2 FPGA 在结构上主要由哪几个部分组成?各部分的主要功能是什么?

8.2 复杂可编程逻辑器件(CPLD)简介

与 FPGA 相比,尽管 CPLD 在电路规模和灵活性方面不如 FPGA,但是它具有可加密性和传输延时的可预知性,使得 CPLD 仍广泛应用于数字系统设计中。

根据体系结构的差别,CPLD[②] 大致可以分为两种类型:一类是传统的 CPLD 结构,另一类是基于查找表的 CPLD 结构。

8.2.1 传统的 CPLD 结构

前面有关章节介绍的 PLA 和 PAL 都属于简单的可编程逻辑器件(SPLD[③]),它们都采用了单一的与-或阵列结构,其集成度只有数十到数百门的程度。而随着集成电路技术的发展,在 20 世纪 80 年代美国的很多创业型企业提出了各种各样的 PLD 架构,以便提高 PLD 器件的集成度,于是涌现出了 CPLD 和 FPGA 等大规模可编程器件。

早期的 CPLD 大多采用 EPROM 编程技术,其编程过程与 SPLD 一样,每次编程需要在专用或通用设备上进行。后来采用 E^2PROM 和快闪存储器技术,使 CPLD 具有了“在系统可编程(in system programmability,ISP)”特性。在系统可编程是指器件写入数据时不需要专门的编程器,计算机将编程数据直接写到置入系统的芯片里。

CPLD 的结构框图如图 8.2.1 所示,它由多个逻辑块、内部可编程的连线区和 I/O 块等组成。CPLD 内部的逻辑块[④]类似于一个 PAL 器件,它是基于与-或阵列来实现逻辑函数的。可见,CPLD 实际上是将多个简单的 PLD 集成到了单个芯片内部,可以使用可编程内部连线来连接片内各个逻辑块。之所以没有采用扩大 PAL 中单一与-或阵列规模来制造

① Altera 公司于 2015 年 6 月被 Intel 公司收购,改名为 Intel PSG(Intel programmable solution group,英特尔可编程解决方案事业部)。

② complex PLD 的简称。

③ simple PLD 的简称。

④ 各公司 CPLD 中的逻辑块名称不一。如 function block,FB(Xilinx);logic array block,LAB(Altera);generic logic block,GLB(Lattice)。

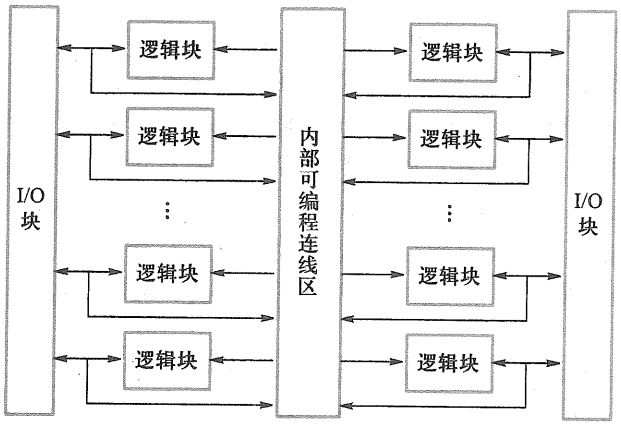


图 8.2.1 CPLD 的结构框图

CPLD，是因为一个大的与-或阵列结构会造成芯片资源的极大浪费。

1. 逻辑块

逻辑块是 CPLD 实现逻辑功能的核心模块，它能实现用乘积项之和表示的逻辑函数。逻辑块的构成如图 8.2.2 所示。

逻辑块主要由可编程乘积项阵列（即与阵列）、乘积项分配、宏单元三部分组成。对于不同厂商、不同型号的 CPLD，逻辑块中乘积项的输入变量个数 n 和宏单元个数 m 不完全相同。例如，Xilinx 公司的 XC9500 系列中，乘积项输入变量有 36 个，宏单元为 18 个。以前 Altera 公司的 MAX7000 系列有 36 个乘积项输入变量，16 个宏单元。

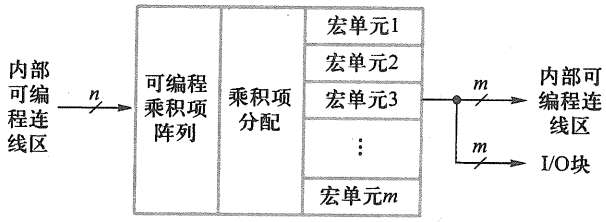


图 8.2.2 逻辑块的构成

（1）可编程乘积项阵列

此处，逻辑块中的可编程乘积项阵列有 n 个输入，可以产生 n 变量的乘积项。一般一个宏单元对应 5 个乘积项，这样，在逻辑块中共有 $5\times m$ 个乘积项。例如，XC9500 系列的逻辑块中有 90 个 36 变量乘积项，MAX7000 系列的逻辑块中有 80 个 36 变量乘积项。

（2）乘积项分配

XC9500 乘积项分配器和宏单元电路如图 8.2.3 所示，点画线框中为乘积项分配器，左侧 5 个与门的输出作为乘积项的输入，标号为 $S_1 \sim S_8$ 的矩形框是可编程的数据分配器，其中 $S_1 \sim S_5$ 为 5 个乘积项提供了不同用途的选择，可以用它们作为控制输入（时钟信号、置位、复位、输出使能），也可以通过分配器 $S_1 \sim S_5$ 最上边的一路输出送到或门 G_4 的输入端实现组

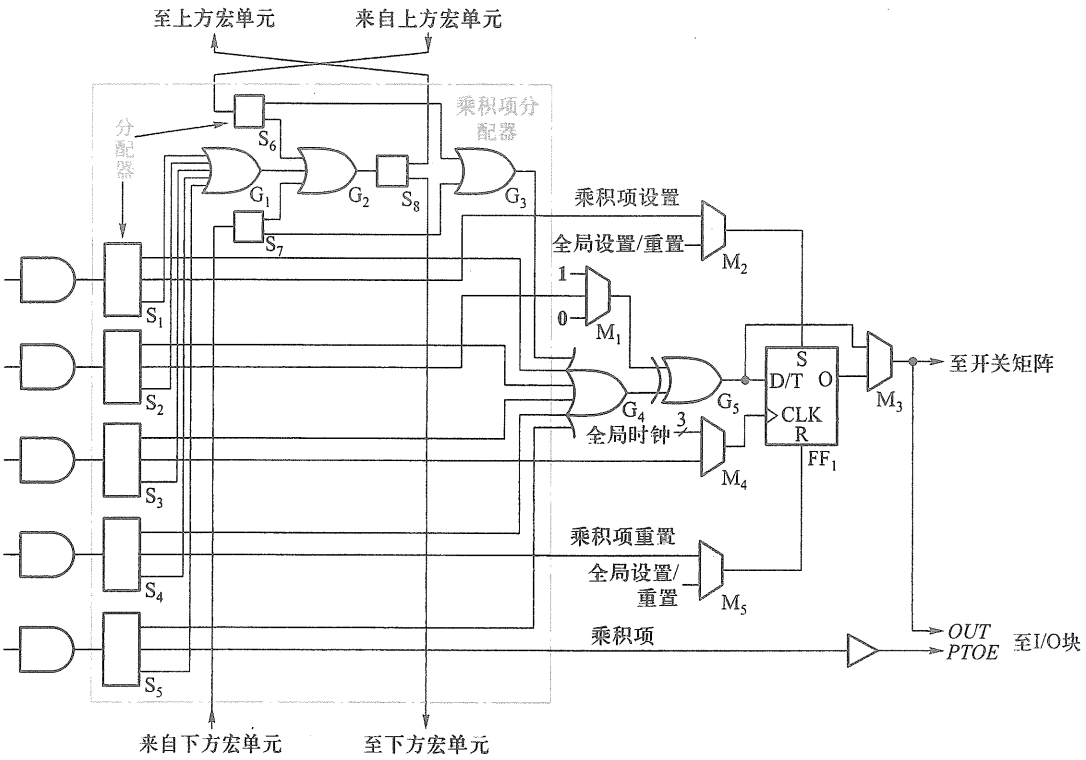


图 8.2.3 XC9500 的乘积项分配和宏单元

合逻辑函数;另外,或门 G_4 最上边的一路输入和或门 G_3 输出相连,由分配器 S_6 和 S_7 控制 G_3 的输入接受来自当前宏单元上方或者下方宏单元的乘积项。由分配器 S_8 控制将该宏单元的 5 个乘积项中的任何一个乘积项分配到上、下宏单元中去,从而增强了逻辑功能实现的灵活性。理论上可以将 XC9500 中 90 个乘积项组合到一个宏单元中,产生 90 个乘积项的与-或式,但此时其余 17 个宏单元将不能使用乘积项了,并且会增加输出信号的延迟。在以前 Altera 公司生产的 CPLD 中,还有乘积项共享电路,使得同一个乘积项可以被多个宏单元同时使用。

(3) 宏单元

此处的宏单元是指点画线框(数据分配器)右边的电路,它包含一个或门(G_4)、一个异或门(G_5)、一个触发器(FF_1)和 5 个可编程的数据选择器($M_1 \sim M_5$)等。或门用来实现逻辑表达式中的或运算。通过对宏单元编程可实现组合逻辑输出、寄存器输出、清零、置位等工作方式。宏单元的输出不仅送至 I/O 块,还送到内部可编程连线区(开关矩阵),以便被其他逻辑块使用。

2. 可编程内部连线

可编程内部连线纵横交错地分布在 CPLD 中,其作用是实现逻辑块与逻辑块之间、逻辑块与 I/O 块之间,以及全局信号到逻辑块和 I/O 块之间的连接。连线区的可编程连接也是基于 E^2CMOS 单元编程实现的。

不同制造商对可编程内部连线区的称呼也不同，Xilinx 公司的称为 switch mattrix（开关矩阵），Altera 公司的称为 PIA（programmable interconnect array，可编程互连矩阵），Lattice 公司的称为 GRP（global routing pool，全局布线区域）。当然，它们之间存在一定的差别，但所承担的任务是相同的。这些连线的编程工作是由开发软件的布线程序自动完成的。

3. I/O 块

I/O 块是 CPLD 外部封装引脚和内部逻辑间的接口。I/O 块由多个 I/O 单元组成，每个 I/O 单元对应一个封装引脚，通过对 I/O 单元中可编程单元的编程，可将引脚定义为输入、输出和双向功能。CPLD 的 I/O 单元简化原理框图如图 8.2.4 所示。

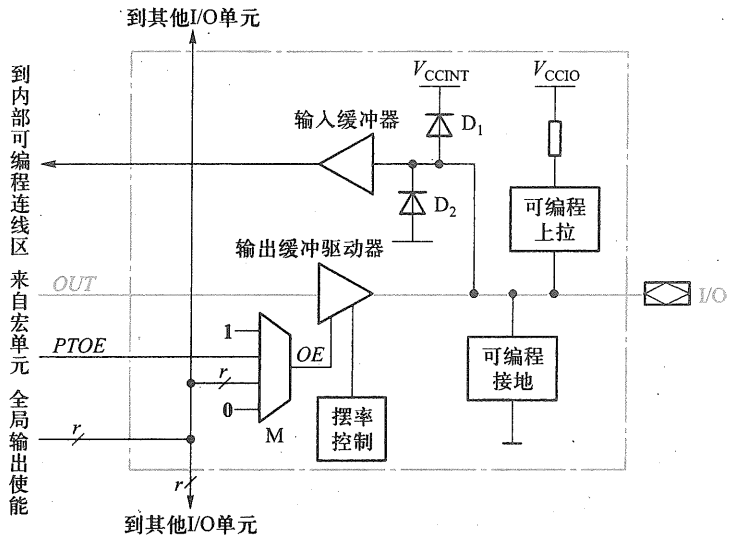


图 8.2.4 I/O 单元的简化结构图

I/O 单元中有输入和输出两条信号通路。当 I/O 引脚作输出时，三态输出缓冲器的输入信号来自宏单元，其使能控制信号 *OE* 由可编程数据选择器 M 选择其来源。其中，有多个全局输出使能控制信号，不同型号的器件，其数量也不同（XC9500 系列中 $r=4$，MAX7000 系列中 $r=6$）。当 *OE* 为低电平时，输出缓冲器为高阻，I/O 引脚可用作输入，引脚上的输入信号经过输入缓冲器送至内部可编程连线区。

图中 D_1 和 D_2 是钳位二极管，用于 I/O 引脚的保护。另外，通过编程可以使 I/O 引脚接上拉电阻或接地，也可以控制输出摆率（转换速率 *SR*）。快速方式可适应频率较高的信号输出，慢速方式则可减小功耗和降低噪声。V_{CCINT} 是器件内部逻辑电路的工作电压（也称为核心工作电压（core voltage）），而 V_{CCIO} 的引入，可以使 I/O 引脚兼容多种电源系统。

8.2.2 基于查找表的 CPLD 结构

实际上，这种查找表 CPLD 是 FPGA 和非易失性存储器相结合的产物（图 8.2.5），也可以被视为低密度的 FPGA。它基于 SRAM 工艺技术的查找表来实现逻辑函数，但 SRAM 具

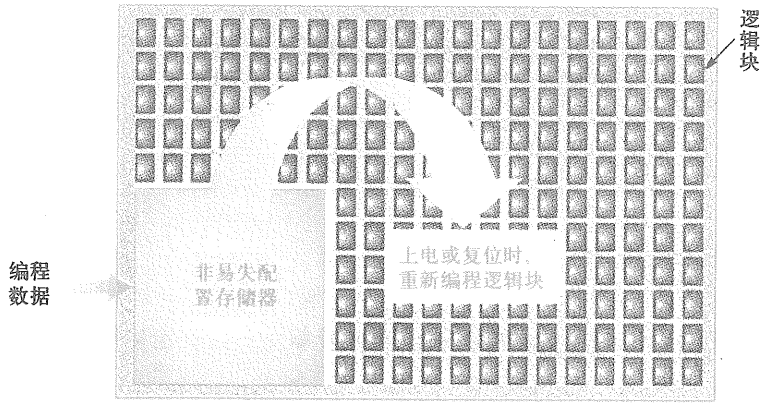


图 8.2.5 带有配置存储器的查找表 CPLD 示意图

有易失性，当电源关闭时，所有已编程的逻辑都会丢失。因此，为了存储配置信息，芯片内部嵌入了一块非易失的 Flash 存储器，加电时，保存在 Flash 内部的配置信息会自主加载到查找表相应的存储单元，从而实现其逻辑功能。

目前，采用这种结构的 CPLD 器件有 Intel PSG 提供的 MAX II 系列和 MAX10 系列，国内紫光同创的 Compact 系列，安路科技的 ELF 系列等。

复习思考题

8.2.1 传统 CPLD 在结构上可分为哪几个部分？各部分的主要功能是什么？

8.2.2 传统 CPLD 的可编程特性是基于什么编程技术？其有什么特点？

8.2.3 查找表 CPLD 在结构上可分为哪几个部分？各部分的主要功能是什么？

8.2.4 查找表 CPLD 内部的配置存储器有何作用？

8.3 可编程逻辑器件开发过程简介

通过上两节的介绍可以看出，无论是 CPLD 还是 FPGA，编程过程就是将编程数据写入这些单元的过程。这一过程也称为下载（download）或配置（configure）。编程数据有时也称为配置数据。

可编程器件的开发（编程实现）过程必须具备计算机、开发软件、通用或专用编程器（或编程电缆）三个条件。各个 FPGA 公司一般都会提供开发软件①，用来支持自己 FPGA 器件开发。其一般开发步骤如图 8.3.1 所示。除了第一步和最后一步，其他步骤必须在可编程开发软件的支持下完成。

（1）逻辑电路设计。设计者根据设计要求，运用前面章节介绍的逻辑电路设计方法设计出逻辑电路。

（2）设计输入。将设计结果以开发软件能够接受的方式输入计算机。大多数开发软件

① Xilinx 早期的开发软件为 Foundation 和 ISE，目前用得较多的是 Vivado；Altera 早期提供 MAX+plus II，现在普遍使用 Quartus Prime。

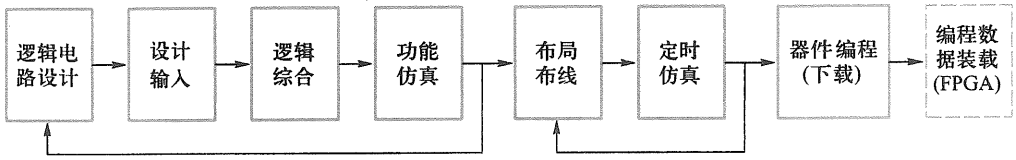


图 8.3.1 可编程器件一般开发步骤

均接受原理图描述方式和 HDL 描述方式。

(3) 逻辑综合。根据设计对象的 HDL 描述生成逻辑电路的过程称为逻辑综合。逻辑综合的结果为电路网表。网表由逻辑单元以及这些单元之间的连接组成。逻辑综合的过程包括 3 个阶段,如图 8.3.2 所示。网表生成阶段会对代码的语法错误进行检查,并生成采用逻辑表达式描述的电路网表。在门级优化阶段,会根据优化目标对网表进行逻辑化简和优化,以得到一个更优的等效电路。工艺映射阶段将决定网表中的每一个元件如何用目标芯片(如 FPGA 或者 CPLD)中的资源进行实现。

图 8.3.2 综合的 3 个阶段

(4) 功能仿真。通过开发软件提供的仿真功能,可以对所设计的电路进行逻辑功能仿真。如果发现逻辑功能不满足设计要求,则需要修改设计。

(5) 布局布线。利用芯片上的逻辑资源(如逻辑门、查找表、触发器等)和布线资源等实现电路网表的过程。开发软件根据事先设定的约束条件,如器件型号、指定的输入/输出引脚、电路工作频率等,以及之前生成的电路网表文件,在目标芯片中确定逻辑块的位置和信号的连接路径,最后生成编程数据文件①和一系列中间文件。一般来说,首先会确定逻辑块的布局,然后再对逻辑块间的连接进行布线。布局布线有时也称为适配。

(6) 定时仿真。布局布线完成后,信号的传递通路也都确定了。定时仿真是包含信号传递通路上的门、触发器、编程开关及连线等延时信息的时序仿真。若定时仿真发现不能满足设计要求时,则需更换器件或修改适配的约束条件重新进行适配;若仍不能满足要求,则需要修改最初的设计。

(7) 器件编程。也称为下载或配置,就是将编程数据写入 CPLD 或 FPGA 器件中。由于 CPLD 和 FPGA 的编程实现技术存在差异,所以这两种器件的编程步骤也略有不同。

传统 CPLD 的编程技术是基于 E^2PROM 的,且绝大部分 CPLD 器件具有 ISP 功能。编程时,先通过专用编程电缆连接计算机的数据传输端口②和被编程器件的 ISP 端口,然后通过编程软件发出编程命令,将编程数据写入芯片中,至此,CPLD 的开发过程结束,CPLD 具有了所设计的逻辑功能。

不同制造商生产的 CPLD 的 ISP 接口也不完全相同,但基本上都支持 JTAG③ 标准编

---

① 编程数据文件有多种叫法,比如配置数据、比特流(bitstream)、编程文件(program file)等,文件扩展名为 bit 或 sof (SRAM object file)等。

② 可以是并口、串口或 USB 口,视编程软件所支持的端口而定。

③ Joint Test Action Group 的缩写,指由该组织制定的 IEEE 1149.1 接口标准。

程,所以 CPLD 中都设有 JTAG 规定的 4 个 I/O 引脚。各引脚作用如表 8.3.1 所示。

图 8.3.3 是 Altera 公司的 MAX7000S 系列 CPLD 器件编程连接示意图。*TDO* 的数据可以用于编程校验,也可以作为多个 CPLD 串行编程时下一个 CPLD 器件的输入数据。多数 CPLD 的这 4 个编程引脚也可以作为用户 I/O 使用。

表 8.3.1 JTAG 标准的编程引脚

引脚	功能
TMS	编程模式控制
TCK	编程时钟
TDI	编程数据输入
TDO	编程数据输出

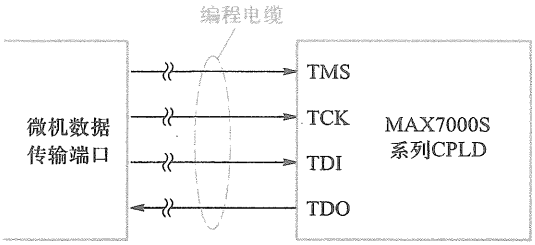


图 8.3.3 ISP 器件编程连接示意图

因为基于 SRAM 编程技术的 FPGA 断电后,其中的编程数据会消失,因此需要一个配套的 PROM 等非易失性存储器长期保存。制造商一般都会提供与 FPGA 配套使用的 PROM、EPROM 等芯片。

当然,在电路实验、电路调试或单片机系统中,也可以由计算机直接控制 FPGA 相应的编程引脚,将存储在计算机中的编程数据直接装载到 FPGA 中。

(8) 编程数据装载。只有基于 SRAM 编程技术的新型 CPLD 或 FPGA 器件才需要该步骤。它是指将第 7 步写入 PROM 中编程数据装载到 FPGA 中的过程。电路每次通电时,编程数据的装载自动完成。对于内置 PROM 的查找表 CPLD 器件,不需要设计额外的配置电路。而对于 SRAM 型 FPGA 器件,一般 PROM 是放在芯片外部的,需要设计好 PROM 和 FPGA 的连接电路。

不同的 FPGA 产品与 PROM 的连接方式和数据装载控制方式也不同,通常都设有专用于数据装载的引脚,并提供几种可选的装载模式。几乎所有的 SRAM 型 FPGA 都可以主动访问外部存储器,获取配置数据并进行配置,这种方式称为主动配置;FPGA 也可以被动地接收外部控制系统发送的数据进行配置,这种方式称为被动配置。

下面以 Xilinx 的 Spartan-Ⅱ系列 FPGA 为例,简要说明装载控制方式和电路实现。Spartan-Ⅱ系列的 FPGA 使用专用数据装载引脚,并结合一些通用 I/O 引脚完成数据装载。装载过程传递的信号包括模式控制、编程使能、数据输入、数据输出、状态指示、时钟等。

当 FPGA 的 3 个专用引脚 *M*2、*M*1 和 *M*0 接不同的逻辑电平时,便可选择一种装载模式进行数据装入。装载模式如表 8.3.2 所示。主动模式利用 FPGA 内部振荡器产生装载时钟信号 *CCLK* 来驱动装有编程数据的 PROM。而被动模式则需要外部电路提供时钟信号来驱动 *CCLK* 和装有编程数据的 PROM。当选择串行模式时,编程数据从 PROM 中以串行方式装入 FPGA 中,此时必须用有串行功能的 PROM。

表 8.3.2 Spartan-Ⅱ系列的 FPGA 装载模式控制

模式	*M2 M1 M0*	装载前相关引脚是否上拉	*CCLK* 方向	数据宽度	*DOUT*
主动串行	0 0 0 0 0 1	否 是	输出	1	串行输出
被动串行	1 1 0 1 1 1	是 否	输入	1	串行输出
被动并行	0 1 0 0 1 1	是 否	输入	8	无

主动串行装入模式电路连接举例如图 8.3.4 所示。装载使能信号 $\overline{PROGRAM}=0$ 时开始装载过程。状态信号 *DONE* 变为高电平后表示装载完成。$\overline{INIT}$ 输入低电平时初始化 FPGA。当选择并行模式时，除了时钟信号外，还需提供其他读写控制信号。

FPGA 不仅能够直接从 PROM 中读取编程数据，而且可以由其他微处理器或单片机控制装入编程数据。

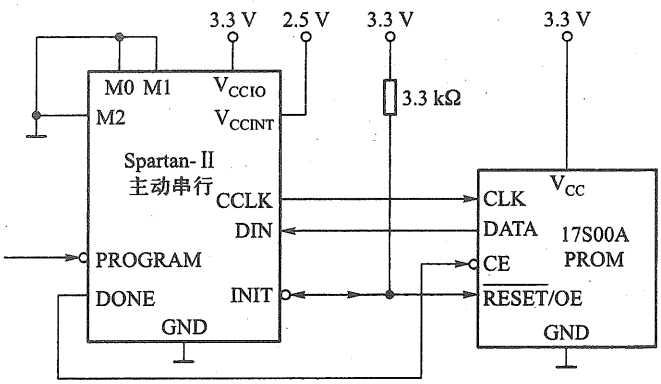


图 8.3.4 主动串行模式装载电路

近年来，FPGA 高速发展，种类型号繁多，实际使用时，需根据所用产品型号的数据手册设计装载电路。

复习思考题

8.3.1 为什么在 FPGA 构成的数字系统中通常要配备一个 PROM 或 EPROM?

8.3.2 Spartan-Ⅱ系列的 FPGA 采用被动模式装载时，是否需要外部时钟信号?

8.4 用 EDA 技术和可编程器件设计举例

例 8.4.1 设计一个定时器。要求：

(1) 定时时间为 24 s，按递减方式计时，每隔 1 s，定时器减 1。

(2) 设置两个外部控制开关(控制功能如表 8.4.1 所示)，控制定时器的直接复位、启动计时、暂停/连续计时。

（3）当定时器递减计时到零（即定时时间到）时，定时器保持零不变，同时发出报警信号。

（4）输入时钟脉冲的频率为 1 Hz，输出为 8421BCD 码。

表 8.4.1 定时器功能表

复位/启动 nRST	暂停/连续 nPAUSE	定时器完成的功能
0	×	定时器复位，置初值 24
1	1	定时器开始计时
1	0	定时器暂停计时

文档 8.2：例 8.4.1 的仿真过程

解：（1）逻辑设计

用计数器对 1 Hz 的时钟信号进行计数，其计数值即为定时时间。根据设计要求可知，电路需要输出 2 组 8421BCD 码，计数器初值为 24，按递减方式计数，减到 0 时，输出报警信号，并能控制计数器暂停/连续计数，所以需要设计一个可预置初值的带使能控制端的递减计数器。实现上述功能的 Verilog HDL 程序如图 8.4.1 所示。

```
/*****************Basketball24.v*****************/
module basketball24(
  input nRST,nPAUSE,CP,
  output reg [3:0] TimerH,TimerL,
  output Alarm
);
assign Alarm =({TimerH,TimerL} == 8'h00) &(nRST == 1'b1);//输出报警信号
always @(posedge CP,negedge nRST,negedge nPAUSE)  //计数处理
begin
    if(~nRST)
        {TimerH,TimerL} <= 8'h24;        //复位时置初值 24
    else if(~nPAUSE)
        {TimerH,TimerL} <= {TimerH,TimerL};  //暂停计时
    else if({TimerH,TimerL} == 8'h00)   //定时时间到,保持 0 不变
        begin {TimerH,TimerL} <= {TimerH,TimerL};end
    else if(TimerL==4'h0)
        begin TimerH <= TimerH - 1'b1;TimerL <= 4'h9;end
    else
        begin TimerH <= TimerH;TimerL<=TimerL-1'b1;end
    end
endmodule
```

图 8.4.1 24 s 定时器的 Verilog 代码

HDL 描述分为 3 部分:第一部分对电路的输入、输出信号进行声明。复位 nRST(低电平有效)、暂停/连续 nPAUSE 和时钟 CP 均为输入信号,TimerH(定时器的十位)、TimerL(定时器的个位)和报警 Alarm 均为输出信号。第二部分用一个连续赋值语句 **assign** 描述了电路报警信号的输出,即计数器递减到 0 时,输出报警信号(Alarm = 1)。第三部分用一个过程块 **always** 描述计数器的计数、处理操作,计数器的个位和十位均按 8421BCD 码方式递减计数。

(2) 设计实现

用可编程逻辑器件实现上述设计的过程如下:

① 在 Quartus Prime 软件中建立一个新的工程项目,输入上述 HDL 文件,对设计项目进行编译。

② 新建一个仿真波形文件,给出输入信号的激励波形(图中为方便仿真将时钟 CP 的周期指定为 2 ms),对设计项目进行时序仿真,得到如图 8.4.2 所示的波形。

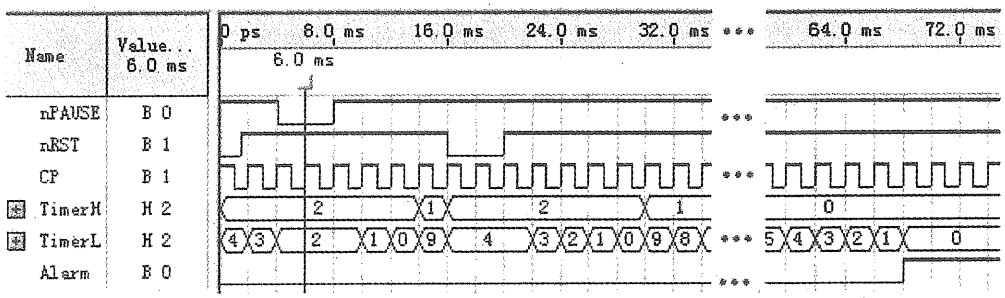


图 8.4.2 定时器的仿真波形图

分析波形图可知,开始和 16~20 ms 时,nRST 为低电平,初值 24 被预置到计数器的输出端 TimerH、TimerL;4 ~ 8 ms 时,nPAUSE 为低电平,计数器暂停计数;其他时间 nRST、nPAUSE 均为高电平,计数器在 CP 上升沿到来时递减计数,到 68 ms 之后,计数器一直输出为 0,报警信号 Alarm 为高电平。仿真结果完全符合设计要求。

③ 选定目标器件(如 EPM240),将输入、输出信号分配到器件相应的引脚上,然后重新编译设计项目,生成下载文件(文件后缀为.pof 或.sof)。

④ 将下载文件写入到目标器件中,该器件就是一块专用的定时器电路。

小 结

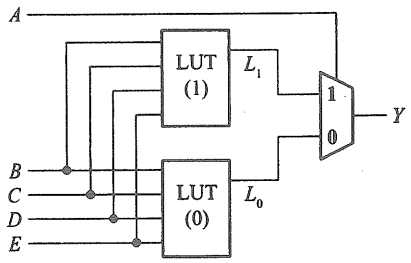
- FPGA 是目前集成度最高的大规模可编程逻辑器件,它是基于 LUT 实现逻辑函数的,且大部分 FPGA 的 LUT 由数据选择器和 SRAM 构成。尽管单个 LUT 比 CPLD 中基于与-或结构的逻辑块要小、功能要弱,但是 FPGA 包含大量的 LUT,因此可以实现规模更大、逻辑更复杂的数字电路。
- SRAM 型 FPGA 编程次数是不受限制的。但断电时,SRAM 中的数据会丢失,FPGA 原有的逻辑功能将消失,所以需要在外部配备一个非易失性存储器保存编程数据。上电后,FPGA 首先从非易失性存储器中读入编程数据进行初始化,然后才开始正常工作。

- 传统 CPLD 是在 SPLD 的基础上发展起来的复杂可编程逻辑器件，其电路结构的核心是与-或阵列和触发器，且可以在系统编程(ISP 特性)。
- 基于查找表的 CPLD，是用 LUT 实现逻辑函数，用 Flash 存储器来存储编程数据。
- 无论是 CPLD 还是 FPGA，其开发过程必须借助相关开发软件和编程设备才能完成。

习　题

8.1　现场可编程门阵列(FPGA)

8.1.1　电路如图题 8.1.1 所示，LUT 的内容如表题 8.1.1 所示。试写出 Y 的逻辑函数表达式。



图题 8.1.1

表题 8.1.1

$B\ C\ D\ E$	$L_1\ L_0$	$B\ C\ D\ E$	$L_1\ L_0$
0 0 0 0	0 1	1 0 0 0	0 1
0 0 0 1	0 0	1 0 0 1	1 0
0 0 1 0	1 0	1 0 1 0	0 0
0 0 1 1	0 0	1 0 1 1	0 1
0 1 0 0	1 0	1 1 0 0	0 1
0 1 0 1	0 0	1 1 0 1	1 0
0 1 1 0	0 1	1 1 1 0	1 0
0 1 1 1	0 1	1 1 1 1	0 1

8.1.2　用 5 个 2 输入 LUT 实现逻辑函数：$f(x_1,x_2,x_3)=x_1\overline{x}_2+x_1x_3+x_2\overline{x}_3$。用图 8.1.3 的风格标出每个 LUT 的真值表，不必画 FPGA 中的无关连线和编程节点。

8.1.3　用 2 个 2 输入 LUT 实现逻辑函数：$f(x_1,x_2,x_3)=\sum m(2,3,4,6,7)$。用图 8.1.3 的风格标出每个 LUT 的真值表，不必画 FPGA 中的无关连线和编程节点。

8.2　复杂可编程逻辑器件(CPLD)简介

8.2.1　若某 CPLD 中的逻辑块有 36 个输入(不含全局时钟、全局使能控制等)，16 个宏单

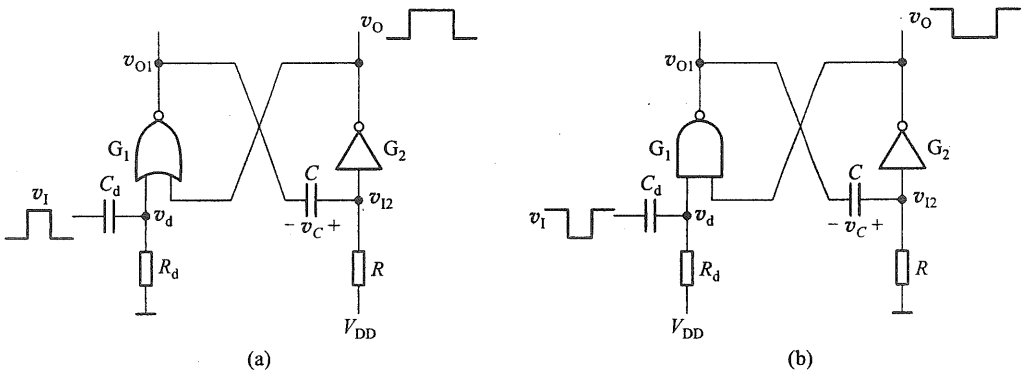


图 9.1.1 CMOS 门电路组成的微分型单稳态电路

(a) 用或非门和非门构成的微分型单稳态电路 (b) 用与非门和非门构成的微分型单稳态电路

下面仅讨论图 9.1.1(a)所示微分型单稳态电路的工作原理。

(1) 没有触发信号时,电路处于稳定状态

在正常情况下,v_I为低电平,由于门 G_2的输入端经电阻 R 接 V_{DD},即 $v_{I2}=V_{DD}$,故 $v_O=0$;这样,或非门 G_1的两输入端均为低电平,故 $v_{O1}=V_{DD}$,电容 C 两端的电压接近 0 V,电路处于稳定状态。在触发脉冲到来之前,电路一直保持这一稳态不变。

(2) 在输入端加触发信号时,电路由稳态翻转至暂稳态

输入正向触发脉冲时,在 R_d和 C_d组成的微分电路输出端得到很窄的正、负脉冲 v_d,如图 9.1.2 所示。当 v_d上升到门 G_1的阈值电压 V_{TH}时,在电路中产生如下正反馈过程:

$$v_d\uparrow \rightarrow v_{O1}\downarrow \rightarrow v_{I2}\downarrow \rightarrow v_O\uparrow$$

这一正反馈过程使 v_{O1}迅速地从高电平跳变为低电平。由于电容 C 两端的电压不能瞬间突变,v_{I2}也跳变为低电平,这又引起 v_O跳变为高电平,电路进入暂稳态,即 $v_{O1}\approx 0$,$v_O\approx V_{DD}$。此时,即使 v_d已返回到低电平,由于 v_O为高电平,或非门 G_1也会使 v_{O1}维持在低电平。但由于电容 C 的存在,电路的这种状态是不能长久保持的,所以称之为暂稳态。

(3) 暂稳态期间电容器 C 充电,电路自动从暂稳态返回至稳态。

在暂稳态期间,$v_{O1}\approx 0$,电源 V_{DD}会经电阻 R 对电容 C 充电,v_{I2}按指数规律升高,一旦 v_{I2}达到门 G_2的阈值电压 V_{TH}时,电路又产生下述正反馈过程:

$$v_{I2}\uparrow \rightarrow v_O\downarrow \rightarrow v_{O1}\uparrow$$

假如此时触发脉冲已消失(即 v_d已回到低电平),上述正反馈过程使 v_{O1}、v_{I2}迅速跳变到高电平,输出返回到 $v_O\approx 0$ V 的状态。此后,电容通过电阻 R 和门 G_2的输入

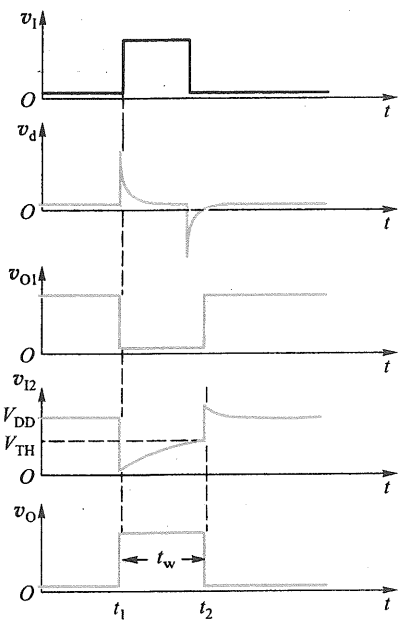


图 9.1.2 微分型单稳态电路中各点的电压波形图

保护电路放电，使电容 C 上的电压最终恢复到稳定状态时的初始值，电路从暂稳态自动返回到稳态。

2. 主要参数的计算

（1）输出脉冲宽度 t_w

输出脉冲宽度 t_w 就是暂稳态维持时间。它是 RC 电路在充电过程中，使 v_{I2}（即电容电压 v_C）从 0 V 上升到 V_{TH} 所需时间。以触发脉冲作用的 t_1 时刻为起点，由图 9.1.2 所示 v_{I2} 的波形可知，$v_C(0)=0$；$v_C(\infty)=V_{DD}$，$\tau=RC$，$v_C(t_w)=V_{TH}=V_{DD}/2$。将这些值代入到 RC 电路过渡过程的计算公式（也称三要素法公式）

$$v_C(t)=v_C(\infty)+[v_C(0)-v_C(\infty)]e^{-t/\tau} \tag{9.1.1}$$

求得输出脉冲宽度

$$t_w=RC\ln\frac{v_C(\infty)-v_C(0)}{v_C(\infty)-v_C(t_w)}=RC\ln 2 \tag{9.1.2}$$

可取近似值

$$t_w\approx 0.7RC \tag{9.1.3}$$

可见，t_w 仅由电路本身的 R、C 参数值决定，与触发脉冲的宽度和幅度无关。

（2）恢复时间 t_{re}

暂稳态结束后，要使电路完全恢复到稳定状态，还需要经过一段恢复时间，以便电容 C 能够释放掉暂稳态期间所充的电荷。一般认为，恢复时间要经过放电时间常数的 3～5 倍，RC 电路才基本达到稳态。

（3）最高工作频率 f_{max}

设触发信号 v_I 的周期为 T，为了使单稳态电路能正常工作，应满足 $T>(t_w+t_{re})$ 的条件，因此，单稳态电路的最高工作频率为

$$f_{max}=\frac{1}{T_{min}}<\frac{1}{t_w+t_{re}} \tag{9.1.4}$$

9.1.2 集成单稳态电路

用逻辑门组成的单稳态电路虽然结构简单，但它存在触发方式单一、输出脉宽调节范围小、稳定性差等缺点。为了提高单稳态电路的性能，可以采用表 9.1.1 所示的单片集成电路。

根据电路工作特性不同，集成单稳态电路分为不可重复触发和可重复触发两种。不可重复触发单稳态电路的逻辑符号和工作波形如图 9.1.3（a）所示，方框中的限定符号“1 ⎍”表示不可重复触发。当触发信号使电路进入暂稳态后，电路不再接受新的触发信号，原有的暂稳态过程会持续下去，直到结束为止。即不可重复触发电路只能在稳态接受触发信号。

可重复触发单稳态电路的逻辑符号和工作波形如图 9.1.3（b）所示，方框中的限定符号“⎍”表示可重复触发。在暂稳态期间，电路能够接受新的触发信号，重新开始暂稳态过程。即电路的暂稳态将从最后一个触发沿到达时刻开始，再延时 t_w 时间后返回到稳态，这样暂稳态的时间就被延长了。

表 9.1.1 部分集成单稳态电路①

型号	说 明	型号	说 明
74121	不可重复触发单稳态	CD4098D	双路带清零端的可重复触发单稳态
74LS122	带清零端的可重复触发单稳态	74HC4538	双路带清零端的高精度可重触发单稳态
74HC123 74HC423	双路带清零端的可重复触发单稳态	CD14538B	双路带清零端的可重复触发单稳态
74HC221	双路带清零端的不可重复触发单稳态	CD4047B	可重复触发单稳态/多谐振荡电路(可选择工作模式)

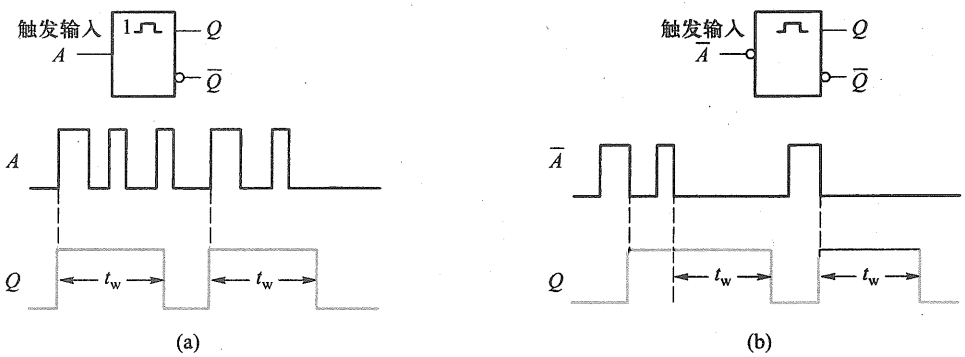


图 9.1.3 两种集成单稳态电路的逻辑符号和工作波形

(a) 上升沿触发的不可重复触发单稳态 (b) 下降沿触发的可重复触发单稳态

下面以 74121 为例,介绍不可重复触发的集成单稳态电路的使用方法。

74121 的组成如图 9.1.4 所示②(点画线框外部的数字为芯片的引脚号)。它的功能如表 9.1.2 所示。功能表的前 4 行说明了电路的稳定状态,Q 为低电平,$\overline{Q}$为高电平;功能表的后 5 行给出了电路进入暂态的条件。

在下述两种情况下,在 Z 点会产生正向触发脉冲,74121 由稳态翻转到暂稳态:

① 若$\overline{A}_1$、$\overline{A}_2$两个输入中有一个或两个为低电平,输入端 B 出现由 **0** 到 **1** 的正向跳变时;

② 若 B 为高电平,输入端$\overline{A}_1$、$\overline{A}_2$中有一个或两个出现由 **1** 到 **0** 的负向跳变时(不产生跳变的输入端必须保持高电平)。

74121 输出脉冲的宽度主要取决于所使用的电容 C_{ext}和电阻 R_{ext}(或 R_{int})的实际值。它的输出脉冲宽度为

$$t_w \approx 0.7 R_{ext} C_{ext} \tag{9.1.5}$$

① 表格中的型号可能有不同工艺生产的产品,例如,74LS123、54LS123 为 TTL 工艺,而 74LV123A、74AHC123A、54HC123 等则为 CMOS 工艺产品。

② 图中门 G_2为具有施密特滞后电压传输特性的与门,将在下一节介绍。

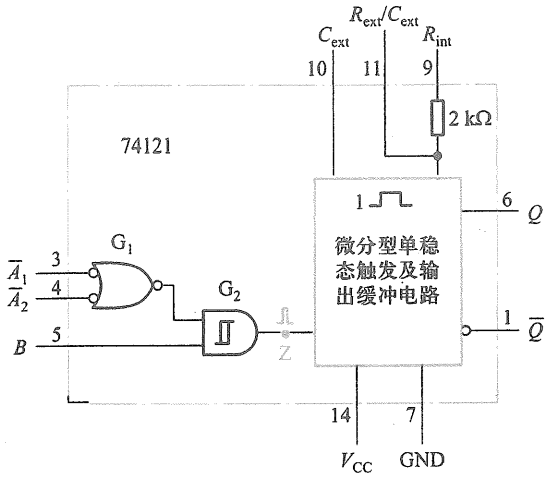


图 9.1.4 单稳态电路 74121 组成框图

定时电容 C_{ext}连接在芯片的 10、11 引脚之间，如果采用电解电容时，其正极必须接在 R_{ext}/C_{ext}(11 脚)上，允许外接电解电容的最大值为 1000 μF。对于定时电阻可选择外接电阻 R_{ext}或芯片内部电阻 R_{int}(2 kΩ)。通常 R_{ext}的取值范围为 2~40 kΩ。

表 9.1.2 74121 功能表

输入			输出	
$\overline{A}_1$	$\overline{A}_2$	B	Q	$\overline{Q}$
L	×	H	L	H
×	L	H	L	H
×	×	L	L	H
H	H	×	L	H
H	↓	H	⎍	⎎
↓	H	H	⎍	⎎
↓	↓	H	⎍	⎎
L	×	↑	⎍	⎎
×	L	↑	⎍	⎎

74121 采用外接电阻 R_{ext}或者内部电阻 R_{int}的电路连接分别如图 9.1.5(a)、(b)所示。74121 的工作波形如图 9.1.6 所示。

9.2 施密特触发电路

在模拟电路中，曾经讨论过由集成运放构成的施密特触发电路（迟滞比较器），这里我们将介绍数字技术中常用的施密特触发电路。施密特触发电路（Schmitt trigger）[①]常用于波形变换、幅度鉴别等。它的工作特点如下：

（1）电路属于电平触发。当输入信号达到某一电压值时，输出电平会发生跳变。但输入信号在增大、减小的过程中，引起输出状态跳变所对应的输入电压值是不相同的。

（2）电路内部采用正反馈来加速电平的转换，可以将边沿变化缓慢的信号变换成边沿陡直的矩形脉冲。

9.2.1 逻辑门组成的施密特触发电路

1. 电路组成

由逻辑门构成的施密特触发电路如图 9.2.1 所示。两个 CMOS 反相器串接，通过电阻 R_1、R_2将输出端的电压反馈到门 G_1的输入端，从而对电路产生影响。

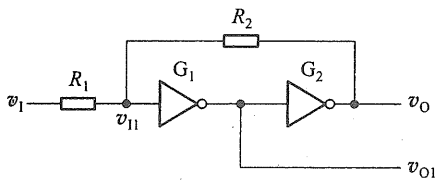


图 9.2.1 CMOS 反相器组成的施密特触发电路

2. 工作原理

设 CMOS 反相器的阈值电压 $V_{TH}\approx\frac{V_{DD}}{2}$，且电阻 $R_1<R_2$。根据图 9.2.1，门 G_1的输入电平 v_{I1}决定着电路的输出状态。根据叠加原理有

$$v_{I1}=\frac{R_2}{R_1+R_2}\cdot v_I+\frac{R_1}{R_1+R_2}\cdot v_O \tag{9.2.1}$$

设输入信号 v_I为三角波。

当 $v_I=0$ V 时，输出 $v_O=V_{OL}\approx 0$ V，此时，$v_{I1}\approx 0$ V。

当 v_I从 0 V 开始逐渐增加时，v_{I1}也会上升，但只要 $v_{I1}<V_{TH}$，电路就会保持 $v_O\approx 0$ V 不变。因为 R_2引入正反馈，当 v_I增加到使 v_{I1}略大于 V_{TH}时，门 G_1就会进入到它的电压传输特性转折区（放大区），并在电路中产生如下正反馈过程：

$$v_I\uparrow\rightarrow v_{I1}\uparrow\rightarrow v_{O1}\downarrow\rightarrow v_O\uparrow$$

① 也称为施密特触发器。但“触发器”一词已在双稳态存储单元电路中使用，而施密特触发电路与双稳态触发器的特性不同，为了避免概念混淆，本书将 Schmitt trigger 翻译为施密特触发电路。

这样，电路状态就会迅速地从低电平跳变为高电平，即 $v_O \approx V_{DD}$。

我们把输入信号在上升过程中，使电路的输出发生状态跳变时所对应的输入电压称为正向阈值电压，用 V_{T+} 表示。在电路状态发生跳变之前，$v_O \approx 0$ V，根据式(9.2.1)得

$$v_{I1} = V_{TH} = \frac{R_2}{R_1+R_2}V_{T+} \tag{9.2.2}$$

$$V_{T+} = \left(1+\frac{R_1}{R_2}\right)V_{TH} \tag{9.2.3}$$

在 $v_{I1} = V_{TH}$ 之后，如果 v_I 继续上升，则 $v_{I1} > V_{TH}$，输出维持 $v_O \approx V_{DD}$ 不变。

当 v_I 经过三角波最高点后开始逐渐减小时，v_{I1} 也会随着下降。当 v_I 减小到使 v_{I1} 略小于 V_{TH} 时，门 G_1 再次进入其电压传输特性转折区，电路产生如下的正反馈过程：

$$v_I \downarrow \rightarrow v_{I1} \downarrow \rightarrow v_{O1} \uparrow \rightarrow v_O \downarrow$$

这样，使电路的输出迅速地从高电平跳变为低电平，即 $v_O \approx 0$ V。

我们将输入信号在下降过程中，使电路的输出发生跳变时所对应的输入电平称为负向阈值电压，用 V_{T-} 表示。在电路状态发生跳变之前，$v_O \approx V_{DD}$，根据式(9.2.1)有

$$v_{I1} \approx V_{TH} = \frac{R_2}{R_1+R_2}V_{T-} + \frac{R_1}{R_1+R_2}V_{DD}$$

将 $V_{DD} = 2V_{TH}$ 代入可得

$$V_{T-} \approx \left(1-\frac{R_1}{R_2}\right)V_{TH} \tag{9.2.4}$$

定义正向阈值电压 V_{T+} 与负向阈值电压 V_{T-} 之差为回差电压，并记作 ΔV_T。由式(9.2.3)和式(9.2.4)可求得

$$\Delta V_T = V_{T+} - V_{T-} \approx 2\frac{R_1}{R_2}V_{TH} = \frac{R_1}{R_2}V_{DD} \tag{9.2.5}$$

式(9.2.5)表明，电路的回差电压与 R_1/R_2 成正比，改变 R_1、R_2 的比值即可调节回差电压的大小。

3. 工作波形及电压传输特性

根据以上分析，可画出电路的工作波形如图 9.2.2 所示。

以 v_O 作为电路的输出端时，其电压传输特性和逻辑符号如图 9.2.3(a)所示。根据输入、输出电平之间的对应关系，可以称之为同相输出施密特触发电路。而以 v_{O1} 作为输出端时，其电压传输特性如图 9.2.3(b)所示，称之为反相输出施密特触发电路。

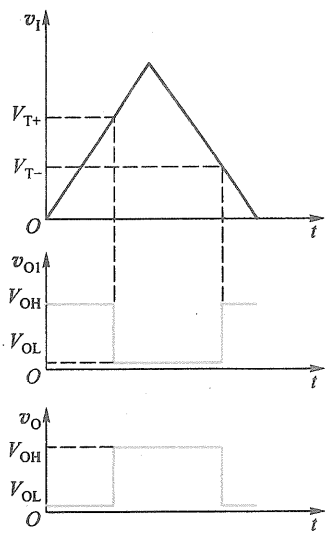


图 9.2.2 施密特触发电路的工作波形

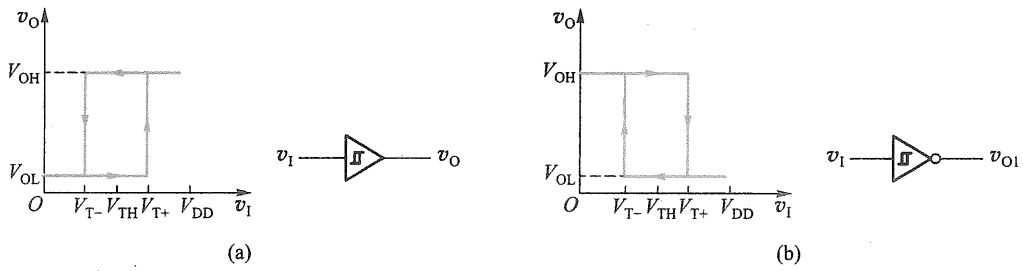


图 9.2.3 施密特触发电路的传输特性

(a) 同相输出施密特电路的传输特性及逻辑符号 (b) 反相输出施密特电路的传输特性及逻辑符号

9.2.2 集成施密特触发电路

集成施密特触发电路性能稳定、使用方便,其正向阈值电压 V_{T+} 与负向阈值电压 V_{T-} 比较稳定,有很强的抗干扰能力,应用较为广泛。

TTL 和 CMOS 集成电路中都有施密特触发电路,表 9.2.1 列出了一些常用的集成施密特触发电路。由于其内部电路较为复杂,这里就不做进一步地介绍了。

表 9.2.1 常用的集成施密特触发电路

型号	说 明	型号	说 明
74HCS00	四路 2 输入施密特触发与非门	74HCS14	六路施密特触发反相器
74HCS02	四路 2 输入施密特触发或非门	74HCS32	四路 2 输入施密特触发或门
74HCS04	六路施密特触发反相器	74HC132	四路 2 输入施密特触发与非门
74HCS08	四路 2 输入施密特触发与门	4093B	四路 2 输入施密特触发与非门
74HCS10	三路 3 输入施密特触发与非门	40106B	六路施密特触发反相器
74HCS11	三路 3 输入施密特触发与门	74AUP3G17	三路施密特触发缓冲器

9.2.3 施密特触发电路的应用

1. 波形变换

施密特触发电路常用于波形变换,如将正弦波、三角波等变换成矩形波等。将幅值大于 V_{T+} 的正弦波送到施密特触发电路的输入端,根据施密特触发电路的电压传输特性,可画出输出电压波形,如图 9.2.4 所示。结果表明,利用施密特触发电路在状态变化过程中的正反馈作用,可将边沿变化缓慢的周期性信号变换成与其同频率、边缘陡直的矩形波。

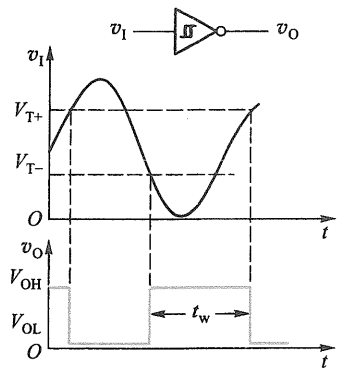


图 9.2.4 用施密特触发电路实现波形变换

2. 波形的整形与抗干扰

在工程实际中,常遇到信号在传输过程中发生

畸变的现象。例如，当传输线上电容较大，矩形波在传输过程中，其上升沿和下降沿都会明显地被延缓，其波形如图 9.2.5(a)所示。又如传输线较长，且接收端的阻抗与传输线的阻抗不匹配，则在波形的上升沿和下降沿将产生阻尼振荡，如图 9.2.5(b)所示中的输入信号。

对上述信号波形在传输过程中产生的畸变，均可采用施密特触发电路整形。图 9.2.5(a)、(b)所示施密特触发电路工作波形说明，只要回差电压合适，就可达到理想的整形效果。

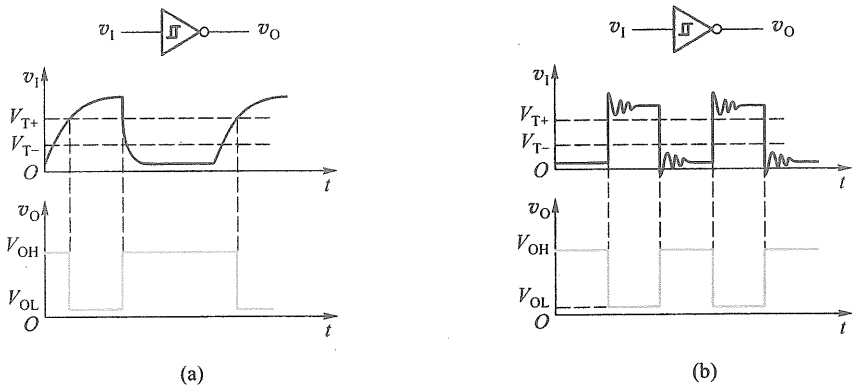


图 9.2.5 用施密特触发电路实现脉冲波形的整形

(a) 改善上升沿和下降沿 (b) 消除振荡影响

3. 幅度鉴别

施密特触发电路属电平触发方式，即其输出状态与输入信号 v_I 的幅值有关。利用这一工作特点，可将它作为幅度鉴别电路。例如，在施密特触发电路输入端输入一串幅度不等的脉冲信号，只有幅度大于 V_{T+} 的那些脉冲才会使施密特触发电路翻转，v_O 有脉冲输出；而对于幅度小于 V_{T+} 的脉冲，施密特触发电路不翻转，v_O 没有脉冲输出。因此，可以选出幅度大于 V_{T+} 的脉冲，电路的输入、输出波形如图 9.2.6 所示。

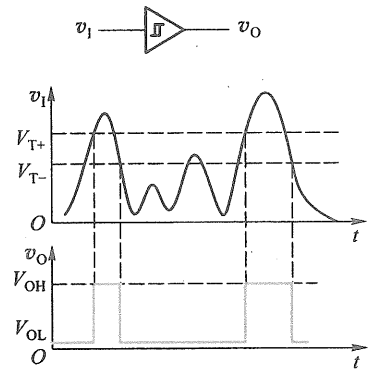


图 9.2.6 用施密特触发电路进行幅度鉴别

复习思考题

9.2.1 施密特触发电路具有怎样的工作特点？试画出其电压传输特性曲线。

9.2.2 如果改变图 9.2.1 中 R_1 的取值，对电路的传输特性有何影响？

于是，振荡周期的表达式为

$$T=T_1+T_2=RC\ln\frac{(V_{DD}+V_{TH})(2V_{DD}-V_{TH})}{V_{TH}(V_{DD}-V_{TH})} \tag{9.3.3}$$

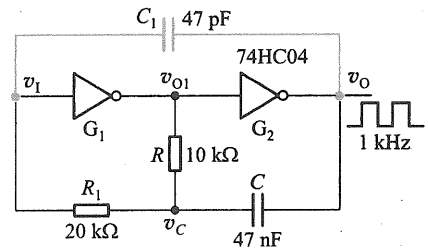
当 $V_{TH}=\frac{V_{DD}}{2}$ 时，得到

$$T=RC\ln 9\approx 2.2RC \tag{9.3.4}$$

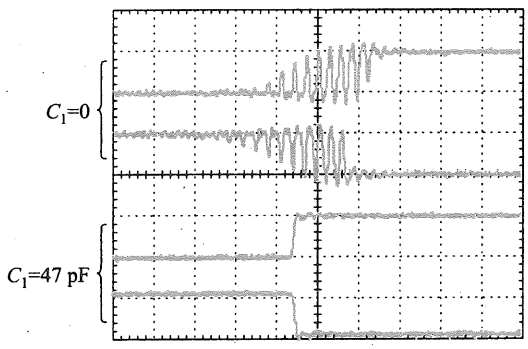
由于相同型号 CMOS 门的 V_{TH} 有差异，这会影响周期 T，因此该电路仅用在对频率准确度和稳定度要求不高的场合。如果去掉电阻 R_1，将 v_C 直接加到门 G_1 输入端，则在 v_O 发生跳变时会使保护二极管导通，并限制 v_C 的跳变幅度，从而改变 C 的充、放电时长，显著地影响输出信号的周期（见习题 9.3.1）。

如果在图 9.3.1 中使用低功耗肖特基 TTL 反相器，也可以输出脉冲波形，但为了使电路容易起振，要求电阻 R_1 为 0，电阻 R 的取值不宜过大，典型值为 500 Ω~1 kΩ，电容 C 可根据振荡频率的要求选择。

注意，在图 9.3.1 所示电路中，由于 R_1 和门 G_1 输入电容的影响，导致门 G_1 输入信号的上升时间相对较长，在每次状态转换期间会受到高频振荡（大约 100 MHz）的困扰。改进方案是增加一个小电容 C_1（47 pF）来加速门 G_1 翻转，一个输出信号频率为 1 kHz 的实用电路如图 9.3.3 所示。



(a)



(b)

图 9.3.3 实用的 1 kHz 多谐振荡电路及其波形

（a）多谐振荡电路 （b）示波器实测波形（增加 C_1 寄生振荡消失）

9.3.2 施密特触发电路组成的多谐振荡电路

由于施密特触发电路有 V_{T+} 和 V_{T-} 两个不同的阈值电压，如果能使它的输入电压在 V_{T+} 和 V_{T-} 之间反复变化，就可以在输出端得到矩形波。将施密特触发电路的输出端经 RC 积分电路接回其输入端，利用 RC 电路充、放电过程改变输入电压，即可用施密特触发电路构成多谐振荡电路，如图 9.3.4(a)所示。

假设在电源接通瞬间，电容器 C 的初始电压为零，则输出电压 v_O 为高电平。v_O 通过电阻 R 对电容器 C 充电，v_C 会逐渐上升，当 v_C 达到 V_{T+} 时，施密特触发电路翻转，v_O 由高电平跳变为低电平。此后，电容器 C 又开始放电，v_C 逐渐下降，当 v_C 下降到 V_{T-} 时，电路又发生翻转，v_O 又由低电平跳变为高电平，C 又被重新充电。如此周而复始，在电路的输出端，就得到了矩形波。其工作波形如图 9.3.4(b)所示。

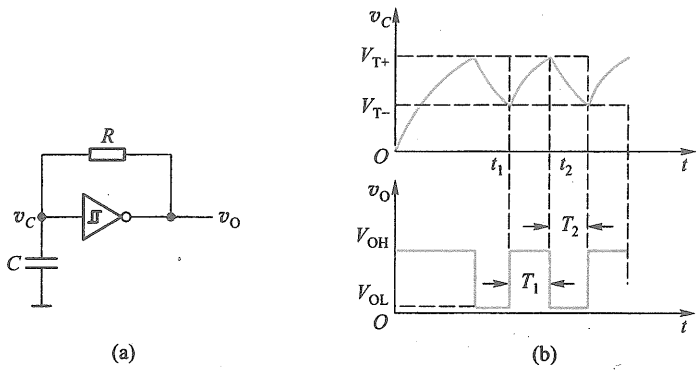


图 9.3.4 用施密特触发电路构成的多谐振荡电路及电压波形
(a) 电路图 (b) 电压波形图

设在图 9.3.4(a)中采用 CMOS 施密特触发电路 CD40106，已知 $V_{OH} \approx V_{DD}$、$V_{OL} \approx 0$ V，则根据式(9.1.1)，可求出图 9.3.4(b)中输出电压 v_O 的周期为

$$T = T_1 + T_2 = RC\left(\ln\frac{V_{DD}-V_{T-}}{V_{DD}-V_{T+}} + \ln\frac{V_{T+}}{V_{T-}}\right)$$

$$= RC\ln\left(\frac{V_{DD}-V_{T-}}{V_{DD}-V_{T+}} \cdot \frac{V_{T+}}{V_{T-}}\right) \tag{9.3.5}$$

注意，使用带有施密特特性的 CMOS 反相器组成振荡电路时，虽然振荡波形较好，但由于 V_{T+}、V_{T-} 各自存在一定的变化范围，例如，CD40106B 在 5 V 供电时，其正向阈值电压介于 2.2~3.6 V 之间，负向阈值电压介于 0.9~2.8 V 之间。这意味着具有相同 R 值和 C 值的不同振荡电路，其输出信号的频率会有差异。

9.3.3 石英晶体多谐振荡电路

前面介绍的多谐振荡电路，振荡频率不够稳定。从式(9.3.1)和式(9.3.5)可知，电路的振荡频率不仅与时间常数 RC 有关，而且还与门电路的阈值电压 V_{TH} 有关。当电源电压波动，温度变化(引起门电路的阈值电压 V_{TH} 变化)时，电路的振荡频率就会发生变化。在现代

数字系统中，普遍采用石英晶体（其化学成分是 SiO_2）振荡电路来获得频率稳定的信号，石英晶体典型的频率范围是 10 kHz~10 MHz。

石英晶体的电路符号、电路模型和阻抗频率特性如图 9.3.5 所示。C_0代表石英晶体不振动时支架静电容量，一般为几到几十皮法；L、C 和 R 代表晶体本身的特性；L 相当于晶体质量（机械振动惯性），其值很大，一般为几十到几百毫亨；C 相当于晶体的等效弹性模数，其值仅为 10^{-4}~10^{-1} pF；R 表示晶片振动时的摩擦损耗，其值为 100 Ω 左右。由于模型中 L/C的比值很大，因而品质因数 Q 高达 10 000~500 000。又由于石英晶体的固有频率仅与石英晶体的结晶方向和外尺寸有关，与电路中的电阻、电容无关。所以，石英晶体振荡电路的频率稳定度极高，其频率稳定度 $\Delta f_S/f_S$可达 10^{-10}~10^{-11}。

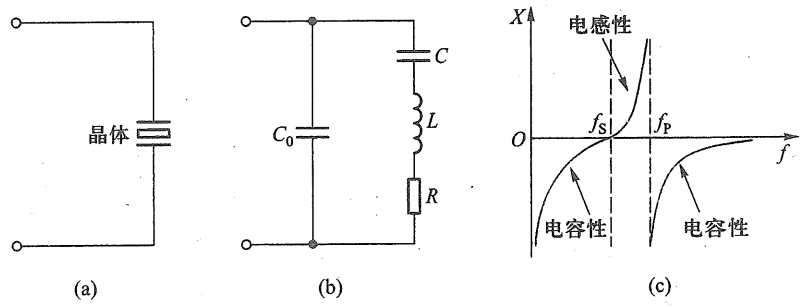


图 9.3.5 石英晶体的电路符号及阻抗频率特性

（a）电路符号 （b）电路模型 （c）阻抗频率特性

由电路模型可知，石英晶体有两个谐振频率。当 R、L、C 支路发生串联谐振时，该支路呈纯阻性，大小为 R，此时的谐振频率为 f_S；当频率高于 f_S时，R、L、C 支路呈感性，可与 C_0发生并联谐振，此时的谐振频率为 f_P。由于 $C \ll C_0$，因此 f_S与 f_P很接近（相差几十~几百 Hz）。

利用石英晶体构成的振荡电路通常有两类：一类是把谐振频率选择在 f_S处，石英晶体构成串联谐振型振荡电路；另一类是把振荡频率选在 f_S与 f_P之间，石英晶体等效电感元件，构成并联谐振型振荡电路。

1. 串联谐振型石英晶体振荡电路

串联谐振型石英晶体振荡电路如图 9.3.6 所示。石英晶体串接在由门 G_1、G_2组成的正反馈电路中。当振荡频率等于晶体的串联谐振频率 f_S时，晶体阻抗最小，呈纯电阻特性，此时正反馈最强，且满足相位条件；而其他频率信号都被石英晶体衰减（由于电抗 X 较大），所以电路的振荡频率就是 f_S。

图中，电阻 R_1和 R_2的作用是使反相器 G_1、G_2在静态（电路没有振荡）时工作在电压传输特性曲线的转折区（放大区），使每个反相器成为具有很强放大能力的放大电路，有利于电路起振。如采用 TTL 门电路，R_1和 R_2通常取值为 0.5~2 kΩ 之间；如采用 CMOS 门电路，其阻值则在 5~100 MΩ 之间。电容 C_1、C_2为两个反相器之间的耦合电容，它们的大小

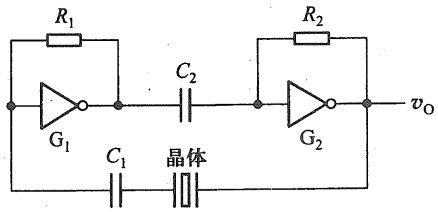


图 9.3.6 串联谐振型石英晶体振荡电路

选择应使其在频率为 f_S 时的容抗可以忽略不计，这样，可保证 G_1 和 G_2 之间形成正反馈环路。

2. 并联谐振型石英晶体振荡电路

并联谐振型石英晶体振荡电路如图 9.3.7 所示。R_f 是偏置电阻，其取值一般在 10～100 MΩ之间，它的作用是设置直流静态工作点（$V_B = V_A = V_{TH} \approx V_{DD}/2$），保证 CMOS 反相器 G_1 能工作在其电压传输特性的转折区——线性放大状态，即反相器 G_1 与 R_f 组成基本放大电路。图中，石英晶体谐振器的工作频率位于串联谐振频率 f_S 和并联谐振频率 f_P 之间，使晶体呈现电感特性，以便与电容 C_1、C_2 一起组成选频反馈网络，这个网络将 B 点输出信号的一部分反馈到输入端 A，再由 G_1 放大以维持振荡。放大电路和选频网络共同组成电容三点式振荡电路。反相器 G_2 起整形缓冲作用，因为振荡电路输出信号接近于正弦波，经 G_2 整形后变成矩形波，并且 G_2 还能提高振荡电路的带负载能力。

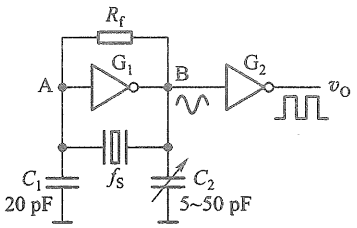


图 9.3.7 并联谐振型石英晶体振荡电路

电路的振荡频率由谐振回路的参数（C_1、C_2 和石英晶体的等效电感 L_{eq}）决定，但与石英晶体本身的谐振频率十分接近。C_1、C_2 的值一般取几十 pF，其中 C_2 用来微调振荡频率。由于振荡电路被限定在 f_S 与 f_P 之间的频率范围内工作，而在这一频率范围内，石英晶体的感抗曲线很陡峭，感抗稍有变化时，频率变化极小，使得电路频率稳定性很高。

石英晶振构成的秒脉冲信号产生电路如图 9.3.8 所示。图中，CC4060 为异步二进制计数器/振荡器。它内部有 14 个串行级联的 T 触发器，其输出端 $Q_4 \sim Q_{10}$，$Q_{12} \sim Q_{14}$ 分别对应 $2^4 \sim 2^{10}$ 和 $2^{12} \sim 2^{14}$ 分频。另外，在 9、10、11 这 3 个引脚上外接电阻、电容和晶振可以构成振荡电路，振荡信号可以从引脚 9 输出，同时也送到内部计数器进行分频，得到多种不同频率的输出信号。按照图中的电路连接，由晶振电路产生 32768 Hz 的基准时钟，经 2^{14} 分频后得到 2 Hz 的脉冲信号输出，再经过 D 触发器构成的二分频器分频后，得到 1 Hz 的时钟信号输出。

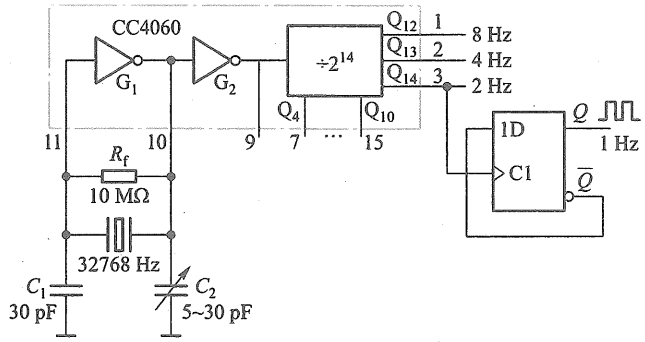


图 9.3.8 秒脉冲信号产生电路

复习思考题

9.3.1 简述多谐振荡电路的工作特点。

9.3.2 石英晶体振荡电路有什么工作特点？其振荡频率等于什么？

9.4 555 定时器及其应用

555 定时器[①]是一种模、数混合的中规模集成电路，它使用方便、灵活，应用极为广泛。用它可组成脉冲的产生、整形、延时和定时电路。下面介绍它的电路结构及其应用。

9.4.1 555 定时器

双极型 555 定时器的电路结构如图 9.4.1 所示[②]，它由 3 个电阻构成的分压器、电压比较器 C_1 和 C_2、基本 *SR* 锁存器、集电极开路的三极管 T 以及缓冲器 G 组成。集成电路 8 个引脚的名称和编号均标在点画线框外边。

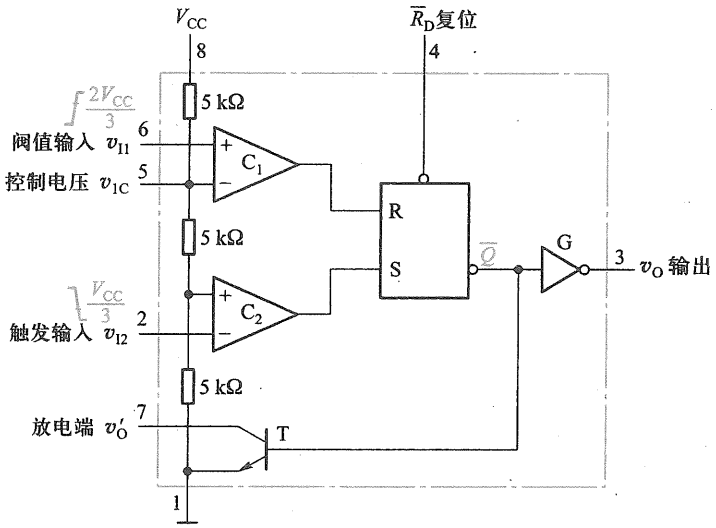


图 9.4.1 555 定时器的电路结构

$\overline{R}_D$ 为直接清零输入端。只要 $\overline{R}_D$ 为低电平，不管其他输入端的状态如何，输出端 v_O 立即变为低电平。正常工作时，必须使 $\overline{R}_D$ 端处于高电平。三极管 T 的作用是为外接的 *RC* 电路提供放电通路；由于其集电极开路，所以使用时，第 7 脚一般都要外接一个上拉电阻。

比较器 C_1 和 C_2 的输出控制着基本 *SR* 锁存器和放电三极管 T 的状态。当控制电压端 v_{IC} 悬空时（一般该端到地之间接 0.01 μF 左右的滤波电容），三个 5 kΩ 电阻串联组成的分压器为比较器提供参考电压。比较器 C_1 的参考电压为 $\frac{2}{3}V_{CC}$，比较器 C_2 的参考电压为 $\frac{1}{3}V_{CC}$。

① 由美国 Signetics 公司于 1972 年研制成功并投放市场，该公司于 1975 年被飞利浦（现在 NXP）公司收购。

② CMOS 型 555 定时器中三极管为 NMOS 管，3 个分压电阻的阻值不是 5 kΩ，而是 100 kΩ。

当 $v_{I1}<\frac{2V_{CC}}{3}$，$v_{I2}<\frac{V_{CC}}{3}$时，比较器 C_1输出低电平，比较器 C_2输出高电平，SR 锁存器置 1，$\overline{Q}$ 输出低电平，三极管 T 截止，输出端 v_O为高电平。

当 $v_{I1}>\frac{2V_{CC}}{3}$，$v_{I2}>\frac{V_{CC}}{3}$时，比较器 C_1输出高电平，比较器 C_2输出低电平，SR 锁存器置 0，$\overline{Q}$ 输出高电平，三极管 T 导通，输出端 v_O为低电平。

当 $v_{I1}<\frac{2V_{CC}}{3}$，$v_{I2}>\frac{V_{CC}}{3}$时，SR 锁存器 $R=0$，$S=0$，锁存器状态不变，电路保持原状态不变。

当 $v_{I1}>\frac{2V_{CC}}{3}$，$v_{I2}<\frac{V_{CC}}{3}$时，$R=S=1$，锁存器输出 $Q=\overline{Q}=0$，T 截止，v_O 为高电平，这时 555 的输出与 $v_{I1}<\frac{2V_{CC}}{3}$，$v_{I2}<\frac{V_{CC}}{3}$时相同，所以将它们合并列于表 9.4.1 中第 2 行。

综上所述，555 定时器功能表如表 9.4.1 所示（表格中用×表示任意状态）。

表 9.4.1 555 定时器功能表

输入			输出	
阈值输入（v_{I1}）	触发输入（v_{I2}）	复位（$\overline{R_D}$）	输出（v_O）	放电管 T
×	×	0	0	导通
×	$<V_{CC}/3$	1	1	截止
$>2V_{CC}/3$	$>V_{CC}/3$	1	0	导通
$<2V_{CC}/3$	$>V_{CC}/3$	1	不变	不变

上面讨论时，假定控制电压端是悬空的，因而比较器 C_1和 C_2的参考电压分别为$\frac{2}{3}V_{CC}$和$\frac{1}{3}V_{CC}$。如果控制电压端（5 脚）外接电压 v_{IC}，则比较器 C_1、C_2的参考电压就变为 v_{IC}和 $v_{IC}/2$。

555 定时器有双极性和 CMOS 两种类型。双极性型的电源电压为（4.5～18）V，输出高电平不低于电源电压的 90%，且能够输出和吸收 200 mA 的输出电流。为了降低功耗，后来又生产了 CMOS 型产品，例如 TLC555、ICM755 都是 CMOS 器件，它的电源电压为（2～18）V，输出高电平不低于电源电压的 95%，其吸收电流为 100 mA，输出电流为 10 mA。为了提高集成度，后来还生产了双定时器产品（如 NE556A，ICM7556 等）和四定时器产品（如 NE558 等）。

9.4.2 用 555 组成的施密特触发电路

将 555 定时器的阈值输入端和触发输入端连接在一起作为信号输入端，便构成了施密特触发电路，如图 9.4.2(a)所示。控制电压端（5）接 0.01 μF 的滤波电容到地，有助于稳定

比较器的参考电压。

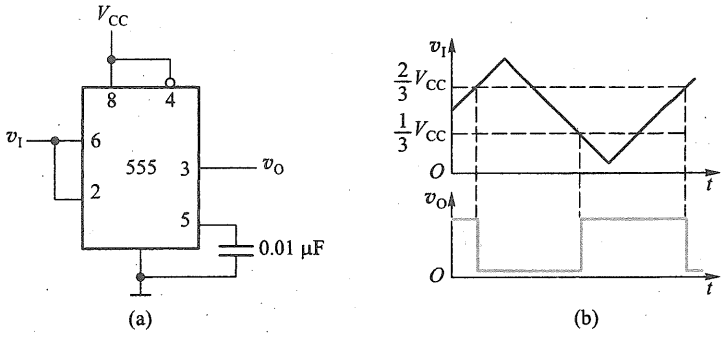


图 9.4.2 用 555 构成的施密特触发电路及工作波形

(a) 电路图 (b) 工作波形图

如果输入 v_I 为三角波，电路的工作波形如图 9.4.2(b)所示。

当 $v_I<\frac{V_{CC}}{3}$ 时，根据 555 定时器功能表可知，输出 v_O 为高电平；当 v_I 增加到满足 $\frac{V_{CC}}{3}<v_I<\frac{2V_{CC}}{3}$ 时，输出 v_O 维持高电平不变；一旦 $v_I>\frac{2V_{CC}}{3}$，v_O 就由高电平跳变为低电平；之后 v_I 再增加，v_O 保持低电平不变。

如果 v_I 由大于 $\frac{2}{3}V_{CC}$ 的电压值逐渐下降，只要 $\frac{V_{CC}}{3}<v_I<\frac{2V_{CC}}{3}$，$v_O$ 仍然维持低电平不变；只有当 $v_I<\frac{V_{CC}}{3}$ 时，电路才再次翻转，v_O 由低电平跳变为高电平。

可见，施密特触发电路能将输入缓慢变化的三角波整形成为矩形脉冲波输出。它的电压传输特性曲线如图 9.4.3 所示。

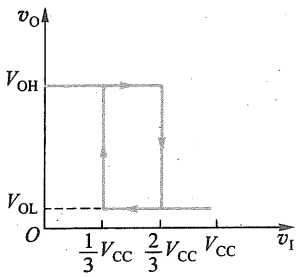


图 9.4.3 施密特触发电路的电压转输特性曲线

v_I 在上升过程中，所对应的正向阈值电压为

$$V_{T+}=\frac{2}{3}V_{CC}$$

v_I 在下降过程中，所对应的负向阈值电压为

$$V_{T-}=\frac{1}{3}V_{CC}$$

其回差电压为

$$\begin{aligned}\Delta V_T &= V_{T+}-V_{T-}\\ &=\frac{2}{3}V_{CC}-\frac{1}{3}V_{CC}=\frac{1}{3}V_{CC}\end{aligned} \tag{9.4.1}$$

若在控制电压端(5 脚)加电压 V_R，则有 $V_{T+}=V_R$，$V_{T-}=V_R/2$，$\Delta V_T=V_R/2$；改变 V_R 可以调节电路的回差电压。

9.4.3 用 555 组成的单稳态电路

用 555 构成的单稳态电路如图 9.4.4 所示。点画线框内的 R_2、C_2是微分电路，用于产生窄的触发脉冲；二极管 D_1用以旁路正向脉冲，保护输入引脚。假设电路输出信号的脉冲宽度为 t_w，则要求 t_w远大于时间常数 R_2C_2，一般按照下式来选取 R_2和 C_2的值

$$R_2C_2=t_w/10 \tag{9.4.2}$$

电路的工作波形如图 9.4.5 所示。电路无触发信号（v_I无负向脉冲）时，v_2为高电平，电路工作在稳定状态（$v_O=0$）。此时，555 内部的三极管 T 导通，电容 C 通过 T 放电，使 $v_C=0$。

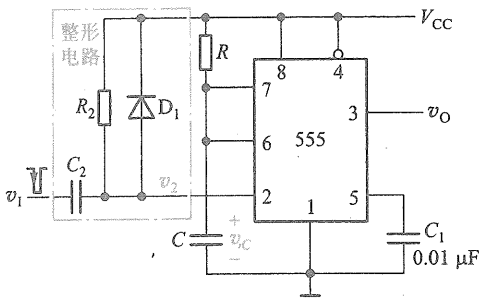


图 9.4.4 用 555 构成的单稳态电路

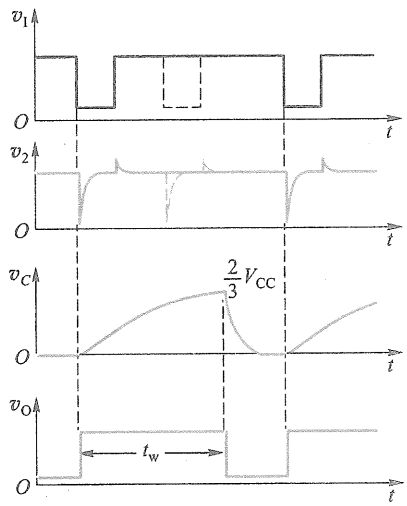


图 9.4.5 工作波形

若在触发输入端加一个负向触发信号$\left(v_I<\dfrac{V_{CC}}{3}\right)$，$v_O$就会由低电平跳变到高电平，电路进入暂稳态，三极管 T 截止。此后，V_{CC}通过 R 向电容 C 充电，v_C按指数规律上升。当 v_C上升到略大于$\dfrac{2V_{CC}}{3}$时，电路的输出 v_O就会由高电平翻转为低电平，同时 T 导通，于是电容 C 放电，电路返回到稳定状态。

如果忽略三极管 T 的饱和压降，则 v_C从 0 V 上升到$\dfrac{2V_{CC}}{3}$的时间，即为输出电压 v_O的脉冲宽度。由 v_C的波形可知，$v_C(0)=0$，$v_C(\infty)=V_{CC}$，$v_C(t_w)=V_{TH}=2V_{CC}/3$，$\tau=RC$，将这些值代入 RC 电路过渡过程的计算公式（9.1.1），求得 v_O的脉冲宽度为

$$t_w=RC\ln\frac{v_C(\infty)-v_C(0)}{v_C(\infty)-v_C(t_w)}=RC\ln\frac{V_{CC}-0}{V_{CC}-\frac{2}{3}V_{CC}}=RC\ln3\approx1.1RC \tag{9.4.3}$$

可见，t_w只与定时元件 R、C 的值有关，与输入触发脉冲和电源电压无关。通常 R 的取

值在 1 kΩ~3.3 MΩ 之间，电容取值为 470 pF~470 μF 之间。这种电路产生的脉冲宽度可从几个微秒到数分钟，精度可达 0.1%。

由图 9.4.5 可知，电路由外加到触发输入端的负向脉冲启动后，就会输出一个单脉冲，因此该电路也被称为单脉冲产生器。如果在电路的暂稳态持续时间内，加入新的触发脉冲（如图 9.4.5 中的虚线所示），则该脉冲并不能改变 555 内部 SR 锁存器的状态，因此对电路不起作用，电路为不可重复触发单稳态电路。

例 9.4.1 设计一个如图 9.4.4 所示的单稳态电路，使得 $t_w = 500$ ms。假设 $V_{CC} = 5$ V。

解： 设计步骤如下：

（1）选择一个合适的 C 值，令 $C = 1$ μF。

（2）求 R 的值。由式(9.4.3)得

$$R = \frac{t_w}{1.1C} = \frac{500\ \text{ms}}{1.1 \times 1\ \mu\text{F}} = 454.5\ \text{k}\Omega\text{（使用一个 470 kΩ 可变电阻）}$$

（3）选择一个合适的 C_2 值，令 $C_2 = 1$ μF。

（4）求 R_2 的值。由式(9.4.2)得

$$R_2 = \frac{t_w}{10C_2} = \frac{500\ \text{ms}}{10 \times 1\ \mu\text{F}} = 50\ \text{k}\Omega$$

用单稳态电路组成的脉宽调制[①]电路如图 9.4.6(a)所示。在单稳态电路的电压控制端输入一个称为调制信号的低频信号（例如，三角波或正弦波等），当调制电压升高时，电路的阈值电压升高，输出的脉冲宽度随之增加；而当调制电压降低时，电路的阈值电压也降低，输出的脉冲宽度则随之减小。随着调制电压的变化，在单稳态电路的输出端，就可得到一串随控制电压变化的脉宽调制波形，如图 9.4.6(b)所示。可见，引脚 2 的每一个触发信号产生一个脉冲输出。由于触发信号的周期为 T，输出信号将是周期为 T 的矩形脉冲序列。调制信号对周期 T 没有影响，但它可以改变每一个输出脉冲的宽度。

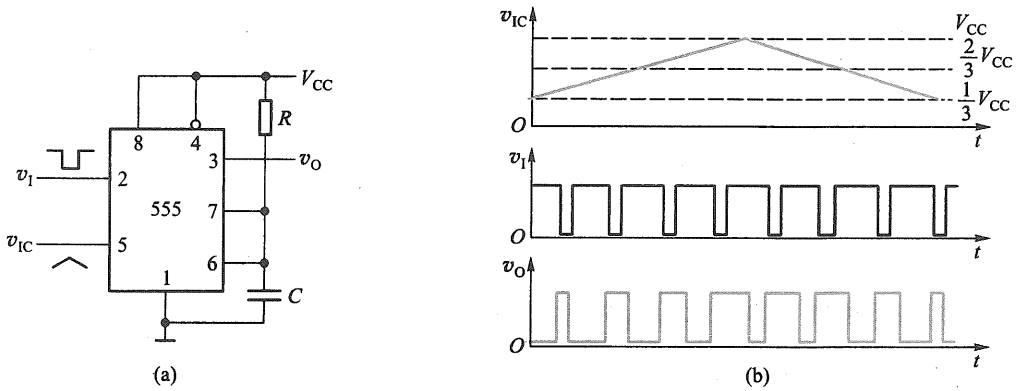


图 9.4.6 脉冲宽度调制器

（a）电路 （b）波形图

① pulse width modulation，PWM。PWM 控制技术广泛应用在从测量、通信到功率控制与变换的许多领域中。

脉宽调制电路常用于通信领域。用低频调制信号(例如,声音)改变高频信号的脉冲宽度,该高频信号称为载波。被调制的载波可以通过导线或者光缆等介质传输到接收器。接收器恢复出调制信号,用来驱动负载(例如,扬声器)。

9.4.4 用 555 组成的多谐振荡电路

由 555 定时器构成的多谐振荡电路如图 9.4.7(a)所示。接通电源时,电容 C 未充电,$v_C=0$ 使 v_O 为高电平,555 内部三极管 T 截止,电源 V_{CC} 通过串联电阻(R_1+R_2)给电容 C 充电,当 v_C 上升到略大于 $\frac{2V_{CC}}{3}$ 时,使 v_O 为低电平,同时 T 导通,此时电容 C 通过 R_2 和 T 放电,v_C 下降。当 v_C 下降到略小于 $\frac{V_{CC}}{3}$ 时,v_O 翻转为高电平。电容器 C 放电所需的时间为

$$t_{pL}=R_2C\ln\frac{0-V_{T+}}{0-V_{T-}}=R_2C\ln 2\approx 0.7R_2C \tag{9.4.4}$$

当放电结束时,T 截止,V_{CC} 将通过 R_1、R_2 向电容器 C 充电,v_C 由 $\frac{V_{CC}}{3}$ 上升到 $\frac{2V_{CC}}{3}$ 所需的时间为

$$\begin{aligned}t_{pH}&=(R_1+R_2)C\ln\frac{V_{CC}-V_{T-}}{V_{CC}-V_{T+}}\\&=(R_1+R_2)C\ln 2\approx 0.7(R_1+R_2)C\end{aligned} \tag{9.4.5}$$

当 v_C 上升到 $\frac{2V_{CC}}{3}$ 时,电路又翻转为低电平。如此周而复始,于是,在电路的输出端就得到一个周期性的矩形波。电路稳定工作时波形如图 9.4.7(b)所示,其振荡频率为

$$f=\frac{1}{t_{pL}+t_{pH}}\approx\frac{1.43}{(R_1+2R_2)C} \tag{9.4.6}$$

由于 555 内部的比较器灵敏度较高,而且采用差分电路形式,用 555 构成的多谐振荡电路的振荡频率受电源电压和温度变化的影响很小。

图 9.4.7(a)所示电路的 $t_{pL}\neq t_{pH}$,而且占空比固定不变。如要实现占空比可调可采用如

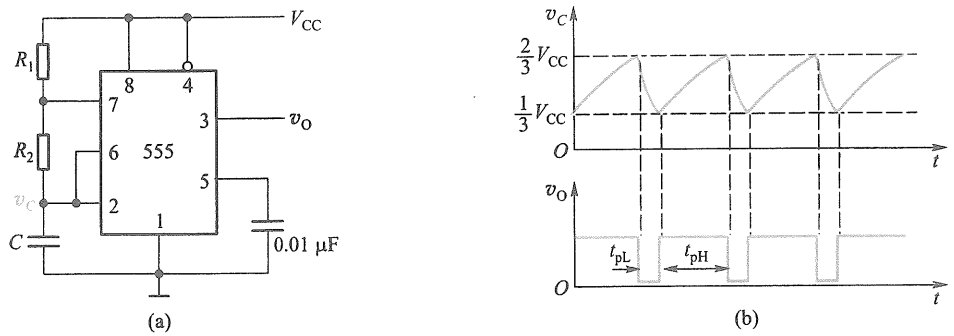


图 9.4.7 多谐振荡电路

(a) 电路图 (b) 工作波形

图 9.4.8 所示电路。由于二极管 D_1、D_2的单向导电特性，使电容器 C 的充电、放电回路分开，调节可变电阻，就可调节多谐振荡电路的占空比。图中，V_{CC}通过 R_A、D_1向电容 C 充电，充电时间为

$$t_{pH} \approx 0.7R_AC \tag{9.4.7}$$

电容器 C 通过 D_2、R_B及 555 中的三极管 T 放电，放电时间为

$$t_{pL} \approx 0.7R_BC \tag{9.4.8}$$

因而，振荡频率为

$$f=\frac{1}{t_{pH}+t_{pL}} \approx \frac{1.43}{(R_A+R_B)C} \tag{9.4.9}$$

电路输出波形的占空比为

$$q(\%)=\frac{R_A}{R_A+R_B}\times 100\% \tag{9.4.10}$$

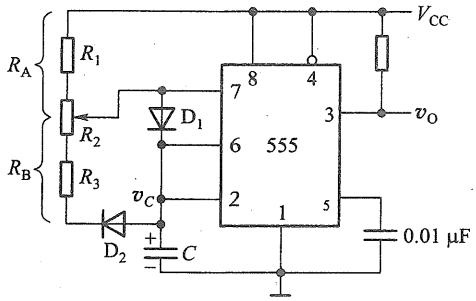


图 9.4.8 占空比可调的方波发生器

上面仅讨论了由 555 定时器组成的施密特触发电路、单稳态电路和多谐振荡电路，实际上，由于 555 定时器的比较器灵敏度高、输出驱动电流大、功能灵活，因而在电子电路中获得了广泛应用，限于篇幅，这里就不一一列举了。

复习思考题

9.4.1 图 9.4.1 中，555 定时器内部的反相器 G 有什么作用？

9.4.2 如将图 9.4.2(a)中 555 定时器的控制电压输入端(5 脚)改接到电压 V_R上，试问电路的输出脉宽 t_w有无改变？t_w与 V_R的关系如何？

小 结

- 单稳态电路分为不可重复触发和可重复触发两大类，在暂稳态期间出现的触发信号对不可重复触发电路的输出脉宽没有影响，而对可重复触发电路可起到连续触发作用。
- 施密特触发电路具有迟滞特性，它有两个阈值电压，使电路的抗干扰能力较强，常用于波形变换、整形及抗干扰电路。
- 多谐振荡电路无须外加输入信号就能在接通电源后自行产生矩形波输出。在频率稳定性要求较高的场合通常采用石英晶体振荡电路。
- 555 定时器内部含有分压电路、两个比较器、一个基本 *SR* 锁存器和一个三极管等，可以用它构成施密特触发电路、单稳态电路和多谐振荡电路等。

习 题

9.1 单稳态电路

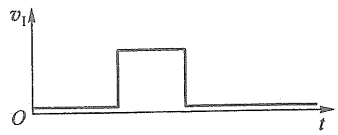
9.1.1 微分型单稳态电路如图 9.1.1(a)所示，已知 $C=0.01\ \mu F$，$R=9.1\ k\Omega$，电源电压

$V_{DD}=10$ V。试对应图题 9.1.1 所示的输入波形画出电路输出 v_O 的波形，并计算输出脉宽 t_w 及幅值大小。

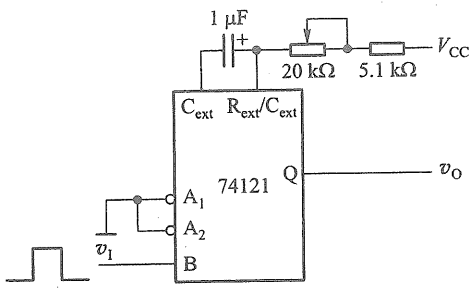
9.1.2 由集成单稳态电路 74121 组成的延时电路及输入波形如图题 9.1.2 所示。要求：

(1) 计算输出脉宽的变化范围。

(2) 解释为什么使用电位器时要串接一个电阻。



图题 9.1.1

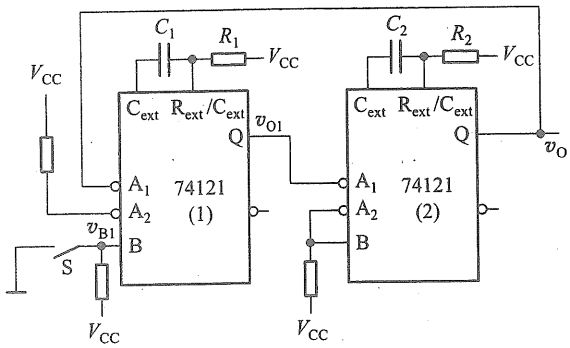


图题 9.1.2

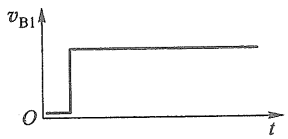
9.1.3 利用两片集成单稳态电路 74121 构成的多谐振荡电路如图题 9.1.3 所示。要求：

(1) 试说明开关 S 从闭合变为断开时，电路的工作原理。并对应 v_{B1} 画出 v_{O1} 和 v_O 波形。

(2) 计算电路的振荡频率。(提示：稳态时，两个单稳态电路的输出均为 0，当开关 S 断开时，电路开始振荡)



(a)



(b)

图题 9.1.3

9.2 施密特触发电路

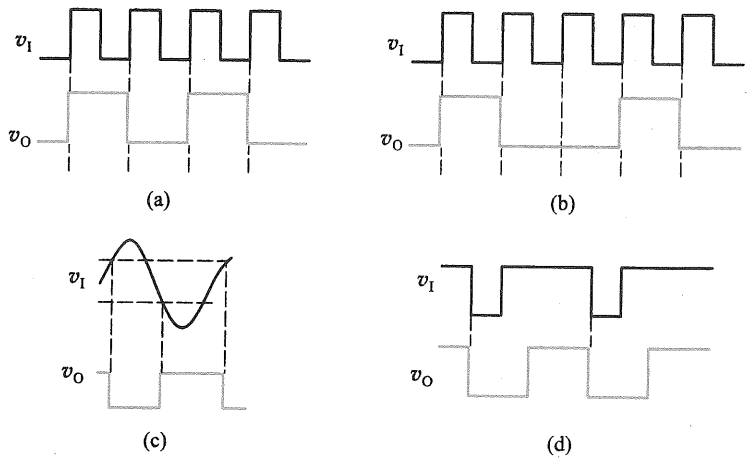
9.2.1 已知几种电路的输入、输出波形如图题 9.2.1 所示，试问应分别选取何种电路才能实现如图所示工作波形？

9.2.2 集成施密特触发电路和 4 位计数器 74LVC161 组成的电路如图题 9.2.2 所示。

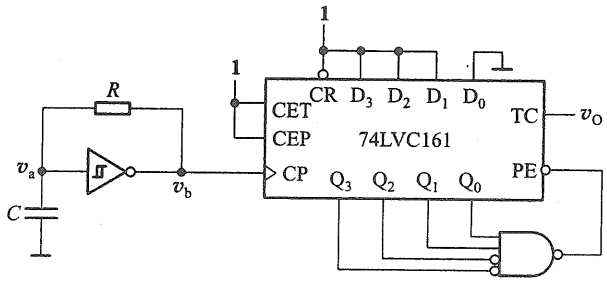
(1) 分别说明图中两部分电路的功能。

(2) 画出图中 74LVC161 组成的电路的状态转换图。

(3) 画出图中 v_a、v_b 和 v_0 的对应波形。



图题 9.2.1

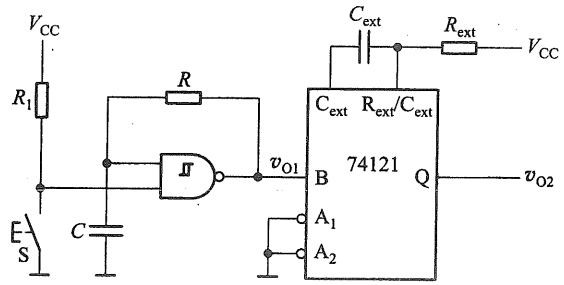


图题 9.2.2

9.2.3 集成施密特电路和集成单稳态电路 74121 构成如图题 9.2.3 所示电路。已知集成施密特电路的 $V_{DD}=10\ V$，$R=100\ k\Omega$，$C=0.01\ \mu F$，$V_{T+}=6.3\ V$，$V_{T-}=2.7\ V$，$C_{ext}=0.01\ \mu F$，$R_{ext}=30\ k\Omega$。

（1）分别计算 v_{O1} 的周期及 v_{O2} 的脉宽。

（2）根据计算结果画出 v_{O1}、v_{O2} 的波形。



图题 9.2.3

9.3 多谐振荡电路

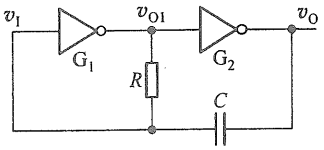
9.3.1 一种最简型多谐振荡电路如图题 9.3.1（a）所示，图（b）还给出了实际 CMOS 反相器

集成电路的内部电路，D_1、D_2、D_3、D_4均为内部的保护二极管，假设二极管的正向导通压降为 V_{DF} = 0.7 V，CMOS 反相器的阈值电压为 V_{TH}。

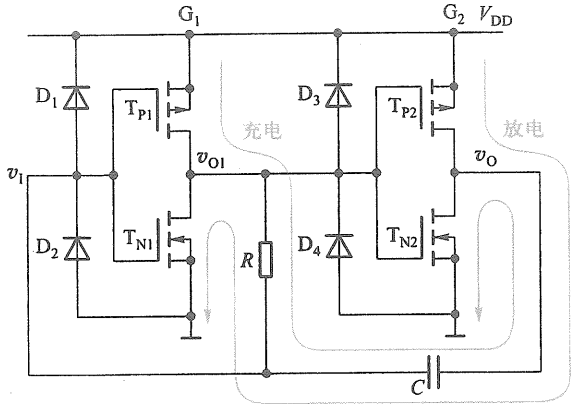
(1) 解释电路的工作原理，画出 v_I和 v_O点的电压波形。

(2) 计算电路的振荡周期。

(3) 当阈值电压 V_{TH}由 $V_{DD}/2$ 改变至 $2V_{DD}/3$ 时，电路的振荡频率如何变化？



(a)



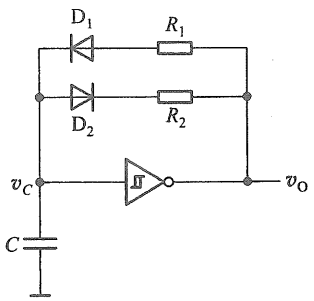
(b)

图题 9.3.1

9.3.2 由集成施密特 CMOS 非门电路组成的脉冲占空比可调多谐振荡电路如图题 9.3.2 所示。设电路中 R_1、R_2、C 及 V_{DD}、V_{T+}、V_{T-}的值已知。

(1) 定性画出 v_C及 v_O波形。

(2) 写出输出信号 v_O频率的表达式。



图题 9.3.2

9.4 555 定时器及其应用

9.4.1 图题 9.4.1 所示为一简易触摸开关电路。当手摸金属片时，发光二极管亮，经过一定时间后，发光二极管自动熄灭。试说明其工作原理，问手摸金属片后发光二极管亮多长时间自动熄灭？

9.4.2 由 555 定时器及场效应管 T 组成如图题 9.4.2 所示电路，其中 T 工作于可变电阻区，其导通电阻为 R_{DS}。

(1) 说明电路功能。

(2) 写出输出 v_O频率的表达式。

10 数模与模数转换器

引言

文档 10.1：第 10 章思维导图

随着数字技术，特别是计算机技术的飞速发展与普及，在现代控制、通信及检测领域中，信号的处理无不广泛地采用了计算机技术。由于自然界中的物理量，如压力、温度、位移、液位等都是模拟量，如要用数字技术处理这些模拟信号，则往往需要一种能在模拟信号与数字信号之间起转换作用的电路——模数转换器和数模转换器。

能将模拟信号转换成数字信号的电路称为模数转换器（analog to digital converter，简称 ADC 或 A/D 转换器）；反之，能把数字信号转换为模拟信号的电路称为数模转换器（digital to analog converter，简称为 DAC 或 D/A 转换器）。A/D 和 D/A 转换器是现代计算机系统中不可缺少的基本组成部分。

A/D 转换器和 D/A 转换器在工业控制系统中的作用如图 10.0.1 所示。模拟传感器将工业生产过程中的各种物理量变换成模拟电信号，A/D 转换器则将传感器送来的模拟电信号转换成数字信号，再送入计算机进行处理、存储并产生数字控制信号，D/A 转换器把来自计算机的数字控制信号转换为模拟信号，经模拟控制器实现对被控对象的控制。

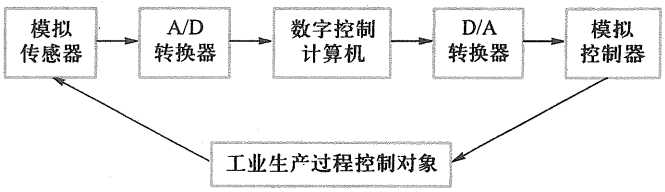


图 10.0.1　A/D 转换器与 D/A 转换器在工业控制系统中的作用

下面介绍几种常用 D/A 转换器与 A/D 转换器的电路结构、工作原理及其应用，以便在实际工作中能更好地选择和运用品种繁多的转换器产品。

10.1 D/A 转换器

10.1.1 权电阻网络 D/A 转换器

根据式（1.2.3），任何一个二进制数 $D_{n-1}D_{n-2}\cdots D_1D_0$ 可以展开为

$$N_B = D_{n-1} \times 2^{n-1} + \cdots + D_1 \times 2^1 + D_0 \times 2^0 \tag{10.1.1}$$

式中，2^{n-1}、…、2^1、2^0为各位的权。上式表明，要实现数模转换，首先要将输入二进制中为 1 的每一位代码按其权的大小，转换成模拟量，然后将这些模拟量相加，相加所得的总量就是与数字量成正比的模拟量。根据这个思路，可以用多种电路方案实现这种转换。

图 10.1.1 是实现 D/A 转换器的一种方案。它主要由寄存器、基准电压、电子开关、权电阻网络、求和电路等组成。

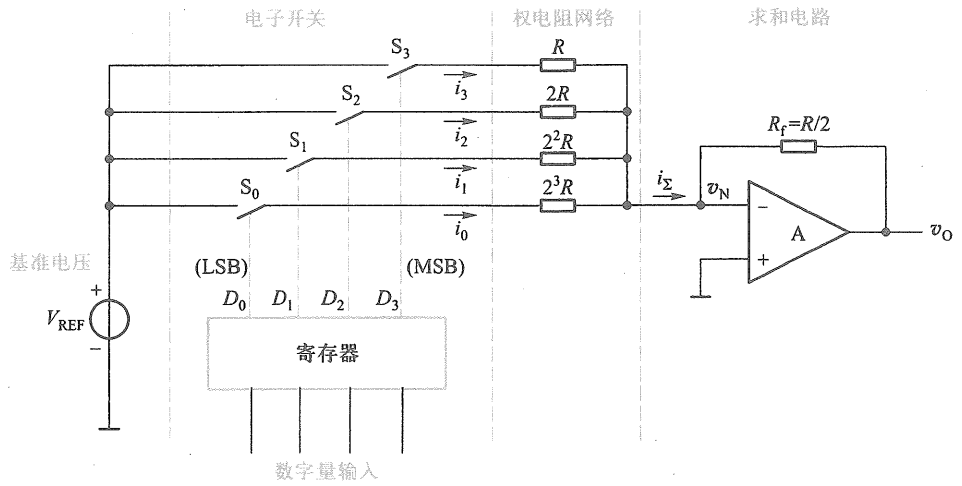


图 10.1.1 D/A 转换器原理电路

寄存器输出的 4 位二进制码 $D_3 \sim D_0$分别控制着电子开关 $S_3 \sim S_0$，当 $D_i = 1(i = 0,1,2,3)$ 时，电子开关 S_i与基准电压 V_{REF}接通，电流 i_i流入求和电路；当 $D_i = 0$ 时，电子开关断开，$i_i = 0$。

求和电路对电阻网络的输出电流进行求和，并转换成电压输出。运放工作在线性区，根据虚短、虚断的特点，可得

$$\begin{aligned} v_O &= -i_\Sigma R_f \\ &= -R_f(i_3 + i_2 + i_1 + i_0) \end{aligned} \tag{10.1.2}$$

由于 $v_N \approx 0$，故 $i_3 = \dfrac{V_{REF} D_3}{R}$，$i_2 = \dfrac{V_{REF} D_2}{2R}$，$i_1 = \dfrac{V_{REF} D_1}{4R}$，$i_0 = \dfrac{V_{REF} D_0}{8R}$，将它们代入式(10.1.2)，同时令 $R_f = \dfrac{R}{2}$，则得

$$\begin{aligned} v_O &= -R_f\left(\frac{V_{REF} D_3}{R} + \frac{V_{REF} D_2}{2R} + \frac{V_{REF} D_1}{4R} + \frac{V_{REF} D_0}{8R}\right) \\ &= -\frac{V_{REF}}{2^4} \sum_{i=0}^{3} (D_i \cdot 2^i) \end{aligned} \tag{10.1.3}$$

结果表明，电路实现了数字量到模拟电压的转换。

由于该电路中每一位电阻的阻值与这一位的“权”相对应，位权越大，对应的阻值越小，所以，该电路也称为权电阻网络 D/A 转换器。这种电路由于各个电阻的阻值不同，当输入

数字量的位数较多时，其阻值相差很大，很难在集成电路里实现这样的精密电阻，因此在集成 D/A 转换器中，很少采用这种电路。

图 10.1.1 中的权电阻网络是完成 D/A 转换的核心电路，通常称为解码网络。常用的解码网络有倒 T 形电阻网络、电阻串联分压式网络、权电流电路等。

寄存器①用来保存外部输入的数字量。根据输入数字量方式的不同，可以分为并行输入 D/A 转换器和串行输入 D/A 转换器；并行输入 D/A 转换器具有更快的转换速度，而串行输入 D/A 转换器具有更少的引脚，适合于便携设备。

10.1.2 倒 T 形电阻网络 D/A 转换器

下面以 4 位 D/A 转换器为例说明其工作原理。

1. 4 位倒 T 形电阻网络 D/A 转换器

4 位倒 T 形电阻网络 D/A 转换器的原理图如图 10.1.2 所示。电路中的电阻只有 R 和 $2R$ 两种阻值，且 R-$2R$ 电阻网络呈倒 T 形，故而得名。

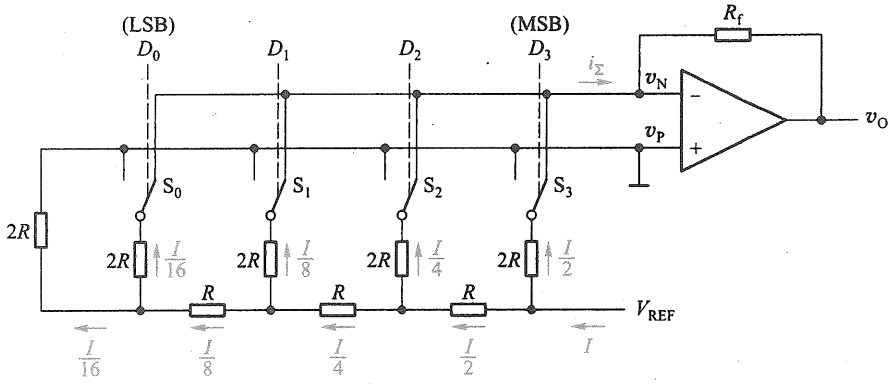


图 10.1.2 4 位倒 T 形电阻网络 D/A 转换器

图中，S_0~S_3为模拟开关，它受到来自寄存器的输入数码 D_0~D_3的控制。当 $D_i=0$，S_i接地；当 $D_i=1$，S_i接运算放大器反相端。这里，电阻网络输出的模拟量为电流 i_Σ，引入电压并联负反馈的运放电路将电流 i_Σ 转换成电压 v_O输出。

运放工作在线性运用状态，其反相端“虚地”。这样，无论模拟开关 S_i置于何种位置，与 S_i相连的 $2R$ 电阻总是接“地”，所以，每条 $2R$ 电阻支路上的电流在开关 S_i切换前、后是不变的。为了计算流过各 $2R$ 电阻支路的电流，画出 4 位倒 T 形电阻网络的等效电路如图 10.1.3 所示。

分析发现，从 AA、BB、CC、DD 向左看，每个端口网络的等效电阻均为 R。因此，从基准电压源流出的总电流始终为

$$I=V_{REF}/R \tag{10.1.4}$$

由于 CC 端口左侧电阻为 R，与右侧 R 串联，再与 $2R$ 并联，因此流入 DD 端口的电流 I

① 该寄存器通常被称为 DAC 寄存器(DAC register)。

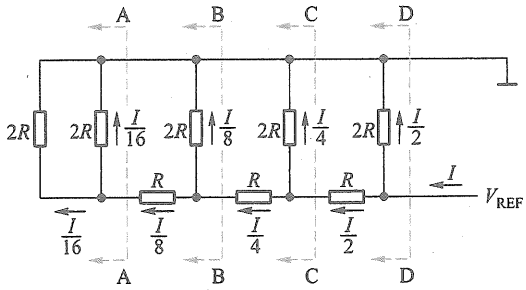


图 10.1.3 4 位倒 T 形电阻网络等效电路

被平分，故最右侧 $2R$ 支路的电流为 $I/2$；同理得，其他 $2R$ 支路（从右到左）的电流分别为 $I/4$、$I/8$ 和 $I/16$。

回到图 10.1.2，同时考虑 $D_0 \sim D_3$ 的值，于是，倒 T 形电阻网络的总电流 i_Σ 为

$$\begin{aligned} i_\Sigma &= \frac{I}{2}D_3 + \frac{I}{4}D_2 + \frac{I}{8}D_1 + \frac{I}{16}D_0 \\ &= \frac{I}{2^4}(2^3D_3 + 2^2D_2 + 2^1D_1 + 2^0D_0) \\ &= \frac{V_{REF}}{2^4 \times R}\sum_{i=0}^{3}(D_i \cdot 2^i) \end{aligned} \tag{10.1.5}$$

输出电压

$$v_O = -i_\Sigma R_f = -\frac{R_f}{R} \cdot \frac{V_{REF}}{2^4}\sum_{i=0}^{3}(D_i \cdot 2^i) \tag{10.1.6}$$

如将输入数字量扩展到 n 位，可得 n 位倒 T 形电阻网络 D/A 转换器输出模拟量与输入数字量之间的一般关系式为

$$v_O = -\frac{V_{REF}}{2^n} \cdot \frac{R_f}{R}\left[\sum_{i=0}^{n-1}(D_i \cdot 2^i)\right] \tag{10.1.7}$$

若将式中 $\frac{V_{REF}}{2^n} \cdot \frac{R_f}{R}$ 用 K 表示，方括号内的 n 位二进制数字量用 N_B 表示，则式（10.1.7）可改写为

$$v_O = -KN_B \tag{10.1.8}$$

上式表明，对应每一个数字量 N_B，在电路的输出端都能得到与之成正比的模拟电压。如果 $R_f = R$，则当数字量为零时，$v_O = 0$；当数字量 N_B 为最大值（即各位都为 1）时，$v_O = -\frac{2^n-1}{2^n}V_{REF}$，即 v_O 的最大变化范围为 $0 \sim -\frac{2^n-1}{2^n}V_{REF}$。当 V_{REF} 为正值，则输出电压 v_O 就为负值；反之亦然。

在图 10.1.2 所示的 D/A 转换器中，开关切换时 $2R$ 支路电流不变，有利于提高转换速度。另外，电路中只需要阻值为 R 和 $2R$ 的电阻，这对集成电路的制造十分有利，因此这种解码网络得到广泛应用。

例 10.1.1 在图 10.1.2 所示电路中，已知 $R_f = R = 10\ \text{k}\Omega$，$V_{REF} = 10\ \text{V}$，试求：

（1）当 $D_3D_2D_1D_0$ 分别为 **1000**、**0100**、**0010**、**0001** 时输出电压 v_O 的值。

（2）画出该电路的输入/输出特性曲线。

解：（1）根据式(10.1.7)可得

$$D_3D_2D_1D_0=\mathbf{1000}\text{ 时},v_O=-\frac{V_{REF}}{2^4}\times2^3=-\frac{V_{REF}}{2}=-\frac{10\text{ V}}{2}=-5\text{ V}$$

$$D_3D_2D_1D_0=\mathbf{0100}\text{ 时},v_O=-\frac{V_{REF}}{2^4}\times2^2=-\frac{V_{REF}}{4}=-\frac{10\text{ V}}{4}=-2.5\text{ V}$$

$$D_3D_2D_1D_0=\mathbf{0010}\text{ 时},v_O=-\frac{V_{REF}}{2^4}\times2^1=-\frac{V_{REF}}{8}=-\frac{10\text{ V}}{8}=-1.25\text{ V}$$

$$D_3D_2D_1D_0=\mathbf{0001}\text{ 时},v_O=-\frac{V_{REF}}{2^4}\times2^0=-\frac{V_{REF}}{16}=-\frac{10\text{ V}}{16}=-0.625\text{ V}$$

根据计算过程可以得到一个结论：对于一个 n 位的 D/A 转换器，当输入二进制的最高位为 **1**、其余为 **0** 时，输出电压 $v_O=-\frac{V_{REF}}{2}$；仅当次高位为 **1**、其余为 **0** 时，输出电压 $v_O=-\frac{V_{REF}}{4}$；其余依此类推。

（2）根据 $D_3D_2D_1D_0$ 的不同取值，分别计算出 v_O 的值。然后以输入的数字量作为横轴，以输出电压 v_O 作为纵轴，画出该 D/A 转换电路的输入/输出特性曲线，如图 10.1.4 所示。可见，输出模拟电压为台阶状，总共有 15 个台阶，每一级台阶的电压值大小（或称步长）为 0.625 V。扩展到 n 位 D/A 转换器，台阶的总数为 2^n-1。

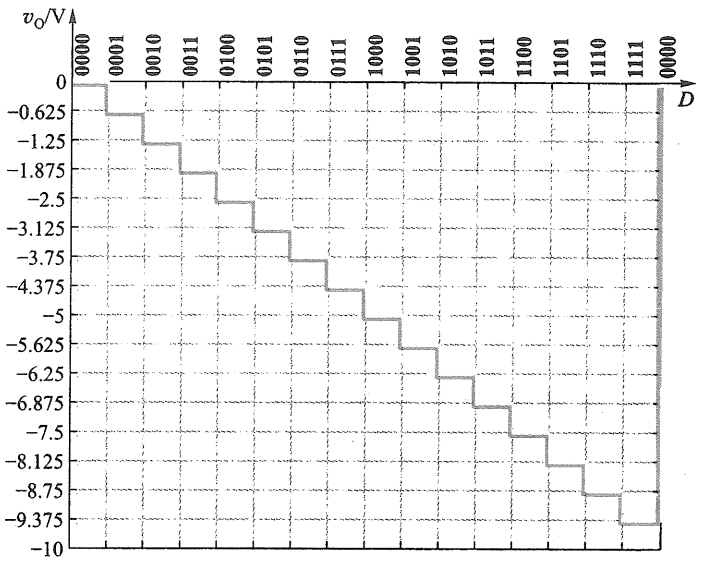


图 10.1.4　4 位倒 T 形电阻网络 D/A 转换器的输入/输出特性曲线

2. 集成倒 T 形电阻网络 D/A 转换器

国产 CB7520（与 ADI 公司的 AD7520 可互换使用）是采用 CMOS 工艺生产的电流输出型 10 位 D/A 转换器，其内部原理电路如图 10.1.5 点画线框内所示，其中包含电子模拟开

关、倒 T 型电阻解码网络和反馈电阻($R=10\ \text{k}\Omega$),基准电压源 V_{REF}需要外部提供。输出电流与输入 10 位自然二进制码的关系为

$$i_\Sigma=\frac{V_{REF}}{2^{10}\times R}\sum_{i=0}^{9}(D_i\cdot 2^i) \tag{10.1.9}$$

上式表明,输出电流与输入的数字量和模拟参考电压的乘积成比例,且电路中的 CMOS 开关能够承载任一方向的电流,即基准电压 V_{REF}可正、可负,甚至可以是交流电压,具有这种特性的 D/A 转换器通常称为"乘法 DAC"(multiplying DAC,简称 MDAC)。由于乘积可以是正数或负数;因此,它的全称是"四象限乘法 DAC"(four-quadrant multiplying DAC)。

在芯片外部接运放电路,将电流转换成电压输出。注意,为了减小输出电压的偏差,建议使用内部的反馈电阻($R=10\ \text{k}\Omega$)实现电流到电压的转换。因为芯片内部电阻值的公差一致性较好,根据式(10.1.7)可以消去电阻值差异带来的影响。

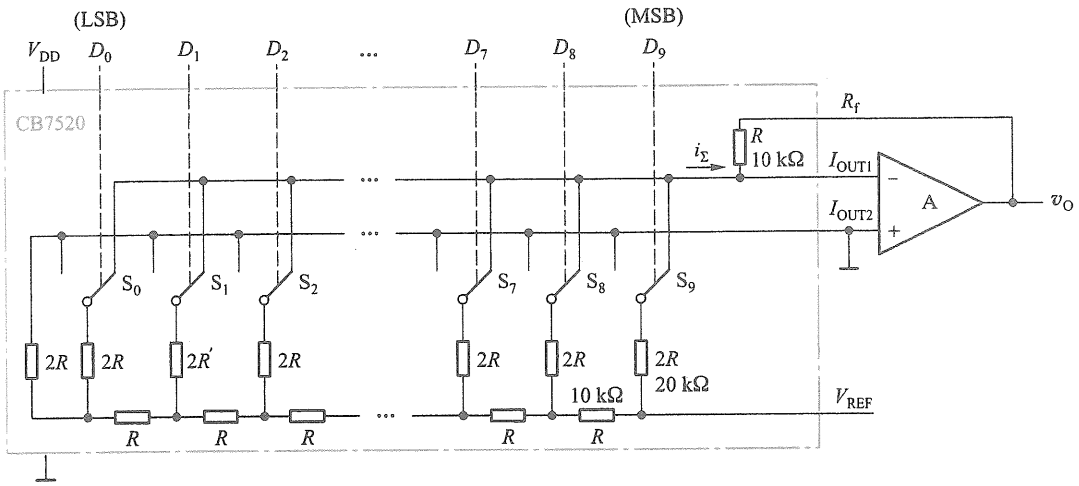


图 10.1.5 CB7520 内部电路

图 10.1.5 中电子开关 S_i的实际电路如图 10.1.6 所示。它是由 9 个 MOS 管组成的 CMOS 模拟开关电路。图中 $T_1\sim T_3$组成电平转移电路,使输入信号能与 TTL 电平兼容。T_4、T_5及 T_6、T_7组成两个反相器分别作为模拟开关管 T_8、T_9的驱动电路,T_8、T_9构成单刀双掷开关。

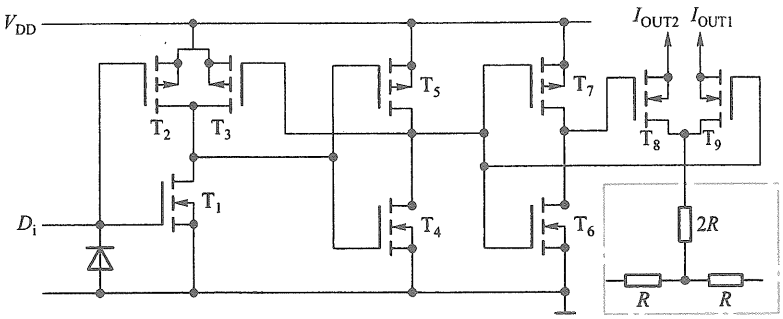


图 10.1.6 CMOS 模拟开关电路

当 $D_i=1$ 时，T_1输出低电平，T_4、T_5反相器输出高电平，而 T_6、T_7反相器输出低电平，从而使 T_8截止、T_9导通，$2R$ 电阻中电流经 T_9汇入 i_Σ（见图 10.1.5）。

当 $D_i=0$ 时，T_1输出高电平，T_4、T_5反相器输出的低电平使 T_9截止，T_6、T_7反相器输出的高电平使 T_8导通，这样 $2R$ 电阻中电流经 T_8接地（见图 10.1.5）。CMOS 模拟开关导通电阻可计入电阻网络。该电路具有功耗低、转换速度较快、温度系数小、通用性强等优点。

例 10.1.2 在图 10.1.5 所示 10 位倒 T 形电阻网络 D/A 转换器电路中，已知 $R_f=R$。

（1）试求输出电压的范围。

（2）若要求电路输入数字量为 $(1000000000)_B$ 时输出电压 $v_O=5$ V，试问 V_{REF} 应取何值？

解：（1）根据式（10.1.7）可知，10 位 D/A 转换器输出电压 v_O 为

$$v_O=-\frac{V_{REF}}{2^{10}}\cdot\frac{R_f}{R}\left[\sum_{i=0}^{9}(D_i\cdot 2^i)\right] \tag{10.1.10}$$

当 $D_9D_8\cdots D_0=00\cdots0$ 时，$v_O=0$ V；当 $D_9D_0\cdots D_0=(11\cdots1)_B$ 且 $R=R_f$ 时，$v_O=-V_{REF}(2^{10}-1)/2^{10}\approx -V_{REF}$。所以输出电压的范围为：$(-V_{REF}\sim 0)$ V。

（2）根据式（10.1.10）可得，

$$V_{REF}=-\frac{2^{10}\cdot v_O\cdot R}{R_f}\cdot\frac{1}{\sum_{i=0}^{9}(D_i\cdot 2^i)}$$

将 $R=R_f$，$v_O=5$ V 和 $D_9\cdots\cdots D_0=(1000000000)_B=512$，代入上式，得

$$V_{REF}=-10\text{ V}$$

10.1.3 权电流型 D/A 转换器

1. 4 位权电流 D/A 转换器

由于实际的倒 T 形电阻网络 D/A 转换器中的模拟开关存在导通电阻和导通压降，这会引起求和电流的误差，从而影响转换精度。如要提高 D/A 转换器的精度，可采用权电流型 D/A 转换器。4 位权电流 D/A 转换器的原理电路如图 10.1.7 所示。图中，用一组恒流源代替了图 10.1.2 中倒 T 形电阻网络，恒流源从高位到低位的电流大小依次为 $\frac{I}{2}$、$\frac{I}{4}$、$\frac{I}{8}$、$\frac{I}{16}$。

图 10.1.7 中，当 $D_i=0$ 时，开关 S_i 接地；$D_i=1$ 时，开关 S_i 与运放的反相端相联，相应的权电流流入求和电路。分析该电路可得

$$\begin{aligned}v_O&=i_\Sigma R_f=R_f\left(\frac{I}{2}D_3+\frac{I}{4}D_2+\frac{I}{8}D_1+\frac{I}{16}D_0\right)\\&=\frac{I}{2^4}\cdot R_f(D_3\cdot 2^3+D_2\cdot 2^2+D_1\cdot 2^1+D_0\cdot 2^0)\\&=\frac{I}{2^4}\cdot R_f\sum_{i=0}^{3}D_i\cdot 2^i\end{aligned} \tag{10.1.11}$$

由于权电流 D/A 转换器中，各支路上的权电流的大小不受开关导通电阻和电压的影响，所以该电路的转换精度较高。

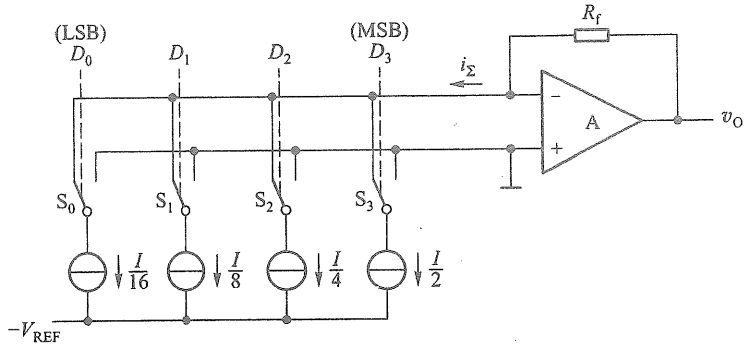


图 10.1.7 权电流 D/A 转换器的原理电路

2. 实际的权电流 D/A 转换器

在集成电路中，通常将电流源电路和 $R-2R$ 网络相结合，直接产生二进制权电流，典型的权电流 D/A 转换器电路如图 10.1.8 所示。

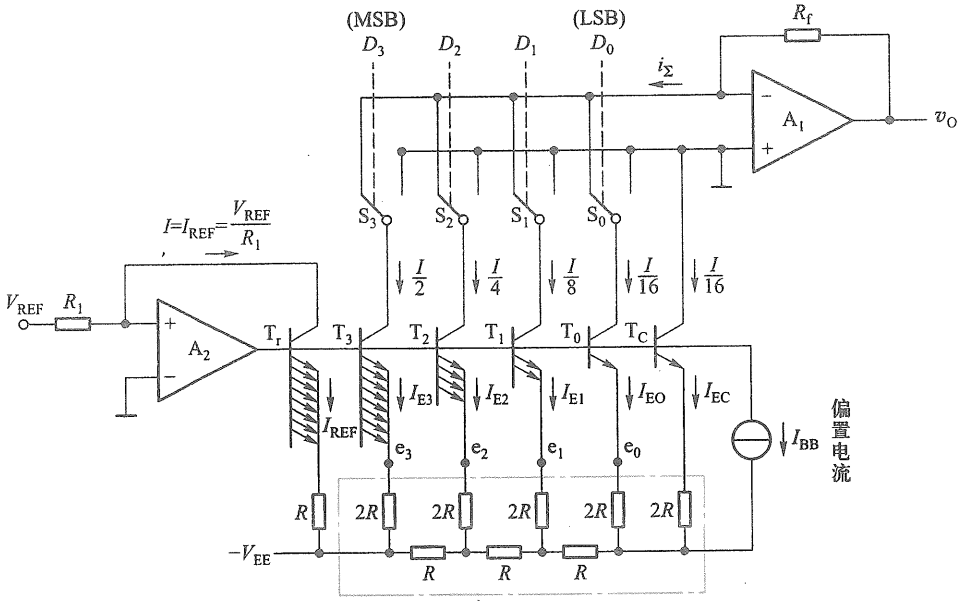


图 10.1.8 实际的权电流 D/A 转换器电路

运放 A_2、R_1、T_r、R 和 $-V_{EE}$组成基准电流 I_{REF}产生电路，A_2的输出端经 T_r的集电结组成电流并联负反馈电路。基准电流 I_{REF}由基准电压 V_{REF}和电阻 R_1确定。即

$$I_{REF}=\frac{V_{REF}}{R_1}=I \tag{10.1.12}$$

由于 T_r和 T_3具有相同的 V_{BE}，而发射极回路的电阻相差一倍，所以它们的发射极电流也相差一倍，于是有

$$I_{E3}=I_{REF}/2=I/2 \tag{10.1.13}$$

图中 T_3～T_0的基极均连接在一起，基极电压相同，只要这些三极管的发射结压降 V_{BE}相

等，则它们的发射极 $e_3 \sim e_0$ 就是等电位，这样，从左到右流过 $2R$ 电阻上的电流就分别为 $\frac{I}{2}$、$\frac{I}{4}$、$\frac{I}{8}$ 和 $\frac{I}{16}$。

为了保证每个 BJT 的发射结电压相等，在发射极电流较大的三极管中按比例地加大了发射结的面积，图中用增加发射极的数目来表示。电路中 $T_3 \sim T_0$ 发射极个数分别是 8、4、2、1（即发射极面积之比为 8∶4∶2∶1）。这样，在各 BJT 射极电流比值为 8∶4∶2∶1 的情况下，$T_3 \sim T_0$ 的射极电流的密度相等，它们的发射结电压 V_{BE} 也就相同，倒 T 形电阻网络每个 $2R$ 电阻的电流值就与图 10.1.7 完全相同。

另外，图 10.1.8 中的恒流源 I_{BB} 用来给 T_r、$T_3 \sim T_0$ 和 T_C 提供必要的基极偏置电流。最右边的三极管 T_C 是为了给倒 T 形电阻网络提供一个与其他端点等效的终端（与图 10.1.2 中最左边 $2R$ 电阻的作用相同），所以它的集电极直接接地，且与 T_0 完全匹配。

将式（10.1.12）代入式（10.1.10）中，得输出电压为

$$v_O = i_\Sigma R_f = \frac{R_f V_{REF}}{2^4 R_1}(D_3 \cdot 2^3 + D_2 \cdot 2^2 + D_1 \cdot 2^1 + D_0 \cdot 2^0)$$

对于 n 位权电流 D/A 转换器，可推得输出电压

$$v_O = \frac{V_{REF}}{R_1} \cdot \frac{R_f}{2^n} \sum_{i=0}^{n-1} (D_i \cdot 2^i) \tag{10.1.14}$$

通常，对于要求分辨率为 10 位以上且信号带宽为几兆赫兹的应用，首选这种权电流型 D/A 转换器。

10.1.4 电阻串联分压式 D/A 转换器①

1. 3 位电阻串联分压式 D/A 转换器

3 位电阻串联分压式 D/A 转换器的原理图如图 10.1.9 所示。8 个阻值相同的电阻 R 串联对基准电压 V_{REF} 进行分压，每一个分压节点得到的电压值为 $V_{REF}/8$ 的整数倍，分压节点与模拟开关相连接；输入 $D_2D_1D_0$ 通过 3 线-8 线译码器控制相应开关是否导通。每输入一个二进制数字量，与之对应的一个开关接通，则该节点的电压值就会被送到输出缓冲器。由于输出缓冲器是运放构成的同相电压跟随器，其输入阻抗高，这样就几乎不会向所连通的分压器取用电流，保证了各节点电压的精度。

电阻串联分压式 D/A 转换器结构简单，输出电压的单调性好。在 12 位以上的 D/A 转换器中，通常将两组电阻串联分压式电路进行级联以减少电阻器的数量。

2. 集成电阻串联分压式 D/A 转换器

当今很多低速高精度单片集成 D/A 转换器都使用串行数字接口。图 10.1.10 是 TI 公司 DAC121S101 的组成框图，它是采用 CMOS 工艺制造的 12 位 D/A 转换器，属于电压输出型转换器。

---

① 英文称为 resistor-string DAC（直译为电阻串 DAC），有时也称为开尔文分压式（Kelvin divider）DAC。

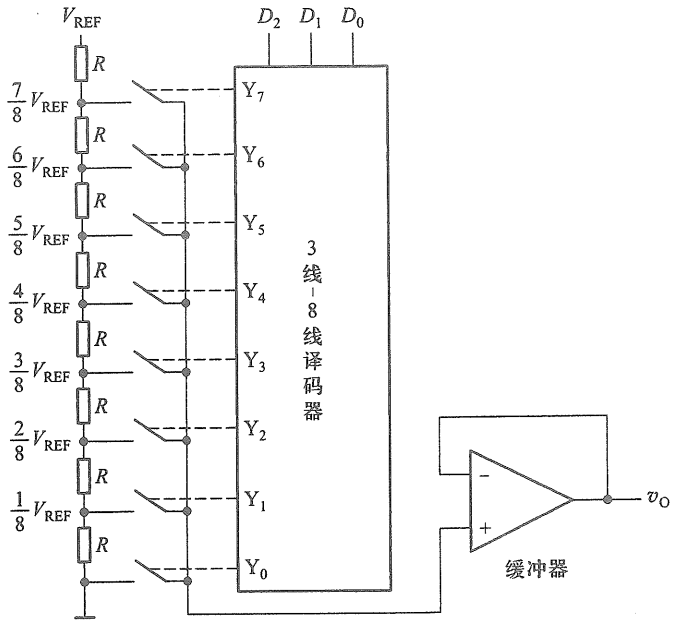


图 10.1.9 电阻串联分压式 3 位 D/A 转换器

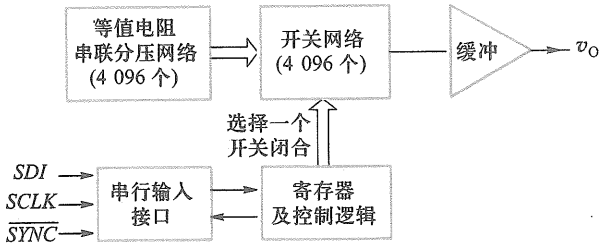


图 10.1.10 DAC121S101 的内部电路

芯片电源电压 V_{DD} 的范围为(2.7~5.5 V),并直接用 V_{DD} 作为基准电压源。分压器由 4 096 个阻值相同的电阻构成。芯片采用时钟输入端(*SCLK*)、串行数据输入端(*SDI*)和帧同步输入($\overline{SYNC}$)串行接口[①]输入数字量,时钟的最高工作频率为 30 MHz。

外部输入的二进制码在 *SCLK* 作用下,从 *SDI* 端依次进入移位寄存器,并转换成并行数据保存在内部的数据寄存器中;在这些二进制码的控制下,选择 4 096 个开关之一闭合,从而将对应的节点电压连接到输出缓冲器,其输出电压与输入数字量之间的关系为

$$v_O = \frac{V_{DD}}{4\,096} \sum_{i=0}^{11} (D_i \times 2^i) \tag{10.1.15}$$

采用这种方案生产的单片集成 D/A 转换器的型号较多,例如 TI 公司的 DAC5311(8

① 这种 3 线接口与 SPI 等标准接口是兼容的。SPI 是串行外设接口(serial peripheral interface)的缩写,是一种高速的,全双工,同步的通信总线,在芯片的引脚上占用四根线,现在越来越多的芯片集成了这种通信协议。有关该接口的详细内容将在后续课程中介绍。

位)、DAC7311/TLV5618A(12 位)等,ADI 公司的 AD5621/AD5681R(12 位)、AD5682R/AD5692R(14 位)、AD5683R/AD5693R(16 位)等。它们通常用在便携式电池供电的产品中,以发挥其低功耗优势。

10.1.5 D/A 转换器的双极性输出方式

1. 具有双极性输出电压的 D/A 转换器

前面介绍的 D/A 转换器均为单极性输出方式,即输出的模拟量全部为正值或者为负值,主要取决于基准电压 V_{REF}的极性;而输入的数字量均为无符号的自然二进制码。

为了转换有符号的数字量,按理说,应该采用双极性的 D/A 转换器。实际上制造商并不专门生产双极性 D/A 转换器,但可以通过改变外部连线,使单极性 D/A 转换器具备双极性转换能力。

有符号的二进制数通常用补码来表示。下面以 4 位二进制补码为例,说明转换的原理。

4 位二进制补码、偏移码与十进制数之间的对应关系如表 10.1.1 中所示。补码的最高位为符号位,**0** 表示正数,**1** 表示负数。表中第 3 列的偏移码是将补码的最高位取反后得到的,它与自然二进制码在形式上相同,但含义不同,偏移码表示的是有符号数,**0** 表示负数,**1** 表示正数。从十进制数的角度来看,将自然二进制码代表的十进制数值减去 8(或者说,偏移负 8),就是偏移码。

表 10.1.1 十进制数、补码与偏移码之间的关系

十进制数	补码	偏移码	电流 i_0
+7	**0111**	**1111**	$(7/16)I$
+6	**0110**	**1110**	$(6/16)I$
+5	**0101**	**1101**	$(5/16)I$
+4	**0100**	**1100**	$(4/16)I$
+3	**0011**	**1011**	$(3/16)I$
+2	**0010**	**1010**	$(2/16)I$
+1	**0001**	**1001**	$(1/16)I$
0	**0000**	**1000**	0
−1	**1111**	**0111**	$-(1/16)I$
−2	**1110**	**0110**	$-(2/16)I$
−3	**1101**	**0101**	$-(3/16)I$
−4	**1100**	**0100**	$-(4/16)I$
−5	**1011**	**0011**	$-(5/16)I$
−6	**1010**	**0010**	$-(6/16)I$
−7	**1001**	**0001**	$-(7/16)I$
−8	**1000**	**0000**	$-(8/16)I$

如果将偏移码看成无符号的二进制码送给单极性 D/A 转换器作为输入，这时转换得到的模拟量并不是偏移码对应的输出，再减去 8 所对应的模拟量才是真正的输出。

按照以上思路实现双极性 D/A 转换的电路如图 10.1.11 所示，图中各电流均为实际方向。双极性二进制补码的最高位 D_3 经过非门 G 取反后变成了偏移码，送到单极性 D/A 转换器，D/A 转换的输出 i_O 减去 8 所对应的模拟量 I_B 后，就是真正的输出，即

$$i_O = i_\Sigma - I_B \tag{10.1.16}$$

式中，I_B 的大小应该设计为单极性 D/A 转换器输入数码 **1000**（即十进制数 8）时对应的电流，即 $I_B = \frac{I}{2}$，其中 $I = V_{REF}/R$（见式(10.1.4)）。通过选择合适的 $-V_B$ 和 R_B，可以获得 I_B。

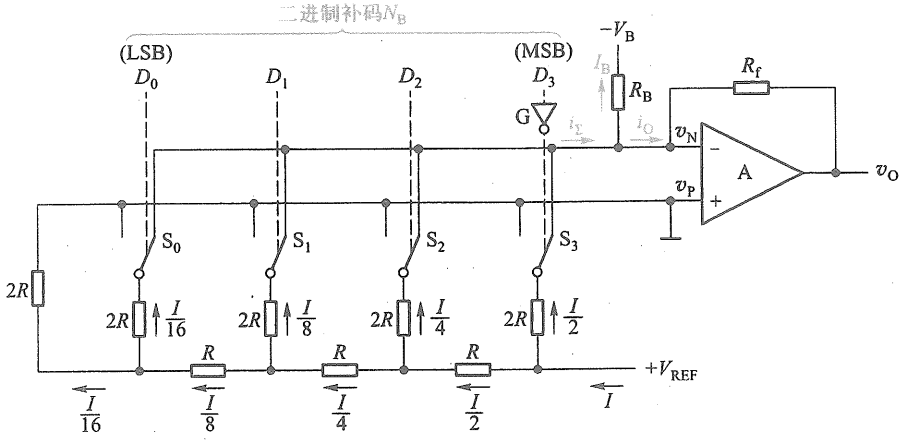


图 10.1.11 具有双极性输出电压的 D/A 转换器

根据式(10.1.5)，得到

$$\begin{aligned} i_\Sigma &= \frac{I}{2}D_3 + \frac{I}{4}D_2 + \frac{I}{8}D_1 + \frac{I}{16}D_0 \\ &= \frac{I}{2}(1-D_3) + \frac{I}{4}D_2 + \frac{I}{8}D_1 + \frac{I}{16}D_0 \end{aligned} \tag{10.1.17}$$

将 $I_B = \frac{I}{2}$ 和式(10.1.17)代入到式(10.1.16)，得到

$$i_O = \frac{I}{2^4}(-2^3D_3 + 2^2D_2 + 2^1D_1 + 2^0D_0) \tag{10.1.18}$$

将 4 位二进制补码分别代入式(10.1.18)，可以得到其对应的输出电流 i_O，如表 10.1.1 中最右边一列所示。可见，电路实现了双极性电流输出。

电流 i_O 经过运放构成的电流-电压变换电路，就能转换成双极性电压输出。

上面 4 位二进制码很容易扩展到 n 位，此时，在运放反相端引入的偏移电流 I_B 的大小仍然为 $\frac{I}{2}$，即

$$\frac{|V_B|}{R_B} = \frac{I}{2} = \frac{|V_{REF}|}{2R} \tag{10.1.19}$$

这种将单极性 D/A 转换器变换成双极性输出的方法十分简便，但极性相反的两个电压源 V_{REF} 和 V_B 的大小必须相等。为了保证精度，有些集成 D/A 转换器内部已经设置了偏流电阻 R_B，并提供内部基准电压源输出，用户只需要适当地连接其引脚，就可以将单极性 D/A 转换器转变成为双极性 D/A 转换器。例如 ADI 公司的 AD7845 和 LTC1821。

2. 用单极性 D/A 转换器实现双极性输出

使用单极性 D/A 转换器得到双极性电压输出的另一种电路如图 10.1.12 所示。电路的输入 N_B 为补码，将 N_B 的最高位取反，得到 4 位二进制偏移码，送到单极性 D/A 转换器，在运放 A_1 的输出端得到电压 v_{O1}，其大小为

$$v_{O1}=-\frac{V_{REF}}{2^4}(N_B+2^3) \tag{10.1.20}$$

式中，N_B 为补码，(N_B+2^3) 为偏移码。

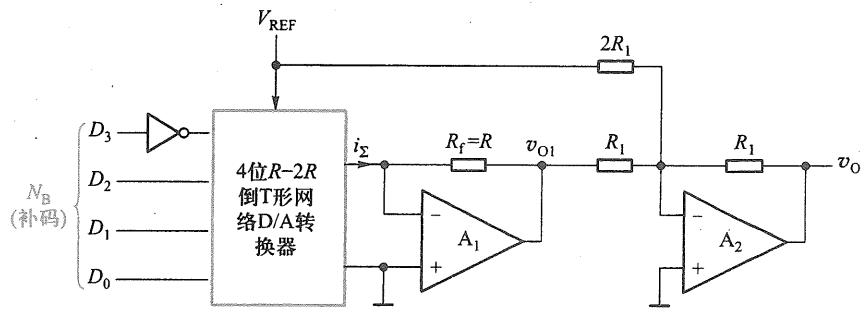


图 10.1.12 具有双极性输出电压的 D/A 转换器电路

由于单极性 D/A 转换器将送入的偏移码当成自然二进制码进行了转换，所以必须在输出 v_{O1} 中扣除十进制数 8 所对应的输出电压 $V_{REF}/2$，于是增加 A_2 构成的反相加法器。最后，A_2 的输出电压为

$$v_O=-\left(\frac{R_1}{R_1}v_{O1}+\frac{R_1}{2R_1}V_{REF}\right)=-\left(v_{O1}+\frac{V_{REF}}{2}\right)$$

将式(10.1.20)代入上式，得到

$$v_O=\frac{V_{REF}}{2^4}\cdot N_B \tag{10.1.21}$$

可见，该电路能将输入的二进制补码 N_B，变成双极性电压输出。

10.1.6 D/A 转换器的主要技术指标

D/A 转换器的技术指标反映了它的特性，也是选用的重要依据，D/A 转换器技术指标有很多，下面仅介绍分辨率、转换精度和工作速度等主要参数。

1. 分辨率

分辨率用来表征 D/A 转换器对输入量微小变化的敏感程度。理论上，它定义为 D/A 转换器最小输出电压 V_{LSB} 与最大输出电压 V_{max} 的比值。n 位 D/A 转换器的分辨率表示为

$$\text{分辨率}=\frac{V_{LSB}}{V_{max}}=\frac{1}{2^n-1} \tag{10.1.22}$$

式中，V_{LSB}是指输入数码仅最低有效位为 1，其余各位均为 0 时，所对应的输出电压值。V_{max}是指输入数码全为 1 时，所对应的输出电压值，有时也称为满刻度(full scale range)输出电压 V_{FSR}。

对于电流输出的 D/A 转换器来说，则有最小输出电流 I_{LSB}和满刻度输出电流 I_{FSR}两个概念。有时，把 V_{LSB}和 I_{LSB}简写为 LSB，把 V_{FSR}和 I_{FSR}简写为 FSR。

分辨率只与位数有关，与基准电压的大小无关，故制造商常用 D/A 转换器的位数表示分辨率。显然，D/A 转换器的位数越多，输出模拟量可能被分离的等级数越多，分辨率越高。

例 10.1.3 已知 10 位 D/A 转换器满刻度输出电压 $V_{max}=10$ V。

(1) 求输入最低位 D_0对应的输出电压增量 V_{LSB}。

(2) 如要求分辨的最小电压为 5 mV($V_{LSB}=5$ mV)，试问至少应选用多少位的 D/A 转换器？

解：(1) 根据式(10.1.22)可知，$\text{分辨率}=\dfrac{1}{2^n-1}=\dfrac{V_{LSB}}{V_{max}}$

将 $n=10$，$V_{max}=10$ V 代入上式得

$$V_{LSB}=\frac{1}{2^n-1}\times V_{max}=\frac{1}{2^{10}-1}\times 10\text{ V}\approx 0.01\text{ V}$$

(2) 将 $V_{LSB}=5$ mV，$V_{max}=10$ V 代入式(10.1.22)中可得

$$\frac{1}{2^n-1}=\frac{0.005\text{ V}}{10\text{ V}}\qquad \text{由此得 } n\approx 11$$

由以上分析可知，在题意给定条件下应选择 11 位 D/A 转换器。

2. 转换精度[①]

转换精度也称为转换误差，它是指 D/A 转换器实际转换特性与理想转换特性曲线之间的最大偏差。转换误差是由各种因素引起的一个综合性指标，根据产生原因的不同，可以分为静态误差和动态误差。静态误差包括失调误差、增益误差和非线性误差等，它们会影响直流信号转换期间转换器的精度；而动态误差通常用于说明转换器的动态性能(如建立时间、转换速率和毛刺脉冲等)，将在后面介绍。

这里主要介绍静态误差。这些误差通常以 LSB 为单位来表示，或以转换器满刻度的百分比(%FSR)表示。

(1) 失调误差

当输入数字量为全 0 时，其模拟量输出的实际值与起始零点之间的偏差量称为失调误差，又称为零点偏移误差。图 10.1.13 说明了这种误差。

产生这一误差的主要原因是运放的输出失调电压(零点漂移)，与输入的数字量无关。运放的零漂会使输出电压的转移特性曲线向上或者向下发生平移，典型的误差应 ≤

① 参考：瞿安连. 电子电路——分析与设计[M]. 武汉：华中科技大学出版社，2010.

(1/2)LSB。

对于外接运放的 D/A 转换器,在一定的温度条件下可以通过调零措施消除初始的零点误差。片内设置放大器的 D/A 转换器则主要取决于芯片的质量。

(2) 增益误差

在修正了失调误差之后,实际转换特性曲线的斜率与理想特性曲线斜率之间的偏差称为增益误差,又称为比例系数误差,如图 10.1.14 所示。

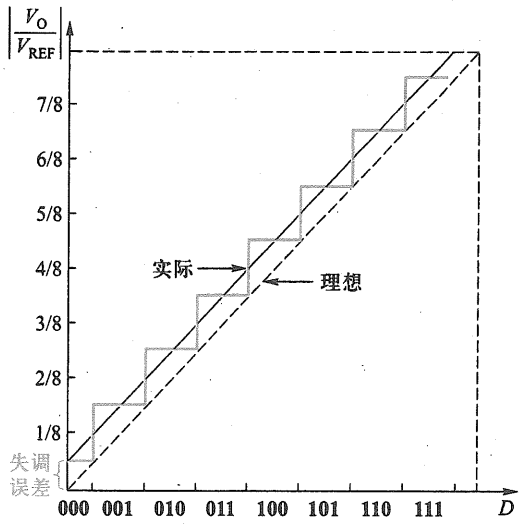


图 10.1.13 3 位 D/A 转换器的失调误差

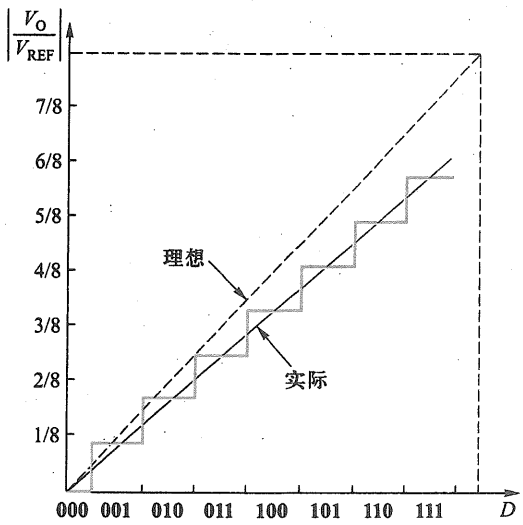


图 10.1.14 3 位 D/A 转换器的增益误差

产生这一误差的主要原因是基准电压 V_{REF} 偏离标准值或者运放闭环增益偏离设计值造成的。所以,增益误差值不是常数,而是与输入数码的值成正比,输入数码的各位都为 1 时,误差最大。

(3) 级差非线性误差

一个理想 D/A 转换器输出模拟电压的步长为 V_{LSB}(或 1 LSB),但 D/A 转换器的实际步长会有偏差,通常用级差非线性(differential nonlinearity,DNL[①])误差来度量,这一误差是在修正了失调误差和增益误差之后测量的。

级差非线性是指任何两个相邻数字量所对应的模拟量的实际差值和理想步长之间的相对偏差。假设输入数字量 k 时,输出模拟量的变化用 Δ_k 表示。换言之,就是数字量从 $k-1$ 变为 k 时,输出模拟量的实际差值为 Δ_k,上述定义可以表达为

$$DNL_k=\frac{\Delta_k}{V_{LSB}}-1 \tag{10.1.23}$$

通常,DNL 是指 $|DNL_k|$ 的最大值。为简单起见,有时用最大绝对偏差表示,即 $|\Delta_k-V_{LSB}|_{max}$,如图 10.1.15 所示。一般要求 DNL 在±0.5 LSB 范围内,以保证转换的单调性。

① 有些著作将 differential nonlinearity 直译为“微分非线性”,其含义不太清楚。实际上这种误差是指每一个量化台阶的误差,而 differential 原意为“有差异的”,所以我们认为翻译为“级差非线性”更合英文原意。

如果 $DNL>1$ LSB 时，则转换特性曲线就会变成非单调的，即输出 v_O 会随着输入数字量的增加而减小，而不是增大。

（4）整体非线性误差

整体非线性（integral nonlinearity，INL[①]）是指 D/A 转换器实际的转换特性曲线与两个端点连成的直线之间的最大偏差值。如图 10.1.16 所示。这种误差也是修正了失调和增益误差之后的最大偏差。整体非线性反映了 DNL 的累积情况，可用 $INL_k=\sum_{j=0}^{k} DNL_j$ 表达。

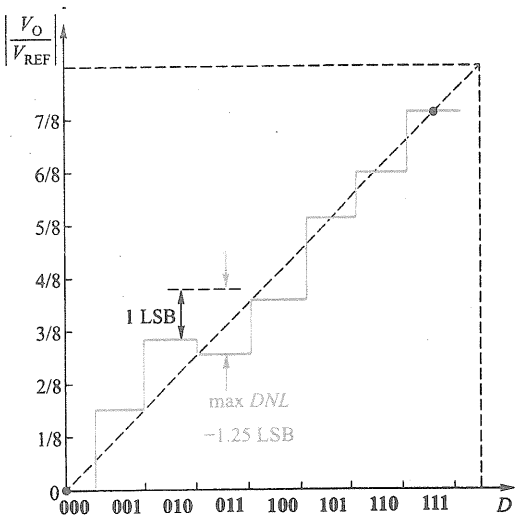


图 10.1.15 级差非线性（DNL）误差

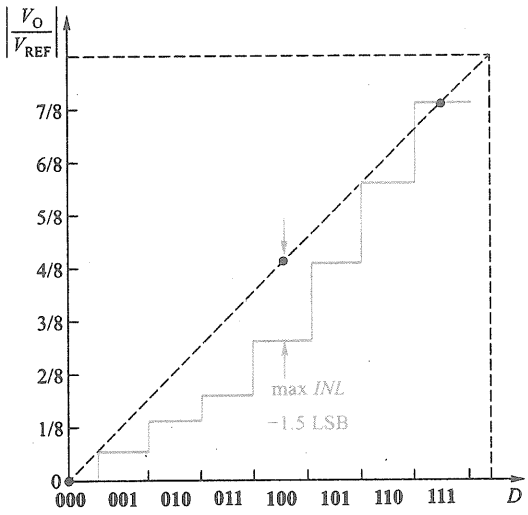


图 10.1.16 整体非线性（INL）误差

3. 转换速度

当 D/A 转换器输入的数字量发生变化时，输出的模拟量并不能立即达到所对应的量值，它要延迟一段时间。通常用稳定时间[②]和转换速率两个参数来描述 D/A 转换器的转换速度。

（1）稳定时间

指输入数字量变化时，输出稳定在规定的误差范围所需的时间。一般用 D/A 转换器输入的数字量 N_B 从全 0 变为全 1 时，输出电压（或电流）进入与稳态值相差（±LSB/2）范围以内所需的时间表示，用 t_{set} 表示，如图 10.1.17 所示。t_{set} 的大小与分辨率、电路结构和制造工艺有关，其典型值范围是 10 ns～15 μs。一般来说，电流输出型 D/A 转换器的稳定时间比电压输出型 D/A 转换器的稳定时间短，造成这种差别的原因是电流电压转换电路中的运放的响应时间。

---

① 有些著作将 integral nonlinearity 直译为“积分非线性”。integral 原意为“完整的、整个的”，所以我们认为翻译为“整体非线性”更合英文原意。

② 是 settling-time 中文翻译，也译为建立时间。

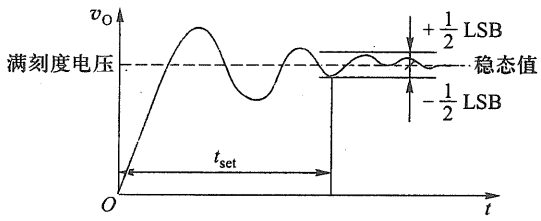


图 10.1.17 D/A 转换器的建立时间

对于高速 D/A 转换器，常用输出更新速率[①]来衡量其转换速度，它和 $1/t_{set}$ 等价。

（2）转换速率

指大信号工作状态下，模拟输出电压的最大变化率。通常以 V/μs 为单位表示。该参数与运放的转换速率 *SR* 类似。

4. 温度系数

这是指在输入不变的情况下，输出模拟电压随温度变化产生的变化量。一般用满刻度输出条件下温度每升高 1 ℃，输出电压变化的百分数作为温度系数。

除了上面介绍的几个指标外，还有输入逻辑电平、单调性、信噪比、无杂散动态范围等参数，这里不再深入讨论。

10.1.7 D/A 转换器的应用

在实践中，D/A 转换器应用很广，它不仅作为数字系统和模拟系统之间的接口电路（如作为微机系统的接口电路），而且还可用于数字量对模拟信号进行处理。下面我们以波形产生电路和数字式可编程增益放大电路为例说明它的应用。

（1）脉冲波产生电路

由 CB7520、运算放大器及 4 位同步二进制计数器 74LVC161 组成的波形产生电路如图 10.1.18(a)所示。图中 74LVC161 采用反馈置数法，组成模 10 计数器，D/A 转换器的高位 $D_4 \sim D_9$ 均为 **0**，低 4 位输入是计数器的输出。在 *CP* 作用下，$Q_3Q_2Q_1Q_0$ 输出分别为 **0000～1001**。

根据式(10.1.7)计算输出电压的值，画出 v_0 的波形如图 10.1.18(b)所示，输出波形是有 10 个阶梯的阶梯波。如改变计数器的模，则改变波形的阶梯数；如采用可逆计数器，经滤波后，在电路的输出端可得到三角波输出。

（2）数字式可编程增益放大电路

数字式可编程增益放大电路如图 10.1.19 所示，CB7520 与运放接成普通的反相放大电路形式。电路中 CB7520 内部的反馈电阻 *R* 为反相放大电路的输入电阻，而由数字量控制的倒 T 形电阻网络是它的反馈电阻。当输入数字量变化时，倒 T 形电阻网络的等效电阻随之改变。这样，在输入电阻一定的情况下，随着电阻网络的等效电阻变化，反相放大电路的增益也就随之改变。

---

① 是 output update rate 中文翻译，其单位常用 MSPS(mega-samples per second，兆次/秒，或 MHz)。

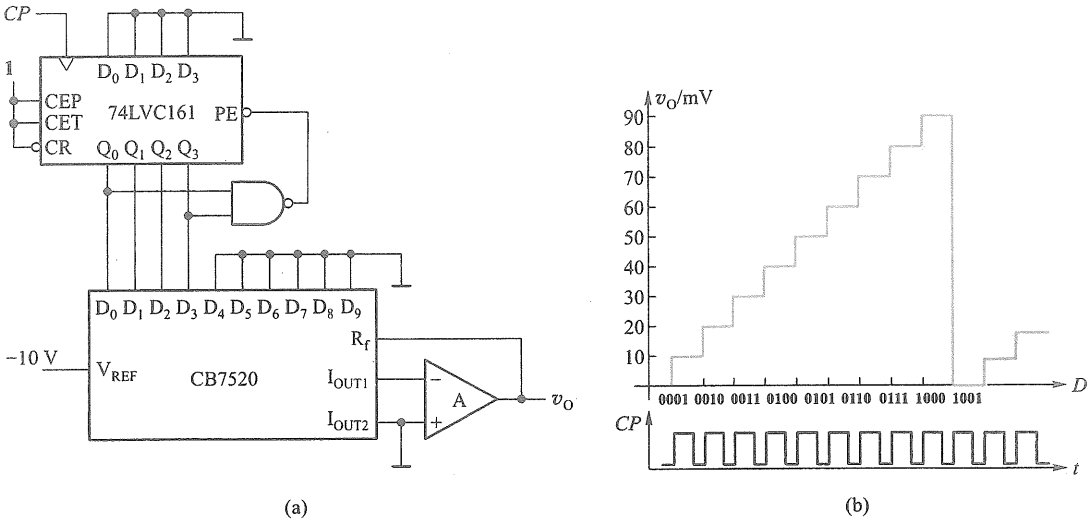


图 10.1.18 波形产生电路

(a) 电路 (b) 工作波形

根据运放虚地原理,可以得到

$$\frac{v_I}{R}=\frac{-v_O}{2^{10}R}(D_0 2^0+D_1 2^1+\cdots+D_9 2^9)$$

所以

$$A_v=\frac{v_O}{v_I}=\frac{-2^{10}}{D_0 2^0+D_1 2^1+\cdots+D_9 2^9}$$

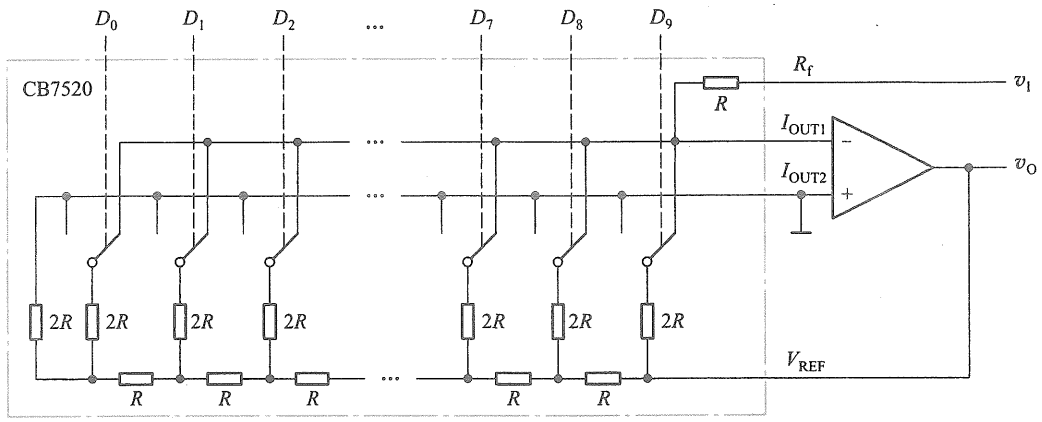


图 10.1.19 数字式可编程增益放大电路

如将 CB7520 芯片中的反馈电阻作为反相运放的反馈电阻,数控 CB7520 的倒 T 形电阻网络连接成运放的输入电阻,读者不难推断出电路为数字式可编程衰减器。

复习思考题

10.1.1 为使倒 T 形电阻网络 D/A 转换器有足够的精度,在电路器件及参数选择上应

注意些什么问题?

10.1.2 已知某 D/A 转换器满刻度输出电压为 10 V,试问如要求分辨的最小电压是 1 mV,应采用多少位的 D/A 转换器?

10.1.3 已知一个 8 位 D/A 转换器满刻度输出电压为 5 V,试问它的最小输出电压 V_{LSB} 是多少?其分辨率是多少(用百分比表示)?

10.1.4 当一个 D/A 转换器的 V_{REF} 增加 20%时,分辨率和最大输出电压值 V_{max} 将会发生什么变化?

10.2 A/D 转换器

A/D 转换器要将时间和幅值都连续的模拟量,转换为时间、幅值都离散的数字量,一般要经过取样、保持、量化和编码这 4 个步骤。在实际电路中,这些步骤有些是合并进行的,例如,取样和保持、量化和编码往往都是在转换过程中同时完成的。

10.2.1 A/D 转换的一般工作过程

1. 取样与保持

取样就是按照一定的时间间隔抽取模拟量的值,将连续变化的模拟量转换为时间上离散的模拟量。取样过程示意图如图 10.2.1 所示。图 10.2.1(a)中,取样信号 $S(t)$ 控制取样过程,$S(t)$ 高电平期间,模拟开关导通,输出信号 $v_O(t)$ 等于输入信号 $v_I(t)$,而在 $S(t)$ 的低电平期间,开关关闭,输出信号 $v_O(t)=0$。电路工作波形如图 10.2.1(b)所示,取样的结果 $v_O(t)$ 是一系列窄脉冲,其顶部与原模拟信号波形相同,幅值仍然是连续的模拟量,称为样本,但时间上已经离散了。

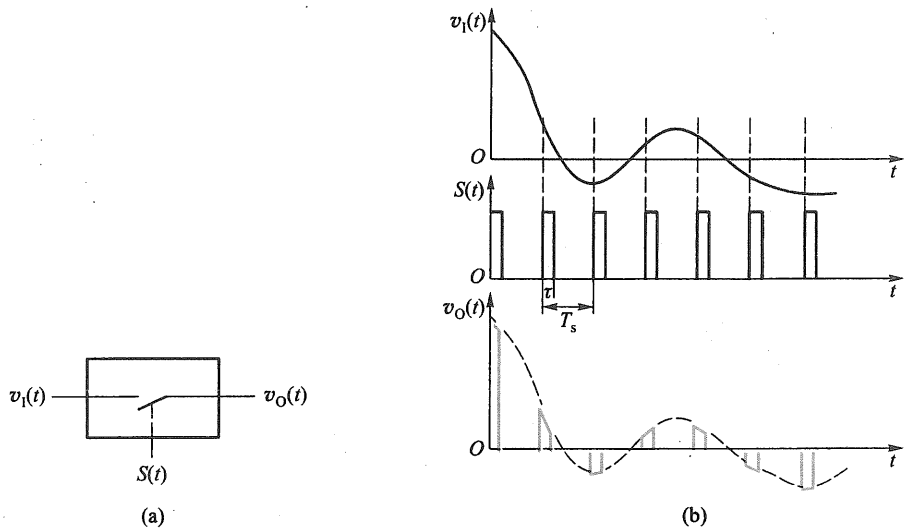


图 10.2.1 取样过程

(a) 取样电路示意图 (b) 各信号波形图

如何选择取样信号 $S(t)$ 的频率 f_s，才能不失真地复现原来的输入信号呢？取样频率 f_s 越高，样值就越多，越能真实地复现输入信号。但得到的数据量多，对后续数据存储和处理的要求就会变高。因此，不宜过度提高取样频率，实际上合理的取样频率由取样定理确定。

取样定理：设取样信号 $S(t)$ 的频率为 f_s，输入模拟信号 $v_I(t)$ 的最高频率分量的频率为 f_{max}[①]，则 f_s 与 f_{max} 必须满足下面的关系

$$f_s \geqslant 2f_{max} \tag{10.2.1}$$

取样定理说明，为了能够不失真地恢复原来的输入信号，取样频率 f_s 必须不小于输入模拟信号频谱中最高频率 f_{max} 的两倍。或者说，在满足式(10.2.1)的条件下，取样得到的信号经低通滤波器后，才能重建原输入信号。例如，话音信号的 $f_{max} \approx 3.4$ kHz，一般取 $f_s = 8$ kHz。再如，音频信号的最高频率为 20 kHz，在数字音频系统中，常用的取样频率是 44.1 kHz 和 48 kHz，其中 48 kHz 最常见，而 44.1 kHz 常用于音频 CD 中。在实际应用中，如果对信号质量要求较高，则取样频率可以高一些，取 $f_s = (3 \sim 5) f_{i(max)}$ 通常能满足要求。

将取样所得信号送入 A/D 转换电路转换成数字信号时，往往需要一定的时间，这样就需要保证转换期间取样点的信号值不变，因此取样以后还需要保持。取样与保持过程一般都是同时完成的，常常将取样-保持电路集成在一个芯片(常见的有 LF198/298/398、AD1154、MAX1565 等)中。取样-保持电路的原理图及输出波形分别如图 10.2.2(a)、(b)所示。取样-保持电路由输入放大器 A_1、输出放大器 A_2，保持电容 C_H 和开关驱动电路组成。电路中要求 $A_{V1} \cdot A_{V2} = 1$，且 A_1 具有较高的输入阻抗，以减小对信号源的影响。A_2 选用有较高输入阻抗和低输出阻抗的运放，这样不仅 C_H 上所存电荷不易泄漏，而且电路还具有较高的带负载能力。

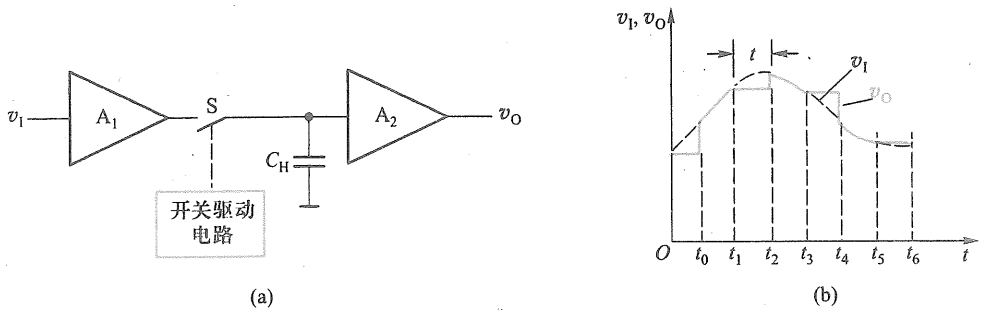


图 10.2.2 取样-保持电路
(a) 原理图 (b) 波形图

$t_0 \sim t_1$ 时段，开关 S 闭合，电路处于取样阶段，电容器 C_H 充电，由于 $A_{V1} \cdot A_{V2} = 1$，因此 $v_O = v_I$；$t_1 \sim t_2$ 时段为保持阶段，在此期间 S 断开，若 A_2 的输入阻抗足够大，且 S 为较理想的开关，可认为 C_H 几乎没有放电回路，输出电压保持 v_O 不变，使输出脉冲波形保持平顶。

2. 量化与编码

以上分析结果表明，经取样和保持电路后的输出信号，在数值上还是连续变化的模拟

① $f_s = 2f_{max}$ 也称为奈奎斯特频率(Nyquist frequency)。

器、寄存器及代码转换器等几个部分组成。输入电压 v_I 是经过取样、保持电路后得到的，其变化范围是 0～V_{REF}，输出为 3 位二进制码 $D_2D_1D_0$。

这里，采用图 10.2.3(b)所示的量化方法，用电阻分压器将基准电压分为(1/15) V_{REF}、(3/15) V_{REF}、…、(13/15) V_{REF} 这 7 个不同的电压值，其量化单位为 $\Delta=(2/15)$ V。再将这 7 个电压值分别加到比较器 C_1～C_7 的反相输入端，作为比较基准。然后，输入电压 v_I 同时加到每个比较器的同相输入端，与这 7 个比较基准进行比较，实现对输入电压 v_I 幅值的量化。

对于任一比较器，如果输入电压高于它的比较基准，则其输出为高电平；否则，输出为低电平。因此，输入电压 v_I 的大小决定着各比较器的输出状态，例如，当 $0\leqslant v_I<(1/15)V_{REF}$ 时，C_7～C_1 的输出状态都为 0；当 $(3/15)V_{REF}\leqslant v_I<(5/15)V_{REF}$ 时，比较器 C_1 和 C_2 的输出 $C_{O1}=C_{O2}=1$，其余各比较器的状态均为 0；……，以此类推。所有比较器的输出形成一种代码①，由 D 触发器进行存储，再经代码转换器后，就得到了数字量输出，如表 10.2.1 所示。

代码转换器是一个多输出组合逻辑电路，读者可以自行分析和设计。

在并行 A/D 转换器中，输入电压 v_I 同时加到所有比较器的输入端，从 v_I 加入，到稳定输出数字量，所经历的时间为比较器、D 触发器和编码器延迟时间的总和。所以，这种并行结构 A/D 转换器的转换时间是最短的，有闪速型 A/D 转换器之称。

表 10.2.1 3 位并行 A/D 转换器输入与输出关系对照表

模拟输入	比较器输出状态							数字输出		
	C_{O7}	C_{O6}	C_{O5}	C_{O4}	C_{O3}	C_{O2}	C_{O1}	D_2	D_1	D_0
$0\leqslant v_I<V_{REF}/15$	0	0	0	0	0	0	0	0	0	0
$V_{REF}/15\leqslant v_I<3\ V_{REF}/15$	0	0	0	0	0	0	1	0	0	1
$3\ V_{REF}/15\leqslant v_I<5\ V_{REF}/15$	0	0	0	0	0	1	1	0	1	0
$5\ V_{REF}/15\leqslant v_I<7\ V_{REF}/15$	0	0	0	0	1	1	1	0	1	1
$7\ V_{REF}/15\leqslant v_I<9\ V_{REF}/15$	0	0	0	1	1	1	1	1	0	0
$9\ V_{REF}/15\leqslant v_I<11\ V_{REF}/15$	0	0	1	1	1	1	1	1	0	1
$11\ V_{REF}/15\leqslant v_I<13\ V_{REF}/15$	0	1	1	1	1	1	1	1	1	0
$13\ V_{REF}/15\leqslant v_I<V_{REF}$	1	1	1	1	1	1	1	1	1	1

并行比较型 A/D 转换器的缺点是元器件数目太多，一个 n 位转换器，分压电阻要用 2^n 个、比较器和触发器各自需要 2^n-1 个。另外，为了获得较高的转换速度，各个比较器必须工作在较高的工作电压下，这样使并行比较型 A/D 转换器的功耗较大，因此，用集成工艺制造输出位数很多的并行比较型 A/D 转换器是非常困难的。常见的并行比较型 A/D 转换器输出数据一般为 6～8 位。MAX1003 是使用这种方案的一个例子，它在一个芯片内部包含两个 6 位的 A/D 转换器，取样率高达 90 MSPS，转换结果在取样后一个时钟周期就可以使用了。

① 这种代码称为温度计码(thermometer code)，因为它看起来非常类似于一个水银温度计中的汞柱。

例 10.2.1 在图 10.2.4 中，已知 $V_{REF}=6$ V，输入模拟电压 $v_I=3.4$ V，试确定 3 位并行比较型 A/D 转换器的输出数码。

解： 根据并行比较型 A/D 转换器的工作原理可知，输入到比较器 $C_1 \sim C_7$ 的基准电压分别为 $(1/15)V_{REF} \sim (13/15)V_{REF}$。将 $V_{REF}=6$ V 代入，求得图 10.2.4 中 $(7/15)V_{REF}=2.8$ V、$(9/15)V_{REF}=3.6$ V。

输入模拟电压 $v_I=3.4$ V，即 $(7/15)V_{REF}<v_I<(9/15)V_{REF}$，对照表 10.2.1，可知输出数码为 **100**。

* 2. 半闪速型 A/D 转换器

为了解决并行比较型 A/D 转换器输出位数增加与电路规模急剧增长的矛盾，可以采用其他不同的解决方案，常用的有“半闪速(half flash)[①]型”“流水线闪速(pipelined flash)型”“折叠/插值(folding/interpolating architecture)型”等。

8 位半闪速型 A/D 转换器框图如图 10.2.5 所示，其中 v_I 是取样-保持电路输出。电路由两个 4 位并行比较型 A/D 转换器、一个 4 位 D/A 转换器、减法器和放大器等组成。ADC_1 将输入的模拟电压转换成输出数字量的高 4 位。为了得到剩余的低 4 位数字量，通过 D/A 转换器重新将高 4 位数字量转换成模拟电压 v_O，再用减法器求得 v_I-v_O，这是 ADC_1 转换后留下来的残差电压，其值较小，通过残差放大器[②]将此残差电压放大 16 倍至 ADC_2 输入的满量程，得到 v_q，经过 ADC_2 转换成数字量的低 4 位，最后，由锁存器和三态缓冲电路同时输出 8 位数字量。

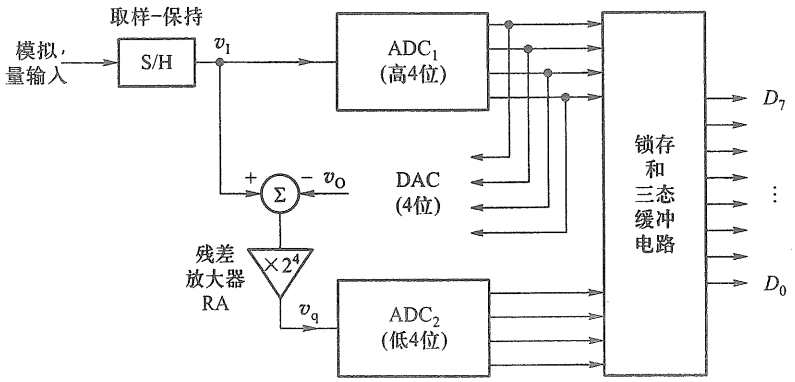


图 10.2.5 8 位半闪速型 A/D 转换器框图

ADC_1 和 ADC_2 各自需要 15 个比较器，加起来总计 30 个比较器，这比 8 位全并行 A/D 转换器所需要的 255 个比较器少得多，但又增加了 D/A 转换器和减法电路等，不过元器件数量还是比全并行 A/D 转换器要少得多。

半闪速型 A/D 转换器的输出数据要经过两级 A/D 转换器串行工作得到，即有两个时钟周期的等待时间，所以其转换速度比完全并行的闪速型 A/D 转换器要慢一些，但它比后面将要介绍的逐次比较型 A/D 转换器的速度快得多，所以这种结构用在许多高速 A/D 转

① 有的文献也称为分步闪速(subranging flash)型转换器。

② 放大残差电压的放大器被称为残差放大器(residue amplifier，简称 RA)。

换器产品中。例如，TI 的 TLC0820、TLC5540，ADI 的 AD7829-1 等都采用了这种方案。

* 3. 流水线型 A/D 转换器

流水线型 A/D 转换器框图如图 10.2.6 所示，它是一种多级串联形式的 A/D 转换器。每一级都是一个位数较少的 A/D 转换子模块。图中虚线框内是这个子模块的组成，它是一个输出为 k 位数字量的半闪速型 A/D 转换器，与图 10.2.5 的结构相同。对于不同的子模块，k 的取值可以不同。图中，第一、二级子模块 k 的值分别为 p 和 q，最后一级 k 的值为 r。流水线型转换器的工作原理如下：

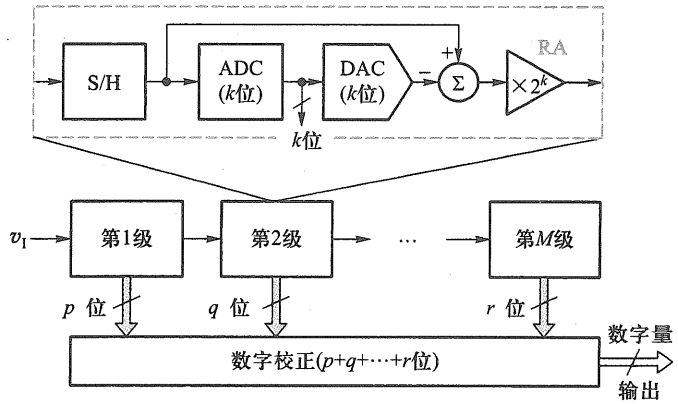


图 10.2.6 流水线型 A/D 转换器框图

第 1 级对 v_I 进行取样，并转换成第一组 p 位数字量，再用 D/A 转换器和减法器得到残差，然后经残差放大器放大后作为流水线中第 2 级的输入。第 2 级与第 1 级一样，重复同样的过程，产生第二组 q 位数字量输出，以此类推，第 M 级子模块得到 r 位数字量输出，最后，经过数字校正等电路的处理，把 M 级子模块输出的数字量组合起来，就能够输出最终的 $(p+q+\cdots+r)$ 位结果。

注意，最后一级子模块已经不再需要产生残差并向下一级传递了，因此，最后一级子模块只需要一个并行比较型 A/D 转换器。

可见，流水线型转换器的转换是一级一级进行的，每个时钟对 v_I 取样一次，得到的每一个样本要经过 M 级子模块才能得到最终的结果，因此，每个样本的转换一定存在着与级数相关的转换延时，称为流水延时。每个样本总的转换时间为各级子模块的转换时间与数字校正电路所需时间的总和。

但是，当流水线正常工作后，每个时钟周期会输出一个转换后的 $(p+q+\cdots+r)$ 结果。这是因为流水线型转换器在第一级转换完成后并将残差送到第二级时，第一级开始接受下一个取样信号，并与第二级同时进行转换，以此类推。各级子模块能够同时并行工作，因而总的转换速率与流水线的级数无关，仅取决于其中一级的速率，通常是第一级。这就像轿车生产总装流水线一样，整条线可以每 2 分钟左右下线一辆车，但每辆车从开始装配到下线却需要好几个小时。

一个 3 级流水线型 A/D 转换器从取样到输出数据的时序图如图 10.2.7 所示。在第 1 个 CP 上升沿取样得到样本 N，经过 3 级流水线处理，到第 4 个 CP 上升沿之后输出样本 N

的转换结果 D_N，输出数据相对于取样时刻延迟了 3 个时钟周期。此后，每个时钟周期会输出一个转换结果。

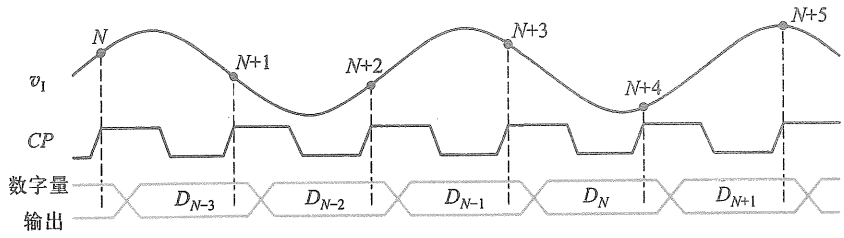


图 10.2.7 流水线型 A/D 转换器的数据延迟

流水线型 A/D 转换器具有较高的精度和较高的速度，其转换精度在 12～16 位，转换速率一般在 1～100 MSPS 范围。并具有功耗低、价格（与并行转换器比较）便宜等优点，广泛适用于数码相机、摄像机和医疗图像设备等分辨率较高的视频信号处理场合。例如，TI 公司的 14 位 A/D 转换器 ADS5424、ADS5474 等都采用了流水线型结构，ADS5424 内部有 3 级流水线，而 ADS5474 内部则有 14 级流水线；ADI 公司的 AD9244 是一个 14 位的 10 级流水线型 A/D 转换器。增加流水线的级数，可以提高转换器的转换精度，但流水延时也会随之增加。

10.2.3 逐次比较型 A/D 转换器

1. 转换原理

逐次比较型 A/D 转换器[①]是应用较多的一种转换器。这种转换器的转换过程与用天平称物体的重量相似。假设所用天平的砝码一个比另一个重量少一半，其称重规则是，从最重的砝码开始试放，与被称物重进行比较，若物体重于砝码，则保留该砝码，否则移去。再加一个次重的砝码，判断砝码的重量是否大于物重，决定第 2 个砝码的去留，照此方法一直加到最小一个砝码为止。最后，将所有留下的砝码重量相加，就得到物体重量。

逐次比较 A/D 转换技术，就是取一个数字量加到 D/A 转换器上，于是得到一个对应的模拟电压 v_o，与待转换的模拟量 v_I 进行比较。通常所取数字量从高位到低位逐位比较，决定该位是留 1 还是留 0，当所有位都尝试后，就能找到与输入电压 v_I 最接近（误差在 ±1/2 LSB 之内）的数字代码。

图 10.2.8 是逐次比较型 A/D 转换器的框图。它包括 D/A 转换器、逐次逼近寄存器（successive approximation register，简称 SAR）和控制逻辑电路、电压比较器和时钟源。为保证转换期间保持信号不变，转换前须有取样-保持电路。

逐次比较型 A/D 转换器通常有一个开始转换的启动信号（START），以及一个转换结束信号（EOC[②]）。对于不同的芯片，这两个信号的极性和名称可能会有所不同，但基本概念是相同的。一个典型的转换时序如图 10.2.9 所示。

---

① 从得到结果的过程来说，这种转换器又被称为逐次逼近型 A/D 转换器（简称 SAR ADC）。

② EOC 是 end-of-convert 的缩写。

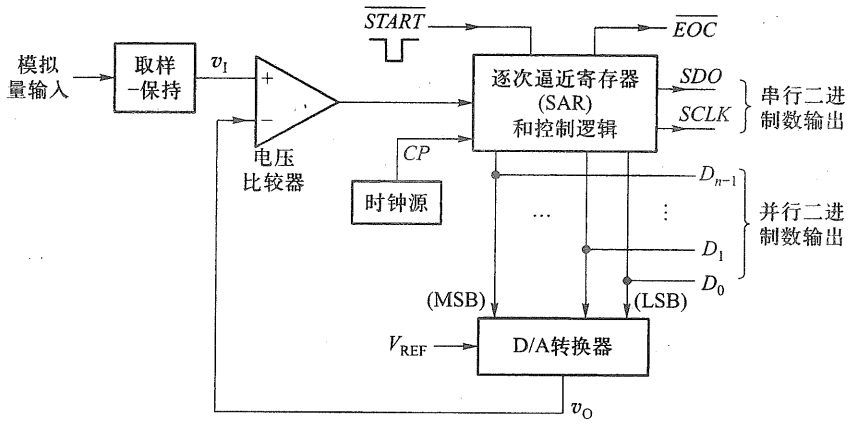


图 10.2.8 逐次比较型 A/D 转换器框图

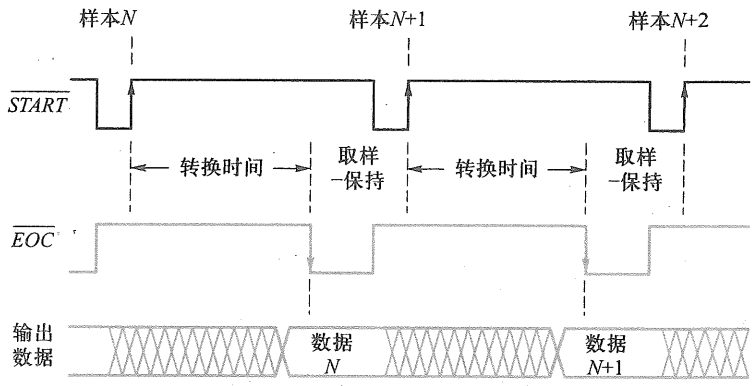


图 10.2.9 典型的逐次比较型 A/D 转换器时序图

在启动信号($\overline{START}$)为低电平时,先将寄存器(SAR)清零,加到 D/A 转换器的数字量为全 **0**。在$\overline{START}$从 **0** 跳变为 **1** 时,开始转换,在转换期间,取样-保持放大电路工作在保持模式,即 v_I保持不变。在时钟信号的作用下,首先将 SAR 的最高位(MSB)D_{n-1}置为 **1**,即将二进制码 **10…0** 送入 D/A 转换器,根据式(10.1.7)可知,$v_O = V_{REF}/2$;v_O与输入电压 v_I比较。如果 $v_I > v_O$,说明数字量不够大,则 D_{n-1}的这个 **1** 应该保留;如果 $v_I < v_O$,说明数字量太大了,这个 **1** 应该去掉(变为 **0**)。接着,按同样的方法将次高位置 **1**,并再次比较 v_O和 v_I,以决定这一位的 **1** 是否应该保留。以此类推,直到完成最低位(LSB)D_0的比较。在转换期间,信号$\overline{EOC}$一直为高电平,输出数据无效,在$\overline{EOC}$下降沿到来时,输出数据才是有效的。

转换后数字量可以并行输出,也可以串行输出,或者两种方式同时输出。目前,逐次比较型 A/D 转换器的采样频率达到兆赫兹,分辨率可达到 18 位,有些芯片是多通道数据采集系统。例如,ADI 公司的 AD7641、AD7328/7329、AD7625/7626 等。

2. 转换电路

图 10.2.10 是一种 4 位逐次比较型 A/D 转换器的逻辑电路。图中,移位寄存器具有同步置数和移位功能,F 为其并行置数控制端,低电平有效,S 为串行数据输入端。5 个 D 触发器组成数据寄存器,输出数字量为 $D_3 \sim D_0 = Q_4 \sim Q_1$。

在启动脉冲下降沿，将 $FF_0 \sim FF_3$ 清零，同时将 FF_5 的输出 Q_5 置 1，时钟脉冲 CP 通过门 G_1 进入移位寄存器。

在启动脉冲为低电平期间，移位寄存器的置数控制端 $F=0$，在第一个 CP 脉冲作用下，移位寄存器将并行数据输入端（E、D、C、B、A）的数据存入寄存器，于是 $Q_E Q_D Q_C Q_B Q_A = 11110$。$Q_A$ 的低电平 FF_4 最高位 Q_4 置 1，其余触发器保持 0 状态，即 $Q_4 Q_3 Q_2 Q_1 = 1000$。该数字量加到 D/A 转换器的输入端上，在其输出端得到相应的模拟电压 $v_O' = V_{REF}/2$；v_O' 送入比较器与输入电压 v_I 比较，若 $v_I > v_O'$，则比较器输出 v_C 为 1，否则为 0。比较结果送到 $FF_1 \sim FF_4$ 的数据输入端 $D_1 \sim D_4$。

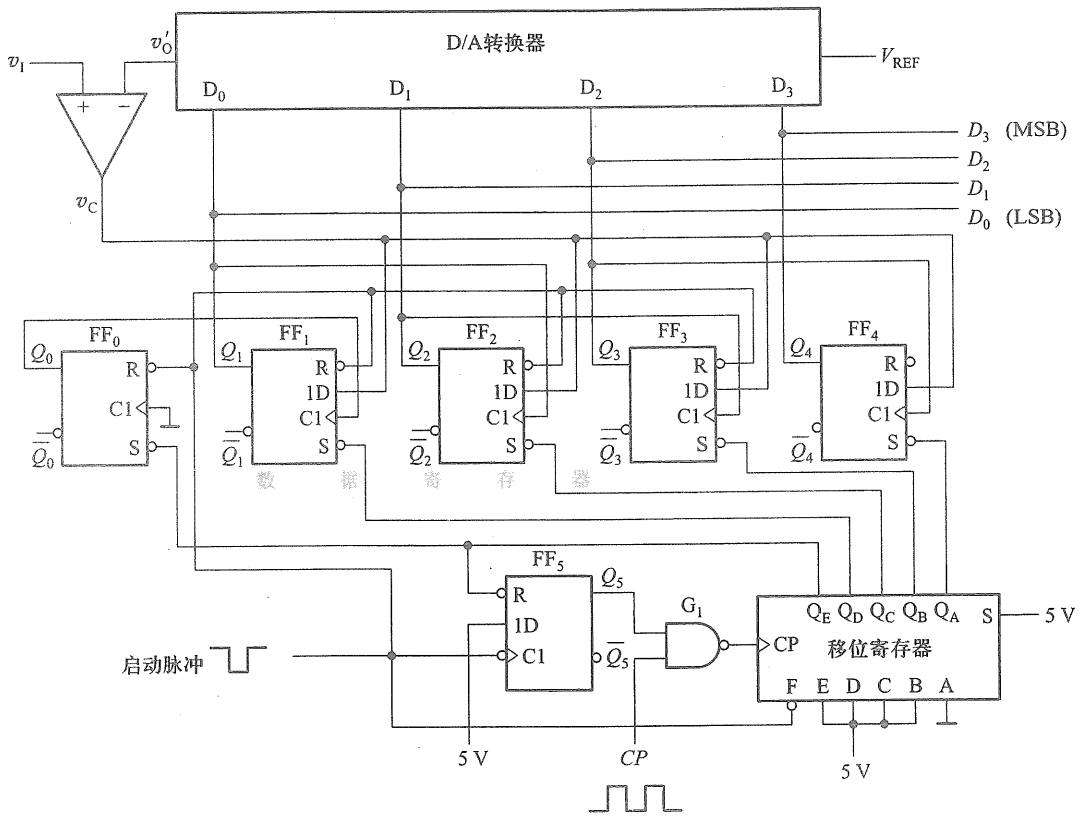


图 10.2.10 4 位逐次比较型 A/D 转换器的逻辑电路

第二个 CP 脉冲到来后（此前启动脉冲已变为高电平），串行数据 S 进入移位寄存器，$Q_A = S = 1$，同时原来移位寄存器的数据向左移动一位，于是 $Q_E Q_D Q_C Q_B Q_A = 11101$。由于 $Q_B = 0$，使触发器 FF_3 的输出 Q_3 由 0 变 1，这个正跳变作为 FF_4 的 CP 信号，使第一次比较的结果 v_C 保存于 Q_4。此时，由于 $FF_0 \sim FF_2$ 时钟输入端无正跳沿，它们保持原状态不变。Q_3 变 1 后建立了新的 D/A 转换器的数据，若原来的 $v_C = 1$，则 $Q_4 Q_3 Q_2 Q_1 = 1100$；若原来的 $v_C = 0$，则 $Q_4 Q_3 Q_2 Q_1 = 0100$；经过 D/A 转换器后，得到新的 v_O' 再与输入电压 v_I 相比较，比较结果在第三个时钟脉冲作用下存于 Q_3……如此进行，直到 Q_E 由 1 变 0，并作用于 FF_5 的异步清零端，使 Q_5 由 1 变 0 后，门 G_1 将时钟信号 CP 封锁，转换完毕。电路的输出端 $D_3 D_2 D_1 D_0$ 得到与输入

电压 v_I 成正比的数字量。

由以上分析可见，逐次比较型 A/D 转换器完成一次转换所需时间与其位数 n 和时钟脉冲频率有关，位数越少，时钟频率越高，转换所需时间越短。这种 A/D 转换器具有转换速度较快、精度高的特点，在工业控制领域获得了广泛的应用。

10.2.4 双积分型 A/D 转换器

双积分型 A/D 转换器是一种间接转换型 A/D 转换器。它采用对输入模拟电压和基准电压分别进行两次积分，将输入电压平均值变换成与之成正比的时间间隔，然后利用时钟脉冲和计数器测出此时间间隔，进而在输出端得到与模拟量相应的数字量。由于该转换电路是对输入电压的平均值进行变换，所以它具有很强的抗工频干扰能力，在数字测量中得到广泛应用。

双积分型 A/D 转换器的原理电路如图 10.2.11 所示，它由积分器，过零比较器（C），时钟脉冲控制门（G）和计数器（$FF_0 \sim FF_{n-1}$）等几部分组成。其中积分器由运放（A）和 RC 组成，计数器由 n 个 T' 触发器组成异步 n 位二进制递增计数器。触发器 FF_n 的输出 Q_n 控制开关 S_1 的切换。

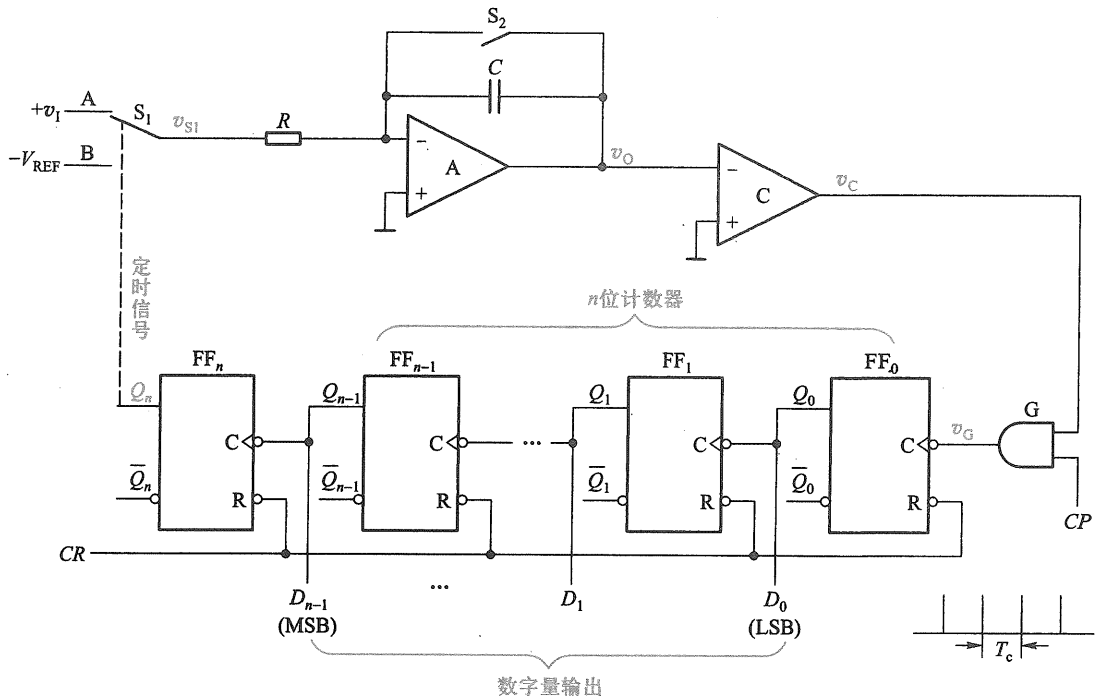


图 10.2.11 双积分型 A/D 转换器的原理电路

下面以输入正极性的直流电压 v_I（$v_I < V_{REF}$）为例，说明电路的基本工作原理。电路的工作过程分为以下几个阶段进行。

（1）准备阶段

转换开始前，CR 信号将计数器清零，开关 S_2 闭合（其控制电路未画出），使积分电容完

全放电完毕。转换开始时，S_2断开，同时 S_1与待转换的输入信号 v_I接通，接着进入下列两次积分阶段。

（2）第一次积分阶段

设 $t=0$ 时，被测电压 v_I加到积分器的输入端，电路进入第一次积分阶段。积分器从初始状态 0 开始对输入电压积分，由于输入电压 v_I是正值，所以积分器的输出电压 v_O从 0 V 开始下降，其波形如图 10.2.12(c)的①段。积分器的输出电压 v_O为

$$v_O=-\frac{1}{\tau}\int_0^t v_I\mathrm{d}t \tag{10.2.2}$$

式中，$\tau=RC$。由于 $v_O<0$，比较器的输出 v_C为高电平，所以，时钟信号可以通过控制门 G，计数器在 CP 作用下从 0 开始递增计数。对于 n 位二进制计数器，从全 **0** 计数到全 **1** 时，需要 (2^n-1)个时钟脉冲。当第 2^n个时钟脉冲到来时，计数器($FF_0\sim FF_{n-1}$)的输出值会变为 **0**，而其进位信号 Q_{n-1}使 $Q_n=1$，开关 S_1由 A 点转接到 B 点，第一次积分结束。假设 CP 信号的周期 T_c为固定值，则第一次积分时间 T_1为

$$T_1=2^nT_c \tag{10.2.3}$$

若用 V_I来表示 T_1期间输入电压 v_I的平均值，将式(10.2.3)代入式(10.2.2)，可得第一次积分结束时积分器的输出电压为

$$v_P=-\frac{T_1}{\tau}V_I=-\frac{2^nT_c}{\tau}V_I \tag{10.2.4}$$

可见，在 T_1一定的条件下，积分器的输出电压 v_P与输入电压的平均值成正比。如果 V_I较小，则 v_P的绝对值也会小一些，如图 10.2.12(c)中斜线①上方的虚线所示。

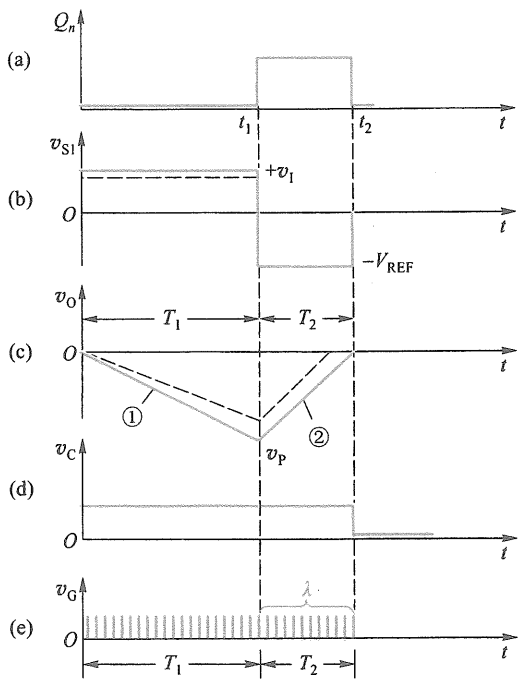


图 10.2.12 双积分型 A/D 转换器的工作波形

教学视频 10.1：双积分式 A/D 转换器

（3）第二次积分阶段

S_1转接到 B 点时（$t=t_1$），开始第二次积分，积分器开始对基准电压$-V_{REF}$进行积分。由于基准电压的极性与输入 v_I相反，所以积分器以 v_p为初始值向反方向积分。在图 10.2.12（c）所示波形中，就是从负值 v_p以固定斜率往正方向回升的斜线②。此时，比较器的输出 v_C仍然为高电平（如图 10.2.12（d）所示），计数器从 0 开始新一轮的计数过程。

当 $t=t_2$时，积分器输出电压 $v_O=0$，比较器的输出 v_C立刻变为 0，时钟脉冲控制门 G 关闭，*CP* 被封锁，计数器停止计数，第二次积分结束。此时计数器 $Q_{n-1}\sim Q_0$的计数值就是本次 A/D 转换得到的数字量，我们用 λ 表示。将计数值 λ 乘以时钟周期 T_C就是第二次积分时间 T_2，即

$$T_2=\lambda T_c \tag{10.2.5}$$

在第二次积分阶段结束后，控制电路又通过 *CR* 信号将计数器清零，同时使开关 S_2闭合，电容 *C* 放电，电路为下一次转换做好准备。

下面，我们来求计数值 λ 与输入电压 v_I之间的关系。

第二次积分阶段从 t_1时刻开始，到 t_2时刻结束，积分的起始电压为 v_P，终止电压 $v_O(t_2)$为 0，所以积分结束时 v_O为

$$v_O(t_2)=v_P-\frac{1}{\tau}\int_{t_1}^{t_2}(-V_{REF})\,dt=0$$

由于 $T_2=t_2-t_1$，于是有

$$v_P=\frac{V_{REF}\cdot T_2}{\tau} \tag{10.2.6}$$

根据式（10.2.4）和式（10.2.6），得到

$$T_2=\frac{2^n T_c}{V_{REF}}V_I \tag{10.2.7}$$

可见，T_2与 V_I成正比，说明电路已经将输入电压的平均值转换成了中间变量时间间隔 T_2。

将式（10.2.7）代入到式（10.2.5），得到

$$\lambda=\frac{T_2}{T_c}=\frac{2^n}{V_{REF}}V_I \tag{10.2.8}$$

式（10.2.8）表明，在计数器中所计的数 λ（$\lambda=Q_{n-1}\cdots Q_1Q_0$）与在取样时间 T_1内输入电压的平均值 V_I成正比，与时钟周期 T_C和积分时间常数 $\tau=RC$ 无关。只要 $V_I<V_{REF}$，T_2期间计数器就不会溢出，转换器就能正常地将输入的模拟电压转换为数字量，并由计数器输出转换结果。

由于双积分型 A/D 转换器在 T_1期间取的是输入电压的平均值，因此只要选择定时积分时间 T_1为 50 Hz 市电周期的整数倍，就可以对 50 Hz 市电信号及其倍频信号具有很强的抑制能力。另一方面，由于在转换过程中，前后两次积分所采用的是同一积分器，这就要求在两次积分期间（一般在几十至数百毫秒之间），*R*、*C* 和脉冲源等元器件参数性能稳定，尤其是积分电容要选用性能稳定、介质损耗小的聚四氟乙烯、聚苯乙烯或聚苯烯等电容，否则，非线性误差会明显增加。

集成双积分型 A/D 转换器有二进制码输出和 BCD 码输出两大类。双积分型 A/D 转换器的主要缺点是转换速度慢，主要用在对转换精度要求较高、对转换速度要求不高的场合。例如，在数字电压表中广泛使用的 CB14433、CB7135、ICL7129、CB7126/7137 等都是双积分型 A/D 转换器。

图 10.2.13 是国产 CB7137 的功能框图，它是 $3\frac{1}{2}$ 位（常称为 3 位半）双积分型 A/D 转换器，能够直接输出七段译码信号并驱动外接的 LED 数码显示器。该芯片输出数码的最高位（千位）只能是 0 或 1，其余 3 位均可为 0～9 中的任何数，其数值范围为 0.000～1.999，故称为 3 位半。

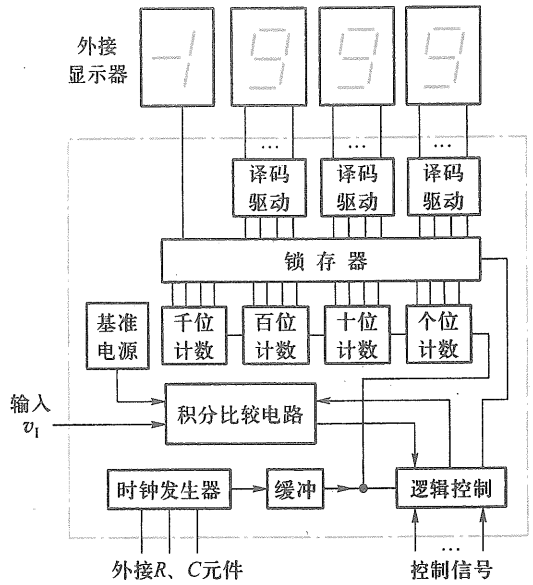


图 10.2.13 CB7137 双积分型 A/D 转换器的功能框图

10.2.5 A/D 转换器的主要技术指标

和 D/A 转换器一样，A/D 转换器的技术指标也较多，这里主要介绍分辨率、转换精度和转换速度等指标。

1. 分辨率

A/D 转换器的分辨率用输出二进制（或十进制）数的位数表示。它说明 A/D 转换器对输入信号的分辨能力。从理论上讲，n 位输出的 A/D 转换器能区分 2^n 个输入模拟电压信号的不同等级，能区分输入电压的最小值为满量程输入的 $1/2^n$。例如，当输入电压的最大值为 5 V 时，输出为 10 位二进制码的 A/D 转换器，可以分辨的最小输入电压为 4.88 mV。因此，在最大输入电压一定时，输出位数越多，量化单位越小，分辨率越高。

例 10.2.2 已知 8 位 A/D 转换器的基准电压 $V_{REF}=5.12$ V，求当输入 $v_I=3.8$ V 时，输出的二进制数字量。

解： 因为 A/D 转换器的基准电压 V_{REF} 就是输入信号的最大值。8 位 A/D 转换器的分

辨率为

$$\Delta = v_{\text{Imax}}/2^8 = V_{\text{REF}}/2^8 = 5.12/256 = 0.02\ \text{V}$$

当输入 $v_{\text{I}} = 3.8$ V 时，数字量为

$$\frac{v_{\text{I}}}{\Delta} = \frac{3.8}{0.02} = (190)_{\text{D}} = (\mathbf{10111110})_{\text{B}}$$

所以，输入 $v_{\text{I}} = 3.8$ V 时，输出的二进制码为 **10111110**。

2. 转换精度

由于模拟量是连续的，而数字量是离散的，所以，一般是某个范围中的模拟量对应于某一个数字量。这就是说，在 A/D 转换时，模拟量和数字量之间并不是一一对应的关系。于是就有一个转换精度的问题。

转换精度（或称为转换误差）反映了 A/D 转换器的实际值接近理想值的精确程度，通常用数字量的最低有效位（LSB）来表示，有时也以转换器满刻度的百分比（%FSR）表示。转换误差包括失调误差、增益误差、级差非线性误差和整体非线性误差等。

3. 转换速度

转换速度通常用转换时间或转换速率来描述。转换时间是指 A/D 转换器完成一次完整的转换所需要的时间。也就是说，从接到转换启动信号开始，到输出端得到稳定的数字量所经过的时间。

对于大多数 A/D 转换器来说，转换速率等于转换时间的倒数，它表示单位时间里完成转换的次数，其单位常用 kSPS（千次取样/秒）或 kHz、MSPS（兆次取样/秒）或 MHz。但某些高速 A/D 转换器（如分级流水线型）的转换速率可能高于转换时间的倒数，因为在前一次 A/D 转换结束之前，就开始了下一次的转换。

A/D 转换器的转换时间与转换电路的类型有关。不同类型的转换器，其转换时间差别较大。其中并行比较型 A/D 转换器的转换速度最快，仅需几十纳秒时间；逐次比较型 A/D 转换器次之，转换时间在几十微秒；双积分型 A/D 转换器的速度最慢，需几百毫秒左右。另外，对于各种不同类型 A/D 转换器而言，比较器的响应时间、运放的带宽、时钟频率、输出位数、输出方式等因素对转换时间也会产生一定的影响。

在实际应用中，应从系统数据总的位数、精度要求、输入模拟信号的范围及输入信号极性、输出数字的编码、工作温度范围和电压稳定度等方面综合考虑 A/D 转换器的选用。

10.2.6 集成 A/D 转换器及其应用

1. 集成 A/D 转换器 ADC0808/0809

国产 CB0808/0809（与 TI 公司的 ADC0808/0809 可互换使用）是采用 CMOS 工艺生产的 8 位逐次比较型 A/D 转换器，两个型号主要差别在于转换精度不同，其中 CB0808 的精度为±0.5 LSB，而 CB0809 的精度为±1 LSB。它们的电源 V_{CC} 为+5 V，其内部结构框图如图 10.2.14 所示。它具有 8 个模拟量输入通道，由于芯片内有三态输出缓冲器，输出数字量可直接与微处理器的数据总线相接，而无须附加接口电路。各引脚的功能如下：

*IN*0～*IN*7：8 路模拟信号输入端，输入电压范围为 0～5 V。

C、*B*、*A*：地址码输入端，其中 *C* 为最高位，*A* 为最低位，不同的地址码（**000～111**）选择不

同通道的模拟量输入；

ALE：地址码锁存允许信号输入端，高电平有效。

START：转换的启动信号输入端。该信号的上升沿到来时，将片内寄存器复位；下降沿开始进行转换。

$D_7 \cdots D_0$：8 位数字信号输出端；D_7为最高位，D_0为最低位。

CP：时钟信号输入端；时钟频率决定了转换速度，一般不高于 640 kHz。若时钟频率为 500 kHz，则转换时间大约为 128 μs。若频率为 640 kHz，则转换时间大约为 100 μs。

$V_{REF}(+)$、$V_{REF}(-)$：分别为基准电压的正、负输入端。一般情况下 $V_{REF}(+)$接 V_{CC}，$V_{REF}(-)$接 GND；

EOC：转换结束信号输出端。

OE：输出数据的控制输入端。当转换完成后，让 $OE=1$，使能三态输出缓冲器，将转换结果送到输出数据线上。

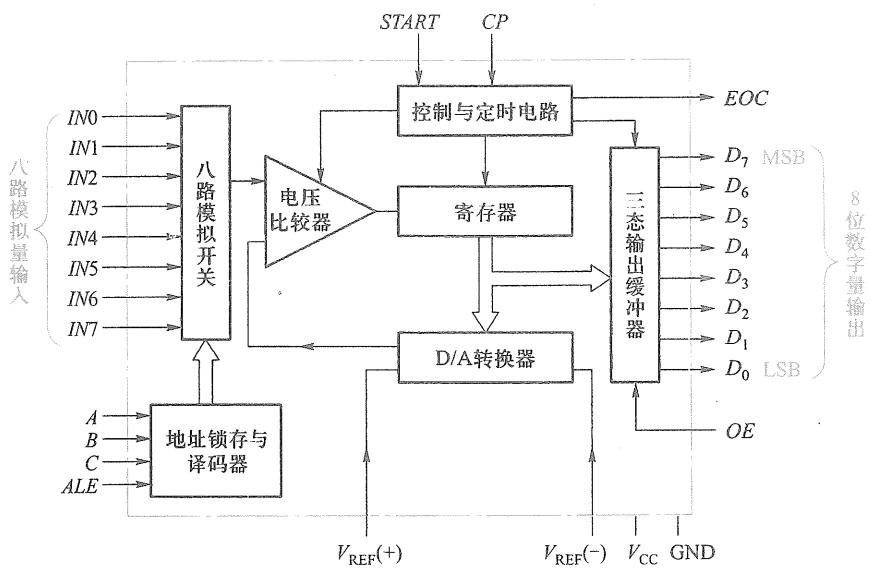


图 10.2.14 CB0808/0809 内部结构框图

2. CB0809 的典型应用

下面以数据采集系统为例，介绍 CB0809 的典型应用。在现代过程控制及各种智能仪器和仪表中，常用微处理器和 A/D 转换器组成数据采集系统，以便达到用计算机对被控（被测）对象进行实时控制、检测的目的。

微机化数据采集系统的示意图如图 10.2.15 所示。系统由微处理器、存储器和 A/D 转换器组成，系统信号采用总线传送方式，它们之间的信号通过地址总线、数据总线和控制总线连接。由于 CB0809 有 8 路不同的模拟量输入通道，可以采用多路复用技术来分时共享一个 A/D 转换器。在地址选择信号 *C*、*B*、*A* 的控制下，每次选择一个通道的模拟量进行转换。因此，该电路实际上是一个多通道数据采集系统。

单个通道数据采集的工作时序如图 10.2.16 所示，它是由 CB0809 控制信号的时序决定

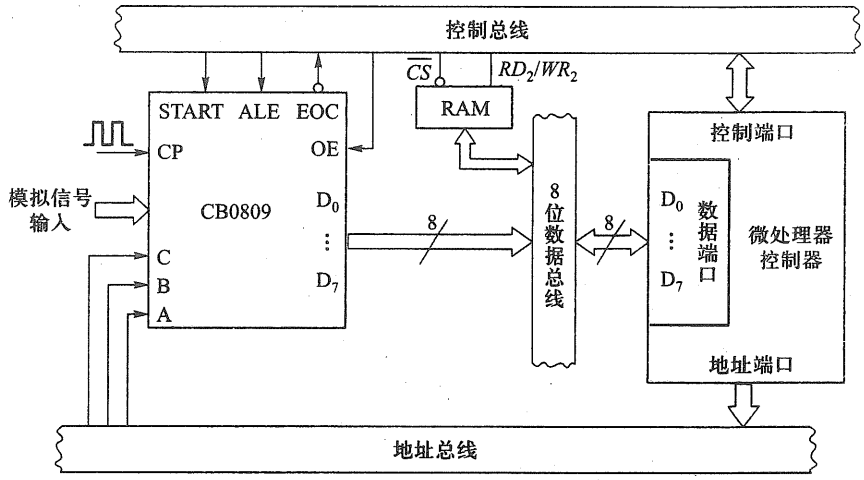


图 10.2.15 微机化数据采集系统示意图

的。在完成单个通道的数据采集与存储工作后，改变 C、B、A 的值，继续对下一个通道的模拟量进行采集，以此类推，就能够完成 8 个通道的数据采集与存储工作。

单个通道数据采集的具体过程如下：

（1）微处理器通过 *ALE* 和 C、B、A 把欲选通的模拟输入通道地址送入 CB0809。在 *ALE* 信号上升沿锁存通道地址于地址锁存器中，经译码后选择其中一个输入通道进行转换。

（2）选定模拟输入通道后，微处理器发出启动脉冲信号 *START*，在其下降沿电路开始采集模拟输入信号，进行 A/D 转换。在转换期间，*EOC* 信号一直为低电平。

（3）经过 t_C 时间后，A/D 转换完成并发出 $EOC=1$ 的信号，微处理器检测到 *EOC* 为高电平后，向 CB0809 发出 *OE* 信号（高电平脉冲）。

（4）CB0809 收到 *OE* 的高电平信号后，经过 t_K 时间，就会送出转换结果。此时，微处理器通过数据总线读入 A/D 转换的结果，并将结果保存在数据存储器 RAM 中。

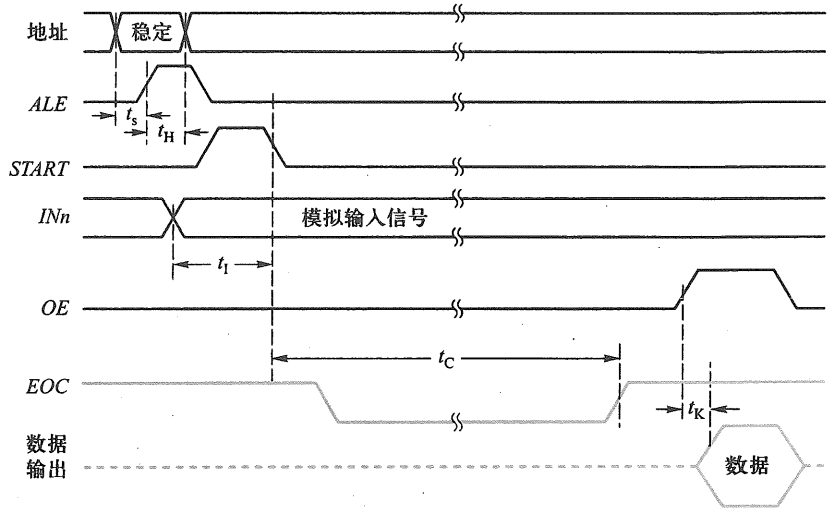


图 10.2.16 系统工作过程的时序波形图

在使用模数、数模转换电路时，必须正确地处理模拟电路和数字电路地线的连接问题，否则会产生严重的干扰，影响转换结果的准确性。A/D、D/A 及取样保持芯片都提供了独立的模拟地（AGND）和数字地（DGND）的引脚，而微处理器属于数字电路。在线路设计中，必须将所有器件的模拟地和数字地分别相连，然后将模拟地与数字地仅在一点上相连接。地线的正确连接方法如图 10.2.17 所示。

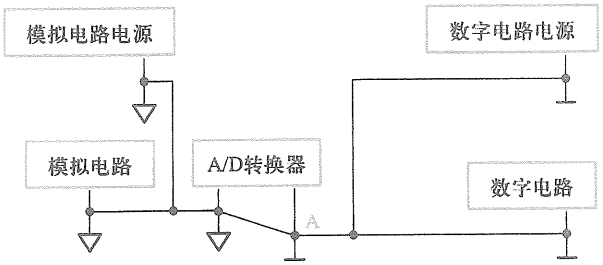


图 10.2.17　正确的地线连接

复习思考题

10.2.1　实现模/数转换一般要经过哪四个过程？按工作原理不同分类，A/D 转换器可分为哪两种？

10.2.2　在图 10.2.4 所示并行比较 A/D 转换电路中，若输入电压 v_I 为负电压，试问电路能否正常进行 A/D 转换？为什么？如不能正常工作需要如何改进电路？

10.2.3　逐次比较型 A/D 转换器完成一次 A/D 转换所需时间为多少？转换时间与哪些因素有关？

10.2.4　试问双积分型 A/D 转换器输出数字量与下述哪些参数有关？

(a) 积分时间常数；(b) 时钟脉冲频率；(c) 输入信号电压；(d) 计数器位数；(e) 运放的零漂；(f) $|V_{REF}|$。

10.2.5　比较并行比较型 A/D 转换器、逐次比较型 A/D 转换器、双积分型 A/D 转换器的优、缺点，试问应如何根据实际系统要求合理选用？

小　结

- D/A 转换器分为权电阻网络型、倒 T 形电阻网络型、电阻串联分压式网络型、权电流型等不同的电路结构。在 CMOS 集成 D/A 转换器中，倒 T 形电阻网络和电阻串联分压式网络电路用得较多；在双极性集成 D/A 转换器中，权电流型电路较为常见。
- 不同结构的 A/D 转换器有各自的特点，在要求转换速度高的场合，可选用并行比较型、半闪速型或者流水线型 A/D 转换器；在要求精度高的情况下，可以采用双积分型 A/D 转换器，当然也可选用高分辨率的其他形式 A/D 转换器，但成本会增加。由于逐次比较型 A/D 转换器在一定程度上兼顾了以上两种转换器的优点，因此得到普遍应用。
- D/A 转换器和 A/D 转换器的主要技术参数是转换精度和转换速度。目前，数据转换器正向着高速度、高分辨率和易于与微型计算机接口的方向发展。

习　题

10.1　D/A 转换器

10.1.1　已知某 D/A 转换器电路满刻度输出电压为 10 V，要求能分辨的最小输出模拟电压

为4.9 mV，试问其输入数字量的位数 n 至少是多少？

10.1.2 在图 10.1.2 所示 4 位 $R-2R$ 倒 T 形电阻网络 D/A 转换器中，已知 $V_{REF}=5$ V，$R_f=R=10$ kΩ。

（1）当输入二进制码分别为 **0001**、**0110**、**1010**、**1111** 时，求输出电压 v_O 的值。

（2）若输出电压 $v_O=-1.25$ V，则输入的二进制码是多少？

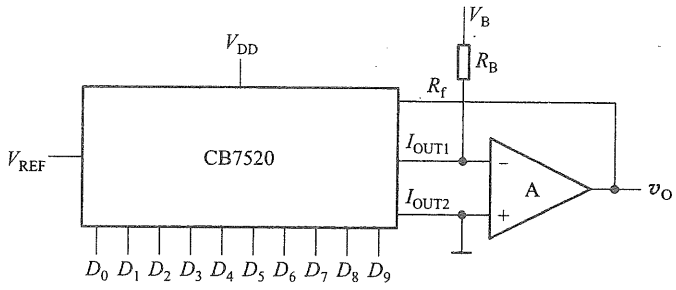
（3）求 V_{LSB}、V_{FSR} 和分辨率。

10.1.3 在图 10.1.2 所示的倒 T 形电阻网络 D/A 转换器中，设 $R_f=R$，外接基准电压 $V_{REF}=-10$ V，为保证 V_{REF} 偏离标准值所引起的误差小于 LSB/2，试计算 V_{REF} 的相对稳定度 $\frac{|\Delta V_{REF}|}{|V_{REF}|}$ 应取多少？

10.1.4 由 CB7520 组成双极性输出 D/A 转换器如图题 10.1.4 所示。

（1）根据电路写出输出电压 v_O 的表达式。

（2）试问实现输入为 2 的补码时的双极性输出电路中，V_B、R_B、V_{REF} 和片内的 R 应满足什么关系？



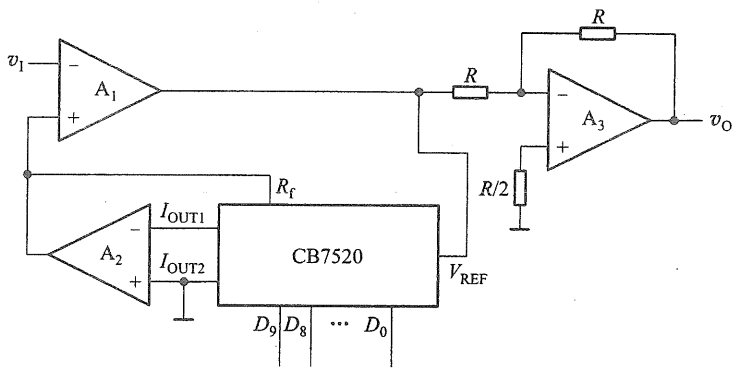
图题 10.1.4

10.1.5 可编程放大器（数控可变增益放大器）电路如图题 10.1.5 所示。

（1）推导电路电压放大倍数 $A_v=v_O/v_I$ 的表达式。

（2）当输入编码为 $(001)_H$ 和 $(3FF)_H$ 时，电压放大倍数 A_v 分别为多少？

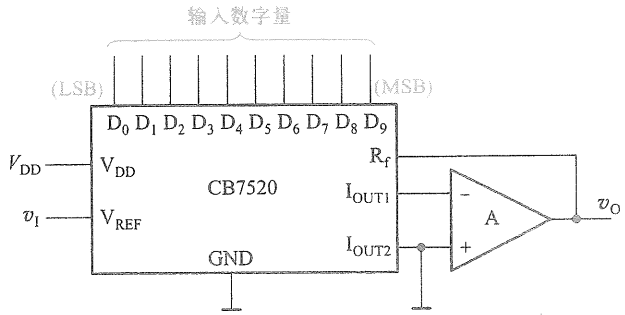
（3）试问当输入编码为 $(000)_H$ 时，运放 A_1 处于什么状态？



图题 10.1.5

10.1.6　4 位权电流 D/A 转换器如图 10.1.8 所示，已知 $V_{REF}=6$ V，$R_1=48$ kΩ，当输入 $D_3D_2D_1D_0=1100$ 时，$v_O=1.5$ V，试确定 R_f 的值。

10.1.7　由 10 位 D/A 转换器 CB7520 和运算放大器组成的电路如图题 10.1.7 所示。试推导电路电压放大倍数 $A_v=v_O/v_I$ 的表达式，确定 A_v 的取值范围，并说明电路的功能。



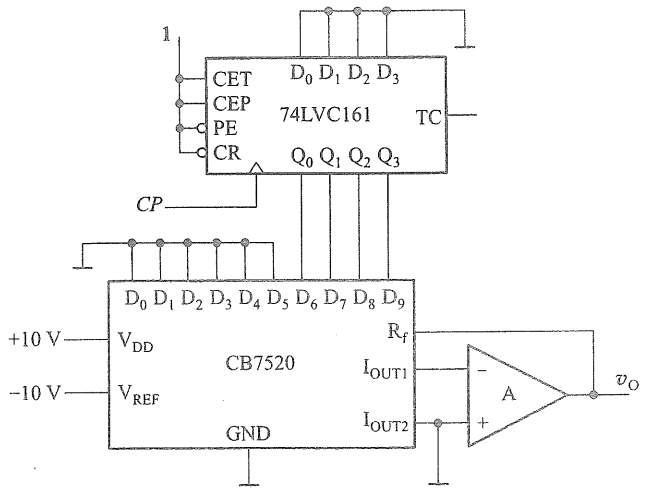
图题 10.1.7

10.1.8　由 10 位 D/A 转换器 CB7520 和计数器 74LVC161 组成的波形发生器电路如图题 10.1.8 所示。

（1）画出 74LVC161 的状态转换图。

（2）求输出电压的最大值 $v_{O(max)}$。

（3）求输入最低位 D_0 为 1、其余各位为 0 时，对应的输出电压值 $V_{O(LSB)}$。



图题 10.1.8

10.2　A/D 转换器

10.2.1　在图 10.2.4 所示并行比较型 A/D 转换器中，$V_{REF}=7$ V，试问电路的最小量化单位 Δ 等于多少？当 $v_I=2.4$ V 时，输出数字量 $D_2D_1D_0=?$

10.2.2　在图 10.2.10 逐次比较 A/D 转换器中，设 $V_{REF}=10$ V，$v_I=8.26$ V，试画出在时钟脉冲作用下，v'_0 的波形并写出转换结果。

10.2.3 在双积分型 A/D 转换器中，输入电压 v_I 和基准电压 V_{REF} 在极性和数值上应满足什么关系？如 $|v_I|>|V_{REF}|$，电路能完成模数转换吗？为什么？

10.2.4 在图 10.2.11 所示双积分型 A/D 转换器中，设时钟脉冲频率为 f_{CP}，分辨率为 n 位，写出 A/D 转换器最低的转换频率表达式。

10.2.5 某双积分型 A/D 转换器中，计数器为十进制计数器，其最大计数容量为 $(3000)_D$。已知计数时钟频率 $f_{CP}=30$ kHz，积分器中 $R=100$ kΩ，$C=1$ μF，输入电压 v_I 的变化范围为 0~5 V。

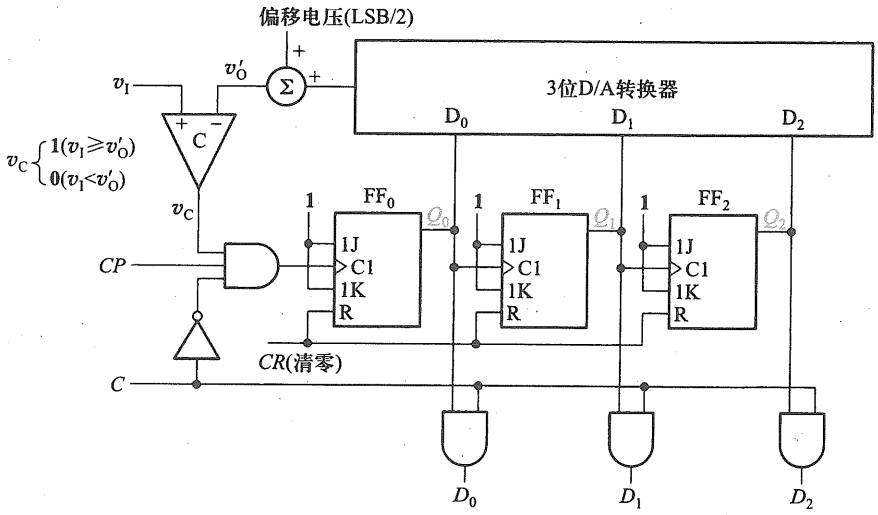
(1) 求第一次积分时间 T_1。

(2) 求积分器的最大输出电压 $|V_{omax}|$。

(3) 当 $V_{REF}=10$ V，第二次积分计数器计数值 $\lambda=(2500)_D$ 时，输入电压的平均值 v_I 为多少？

10.2.6 在应用 A/D 转换器的过程中，通常应注意哪些主要问题？假设输入信号的幅值为 0.5 V，如果采用满度值为 10 V 的 8 位 A/D 转换器进行模数转换，你认为这样使用正确吗？为什么？

10.2.7 计数型 A/D 转换器如图题 10.2.7 所示，试分析其工作原理。



图题 10.2.7

文档 10.2：第 10 章部分习题答案

第11章 数字系统设计基础

引言

前面几章讨论了组合逻辑电路和时序逻辑电路的分析和设计方法。这些方法是建立在真值表、卡诺图和状态转换表的基础上，可以满足功能比较简单的基本逻辑单元电路的分析和设计。对于由基本逻辑模块组成规模更大、功能更复杂的数字电路的设计，若采用经典的方法进行描述和设计，则设计过程比较复杂。因此，需要采用新的方法，即分层次设计。

本章首先介绍数字系统的基本概念、设计方法和实现方法。然后介绍常用的设计工具，即算法状态机 ASM 图，并列举简单的实例说明数字系统的设计过程。通过讲述这些引导性的内容以期初学者获得数字系统设计的基础知识和设计技巧。

文档 11.1：第 11 章思维导图

11.1 数字系统概述

11.1.1 数字系统的组成

前面几章介绍的数据选择器、比较器、加法器、计数器等电路，都是只能实现某种单一的特定功能，因此称为逻辑功能模块。由若干这样的模块作为子电路构成的，按一定顺序处理和传输数字信号的设备，称为**数字系统**。电子计算机、数字照相机、数字电视等就是常见的数字系统。当数字系统比较复杂，内部状态数目很大时，若采用经典的状态图、状态转换表等方法进行设计时，设计过程过于复杂，不易于完成设计任务。这时通常采用层次化设计方法，即先将数字系统划分为几个子系统，再将每个子系统分解为几个模块，每个模块再细分为几个功能块，最终用各种门和触发器等实现每个功能块。

数字系统从结构上可以划分为**数据通路**(datapath)和**控制单元**(control unit)两部分，如图 11.1.1 所示。因此，数字系统中的二进制信息也划分成数据信息和控制信息两大类。

数据通路接受控制单元发来的控制信号，对输入的数据进行算术运算、逻辑运算、移位操作等处理，然后输出数据，并将处理过程中产生的状态信息反馈到控制单元，数据通路也称为数据处理单元。

控制单元根据外部输入信号及数据通路提供的状态信息，决定下一步要完成的操作，并

11.1.3 数字系统的实现

随着集成电路技术的发展和计算机的应用,数字系统的实现也经历了由分立元件、小规模、中规模到大规模、超大规模,直至今天的专用集成电路(ASIC)的过程。数字系统的实现大致有以下几种方法:

(1) 采用通用的集成逻辑器件组成。

(2) 采用单片微处理器作为核心实现。

(3) 采用可编程逻辑器件实现。

(4) 设计功能完整的数字系统芯片。

第一种是传统的方法,曾经得到广泛应用,目前在设计简单电路时还在使用。第二种方法所用器件少,使用灵活,也得到广泛应用,但工作速度相对后面两种方法低些。第三种方法设计的系统体积小、功耗低、可靠性高、易于进行修改等,成为当今实现数字系统设计的首选方案。目前的可编程器件功能已经非常强大,一片或几片就可实现一个复杂的数字系统。第四种方法是将一个完整的系统集成在一个芯片上,称为**芯片系统或片上系统**(system on chip,SOC)。微电子制造工艺的进步和 CAD 软件技术的发展,使得数字系统的设计效率和复杂程度都在不断提高。

复习思考题

11.1.1 从结构上划分,数字系统由哪几部分组成?所传输的二进制信息分为哪几类?

11.1.2 自上而下的设计方法有哪些优点?

11.2 算法状态机

时序电路在有限个状态之间按照一定的规律进行转换,称为**有限状态机**(FSM)。对于数字系统这样较大型状态机,其输入和输出信号的数目较多。若仍然采用第 6 章介绍的状态转换图或状态转换表等描述方法,则设计过程过于复杂,甚至无法完成整个系统的设计。因此,要使用另一种称为**算法状态机**(algorithmic state machine,ASM)的图形描述方法。ASM 图的作用是描述数字系统控制单元的工作流程,其结构形式类似于计算机中的程序流程图。主要用来描述控制单元的时序操作特性,说明控制条件及控制单元状态转换过程。应用 ASM 图设计控制单元,很容易获得状态函数和输出函数,从而得出相应的硬件电路。

11.2.1 ASM 图形符号

ASM 图中有三种基本的符号,即状态框、判断框和输出框。

1. 状态框

状态框用于表示控制单元的一个状态,用矩形框表示,如图 11.2.1(a)所示。状态名称置于状态框左上角,分配给状态的代码位于状态框的右上角。框内标出在此状态下输出等于 1 的逻辑变量,即穆尔型输出。因为这些输出只与电路的状态有关。一般只标出信号名,例如 Z,就表示其值为 1,而不必写 $Z=1$。有时在状态框内,也标明此状态下系统要执行的

操作，并以助记符的形式表示，例如：$A \leftarrow A+1$，即计数器的内容必须加 1。或者表示移位寄存器的一次移位等。图中的箭头表示控制单元状态的流向，在时钟脉冲触发沿的触发下，控制单元进入该状态，并且一个状态框占用一个时钟脉冲周期。

2. 判断框

判断框表示根据对输入条件的测试确定状态转换的分支，用菱形框表示，如图 11.2.1(b)所示。它有一个入口和多个出口，框内填判断条件，如果条件是真，选择一个出口，若条件是假，选择另一个出口。

判断框的入口来自某一个状态框，在该状态占用的一个时钟周期内，根据判断框中的条件，以决定下一个时钟脉冲触发沿来到时，该状态从判断框的哪个出口出去，因此，判断框不占用时间。

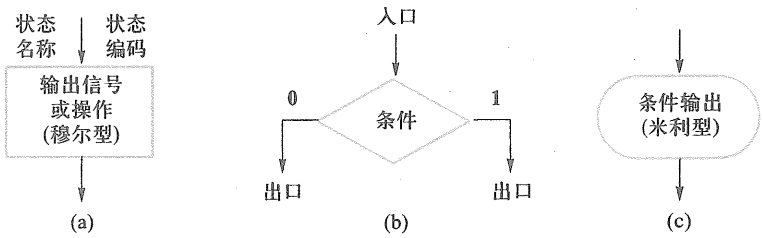


图 11.2.1 ASM 图中的三种图形符号

(a) 状态框 (b) 判断框 (c) 条件输出框

3. 条件输出框

条件输出框用椭圆形框表示，如图 11.2.1(c)所示，它描述了对判断框中判断条件的响应。条件框的入口必定与判断框的输出相连。列在条件框内的变量是米利型输出，当满足所接判断框出口的条件时，条件框内的逻辑变量等于 1，否则为 0。

有时系统的输出有多个，既有穆尔型又有米利型。穆尔型输出写在状态框内，米利型输出写在条件输出框内。

4. ASM 图的定时关系

一个 ASM 图由多个 ASM 模块组成。一般情况下每个 ASM 模块包含一个状态框、若干个判断框和条件输出框。要特别强调的是一个 ASM 模块只能有一个状态框和一个入口，但可以有多个连接不同次态的出口。在同一个 ASM 模块中，状态框和条件输出框内标出的变量的操作都是在同一个系统时钟信号控制下完成的，同时系统控制器在下一个时钟信号到达时进入下一个状态。例如在图 11.2.2 所示的 ASM 举例中，虚线内为一个 ASM 模块。当系统控制器进入 S_0 状态后，计数器的内容加 1。若这时 $E=0$、$F=0$，则下一个时钟信号到达时，控制器转入 S_1 状态。若这时 $E=0$、$F=1$，下一个时钟信号到达时，控制器转入 S_2 状态。如果 $E=1$，则 R 清零，下一个时钟信号到达时，控制器转入 S_3 状态。

表面上看 ASM 图与程序流程图很相似，但实际上有很大的差异。程序流程图只表示事件发生的先后顺序，没有时间概念，而 ASM 图则不同，它包含了每个有效时钟沿之后，由一个状态转换为另一个状态的时序信息。对于图11.2.2给出的 ASM 图，假设系统中所有触发器都是上升沿触发的，在第一个时钟脉冲上升沿来到时，系统转换到 S_0 状态，随后根据条件

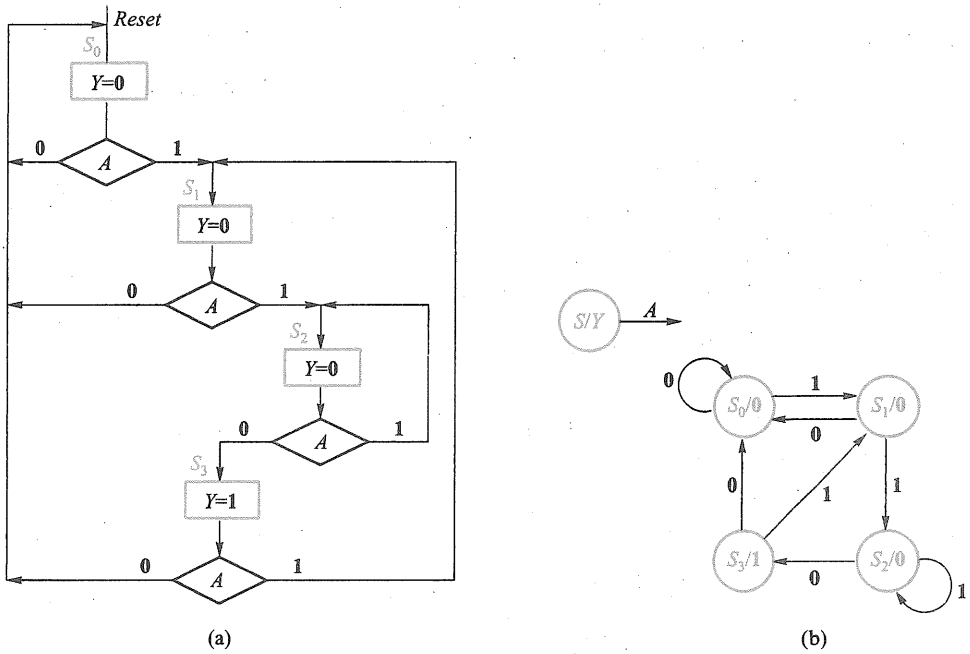


图 11.2.5 110 序列编码检测的穆尔型状态机

(a) ASM 图 (b) 状态转换图

有米利型输出，又有穆尔型输出。电路的 ASM 图既包含状态输出框，也包含条件输出框。

复习思考题

11.2.1 ASM 图中有哪几种基本符号？各符号的含义是什么？

11.2.2 ASM 图中条件分支与条件输出有什么区别，条件输出与状态输出有什么区别？

11.2.3 ASM 图与程序流程图的根本区别是什么？

11.2.4 米利型和穆尔型电路的 ASM 图最大的区别是什么？

11.3 交通信号灯控制系统

交通信号灯自问世以来，经历了机电定时控制，继电器控制，到现在计算机控制的发展过程。随着经济发展、城市人口增加、汽车的普及，交通问题成为各个国家面临的共同问题。许多国家投入巨大人力和物力研究智能交通系统，以提高整个路网的通行能力。

作为数字系统设计入门，下面以最简单的交通灯控制系统为例，说明数字系统设计过程，以及 ASM 图法在数字系统设计过程中的应用。

11.3.1 交通信号灯控制系统 ASM 图

设计一个主干道和支干道十字路口的交通信号灯控制电路，其技术要求如下：

(1) 一般情况下，保持主干道畅通，主干道绿灯亮、支干道红灯亮，并且主干道绿灯亮的时间不得少于 60 s。

（2）当主干道绿灯亮的时间超过 60 s，且支干道有车时，主干道红灯亮、支干道绿灯亮，但支干道绿灯亮的时间不得超过 30 s。

（3）每次主干道或支干道绿灯变红灯时，黄灯先亮 5 s。

1. 明确系统的整体方案

图 11.3.1 表示位于主干道和支干道的十字路口交通灯系统，支干道两边安装传感器 S，要求优先保证主干道的畅通。平时处于主干道绿灯、支干道红灯的状态。当支干道有车时，传感器发出信号 $S=1$，主干道绿灯先转换成黄灯、再变成红灯，支干道由红灯变成绿灯。如果支干道继续有车通过时，则传感器继续有信号，使支干道保持绿灯亮，但支干道绿灯持续亮的时间不得超过 30 s，否则支干道绿灯先转换成黄灯再变成红灯，同时主干道由红灯变成绿灯。主干道每次通行时间不得短于 60 s，在此期间，即使支干道 S 有信号，也不能中止主干道的绿灯亮。

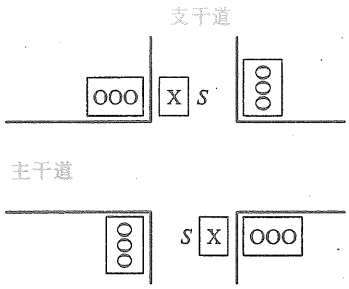


图 11.3.1 交通灯示意图

2. 确定控制单元算法并画出 ASM 图

系统由控制单元和处理单元（数据通路）组成，控制单元接收外部系统时钟和传感器信号。处理单元由定时器和译码显示器组成。定时器能向控制单元发出 60 s、30 s 或 5 s 定时信号，译码显示电路在控制单元的控制下，改变交通灯信号。根据题目要求画出系统框图如图 11.3.2 所示，其中：

HG、HY、HR 分别表示主干道绿、黄、红三色灯。

FG、FY、FR 分别表示支干道绿、黄、红三色灯。

交通灯系统工作时，会用到三个不同时长的定时信号 T_L、T_S 和 T_Y。

T_L：主干道绿灯亮的最短时间，不少于 60 s。

T_S：支干道绿灯亮的最长时间，不多于 30 s。

T_Y：主干道或支干道黄灯亮的时间为 5 s。

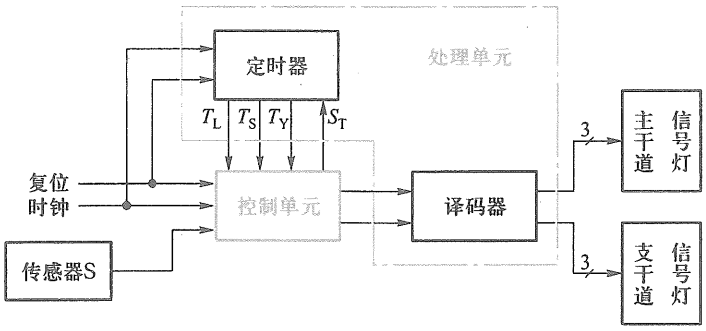


图 11.3.2 交通灯系统框图

因此，用定时器分别产生 T_L、T_S 和 T_Y 后，向控制单元发出时间已到信号，控制单元根据定时器及传感器的信号，决定是否进行状态转换。如果肯定，则控制单元发出状态转换信号 S_T，定时器开始清零，准备重新计时。

交通灯控制单元的控制过程分为四个阶段，对应四种状态，分别用 S_0、S_1、S_2和 S_3表示：

S_0状态：主干道绿灯亮，支干道红灯亮，此时若支干道有车等待通过，而且主干道绿灯已亮足规定的时间 T_L，控制器发出状态转换信号 S_T，状态从 S_0转换到 S_1。

S_1状态：主干道黄灯亮，支干道红灯亮，进入此状态，黄灯亮足规定的时间 T_Y时，控制器发出状态转换信号 S_T，状态从 S_1转换到 S_2。

S_2状态：支干道绿灯亮，主干道红灯亮，若此时支干道继续有车，则继续保持此状态，但支干道绿灯亮的时间不得超过 T_S，否则控制单元发出状态转换信号 S_T，使状态转换到 S_3。若此时支干道没有车，则控制单元立即发出状态转换信号 S_T，使状态转换到 S_3。

S_3状态：支干道黄灯亮，主干道红灯亮，此时状态与 S_1状态持续的时间相同，均为 T_Y，时间到时，控制器发出 S_T信号，状态从 S_3回到 S_0。

对上述 S_0、S_1、S_2和 S_3四种状态按照格雷码进行编码分别为 **00**，**01**，**11** 和 **10**，由此得到交通灯控制单元的 ASM 图如图 11.3.3 所示。异步复位信号 *Reset* 使控制单元直接进入主干道绿灯亮、支干道红灯亮的初始状态，用 S_0状态框表示。当 S_0状态持续时间 T_L大于等于 60 s，并且支干道有车等待通过使传感器 $S=1$。两个条件同时满足使 $T_L \cdot S=1$，即满足判断框中的 $T_L \cdot S=1$条件，控制单元发出状态转换信号 S_T，由条件输出框表示，同时系统从状态 S_0转到主干道黄灯亮、支干道红灯亮的 S_1状态。

在 S_1状态，黄灯亮足规定的时间 T_Y，即状态转换的条件 $T_Y=1$ 时，产生状态转换信号 S_T，系统转到 S_2状态。

在 S_2状态，有两个条件控制系统状态转换，一个是支干道绿灯亮足规定的时间 T_S，另一个是支干道没有车 $S=0$。只要满足两个条件之一，即 $T_S+\overline{S}=1$，系统立即转换到 S_3状态。

以此类推得出图 11.3.3 所示的交通灯控制系统 ASM 图。

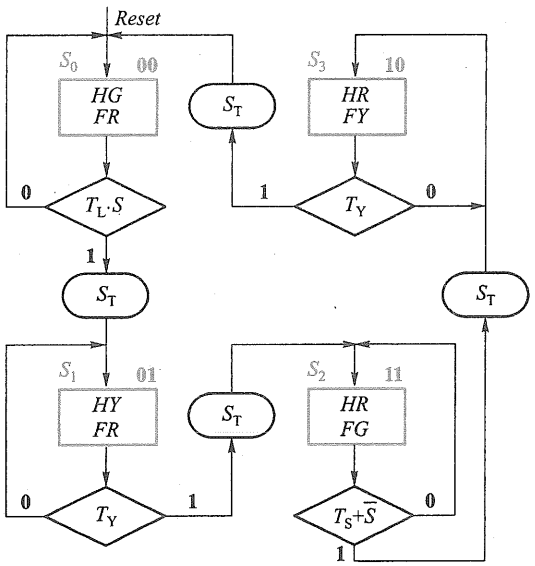


图 11.3.3　交通灯控制系统 ASM 图

11.3.2 用典型电路基本模块设计交通灯控制系统

下面介绍用典型电路基本模块设计交通灯控制系统的控制单元和处理单元。

1. 控制单元电路

根据交通灯控制系统的 ASM 图，得出其状态转换图如图 11.3.4 所示。ASM 图中的状态框与状态转换图中的状态相对应，判断框中的条件是状态转换的输入条件，条件输出框与状态转换图中的输出相对应。

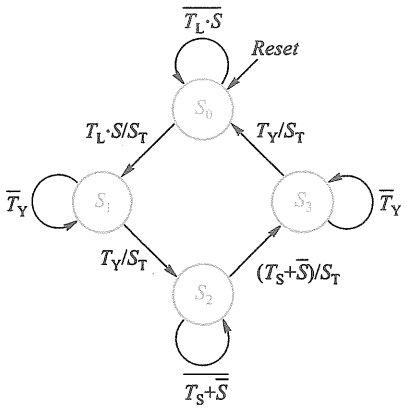


图 11.3.4 交通灯控制单元状态转换图

根据状态转换图，用 6.3 节介绍的同步时序电路的设计方法，可以设计出交通灯控制单元电路，其中状态分配采用格雷码。

状态转换图中，S_T是控制单元的输出信号，用来控制定时器的工作。图 11.3.4 中未画出主干道和支干道交通灯的控制信号，它们是处理单元的输出，由控制单元的四种输出状态确定。

交通灯控制单元有 4 种状态，选用两个触发器 FF_1、FF_0的 4 种输出组合对其进行格雷码的编码，可以列出如表 11.3.1 所示的状态转换表，状态转换的条件为 $T_L \cdot S$、T_Y和 $T_S+\overline{S}$。*Reset* 信号将控制单元置为初始状态$Q_1^nQ_0^n=00$ 以后，如果 $T_L \cdot S=0$，则控制单元保持在 00 状态；如果 $T_L \cdot S=1$，则控制器转换到$Q_1^{n+1}Q_0^{n+1}=01$ 状态。这两种情况与条件 T_Y和 $T_S+\overline{S}$无关，所以用无关项“×”表示。其余情况以此类推，同时表中列出了状态转换信号 S_T。

表 11.3.1 控制单元状态转换表

现态符号	输入					输出		
	现态		状态转换条件			次态		状态转换信号
	Q_1^n	Q_0^n	$T_L \cdot S$	T_Y	$T_S+\overline{S}$	Q_1^{n+1}	Q_0^{n+1}	S_T
S_0	0	0	0	×	×	0	0	0
S_0	0	0	1	×	×	0	1	1
S_1	0	1	×	0	×	0	1	0
S_1	0	1	×	1	×	1	1	1
S_2	1	1	×	×	0	1	1	0
S_2	1	1	×	×	1	1	0	1
S_3	1	0	×	0	×	1	0	0
S_3	1	0	×	1	×	0	0	1

由表 11.3.1 可得状态转换方程和状态转换信号方程如下：

$$Q_1^{n+1}=\overline{Q}_1^nQ_0^nT_Y+Q_1^n\overline{Q}_0^n\overline{T}_Y+Q_1^nQ_0^n$$

$$Q_0^{n+1}=\overline{Q}_1^n\overline{Q}_0^nT_L\cdot S+\overline{Q}_1^nQ_0^n+Q_1^nQ_0^n\overline{T_S+\overline{S}}$$

$$S_T=\overline{Q}_1^n\overline{Q}_0^nT_L\cdot S+\overline{Q}_1^nQ_0^nT_Y+Q_1^n\overline{Q}_0^nT_Y+Q_1^nQ_0^n(T_S+\overline{S})$$

FF_1和FF_0选用D触发器，每个触发器的激励函数用数据选择器选择，所以数据选择器的数目等于触发器的数目。考虑到控制信号S_T与输入变量的关系，也需要用一个数据选择器，所以共选用三个数据选择器，并将触发器的现态值加到数据选择器的选择变量端，数据选择器的输入端信号可以根据状态转换方程和状态转换信号方程得出。交通灯控制单元的逻辑图如图 11.3.5 所示。

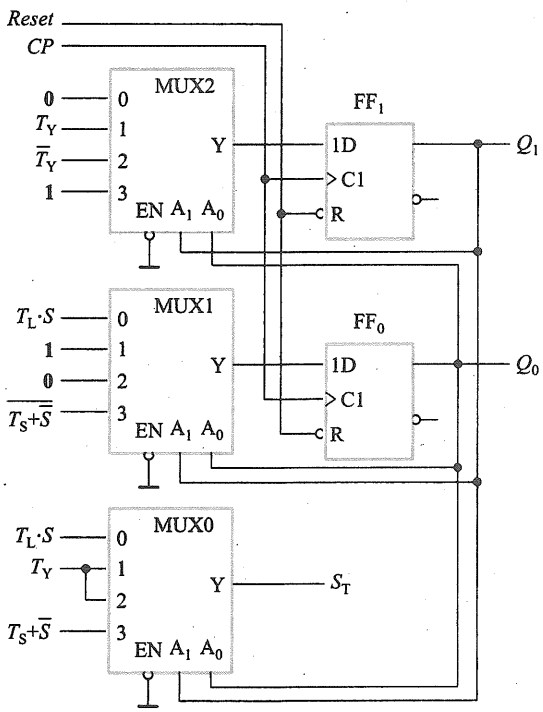


图 11.3.5 交通灯控制单元电路

2. 处理单元电路

（1）定时器

定时器由计数器构成。该计数器与系统时钟（由时钟脉冲产生器提供）同步，时钟脉冲为秒脉冲，其上升沿到来时，在控制信号S_T作用下，计数器从零开始计数，并向控制单元提供模 M5、M30 和 M60 信号，即T_Y、T_S和T_L三个不同时长的定时信号。计数器工作在 M60 状态，并且当计数到 4 和 29 时，也给出T_Y、T_S信号，但此时若系统处于S_0状态，控制单元只检测T_L信号而不响应T_Y、T_S信号。

当系统处于S_0状态，为满足主干道绿灯亮，支干道红灯亮的持续时间$T_L\geqslant 60$ s，所以要将 M60 的输出端反馈到计数器的使能端EN，使它计到 59 时停止计数，并保持在$M=60$的状态直到支干道有车要通过时，才转换到S_1状态。

要求计数器在状态转换信号S_T作用下，首先清零，然后开始计数。定时器框图如图 11.3.6所示。

计数器具有高电平有效使能端EN，异步清零端CLR，同步置数端LD和进位输出端C_O。异步清零端接整个系统复位端$Reset$。当计数到$Q_5Q_4Q_3Q_2Q_1Q_0=\mathbf{111011}$，即$M=60$时，$C_O=1$，将其反相后接入使能端，就可以保持在$M=60$状态。控制单元发出的状态转换信号$S_T$是高电平有效，接至置数端，使计数器置零。

（2）译码器

系统在$S_0\sim S_3$状态下，由译码电路将控制单元的输出Q_1Q_0转换成六个信号灯的驱动信号，信号灯的译码电路框图如图 11.3.7 所示。各状态与信号灯的关系由表 11.3.2 给出。因此，得到灯光信号与控制单元状态变量的关系为

$$HG=\overline{Q}_1\overline{Q}_0\qquad HY=\overline{Q}_1Q_0\qquad HR=Q_1$$

$FG=Q_1Q_0$　　$FY=Q_1\overline{Q}_0$　　$FR=\overline{Q}_1$

根据上述逻辑函数表达式就可以实现译码电路。

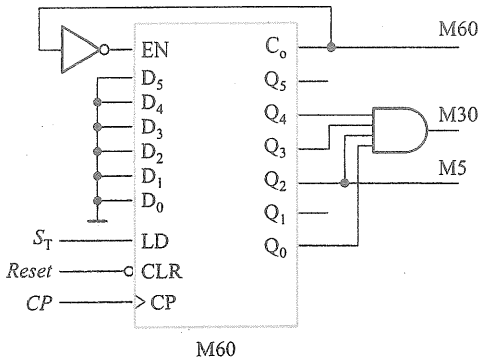


图 11.3.6　定时器框图

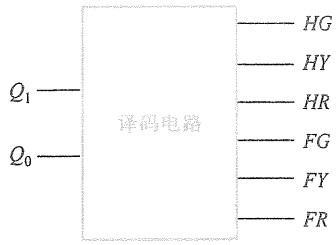


图 11.3.7　信号灯的译码电路框图

将控制单元和处理单元组合起来就可以构成交通信号灯控制系统，交通灯系统电路图略。

表 11.3.2　信号灯与控制器状态编码表

状态	状态编码 Q_1Q_0	*HG*	*HY*	*HR*	*FG*	*FY*	*FR*
S_0	00	1	0	0	0	0	1
S_1	01	0	1	0	0	0	1
S_2	11	0	0	1	1	0	0
S_3	10	0	0	1	0	1	0

11.3.3　交通灯控制系统 Verilog 建模

当采用可编程逻辑器件实现数字系统时，一个重要任务是用 HDL 描述数字系统。其描述可以在结构级或行为级进行，行为级又可分为寄存器级或算法级。因此，对数字系统的描述可以分为结构级描述、寄存器传输级(RTL)描述和基于算法的行为级描述。

结构级描述是最底层、最详细的描述。它是根据具体物理元件以及它们之间的连接描述系统。这些元件包括门、触发器及选择器、计数器等标准部件。描述过程首先是将系统划分为多个不同功能的模块，然后用 HDL 描述每个功能模块，最后将所有这些底层模块组合起来构成顶层模块，即完成了整个系统的设计。

寄存器传输级描述是根据寄存器要完成的操作，以及操作的顺序来描述系统。这种类型的描述是用过程语句说明各种操作的关系，不涉及具体硬件电路结构。但是，寄存器传输级描述隐含了寄存器的硬件电路结构，可以用标准部件实现系统。

基于算法的行为级描述是最抽象的。用类似于编程语言中的过程算法形式描述系统的功能，不涉及任何的硬件电路实现。因此，这一层设计的某些描述不能被开发软件综合成具

体结构形式。这种行为级描述适合于复杂数字系统的仿真,用来证明设计是否正确。

下面我们将通过实例介绍数字系统的结构级和 RTL 级描述。

1. 寄存器传输级描述

交通灯控制系统的寄存器传输级描述如图 11.3.8 所示,整个代码分为 4 部分。第一部分声明了系统的输入、输出和一些内部信号。其输入信号为时钟 CLK、传感器 S 和复位信号 RESET。输出信号有主干道和支干道信号灯 HG、HY、HR 及 FR、FY、FG。

第二部分描述控制单元工作的时序关系。它是根据图 11.3.4 所示的状态图编写的,控制电路有 4 个状态,由两个 *D* 触发器构成,用 **reg** [1:0] CurrentState, NextState 表示;其中 NextState 表示 *D* 触发器的输入,CurrentState 表示 *D* 触发器的输出。整个控制电路用两个 **always** 语句描述。第一个 **always** 说明了两个操作过程:异步复位信号 RESET 使系统进入初态 S0 和时钟 CLK 上升沿作用下系统进行状态转换。第二个 **always** 是由 **case** 语句构成的一个组合逻辑电路,准备好现态到次态的转换条件。例如,现态是 S0,如果 Tl·S = **1**,则次态应该为 S1 状态,只需要等待 CLK 的上升沿到来。如果 Tl·S = **0**,则保持在 S0 状态。

第三部分是定时器的描述。它是由具有异步清零功能的 BCD 码计数器组成的,RESET 信号使计数器异步清零。St 是由控制单元发出的状态转换信号,每次状态转换时,St 使定时器同步清零后重新计数。在 S0、S2、S1(或 S3)状态下,定时器分别给出 3 个定时信号 Tl、Ts 和 Ty,以供控制单元决定是否进行状态转换。

第四部分是信号灯译码电路的描述,根据图 11.3.7 编写。由 **case** 语句说明在不同的状态下,两组输出信号(HG、HY、HR 或 FR、FY、FG)中均有一个为 **1**。

```
module Traffic_RTL (
    /*======声明输入/输出端口,见图11.3.2 ======*/
     input        CLK, S, RESET,
     output reg   HG, HY, HR, FG, FY, FR
     );
     //声明系统内部的信号,见图11.3.2
     reg [3:0]    TimerH, TimerL;          //用于定时器的寄存器
     wire         Tl, Ts, Ty;
     reg          St;                      //状态转换信号
     reg [1:0]    CurrentState, NextState; //FSM状态寄存器
     //对状态进行编码
     parameter  S0=2'b00, S1=2'b01, S2=2'b11, S3=2'b10;

     /*======描述控制单元的状态转换,见图11.3.4 ======*/
     always@(posedge CLK, negedge RESET )
       begin:statereg
          if (~RESET)
               CurrentState<= S0;
          else
               CurrentState<= NextState;
       end //statereg
```

```
always@(S, CurrentState, Tl, Ts, Ty)
    begin: fsm
    case(CurrentState)
        S0: begin
              NextState = (Tl && S) ? S1 : S0;
              St = (Tl && S) ? 1:0;
            end
        S1: begin
              NextState = (Ty) ? S2 : S1;
              St = (Ty) ? 1:0;
            end
        S2: begin
              NextState = (Ts || ~S) ? S3 : S2;
              St = (Ts || ~S) ? 1:0;
            end
        S3: begin
              NextState = (Ty) ? S0 : S3;
              St = (Ty) ? 1:0;
            end
    endcase
 end   //fsm

/*===== 描述定时器 =====*/
always@(posedge CLK,negedge RESET )
    begin:counter
       if (~RESET)
              {TimerH,TimerL} = 8'h00;          //异步清零
    else if (St)
              {TimerH,TimerL} = 8'h00;          //同步置数
    else if ((TimerH == 5) & (TimerL == 9))     //保持不变
        begin      {TimerH,TimerL} = {TimerH, TimerL}; end
    else if (TimerL == 9)
        begin      TimerH = TimerH + 1;  TimerL = 0; end
    else
        begin      TimerH = TimerH;  TimerL = TimerL + 1; end
 end   //counter
 assign  Ty = (TimerH==0)&(TimerL==4);
 assign  Ts = (TimerH==2)&(TimerL==9);
 assign  Tl = (TimerH==5)&(TimerL==9);

/*===== 描述输出译码电路 =====*/
always @(CurrentState)
    begin
    case (CurrentState)
```

```
            S0: begin
                  {HG, HY, HR} = 3'b100; //主干道绿灯亮
                  {FG, FY, FR}  = 3'b001; //支干道红灯亮
                end
            S1: begin
                  {HG, HY, HR} = 3'b010; //主干道黄灯亮
                  {FG, FY, FR}  = 3'b001; //支干道红灯亮
                end
            S2: begin
                  {HG, HY, HR} = 3'b001; //主干道红灯亮
                  {FG, FY, FR}  = 3'b100; //支干道绿灯亮
                end
            S3: begin
                  {HG, HY, HR} = 3'b001; //主干道红灯亮
                  {FG, FY, FR}  = 3'b010; //支干道黄灯亮
                end
          endcase
      end
endmodule
```

图 11.3.8 交通灯控制系统的寄存器传输级描述

2. 结构级描述

前面介绍了用 HDL 中的过程语句，对交通灯控制系统的功能进行寄存器传输级描述。使用开发软件中的逻辑综合工具对其进行编译，就可以获得等效的门级电路描述。我们也可以根据构成电路的元器件，以及它们之间的连接关系，直接对交通灯控制系统进行结构级描述。

根据交通灯控制系统层次结构框图 11.3.2 可知，电路由三部分组成：控制单元、定时器和输出译码电路。按照数字系统自上而下分模块、分层次的设计方法，整个系统 HDL 描述分为三个层次，如图 11.3.9 所示。系统顶层的 HDL 描述是由控制单元、定时器和输出译码电路三部分的 HDL 描述组成。而控制单元又由底层的两个 *D* 触发器和 3 个 4 选 1 数据选择器构成。

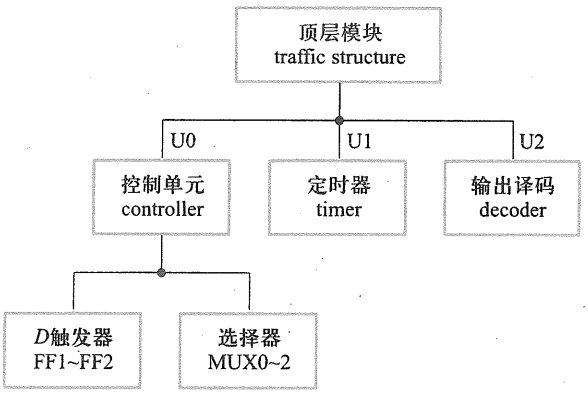


图 11.3.9 交通灯控制系统层次结构框图

下面是交通灯控制系统结构级描述。它由 6 个模块构成，可以分为 4 部分：

（1）顶层模块由 3 个子模块组成，其代码如图 11.3.10 所示。调用时端口采用了两种连接方式，U0、U2 是位置对应的调用方式，即调用时端口的排列顺序必须跟子模块定义时端口的排列顺序一致；U1 是端口名称对应的调用方式，端口顺序可任意排列，圆括号内部是顶层模块使用的端口名称，圆括号外部是子模块使用的端口名称。

```
module traffic_structure (
/*====声明输入/输出端口,见图11.3.2====*/
    input     CLK, S, RESET,
    output    HG, HY, HR, FG, FY, FR
    );
    //声明内部信号
    wire [3:0] TimerH, TimerL;      //模60计数器（BCD码）
    wire       Tl, Ts, Ty;          //定时器输出
    wire       St;                  //状态转换
    wire       Q1, Q0;              //控制器的状态

    Controller U0(St, Q1, Q0, S, Tl, Ts, Ty, RESET, CLK);
    Timer U1(.M60(Tl), .M30(Ts), .M5(Ty), .CLR(RESET), .St(St),
        .CP(CLK), .TimerH(TimerH), .TimerL(TimerL));
    Decoder U2( HG, HY, HR, FG, FY, FR, Q1, Q0);
endmodule
```

图 11.3.10 交通灯控制系统的顶层模块代码

（2）控制单元由 3 个模块组成，其代码如图 11.3.11 所示。它调用了 3 个选择器子模块和 2 个 *D* 触发器子模块，这几个子模块之间的连接是由图 11.3.5 决定的。

```
module Controller (
    output    St, Q1, Q0,
    input     S, Tl, Ts, Ty, CLR, CP
    );
    supply1   Vcc;      //高电平
    supply0   GND;      //低电平
    wire      Y2,Y1;    //内部信号
    Mux4x1    MUX2(Y2,GND,Ty,~Ty,Vcc,GND,{Q1,Q0});
    Mux4x1    MUX1(Y1,(Tl&S),Vcc,GND,~(Ts|~S),GND,{Q1,Q0});
    Mux4x1    MUX0(St,(Tl&S),Ty,Ty,(Ts|~S),GND,{Q1,Q0});
    UDFF      FF1(Q1,Y2,CLR,CP);
    UDFF      FF0(Q0,Y1,CLR,CP);
endmodule

/*===== 描述D触发器=====*/
module UDFF (
```

```
    output reg          Q,
    input               D,CLR,CP
    );
     always@(posedge CP, negedge CLR)
    if (~CLR)           Q <= 1'b0;
    else                Q <= D;
endmodule

/*===== 描述4选1数据选择器 =====*/
module Mux4x1 (
    output reg          Y,
    input               I0,I1,I2,I3,EN,
    input [1:0] SEL
    );
     always@(I0, I1, I2, I3, SEL, EN)
        if (EN)     Y = 1'b0;
        else
        case (SEL)
            2'b00:      Y=I0;
            2'b01:      Y=I1;
            2'b10:      Y=I2;
            2'b11:      Y=I3;
        endcase
endmodule
```

图 11.3.11 控制单元代码

（3）定时器代码如图 11.3.12 所示。这个模块由具有异步清零和同步置数功能的 BCD 码计数器组成。

```
module Timer (
    output          M60, M30, M5,
    output reg [3:0]  TimerH, TimerL,
    input           CLR, St, CP
    );
     always@(posedge CP, negedge CLR )
       begin:counter
         if (~CLR)
             {TimerH,TimerL} <= 8'h00;          //异步清零
         else if (St)
             {TimerH,TimerL} <= 8'h00;          //同步置数
         else if ({TimerH, TimerL} == 8'h59)    //保持不变
             begin {TimerH, TimerL} <= {TimerH, TimerL}; end
```

```
        else if (TimerL == 9)
            begin TimerH<= TimerH + 1'b1;  TimerL<= 4'b0000; end
        else
            begin TimerH<= TimerH;   TimerL<= TimerL + 1'b1; end
    end  //counter
    assign  M5 = (TimerH==0)&(TimerL==4);
    assign  M30 = (TimerH==2)&(TimerL==9);
    assign  M60 = (TimerH==5)&(TimerL==9);
endmodule
```

图 11.3.12 定时器代码

(4) 输出译码模块电路代码如图 11.3.13 所示。

```
module Decoder (
    output    HG, HY, HR, FG, FY, FR,
    input     Q1, Q0
    );
    assign    HG = ~Q1 & ~Q0;
    assign    HY = ~Q1 & Q0;
    assign    HR = Q1;
    assign    FG = Q1 & Q0;
    assign    FY = Q1 & ~Q0;
    assign    FR = ~Q1;
endmodule
```

图 11.3.13 输出译码电路代码

比较上述交通灯控制系统寄存器传输级和结构级的 Verilog 描述可知,寄存器传输级描述简单方便,不需要知道电路的具体结构。随着数字系统复杂性的增加,寄存器传输级描述更具优势。

3. 用 FPGA 器件实现交通灯控制系统

用 Intel FPGA 器件实现上述设计的过程如下:

(1) 在 Quartus 软件中建立一个新的工程项目,输入上述 HDL 代码,对设计项目进行编译。

(2) 对项目进行时序仿真,得到如图 11.3.14 所示的波形。

分析波形图可知,在 AB 段开始时,低电平有效的 RESET 信号使系统直接进入主干道绿灯亮、支干道红灯亮的初始状态。BC 段表示支干道有车,$S=1$,并且主干道绿灯需亮足 60 s,系统转换到 CD 段,主干道黄灯亮、支干道红灯亮,持续 5 s 后转换到 DE 段,此时主干道红灯亮、支干道绿灯亮。由于支干道一直有车,DE 段持续 30 s 后,系统才进入主干道红灯亮、支干道黄灯亮的 EF 段,5 s 后又回到主干道绿灯亮、支干道红灯亮的状态,如图中 FG 段。在主干道红灯亮、支干道绿灯亮的 HI 段,当支干道没有车 $S=0$ 时,系统立刻转换到下

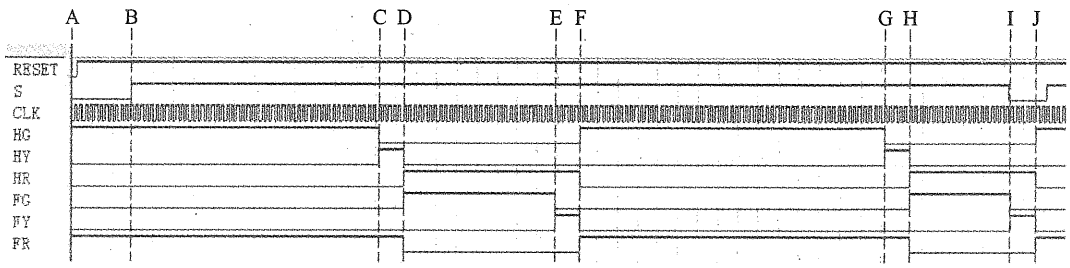


图 11.3.14 交通灯控制系统的仿真波形图

一个状态,并准备进入主干道绿灯亮、支干道红灯亮的状态。仿真结果完全符号设计要求。

(3) 选定目标器件,将输入、输出信号分配到器件相应的引脚上。然后重新编译设计项目,生成下载文件(文件后缀为.pof 或.sof)。

(4) 将下载文件写入到目标器件中,就是一块专用的交通灯控制电路。

复习思考题

11.3.1 用 Verilog HDL 描述数字系统时,可以分为几种形式?各种描述形式的特点是什么?

11.3.2 数字系统中,通常借助于什么编写控制单元的 Verilog HDL 描述?

11.3.3 比较交通灯控制系统的结构级描述和寄存器传输级描述,你得出什么结论?

11.4 数字密码锁

电子技术的发展使许多智能密码锁问世,如刷卡式、指纹类、按键式等。前面两类密码锁适应于保密要求高,仅供个人使用的场合,并且成本一般较高,其中 IC 卡有易丢失的缺点,在一定程度上限制了其使用范围。数字密码锁适用范围广,特别适合人员变动大,开锁钥匙不止一人拥有的场合。例如办公室、共享单车等。

下面以简单的数字密码锁为例,说明其设计实现方法。

11.4.1 数字密码锁的 ASM 图

设计一个键盘输入的 4 位十进制数字密码锁,只有当依次收到的 4 位十进制数码与规定的密码一致时,该锁才能被打开。

1. 明确系统总体方案

数字密码锁有密码设置和解锁两部分功能,通过安装在内部的 *MODE* 开关选择。当 *MODE* = **0** 时,通过键盘设置 4 位十进制密码。*MODE* = **1** 时,通过键盘操作解锁。数字密码锁框图如图 11.4.1 所示。

现假设已有键盘及处理电路,包括按键去抖电路、8421BCD 码编码器、时钟脉冲 *CLK* 产生电路及按键的同步化处理电路等。键盘有 0~9 十个数字键和“ * ”“#”共 12 个键。当按下一个数字键时,产生 4 位编码 *CODE*,同时 *PRESS* = **1**,表示有数字输入。“ * ”作为复位键 *RESET*,“#”相当于确认键 *ENTER*。每当按下一次按键都可以产生一个时钟脉冲 *CLK*。

要求设计数字锁的控制与处理单元电路，其输入是 *MODE* 以及键盘电路产生的信号 *CODE*、*PRESS*、*RESET*、*ENTER* 和 *CLK*。*RESET* 信号使系统进入初始状态（即上锁），当输入 4 位数码并按下 *ENTER* 后，若送入的 4 位数码与开锁密码一致，系统便输出 *OPEN* 信号打开锁；否则数字锁不开，并输出错误信号 *ERROR*。

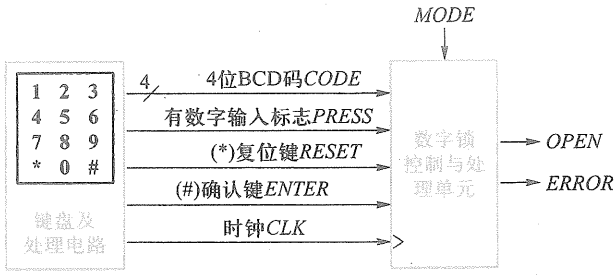


图 11.4.1 数字密码锁框图

2. 将系统划分为控制单元和处理单元

将数字密码锁划分为控制单元和处理单元两部分如图 11.4.2 所示。寄存器组 *RA* 和 *RB* 分别存放设置的密码和开锁时输入的数码。当 *MODE* = 0 时，通过键盘输入的 4 位十进制密码存入寄存器组 *RA* 中。当 *MODE* = 1 时，*RESET* 使控制单元发出的低电平有效的清零信号 $\overline{CLR}$ 将寄存器 *RB* 及计数器清零，在 *PRESS* 信号作用下，使处理单元接受由键盘按键电路依次产生的 4 组 *CODE* 数码，存入寄存器组 *RB* 中，并由 *ENTER* 确认。当有数字键按下即 *PRESS* 为 1，并且输入数码没有达到规定的位数时，控制单元输出的使能信号 *EN* 为 1。随后由数值比较器将寄存器组 *RA* 和 *RB* 保存的数码进行数值比较，比较结果 *AEQB* 送回到控制单元作为控制器状态转换条件之一。计数器用于记录数字键按下的次数，由位数比较器对计数器的输出与规定的密码位数进行比较，如果相等则输出控制信号 *BITEQ*。

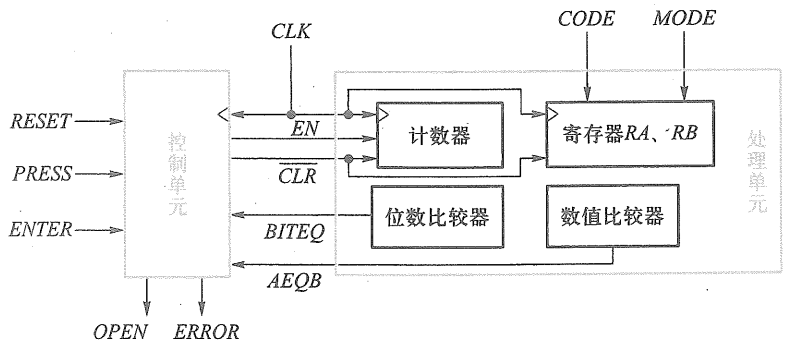


图 11.4.2 数字锁系统结构图

图 11.4.3 给出了处理单元逻辑结构图，其中 *RA* 或 *RB* 均由 4 个 4 位寄存器构成。*MODE* = 0 使寄存器组 *RA*0 ~ *RA*3 为工作状态，寄存器组 *RB*0 ~ *RB*3 为禁止状态，由键盘按键输入数码，并经过 8421BCD 码编码器（参见例 4.4.1）产生 4 个 4 位 *CODE* 输入，在时钟脉冲 *CLK* 的控制下，依次串行存入寄存器组 *RA* 中。

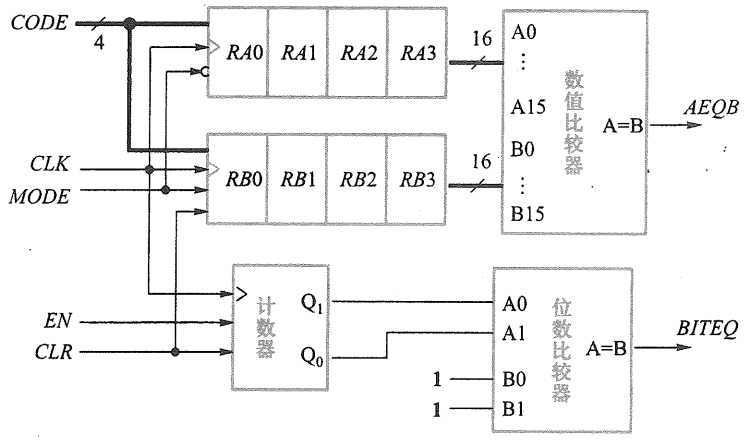


图 11.4.3 处理单元逻辑结构图

MODE=**1** 为解锁过程。首先 *RESET*(即 * 键)使控制单元发出 *CLR* 命令将计数器及寄存器组 *RB* 清零。再由键盘被按下的数字键产生的 4 位 *CODE* 并在 *CLK* 的控制下,依次串行存入寄存器组 *RB* 中。同时计数器在使能信号 *EN* 的控制下,对输入数码的次数进行计数。可见使能信号 *EN* 受输入 *PRESS* 的直接控制,是米利型输出。位数比较器则将计数器的输出与设定密码位数进行比较,当两者相等时,比较器输出 *BITEQ* 为 **1**,否则为 **0**。数值比较器将寄存器组 *RA* 与 *RB* 的对应位数值相比较,两者相同时输出 *AEQB* 为 **1**,不同时为 **0**。控制单元根据处理电路反馈回来的信息 *BITEQ* 和 *AEQB*,决定处理器下一步的动作。

这里我们重点介绍解锁过程,假设 4 位十进制数密码已经存入 *RA*0~*RA*3 中。

3. 确定控制单元的算法,画出 ASM 图

根据上述分析得出解锁过程控制单元的 ASM 图,如图 11.4.4 所示,其中 S_0为初始状态,S_1为接收数码状态,并且每接收一次数码,计数器 *C* 加 1,S_2为准备开锁状态,S_3为开锁状态,S_4为错误状态。

异步复位信号 *RESET* 使得控制单元进入初始状态 S_0,在该状态时,控制单元发出低电平有效的$\overline{CLR}$命令使计数器 *C* 及寄存器组 *RB* 清零。没有数码输入时,即 *PRESS* 为 **0**,系统保持在 S_0状态,有数码输入时 *PRESS* 为 **1**,系统进入 S_1状态。

在 S_1状态下,此时若开锁信号 *ENTER* 为 **1**,控制单元进入错误状态 S_4。*ENTER* 为 **0** 时,同时没有数码输入即 *PRESS* 为 **0**,控制单元保持在 S_1状态,如果 *PRESS* 为 1,接着判断位数比较器输出 *BITEQ* 的结果,*BITEQ* 为 **0** 时,使能信号 *EN* 为 **1**,同时计数器 *C* 加 1。由于使能信号 *EN* 为米利型输出,因此用椭圆形条件输出框表示。随后返回 S_1状态,继续接受数码输入。*BITEQ* 为 **1** 时,系统转入 S_2状态。

S_2为准备开锁状态,在此状态下,对输入的 4 位密码的数值与设置密码的数值进行比较,若不等则 *AEQB*=**0**,控制单元进入错误状态 S_4并输出 *ERROR* 命令。若相等,则 *AEQB* 为 **1**,再输入数码,即 *PRESS* 为 **1** 时,控制单元进入错误状态 S_4,否则等待确认信号 *ENTER*,*ENTER* 为 **0** 时,返回 S_2状态,继续等待。*ENTER* 为 **1** 时,控制单元由 S_2状态进入开锁状态 S_3,并输出 *OPEN* 开锁命令。

在上述过程中，任何一次送入的数码与开锁密码的数值不一致，或者输入数码（即 $PRESS=1$）和 $ENTER$ 按键使用的顺序与规定的不符，都将使控制单元进入错误状态 S_4，同时控制单元发出错误信息 $ERROR$。

根据图 11.4.4 所示的数字密码锁解锁过程控制单元的 ASM 图，得出状态转换图如图 11.4.5 所示。ASM 图中的状态框与状态转换图中的状态相对应，判断框中的条件是状态转换的输入条件，状态框中的输出与状态转换图中的穆尔型输出相对应，写在圆圈内。条件输出框中的输出与米利型输出相对应，写在箭头线旁。例如在接收数码 S_1 状态，根据输入信号的不同取值，决定下一个状态。如果 $ENTER$ 为 1，进入 S_4 状态。如果条件 $\overline{ENTER}\cdot PRESS\cdot BITEQ$ 为 1，系统转到 S_2 状态。如果 $\overline{ENTER}\cdot\overline{PRESS}$ 为 1 时，EN 为 0，系统保持在 S_1 状态，计数器也处于保持状态。如果 $\overline{ENTER}\cdot PRESS\cdot\overline{BITEQ}$ 为 1，则 EN 为 1，同时计数器 C 加 1，系统仍处于 S_1 状态。

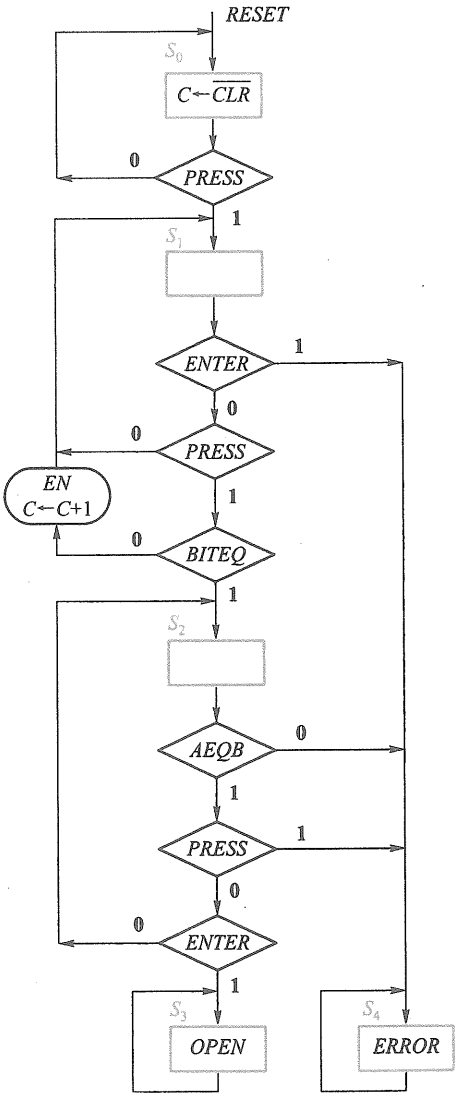


图 11.4.4 解锁过程控制单元 ASM 图

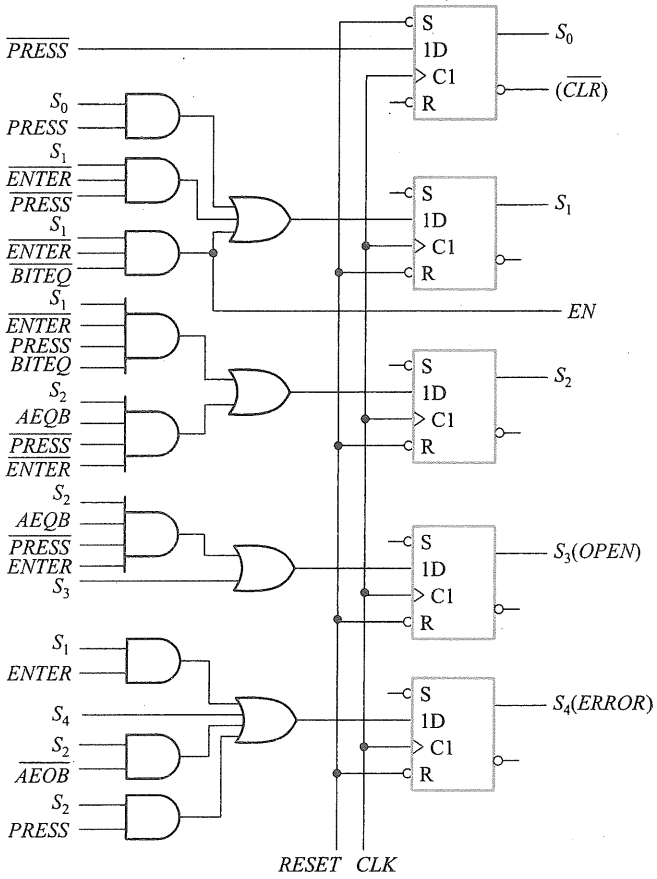


图 11.4.6 数字锁控制单元电路

比较交通灯控制系统和数字密码锁两种控制单元的设计方法可知，交通灯控制系统采用的状态编码方法所用的触发器少，但组合电路相对复杂。数字密码锁采用一个触发器对应一个状态，所用的触发器较多，但其优点是：任意两个状态转换时，只需变化两位，可以简化组合逻辑电路，并能降低电路的功耗。对于寄存器数量多，而逻辑门相对缺乏的 FPGA 器件，采用后一种方法可以提高器件资源的利用率，提高电路的工作速度和可靠性。

11.4.3 数字密码锁 Verilog 建模

本节使用 Verilog HDL 对图 11.4.2 所示的简单数字密码锁进行建模。根据上述设计过程，采用自下而上的方式进行描述。

1. 处理单元描述

处理单元(图 11.4.3)由计数器、寄存器和比较器 3 个子模块构成，其代码如图 11.4.7 所示。其中，Regn 为具有同步并行置数功能的寄存器子模块，改变参数 n 的值，就可以得到位宽不同的寄存器。Comparator 为判断数值是否相等的比较器子模块，其参数也是可以修改的。

```
// === 2 位递增计数器 ===
module Upcount (
    input CLR,EN,CP,
    output reg [1:0] Q
    );

    always @(posedge CP or negedge CLR)
    begin
     if(~CLR)
          Q <= 2'b00;
     else if(~EN)
          Q <= Q;
     else if (Q==2'b11)
          Q <= 2'b11;
     else
          Q <= Q+1'b1;
    end

endmodule
```

```
// === n 位并行寄存器 ===
module Regn#(parameter n = 8)(
     input [n-1:0] Pdata,
     input   LD, CP,
     output reg [n-1:0] Q
     );
    always @(posedge CP)
    begin
     if (LD)      Q <= Pdata;
     else         Q <= Q;
    end
endmodule
// === 数值比较器 ===
module Comparator#(parameter n) (
     input [n-1:0] A, B,
     output EQU
     );
    assign EQU = (A == B);
endmodule
```

图 11.4.7 计数器、寄存器和数值比较器子模块

实例化引用上述 3 个子模块可以组成数字密码锁的处理单元，其代码如图 11.4.8 所示。其中，寄存器子模块 Regn 的参数修改为 4，即为 4 位并行置数寄存器。第 1 次实例引用比较器子模块 Comparator 时，将其参数修改成 16，以便于对输入的 4 位 BCD 码（即 16 位二进制数）进行比较；第 2 次引用子模块 Comparator 时，将其参数修改成 2，比较输入的 BCD 码位数是否为 3。

```
module Lock_datapath (
    output          AEQB, BITEQ,
    input   [3:0]   CODE,       //按键的 BCD 码
    input           PRESS,      //1 : 有数字键按下
    input           CLK, RESET,
    input           MODE        //0:设置密码 ; 1 : 解锁
    );
    wire [3:0] QA0,QA1,QA2,QA3;
    wire [3:0] QB0,QB1,QB2,QB3;
    Regn #(.n(4)) RA0(.Pdata(CODE), .LD(!MODE & PRESS), .CP(CLK), .Q(QA0));//设置密码
    Regn #(.n(4)) RA1(.Pdata(QA0), .LD(!MODE & PRESS), .CP(CLK), .Q(QA1));
    Regn #(.n(4)) RA2(.Pdata(QA1), .LD(!MODE & PRESS), .CP(CLK), .Q(QA2));
```

```
    Regn #(.n(4)) RA3(.Pdata(QA2), .LD(!MODE & PRESS), .CP(CLK), .Q(QA3));
    Regn #(.n(4)) RB0(.Pdata(CODE), .LD(MODE & PRESS), .CP(CLK), .Q(QB0));//输入数字
    Regn #(.n(4)) RB1(.Pdata(QB0), .LD(MODE & PRESS), .CP(CLK), .Q(QB1));
    Regn #(.n(4)) RB2(.Pdata(QB1), .LD(MODE & PRESS), .CP(CLK), .Q(QB2));
    Regn #(.n(4)) RB3(.Pdata(QB2), .LD(MODE & PRESS), .CP(CLK), .Q(QB3));
    Comparator #(.n(16)) RAB0(
        .A({QA0,QA1,QA2,QA3}),
        .B({QB0,QB1,QB2,QB3}),
        .EQU(AEQB)
        );
    wire [1:0]  CNT;
    Upcount CNT_inst(
        .CLR(RESET),
        .EN(PRESS & MODE & !BITEQ),
        .CP(CLK),
        .Q(CNT)
        );
    Comparator #(.n(2)) RAB1(
        .A(2'b11),
        .B(CNT),
        .EQU(BITEQ)
        );
endmodule
```

图 11.4.8 密码锁的处理单元

2. 控制单元描述

数字密码锁的控制单元可以根据图 11.4.4(或图 11.4.5)进行描述,其代码如图 11.4.9 所示。这里用两个 **always** 描述该同步时序电路。第一个 **always** 说明异步复位 RESET = **0** 时,使系统进入初态 S0,并且在时钟 CLK 的上升沿进行状态转换。第二个 **always** 用多路分支语句 **case** 描述了一个组合逻辑电路,说明控制单元的 5 种状态,以及电路从现态转换到次态的条件。

```
module Lock_control(
    output reg  OPEN, ERROR, //EN, CLR,
    input       CLK, RESET, PRESS, ENTER,
    input       BITEQ, AEQB,
    output reg [4:0]  CurrentState, NextState   //FSM 状态寄存器 (方便仿真添加的)
    );
```

```
    //对状态进行编码
    parameter S0=5'b00001, S1=5'b00010, S2=5'b00100, S3=5'b01000, S4=5'b10000;
   /*=====控制单元状态转换的描述 ( 图 11.4.4 或图 11.4.5 ) =====*/
   always @(posedge CLK or negedge RESET )
   begin:statereg
    if (~RESET)
        begin      CurrentState<= S0;    end
    else if(PRESS|ENTER)
        begin      CurrentState<= NextState;   end
   end //statereg
   always @(PRESS or ENTER or AEQB or BITEQ or CurrentState )
   begin: fsm        //现态转次态的条件
    OPEN = 1'b0;
    ERROR = 1'b0;
    case(CurrentState)
        S0: begin   NextState = PRESS ? S1 : S0;   end
        S1: begin
            if (ENTER) NextState = S4;
            else if (~PRESS) NextState = S1;
            else if (~BITEQ) NextState = S1;
            else NextState = S2;
            end
        S2: begin
            if (~AEQB) NextState = S4;
            else if (PRESS) NextState = S4;
            else if (~ENTER) NextState = S2;
            else NextState = S3;
            end
        S3: begin
            OPEN = 1'b1;
            NextState = S3;
            end
        S4: begin
            ERROR = 1'b1;
            NextState = S4;
            end
        default: NextState = S0;
    endcase
   end
endmodule
```

图 11.4.9 密码锁的控制单元

3. 密码锁整体电路的描述

密码锁的顶层代码如图 11.4.10 所示。它实例引用了上述的处理单元和控制单元两个子模块,其端口采用名称关联的方式进行映射,这是实际项目中值得推荐的编码风格。

```
module Lock_Password (
    output          OPEN, ERROR,    //EN, CLR,
    input [3:0]     CODE,           //0~9对应的BCD码
    input           PRESS,          //1: 数字键被按下
    input           ENTER,          //按 # 键, 开锁
    input           CLK,            //系统时钟
    input           RESET,          //按 * 键, 上锁(系统复位)
    input           MODE,           //0:设置密码; 1: 解锁
    output          AEQB,BITEQ,     //方便仿真添加的输出信号
    output [4:0] CurrentState, NextState //FSM 状态寄存器
    );
    Lock_datapath U0(
        .AEQB(AEQB),
        .BITEQ(BITEQ),
        .CODE(CODE),
        .PRESS(PRESS),
        .CLK(CLK),
        .RESET(RESET),
        .MODE(MODE) //0:设置密码; 1: 解锁
        );
    Lock_control U1(
        .OPEN(OPEN),
        .ERROR(ERROR),
        .RESET(RESET),
        .CLK(CLK),
        .PRESS(PRESS),
        .ENTER(ENTER),
        .BITEQ(BITEQ),
        .AEQB(AEQB),
        .CurrentState(CurrentState),
        .NextState(NextState)
        );
endmodule
```

图 11.4.10 密码锁顶层代码

4. 时序仿真

在 Quartus 软件中建立一个新的工程项目,输入上述 HDL 代码,对设计项目进行编译。然后对项目进行时序仿真,得到如图 11.4.11 所示的波形。

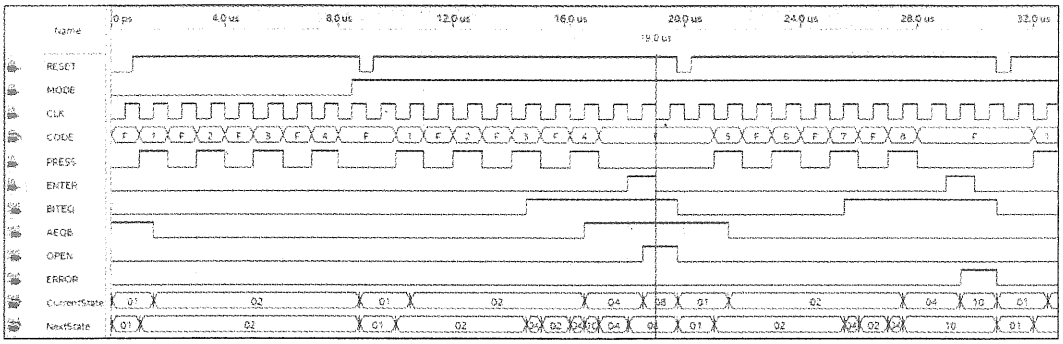


图 11.4.11 数字密码锁的仿真波形图

刚开始,MODE=0,设置 4 位 BCD 密码为 1234。在 9 μs 附近,复位信号 RESET=0,系统复位(即上锁);接着,进入解锁过程。分别在 10 μs、12 μs、14 μs、16 μs 有数字键被按下,输入的 4 位数码为 1234,在 18 μs 按下开锁确认键(ENTER=1),经过比较,送入的数码与设置密码相同(AEQB=1),所以在 18.5 μs 时,系统产生高电平开锁信号(OPEN=1)。在 20 μs附近,RESET=0,系统再次复位;接着输入 5678 这 4 位数码,虽然输入的数码位数正确(BITEQ=1),但由于送入的数码与设置的密码数值不一致(AEQB=0),系统发出错误信息 ERROR。同样,可以测试不按按照规定流程操作的情况,例如,输入了 5 位 BCD 码(或者输入位数不够时,按下确认键 ENTER)时,系统也会产生 ERROR 信号。

为便于分析开锁过程中系统状态之间的转换,图中给出了中间信号的波形图,具体含义这里不再赘述,读者可自行分析。

复习思考题

11.4.1 用 Verilog HDL 描述数字系统时,可以分为几种形式?各种描述形式的特点是什么?

11.4.2 数字系统中,通常借助于什么编写控制单元的 Verilog HDL 描述?

11.4.3 比较交通灯控制系统的结构级描述和寄存器传输级描述,你得出什么结论?

小 结

- 数字系统从结构上分为数据处理单元和控制单元。因此,数字系统中的二进制信息也分成数据信息和控制信息两大类。
- 数字系统的设计方法有两种,自下而上的设计方法和自上而下的设计方法。现代数字系统的设计常采用自上而下的设计方法。
- 数字系统自下而上的设计是设计者根据经验将数字系统按逻辑功能划分成若干子模块,一直分到这些子模块可以用经典的方法和标准的逻辑功能部件进行设计,最后将整个系统安装、调试达到设计要求。
- 自上而下的设计方法是将系统的设计分层次、分模块进行。通常将系统分为控制器和处理器两部分。如果控制器和处理器仍比较复杂,可以在它们内部多重地分解成几个子模块进行

逻辑设计，给出实现系统的硬件和软件描述，最后得到所要求的数字系统。

• ASM 图是数字系统自上而下设计方法中经常采用的设计工具，它是指数字系统控制算法的流程图，可以将语言描述的设计问题变成时序流程图的描述，从而获得状态函数或输出函数，得出相应的硬件电路。

• 数字系统的实现方法有多种，采用可编程逻辑器时，用户可以将设计的数字系统，用图形输入方式或硬件描述语言方式或两者的混合方式，输入到开发系统，生成编程文件，将编程文件写到芯片中。Verilog HDL 是常用的硬件描述语言，采用 Verilog HDL 语言可以简化数字系统的实现过程。

• 以交通信号灯控制系统和数字密码锁为例介绍数字系统中控制单元和处理单元（数据通路）的设计过程。包括系统方案的确定，控制单元和数据通路（处理单元）的划分以及两者之间的控制、状态反馈等信号传输的重要内容。最后用 Verilog HDL 语言对各数字系统建模。

习　题

11.1　数字系统概述

11.1.1　数字电路与数字系统的根本区别是什么？大容量存储器属于数字电路还是数字系统？

11.1.2　数字系统自上而下的设计步骤是什么？

11.1.3　数字系统的实现有哪几种方法？各种方法的特点是什么？

11.2　算法状态机

11.2.1　初始状态为 T_0 的数字系统，有两个控制信号 X 和 Y，当 $XY=10$ 时，寄存器 R 加 1，系统转到第二个状态 T_1。如果 $XY=01$ 时，寄存器 R 清零，同时系统从 T_0 转到第三个状态 T_2。试画出该数字系统的 ASM 图。

11.2.2　一个数字系统的数据处理单元由触发器 E 和 F、4 位二进制计数器 A 以及必要的门电路组成。计数器的各位为 A_4、A_3、A_2、A_1。系统开始处于初始状态，当信号 $S=0$ 时，系统保持在初始状态；当 $S=1$ 时，计数器 A 和触发器 F 清零。从下一个时钟脉冲开始，计数器进行加 1 计数，直到系统操作停止。A_4 和 A_3 的值决定了系统的操作顺序。

当 $A_3=0$ 时，触发器 E 清零，计数器继续计数。

当 $A_3=1$ 时，触发器 E 置 1，并检测 A_4，$A_4=0$ 时，继续计数；$A_4=1$ 时，触发器 F 置 1，并停止计数，回到系统初始状态。

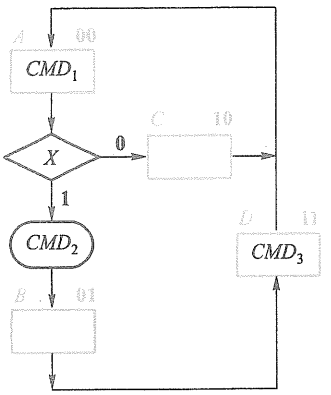
（1）试画出该系统的 ASM 图；

（2）画出该系统控制单元的状态图，并用 D 触发器及必要的门电路设计控制单元。

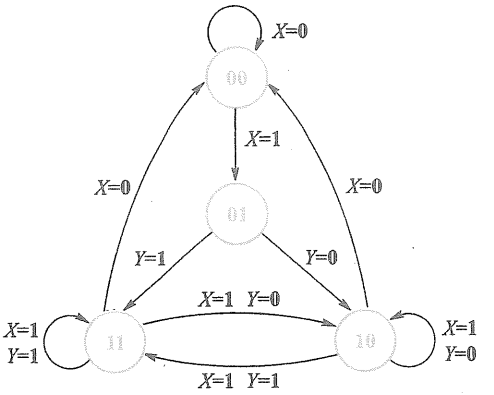
11.2.3　试画出一个序列检测器电路的 ASM 图及状态图。该电路的功能是：当输入的串行数据为 **0101**（从左到右）时，输出为 1，否则输出为 0。要求考虑序列重叠性，当输入为 **010101**，相当于出现两个 **0101** 序列。

11.2.4　用一个触发器一个状态方法实现图题 11.2.4 所示的 ASM 图。

11.2.5　一个数字系统控制单元的状态图如图题 11.2.5 所示，试画出等效的 ASM 图（状态框是空的），并用 D 触发器和数据选择器实现控制单元电路。

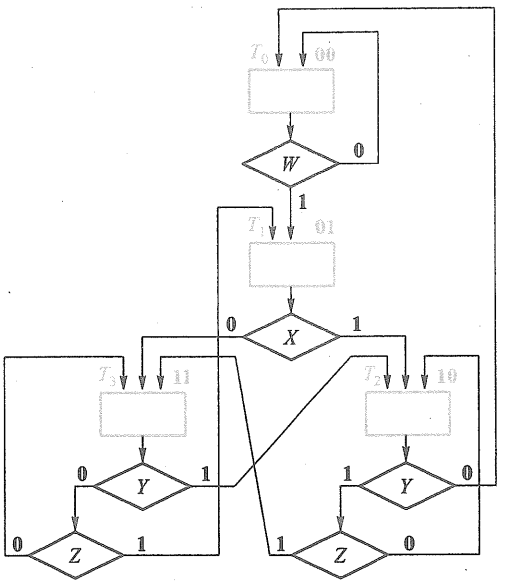


图题 11.2.4



图题 11.2.5

11.2.6 数字系统的 ASM 图如图题 11.2.6 所示，试设计系统的控制单元电路。



图题 11.2.6

11.3 交通信号灯控制系统

11.3.1 设计一个可以控制计数功能的计数器，当 $X=0$ 时，计数器以二进制数序列 000、001、010、011、100、101、110、111 计数并重复。当 $X=1$ 时，计数器以格雷码序列 000、001、011、010、110、111、101、100 计数并重复，要求：

(1) 画出计数器的状态图和 ASM 图；

(2) 用 Verilog HDL 描述系统的工作过程，并给出仿真结果。

11.3.2 设计一个用移位相加实现的乘法器，乘数与被乘数均为同步输入的 4 位无符号二进制数。要求：

(1) 确定乘法器算法，画出乘法器系统方案框图；

(2) 画出系统控制器的 ASM 图。用一个触发器对应一个状态的方法设计控制电路;

(3) 用 Verilog HDL 描述系统的工作过程,并给出仿真结果。

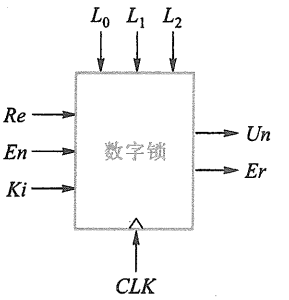
11.4 数字密码锁

11.4.1 设计一个简单的 3 位二进制数字密码锁,控制房门的打开,当接收到的串行输入数码与原设定的密码相同时,发出开锁信号,锁被打开。要求:

(1) 画出系统的 ASM 图;

(2) 画出系统的状态转换图;

(3) 用 Verilog HDL 描述其工作过程,并给出仿真结果。



图题 11.4.1

提示:数字锁系统的示意图如图题 11.4.1 所示,由于开锁过程比较简单,所以没有划分控制单元和处理单元。L_0、L_1、L_2 用于从低位到高位设置密码,另外三个输入端 Re、En、Ki 中,Re 是复位端,当 $Re=1$ 时,系统复位进入初始状态;En 是控制开关,Ki 是数据输入端,当按一下 En 键使 $En=1$ 时,从 Ki 端输入一个数码,再按一下 En 键,再送一个数码,直到将三个数码送完为止。每输入一个数据都要同原设定的密码比较,依次从低位到高位进行。若相等则准备接收下一位数码,若不相等,系统应进入错误状态。输入数码的位数也是开锁的条件。当输入数码的位数和位值与相应密码都相等时,系统发出开锁信号 $Un=1$,锁被打开,否则 $Er=1$ 表示开锁过程错误。为保密起见,中间错误状态不显示,并且不能返回初态,直到三个数码全部送完为止。

11.4.2 设计一个体育比赛中常用的数字跑表。它通过两个按键来控制计时开始和停止,一个是清零控制按键 Reset(简称 R 键),另一个是 Start/Stop 控制按键(简称 S 键),其工作过程如下:

开始时,按一下 R 键使跑表进入零初始状态。在 R 键无效的时候,按一下 S 键则计时器开始计时,在此计时状态下,按一下 S 键暂停计时,再按一下 S 键则继续计时,并且这一过程可由 S 键控制重复进行。如果在暂停状态按一下 R 键,跑表被清零。

如果在计时状态下,按一下 R 键则暂停计时,再按一下 R 键则继续计时,并且这一过程也可由 R 键控制重复进行。当按 R 键使计时暂停时,再按 S 键不起作用。

要求跑表的计时范围为 0.01 s~59 min 59.99 s,计时精度为 10 ms;跑表的输出能够直接驱动共阳极 7 段数码管显示。输入信号的频率为 100 Hz。

(1) 画出跑表的结构框图;

(2) 画出控制单元的 ASM 图及状态图;

(3) 用 Verilog HDL 描述跑表的功能,并给出仿真结果。

文档 11.2:
第 11 章部分
习题答案

附录 A 电气简图用图形符号——二进制逻辑单元[①](GB/T 4728.12—2008)简介

《电气简图用图形符号——二进制逻辑单元》(GB/T 4728.12—2008)是由国家标准化管理委员会颁布的用于绘制二进制逻辑单元电路的符号标准。该标准是绘制电路图、功能图、概略图等功能性简图的依据,是电气技术的工程语言。

A.1 二进制逻辑单元图形符号的组成

二进制逻辑单元图形符号是由一个方框(或几个方框的组合)和标注其上的一个或多个限定性符号组成,如图 A.1.1 所示。图中 ** 为总限定符号,用来说明该逻辑单元所完成的逻辑功能。* 表示与输入和输出有关的限定符号。方框的大小和长宽比可以是任意的。标注在方框外的字母和其他字符不是符号的组成部分,仅用于区分多个输入和输出。通常输入线画在方框的左边或上边;输出线画在方框的右边或下边,以使信息流方向是从左到右,或从上到下。

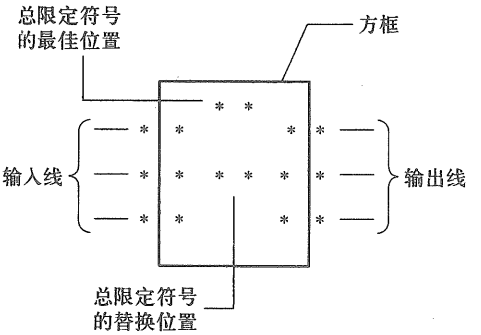


图 A.1.1 符号的组成

除上述基本单元框外,为减小图形所占的篇幅,还可以使用公共控制框和公共输出单元框。图 A.1.2 所示为公共控制框举例。图中如果输入 a 不加任何限定符号时,表示它同时加

① 应为二值逻辑单元。

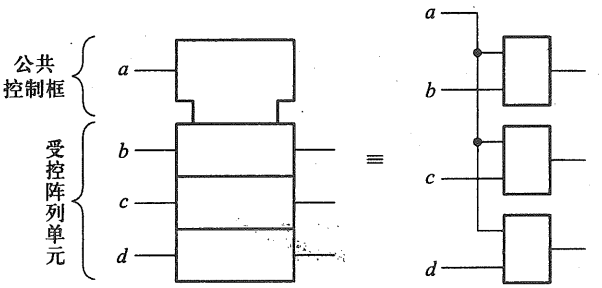


图 A.1.2 公共控制框

到公共控制框下面每个受控的阵列单元上。

图 A.1.3 所示为公共输出单元框举例。图中公共输出单元的输出 e 与阵列中每一个单元都有连接。此外,也可以有其他的输入,如输入信号 a,即 b、c 和 a 同时加到公共输出单元框上。公共输出单元的逻辑功能需另外标注限定符号加以说明。

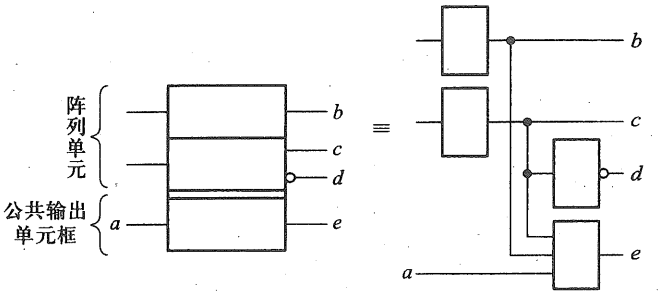


图 A.1.3 公共输出框

为了缩小图形所占的篇幅,可以将相邻的单元框组合在一起。组合的方式有邻接法和镶嵌法。在有邻接单元或镶嵌单元的符号中,如果单元框之间的公共线是沿着信息流方向,就表明这些单元之间无逻辑连接;如果单元框之间的公共线是垂直于信息流方向,则表明单元之间至少有一种逻辑连接,如图 A.1.4(a)、(b)所示。

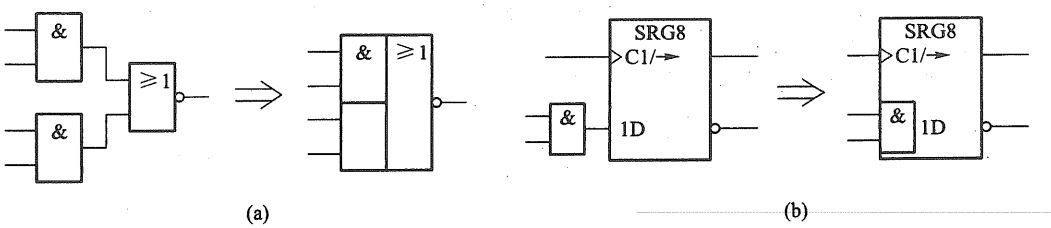


图 A.1.4 邻接法与镶嵌法

(a) 邻接法 (b) 镶嵌法

A.2 限定性符号

限定性符号标注在逻辑单元图形符号的方框中,用以表明该单元的逻辑功能,或者是输

入或输出的物理特性或逻辑特性。限定性符号大体可以分为总限定符号，以及与输入、输出有关的限定符号等。

1. 总限定符号

总限定符号用来表示逻辑单元总的逻辑功能。这里的逻辑功能是指符号框内部输入、输出之间的逻辑关系。表 A.2.1 列出了部分总限定符号及其所表达的逻辑功能。

表 A.2.1 部分总限定符号

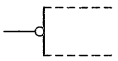
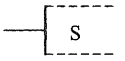
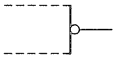
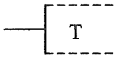
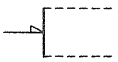
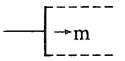
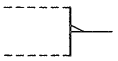
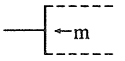
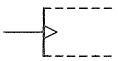
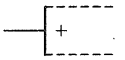
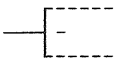
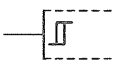
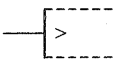
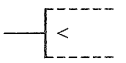
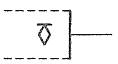
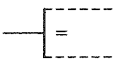
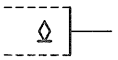
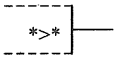
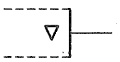
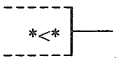
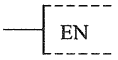
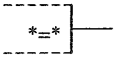
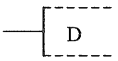
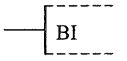
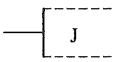
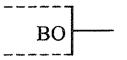
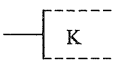
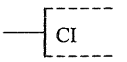
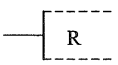
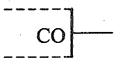
符号	说明	符号	说明
&	与	CPG	先行(超前)进位
≥1	或	MUX	多路选择
1	缓冲	DX 或 DMUX	多路分配
=1	异或	X/Y	编码、译码或代码转换
=	逻辑恒等(所有输入状态相同时，输出才为 1 状态)	I=0	触发器的初始状态为 0
2k	偶数(输入 1 的个数为偶数时，输出为 1 状态)	I=1	触发器的初始状态为 1
2k+1	奇数(输入 1 的个数为奇数时，输出为 1 状态)		不可重复触发的单稳态电路
▷	缓冲/驱动		可重复触发的单稳态电路
* ∫∫	磁滞特性		非稳态电路
Σ	加法运算	SRGm	m 位的移位寄存
P-Q	减法运算	CTRm	循环长度为 2^m 的计数
Π	乘法运算	ROM **	只读存储
COMP	数值比较	PROM **	可编程只读存储
ALU	算术逻辑单元	RAM **	随机存储

注：* 号用表明单元逻辑功能的总限定符号代替；** 号用“地址×位数”来代替。

2. 与输入、输出和其他连接有关的定性符号

与输入、输出有关的限定符号用来说明相应输入端或输出端所具有的逻辑功能或物理特性，如表 A.2.2 所示。

表 A.2.2 与输入、输出有关的限定符号

符号	说明	符号	说明
	逻辑非,在输入端		*S* 输入
	逻辑非,在输出端		*T* 输入
	低有效输入极性指示符		移位输入、从左到右或从顶到底
	低有效输出极性指示符		移位输入、从右到左或从底到顶
	动态输入		加计数输入
	延迟输出		减计数输入
	具有磁滞特性的输入		数值比较器的“大于”输入
	开路输出(例如集电极开路、发射极开路、漏极开路、源极开路)		数值比较器的“小于”输入
	H 型开路输出(输出高电平时输出阻抗低)		数值比较器的“等于”输入
	L 型开路输出(输出低电平时输出阻抗低)		数值比较器的“大于”输出
	三态输出		数值比较器的“小于”输出
	使能输入		数值比较器的“等于”输出
	*D* 输入		运算单元的借位输入
	*J* 输入		运算单元的借位输出
	*K* 输入		运算单元的进位输入
	*R* 输入		运算单元的进位输出

相邻单元的方框可以采用如表 A.2.3 所示的内部连接符号表示。

表 A.2.3 内部连接符号

符号	说明	符号	说明
	内部连接(信号流方向从左到右)		具有动态特性的内部连接
	内部连接(信号流方向从左到右，不混淆逻辑数量时使用)		具有逻辑非和动态特性的内部连接
	从右到左信号流的内部连接		内部输入(左边)
	具有逻辑非的内部连接		内部输出(右边)

表 A.2.4 给出了非逻辑连接和信号流指示符号。其中，非逻辑连接符号表示逻辑图中，如果信号线不是逻辑信号的连接线，用信号线上加“×”表示。

原则上，信号流的方向是从左到右、从上到下。如果不符合此规则或信号流方向不明显时，应在信息线上标出指示信号流方向的箭头，如表 A.2.4 所示。

表 A.2.4 非逻辑连接和信号流指示符

符号	说明
	非逻辑连接，在左边示出
	单向信号流
	双向信号流

A.3 关联标注法

有时只使用限定性符号不能充分说明逻辑单元的各输入之间、各输出之间或输入与输出之间的关系，为此，采用关联标注法加以补充说明。

关联标注法引用“影响”和“受影响”两个术语，用以表示信号之间内在的“影响”和“受影响”的关系。例如，在第 5 章锁存器和触发器中涉及关联标注法，如图 A.3.1 所示 D 锁存器符号。其中输入 D 受到输入 E 的影响。当 $E=1$ 时，输出 $Q=D$；当 $E=0$ 时，输入 D 不影响输出 Q。在符号框中用 D 前面加 1，即 1D 表示受 C1 的影响。

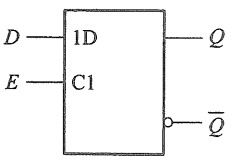


图 A.3.1 D 锁存器的关联标注法

关联标记的标注方法：

(1) 用一个表示内在关系的字母和后面加标识序号来表示影响其他输入(或输出)。

(2) 用相同的标识序号来标记每一个受该“影响输入”影响的输入(或输出)。

(3) 若起影响作用的是“影响输入(或输出)”内部逻辑状态的补状态，则应在受“影响

输入(或输出)”的标识序号上面加一个横线。

(4) 若一个输入(或输出)受两个以上“影响输入”的影响时,则所有“影响输入”的标识序号都应出现在“受影响输入”的标记中,并以逗号隔开。

这里列出了 10 种关联类型,如表 A.3.1 所示,其中:

与关联、**或**关联和**非**关联用来注明输入和(或)输出之间的逻辑关系。

互连关联是用来表示一个输入或输出把其逻辑状态强加到另一个或多个输入和(或)输出上。

控制关联用来标识时序单元的定时输入或时钟输入,并指出受它控制的输入。

置位关联和复位关联用来规定当 R 输入和 S 输入均处在它们的内部 **1** 状态时,基本触发器的内部逻辑状态。

使能关联用来标识使能输入并指出由它控制的输入和/或输出(例如,哪些输出呈现高阻抗状态)。

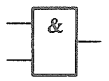
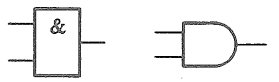
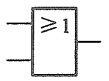
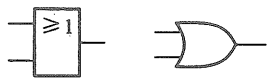
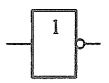
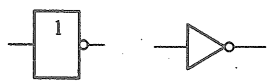
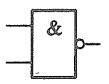
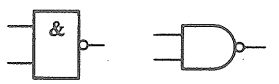
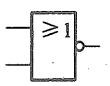
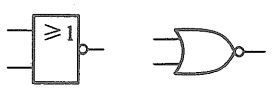
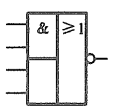
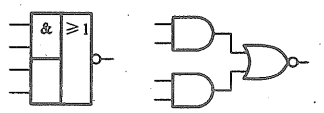
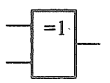
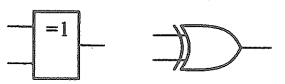
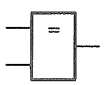
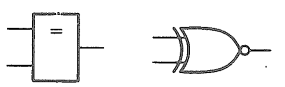
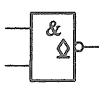
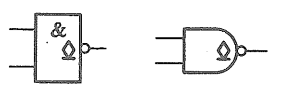
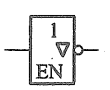
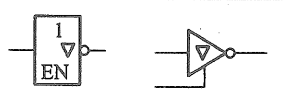
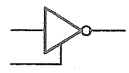
方式关联用来标识选择单元操作方式的输入,并指出取决于该方式的输入和/或输出。

地址关联用来标识存储器的地址输入。

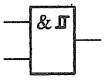
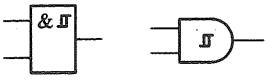
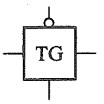
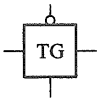
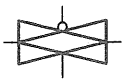
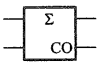
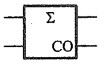
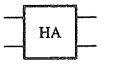
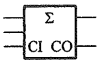
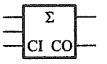
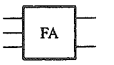
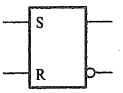
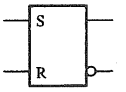
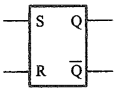
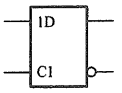
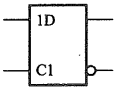
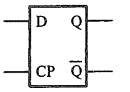
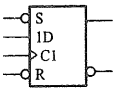
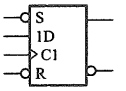
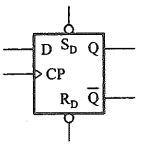
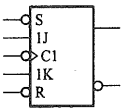
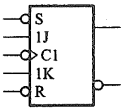
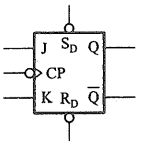
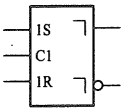
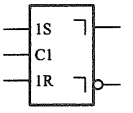
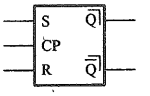
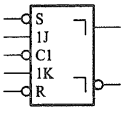
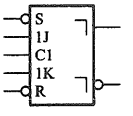
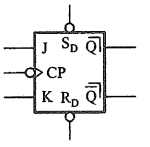
表 A.3.1 关 联 类 型

关联类型	关联符号	对“受影响输入”或“受影响输出”的影响	
		当“影响输入(出)”=**1** 时	当“影响输入(出)”=**0** 时
地址	A	允许动作(已选地址)	禁止动作(未选地址)
控制	C	允许动作	禁止动作
使能	EN	允许动作	禁止“受影响输入”动作、置开路或三态输出于外部高阻抗状态,置其他输出于 **0** 状态
与	G	允许动作	置 **0** 状态
方式	M	允许动作(已选方式)	禁止动作(未选方式)
非	N	求补状态	不起作用
复位	R	“受影响输出”复位	不起作用
置位	S	“受影响输出”置位	不起作用
或	V	置 **1** 状态	允许动作
互连	Z	置 **1** 状态	置 **0** 状态

附录 B
常用基本逻辑符号对照表

名　称	国标符号	IEEE 标准符号	曾用逻辑符号
与门			
或门			
非门			
与非门			
或非门			
与或非门			
异或门			
同或门			
漏极(集电极) 开路与非门			
三态输出 非门			

续表

名 称	国标符号	IEEE 标准符号	曾用逻辑符号
带施密特触发特性的与门			
传输门			
半加器			
全加器			
SR 锁存器（触发器）			
逻辑门控 *D* 锁存器			
上升沿触发 *D* 触发器			
下降沿触发 *JK* 触发器			
正脉冲触发 *SR* 触发器			
负脉冲触发 *JK* 触发器			

附录 C 编写 Verilog HDL 测试模块

C.1 测试模块的基本结构

在前面有关章节中，我们重点讨论了可综合的 Verilog HDL 语言结构，并对组合逻辑电路和时序逻辑电路进行描述，本节重点讨论如何使用 Verilog HDL 编写测试模块，以便对设计模块进行逻辑功能的测试。

将测试模块和设计模块分开设计是一种良好的设计风格，测试模块一般被称为测试平台（testbench）。在实际工作中，常常使用不同的测试平台对设计模块进行全面的功能测试。

测试模块通常是顶层模块，它的基本结构如图 C.1.1 所示。它也是以关键词 **module** 开始，并以 **endmodule** 作为结尾的，但没有端口列表和端口类型声明部分。在测试模块内部，通常由以下几个部分组成：声明信号类型、实例引用设计模块、施加激励信号、显示测试结果。

```
module 测试模块名( );
        …
    reg  输入信号名;
        …
    wire 输出信号名;
        …
    实例引用设计模块;
    initial begin
        … //此处添加激励
        end
    always begin
        … //此处添加时钟信号
        end
    initial begin
        … //此处添加输出语句
        end
endmodule
```

图 C.1.1 测试模块的基本结构

可见，Verilog HDL 测试平台是一个 Verilog HDL 模块，它实例引用设计模块，向其提供输入数据并监视其输出。

一个设计模块及其相应的测试平台形成一个仿真模型，由于测试平台也是用 Verilog HDL 编写的，所以无论哪种仿真环境，都可以使用相同的输入数据对设计模块进行测试。有一些仿真环境提供波形编辑器（例如，Quartus Prime 中的 Simulation Waveform Editor），可以使用波形图的方式将测试数据输入到正在测试的设计模块中，但这类工具只适用于小型设计，如果更换仿真器时，需要重新编辑测试数据。

下面，举例说明测试平台的编写方法。

C.2 组合逻辑电路的测试

编写组合逻辑电路的测试模块较简单，主要是给被测模块提供输入信号。下面通过一

个例题进行说明。

例 C.2.1 一个多功能算术逻辑运算(ALU)的 Verilog HDL 代码如图 C.2.1 所示,试说明其具体的功能,画出组成框图,并写出测试平台,给出仿真波形。

附录教学视频 C.1:
编写组合电路
的测试代码

```
module alu_4bit (
    input [3:0] a, b,
    input [1:0] f,
    input oe,
    output [3:0] y,
    output reg co, ov, neg,
    output zero,
    output reg a_gt_b, a_eq_b, a_lt_b
    );
    reg [4:0] node_y;
    always@(a or b or f ) begin : alu
        co= 1'b0;   ov = 1'b0;
        node_y =5'b00000;
        case( f )
            2'b00 :begin
                        node_y = a + b;
                        co = node_y[4];
                        ov = co^ node_y[3]^a[3]^b[3] ;
            end
            2'b01 :begin
                        node_y = a - b;
                        co = node_y[4];
                        ov = co^ node_y[3]^a[3]^b[3] ;
            end
            2'b10 :node_y[3:0] = a & b;
            2'b11 :node_y[3:0] = a ^ b;
            default: node_y[3:0] = 4'b0000;
        endcase
        neg = node_y[3];
    end  //alu
    always@( a or b ) begin : comparator
        if( a> b )    { a_gt_b, a_eq_b, a_lt_b} = 3'b100;
        else if( a< b )    { a_gt_b, a_eq_b, a_lt_b } = 3'b001;
        else               { a_gt_b, a_eq_b, a_lt_b } = 3'b010;
    end
    assign y = oe ?node_y[3:0] : 4'bz;
    assign zero = ~ | node_y[3:0];
endmodule
```

图 C.2.1 ALU 的设计模块

解：分析上述代码可知，该设计模块由 2 个 **always** 块和 2 条 **assign** 语句组成。第 1 个 **always** 块是根据 f 选择对 a 和 b 进行算术或者逻辑运算，并经过三态逻辑输出结果。对于加、减运算，还能给出与运算结果的有关标志信号，即进位标志、溢出标志、负数标志和零标志。第 2 个 **always** 块是对 a 和 b 的大小进行比较。整个模块的组成框图如图 C.2.2 所示。

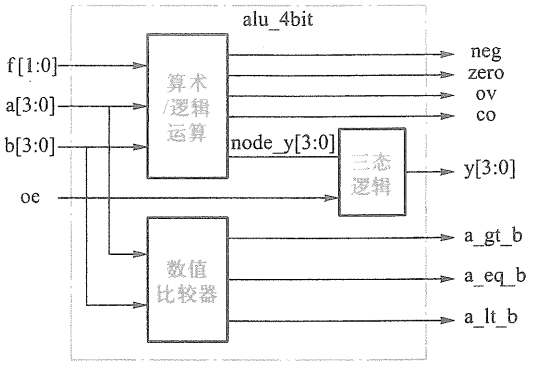


图 C.2.2 ALU 模块的组成框图

在代码中，带有“:alu”的 **begin…end** 块完成了算术/逻辑运算的功能，带有“:comparator”的语句块完成了数值比较的功能，这种带有名称的 **begin…end** 语句块称为有名块。在描述一个较为复杂的电路时，这是一种值得推荐的代码描述风格。

alu_4bit 的测试模块如图 C.2.3 所示。以`(反引号)开始的第 1 条语句是编译器指令，该指令将测试模块中的延迟时间单位设置成 1 ns，时间精度(指延时值的最小分辨度)设置为 100 ps，例如，下面模块中的#20 代表延时 20 ns。

测试模块取名为 test_alu_4bit，它没有输入或输出端口。接着，将输入到被测模块的信号声明为 **reg**，以便保持激励值不变；将输出信号声明为 **wire**，以便能随时跟踪激励信号的变化。这里，在声明 **reg** 信号时，已经给 a、b、f 和 oe 赋了初始值。

接下来，调用(也称为实例引用)设计模块 alu_4bit，并将 UUT 用作其实例名称。根据 alu_4bit 电路的端口列表顺序，将测试模块中声明的信号一一对应地与被测模块中的端口连接起来。

在 **initial** 语句块中给出激励信号数据。仿真时，刚进入 **initial** 语句的时刻为 0，在前 60 ns，每隔 20 ns 给 b 赋一个新的值；然后，**initial** 块等待 80 ns，将 oe 变为 **0**，禁止 ALU 输出结果，最后，等待 20 ns 后，结束仿真。

在 **always** 语句中，对送到 alu_4bit 的输入信号 f 的值进行修改。从初始值 0 开始，每隔 23 ns，f 的值递增 1。

在 **initial** 块中的系统任务 **$finish** 语句在 160 ns 时结束。此时，所有活动的程序块均停止，仿真终止并退出。另一个常用的系统任务是 **$stop**，该任务仅暂停仿真，并允许恢复仿真运行。

```
`timescale 1ns/100ps
module test_alu_4bit;
    reg [3:0] a=4'b1011,b=4'b0110;
    reg [1:0] f=2'b00;
    reg oe=1;

    wire [3:0] y;          //声明输出信号

    wire co, ov, neg, zero, a_gt_b, a_eq_b, a_lt_b;

    //实例引用设计模块

    alu_4bit UUT( a, b, f, oe, y, co, ov, neg, zero,
                  a_gt_b, a_eq_b, a_lt_b );
    initial begin

        #20  b=4'b1011;     //改变 b 的值

        #20  b=4'b1110;
        #20  b=4'b1111;
        #80  oe=1'b0;

        #20  $finish;    //结束仿真

    end

    always #23 f = f + 1; //改变f的值

endmodule
```

图 C.2.3 alu_4bit 的测试模块

在仿真软件 ModelSim 中，输入上述所有代码，并进行编译和仿真，得到 alu_4bit 模块的仿真波形如图 C.2.4 所示。在整个仿真过程中，a 保持恒定，b 在 20 ns 时从 6 变为 B，然后在 40 ns、60 ns 时分别变为 E 和 F。工作模式输入 f 每 23 ns 变化一次，从而检查了 ALU 的各种功能。在 140 ns 时，oe 变为 0，导致 y 输出变为高阻 z。

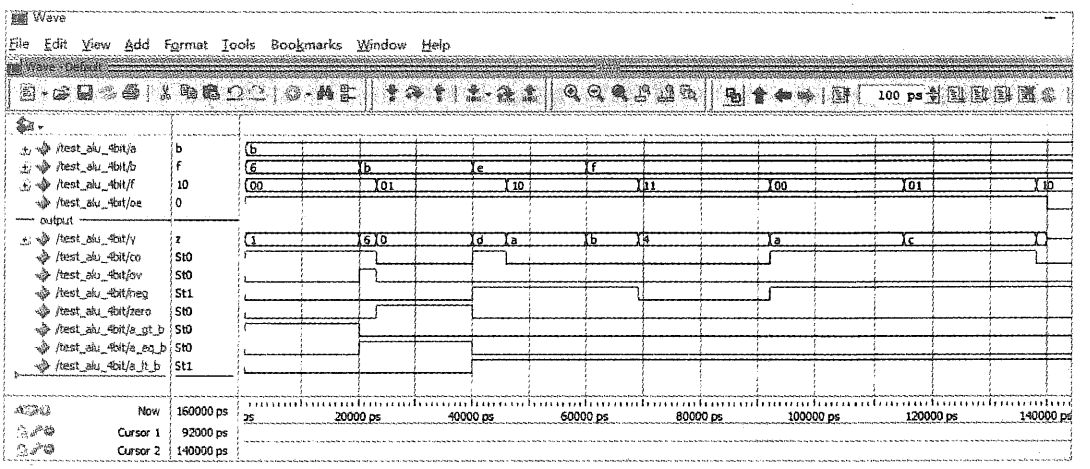


图 C.2.4 alu_4bit 的仿真波形

C.3 时序逻辑电路的测试

在测试时序电路时，会涉及电路时钟信号与其他数据输入的同步问题。下面先看一个产生时钟信号的例子。

例 C.3.1 分析图 C.3.1 所示的 4 段代码，说明其功能。

(1)
```
reg clk;
always
  begin
    #5 clk = 0;
    #5 clk = 1;
end
```

(3)
```
reg clk;
parameter PERIOD = 10;
initial      clk = 1'b0;
always begin
      #(PERIOD-PERIOD/5) clk = 1;
      #(PERIOD/5) clk = 0;
end
```

(2)
```
initial begin
    clk=0;
    forever #5 clk = ~clk;
end
```

(4)
```
initial begin
    clk=0;
end
    repeat(6) #5 clk = ~clk;
```

图 C.3.1 产生时钟信号的代码

附录教学视频 C.2：编写时序电路的测试代码

解：(1) 这是单独使用 **always** 块产生时钟信号的例子。由于没有用编译指令`timescale 规定时间单位，故使用默认的时间单位 ps。每隔 5 ps，clk 翻转一次，不断地循环。所以，clk 是占空比为 50%的方波，但 clk 最初 5 ps 的初值为 x(不定状态)。

(2) 这是单独使用 **initial** 块产生时钟信号的例子。clk 初值为 **0**，隔 5 个时间单位，clk = **1**，再隔 5 个时间单位，clk = **0**，如此不断地循环。所以，信号 clk 是占空比为 50%的方波。

(3) 产生占空比为 20%的方波。clk 初值为 **0**，隔 8 个时间单位，clk = **1**，再隔 2 个时间单位，clk = **0**，如此不断地循环。这里使用了参数定义语句，以方便修改时钟周期的值。

(4) clk 初值为 **0**，接着，用 **repeat** 语句产生 3 个时钟脉冲。

例 C.3.2 分析图 C.3.2 所示的代码，说明其功能，并给出仿真波形。

解：这段测试代码首先声明了两个输入信号，即清零信号 clr 和时钟信号 clk。并用 **initial** 和 **always** 产生周期为 10 个时间单位的时钟信号 clk。

```
module test_clkreset ;
    reg clr; //Clear
    reg clk; //Clock
//产生时钟信号，周期为10个时间单位
  initial clk = 1'b0;
  always #5 clk = ~clk;
//产生清零信号
  initial begin
            clr = 1'b0;
      wait (clk == 1'b1);
      @(negedge clk); //等待clk下降沿
          clr<= 1'b1;
      repeat(2) @(negedge clk);
          clr<= 1'b0;
  end
endmodule
```

图 C.3.2 产生时钟和清零信号

接着,在第 2 个 **initial** 语句块中,产生清零信号。仿真的 0 时刻,给 clr 赋初值 **0**;接着,出现了一个新语句 **wait**(等待)。**wait** 是电平敏感的时序控制语句,代码中的 **wait**(clk == 1'b1),就是等待 clk 出现高电平时继续执行下一条语句。这里,下一条语句是@(**negedge** clk),即等到 clk 的下一个下降沿到来时,执行下一条语句,把 clr 变为 **1**,让清零信号有效。

接着,又出现了重复执行的 **repeat** 语句,要求重复等待两次 clk 的下降沿,再执行下一条语句,把 clr 变为 **0**,让清零信号无效。

用这种方法产生清零信号的好处是:当改变时钟周期的值时,清零信号 clr 高电平宽度可以保持为 2 个时钟周期的时间不变。

仿真波形如图 C.3.3 所示。

注意,该代码在第 2 个 **initial** 语句中,混合使用了阻塞赋值和非阻塞赋值两种语句,这种情况只能在测试模块中使用。因为测试模块不需要被综合成门级网表,所以对语法没有严格的限制,只要得到所有测试向量即可。

这里,阻塞赋值用于刚开始时,给 clr 赋初值,后面的非阻塞赋值是为了在时钟边沿的位置不要出现时钟和复位信号的竞争关系。

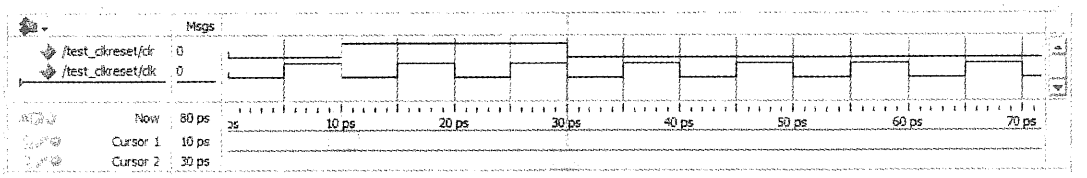


图 C.3.3 图 C.3.2 的仿真波形

例 C.3.3 4 位同步二进制递增计数器的代码如图 C.3.4 所示,试按图 C.3.5 所示的波形图编写一个测试模块,并给出仿真波形。

解:根据图 C.3.5 所给出的计数值来计算,输出 Q 从 0 开始计数到 15,然后循环。整个仿真需要 22 个时钟周期。

假设时钟 CP 的周期为 10 ns,则清零信号 CLR 在 0~15 ns 和 195~205 ns 期间为 **1**,其余时间为 **0**。于是,写出来的测试模块如图 C.3.6 所示。

```
module Counter(CP, CLR, Q);
    input CP, CLR;
    output reg [3:0] Q;
always@(negedge CP or posedge CLR)
begin
    if (CLR)
        Q <= 4'd0;              //异步清零
    else
        Q <= Q + 1'b1;          //递增计数
end
endmodule
```

图 C.3.4 4 位同步二进制递增计数器

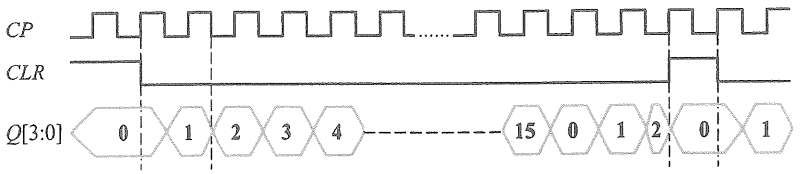


图 C.3.5 测试模块要求的波形图

```
`timescale 10ns /1ns
 module Test_Counter ;
        reg CP,CLR; //Clock and Clear
        wire [3:0] Q; //Register output
 Counter U0(CP,CLR,Q); //实例引用设计块
 initial //产生时钟信号CP, 周期为10 ns
                CP = 1'b0;    //将CP初值设为0
        always                //每隔5 ns, CP翻转一次
            #5   CP = ~CP;
  initial begin:Clear    //产生清零信号
          CLR = 1'b1;  //在0~15 期间为高电平
     #15  CLR = 1'b0;
     #180 CLR = 1'b1;  //在 195~205期间为高电平
     #10  CLR = 1'b0;
     #15 $stop;          //暂停仿真
   end
   initial                //输出结果到屏幕上
        $monitor($time,": Output Q= %d\n",Q);
endmodule
```

图 C.3.6 满足图 C.3.5 要求的测试模块

先声明输入、输出信号,并实例引用前面的设计模块 Counter。接着,产生时钟和清零信号。然后,到了需要结束仿真的时间点,就用系统任务 **$stop** 来暂停仿真。由于这里有多条语句,所以使用 **begin…end** 将这些语句组合成为一个串行语句块。

最后一个 **initial** 块的显示结果如图 C.3.7 所示(为节省篇幅,中间省略了部分结果)。这里,使用系统任务 **$monitor** 来监视计数器输出信号 Q 的变化情况。仿真时会在 ModelSim

```
0: Output Q= 0
20: Output Q= 1
30: Output Q= 2
...
190: Output Q= 2
195: Output Q= 0
210: Output Q= 1
```

图 C.3.7 测试模块中系统任务 $monitor 的显示结果

仿真器的 Transcript(信息报告)窗口中,同步地以文本方式显示出 Q 的结果。另外,$time 是一个系统函数,运行该函数时将返回当前的仿真时间。下一节介绍系统任务和系统函数的内容。

C.4 编译指令、系统任务和系统函数

在编写测试代码时,经常会用到编译指令、系统任务和系统函数。这里,做一个简要介绍。

C.4.1 Verilog HDL 编译器指令

在 Verilog 中,有一些硬件建模要求是由编译指令(compiler directives)提供的,而不是包含在语言的主要语法中。

编译指令使用"<关键词>"的结构进行定义。在 Verilog 代码编译的整个过程中,编译指令始终有效(编译过程可能跨越多个文件),直至遇到其他不同的编译器指令为止。这里,讨论 3 个最常用的编译器指令,其他编译指令可参考文献[9,10]。

1. `timescale

该指令用于指定 Verilog HDL 模型中延迟的时间单位和时间精度。其格式为:

```
`timescale time_unit/time_precision
```

其中,time_unit 和 time_precision 由值 1、10、100 以及单位 s、ms、μs、ps、fs 组成。例如,`**timescale** 10 ns/100 ps 表示时间单位为 10 ns,时间精度(即最小分辨度)为 100 ps。

`**timescale** 放在模块声明的外部,在编译过程中,它影响其后所有模块内部的延迟值,直到遇到另一个`**timescale** 指令或者`**resetall** 指令。

2. `include

编译器指令`**include** 用于在代码中包含其他文件的内容,这与 C 语言中的#include 类似。以便在模块中包括参数定义文件或共享部分代码,被包含的文件既可以用相对路径名定义,也可以用全路径名定义,例如:

```
`include "../../header.v"
```

编译时,这一行由文件"../../header.v"的内容替换。

3. `define 和`undef

为了提高代码的可读性,可以定义一个有意义的字符串(称为文本宏)来表示数字或表达式。`**define** 指令用于设置文本替换的宏[①],这与 C 语言中的#define 类似。

定义的字符串可以在 Verilog 代码文本中使用,方法是在定义的名称前加上`(反引号)。编译时,当编译器碰到<宏名>,使用预定义的宏文本进行替换。举例说明如下:

---

① 宏名,通常全部用大写字母表示。

```
`define WORD_SIZE   32                  //定义文本宏
        ……
    reg  [`WORD_SIZE-1:0]    Q;   //使用时，格式为`WORD_SIZE

`define   S    $stop                    //定义别名，可以用`S 来代替 $stop

`define WORD_REG   reg  [31:0]      //定义经常用的字符串，
                                    //可以用`WORD_REG 来代替 reg  [31:0]
```

一旦`**define** 指令被编译通过，则由其规定的宏定义在整个编译过程中都保持有效。例如，在某个文件中通过`**define** 指令定义的宏 WORD_SIZE 可以在多个文件中使用①。

`**undef** 指令用来取消前面定义的宏。举例说明如下：

```
`define WORD_SIZE 16                //定义文本宏
        ……
    reg  [`WORD_SIZE-1:0]    Q;   //使用时，格式为`WORD_SIZE
        ……
`undef WORD_SIZE                    //在该语句以后，宏定义 WORD_SIZE 不再有效
```

C.4.2 Verilog HDL 系统任务

Verilog HDL 提供了很多系统任务（System Tasks），用来完成仿真过程中的一些常规操作，例如显示任务、仿真控制任务、文件输入/输出任务等。

系统任务使用“ $<关键词>”的结构进行定义。这里，仅介绍几个常用的系统任务。

1. 显示任务（Display Task）

$display 是 Verilog 中最有用的任务之一，用于将指定信息（被引用的字符串、变量值或者表达式）以及结束符显示到标准输出设备上。其格式如下：

$display(format_specification1,argument_list1,
format_specification2,argument_list2,
……,);

其中，format_specification1 为显示格式说明，它与 C 语言中 printf( ) 的格式非常类似。argument_list1 为显示内容。默认情况下，**$display** 在信息显示完后会插入新的一行。

字符串的显示格式说明如表 C.4.1 所示。

附录教学视频 C.3：常用的系统任务和系统函数

① 建议：将多个`**define** 指令放在一个独立的名为<design>_defines.v 的文件中。用`**include** 指令在设计文件中包含该文件。

表 C.4.1 字符串的显示格式说明

格式符	显示说明	格式符	显示说明
%d 或%D	用十进制数显示信号值	%o 或%O	用八进制显示信号值
%b 或%B	用二进制显示信号值	%t 或%T	显示目前时间格式
%s 或%S	显示字符串	%e 或%E	用科学记数方式显示实数(例 3e10)
%h 或%H	用十六进制显示信号值	%f 或%F	用十进制方式显示实数(例 2.13)
%c 或%C	显示 ASCII 字符	%g 或%G	选择科学记数和十进制方式中较短的来显示实数
%m 或%M	显示层次名(不需参数)		
%v 或%V	显示信号强度		

下面举例说明其用法。

```
$display ("Hello Verilog World!") //用新的一行显示引用的字符串
/* 显示结果: Hello Verilog World!    * /

reg  [4:0]  Port_id;  //假设 Port_id 的值为 00101
$display ("ID of the port is %b ", Port_id);
/* 显示结果: ID of the port is 00101   * /

reg  [3:0]  Bus;  //假设 Bus 的值为 10xx
$display ("bus value is %b ",Bus);
/* 显示结果: bus value is 10xx   * /
```

2. 监视任务(Monitor Task)

在仿真时,**$monitor** 用来持续监视一个或者多个信号值发生改变的情况。如果被监视的某个信号的值发生变化,仿真器就会打印这些指定值。但是它不能用来监控时间变量或者返回时间值的函数(如,**$time**)。

$monitor 任务的用法与 **$display** 相同,但是在任何时刻只能有一个监控任务处于活动(active)状态。如果在仿真中有多条 **$monitor** 语句,则只有最后一条语句处于活动状态,前面的 **$monitor** 语句将不起作用。

可以通过下面两个系统任务启动和关闭监控任务。

```
$monitoroff;        //关闭激活的监控任务
$monitoron;         //启动最近关闭的监控任务
```

3. 仿真的中止(stopping)和完成(finishing)任务

这两个任务的格式是:

```
$stop;     //在仿真期间,暂停执行,未退出仿真环境
$finish;   //仿真完成,退出仿真环境,并将控制权返回给操作系统
```

系统任务 **$stop** 使得仿真进入交互模式,然后设计者可以进行调试。当设计者希望检查信号的值时,就可以使用 **$stop** 暂停仿真。然后可以发送交互命令给仿真器继续仿真。

下面举例说明其用法。

```
initial  //进入仿真时，time=0
begin
                Clock = 0;
                Reset = 1;
        #100  $stop;     //在 time=100 时，暂停仿真
        #900  $finish;   //在 time=1000 时，结束仿真，退出仿真器
end
```

4. 初始化存储器的系统任务

有两个系统任务能够从文本文件中读取数据，并将数据加载到存储器中。这两个系统任务分别是：**$readmemb** 和 **$readmemh**。它们的用法相同，只是文本文件中数据的格式有区别。**$readmemb** 要求文件中的数据以二进制格式存放；而 **$readmemh** 则要求文件中的数据以十六进制格式存放。

$readmemb 的用法如下：

```
$readmemb("文件名",存储器名);
$readmemb("文件名",存储器名,起始地址);
$readmemb("文件名",存储器名,起始地址,结束地址);
```

这里，文件名和存储器名是必须的。文件名可以包含相应的路径，必须使用双引号，其内容为初始化存储器的数据。存储器名用来指定一个存储器，用文件中读出来的数据对存储器进行初始化。

起始地址和结束地址是可选的。它们用来表明读出来的数据存放在存储器的什么地方。缺省起始地址时，表明从存储器的第一个地址开始存放；缺省结束地址时，表明一直存放到存储器的最后一个地址为止。

关于文件中存放的数据，有两点注意事项：

（1）各个数据之间要求用空格隔开，每个数据位允许出现逻辑值的四种状态（0、1、x、z），也允许包含下划线。

（2）文本文件中允许包含地址说明，形式为@ hhhh，@ 为地址说明标识符，h 表示地址必须以十六进制形式给出。这样，系统任务将数据读出来后，会存入到指定的地址，后面的数据会从指定地址开始向后存放。

一个存储器初始化模块的例子如图 C.4.1 所示。首先，调用系统任务 **$readmemb** 对存储器进行初始化，最后在 for 循环语句中，调用显示系统任务，将初始化后存储器中的值显示出来。

假设文本文件 init.dat 的内容如图 C.4.2 所示。文件中的@ 002、@ 006 是用 16 进制格式表示的存储器的地址。存储器中未初始化位置的默认值为 x。

对这个模块进行仿真时，得到如图 C.4.3 所示的结果。从文件中规定的 2 号地址开始，对存储器进行初始化。刚开始的两个存储单元没有初始化，所以它的值为 x。

```
module Test_Memory;
    reg [3:0] mem [0:7];  //声明一个8×4位的存储器
    integer i;            //声明循环变量
    initial begin
        //从init.dat读取数据，并存放到存储器中
        $readmemb("init.dat",mem);
        //显示初始化后的存储器内容
        for(i=0;i<8;i=i+1)
            $display("Mem[%0d] = %b",i,mem[i]);
    end
endmodule
```

图 C.4.1 存储器初始化模块

```
@002
1111 0101
0000 1010
@006
11zz 0011
```

图 C.4.2 init.dat 的内容

```
# Mem [0] = xxxx
# Mem [1] = xxxx
# Mem [2] = 1111
# Mem [3] = 0101
# Mem [4] = 0000
# Mem [5] = 1010
# Mem [6] = 11zz
# Mem [7] = 0011
```

图 C.4.3 图 C.4.1 的仿真结果

C.4.3 Verilog HDL 系统函数

系统函数和系统任务一样，也使用“$<关键词>”的结构进行定义。两者的主要区别是：系统任务主要用来完成仿真过程中的一些常规操作。调用系统任务时，只完成某项任务，没有返回值。而系统函数（system function）主要由系统内部完成某种功能的运算。调用系统函数时，会返回一个值。这里，仅介绍两个最常用的系统函数。

1. 仿真时间函数

下列 2 个系统函数，都能返回从仿真开始执行到该函数被调用时刻的时间。具体区别如下：

$time：按照所在模块的时间单位，返回的仿真时间是一个 64 位整数。

$realtime：按照所在模块的时间单位，返回的仿真时间是一个实数。

2. 产生整型随机数的函数

有时，为了生成一组随机的测试向量，就需要使用生成随机数的系统函数。随机测试非常重要，因为它常常能够发现设计中隐含的错误。

$random 是一个用来产生随机数的系统函数。一般用法如下：

```
$random;
$random(seed);
```

这里，种子 seed 的值是可选的。seed 不同，产生的随机数的序列也不同。如果 seed 的

初值是确定的，则仿真时产生的随机数序列也是确定的，就不能覆盖更多的情况。因此在仿真时，要改变 seed 的初值。

$random 函数的返回值是一个有符号的、32 位二进制整型数。可以使用这 32 位随机数的所有位，也可以只选择其中的部分位来使用。

注意：(1) seed 必须是 reg 型、integer 型或者 time 类型的变量。在调用 $random (seed) 之前，应该为这个变量赋值。

(2) $random 函数使用的算法不是标准化的，不同的仿真器返回的随机数可能不一样。

例如，在测试 4 位二进制加法器时，为了随机地生成加数、被加数和进位数据，就可以用 $random 系统函数来产生测试数据。其测试模块如图 C.4.4 所示，代码中使用了 3 个种子，分别赋予了三个值，这样生成的 32 位二进制整数就是不同的。在 ModelSim 中的仿真波形如图 C.4.5 所示。

```
module Test_Random ;
    reg [3:0] A,B;
    reg Cin;
    integer seed1,seed2,seed3;
    initial begin
        seed1 =1; seed2 =2; seed3 =3; //种子初始化
    end
    always begin  //每隔10个时间单位生成一组随机值
        #10 A=$random(seed1);
            B=$random(seed2);
            Cin=$random(seed3);
    end
endmodule
```

图 C.4.4 使用 $random 的测试模块

如果不指定种子变量的值，直接使用($random)来生成数值，则 A、B 的数值都相同，就起不到随机测试的本来目的了。如果使用相同的种子，也会出现这个问题。所以一般要生成几个随机向量，就使用几个种子值。

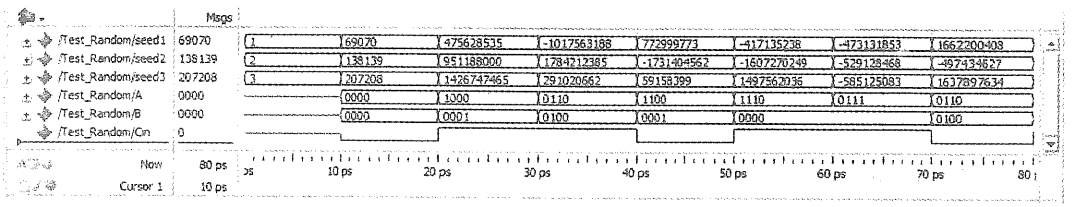


图 C.4.5 图 C.4.1 的仿真波形

附录 D
ModelSim 仿真软件的使用

ModelSim 是明导(Mentor Graphics)公司的仿真软件,它是针对 Verilog、SystemVerilog 和 VHDL 语言的验证和仿真工具。

ModelSim 分为 SE、AE、XE 等不同的版本,其中 SE 是功能最齐全的版本,AE、XE 则分别是为 Intel FPGA、Xilinx FPGA 等专门定制的版本(OEM 版)。在用特定公司的 OEM 版进行仿真时,不需要编译该公司的库文件,但是仿真速度等性能指标都要落后于 SE 的版本。目前,ModelSim 可在 Windows 7/8/10 操作系统上运行。

本文档编译和仿真使用 ModelSim-Intel FPGA Starter Edition 10.5b(Quartus Prime 18.1)软件(以下简称 ModelSim),并以举例的方式来说明仿真操作,以便初学者能够学以致用。

读者可以参考帮助文档,选择菜单 Help→PDF Documentation→Tutorial 命令,打开 ModelSim Tutorial 文档,从而获得更多的使用信息。

本附录简要介绍该软件的基本仿真功能和使用方法,以期读者能用它对 Verilog 描述的数字逻辑电路进行仿真。限于篇幅,这里的内容以电子档的形式提供,读者可扫码阅读。另外,为便于学习,同时提供了软件使用操作演示的讲解视频,也可扫码观看。

附录文档 D.1:
基于工作库的
基本仿真流程

附录文档 D.2:
基于工程项目的
仿真流程

附录教学视频 D.1:
基于 ModelSim
功能仿真

附录 E
Quartus Prime 软件的使用

Quartus Prime 软件是由 Intel(前身为 Altera)公司开发的一个 EDA 工具,它集成了设计输入、逻辑综合、布局布线、器件编程、时序分析等开发 FPGA 和 CPLD 器件所需的多个软件工具。

随着 FPGA 器件集成度的提高、器件结构和性能的改进,各公司的开发软件在持续地改进和更新之中。目前,Quartus Prime(简称 Quartus)软件提供 3 个版本,即专业版、标准版和精简版,并以当年年号的后两位数字作为软件的主版本号(例如,2017 年推出的软件为 Quartus Prime 17.0),软件的次版本号从 0 开始顺序编号。这些版本之间的差别主要是支持的器件不同。精简版软件不需要许可授权文件,免费使用,但支持的器件不太多,是初学者的理想选择。

这里,使用 2018 年推出的 Quartus Prime 18.1-Lite 精简版,可以安装运行在 Windows 7/10 (64 位)操作系统的计算机上。

本附录简要介绍该软件的基本功能和使用方法,以期读者能用它和 FPGA 器件来实现数字逻辑电路,并能够对电路进行仿真。限于篇幅,这里的内容以电子档的形式提供,读者可扫码阅读。

附录文档 E.1:
Quartus Prime
软件主界面

附录文档 E.2:
基于 Verilog HDL
输入的电路实现

附录文档 E.3:
基于 Verilog HDL
输入的电路仿真

参考文献

[1] M. Morris Mano. Digital Design [M]. 5th ed. 北京:电子工业出版社,2017.

[2] John F. Wakerly. Digital Design:Principles and Practices [M]. 5th ed. 北京:机械工业出版社,2018.

[3] Stephen Brown. 数字逻辑基础与 Verilog 设计[M]. 3 版. 吴建辉,黄成,等译. 北京:机械工业出版社,2018.

[4] Sergio Franco. Design with Operational Amplifiers and Analog Integrated Circuits [M]. 4th ed. 北京:机械工业出版社,2015.

[5] Paul Horowitz, Winfield Hill. The Art of Electronics [M]. 3rd ed. Cambridye: Cambridge University Press,2015.

[6] Ulrich Tietze, Christoph Schenk, Eberhard Gamm. Electronic Circuits Handbook for Design and Applications [M]. 2nd ed. Berlin Springer,2008.

[7] Adel S. Sedra, Keneth C. Smith. Microelectronic Circuits[M]. 7th ed. New York: Oxford Univesity Press,2015.

[8] William Kleitz. Digital Electronic: A Practical Approach with VHDL [M]. 9th ed. New York:Pearson Education Inc. 2014.

[9] J. Bhasker. Verilog HDL 入门[M]. 3 版. 夏宇闻,甘伟,译. 北京:北京航空航天大学出版社,2008.

[10] Donald E. Thomas, Philip R. Moorby. The Verilog Hardware Description Language[M]. 5th ed. Boston:Kluwer Academic Publishers,2002.

[11] 清华大学电子学教研组编,阎石主编. 数字电子技术基础[M]. 6 版. 北京:高等教育出版社,2016.

[12] 西安交通大学电子学教研室编,沈尚贤主编. 电子技术导论(上)[M]. 北京:高等教育出版社,1985.

[13] 李士雄,丁康源. 数字集成电子技术教程[M]. 北京:高等教育出版社,1993.

[14] 罗杰,彭容修. 数字电子技术基础[M]. 3 版. 北京:高等教育出版社,2014.

[15] 瞿安连. 电子电路——分析与设计[M]. 武汉:华中科技大学出版社,2010.

[16] 李哲英. 电子技术及其应用基础(数字部分)[M]. 2 版. 北京:高等教育出版社,2009.

[17] 蔡惟铮. 基础电子技术[M]. 北京:高等教育出版社,2004.

[18] 李文渊. 数字电路与系统[M]. 北京:高等教育出版社,2017.

[19] 罗杰. Verilog HDL 与 FPGA 数字系统设计[M]. 北京:机械工业出版社,2015.

索引(汉英对照)

二值数字逻辑(binary digital logic) 7
二进制(binary) 11
~数(number) 12
~权(weights) 12
二进制编码的十进制(binary-coded-decimal,BCD) 23
二极管(diode) 3
发光~(light-emitting~) 98
肖特基势垒~(schottky barrier~,SBD) 111
十进制(decimal) 11
十六进制(hexadecimal) 16
八进制(octal) 17
七段显示器(seven-segment display) 164
二的补码(two's complement) 20

三极管(bipolar Junction transistor.BJT) 78,109
肖特基~(Schottky~) 111
三极管-三极管逻辑(transistor-transistor logic,TTL) 3,110
下降沿(fall edge) 9
下降时间(fall time) 9
上拉电阻(pull-up resistor) 95
上升沿(rise edge) 9
上升时间(rise time) 9
门(gate) 29
~阵列(array) 190
与(AND)~ 29
或(OR)~ 30
非(NOT)~ 30
与或非(AND-OR-INVERT)~ 91
异或(exclusive OR)~ 32,91
同或(exclusive NOR)~ 32,91
与非(NAND)~ 31,89
或非(NOR)~ 32,90
三态(three state)~ 99
漏极开路(open drain)~ 94
集电极开路(open collector,OC)~ 112
门级元件(gate level primitives) 70

四　画

专用集成电路(application specific integrated circuit,ASIC)　4
无关项(don't care terms)　57
开关特性(switching characteristics)　81
开关时间(switching time)　111
互补 MOS(complementary metal-oxide-semiconductor,CMOS)　3,85
　增强型～(enhancement-mode～)　81
　耗尽型～(depletion-mode～)　83
卡诺图(karnaugh map)　52
分配器(demultiplexer)　167
分辨率(resolution)　414,433
分频(frequency division)　296
反相器(inverter)　85
　反演规则(complementary operation theorem)　43
　反码(one's complement)　19
计数器(counter)　295
　模 n～(modulo-n～)　295
　异步～(asynchronous～)　296
　同步～(synchronous～)　297
　二进制～(binary～)　296
　递增～(up～)　297
　递减～(down～)　298
　纹波～(ripple～)　297
　环形～(ring～)　306
　扭环形～(twisted-ring ～,Johnson ～)　307
双稳态(bistable)　220
双向移位寄存器(bidirectional shift register)　294
双列直插封装(dual in-line package,DIP)　79
比特率(bit rate)　8

五　画

可编程逻辑器件(programmable logic device,PLD)　189
可编程阵列逻辑(programmable array logic,PAL)　192
布尔代数(boolean algebra)　41
正逻辑(positive logic)　113
功能表(function table)　153
只读存储器(read-only memory,ROM)　333
　可编程～(programmable ～,PROM)　336
　可擦除可编程～(erasable programmable ～,EPROM)　336
　电可擦除可编程～(electrical erasable programmable～,E^2PROM)　336
电子设计自动化(electronic design automation, EDA)　5
电平(level)　7
　有效～　114
　高～有效(active high)　115
　低～有效(active low)　115
占空比(pulse duration ratio)　8
印制电路板(printed circuit board,PCB)　58
片上系统(system on chip, SOC)　444
加法器(adder)　182
　半(half)～　181
　全(full)～　181
　串行进位(serial carry)～　182

六 画

延时-功耗积(time delay-power dissipation product) 104
权(weight) 11
存储器(memory) 333
只读(read-only~,ROM) 333
读写(read-write~,RWM) 333
随机存取(random access~,RAM) 333,341
静态(static)~ 341
同步~(synchronous~,SSRAM) 344
异步~(asynchronous~,ASRAM) 344
双口~(dual-port~) 343
先进先出~(first-in first-out,FIFO~) 343
动态(dynamic)~ 341,345
同步~(synchronous~,SDRAM) 345
闪存存储器(flash~) 333,336
双倍数据传输率~(double data rate,DDR~) 345
四倍数据传输率~(quad data rate,QDR~) 345
单倍数据传输率~(single data rate,SDR~) 345
存储阵列(memory array) 333,335,341
存储单元(memory cell) 333
动态(dynamic)~ 345
静态(static)~ 342
存储时间(storage time) 339
地址(address) 334
~单元(~cell) 334
在系统可编程(in system programmability,ISP) 361
再生(刷新)(regenerate,refresh) 346
列选择线(column-select line) 335,342
同或(exclusive NOR) 32
回差电压(backlash voltage) 381
异或(exclusive OR) 32
传输延迟(propagation delay) 102
自校正(self-correcting) 274,276,280
传输延迟时间(propagation delay time) 102
传输特性(transfer characteristic) 86
传输门(transmission gate,TG) 91
行选择线(row-select line) 335,342
字(word) 333
~长(~length) 333

七 画

取样-保持电路(sample-hold circuit) 421
时钟(clock) 232
~脉冲(~pulse) 232
~偏移(~skew) 285
时序图(timing sequence diagram) 10,265
时序逻辑电路(sequential logic circuit) 259
同步~(synchronous~) 260
异步~(asynchronous~) 260
米利型~(Mealy~) 261
穆尔型~(Moor~) 261
余3码(excess three code) 23
位(bit) 8
运算电路(arithmetic circuit) 181
运算符(operator) 68
译码器(decoder) 157
二进制(binary)~ 157
二-十进制(BCD)~ 162

七段显示～(seven-segment display decoder) 164
状态(state) 260
～变量(～variable) 260
～化简(～reduction) 273
～分配(～assignment) 273
现态(present～) 238
次态(next～) 238
初始～(initial～) 222
无效～(unused～) 270,274
等价～(equivalent～) 273
状态转换表(state transition table) 263
状态转换方程(state transition equation) 260
状态机(state machine) 260
有限～(finite～) 260
时钟同步～(clocked synchronous～) 260
状态转换图(state transition diagram) 264

八 画

现场可编程门阵列(field programmable gate array,FPGA) 4,354
奇偶校验(odd-even check) 142
拉电流(draw-off current) 88
或门(OR) 30
或非门(NOR) 32,90
非门(NOT) 30
和之积 (products of sum,POS) 45
单稳态电路(monostable multivibrator) 373
定时器(timer) 390
定时参数(timing parameters) 238
线与(wire-AND) 94
组合逻辑电路(combinational logic circuit) 140
建立时间(setup time) 230

九 画

查找表(look-up table) 175,354
复位(reset) 221
复杂可编程逻辑器件(complex programmable logic device,CPLD) 4,361
保持时间(hold time) 230
施密特触发电路(schmitt trigger) 380
总线(bus) 100
宽～(widebus) 124

十 画

真值表(truth table) 29
逐次逼近 A/D 转换器(见模数转换器) 427
乘积项(product term,PT) 34,45
积之和 (sum of products,SOP) 45
特性表(characteristic table) 239
特性方程(characteristic equation) 239
扇出(fan out) 104
扇入(fan in) 104
竞争-冒险(race and hazard) 149
高阻态(high impedance state) 99
高电平输入电流(high level input current) 97
预置(preset) 234,274

十一画

逻辑（logic） 7
~电平（level） 7
~函数（function） 33
~变量（variables） 28
~门（gate） 78
~表达式（expression） 34
~图（diagram） 29
正~ （positive） 113
负~ （negative） 113
三极管-三极管~（transistor-transistor~，TTL） 3,110
小~（little logic） 124
~仿真（logic simulation） 58
~综合 （logic synthesis） 58
清零（clear） 233
寄存器（register） 291
移位~（shift~） 292
双向移位~（bidirectional shift~） 294
阈值电压（threshold voltage） 81

十二画

硬件描述语言（hardware description language，HDL） 4,58
超前进位（look~ahead carry） 183
最大项（maxterm） 47
量化（quantification） 421
最小项（miniterm） 45
集成电路（integrated circuit） 3
小规模~（small scale integration，SSI） 3
中规模~（medium scale integration，MSI） 3
大规模~（large scale integration，LSI） 3
超大规模~（very large scale integration，VISI） 3
甚大规模~（ultra large scale integration，UISI） 3
集成度（integration） 3
锁存器（Latch） 221
SR~（*SR*~） 221
门控 *SR*~（gated *SR*~） 225
D~（*D*~） 227
编码（encoding） 152
状态~（state~） 273
一对一~（one-hot~） 282
编码器（encoder） 152
优先（priority）~ 152

十三画

输出方程（output equation） 260
置位（set） 221
触发（triggering） 232
触发器（flip-flop） 232
主从~（master-slave ~） 232
D~（*D*~） 232
JK~（*JK*~） 240
T~（*T*~） 241
T'~（*T'* ~） 242
SR~（*SR*~） 242
数字电路（digital circuit） 3
数码显示器（digital display） 164

数值比较器(digital comparator) 178
数模转换器(digital to analog converter, DAC) 402
电阻串(resistor-string)~ 410
权电阻(weighted resistor)~ 402
权电流(weighted current)~ 408
倒T形(inverted T)~ 404
数据选择器/多路复用器(data selector/Multiplexer) 169,177

十四画

模拟开关(analog switch) 91
漏极开路门(open drain,OD) 94
模数转换器(analog to digital converter, ADC) 402
半闪速(half flash)~ 425
闪速(flash)~ 423
流水线闪速(pipelined flash)~ 425
双积分(dual slope)~ 430
逐次比较(successive comparation)~ 427
算法状态机(algorithmic state machine, ASM) 444

十五画

摩根定理(De Morgan's theorem) 42

十六画

激励表(excitation table) 278
激励方程(excitation equation) 260
噪声容限(noise margin) 101
高电平~(high-state~) 101
低电平~(low-state~) 101

郑重声明

高等教育出版社依法对本书享有专有出版权。任何未经许可的复制、销售行为均违反《中华人民共和国著作权法》,其行为人将承担相应的民事责任和行政责任;构成犯罪的,将被依法追究刑事责任。为了维护市场秩序,保护读者的合法权益,避免读者误用盗版书造成不良后果,我社将配合行政执法部门和司法机关对违法犯罪的单位和个人进行严厉打击。社会各界人士如发现上述侵权行为,希望及时举报,本社将奖励举报有功人员。

反盗版举报电话　(010)58581999　58582371　58582488

反盗版举报传真　(010)82086060

反盗版举报邮箱　dd@hep.com.cn

通信地址　北京市西城区德外大街4号

高等教育出版社法律事务与版权管理部

邮政编码　100120

防伪查询说明

用户购书后刮开封底防伪涂层,利用手机微信等软件扫描二维码,会跳转至防伪查询网页,获得所购图书详细信息。用户也可将防伪二维码下的20位密码按从左到右、从上到下的顺序发送短信至106695881280,免费查询所购图书真伪。

反盗版短信举报

编辑短信“JB,图书名称,出版社,购买地点”发送至10669588128

防伪客服电话

(010) 58582300